

Dokumentacja WPF (Windows Presentation Foundation)

Spis treści

- 1. Wprowadzenie do WPF i aplikacji desktopowych
 - 1.1. Czym jest WPF
 - 1.2. Jak działa aplikacja okienkowa
 - 1.3. XAML i C# jako dwa filary WPF
 - 1.4. Code-behind i logika za widokiem
 - 1.5. Typowy schemat zadań desktopowych
- 2. Przygotowanie środowiska i struktura projektu
 - 2.1. Instalacja Visual Studio
 - 2.2. Tworzenie projektu WPF
 - 2.3. Struktura plików projektu
 - 2.4. Cały folder projektu
 - 2.5. Uruchamianie i debugowanie projektu
- 3. Podstawy C# potrzebne do WPF
 - 3.1. Zmienne i typy danych
 - 3.2. Instrukcje warunkowe if, else if, else
 - 3.3. Instrukcja switch
 - 3.4. Pętle for, foreach, while
 - 3.5. Metody pomocnicze
 - 3.6. Parsowanie liczb — szczegółowo
 - 3.7. Fallback do wartości domyślnej
 - 3.8. Praca z tekstem: string
 - 3.9. Praca ze znakami: char
 - 3.10. Klasa Random
- 4. Klasy, obiekty i modele danych
 - 4.1. Czym jest klasa
 - 4.2. Pola i właściwości
 - 4.3. Konstruktor — tworzenie obiektu z danymi
 - 4.4. List
 - 4.5. Aktualny obiekt — jeden z wielu rekordów na ekranie
 - 4.6. Aktualizacja pojedynczych właściwości
- 5. Kolekcje i praca na danych
 - 5.1. Tablice
 - 5.2. List
 - 5.3. Indeks aktualnego elementu
 - 5.4. Przechodzenie do następnego i poprzedniego elementu
 - 5.5. Przechodzenie do następnego i poprzedniego elementu
 - 5.6. Zawijanie indeksu na początku i końcu listy
 - 5.7. List
- 6. Podstawy XAML
 - 6.1. Czym jest XAML
 - 6.2. Struktura pliku MainWindow.xaml
 - 6.3. Znaczniki i atrybuty
 - 6.4. x:Name i odwoływanie się do kontroltek w C#
 - 6.5. Najważniejsze właściwości kontroltek
 - 6.6. Komentarze w XAML
 - 6.7. Czcionki
 - 6.8. Marginesy, padding i odstępy
 - 6.9. Wyrównanie — HorizontalAlignment, VerticalAlignment
 - 6.10. Konwersja bool → Visibility
- 7. Layout i rozmieszczanie elementów
 - 7.1. Czym jest layout
 - 7.2. Grid — wiersze i kolumny
 - 7.3. Formularze na Grid
 - 7.4. StackPanel

- 7.5. DockPanel
- 7.6. WrapPanel
- 7.7. Canvas
- 7.8. Marginesy, odstępy i wyrównanie — praktyczne wskazówki
- 7.9. Zagnieżdżanie paneli
- 8. Kontrolki tekstowe
 - 8.1. TextBlock
 - 8.2. Label
 - 8.3. Text
 - 8.4. Wielowierszowy TextBox
 - 8.5. PasswordBox
- 9. Przyciski i akcje użytkownika
 - 9.1. Button
 - 9.2. Zdarzenie Click
 - 9.3. Jeden przycisk — jedna odpowiedzialność
 - 9.4. Kilka przycisków w jednym oknie
 - 9.5. Włączanie i wyłączanie przycisków: IsEnabled
- 10. Kontrolki wyboru
 - 10.1. CheckBox
 - 10.2. CheckBox
 - 10.3. Button
 - 10.4. Button
 - 10.5. ComboBox
 - 10.6. List
 - 10.7. List
- 11. Suwaki i wartości liczbowe
 - 11.1. Slider
 - 11.2. Zdarzenie ValueChanged
 - 11.3. TextBlock
 - 11.4. Slider
 - 11.5. SolidColorBrush
 - 11.6. Rectangle
- 12. Obrazy, grafiki i zasoby
 - 12.1. Kontrolka Image
 - 12.2. Build Action: Resource
 - 12.3. Wyświetlanie obrazu w XAML — BitmapImage, pack://application
 - 12.4. Dynamiczne ładowanie obrazu z obsługą błędów — kompletny przykład
 - 12.5. Button
- 13. Kolory, kształty i elementy wizualne
 - 13.1. Rectangle
 - 13.2. Border
 - 13.3. SolidColorBrush
 - 13.4. Text
 - 13.5. Rectangle
 - 13.6. ProgressBar
 - 13.7. Color, Brush i SolidColorBrush
 - 13.8. Pełna tabela kolorów nazwanych
 - 13.9. Kolory hex, RGB i ARGB
 - 13.10. Gradienty i pędzle obrazkowe
- 14. Zdarzenia w WPF
 - 14.1. Czym są zdarzenia
 - 14.2. Podpinanie zdarzeń w XAML
 - 14.3. Parametry sender i RoutedEventArgs
 - 14.4. Click
 - 14.5. TextChanged
 - 14.6. LostFocus
 - 14.7. List
 - 14.8. Button

- 14.9. Slider
- 14.10. Loaded
- 14.11. Zdarzenia klawiatury: KeyDown, KeyUp
- 14.12. Closing
- 15. Odczytywanie i ustawianie wartości kontrolek w C#
 - 15.1. Text
 - 15.2. CheckBox
 - 15.3. Button
 - 15.4. ComboBox
 - 15.5. Slider
 - 15.6. TextBlock
 - 15.7. Source
 - 15.8. Background
- 16. Formularze i pobieranie danych od użytkownika
 - 16.1. Budowa prostego formularza
 - 16.2. Text
 - 16.3. CheckBox
 - 16.4. Button
 - 16.5. ComboBox
 - 16.6. Slider
 - 16.7. Czyszczenie formularza po akcji
- 17. Walidacja danych
 - 17.1. Czym jest walidacja
 - 17.2. Sprawdzanie pustych pól
 - 17.3. Sprawdzanie długości tekstu
 - 17.4. Sprawdzanie, czy tekst zawiera tylko cyfry
 - 17.5. Sprawdzanie poprawności liczby
 - 17.6. Walidacja zakresu liczbowego
 - 17.7. Walidacja formatu przez Regex
 - 17.8. TextBlock
 - 17.9. Walidacja z fallbackiem
- 18. Komunikaty dla użytkownika
 - 18.1. MessageBox.Show()
 - 18.2. Button
 - 18.3. MessageBoxResult
 - 18.4. MessageBoxResult
 - 18.5. Komunikaty błędów
 - 18.6. Komunikaty wyniku działania
 - 18.7. Potwierdzanie akcji — dialog Tak/Nie przed niebezpieczną operacją
- 19. Stan aplikacji i aktualizacja widoku
 - 19.1. Czym jest stan aplikacji
 - 19.2. Zmienne przechowujące stan — pola klasy okna
 - 19.3. Stan lokalny i globalny
 - 19.4. Stan bieżący i zatwierdzony
 - 19.5. Aktualizacja danych tylko w pamięci
 - 19.6. Aktualizacja widoku na podstawie stanu — metoda OdswiezWidok()
- 20. Dynamiczna aktualizacja interfejsu
 - 20.1. TextBlock
 - 20.2. Pokazywanie wyniku dopiero po kliknięciu — wynik ukryty (Collapsed), pokazany po akcji
 - 20.3. Ukrywanie i pokazywanie elementów — Visibility.Visible, Hidden, Collapsed — różnice
 - 20.4. Source
 - 20.5. TextBlock
 - 20.6. Metoda OdswiezWidok() — centralna metoda aktualizująca wiele elementów naraz
- 21. Pliki i dane zewnętrzne
 - 21.1. Text
 - 21.2. Format danych w pliku — parsowanie struktury (Split, TryParse, kolejność linii)
 - 21.3. Tworzenie obiektów z danych tekstowych
 - 21.4. Text

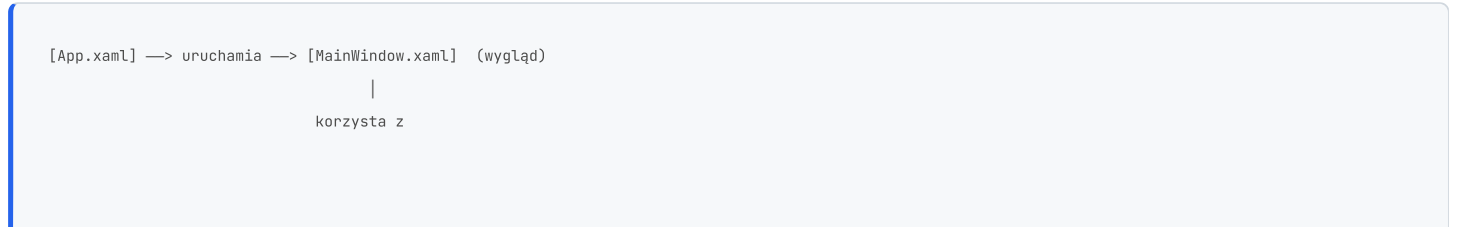
- 21.5. SaveFileDialog
- 21.6. OpenFileDialog
- 21.7. Sprawdzanie istnienia pliku: File.Exists
- 21.8. Ścieżki do plików — relatywne i absolutne, AppDomain.CurrentDomain.BaseDirectory
- 21.9. Obsługa błędów przy plikach
- 22. Algorytmy w aplikacjach WPF
 - 22.1. Oddzielenie algorytmu od interfejsu — metoda ZaszyfrujTekst(string tekst, int klucz) osobno
 - 22.2. Generator haseł
 - 22.3. Szyfr Cezara
 - 22.4. Walidator kodu pocztowego
 - 22.5. Przeliczanie ceny lub wyniku
 - 22.6. Aktualizacja licznika — int _liczbaPobran++, wyświetlanie w etykiecie
 - 22.7. StringBuilder
- 23. Generowanie losowych danych
 - 23.1. Klasa Random
 - 23.2. StringBuilder
 - 23.3. CheckBox
 - 23.4. List
- 24. Programowanie obiektowe w WPF
 - 24.1. Klasy i obiekty — tworzenie klasy Album z właściwościami
 - 24.2. Właściwości z get i set
 - 24.3. Hermetyzacja: private i public
 - 24.4. Generator haseł
 - 24.5. List
- 25. Binding — podstawy
 - 25.1. Czym jest binding
 - 25.2. Text
 - 25.3. Binding do właściwości
 - 25.4. Mode
 - 25.5. INotifyPropertyChanged — interfejs, PropertyChanged event, OnPropertyChanged(), kompletna implementacja
 - 25.6. List
 - 25.7. Mode
 - 25.8. ICommand
- 26. Style i wygląd aplikacji
 - 26.1. Background
 - 26.2. Source
 - 26.3. Source
 - 26.4. Button
 - 26.5. Trigger — PropertyTrigger, zmiana wyglądu gdy IsMouseOver=True, IsEnabled=False
 - 26.6. List
 - 26.7. SolidColorBrush
- 27. Nawigacja i wiele widoków
 - 27.1. Wiele okien
 - 27.2. UserControl
 - 27.3. Content
 - 27.4. Widok startowy, roboczy i wynikowy
- 28. Algorytmy w aplikacjach WPF
 - 28.1. Wzorzec: odczyt kontrolek → logika → aktualizacja UI — schemat każdej obsługi zdarzenia
 - 28.2. Przechowywanie stanu w polach klasy okna — private fields, kiedy i jak
 - 28.3. Loaded
 - 28.4. Obsługa wyjątków — try/catch w zdarzeniach, co łapać, co logować
 - 28.5. Blokowanie przycisku — IsEnabled = false przed operacją, true po zakończeniu
 - 28.6. Czyszczenie formularza po akcji
- 29. Porządkowanie kodu
 - 29.1. TextBlock
 - 29.2. Nazewnictwo metod
 - 29.3. Button
 - 29.4. Generator haseł

- 29.5. Komentarze i czytelność kodu — kiedy pisać komentarz, kiedy kod mówi sam za siebie
- 30. Debugowanie i typowe błędy
 - 30.1. Podgląd zmiennych
 - 30.2. Debugowanie XAML — Output window, brak zamknięcia znacznika, literówka w x:Name, brak metody zdarzenia
 - 30.3. Brak x:Name
 - 30.4. Błędy z plikami — FileNotFoundException, ścieżka względna vs absolutna, Debug vs Release
 - 30.5. List
- 31. Testowanie logiki aplikacji
 - 31.1. Debug i Release
 - 31.2. Budowanie aplikacji — Build Solution (Ctrl+Shift+B), gdzie powstają pliki .exe
 - 31.3. Publikowanie folderowe
 - 31.4. Aplikacja zależna od runtime
- 32. INotifyPropertyChanged i ObservableCollection
 - 32.1. Po co widokowi powiadomienia o zmianach
 - 32.2. Interfejs INotifyPropertyChanged
 - 32.3. Metoda OnPropertyChanged
 - 32.4. Aktualizacja pojedynczych właściwości
 - 32.5. Aktualizacja list przez ObservableCollection
 - 32.6. Typowe błędy przy braku powiadomień
- 33. Konwertery danych
 - 33.1. Po co stosować konwertery
 - 33.2. IValueConverter
 - 33.3. IMultiValueConverter
 - 33.4. Konwersja bool → Visibility
 - 33.5. Konwersja liczby na kolor
 - 33.6. Konwersja enum na tekst
 - 33.7. Parametry konwertera
- 34. Walidacja w bindingu
 - 34.1. Walidacja ręczna a walidacja bindingowa
 - 34.2. ValidationRule
 - 34.3. IDataErrorInfo
 - 34.4. INotifyDataErrorInfo
 - 34.5. Komunikaty błędów przy kontrolce
 - 34.6. Style dla błędów walidacji
- 35. MVVM w praktyce
 - 35.1. Model, View, ViewModel
 - 35.2. Kiedy warto używać MVVM
 - 35.3. ViewModel jako stan widoku
 - 35.4. ICommand
 - 35.5. RelayCommand
 - 35.6. CanExecute
 - 35.7. Parametry komend
 - 35.8. Przenoszenie logiki z code-behind
- 36. Zasoby i style
 - 36.1. Resources
 - 36.2. StaticResource
 - 36.3. DynamicResource
 - 36.4. Style lokalne
 - 36.5. Style globalne w App.xaml
 - 36.6. Setter
 - 36.7. TargetType
 - 36.8. Zasoby kolorów
 - 36.9. Prosty motyw aplikacji
- 37. Szablony i wygląd kontrolek
 - 37.1. DataTemplate
 - 37.2. ControlTemplate
 - 37.3. TemplateBinding
 - 37.4. Triggery w stylach

- 37.5. Proste motywy kontrolek
- 37.6. Różnica między stylem, szablonem i zasobem
- 38. Drzewo logiczne i wizualne
 - 38.1. Logical tree
 - 38.2. Visual tree
 - 38.3. Dziedziczenie właściwości
 - 38.4. Szukanie zasobów
 - 38.5. Wpływ drzewa na style
 - 38.6. Wpływ drzewa na binding
- 39. DependencyProperty i właściwości WPF
 - 39.1. Czym jest DependencyProperty
 - 39.2. Zwykła właściwość C# a właściwość zależności
 - 39.3. Dependency properties w kontrolkach WPF
 - 39.4. Wartości lokalne, style i binding
 - 39.5. Tworzenie własnej prostej właściwości zależności
- 40. CollectionView i prezentacja kolekcji
 - 40.1. Po co używać widoku kolekcji
 - 40.2. ICollectionView
 - 40.3. CollectionViewSource
 - 40.4. Sortowanie danych w widoku
 - 40.5. Filtrowanie danych w widoku
 - 40.6. Grupowanie danych
 - 40.7. Aktualny element kolekcji
- 41. Nawigacja i wiele widoków
 - 41.1. Wiele okien
 - 41.2. Show()
 - 41.3. ShowDialog()
 - 41.4. Przekazywanie danych między oknami
 - 41.5. UserControl
 - 41.6. ContentControl
 - 41.7. Widok startowy, roboczy i wyników
- 42. Asynchroniczność i responsywność UI
 - 42.1. Dlaczego interfejs może się zawieszać
 - 42.2. async i await
 - 42.3. Zadania długotrwałe
 - 42.4. Dispatcher
 - 42.5. Aktualizacja UI z innego wątku
 - 42.6. CancellationToken
 - 42.7. Blokowanie przycisku podczas operacji
- 43. Testowanie logiki aplikacji
 - 43.1. Co testować w aplikacji WPF
 - 43.2. Testy jednostkowe algorytmów
 - 43.3. Testowanie klas pomocniczych
 - 43.4. Testowanie ViewModeli
 - 43.5. Oddzielanie logiki od UI pod testy
 - 43.6. Przykładowe przypadki testowe dla walidatorów
- 44. Debugowanie i typowe błędy
 - 44.1. Breakpointy
 - 44.2. Podgląd zmiennych
 - 44.3. Błędy składni XAML
 - 44.4. Brak x:Name
 - 44.5. Niezgodne nazwy kontrolek
 - 44.6. Błędne ścieżki obrazów
 - 44.7. Błędy z listą i indeksem
 - 44.8. Brak odświeżenia widoku
- 45. Porządkowanie kodu
 - 45.1. Nazewnictwo kontrolek
 - 45.2. Nazewnictwo metod

- 45.3. Metody pomocnicze
- 45.4. Klasy serwisowe
- 45.5. Oddzielanie logiki od obsługi zdarzeń
- 45.6. Komentarze w kodzie
- 45.7. Formatowanie kodu
- 46. Budowanie, publikowanie i przekazywanie projektu
 - 46.1. Build projektu
 - 46.2. Debug i Release
 - 46.3. Folder wynikowy aplikacji
 - 46.4. Publikowanie folderowe
 - 46.5. Aplikacja zależna od runtime
 - 46.6. Aplikacja samowystarczalna
 - 46.7. Plik .exe
- 47. Dokumentowanie działania aplikacji
 - 47.1. Po co dokumentować stany aplikacji
 - 47.2. Zrzut ekranu stanu początkowego
 - 47.3. Zrzuty ekranu po interakcjach
 - 47.4. Widoczne okno programu
 - 47.5. Widoczne środowisko programistyczne
 - 47.6. Nazewnictwo plików ze zrzutami
 - 47.7. Dokumentowanie błędów kompilacji
- 48. Pakowanie projektu i plików źródłowych
 - 48.1. Co powinien zawierać przekazywany projekt
 - 48.2. Pliki XAML i C#
 - 48.3. Cały folder projektu
 - 48.4. Pliki wykonywalne
 - 48.5. Zasoby graficzne i pliki tekstowe
 - 48.6. Tworzenie archiwum .zip
 - 48.7. Kontrola poprawności paczki
- 49. Projekty praktyczne od prostych do średnich
 - 49.1. Licznik kliknięć
 - 49.2. Formularz danych osobowych
 - 49.3. Generator haseł
 - 49.4. Nadawanie przesyłki
 - 49.5. Dane paszportowe
 - 49.6. Przeglądarka albumów
 - 49.7. Wzornik kolorów RGB
 - 49.8. Szyfr Cezara
 - 49.9. Lista zadań
 - 49.10. Mini aplikacja wielowidokowa
- 50. Kluczowe mechaniki do powtarzania
 - 50.1. Formularz z wieloma kontrolkami
 - 50.2. Główny przycisk akcji
 - 50.3. Dynamiczna zmiana tekstu
 - 50.4. Dynamiczna zmiana obrazu
 - 50.5. Zdarzenie LostFocus
 - 50.6. Walidacja cyfr i długości tekstu
 - 50.7. Fallback do wartości domyślnej
 - 50.8. Odczyt listy obiektów z pliku
 - 50.9. Indeks cykliczny
 - 50.10. Suwaki RGB
 - 50.11. Zapis przez dialog systemowy
 - 50.12. Stan bieżący i zatwierdzony
 - 50.13. Atlas najważniejszych właściwości WPF
 - 50.14. Atlas najważniejszych zdarzeń WPF
- 51. Najczęstsze błędy (Wyjątki i błędy kompilacji)
 - 51.1. NullReferenceException
 - 51.2..XamlParseException

- ## 1. Wprowadzenie do WPF i aplikacji desktopowych



Ważna zasada: **plik XAML i plik C# to jedna całość**. Razem tworzą jedno okno. Nie można ich od siebie oddzielić.

1.3. XAML i C# jako dwa filary WPF

XAML (Extensible Application Markup Language) to język oparty na XML. Używasz go do opisania, jak wygląda okno — gdzie jest przycisk, jaki ma rozmiar, jakiego koloru jest tło. XAML jest czytelny i łatwy do edytowania. Visual Studio pokazuje podgląd w czasie rzeczywistym.

C# to język programowania. Pisziesz w nim logikę: co się dzieje po kliknięciu przycisku, jak obliczać wynik, skąd brać dane. C# nie opisuje wyglądu — tym zajmuje się XAML.

Poniżej widzisz najprostszą aplikację WPF — okno z jednym przyciskiem i etykietą:

MainWindow.xaml:

```
<Window x:Class="MojaAplikacja.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Moja pierwsza aplikacja WPF" Height="200" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Witaj w WPF!" FontSize="18" Margin="0,0,0,10"/>
        <Button Name="btnKliknij" Content="Kliknij mnie" Width="150" Click="btnKliknij_Click"/>
        <TextBlock Name="lblWynik" Text="" FontSize="14" Margin="0,10,0,0" Foreground="Green"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace MojaAplikacja
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnKliknij_Click(object sender, RoutedEventArgs e)
        {
            lblWynik.Text = "Działa! Kliknąłeś przycisk.";
        }
    }
}
```

Co tu się dzieje:

- `StackPanel` — układ, który ustawia elementy jeden pod drugim
- `TextBlock` — etykieta do wyświetlania tekstu (tylko do odczytu)
- `Button` — przycisk; atrybut `Click` wskazuje metodę w C#
- `Name="btnKliknij"` — nadajesz nazwę, żeby odwołać się do kontrolki w C#
- `btnKliknij_Click` — metoda wywoływana po kliknięciu przycisku

1.4. Code-behind i logika za widokiem

Code-behind (kod za widokiem) to plik C# powiązany z konkretnym plikiem XAML. W projekcie WPF każdy plik `.xaml` ma swój plik `.xaml.cs`. Razem tworzą jedną klasę — dlatego w XAML widzisz `x:Class="MojaAplikacja.MainWindow"`, a w C# `public partial class MainWindow`.

Słowo kluczowe `partial` oznacza, że klasa jest podzielona na dwa pliki — jeden generuje Visual Studio (z XAML), drugi piszesz Ty (logika).

W code-behind możesz:

- Obsługiwać kliknięcia przycisków
- Odczytywać i ustawiać tekst w kontrolkach
- Tworzyć obiekty i listy
- Walidować dane wpisane przez użytkownika
- Otwierać inne okna

Przykład — pobieranie tekstu z TextBox i wyświetlanie w TextBlock:

MainWindow.xaml:

```
<Window x:Class="CodeBehindDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Code-Behind Demo" Height="300" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Wpisz swoje imię:"/>
        <TextBox Name="txtImie" Width="200" HorizontalAlignment="Left"/>
        <Button Name="btnPowitaj" Content="Przywitaj mnie" Width="150"
                HorizontalAlignment="Left" Click="btnPowitaj_Click"/>
        <TextBlock Name="lblPowitanie" FontSize="16" Foreground="DarkBlue"
                TextWrapping="Wrap"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace CodeBehindDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnPowitaj_Click(object sender, RoutedEventArgs e)
        {
            // Odczytujemy tekst z pola TextBox
            string imie = txtImie.Text;

            // Sprawdzamy czy pole nie jest puste
            if (string.IsNullOrEmpty(imie))
            {
                lblPowitanie.Text = "Proszę wpisać imię!";
                lblPowitanie.Foreground = System.Windows.Media.Brushes.Red;
                return;
            }

            // Wyświetlamy powitanie
            lblPowitanie.Text = $"Witaj, {imie}! Miło Cię poznać.";
            lblPowitanie.Foreground = System.Windows.Media.Brushes.DarkBlue;
        }
    }
}
```

Kluczowe elementy:

- `txtImie.Text` — odczytujesz tekst wpisany przez użytkownika
- `lblPowitanie.Text = "..."` — ustawiasz tekst etykiety
- `string.IsNullOrEmpty()` — sprawdzasz, czy pole nie jest puste lub same spacje
- `return` — przerywasz metodę, jeśli dane są nieprawidłowe
- Interpolacja stringu — `$"Witaj, {imie}!"` — wstawiasz zmienną do tekstu

1.5. Typowy schemat zadań desktopowych

Większość aplikacji WPF, które będziesz pisać, opiera się na kilku powtarzalnych wzorcach. Warto je znać, bo przyspieszają pracę.

Wzorzec 1: Pobierz dane → Oblicz → Wyświetl

Użytkownik wpisuje liczby, klikasz przycisk, obliczasz wynik i pokazujesz go.

Wzorzec 2: Przeglądaj listę rekordów

Masz listę obiektów (np. albumów, pracowników). Użytkownik klika "Następny" / "Poprzedni" i widzi kolejne rekordy.

Wzorzec 3: Walidacja i komunikaty

Sprawdzasz, czy użytkownik wpisał poprawne dane. Jeśli nie — pokazujesz komunikat o błędzie.

Wzorzec 4: Losowanie

Klikasz przycisk, losuje się element z listy i wyświetla na ekranie.

Poniżej kompletny przykład łączący kilka wzorców — kalkulator BMI:

MainWindow.xaml:

```
<Window x:Class="KalkulatorBMI.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kalkulator BMI" Height="350" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Kalkulator BMI" FontSize="20" FontWeight="Bold"
                   HorizontalAlignment="Center" Margin="0,0,0,20"/>

        <TextBlock Text="Waga (kg):"/>
        <TextBox Name="txtWaga" Margin="0,2,0,10"/>

        <TextBlock Text="Wzrost (cm):"/>
        <TextBox Name="txtWzrost" Margin="0,2,0,10"/>

        <Button Name="btnOblicz" Content="Oblicz BMI" Click="btnOblicz_Click"
                Background="SteelBlue" Foreground="White" Padding="10,5" Margin="0,10,0,0"/>

        <TextBlock Name="lblWynik" FontSize="16" Margin="0,15,0,0"
                   HorizontalAlignment="Center" TextWrapping="Wrap"/>
        <TextBlock Name="lblKategoria" FontSize="14" Margin="0,5,0,0"
                   HorizontalAlignment="Center" FontWeight="Bold"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace KalkulatorBMI
{
    public partial class MainWindow : Window
    {
        public MainWindow()
```

```

{
    InitializeComponent();
}

private void btnOblicz_Click(object sender, RoutedEventArgs e)
{
    // Walidacja i parsowanie danych
    if (!double.TryParse(txtWaga.Text, out double waga) || waga <= 0)
    {
        MessageBox.Show("Podaj prawidłową wagę!", "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    if (!double.TryParse(txtWzrost.Text, out double wzrost) || wzrost <= 0)
    {
        MessageBox.Show("Podaj prawidłowy wzrost!", "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // Obliczenie BMI
    double wzrostM = wzrost / 100.0;
    double bmi = waga / (wzrostM * wzrostM);

    // Wyświetlenie wyniku
    lblWynik.Text = $"Twoje BMI: {bmi:F2}";

    // Kategoria BMI
    string kategoria = OkreslKategorie(bmi);
    lblKategoria.Text = kategoria;
}

private string OkreslKategorie(double bmi)
{
    if (bmi < 18.5)
        return "Niedowaga";
    else if (bmi < 25.0)
        return "Waga prawidłowa";
    else if (bmi < 30.0)
        return "Nadwaga";
    else
        return "Otyłość";
}
}

```

Co tu się dzieje:

- `double.TryParse()` — bezpieczne parsowanie liczby z TextBox
- `OkreslKategorie()` — osobna metoda z logiką — nie wszystko w Click!
- `MessageBox.Show()` — systemowe okienko z komunikatem
- `bmi:F2` — formatowanie liczby do 2 miejsc po przecinku
- `return` — przerywamy metodę przy błędnych danych

2. Przygotowanie środowiska i struktura projektu

2.1. Instalacja Visual Studio

Visual Studio to zintegrowane środowisko programistyczne (IDE) firmy Microsoft. Do tworzenia aplikacji WPF potrzebujesz wersji **Visual Studio Community** (bezpłatna) lub wyższej.

Kroki instalacji:

1. Wejdź na stronę visualstudio.microsoft.com i pobierz **Visual Studio Community**.
2. Uruchom instalator. Pojawi się okno z wyborem "obciążeń" (ang. workloads).
3. Zaznacz **"Programowanie aplikacji klasycznych dla platformy .NET"** (ang. *.NET desktop development*).
4. Kliknij **Instaluj** i poczekaj — pobieranie może zająć kilka minut.
5. Po instalacji uruchom Visual Studio i zaloguj się kontem Microsoft (możesz je założyć bezpłatnie).

Minimalne wymagania systemowe:

ELEMENT	WYMAGANIE
System	Windows 10 (wersja 1909+) lub Windows 11
RAM	Minimum 8 GB (zalecane 16 GB)
Dysk	Minimum 20 GB wolnego miejsca
Procesor	1,8 GHz lub szybszy

Po uruchomieniu Visual Studio zobaczysz ekran startowy z opcjami "Utwórz nowy projekt" i "Otwórz projekt".

2.2. Tworzenie projektu WPF

Po uruchomieniu Visual Studio:

1. Kliknij **"Utwórz nowy projekt"** (ang. *Create a new project*).
2. W polu wyszukiwania wpisz **"WPF"**.
3. Wybierz szablon **"Aplikacja WPF"** (ang. *WPF Application*) — upewnij się, że jest to wersja **C#**, nie VB.NET.
4. Kliknij **Dalej**.
5. Podaj **nazwę projektu** — np. `MojaAplikacja`. Ważne: bez polskich znaków i spacji!
6. Wybierz **lokalizację** na dysku.
7. Kliknij **Dalej**.
8. Wybierz **wersję .NET** — wybierz najnowszą dostępną (np. .NET 8.0 lub .NET 9.0).
9. Kliknij **Utwórz**.

Visual Studio automatycznie wygeneruje strukturę projektu z podstawowymi plikami.

Nazwa projektu jest równocześnie **namespace** w C#. Jeśli podasz nazwę z polskimi znakami lub spacją, C# nie skompiluje projektu.

2.3. Struktura plików projektu

Po stworzeniu projektu w panelu **Solution Explorer** (po prawej stronie) zobaczysz następujące pliki:

```
MojaAplikacja/
├─ App.xaml
├─ App.xaml.cs
├─ MainWindow.xaml
├─ MainWindow.xaml.cs
├─ MojaAplikacja.csproj
├─ Properties/
└─ launchSettings.json
```

Opis każdego pliku:

APP.XAML

Plik konfiguracyjny całej aplikacji. Definiuje, które okno otwiera się jako pierwsze (`StartupUri`) oraz globalne zasoby (style, kolory).

```
<Application x:Class="MojaAplikacja.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    StartupUri="MainWindow.xaml">

    <Application.Resources>
        <!-- Tutaj możesz dodać globalne style i zasoby -->
    </Application.Resources>
</Application>
```

`StartupUri="MainWindow.xaml"` — to ten plik XAML zostanie otwarty przy starcie aplikacji.

APP.XAML.CS

Code-behind dla `App.xaml`. Rzadko edytowany, ale można tu wykonać kod przed startem aplikacji.

```
using System;
using System.Windows;

namespace MojaAplikacja
{
    public partial class App : Application
    {
        // Możesz tutaj dodać kod inicjalizujący aplikację
        // np. ładowanie danych z pliku, sprawdzanie licencji itp.
    }
}
```

MAINWINDOW.XAML

Główne okno aplikacji — tutaj projektujesz interfejs użytkownika. Zawiera wszystkie kontrolki: przyciski, pola tekstowe, etykiety.

```
<Window x:Class="MojaAplikacja.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Moja Aplikacja" Height="450" Width="800">
    <Grid>
        <!-- Tutaj dodajesz kontrolki -->
    </Grid>
</Window>
```

MAINWINDOW.XAML.CS

Code-behind dla okna głównego. Tu piszesz całą logikę obsługi interfejsu.

```
using System;
using System.Windows;

namespace MojaAplikacja
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent(); // ZAWSZE musi być w konstruktorze!
        }
    }
}
```

MOJAAPLIKACJA.CSPROJ

Plik projektu w formacie XML. Zawiera informacje o docelowej platformie .NET, referencjach do bibliotek i ustawieniach kompilacji. Zazwyczaj nie edytujesz go ręcznie.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net8.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
  </PropertyGroup>
</Project>
```

`UseWPF>true` — to kluczowa linia włączająca obsługę WPF.

2.4. Cały folder projektu

Gdy projekt rośnie, warto zorganizować pliki w folderach. Standardowa struktura dla aplikacji WPF wygląda tak:

```
MojaAplikacja/
├─ Models/          ← klasy danych (Album.cs, Pracownik.cs)
├─ Services/        ← logika biznesowa, pomocnicze metody
├─ Views/           ← dodatkowe okna (jeśli masz więcej niż jedno)
├─ Images/          ← obrazki, ikony
├─ App.xaml
├─ App.xaml.cs
├─ MainWindow.xaml
└─ MainWindow.xaml.cs
```

Tworzenie folderu w Visual Studio:

1. Kliknij prawym przyciskiem myszy na nazwie projektu w Solution Explorer.
2. Wybierz **Dodaj** → **Nowy folder** (Add → New Folder).
3. Wpisz nazwę folderu.

Tworzenie klasy w folderze Models:

1. Kliknij prawym przyciskiem na folderze `Models`.
2. Wybierz **Dodaj** → **Klasa** (Add → Class).
3. Wpisz nazwę klasy, np. `Album.cs`.

Przykładowa klasa w folderze Models:

```
namespace MojaAplikacja.Models
{
    public class Album
    {
        public string Tytuł { get; set; }
        public string Artysta { get; set; }
        public int RokWydania { get; set; }
    }
}
```

W MainWindow.xaml.cs importujesz ją przez:

```
using MojaAplikacja.Models;
```

2.5. Uruchamianie i debugowanie projektu

Uruchamianie aplikacji:

- Naciśnij **F5** — uruchamia aplikację z debuggerem (możesz zatrzymywać w punktach przerwania).
- Naciśnij **Ctrl+F5** — uruchamia aplikację bez debuggera (szybciej).
- Kliknij zielony przycisk ► **Start** na pasku narzędzi.

Punkty przerwania (breakpoints):

Kliknij szary margines po lewej stronie numeru linii — pojawi się czerwona kropka. Gdy wykonanie kodu dojdzie do tej linii, aplikacja zatrzyma się i możesz sprawdzić wartości zmiennych.

Okno Output:

Menu **Widok** → **Dane wyjściowe** (View → Output) — pokazuje komunikaty błędów kompilacji.

Lista błędów:

Menu **Widok** → **Lista błędów** (View → Error List) — po nieudanej kompilacji widzisz tu wszystkie błędy z informacją o pliku i numerze linii.

Debugowanie krok po kroku:

SKRÓT	DZIAŁANIE
F5	Uruchom z debuggerem
F9	Ustaw/usuń punkt przerwania
F10	Wykonaj następną linię (bez wchodzenia w metody)
F11	Wejdź do metody
Shift+F5	Zatrzymaj debugowanie

Przykład z punktem przerwania:

Ustaw punkt przerwania w metodzie obsługi kliknięcia i sprawdź, jaką wartość ma zmienna `imie` po wpisaniu tekstu przez użytkownika. Najedź myszką na zmienną — zobaczysz jej wartość w dymku (tooltip).

3. Podstawy C# potrzebne do WPF

3.1. Zmienne i typy danych

Zmienna to "pudełko" z etykietą, które przechowuje wartość. W C# każda zmienna ma określony **typ** — mówi on, co można w niej przechowywać.

Najważniejsze typy w WPF:

TYP	CO PRZECHOWUJE	PRZYKŁAD
<code>int</code>	Liczby całkowite	<code>42</code> , <code>-7</code> , <code>0</code>
<code>double</code>	Liczby zmiennoprzecinkowe (duże)	<code>3.14</code> , <code>-0.5</code>
<code>decimal</code>	Liczby pieniężne (bardzo dokładne)	<code>19.99m</code> , <code>1234.56m</code>
<code>string</code>	Tekst	<code>"Witaj"</code> , <code>"123"</code>
<code>bool</code>	Prawda lub fałsz	<code>true</code> , <code>false</code>
<code>char</code>	Pojedynczy znak	<code>'A'</code> , <code>'5'</code> , <code>'!'</code>

Dane z `TextBox` to ZAWSZE `string`! Nawet jeśli użytkownik wpisał `"42"`, to jest tekst, nie liczba. Żeby użyć tego jako liczby, musisz ją **sparsować** (patrz sekcja 3.6).

Deklaracja zmiennych w C#:

```
// Typy liczbowe
int wiek = 25;
int liczbaStudentow = 0;

double wzrost = 1.75;
double temperatura = -3.5;

decimal cena = 29.99m; // 'm' na końcu = decimal
decimal rabat = 0.10m;

// Tekst
string imie = "Kacper";
string pusteTekst = "";
string? nazwaOpcjonalna = null; // może być null

// Logiczne
bool czyZalogowany = false;
bool czyPoprawne = true;

// Znak
char pierwszaLitera = 'K';
char cyfra = '7';
```

Słowo kluczowe `var`:

Możesz użyć `var` zamiast nazwy typu — C# sam wywnioskuje typ:

```
var liczba = 42;           // int
var tekst = "Witaj";      // string
var cena = 9.99m;         // decimal
var aktywny = true;       // bool
```

Przykład zastosowania w WPF — odczyt i wyświetlanie:

MainWindow.xaml:

```
<Window x:Class="TypyDanych.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Typy danych" Height="300" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Podaj rok urodzenia:"/>
        <TextBox Name="txtRok" Margin="0,4,0,10"/>
        <Button Content="Oblicz wiek" Click="btnOblicz_Click"
                HorizontalAlignment="Left" Padding="10,5"/>
        <TextBlock Name="lblWynik" Margin="0,10,0,0" FontSize="14"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace TypyDanych
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnOblicz_Click(object sender, RoutedEventArgs e)
        {
            // txtRok.Text to STRING, nie int!
            string tekstRoku = txtRok.Text;

            // Parsujemy string na int
            if (!int.TryParse(tekstRoku, out int rokUrodzenia))
            {
                lblWynik.Text = "Podaj prawidłowy rok (np. 2000)";
                return;
            }

            // Teraz rokUrodzenia to int, możemy obliczać
            int aktualnyRok = DateTime.Now.Year;
            int wiek = aktualnyRok - rokUrodzenia;

            // Wyświetlamy wynik jako string
            lblWynik.Text = $"Masz około {wiek} lat.";
        }
    }
}
```

```
}  
  
}
```

3.2. Instrukcje warunkowe if, else if, else

Instrukcja `if` pozwala wykonać kod tylko wtedy, gdy warunek jest spełniony.

Składnia:

```
if (warunek)  
{  
    // kod wykonywany gdy warunek = true  
}  
else if (inny_warunek)  
{  
    // kod gdy pierwszy warunek = false, ale ten = true  
}  
else  
{  
    // kod gdy żaden warunek nie jest spełniony  
}
```

Operatory porównania:

OPERATOR	ZNACZENIE
<code>=</code>	równa się
<code>≠</code>	różni się
<code>></code>	większy niż
<code><</code>	mniej niż
<code>≥</code>	większy lub równy
<code>≤</code>	mniej niż lub równy

Operatory logiczne:

OPERATOR	ZNACZENIE	PRZYKŁAD
<code>&&</code>	i (AND)	<code>wiek > 18 && wiek < 65</code>
<code> </code>	lub (OR)	<code>ocena < 2 ocena > 6</code>
<code>!</code>	nie (NOT)	<code>!czyZalogowany</code>

Przykład 1 — ocenianie wyników:

MainWindow.xaml:

```

<Window x:Class="WarunkiDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Sprawdzenie wyniku" Height="320" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Podaj wynik procentowy (0-100):" FontSize="13"/>
        <TextBox Name="txtWynik" Margin="0,5,0,10" Width="120" HorizontalAlignment="Left"/>
        <Button Content="Sprawdź ocenę" Click="btnSprawdz_Click"
                HorizontalAlignment="Left" Padding="10,5"/>
        <TextBlock Name="lblOcena" FontSize="18" Margin="0,15,0,0" FontWeight="Bold"/>
        <TextBlock Name="lblKomentarz" FontSize="13" Margin="0,5,0,0" TextWrapping="Wrap"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WarunkiDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnSprawdz_Click(object sender, RoutedEventArgs e)
        {
            if (!int.TryParse(txtWynik.Text, out int wynik))
            {
                lblOcena.Text = "Błąd!";
                lblKomentarz.Text = "Podaj liczbę całkowitą.";
                return;
            }

            if (wynik < 0 || wynik > 100)
            {
                lblKomentarz.Text = "Wynik musi być w zakresie 0-100.";
                return;
            }

            // Sprawdzanie oceny
            if (wynik ≥ 90)
            {
                lblOcena.Text = "Ocena: 5 (celujący)";
                lblKomentarz.Text = "Doskonały wynik! Brawo!";
                lblOcena.Foreground = System.Windows.Media.Brushes.DarkGreen;
            }
            else if (wynik ≥ 75)
            {
                lblOcena.Text = "Ocena: 4 (dobry)";
                lblKomentarz.Text = "Dobry wynik. Tak trzymaj!";
                lblOcena.Foreground = System.Windows.Media.Brushes.Green;
            }
            else if (wynik ≥ 60)
            {
                lblOcena.Text = "Ocena: 3 (dostateczny)";
                lblKomentarz.Text = "Wynik wystarczający. Warto powtórzyć materiał.";
            }
        }
    }
}

```

```

        lblOcena.Foreground = System.Windows.Media.Brushes.DarkOrange;
    }
    else if (wynik ≥ 50)
    {
        lblOcena.Text = "Ocena: 2 (dopuszczający)";
        lblKomentarz.Text = "Minimalny próg zaliczenia.";
        lblOcena.Foreground = System.Windows.Media.Brushes.Orange;
    }
    else
    {
        lblOcena.Text = "Ocena: 1 (niedostateczny)";
        lblKomentarz.Text = "Nie zaliczono. Konieczna poprawa.";
        lblOcena.Foreground = System.Windows.Media.Brushes.Red;
    }
    }
}
}

```

Przykład 2 — sprawdzanie czy liczba jest parzysta:

```

private void btnSprawdzParzystosc_Click(object sender, RoutedEventArgs e)
{
    if (!int.TryParse(txtLiczba.Text, out int liczba))
    {
        lblWynik.Text = "Podaj liczbę całkowitą!";
        return;
    }

    // Operator modulo % zwraca resztę z dzielenia
    if (liczba % 2 == 0)
    {
        lblWynik.Text = $"{liczba} jest parzysta.";
    }
    else
    {
        lblWynik.Text = $"{liczba} jest nieparzysta.";
    }
}

```

3.3. Instrukcja switch

Instrukcja `switch` to alternatywa dla długiego łańcucha `if-else if`. Sprawdza wartość jednej zmiennej i wykonuje odpowiedni blok kodu.

Składnia klasyczna:

```

switch (zmienna)
{
    case wartość1:
        // kod dla wartość1
        break;
    case wartość2:
        // kod dla wartość2
        break;
    default:
        // kod gdy żaden case nie pasuje
        break;
}

```

Przykład — dni tygodnia:

MainWindow.xaml:

```
<Window x:Class="SwitchDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Dni tygodnia" Height="300" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Podaj numer dnia (1-7):" FontSize="13"/>
        <TextBox Name="txtDzien" Margin="0,5,0,10" Width="80" HorizontalAlignment="Left"/>
        <Button Content="Sprawdź" Click="btnSprawdz_Click"
                HorizontalAlignment="Left" Padding="10,5"/>
        <TextBlock Name="lblDzien" FontSize="16" Margin="0,15,0,0"/>
        <TextBlock Name="lblInfo" FontSize="12" Margin="0,5,0,0" Foreground="Gray"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace SwitchDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnSprawdz_Click(object sender, RoutedEventArgs e)
        {
            if (!int.TryParse(txtDzien.Text, out int numer))
            {
                lblDzien.Text = "Podaj numer 1-7!";
                return;
            }

            string dzien;
            string info;

            switch (numer)
            {
                case 1:
                    dzien = "Poniedziałek";
                    info = "Pierwszy dzień tygodnia roboczego.";
                    break;
                case 2:
                    dzien = "Wtorek";
                    info = "Drugi dzień tygodnia roboczego.";
                    break;
                case 3:
                    dzien = "Środa";
                    info = "Połowa tygodnia roboczego.";
                    break;
                case 4:
                    dzien = "Czwartek";
                    info = "Przedostatni dzień roboczy.";
                    break;
                case 5:
                    dzien = "Piątek";
```

```

        info = "Ostatni dzień tygodnia roboczego!";
        break;
    case 6:
        dzien = "Sobota";
        info = "Weekend!";
        break;
    case 7:
        dzien = "Niedziela";
        info = "Dzień odpoczynku.";
        break;
    default:
        dzien = "Nieznany dzień";
        info = "Podaj liczbę od 1 do 7.";
        break;
    }

    lblDzien.Text = dzien;
    lblInfo.Text = info;
}
}
}

```

Switch expression (nowszy styl C# 8+):

```

string nazwaSezon = numer switch
{
    1 => "Wiosna",
    2 => "Lato",
    3 => "Jesień",
    4 => "Zima",
    _ => "Nieznany sezon" // '_' to default
};

```

Switch z wieloma case dla jednego bloku:

```

switch (miesiac)
{
    case 12:
    case 1:
    case 2:
        lblSezon.Text = "Zima";
        break;
    case 3:
    case 4:
    case 5:
        lblSezon.Text = "Wiosna";
        break;
    case 6:
    case 7:
    case 8:
        lblSezon.Text = "Lato";
        break;
    default:
        lblSezon.Text = "Jesień";
        break;
}

```

3.4. Pętle for, foreach, while

Pętla pozwala powtarzać kod wielokrotnie bez pisania go kilka razy.

PĘTLA FOR

Używasz jej gdy wiesz z góry, ile razy coś ma się powtórzyć.

```
for (int i = 0; i < 10; i++)
{
    // kod wykonywany 10 razy (i = 0, 1, 2, ..., 9)
}
```

Przykład — suma liczb i wyświetlanie:

MainWindow.xaml:

```
<Window x:Class="PetleDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Pętla w WPF" Height="400" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Sumuj liczby od 1 do N:" FontSize="13"/>
        <TextBox Name="txtN" Margin="0,5,0,10" Width="100" HorizontalAlignment="Left"/>
        <Button Content="Oblicz sumę" Click="btnSuma_Click"
                HorizontalAlignment="Left" Padding="10,5"/>
        <TextBlock Name="lblWynik" FontSize="16" Margin="0,10,0,0"/>

        <Separator Margin="0,15"/>
        <Button Content="Dodaj 5 elementów do listy" Click="btnDodaj_Click"
                HorizontalAlignment="Left" Padding="10,5"/>
        <ListBox Name="listaElementow" Height="120" Margin="0,10,0,0"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace PetleDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnSuma_Click(object sender, RoutedEventArgs e)
        {
            if (!int.TryParse(txtN.Text, out int n) || n ≤ 0)
            {
                lblWynik.Text = "Podaj dodatnią liczbę całkowitą!";
                return;
            }

            int suma = 0;
            for (int i = 1; i ≤ n; i++)
            {
                suma += i; // suma = suma + i
            }
        }
    }
}
```

```

        lblWynik.Text = $"Suma liczb od 1 do {n} = {suma}";
    }

    private void btnDodaj_Click(object sender, RoutedEventArgs e)
    {
        listaElementow.Items.Clear();

        // Pętla for dodająca elementy do ListBox
        for (int i = 1; i ≤ 5; i++)
        {
            listaElementow.Items.Add($"Element numer {i}");
        }
    }
}

```

PĘTLA FOREACH

Używasz jej do przechodzenia przez każdy element kolekcji (listy, tablicy).

```

string[] owoce = { "Jabłko", "Banan", "Gruszka" };

foreach (string owoc in owoce)
{
    // owoc przyjmuje kolejno: "Jabłko", "Banan", "Gruszka"
    Console.WriteLine(owoc);
}

```

Przykład foreach w WPF:

```

private void btnWyswietlListę_Click(object sender, RoutedEventArgs e)
{
    var nazwy = new List<string> { "Anna", "Bartosz", "Celina", "Dawid" };

    listaOsob.Items.Clear();

    foreach (string nazwa in nazwy)
    {
        listaOsob.Items.Add(nazwa);
    }
}

```

PĘTLA WHILE

Wykonuje kod dopóki warunek jest prawdziwy. Używasz jej gdy nie wiesz z góry ile razy pętla ma się wykonać.

```

int licznik = 0;
while (licznik < 5)
{
    // kod wykonywany dopóki licznik < 5
    licznik++;
}

```

Przykład — szukanie pierwszej parzystej liczby większej od N:

```

private void btnSzukaj_Click(object sender, RoutedEventArgs e)
{
    if (!int.TryParse(txtPoczek.Text, out int start))

```

```

        return;

        int liczba = start + 1;
        while (liczba % 2 != 0)
        {
            liczba++;
        }

        lblWynik.Text = $"Pierwsza parzysta liczba po {start} to {liczba}";
    }
}

```

3.5. Metody pomocnicze

Bardzo częsty błąd początkujących: wpisanie całej logiki aplikacji bezpośrednio do metody obsługi kliknięcia. To prowadzi do kodu, który jest trudny do czytania, testowania i naprawiania.

Złe podejście:

```

private void btnOblicz_Click(object sender, RoutedEventArgs e)
{
    // 50 linii logiki tutaj – ZłE!
    double waga = double.Parse(txtWaga.Text);
    double wzrost = double.Parse(txtWzrost.Text) / 100.0;
    double bmi = waga / (wzrost * wzrost);
    string kategoria;
    if (bmi < 18.5) kategoria = "Niedowaga";
    else if (bmi < 25) kategoria = "Prawidłowa";
    else if (bmi < 30) kategoria = "Nadwaga";
    else kategoria = "Otyłość";
    lblBMI.Text = bmi.ToString("F2");
    lblKategoria.Text = kategoria;
    // itd...
}

```

Dobre podejście — osobne metody:

MainWindow.xaml:

```

<Window x:Class="MetodyDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Metody pomocnicze" Height="380" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Waga (kg):" />
        <TextBox Name="txtWaga" Margin="0,3,0,8" />
        <TextBlock Text="Wzrost (cm):" />
        <TextBox Name="txtWzrost" Margin="0,3,0,8" />
        <Button Content="Oblicz BMI" Click="btnOblicz_Click"
                Padding="10,5" HorizontalAlignment="Left" />
        <TextBlock Name="lblBMI" FontSize="16" Margin="0,12,0,0" />
        <TextBlock Name="lblKategoria" FontSize="13" Margin="0,5,0,0" Foreground="Gray" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

```

```
namespace MetodyDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        // Metoda obsługi kliknięcia – tylko koordynuje
        private void btnOblicz_Click(object sender, RoutedEventArgs e)
        {
            if (!PobierzDane(out double waga, out double wzrost))
                return;

            double bmi = ObliczBMI(waga, wzrost);
            string kategoria = OkreslKategorieBMI(bmi);

            WyszwietlWynik(bmi, kategoria);
        }

        // Metoda walidacji i pobierania danych
        private bool PobierzDane(out double waga, out double wzrost)
        {
            waga = 0;
            wzrost = 0;

            if (!double.TryParse(txtWaga.Text, out waga) || waga ≤ 0)
            {
                MessageBox.Show("Podaj prawidłową wagę!", "Błąd");
                return false;
            }

            if (!double.TryParse(txtWzrost.Text, out wzrost) || wzrost ≤ 0)
            {
                MessageBox.Show("Podaj prawidłowy wzrost!", "Błąd");
                return false;
            }

            return true;
        }

        // Metoda obliczeniowa – tylko liczy
        private double ObliczBMI(double waga, double wzrostCm)
        {
            double wzrostM = wzrostCm / 100.0;
            return waga / (wzrostM * wzrostM);
        }

        // Metoda klasyfikująca wynik
        private string OkreslKategorieBMI(double bmi)
        {
            if (bmi < 18.5) return "Niedowaga";
            if (bmi < 25.0) return "Waga prawidłowa";
            if (bmi < 30.0) return "Nadwaga";
            return "Otyłość";
        }

        // Metoda wyświetlająca wynik
        private void WyszwietlWynik(double bmi, string kategoria)
        {
            lblBMI.Text = $"BMI: {bmi:F2}";
        }
    }
}
```

```

        lblKategoria.Text = $"Kategoria: {kategoria}";
    }
}
}

```

Zalety osobnych metod:

- **Czytelność** — od razu widać co metoda robi (po nazwie)
- **Wielokrotne użycie** — tę samą metodę możesz wywołać z kilku miejsc
- **Testowanie** — możesz sprawdzić każdą metodę osobno
- **Łatwiejsze naprawianie błędów** — wiesz dokładnie, gdzie szukać problemu

3.6. Parsowanie liczb — szczegółowo

Dane z `TextBox.Text` to zawsze `string`. Zanim użyjesz ich jako liczby, musisz je **sparsować** (zamienić na odpowiedni typ).

METODA 1: INT.PARSE() / DOUBLE.PARSE()

Prosta, ale rzuca wyjątek gdy tekst nie jest liczbą. Używaj tylko gdy masz pewność, że dane są poprawne (rzadko).

```

string tekst = "42";

int liczba = int.Parse(tekst);      // OK, liczba = 42
double d = double.Parse("3.14");   // OK, d = 3.14

// NIEBEZPIECZNE:
int blad = int.Parse("abc");        // Wyjątek! FormatException
int blad2 = int.Parse("");          // Wyjątek! FormatException

```

METODA 2: CONVERT.TOINT32() / CONVERT.TODOUBLE()

Podobna do Parse, ale obsługuje null. Nadal rzuca wyjątek dla nieprawidłowego formatu.

```

int liczba = Convert.ToInt32("42");    // OK
double d = Convert.ToDouble("3.14");   // OK
int z_null = Convert.ToInt32(null);    // OK, zwraca 0 (nie rzuca wyjątku)

```

METODA 3: INT.TRYPARSE() — ZALECANA W WPF

Bezpieczna metoda. Nie rzuca wyjątku — zwraca `true` jeśli parsowanie się powiodło, `false` jeśli nie.

```

// Składnia:
bool sukces = int.TryParse(tekst, out int wynik);

// Przykład:
if (int.TryParse(txtLiczba.Text, out int liczba))
{
    // Parsowanie się powiodło, liczba ma wartość
    lblWynik.Text = $"Wpisałeś: {liczba}";
}
else
{
    // Parsowanie nie powiodło się
    lblWynik.Text = "To nie jest liczba całkowita!";
}

```

Tabela porównawcza metod parsowania:

METODA	BŁĘDNE DANE	NULL	ZALECANA DLA
<code>int.Parse()</code>	Wyjątek	Wyjątek	Pewnych danych
<code>Convert.ToInt32()</code>	Wyjątek	0	Pewnych danych
<code>int.TryParse()</code>	<code>false</code>	<code>false</code>	Danych od użytkownika ✓

Kompletny przykład — wszystkie typy parsowania:

MainWindow.xaml:

```
<Window x:Class="ParsowanieDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Parsowanie liczb" Height="420" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Liczba całkowita:" FontSize="13"/>
        <TextBox Name="txtInt" Margin="0,4,0,10"/>

        <TextBlock Text="Liczba dziesiętna:" FontSize="13"/>
        <TextBox Name="txtDouble" Margin="0,4,0,10"/>

        <TextBlock Text="Kwota (pieniądze):" FontSize="13"/>
        <TextBox Name="txtDecimal" Margin="0,4,0,10"/>

        <Button Content="Parsuj wszystkie" Click="btnParsuj_Click"
                Padding="10,5" HorizontalAlignment="Left"/>
        <TextBlock Name="lblWyniki" Margin="0,12,0,0" TextWrapping="Wrap"
                FontFamily="Consolas" FontSize="12"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace ParsowanieDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnParsuj_Click(object sender, RoutedEventArgs e)
        {
            string wyniki = "";

            // Parsowanie int
            if (int.TryParse(txtInt.Text, out int liczbaInt))
                wyniki += $"int: {liczbaInt} (OK)\n";
            else
                wyniki += $"int: BŁĄD - '{txtInt.Text}' to nie int\n";

            // Parsowanie double
            // Uwaga: separator dziesiętny zależy od ustawień systemu
            // W Polsce zwykle ',' ale w kodzie często używamy '.'
            if (double.TryParse(txtDouble.Text, out double liczbaDouble))
                wyniki += $"double: {liczbaDouble:F4} (OK)\n";
```



```

        else
            wyniki += $"double: Błąd - '{txtDouble.Text}' to nie double\n";

        // Parsowanie decimal
        if (decimal.TryParse(txtDecimal.Text, out decimal liczbaDecimal))
            wyniki += $"decimal: {liczbaDecimal:C} (OK)\n"; // :C = format walutowy
        else
            wyniki += $"decimal: Błąd - '{txtDecimal.Text}' to nie decimal\n";

        lblWyniki.Text = wyniki;
    }
}
}

```

Parsowanie z przecinkiem — ważna uwaga:

W Polsce separator dziesiętny to przecinek (,), ale w kodzie C# literały używają kropki (.). `double.TryParse("3,14")` zadziała na systemie z polskimi ustawieniami regionalnymi, ale `double.TryParse("3.14")` może nie zadziałać. Żeby uniknąć problemów, używaj:

```

// Parsowanie z obsługą obu separatorów:
using System.Globalization;

// Obsługuje "3.14" niezależnie od ustawień systemu:
if (double.TryParse(txtWartość.Text,
    NumberStyles.Any,
    CultureInfo.InvariantCulture,
    out double wynik))
{
    // sukces
}

```

3.7. Fallback do wartości domyślnej

Fallback to wzorec, w którym gdy parsowanie się nie uda lub dane są puste, używasz z góry ustalonej wartości domyślnej zamiast przerywać działanie programu.

Wzorec 1 — operator warunkowy `?:`:

```

// Jeśli TryParse zwróci false, liczba = 0 (wartość domyślna)
int liczba = int.TryParse(txtWartość.Text, out int parsowana) ? parsowana : 0;

```

Wzorec 2 — własna metoda pomocnicza:

```

private int PobierzInt(string tekst, int domyslne = 0)
{
    return int.TryParse(tekst, out int wynik) ? wynik : domyslne;
}

private double PobierzDouble(string tekst, double domyslne = 0.0)
{
    return double.TryParse(tekst, out double wynik) ? wynik : domyslne;
}

```

Kompletny przykład zastosowania wzorca fallback:

MainWindow.xaml:

```

<Window x:Class="FallbackDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kalkulator z fallback" Height="340" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Liczba 1 (domyślnie 0):" />
        <TextBox Name="txtA" Margin="0,4,0,10" />
        <TextBlock Text="Liczba 2 (domyślnie 1):" />
        <TextBox Name="txtB" Margin="0,4,0,10" />
        <Button Content="Podziel A / B" Click="btnPodziel_Click"
                Padding="10,5" HorizontalAlignment="Left" />
        <TextBlock Name="lblWynik" FontSize="15" Margin="0,12,0,0" />
        <TextBlock Name="lblInfo" FontSize="11" Foreground="Gray" Margin="0,4,0,0" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace FallbackDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnPodziel_Click(object sender, RoutedEventArgs e)
        {
            // Fallback: A domyślnie 0, B domyślnie 1
            double a = PobierzDouble(txtA.Text, domyslNa: 0.0);
            double b = PobierzDouble(txtB.Text, domyslNa: 1.0);

            // Zabezpieczenie przed dzieleniem przez zero
            if (b == 0)
            {
                lblWynik.Text = "Błąd: dzielenie przez zero!";
                lblInfo.Text = "B nie może być równe zero.";
                return;
            }

            double wynik = a / b;
            lblWynik.Text = $"{a} / {b} = {wynik:F4}";
            lblInfo.Text = "Obliczono pomyślnie.";
        }

        // Metoda pomocnicza z fallback
        private double PobierzDouble(string tekst, double domyslNa)
        {
            if (double.TryParse(tekst, out double wynik))
                return wynik;

            lblInfo.Text += $" [użyto wartości domyślnej: {domyslNa}]";
            return domyslNa;
        }
    }
}

```

```
}  
  
}
```

3.8. Praca z tekstem: string

`string` w C# ma wiele wbudowanych metod do manipulacji tekstem.

Najważniejsze właściwości i metody:

METODA/WŁAŚCIWOŚĆ	CO ROBI	PRZYKŁAD
<code>.Length</code>	Długość tekstu	<code>"Witaj".Length</code> → 5
<code>.ToUpper()</code>	Zamień na duże litery	<code>"hello".ToUpper()</code> → "HELLO"
<code>.ToLower()</code>	Zamień na małe litery	<code>"HELLO".ToLower()</code> → "hello"
<code>.Trim()</code>	Usuń spacje z początku i końca	<code>" abc ".Trim()</code> → "abc"
<code>.TrimStart()</code>	Usuń spacje z początku	<code>" abc".TrimStart()</code> → "abc"
<code>.TrimEnd()</code>	Usuń spacje z końca	<code>"abc ".TrimEnd()</code> → "abc"
<code>.Contains(s)</code>	Czy zawiera podciąg?	<code>"Witaj".Contains("ita")</code> → true
<code>.StartsWith(s)</code>	Czy zaczyna się od?	<code>"Witaj".StartsWith("Wi")</code> → true
<code>.EndsWith(s)</code>	Czy kończy się na?	<code>"Witaj".EndsWith("aj")</code> → true
<code>.Replace(a, b)</code>	Zamień tekst	<code>"Cześć".Replace("ść", "sc")</code> → "Czesc"
<code>.Substring(i, n)</code>	Pobierz podciąg	<code>"Witaj".Substring(2, 3)</code> → "taj"
<code>.Split(',')</code>	Podziel po znaku	<code>"a,b,c".Split(',')</code> → ["a","b","c"]
<code>.IndexOf(s)</code>	Pozycja podciągu (-1 = brak)	<code>"Witaj".IndexOf("ta")</code> → 2

Interpolacja stringów:

Zamiast konkatencji (`"Imię: " + imię + "!"`) używaj interpolacji:

```
string imie = "Anna";  
int wiek = 25;  
  
// Stary sposób (brzydki):  
string wiadomosc1 = "Imię: " + imie + ", wiek: " + wiek + " lat.";  
  
// Interpolacja (zalecana):  
string wiadomosc2 = $"Imię: {imie}, wiek: {wiek} lat.";  
  
// Formatowanie w interpolacji:  
double cena = 49.9;  
string cenaTxt = $"Cena: {cena:F2} zł";           // "Cena: 49.90 zł"  
double procent = 0.756;  
string procentTxt = $"Wynik: {procent:P0}";       // "Wynik: 76%"
```

Kompletny przykład — analiza tekstu:

MainWindow.xaml:

```
<Window x:Class="TekstDemo.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Analiza tekstu" Height="480" Width="500">  
    <StackPanel Margin="20">  
        <TextBlock Text="Wpisz tekst do analizy:" FontSize="13"/>  
        <TextBox Name="txtWejscie" Margin="0,5,0,10" AcceptsReturn="False"/>  
    </StackPanel>  
</Window>
```

```

        <WrapPanel>
            <Button Content="Analizuj" Click="btnAnalizuj_Click" Padding="10,5" Margin="0,0,8,0" />
            <Button Content="Wyczyść" Click="btnWyczysc_Click" Padding="10,5" />
        </WrapPanel>
        <TextBlock Text="Wyniki analizy:" FontSize="13" Margin="0,15,0,5" />
        <TextBlock Name="lblWyniki" TextWrapping="Wrap" FontFamily="Consolas"
            FontSize="11" LineHeight="18" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace TekstDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnAnalizuj_Click(object sender, RoutedEventArgs e)
        {
            string tekst = txtWejscie.Text;

            if (string.IsNullOrEmpty(tekst))
            {
                lblWyniki.Text = "Wpisz jakiś tekst!";
                return;
            }

            string wyniki = "";
            wyniki += $"Długość: {tekst.Length} znaków\n";
            wyniki += $"Wielkie litery: {tekst.ToUpper()}\n";
            wyniki += $"Małe litery: {tekst.ToLower()}\n";
            wyniki += $"Po Trim(): '{tekst.Trim()}'\n";
            wyniki += $"Zawiera 'a': {tekst.Contains("a")}\n";
            wyniki += $"Zaczyna się od 'W': {tekst.StartsWith("W")}\n";
            wyniki += $"Pozycja spacji: {tekst.IndexOf(' ')}\n";

            // Zamiana tekstu
            string zamieniony = tekst.Replace("a", "@");
            wyniki += $"Po Replace('a'→'@'): {zamieniony}\n";

            // Podciąg (jeśli wystarczająco długi)
            if (tekst.Length ≥ 5)
            {
                wyniki += $"Pierwsze 5 znaków: {tekst.Substring(0, 5)}\n";
            }

            lblWyniki.Text = wyniki;
        }

        private void btnWyczysc_Click(object sender, RoutedEventArgs e)
        {
            txtWejscie.Text = "";
            lblWyniki.Text = "";
        }
    }
}

```

```
}  
  
}
```

3.9. Praca ze znakami: char

`char` przechowuje jeden znak. Klasa `char` ma przydatne metody statyczne do sprawdzania właściwości znaków.

Metody statyczne char:

METODA	CO SPRAWDZA	PRZYKŁAD
<code>char.IsDigit(c)</code>	Czy to cyfra (0-9)?	<code>char.IsDigit('5')</code> → <code>true</code>
<code>char.IsLetter(c)</code>	Czy to litera?	<code>char.IsLetter('A')</code> → <code>true</code>
<code>char.IsUpper(c)</code>	Czy to duża litera?	<code>char.IsUpper('A')</code> → <code>true</code>
<code>char.IsLower(c)</code>	Czy to mała litera?	<code>char.IsLower('a')</code> → <code>true</code>
<code>char.IsWhiteSpace(c)</code>	Czy to spacja/tab?	<code>char.IsWhiteSpace(' ')</code> → <code>true</code>
<code>char.IsPunctuation(c)</code>	Czy to znak interpunkcji?	<code>char.IsPunctuation('.')</code> → <code>true</code>
<code>char.ToUpper(c)</code>	Zamień na wielką literę	<code>char.ToUpper('a')</code> → <code>'A'</code>
<code>char.ToLower(c)</code>	Zamień na małą literę	<code>char.ToLower('A')</code> → <code>'a'</code>

Dostęp do znaków w stringu:

```
string tekst = "Witaj";  
char pierwszyZnak = tekst[0]; // 'W'  
char ostatniZnak = tekst[tekst.Length - 1]; // 'j'
```

Kompletny przykład — analiza znaków w tekście:

MainWindow.xaml:

```
<Window x:Class="CharDemo.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Analiza znaków" Height="400" Width="450">  
    <StackPanel Margin="20">  
        <TextBlock Text="Wpisz tekst:" FontSize="13"/>  
        <TextBox Name="txtWejscie" Margin="0,5,0,10"/>  
        <Button Content="Analizuj znaki" Click="btnAnalizuj_Click"  
            Padding="10,5" HorizontalAlignment="Left"/>  
        <TextBlock Name="lblWyniki" Margin="0,12,0,0" TextWrapping="Wrap"  
            FontFamily="Consolas" FontSize="12"/>  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.Windows;  
  
namespace CharDemo  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
    }  
}
```

```

    }

    private void btnAnalizuj_Click(object sender, RoutedEventArgs e)
    {
        string tekst = txtWejscie.Text;

        if (string.IsNullOrEmpty(tekst))
        {
            lblWyniki.Text = "Podaj jakiś tekst!";
            return;
        }

        int liczbaLiter = 0;
        int liczbaCyfr = 0;
        int liczbaDuzych = 0;
        int liczbaMałych = 0;
        int liczbaSpacji = 0;
        int liczbaPozostale = 0;

        foreach (char c in tekst)
        {
            if (char.IsLetter(c))
            {
                liczbaLiter++;
                if (char.IsUpper(c)) liczbaDuzych++;
                else liczbaMałych++;
            }
            else if (char.IsDigit(c))
                liczbaCyfr++;
            else if (char.IsWhiteSpace(c))
                liczbaSpacji++;
            else
                liczbaPozostale++;
        }

        string wyniki = "";
        wyniki += $"Łącznie znaków: {tekst.Length}\n";
        wyniki += $"Liter: {liczbaLiter} (duże: {liczbaDuzych}, małe: {liczbaMałych})\n";
        wyniki += $"Cyfry: {liczbaCyfr}\n";
        wyniki += $"Spacje: {liczbaSpacji}\n";
        wyniki += $"Inne znaki: {liczbaPozostale}\n";

        if (tekst.Length > 0)
        {
            char pierwszy = tekst[0];
            wyniki += $"Pierwszy znak: '{pierwszy}' – ";
            wyniki += char.IsUpper(pierwszy) ? "DUŻA litera" : "nie duża litera";
        }

        lblWyniki.Text = wyniki;
    }
}

```

3.10. Klasa Random

Klasa `Random` pozwala generować liczby pseudolosowe. Bardzo przydatna do losowania pytań, elementów listy, kart itp.

Tworzenie obiektu Random:

```
// Utwórz raz – najlepiej jako pole klasy
private Random rng = new Random();
```

Metody klasy Random:

METODA	OPIS	ZAKRES
<code>rng.Next()</code>	Losowa nieujemna liczba całkowita	0 do <code>int.MaxValue</code>
<code>rng.Next(max)</code>	Losowa liczba od 0 do max-1	$0 \leq x < \text{max}$
<code>rng.Next(min, max)</code>	Losowa liczba od min do max-1	$\text{min} \leq x < \text{max}$
<code>rng.NextDouble()</code>	Losowa liczba od 0.0 do 1.0	$0.0 \leq x < 1.0$

Kompletny przykład — losowanie pytania z listy:

MainWindow.xaml:

```
<Window x:Class="LosowanieDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Losowe pytanie" Height="360" Width="480">
    <StackPanel Margin="20">
        <TextBlock Text="Generator pytań quizowych" FontSize="18"
                  FontWeight="Bold" HorizontalAlignment="Center" Margin="0,0,0,20"/>
        <Border BorderBrush="LightGray" BorderThickness="1" Padding="15"
                Background="AliceBlue" CornerRadius="5">
            <TextBlock Name="lblPytanie" TextWrapping="Wrap" FontSize="14"
                      Text="Naciśnij przycisk, aby wylosować pytanie."/>
        </Border>
        <Button Content="Losuj pytanie" Click="btnLosuj_Click"
                Padding="15,8" Margin="0,15,0,0" HorizontalAlignment="Center"
                Background="SteelBlue" Foreground="White" FontSize="13"/>
        <TextBlock Name="lblNumer" HorizontalAlignment="Center"
                Margin="0,10,0,0" Foreground="Gray" FontSize="11"/>

        <Separator Margin="0,15"/>
        <TextBlock Text="Losowa liczba (1-100):" FontSize="13"/>
        <WrapPanel Margin="0,5,0,0">
            <Button Content="Losuj liczbę" Click="btnLosujLiczbe_Click"
                    Padding="10,5" Margin="0,0,10,0"/>
            <TextBlock Name="lblLiczba" FontSize="16" VerticalAlignment="Center"/>
        </WrapPanel>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Collections.Generic;
using System.Windows;

namespace LosowanieDemo
{
    public partial class MainWindow : Window
    {
        // Random tworzymy raz jako pole klasy
        private Random rng = new Random();

        private List<string> pytania = new List<string>
        {
```

```

        "Co to jest WPF?",
        "Czym różni się string od int?",
        "Jak działa pętla foreach?",
        "Co to jest klasa w C#?",
        "Jak bezpiecznie parsować liczby z TextBox?",
        "Co to jest constructor?",
        "Czym jest List<T>?"
    };

    public MainWindow()
    {
        InitializeComponent();

    }

    private void btnLosuj_Click(object sender, RoutedEventArgs e)
    {
        // Losujemy indeks od 0 do liczba pytań - 1
        int indeks = rng.Next(0, pytania.Count);

        lblPytanie.Text = pytania[indeks];
        lblNumer.Text = $"Pytanie {indeks + 1} z {pytania.Count}";
    }

    private void btnLosujLiczbe_Click(object sender, RoutedEventArgs e)
    {
        // Losuj liczbę od 1 do 100 włącznie (Next(1, 101) = 1..100)
        int liczba = rng.Next(1, 101);
        lblLiczba.Text = liczba.ToString();
    }
}

```

Losowanie elementu z tablicy:

```

string[] kolory = { "Czerwony", "Zielony", "Niebieski", "Żółty", "Fioletowy" };
int i = rng.Next(0, kolory.Length);
string losowyKolor = kolory[i];
lblKolor.Text = $"Wylosowany kolor: {losowyKolor}";

```

Losowanie bez powtórzeń (tasowanie):

```

// Algorytm Fishera-Yatesa - tasuje listę
private void Potasuj<T>(List<T> lista)
{
    for (int i = lista.Count - 1; i > 0; i--)
    {
        int j = rng.Next(0, i + 1);
        T temp = lista[i];
        lista[i] = lista[j];
        lista[j] = temp;
    }
}

```


4. Klasy, obiekty i modele danych

4.1. Czym jest klasa

W C# klasa to **projekt** (blueprints) własnego, złożonego typu danych. Tak jak `int` przechowuje jedną liczbę, a `string` jeden tekst — Twoja własna klasa może przechowywać wiele powiązanych danych razem.

Przykład z życia:

Zamiast trzymać dane albumu w 3 osobnych zmiennych:

```
string tytułAlbumu = "Thriller";
string artystaAlbumu = "Michael Jackson";
int rokAlbumu = 1982;
```

Tworzysz klasę `Album`, która łączy te dane:

```
public class Album
{
    public string Tytuł { get; set; }
    public string Artysta { get; set; }
    public int RokWydania { get; set; }
}
```

I używasz jej tak:

```
Album album = new Album();
album.Tytuł = "Thriller";
album.Artysta = "Michael Jackson";
album.RokWydania = 1982;
```

Inne przykłady klas:

```
// Klasa Pracownik
public class Pracownik
{
    public string Imie { get; set; }
    public string Nazwisko { get; set; }
    public decimal Pensja { get; set; }
    public string Stanowisko { get; set; }
}

// Klasa Pytanie (np. do quizu)
public class Pytanie
{
    public string Treść { get; set; }
    public string[] Odpowiedzi { get; set; }
    public int PoprawnaOdpowiedz { get; set; } // indeks 0-3
}

// Klasa Produkt
public class Produkt
{
    public string Nazwa { get; set; }
    public decimal Cena { get; set; }
    public int StanMagazynowy { get; set; }
    public string Kategoria { get; set; }
}
```

4.2. Pola i właściwości

Właściwości (ang. properties) to dane przechowywane przez klasę. Deklarujesz je za pomocą słów kluczowych `get` i `set`.

Składnia właściwości:

```
public class Album
{
    // Auto-property – najprostszy sposób
    public string Tytuł { get; set; }

    // Właściwość tylko do odczytu
    public string Artysta { get; }

    // Właściwość z wartością domyślną (C# 6+)
    public int LiczbaPobran { get; set; } = 0;

    // Właściwość z logiką
    private decimal _cena;
    public decimal Cena
    {
        get { return _cena; }
        set
        {
            if (value < 0)
                throw new ArgumentException("Cena nie może być ujemna!");
            _cena = value;
        }
    }
}
```

Pełna klasa Album z właściwościami:

```
namespace MojaAplikacja.Models
{
    public class Album
    {
        public string Tytuł { get; set; } = "";
        public string Artysta { get; set; } = "";
        public int RokWydania { get; set; }
        public string Gatunek { get; set; } = "";
        public decimal Cena { get; set; }
        public int LiczbaPobran { get; set; } = 0;

        // Właściwość wyliczana (bez set) – wartość wylicza się z innych
        public int WiekAlbumu => DateTime.Now.Year - RokWydania;

        public override string ToString()
        {
            return $"{Tytuł} – {Artysta} ({RokWydania})";
        }
    }
}
```

`override string ToString()` — gdy WPF wyświetla obiekt w ListBox, wywołuje tę metodę, żeby wiedzieć jak go pokazać.

4.3. Konstruktor — tworzenie obiektu z danymi

Konstruktor to specjalna metoda wywoływana przy tworzeniu obiektu (`new Album()`). Możesz w nim od razu przekazać wszystkie dane.

Bez konstruktora (verbose):

```
Album a = new Album();
a.Tytul = "Thriller";
a.Artysta = "Michael Jackson";
a.RokWydania = 1982;
```

Z konstruktorem (zwięźle):

```
Album a = new Album("Thriller", "Michael Jackson", 1982);
```

Definiowanie konstruktora:

```
public class Album
{
    public string Tytul { get; set; }
    public string Artysta { get; set; }
    public int RokWydania { get; set; }
    public int LiczbaPobran { get; set; }

    // Konstruktor bezparametrowy
    public Album()
    {
        LiczbaPobran = 0;
    }

    // Konstruktor z parametrami
    public Album(string tytul, string artysta, int rokWydania)
    {
        Tytul = tytul;
        Artysta = artysta;
        RokWydania = rokWydania;
        LiczbaPobran = 0;
    }
}
```

Inicjator obiektu (alternatywa — często używana):

```
// Bez konstruktora z parametrami — wygodna inicjalizacja:
Album a = new Album
{
    Tytul = "Thriller",
    Artysta = "Michael Jackson",
    RokWydania = 1982
};
```

Kompletny przykład — klasa i okno WPF:

Models/Pracownik.cs:

```
namespace PracownikApp.Models
{
    public class Pracownik
    {
        public string Imie { get; set; }
        public string Nazwisko { get; set; }
        public decimal Pensja { get; set; }
        public string Stanowisko { get; set; }
    }
}
```

```

public Pracownik(string imie, string nazwisko, string stanowisko, decimal pensja)
{
    Imie = imie;
    Nazwisko = nazwisko;
    Stanowisko = stanowisko;
    Pensja = pensja;
}

public string PelneImie => $"{Imie} {Nazwisko}";

public override string ToString()
{
    return $"{PelneImie} ({Stanowisko})";
}
}
}

```

MainWindow.xaml:

```

<Window x:Class="PracownikApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Dane pracownika" Height="350" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Dane pracownika" FontSize="18" FontWeight="Bold"
                  HorizontalAlignment="Center" Margin="0,0,0,15"/>
        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="120"/>
                <ColumnDefinition Width="*"/>
            </Grid.ColumnDefinitions>
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
            </Grid.RowDefinitions>

            <TextBlock Grid.Row="0" Grid.Column="0" Text="Imię:" Margin="0,5"/>
            <TextBox Grid.Row="0" Grid.Column="1" Name="txtImie" Margin="0,5"/>

            <TextBlock Grid.Row="1" Grid.Column="0" Text="Nazwisko:" Margin="0,5"/>
            <TextBox Grid.Row="1" Grid.Column="1" Name="txtNazwisko" Margin="0,5"/>

            <TextBlock Grid.Row="2" Grid.Column="0" Text="Stanowisko:" Margin="0,5"/>
            <TextBox Grid.Row="2" Grid.Column="1" Name="txtStanowisko" Margin="0,5"/>

            <TextBlock Grid.Row="3" Grid.Column="0" Text="Pensja (zł):" Margin="0,5"/>
            <TextBox Grid.Row="3" Grid.Column="1" Name="txtPensja" Margin="0,5"/>
        </Grid>
        <Button Content="Utwórz pracownika" Click="btnUtwórz_Click"
                Padding="10,5" Margin="0,15,0,0" HorizontalAlignment="Center"/>
        <TextBlock Name="lblWynik" Margin="0,10,0,0" TextWrapping="Wrap" FontSize="13"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

```

```

using PracownikApp.Models;

namespace PracownikApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnUtwórz_Click(object sender, RoutedEventArgs e)
        {
            string imie = txtImie.Text.Trim();
            string nazwisko = txtNazwisko.Text.Trim();
            string stanowisko = txtStanowisko.Text.Trim();

            if (string.IsNullOrEmpty(imie) || string.IsNullOrEmpty(nazwisko))
            {
                MessageBox.Show("Podaj imię i nazwisko!", "Błąd");
                return;
            }

            if (!decimal.TryParse(txtPensja.Text, out decimal pensja))
            {
                MessageBox.Show("Podaj prawidłową pensję!", "Błąd");
                return;
            }

            // Tworzymy obiekt klasy Pracownik
            Pracownik p = new Pracownik(imie, nazwisko, stanowisko, pensja);

            // Używamy właściwości obiektu
            lblWynik.Text = $"Utworzono pracownika:\n" +
                $"Pełne imię: {p.PełneImię}\n" +
                $"Stanowisko: {p.Stanowisko}\n" +
                $"Pensja: {p.Pensja:C}\n" +
                $"ToString: {p}";
        }
    }
}

```

4.4. List

`List<T>` to lista obiektów tego samego typu. `T` (Type) zastępuje rzeczywistym typem, np. `List<Album>`, `List<Pracownik>`, `List<string>`.

Tworzenie listy:

```

List<Album> albumy = new List<Album>();

// Dodawanie elementów
albumy.Add(new Album("Thriller", "Michael Jackson", 1982));
albumy.Add(new Album("Back in Black", "AC/DC", 1980));
albumy.Add(new Album("The Dark Side of the Moon", "Pink Floyd", 1973));

```

Inicjalizacja z góry (inicjator kolekcji):

```

List<Album> albumy = new List<Album>
{
    new Album("Thriller", "Michael Jackson", 1982),

```

```
new Album("Back in Black", "AC/DC", 1980),  
new Album("The Dark Side of the Moon", "Pink Floyd", 1973)  
};
```

Kompletny przykład — lista albumów wyświetlana w ListBox:

MainWindow.xaml:

```
<Window x:Class="AlbumyApp.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Lista albumów" Height="500" Width="500">  
    <Grid Margin="15">  
        <Grid.RowDefinitions>  
            <RowDefinition Height="Auto" />  
            <RowDefinition Height="*" />  
            <RowDefinition Height="Auto" />  
        </Grid.RowDefinitions>  
  
        <TextBlock Grid.Row="0" Text="Lista albumów muzycznych"  
            FontSize="18" FontWeight="Bold" Margin="0,0,0,10" />  
  
        <ListBox Grid.Row="1" Name="listaAlbumow"  
            SelectionChanged="listaAlbumow_SelectionChanged"  
            Margin="0,0,0,10" />  
  
        <Border Grid.Row="2" BorderBrush="LightGray" BorderThickness="1"  
            Padding="10" Background="#F8F8F8">  
            <TextBlock Name="lblSzczegoly" TextWrapping="Wrap" FontSize="12" />  
        </Border>  
    </Grid>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.Collections.Generic;  
using System.Windows;  
using System.Windows.Controls;  
  
namespace AlbumyApp  
{  
    public class Album  
    {  
        public string Tytuł { get; set; }  
        public string Artysta { get; set; }  
        public int RokWydania { get; set; }  
        public int LiczbaPobran { get; set; } = 0;  
  
        public Album(string tytuł, string artysta, int rok)  
        {  
            Tytuł = tytuł;  
            Artysta = artysta;  
            RokWydania = rok;  
        }  
  
        public override string ToString() => $"{Tytuł} – {Artysta} ({RokWydania})";  
    }  
  
    public partial class MainWindow : Window
```

```

{
    private List<Album> albumy = new List<Album>
    {
        new Album("Thriller", "Michael Jackson", 1982),
        new Album("Back in Black", "AC/DC", 1980),
        new Album("The Dark Side of the Moon", "Pink Floyd", 1973),
        new Album("Led Zeppelin IV", "Led Zeppelin", 1971),
        new Album("Hotel California", "Eagles", 1977)
    };

    public MainWindow()
    {
        InitializeComponent();
        ZaładujListę();
    }

    private void ZaładujListę()
    {
        listaAlbumow.Items.Clear();
        foreach (Album a in albumy)
        {
            listaAlbumow.Items.Add(a); // ToString() wywoła się automatycznie
        }
    }

    private void listaAlbumow_SelectionChanged(object sender, SelectionChangedEventArgs e)
    {
        if (listaAlbumow.SelectedItem is Album wybrany)
        {
            lblSzczegoly.Text = $"Tytuł: {wybrany.Tytuł}\n" +
                                $"Artysta: {wybrany.Artysta}\n" +
                                $"Rok wydania: {wybrany.RokWydania}\n" +
                                $"Wiek albumu: {DateTime.Now.Year - wybrany.RokWydania} lat\n" +
                                $"Pobrano razy: {wybrany.LiczbaPobran}";
        }
    }
}

```

4.5. Aktualny obiekt — jeden z wielu rekordów na ekranie

Częsty wzorec: masz listę obiektów i jedno pole wyświetla **aktualny** rekord. Użytkownik może przechodzić między nimi przyciskami "Poprzedni" / "Następny".

Potrzebujesz zmiennej `aktualnyIndeks` śledzącej, który element jest aktualnie wyświetlany.

```

private List<Album> albumy = new List<Album>();
private int aktualnyIndeks = 0; // Pole klasy!

```

Metoda wyświetlająca aktualny rekord:

```

private void WyświetlAlbum()
{
    if (albumy.Count == 0) return;

    Album a = albumy[aktualnyIndeks];
    lblTytuł.Text = a.Tytuł;
    lblArtysta.Text = a.Artysta;
    lblRok.Text = a.RokWydania.ToString();
}

```

```
lblNumer.Text = $"{aktualnyIndeks + 1} / {albumy.Count}";  
}
```

4.6. Aktualizacja pojedynczych właściwości

Możesz zmieniać właściwości obiektu w dowolnym momencie. Na przykład zwiększać licznik pobrań:

Kompletny przykład — aplikacja przeglądania albumów z licznikiem pobrań:

MainWindow.xaml:

```
<Window x:Class="AlbumPrzegląd.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Przeglądarka albumów" Height="420" Width="480">  
    <StackPanel Margin="20">  
        <TextBlock Text="Przeglądarka albumów" FontSize="18" FontWeight="Bold"  
            HorizontalAlignment="Center" Margin="0,0,0,15"/>  
  
        <Border BorderBrush="#CCCCCC" BorderThickness="1" Padding="15"  
            Background="#F0F8FF" CornerRadius="6" Margin="0,0,0,10">  
            <StackPanel>  
                <TextBlock Name="lblTytuł" FontSize="16" FontWeight="Bold" TextWrapping="Wrap"/>  
                <TextBlock Name="lblArtysta" FontSize="13" Foreground="Gray" Margin="0,4,0,0"/>  
                <TextBlock Name="lblRok" FontSize="12" Foreground="DimGray" Margin="0,2,0,0"/>  
                <TextBlock Name="lblPobran" FontSize="12" Foreground="SteelBlue" Margin="0,6,0,0"/>  
            </StackPanel>  
        </Border>  
  
        <TextBlock Name="lblNumer" HorizontalAlignment="Center"  
            Foreground="Gray" FontSize="11" Margin="0,0,0,10"/>  
  
        <WrapPanel HorizontalAlignment="Center">  
            <Button Content="← Poprzedni" Click="btnPoprzedni_Click"  
                Padding="12,6" Margin="5"/>  
            <Button Content="↓ Pobierz" Click="btnPobierz_Click"  
                Padding="12,6" Margin="5" Background="SteelBlue" Foreground="White"/>  
            <Button Content="Następny →" Click="btnNastepny_Click"  
                Padding="12,6" Margin="5"/>  
        </WrapPanel>  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.Collections.Generic;  
using System.Windows;  
  
namespace AlbumPrzegląd  
{  
    public class Album  
    {  
        public string Tytuł { get; set; }  
        public string Artysta { get; set; }  
        public int RokWydania { get; set; }  
        public int LiczbaPobran { get; set; } = 0;  
  
        public Album(string tytuł, string artysta, int rok)  
        {  

```



```

        Tytuł = tytuł; Artysta = artysta; RokWydania = rok;
    }
}

public partial class MainWindow : Window
{
    private List<Album> albumy = new List<Album>
    {
        new Album("Thriller", "Michael Jackson", 1982),
        new Album("Back in Black", "AC/DC", 1980),
        new Album("The Dark Side of the Moon", "Pink Floyd", 1973),
        new Album("Led Zeppelin IV", "Led Zeppelin", 1971)
    };

    private int aktualnyIndeks = 0;

    public MainWindow()
    {
        InitializeComponent();
        WyszwietaAlbum();
    }

    private void WyszwietaAlbum()
    {
        if (albumy.Count == 0) return;

        Album a = albumy[aktualnyIndeks];
        lblTytuł.Text = a.Tytuł;
        lblArtysta.Text = a.Artysta;
        lblRok.Text = $"Rok: {a.RokWydania}";
        lblPobran.Text = $"Pobrano: {a.LiczbaPobran} razy";
        lblNumer.Text = $"Album {aktualnyIndeks + 1} z {albumy.Count}";
    }

    private void btnNastepny_Click(object sender, RoutedEventArgs e)
    {
        aktualnyIndeks++;
        if (aktualnyIndeks >= albumy.Count)
            aktualnyIndeks = 0; // Zawijanie - wróć do pierwszego
        WyszwietaAlbum();
    }

    private void btnPoprzedni_Click(object sender, RoutedEventArgs e)
    {
        aktualnyIndeks--;
        if (aktualnyIndeks < 0)
            aktualnyIndeks = albumy.Count - 1; // Zawijanie - wróć do ostatniego
        WyszwietaAlbum();
    }

    private void btnPobierz_Click(object sender, RoutedEventArgs e)
    {
        if (albumy.Count == 0) return;

        // Aktualizujemy właściwość aktualnego obiektu
        albumy[aktualnyIndeks].LiczbaPobran++;
        WyszwietaAlbum();
    }
}
}

```

Kluczowe elementy:

- `aktualnyIndeks` — pole klasy (nie lokalna zmienna!) przechowuje aktualną pozycję
- `albumy[aktualnyIndeks].LiczbaPobran++` — bezpośrednia modyfikacja właściwości obiektu na liście
- `WyswietlAlbum()` — jedna metoda, którą wywołujesz po każdej zmianie — nie duplikujesz kodu

5. Kolekcje i praca na danych

5.1. Tablice

Tablica (ang. array) to najprostszy sposób przechowywania wielu wartości tego samego typu. Jej rozmiar jest **ustalony przy tworzeniu** i nie można go zmienić.

Deklaracja i inicjalizacja tablic:

```
// Tablica stringów
string[] imiona = { "Anna", "Bartosz", "Celina", "Dawid" };

// Tablica liczb
int[] wyniki = { 85, 72, 91, 60, 77 };

// Tablica z góry określonego rozmiaru
double[] temperatury = new double[7]; // 7 elementów, początkowo 0.0

// Dostęp po indeksie (od 0!)
string pierwsze = imiona[0]; // "Anna"
string ostatnie = imiona[3]; // "Dawid"
int dlugosc = imiona.Length; // 4

// Modyfikacja elementu
wyniki[0] = 90;

// Pętla po tablicy
for (int i = 0; i < imiona.Length; i++)
{
    Console.WriteLine($"{i}: {imiona[i]}");
}

// Lub foreach
foreach (string imie in imiona)
{
    Console.WriteLine(imie);
}
```

Przykład tablicy w WPF — wyświetlanie elementów:

MainWindow.xaml:

```
<Window x:Class="TablicaDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Tablica w WPF" Height="350" Width="420">
    <StackPanel Margin="20">
        <TextBlock Text="Dni tygodnia (tablica):" FontSize="13" FontWeight="Bold"/>
        <ListBox Name="listaDni" Height="140" Margin="0,8,0,10"/>
        <WrapPanel>
            <Button Content="Załaduj tablicę" Click="btnZaładuj_Click"
                    Padding="10,5" Margin="0,0,8,0"/>
            <Button Content="Wyczyść" Click="btnWyczysc_Click" Padding="10,5"/>
        </WrapPanel>
        <TextBlock Name="lblInfo" Margin="0,10,0,0" Foreground="Gray"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace TablicaDemo
{
    public partial class MainWindow : Window
    {
        // Tablica jako pole klasy
        private string[] dniTygodnia = {
            "Poniedziałek", "Wtorek", "Środa",
            "Czwartek", "Piątek", "Sobota", "Niedziela"
        };

        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnZaladuj_Click(object sender, RoutedEventArgs e)
        {
            listaDni.Items.Clear();

            for (int i = 0; i < dniTygodnia.Length; i++)
            {
                listaDni.Items.Add($"{i + 1}. {dniTygodnia[i]}");
            }

            lblInfo.Text = $"Załadowano {dniTygodnia.Length} elementów.";
        }

        private void btnWyczysc_Click(object sender, RoutedEventArgs e)
        {
            listaDni.Items.Clear();
            lblInfo.Text = "Lista wyczyszczona.";
        }
    }
}
```

Ograniczenia tablic:

- Nie możesz dodawać nowych elementów po utworzeniu
- Nie możesz usuwać elementów
- Musisz znać rozmiar z góry

Dlatego w WPF częściej używasz `List<T>`.

5.2. List

`List<T>` to dynamiczna lista — możesz dodawać i usuwać elementy w dowolnym momencie.

Najważniejsze operacje na `List<T>`:

OPERACJA	METODA/WŁAŚCIWOŚĆ	OPIS
Dodaj na końcu	<code>lista.Add(element)</code>	Dodaje element na końcu
Wstaw w miejscu	<code>lista.Insert(i, element)</code>	Wstawia element na pozycji i
Usuń element	<code>lista.Remove(element)</code>	Usuwa pierwsze wystąpienie
Usuń po indeksie	<code>lista.RemoveAt(i)</code>	Usuwa element na pozycji i
Wyczyść	<code>lista.Clear()</code>	Usuwa wszystkie elementy
Liczba elementów	<code>lista.Count</code>	Aktualna liczba elementów
Odczyt po indeksie	<code>lista[i]</code>	Element na pozycji i
Czy zawiera	<code>lista.Contains(x)</code>	true/false
Znajdź indeks	<code>lista.IndexOf(x)</code>	-1 jeśli brak
Sortuj	<code>lista.Sort()</code>	Sortuje w miejscu

Kompletny przykład — menadżer zadań:

MainWindow.xaml:

```
<Window x:Class="ListaZadan.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Lista zadań" Height="480" Width="480">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <TextBlock Grid.Row="0" Text="Lista zadań" FontSize="20" FontWeight="Bold"
            HorizontalAlignment="Center" Margin="0,0,0,12" />

        <Grid Grid.Row="1" Margin="0,0,0,8">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>
            <TextBox Grid.Column="0" Name="txtNoweZadanie" Margin="0,0,8,0" />
            <Button Grid.Column="1" Content="Dodaj" Click="btnDodaj_Click"
                Padding="12,5" Background="Green" Foreground="White" />
        </Grid>

        <ListBox Grid.Row="2" Name="listaZadan" Margin="0,0,0,8" />

        <WrapPanel Grid.Row="3" HorizontalAlignment="Right">
            <Button Content="Usuń zaznaczone" Click="btnUsun_Click"
                Padding="10,5" Margin="0,0,8,0" Background="Red" Foreground="White" />
            <Button Content="Wyczyść wszystko" Click="btnWyczysc_Click"
                Padding="10,5" />
        </WrapPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Collections.Generic;
using System.Windows;

namespace ListaZadan
{
    public partial class MainWindow : Window
    {
        private List<string> zadania = new List<string>();

        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnDodaj_Click(object sender, RoutedEventArgs e)
        {
            string noweZadanie = txtNoweZadanie.Text.Trim();

            if (string.IsNullOrEmpty(noweZadanie))
            {
                MessageBox.Show("Wpisz treść zadania!", "Błąd");
                return;
            }

            // Dodajemy do listy
            zadania.Add(noweZadanie);
            OdswiezListBox();

            txtNoweZadanie.Text = "";
            txtNoweZadanie.Focus();
        }

        private void btnUsun_Click(object sender, RoutedEventArgs e)
        {
            if (listaZadan.SelectedIndex < 0)
            {
                MessageBox.Show("Zaznacz zadanie do usunięcia!", "Info");
                return;
            }

            int indeks = listaZadan.SelectedIndex;
            zadania.RemoveAt(indeks); // Usuwa z listy po indeksie
            OdswiezListBox();
        }

        private void btnWyczysc_Click(object sender, RoutedEventArgs e)
        {
            zadania.Clear(); // Czyści całą listę
            OdswiezListBox();
        }

        private void OdswiezListBox()
        {
            listaZadan.Items.Clear();
            foreach (string zadanie in zadania)
            {
                listaZadan.Items.Add(zadanie);
            }
        }
    }
}
```

```
}  
  
}
```

5.3. Indeks aktualnego elementu

Gdy przeglądasz listę rekordów (album, pytanie, pracownik) i pokazujesz jeden na raz, potrzebujesz zmiennej `aktualnyIndeks` jako pola klasy.

Dlaczego pole klasy, a nie lokalna zmienna?

Lokalna zmienna żyje tylko podczas jednego wywołania metody. Kiedy klikasz "Następny" po raz drugi, poprzednia lokalna zmienna już nie istnieje. Pole klasy żyje przez cały czas życia okna.

```
public partial class MainWindow : Window  
{  
    // POLE KLASY – żyje przez cały czas działania okna  
    private List<string> elementy = new List<string>();  
    private int aktualnyIndeks = 0;  
  
    // ...  
}
```

Kompletny przykład — przeglądarka cytatów:

MainWindow.xaml:

```
<Window x:Class="CytatyApp.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Cytaty" Height="380" Width="500">  
    <StackPanel Margin="25">  
        <TextBlock Text="Inspirujące cytaty" FontSize="20" FontWeight="Bold"  
            HorizontalAlignment="Center" Margin="0,0,0,20"/>  
  
        <Border BorderBrush="#DDDDDD" BorderThickness="1" Padding="20"  
            CornerRadius="8" Background="#FFF0F0" MinHeight="100">  
            <TextBlock Name="lblCytat" TextWrapping="Wrap" FontSize="14"  
                FontStyle="Italic" TextAlignment="Center"/>  
        </Border>  
  
        <TextBlock Name="lblAutor" HorizontalAlignment="Right"  
            Margin="0,8,0,0" FontWeight="Bold" FontSize="12"/>  
  
        <TextBlock Name="lblNumer" HorizontalAlignment="Center"  
            Margin="0,8,0,15" Foreground="Gray" FontSize="11"/>  
  
        <WrapPanel HorizontalAlignment="Center">  
            <Button Content="← Poprzedni" Click="btnPoprzedni_Click"  
                Padding="14,7" Margin="5"/>  
            <Button Content="Następny →" Click="btnNastepny_Click"  
                Padding="14,7" Margin="5"/>  
        </WrapPanel>  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.Collections.Generic;  
using System.Windows;
```

```

namespace CytatyApp
{
    public class Cytat
    {
        public string Tresc { get; set; }
        public string Autor { get; set; }

        public Cytat(string tresc, string autor)
        {
            Tresc = tresc;
            Autor = autor;
        }
    }

    public partial class MainWindow : Window
    {
        private List<Cytat> cytaty = new List<Cytat>
        {
            new Cytat("Sukces to suma małych wysiłków powtarzanych każdego dnia.", "Robert Collier"),
            new Cytat("Jedyną granicą naszych jutrzejszych osiągnięć są nasze dzisiejsze wątpliwości.", "Franklin D. Roosevelt"),
            new Cytat("Nie liczy się to, ile razy upadniesz, ale to, ile razy wstaniesz.", "Nelson Mandela"),
            new Cytat("Droga do sukcesu i droga do porażki są prawie identyczne.", "Colin R. Davis"),
            new Cytat("Wyobraźnia jest ważniejsza od wiedzy.", "Albert Einstein")
        };

        // POLE KLASY – śledzi aktualną pozycję
        private int aktualnyIndeks = 0;

        public MainWindow()
        {
            InitializeComponent();
            WyszwietlCytat();
        }

        private void WyszwietlCytat()
        {
            if (cytaty.Count == 0) return;

            Cytat c = cytaty[aktualnyIndeks];
            lblCytat.Text = $"{c.Tresc}\n";
            lblAutor.Text = $"– {c.Autor}";
            lblNumer.Text = $"{aktualnyIndeks + 1} z {cytaty.Count}";
        }

        private void btnNastepny_Click(object sender, RoutedEventArgs e)
        {
            aktualnyIndeks++;
            if (aktualnyIndeks ≥ cytaty.Count)
                aktualnyIndeks = 0;
            WyszwietlCytat();
        }

        private void btnPoprzedni_Click(object sender, RoutedEventArgs e)
        {
            aktualnyIndeks--;
            if (aktualnyIndeks < 0)
                aktualnyIndeks = cytaty.Count - 1;
            WyszwietlCytat();
        }
    }
}

```

5.4. Przechodzenie do następnego i poprzedniego elementu

Przejsięcie do następnego elementu to zwiększenie `aktualnyIndeks` o 1. Pamiętaj o zabezpieczeniu przed wyjściem poza zakres listy.

Wzorzec bez zawijania (zatrzymaj na końcu):

```
private void btnNastepny_Click(object sender, RoutedEventArgs e)
{
    if (aktualnyIndeks < lista.Count - 1)
    {
        aktualnyIndeks++;
        WyszwietlElement();
    }
    else
    {
        MessageBox.Show("To już ostatni element!");
    }
}
```

Wzorzec z zawijaniem (po ostatnim wróć do pierwszego):

```
private void btnNastepny_Click(object sender, RoutedEventArgs e)
{
    aktualnyIndeks++;
    if (aktualnyIndeks ≥ lista.Count)
        aktualnyIndeks = 0;
    WyszwietlElement();
}
```

Wzorzec z operatorem modulo (elegancki):

```
private void btnNastepny_Click(object sender, RoutedEventArgs e)
{
    aktualnyIndeks = (aktualnyIndeks + 1) % lista.Count;
    WyszwietlElement();
}
```

5.5. Przechodzenie do następnego i poprzedniego elementu

Analogicznie — zmniejszasz `aktualnyIndeks` o 1. Zabezpieczenie przed ujemną wartością.

Wzorzec bez zawijania:

```
private void btnPoprzedni_Click(object sender, RoutedEventArgs e)
{
    if (aktualnyIndeks > 0)
    {
        aktualnyIndeks--;
        WyszwietlElement();
    }
    else
    {
        MessageBox.Show("To już pierwszy element!");
    }
}
```

Wzorzec z zawijaniem:


```
private void btnPoprzedni_Click(object sender, RoutedEventArgs e)
{
    aktualnyIndeks--;
    if (aktualnyIndeks < 0)
        aktualnyIndeks = lista.Count - 1;
    WyszwietlElement();
}
```

5.6. Zawijanie indeksu na początku i końcu listy

Zawijanie indeksu to jeden z najczęstszych wzorców w aplikacjach przeglądających listy. Zamiast zatrzymywać się na końcu/początku, aplikacja "zawija" się i przechodzi na drugi koniec.

Kompletny przykład — quiz z zawijaniem:

MainWindow.xaml:

```
<Window x:Class="QuizApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Mini Quiz" Height="480" Width="520">
    <StackPanel Margin="20">
        <TextBlock Text="Mini Quiz" FontSize="22" FontWeight="Bold"
            HorizontalAlignment="Center" Margin="0,0,0,15"/>

        <TextBlock Name="lblNumerPytania" HorizontalAlignment="Center"
            Foreground="Gray" FontSize="11" Margin="0,0,0,8"/>

        <Border BorderBrush="SteelBlue" BorderThickness="2" Padding="15"
            CornerRadius="6" Background="#FFF5FF">
            <TextBlock Name="lblPytanie" TextWrapping="Wrap"
                FontSize="14" FontWeight="Bold"/>
        </Border>

        <RadioButton Name="rbA" Margin="0,12,0,4" GroupName="Odpowiedz"/>
        <RadioButton Name="rbB" Margin="0,4,0,4" GroupName="Odpowiedz"/>
        <RadioButton Name="rbC" Margin="0,4,0,4" GroupName="Odpowiedz"/>
        <RadioButton Name="rbD" Margin="0,4,0,12" GroupName="Odpowiedz"/>

        <WrapPanel HorizontalAlignment="Center">
            <Button Content="← Wróć" Click="btnPoprzedni_Click"
                Padding="12,6" Margin="5"/>
            <Button Content="Sprawdź ✓" Click="btnSprawdz_Click"
                Padding="12,6" Margin="5" Background="Green" Foreground="White"/>
            <Button Content="Dalej →" Click="btnNastepny_Click"
                Padding="12,6" Margin="5"/>
        </WrapPanel>

        <TextBlock Name="lblWynikOdp" HorizontalAlignment="Center"
            FontSize="14" FontWeight="Bold" Margin="0,10,0,0"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Collections.Generic;
using System.Windows;
using System.Windows.Controls;
```

```
namespace QuizApp
{
    public class Pytanie
    {
        public string Tresc { get; set; }
        public string[] Odpowiedzi { get; set; }
        public int PoprawnaOdpowiedz { get; set; }

        public Pytanie(string tresc, string[] odpowiedzi, int poprawna)
        {
            Tresc = tresc;
            Odpowiedzi = odpowiedzi;
            PoprawnaOdpowiedz = poprawna;
        }
    }

    public partial class MainWindow : Window
    {
        private List<Pytanie> pytania = new List<Pytanie>
        {
            new Pytanie(
                "Co to jest WPF?",
                new[] { "A) Web Page Framework", "B) Windows Presentation Foundation",
                    "C) Windows Program Files", "D) Web Presentation Format" },
                1 // B) jest poprawne (indeks 1)
            ),
            new Pytanie(
                "Jakiego słowa kluczowego używamy do tworzenia obiektu?",
                new[] { "A) create", "B) make", "C) new", "D) object" },
                2 // C) new
            ),
            new Pytanie(
                "Co przechowuje typ string?",
                new[] { "A) Liczby całkowite", "B) Liczby dziesiętne",
                    "C) Wartości logiczne", "D) Tekst" },
                3 // D) Tekst
            )
        };

        private int aktualnyIndeks = 0;
        private RadioButton[] radioButtons;

        public MainWindow()
        {
            InitializeComponent();
            radioButtons = new RadioButton[] { rbA, rbB, rbC, rbD };
            WyszwietlPytanie();
        }

        private void WyszwietlPytanie()
        {
            // Zawijanie indeksu
            if (aktualnyIndeks ≥ pytania.Count)
                aktualnyIndeks = 0;
            if (aktualnyIndeks < 0)
                aktualnyIndeks = pytania.Count - 1;

            Pytanie p = pytania[aktualnyIndeks];
            lblPytanie.Text = p.Tresc;
            lblNumerPytania.Text = $"Pytanie {aktualnyIndeks + 1} z {pytania.Count}";
            lblWynikOdp.Text = "";
        }
    }
}
```

```

        // Ustaw treść RadioButtonów
        for (int i = 0; i < radioButtons.Length; i++)
        {
            radioButtons[i].Content = p.Odpowiedzi[i];
            radioButtons[i].IsChecked = false;
        }
    }

    private void btnNastepny_Click(object sender, RoutedEventArgs e)
    {
        aktualnyIndeks = (aktualnyIndeks + 1) % pytania.Count; // Zawijanie modulo
        WyszwietlPytanie();
    }

    private void btnPoprzedni_Click(object sender, RoutedEventArgs e)
    {
        aktualnyIndeks--;
        if (aktualnyIndeks < 0)
            aktualnyIndeks = pytania.Count - 1; // Zawijanie przy cofaniu
        WyszwietlPytanie();
    }

    private void btnSprawdz_Click(object sender, RoutedEventArgs e)
    {
        // Znajdź zaznaczoną odpowiedź
        int zaznaczona = -1;
        for (int i = 0; i < radioButtons.Length; i++)
        {
            if (radioButtons[i].IsChecked == true)
            {
                zaznaczona = i;
                break;
            }
        }

        if (zaznaczona == -1)
        {
            lblWynik0dp.Text = "Zaznacz odpowiedź!";
            lblWynik0dp.Foreground = System.Windows.Media.Brushes.Orange;
            return;
        }

        Pytanie p = pytania[aktualnyIndeks];
        if (zaznaczona == p.PoprawnaOdpowiedz)
        {
            lblWynik0dp.Text = "✓ Poprawna odpowiedź!";
            lblWynik0dp.Foreground = System.Windows.Media.Brushes.Green;
        }
        else
        {
            lblWynik0dp.Text = $"× Błąd! Poprawna: {p.Odpowiedzi[p.PoprawnaOdpowiedz]}";
            lblWynik0dp.Foreground = System.Windows.Media.Brushes.Red;
        }
    }
}
}

```

5.7. List

`ObservableCollection<T>` to specjalna kolekcja z przestrzeni nazw `System.Collections.ObjectModel`. Jej główna zaleta: automatycznie powiadamia interfejs użytkownika o zmianach (dodaniu, usunięciu elementu).

Kiedy używać ObservableCollection vs List:

CECHA	LIST<T>	OBSERVABLECOLLECTION<T>
Ręczne odświeżanie UI	Wymagane	Automatyczne
Prostota użycia	Prosta	Podobna
Powiadamianie UI	Brak	Tak (INotifyCollectionChanged)
Dla prostych okien	✓ Wystarczy	Możesz użyć
Dla Data Binding	Nieoptymalne	✓ Zalecana

Przykład z List<T> — musisz ręcznie odświeżać ListBox:

```
private List<string> elementy = new List<string>();

private void DodajElement(string tekst)
{
    elementy.Add(tekst);
    // Ręczne odświeżenie ListBox:
    listaElementow.Items.Clear();
    foreach (var e in elementy)
        listaElementow.Items.Add(e);
}
```

Kompletny przykład z ObservableCollection — automatyczne odświeżanie:

MainWindow.xaml:

```
<Window x:Class="ObservableDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ObservableCollection demo" Height="420" Width="460">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <TextBlock Grid.Row="0" Text="ObservableCollection<T>"
            FontSize="16" FontWeight="Bold" Margin="0,0,0,10" />

        <Grid Grid.Row="1" Margin="0,0,0,8">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>
            <TextBox Grid.Column="0" Name="txtNazwa" Margin="0,0,0,0" />
            <Button Grid.Column="1" Content="Dodaj" Click="btnDodaj_Click"
                Padding="12,5" />
        </Grid>

        <!-- ItemsSource = binding do ObservableCollection -->
        <ListBox Grid.Row="2" Name="listaElementow" Margin="0,0,0,8" />

        <WrapPanel Grid.Row="3" HorizontalAlignment="Right">
            <TextBlock Name="lblCount" VerticalAlignment="Center"
```

```

        Margin="0,0,15,0" Foreground="Gray"/>
        <Button Content="Usuń zaznaczony" Click="btnUsun_Click"
            Padding="10,5" />
    </WrapPanel>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.ObjectModel;
using System.Windows;

namespace ObservableDemo
{
    public partial class MainWindow : Window
    {
        // ObservableCollection zamiast List
        private ObservableCollection<string> elementy = new ObservableCollection<string>();

        public MainWindow()
        {
            InitializeComponent();

            // Wiążemy ListBox z kolekcją – odtąd zmiany są automatyczne!
            listaElementow.ItemsSource = elementy;

            // Subskrybujemy zdarzenie zmiany kolekcji
            elementy.CollectionChanged += (s, e) =>
            {
                lblCount.Text = $"Elementów: {elementy.Count}";
            };

            // Dodaj przykładowe elementy
            elementy.Add("Element startowy 1");
            elementy.Add("Element startowy 2");
        }

        private void btnDodaj_Click(object sender, RoutedEventArgs e)
        {
            string nazwa = txtNazwa.Text.Trim();
            if (string.IsNullOrEmpty(nazwa)) return;

            // Dodajemy – ListBox odświeży się automatycznie!
            elementy.Add(nazwa);
            txtNazwa.Text = "";
        }

        private void btnUsun_Click(object sender, RoutedEventArgs e)
        {
            if (listaElementow.SelectedIndex >= 0)
            {
                // Usuwamy – ListBox odświeży się automatycznie!
                elementy.RemoveAt(listaElementow.SelectedIndex);
            }
        }
    }
}

```

Kluczowe różnice w kodzie:

- `listaElementow.ItemsSource = elementy;` — łączysz ListBox z kolekcją przez `ItemsSource`
- Nie wywołujesz `Items.Clear()` i `Items.Add()` — ListBox odświeża się sam
- `CollectionChanged` — zdarzenie wywoływane przy każdej zmianie kolekcji

Kiedy wybrać `ObservableCollection`:

- Gdy używasz Data Binding (wiązania danych)
- Gdy lista zmienia się często i nie chcesz ręcznie odświeżać UI
- Gdy uczysz się wzorca MVVM

Kiedy wystarczy `List<T>`:

- Gdy dane ładujesz raz (przy starcie) i tylko wyświetlasz
- Gdy nie używasz Data Binding
- Gdy ręcznie kontrolujesz odświeżanie ListBox

6. Podstawy XAML

XAML to język opisu interfejsu użytkownika w aplikacjach WPF. Zanim zaczniesz tworzyć okna z kontrolkami, musisz zrozumieć jak działa ten język i jak łączy się z kodem C#.

6.1. Czym jest XAML

XAML (ang. **eXtensible Application Markup Language**) to język oparty na XML, który służy do opisywania wyglądu i struktury interfejsu użytkownika. Każdy element XAML odpowiada obiektowi C#. Każdy atrybut XAML odpowiada właściwości tego obiektu.

Zamiast pisać w C#:

```
Button btn = new Button();
btn.Content = "Kliknij mnie";
btn.Width = 100;
```

Piszesz w XAML:

```
<Button Content="Kliknij mnie" Width="100"/>
```

Oba zapisy dają ten sam efekt — XAML jest po prostu czytelniejszy dla opisu interfejsu.

Dlaczego XAML?

- Wyraźnie oddziela wygląd od logiki
- Łatwiej zobaczyć strukturę okna
- Visual Studio wyświetla podgląd na żywo
- Można edytować wygląd bez dotykania kodu C#

6.2. Struktura pliku `MainWindow.xaml`

Każdy nowy projekt WPF tworzy plik `MainWindow.xaml`. Zobaczysz tam następującą strukturę:

MainWindow.xaml:

```
<Window x:Class="MojaAplikacja.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Moje pierwsze okno" Height="450" Width="800">
    <Grid>
        <!-- Tutaj umieszczasz kontrolki -->
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace MojaAplikacja
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Wyjaśnienie poszczególnych elementów:

- `x:Class="MojaAplikacja.MainWindow"` — łączy plik XAML z klasą C# o nazwie `MainWindow` w przestrzeni nazw `MojaAplikacja`
- `xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"` — główna przestrzeń nazw WPF; dzięki niej możesz używać `Button`, `Grid`, `TextBox` itd.
- `xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"` — przestrzeń nazw samego XAML; dzięki niej działa `x:Name`, `x:Class` itp.
- `Title="Moje pierwsze okno"` — tekst wyświetlany na pasku tytułowym okna
- `Height="450"` — wysokość okna w pikselach
- `Width="800"` — szerokość okna w pikselach
- `<Grid>` — jeden główny element zawierający całą zawartość okna (Window może mieć dokładnie jeden element podrzędny)

Element `<Window>` może mieć **dokładnie jeden** bezpośredni element podrzędny. Tym elementem jest zazwyczaj panel układu (Grid, StackPanel, DockPanel itp.), który z kolei może zawierać wiele kontrolek.

Kompletny przykład okna z kilkoma właściwościami:

MainWindow.xaml:

```

<Window x:Class="StrukturaPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Przykład struktury XAML"
        Height="350"
        Width="500"
        MinHeight="200"
        MinWidth="300"
        WindowStartupLocation="CenterScreen"
        ResizeMode="CanResize">
    <Grid>
        <TextBlock Text="Witaj w WPF!"
                    HorizontalAlignment="Center"
                    VerticalAlignment="Center"
                    FontSize="24"/>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace StrukturaPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()

```

```
{
    InitializeComponent();
}
}
```

Dodatkowe właściwości okna:

- `MinHeight`, `MinWidth` — minimalne wymiary okna
- `WindowStartupLocation="CenterScreen"` — okno pojawi się na środku ekranu
- `ResizeMode="CanResize"` — użytkownik może zmieniać rozmiar okna

6.3. Znaczniki i atrybuty

XAML używa składni XML. Każdy element ma tag otwierający i zamykający, albo jest samozamykający.

Tag otwierający i zamykający:

```
<Button>Kliknij mnie</Button>
```

Tag samozamykający (gdy brak zawartości):

```
<Button Content="Kliknij mnie"/>
```

Zagnieżdżanie — element wewnątrz elementu:

```
<StackPanel>
    <TextBlock Text="Pierwszy" />
    <TextBlock Text="Drugi" />
    <Button Content="Trzeci" />
</StackPanel>
```

Atrybuty to właściwości zapisywane wewnątrz tagu:

```
<Button Content="OK"
        Width="100"
        Height="30"
        Background="Blue"
        Foreground="White"/>
```

Pełny przykład pokazujący zagnieżdżanie i różne rodzaje tagów:

MainWindow.xaml:

```
<Window x:Class="ZagniezdzeniePrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zagnieżdżanie elementów" Height="300" Width="400">
    <StackPanel Margin="10">
        <!-- TextBlock z atrybutem Text -->
        <TextBlock Text="Nagłówek aplikacji" FontSize="18" FontWeight="Bold"/>

        <!-- Separator poziomy -->
        <Separator Margin="0,5"/>

        <!-- Button z zawartością jako tekst między tagami -->
        <Button Width="150" Height="35" Margin="0,5">
            Przycisk z tekstem
        </Button>
    </StackPanel>
</Window>
```



```

</Button>

<!-- Button z atrybutem Content -->
<Button Content="Przycisk z atrybutem"
        Width="150"
        Height="35"
        Margin="0,5" />

<!-- StackPanel zagnieżdżony wewnątrz StackPanel -->
<StackPanel Orientation="Horizontal" Margin="0,10">
    <TextBlock Text="Lewy " VerticalAlignment="Center" />
    <TextBlock Text="Środkowy " VerticalAlignment="Center" FontWeight="Bold" />
    <TextBlock Text="Prawy" VerticalAlignment="Center" />
</StackPanel>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace ZagniezdzeniePrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

6.4. x:Name i odwoływanie się do kontroltek w C#

Aby móc korzystać z kontrolki w kodzie C#, musisz nadać jej nazwę za pomocą atrybutu `x:Name`. Dzięki temu Visual Studio generuje pole w klasie, przez które możesz odwoływać się do tej kontrolki.

Składnia:

```
<TextBox x:Name="txtImie" Width="200" />
```

Teraz w C# możesz pisać:

```

string imie = txtImie.Text;
txtImie.Text = "Nowy tekst";
txtImie.IsEnabled = false;

```

Konwencja nazewnictwa — stosuj prefiksy wskazujące typ kontrolki:

KONTROLKA	PREFIKS	PRZYKŁAD
TextBox	txt	txtImie
TextBlock	tb lub lbl	tbWynik
Button	btn	btnZatwierdz
Label	lbl	lblOpis
ComboBox	cmb	cmbKategoria
ListBox	lst	lstProdukty
CheckBox	chk	chkZgoda
RadioButton	rdb	rdbMeski
PasswordBox	pwd	pwdHaslo

Kompletny przykład z wieloma nazwanymi kontrolkami:

MainWindow.xaml:

```
<Window x:Class="NazwyKontrollek.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Przykład x:Name" Height="300" Width="400">
    <StackPanel Margin="15">
        <TextBlock Text="Podaj swoje imię:" Margin="0,0,0,5"/>
        <TextBox x:Name="txtImie"
                 Width="200"
                 HorizontalAlignment="Left"
                 Margin="0,0,0,10"/>

        <Button x:Name="btnPowitaj"
                Content="Przywitaj mnie"
                Width="150"
                HorizontalAlignment="Left"
                Click="btnPowitaj_Click"
                Margin="0,0,0,10"/>

        <TextBlock x:Name="tbWynik"
                 FontSize="16"
                 Foreground="DarkBlue"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace NazwyKontrollek
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnPowitaj_Click(object sender, RoutedEventArgs e)
        {
            // Pobieramy tekst z TextBox przez jego nazwę
            string imie = txtImie.Text;
        }
    }
}
```

```

        if (string.IsNullOrEmpty(imie))
        {
            tbWynik.Text = "Wpisz swoje imię!";
            tbWynik.Foreground = System.Windows.Media.Brushes.Red;
        }
        else
        {
            tbWynik.Text = $"Witaj, {imie}!";
            tbWynik.Foreground = System.Windows.Media.Brushes.DarkBlue;
        }
    }
}
}
}

```

6.5. Najważniejsze właściwości kontroltek

Każda kontrolka w WPF ma zestaw właściwości, które decydują o jej wyglądzie i zachowaniu. Poniżej omówione są najważniejsze z nich.

TEXT I CONTENT

- **Text** — używane przez **TextBlock**, **TextBox**, **Label** (przez właściwość Content)
- **Content** — używane przez **Button**, **Label**, **CheckBox** — może to być tekst lub inna kontrolka

```

<TextBlock Text="To jest TextBlock"/>
<TextBox Text="Edytowalny tekst"/>
<Button Content="Kliknij"/>
<Label Content="Etykieta"/>

```

WIDTH I HEIGHT

Ustalają szerokość i wysokość kontrolki. Możesz użyć wartości **Auto** (domyślna — dopasowuje do zawartości) albo liczby w pikselach.

```

<Button Content="Małe" Width="80" Height="25"/>
<Button Content="Duże" Width="200" Height="50"/>
<Button Content="Auto szerokość" Height="30"/>

```

FONTSIZE, FOREGROUND, BACKGROUND, ISENALED, VISIBILITY

Kompletny przykład pokazujący wszystkie kluczowe właściwości:

MainWindow.xaml:

```

<Window x:Class="WlasciwosciKontrollek.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Właściwości kontrollek" Height="450" Width="500">
    <StackPanel Margin="15" >

        <!-- FontSize - rozmiar czcionki -->
        <TextBlock Text="Mały tekst (12)" FontSize="12" Margin="0,5"/>
        <TextBlock Text="Normalny tekst (16)" FontSize="16" Margin="0,5"/>
        <TextBlock Text="Duży tekst (24)" FontSize="24" Margin="0,5"/>

        <!-- Foreground - kolor tekstu -->
        <TextBlock Text="Czerwony tekst" Foreground="Red" FontSize="14" Margin="0,5"/>
        <TextBlock Text="Niebieski tekst" Foreground="Blue" FontSize="14" Margin="0,5"/>
        <TextBlock Text="Zielony tekst" Foreground="#228B22" FontSize="14" Margin="0,5"/>

        <!-- Background - kolor tła -->
        <TextBox x:Name="txtTlo"
                Text="Żółte tło"

```

```

        Background="Yellow"
        Width="200"
        HorizontalAlignment="Left"
        Margin="0,5" />

<Button x:Name="btnNiebieskie"
        Content="Niebieski przycisk"
        Background="SteelBlue"
        Foreground="White"
        Width="180"
        HorizontalAlignment="Left"
        Margin="0,5" />

<!-- IsEnabled - czy kontrolka jest aktywna -->
<Button x:Name="btnAktywny"
        Content="Aktywny przycisk"
        IsEnabled="True"
        Width="180"
        HorizontalAlignment="Left"
        Margin="0,5" />

<Button x:Name="btnNieaktywny"
        Content="Nieaktywny przycisk"
        IsEnabled="False"
        Width="180"
        HorizontalAlignment="Left"
        Margin="0,5" />

<!-- Visibility -->
<TextBlock x:Name="tbWidoczny"
        Text="Ten tekst jest widoczny (Visible)"
        Visibility="Visible"
        Foreground="Green"
        Margin="0,5" />

<TextBlock x:Name="tbUkryty"
        Text="Ten tekst jest ukryty (Hidden) - zajmuje miejsce"
        Visibility="Hidden"
        Foreground="Orange"
        Margin="0,5" />

<TextBlock x:Name="tbZwiniety"
        Text="Ten tekst jest zwinięty (Collapsed) - nie zajmuje miejsca"
        Visibility="Collapsed"
        Foreground="Purple"
        Margin="0,5" />

<Button Content="Przełącz widoczność"
        Width="180"
        HorizontalAlignment="Left"
        Margin="0,10"
        Click="btnPrzelacz_Click" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WlasciwosciKontroliek

```

```
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnPrzełącz_Click(object sender, RoutedEventArgs e)
        {
            // Przełączamy widoczność elementów
            if (tbUkryty.Visibility == Visibility.Hidden)
            {
                tbUkryty.Visibility = Visibility.Visible;
                tbZwiniety.Visibility = Visibility.Visible;
            }
            else
            {
                tbUkryty.Visibility = Visibility.Hidden;
                tbZwiniety.Visibility = Visibility.Collapsed;
            }
        }
    }
}
```

Zestawienie właściwości:

WŁAŚCIWOŚĆ	TYP	PRZYKŁADOWE WARTOŚCI
Text	string	"Hello", ""
Content	object	"OK", (kontrolka)
Width	double	100, 200, Double.NaN (Auto)
Height	double	30, 50, Double.NaN (Auto)
FontSize	double	12, 14, 16, 24
Foreground	Brush	Red, Blue, #FF0000
Background	Brush	White, Yellow, #FFFF00
IsEnabled	bool	True, False
Visibility	Visibility	Visible, Hidden, Collapsed

6.6. Komentarze w XAML

W XAML możesz definiować kolory na trzy sposoby.

SPOSÓB 1: NAZWY KOLORÓW

WPF obsługuje ponad 140 nazwanych kolorów:

```
<TextBlock Foreground="Red" />
<TextBlock Foreground="DarkBlue" />
<TextBlock Foreground="LimeGreen" />
<TextBlock Foreground="Orange" />
<TextBlock Foreground="Transparent" />
```

SPOSÓB 2: ZAPIS SZESNASTKOWY (HEX)

Format: #RRGGBB lub #AARRGGBB (AA = przezroczystość, 00=całkowicie przezroczysty, FF=nieprzezroczysty)

```
<TextBlock Foreground="#FF0000" />      ←!— Czerwony →
<TextBlock Foreground="#0000FF" />      ←!— Niebieski →
<TextBlock Foreground="#228B22" />      ←!— Ciemnozielony →
```

```
<TextBlock Foreground="#80FF0000"/>      ←!— Czerwony z 50% przezroczystością →
<Button Background="#2196F3"/>           ←!— Material Blue →
```

SPOSÓB 3: SOLIDCOLORBRUSH JAKO ELEMENT PODRZĘDNY

Gdy chcesz ustawić kolor bardziej szczegółowo, używasz składni elementu właściwości:

```
<Button Content="Kolorowy przycisk">
    <Button.Background>
        <SolidColorBrush Color="SteelBlue" Opacity="0.8"/>
    </Button.Background>
</Button>
```

Kompletny przykład z kolorami:

MainWindow.xaml:

```
<Window x:Class="KoloryPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kolory w XAML" Height="420" Width="450">
    <StackPanel Margin="15">
        <TextBlock Text="Kolory – różne sposoby zapisu"
                   FontSize="18" FontWeight="Bold" Margin="0,0,0,15"/>

        ←!— Nazwane kolory →
        <TextBlock Text="● Kolor nazwany: Red"
                   Foreground="Red" FontSize="14" Margin="0,3"/>
        <TextBlock Text="● Kolor nazwany: DarkBlue"
                   Foreground="DarkBlue" FontSize="14" Margin="0,3"/>
        <TextBlock Text="● Kolor nazwany: ForestGreen"
                   Foreground="ForestGreen" FontSize="14" Margin="0,3"/>
        <TextBlock Text="● Kolor nazwany: DarkOrange"
                   Foreground="DarkOrange" FontSize="14" Margin="0,3"/>

        ←!— Kolory hex →
        <TextBlock Text="● Hex #E53935 (ciemnoczerwony)"
                   Foreground="#E53935" FontSize="14" Margin="0,3"/>
        <TextBlock Text="● Hex #1565C0 (ciemnoniebieski)"
                   Foreground="#1565C0" FontSize="14" Margin="0,3"/>
        <TextBlock Text="● Hex #2E7D32 (ciemnozielony)"
                   Foreground="#2E7D32" FontSize="14" Margin="0,3"/>

        ←!— Przyciski z tłem →
        <Button Content="Tło: Tomato"
                Background="Tomato" Foreground="White"
                Width="200" Height="35"
                HorizontalAlignment="Left" Margin="0,5"/>

        <Button Content="Tło: #3F51B5"
                Background="#3F51B5" Foreground="White"
                Width="200" Height="35"
                HorizontalAlignment="Left" Margin="0,5"/>

        ←!— SolidColorBrush z przezroczystością →
        <Button Content="SolidColorBrush Opacity=0.6"
                Width="200" Height="35"
                HorizontalAlignment="Left" Margin="0,5">
            <Button.Background>
                <SolidColorBrush Color="DarkSlateBlue" Opacity="0.6"/>
            </Button.Background>
```

```
</Button>

<TextBox x:Name="txtKolorTla"
         Text="TextBox z żółtym tłem"
         Background="#FFF9C4"
         Foreground="#5D4037"
         Width="250"
         HorizontalAlignment="Left"
         Padding="5"
         Margin="0,5" />

</StackPanel>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace KoloryPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

6.7. Czcionki

WPF oferuje rozbudowane możliwości formatowania tekstu.

FONTSIZE — ROZMIAR CZCIONKI

Podawany w pikselach (domyślnie 12):

```
<TextBlock Text="Rozmiar 10" FontSize="10"/>
<TextBlock Text="Rozmiar 14" FontSize="14"/>
<TextBlock Text="Rozmiar 20" FontSize="20"/>
<TextBlock Text="Rozmiar 32" FontSize="32"/>
```

FONTWEIGHT — GRUBOŚĆ CZCIONKI

WARTOŚĆ	EFEKT
Thin	Bardzo cienka
Light	Lekka
Normal	Normalna (domyślna)
Medium	Średnia
SemiBold	Pół pogrubiona
Bold	Pogrubiona
Black	Bardzo gruba

```
<TextBlock Text="Normal" FontWeight="Normal"/>
<TextBlock Text="SemiBold" FontWeight="SemiBold"/>
```

```
<TextBlock Text="BoLd" FontWeight="BoLd" />
<TextBlock Text="Black" FontWeight="Black" />
```

FONTSTYLE — STYL CZCIONKI

```
<TextBlock Text="Normal" FontStyle="Normal" />
<TextBlock Text="Italic — kursywa" FontStyle="Italic" />
<TextBlock Text="Oblique" FontStyle="Oblique" />
```

FONTFAMILY — RODZINA CZCIONEK

```
<TextBlock Text="Arial" FontFamily="Arial" />
<TextBlock Text="Times New Roman" FontFamily="Times New Roman" />
<TextBlock Text="Courier New — mono" FontFamily="Courier New" />
<TextBlock Text="Segoe UI (domyślna WPF)" FontFamily="Segoe UI" />
```

Kompletny przykład z czcionkami:

MainWindow.xaml:

```
<Window x:Class="CzcionkiPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Czcionki w XAML" Height="500" Width="500">
    <StackPanel Margin="15">
        <TextBlock Text="Demonstracja czcionek" FontSize="20" FontWeight="BoLd"
            Margin="0,0,0,15" />

        <!-- FontSize -->
        <TextBlock Text="FontSize=10" FontSize="10" Margin="0,3" />
        <TextBlock Text="FontSize=14" FontSize="14" Margin="0,3" />
        <TextBlock Text="FontSize=18" FontSize="18" Margin="0,3" />
        <TextBlock Text="FontSize=24" FontSize="24" Margin="0,3" />

        <Separator Margin="0,8" />

        <!-- FontWeight -->
        <TextBlock Text="FontWeight=Light" FontWeight="Light" FontSize="16" Margin="0,3" />
        <TextBlock Text="FontWeight=Normal" FontWeight="Normal" FontSize="16" Margin="0,3" />
        <TextBlock Text="FontWeight=SemiBoLd" FontWeight="SemiBoLd" FontSize="16" Margin="0,3" />
        <TextBlock Text="FontWeight=BoLd" FontWeight="BoLd" FontSize="16" Margin="0,3" />
        <TextBlock Text="FontWeight=Black" FontWeight="Black" FontSize="16" Margin="0,3" />

        <Separator Margin="0,8" />

        <!-- FontStyle -->
        <TextBlock Text="FontStyle=Normal" FontStyle="Normal" FontSize="16" Margin="0,3" />
        <TextBlock Text="FontStyle=Italic" FontStyle="Italic" FontSize="16" Margin="0,3" />

        <Separator Margin="0,8" />

        <!-- Kombinacje -->
        <TextBlock Text="BoLd + Italic + DarkRed"
            FontWeight="BoLd" FontStyle="Italic"
            Foreground="DarkRed" FontSize="18" Margin="0,3" />

        <TextBlock Text="Courier New — kod"
            FontFamily="Courier New"
            FontSize="14" Margin="0,3" />
```



```
<TextBox x:Name="txtPrzyklad"
        Text="Edytowalny tekst"
        FontFamily="Segoe UI"
        FontSize="14"
        FontWeight="SemiBold"
        Margin="0,8"
        Padding="5"
        Width="300"
        HorizontalAlignment="Left"/>

</StackPanel>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace CzcionkiPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

6.8. Marginesy, padding i odstępy

Margin to zewnętrzny odstęp — odległość od krawędzi kontrolki do elementów sąsiadujących lub do krawędzi kontenera.

Padding to wewnętrzny odstęp — odległość od krawędzi kontrolki do jej zawartości (np. tekstu na przycisku).

SKŁADNIA MARGIN I PADDING

ZAPIS	ZNACZENIE
"10"	Wszystkie cztery strony = 10
"10,5"	Poziomo=10, Pionowo=5 (lewo/prawo=10, góra/dół=5)
"10,5,10,5"	Lewo=10, Góra=5, Prawo=10, Dół=5
"5,10,0,15"	Lewo=5, Góra=10, Prawo=0, Dół=15

```
←!— Margin taki sam ze wszystkich stron →
<Button Content="Margin 10" Margin="10"/>

←!— Margin poziomy 20, pionowy 5 →
<Button Content="Margin 20,5" Margin="20,5"/>

←!— Margin: lewo=5, góra=10, prawo=5, dół=20 →
<Button Content="Margin szczegółowy" Margin="5,10,5,20"/>

←!— Padding wewnętrzny →
<Button Content="Padding 15,8" Padding="15,8"/>
```

Kompletny przykład:

MainWindow.xaml:

```

<Window x:Class="MarginPaddingPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Margin i Padding" Height="500" Width="450">
    <StackPanel Background="LightGray">

        <!-- Bez marginesu - elementy stykają się ze sobą -->
        <Button Content="Bez marginesu" Background="White"/>
        <Button Content="Bez marginesu" Background="White"/>

        <!-- Margin z każdej strony po 10 -->
        <Button Content="Margin=&quot;10&quot;" Margin="10" Background="LightBlue"/>

        <!-- Margin poziomy 20, pionowy 5 -->
        <Button Content="Margin=&quot;20,5&quot;" Margin="20,5" Background="LightGreen"/>

        <!-- Margin szczegółowy -->
        <Button Content="Margin=&quot;5,15,50,15&quot;" Margin="5,15,50,15"
                Background="LightSalmon"/>

        <!-- Padding - zwiększa rozmiar przycisku przez wewnętrzny odstęp -->
        <Button Content="Padding=&quot;5,3&quot; (mały)"
                Padding="5,3" Background="Lavender" Margin="10,5"/>

        <Button Content="Padding=&quot;20,12&quot; (duży)"
                Padding="20,12" Background="Lavender" Margin="10,5"/>

        <!-- TextBox z paddingiem - tekst nie przylega do krawędzi -->
        <TextBox Text="TextBox bez Padding" Margin="10,5"/>
        <TextBox Text="TextBox z Padding=&quot;8,5&quot;"
                Padding="8,5" Margin="10,5"/>

        <!-- Label z paddingiem -->
        <Label Content="Label z Padding=&quot;15,8&quot;"
                Background="Wheat"
                Padding="15,8"
                Margin="10,5"/>

    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace MarginPaddingPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

6.9. Wyrównanie — HorizontalAlignment, VerticalAlignment

HorizontalAlignment kontroluje, jak element jest wyrównany poziomo wewnątrz swojego kontenera.

VerticalAlignment kontroluje wyrównanie pionowe.

HORIZONTALALIGNMENT

WARTOŚĆ	OPIS
Left	Wyrównaj do lewej
Center	Wycentrum poziomo
Right	Wyrównaj do prawej
Stretch	Rozciągnij na całą szerokość (domyślna)

VERTICALALIGNMENT

WARTOŚĆ	OPIS
Top	Wyrównaj do góry
Center	Wycentrum pionowo
Bottom	Wyrównaj do dołu
Stretch	Rozciągnij na całą wysokość (domyślna)

MainWindow.xaml:

```
<Window x:Class="WyrownaniePrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Wyrównanie kontroltek" Height="500" Width="500">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="200"/>
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <!-- Górny panel - wyrównanie poziome -->
        <StackPanel Grid.Row="0" Background="AliceBlue" Margin="5">
            <TextBlock Text="HorizontalAlignment:" FontWeight="Bold" Margin="5" />
            <Button Content="Left" HorizontalAlignment="Left"
                    Width="100" Margin="5,2" />
            <Button Content="Center" HorizontalAlignment="Center"
                    Width="100" Margin="5,2" />
            <Button Content="Right" HorizontalAlignment="Right"
                    Width="100" Margin="5,2" />
            <Button Content="Stretch (domyślne)" HorizontalAlignment="Stretch"
                    Margin="5,2" />
        </StackPanel>

        <!-- Dolny panel - wyrównanie pionowe -->
        <Grid Grid.Row="1" Background="LavenderBlush" Margin="5">
            <Button Content="Top" VerticalAlignment="Top"
                    HorizontalAlignment="Left" Width="100" Margin="10" />
            <Button Content="Center" VerticalAlignment="Center"
                    HorizontalAlignment="Center" Width="100" />
            <Button Content="Bottom" VerticalAlignment="Bottom"
                    HorizontalAlignment="Right" Width="100" Margin="10" />
        </Grid>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;
```

```

namespace WyrownaniePrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

6.10. Konwersja bool -> Visibility

Właściwość **Visibility** ma trzy wartości, które różnią się zachowaniem:

WARTOŚĆ	WIDOCZNOŚĆ	MIEJSCE W UKŁADZIE
Visible	Tak	Tak — zajmuje przestrzeń
Hidden	Nie	Tak — nadal zajmuje przestrzeń
Collapsed	Nie	Nie — nie zajmuje przestrzeń

Różnica między **Hidden** a **Collapsed** jest kluczowa przy projektowaniu układu. Użyj **Collapsed**, gdy chcesz by pozostałe elementy "weszły na jego miejsce".

MainWindow.xaml:

```

<Window x:Class="WidocznoscPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Visibility – demo" Height="400" Width="420">
    <StackPanel Margin="15">
        <TextBlock Text="Demonstracja Visibility"
            FontSize="18" FontWeight="Bold" Margin="0,0,0,15"/>

        <TextBlock Text="Element 1 (zawsze Visible)"
            Background="LightGreen" Padding="8" Margin="0,3"/>

        <TextBlock x:Name="tbHidden"
            Text="Element 2 – Hidden (zajmuje miejsce)"
            Background="LightCoral" Padding="8" Margin="0,3"
            Visibility="Visible"/>

        <TextBlock x:Name="tbCollapsed"
            Text="Element 3 – Collapsed (nie zajmuje miejsca)"
            Background="LightBlue" Padding="8" Margin="0,3"
            Visibility="Visible"/>

        <TextBlock Text="Element 4 (zawsze Visible)"
            Background="LightGreen" Padding="8" Margin="0,3"/>

        <StackPanel Orientation="Horizontal" Margin="0,15,0,0">
            <Button Content="Ukryj (Hidden)"
                Width="140" Margin="0,0,5,0"
                Click="btnHidden_Click"/>
            <Button Content="Schowaj (Collapsed)"
                Width="150"
                Click="btnCollapsed_Click"/>
        </StackPanel>

        <Button Content="Przywróć wszystko"

```

```

        Width="180" HorizontalAlignment="Left"
        Margin="0,5"
        Click="btnPrzywroc_Click"/>

        <TextBlock x:Name="tbStatus" Margin="0,10"
            FontStyle="Italic" Foreground="Gray"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WidocznoscPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void btnHidden_Click(object sender, RoutedEventArgs e)
            {
                tbHidden.Visibility = Visibility.Hidden;
                tbStatus.Text = "Element 2 ma Hidden – widzisz pustą przestrzeń po nim.";
            }

            private void btnCollapsed_Click(object sender, RoutedEventArgs e)
            {
                tbCollapsed.Visibility = Visibility.Collapsed;
                tbStatus.Text = "Element 3 ma Collapsed – Element 4 wsunął się wyżej.";
            }

            private void btnPrzywroc_Click(object sender, RoutedEventArgs e)
            {
                tbHidden.Visibility = Visibility.Visible;
                tbCollapsed.Visibility = Visibility.Visible;
                tbStatus.Text = "Wszystkie elementy przywrócone.";
            }
        }
    }
}

```

7. Layout i rozmieszczanie elementów

Layout (układ) to sposób, w jaki WPF rozmieszcza kontrolki w oknie. WPF oferuje kilka paneli układu, z których każdy ma inne zachowanie.

7.1. Czym jest layout

W tradycyjnym programowaniu (Windows Forms) każda kontrolka miała ustalone  i  — dokładną pozycję w pikselach. W WPF **nie robimy tego** dla zwykłych layoutów.

Dlaczego?

- Okno można powiększyć — przy stałych pozycjach kontrolki zostają w miejscu, a nie wypełniają okna
- Na różnych monitorach (DPI) stałe piksele dają różne wyniki
- Elementy nie dostosowują się do zawartości

Zamiast tego używamy paneli layoutu, które automatycznie obliczają pozycje.

Złe podejście (tylko Canvas):

```
<Canvas>
    <Button Canvas.Left="50" Canvas.Top="100" Content="Przycisk"/>
</Canvas>
```

Dobre podejście (Grid, StackPanel):

```
<StackPanel>
    <Button Content="Przycisk"/>
</StackPanel>
```

7.2. Grid — wiersze i kolumny

Grid to najczęściej używany panel w WPF. Działa jak tabela — definiujesz wiersze i kolumny, a potem umieszczasz elementy w wybranych komórkach.

DEFINIOWANIE WIERSZY I KOLUMN

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="50"/>      <!-- Stała wysokość 50px -->
        <RowDefinition Height="Auto"/>    <!-- Wysokość dopasowana do zawartości -->
        <RowDefinition Height="*/>        <!-- Reszta dostępnego miejsca -->
        <RowDefinition Height="2*"/>      <!-- Dwa razy więcej niż * -->
    </Grid.RowDefinitions>

    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="100"/>    <!-- Stała szerokość -->
        <ColumnDefinition Width="Auto"/>   <!-- Szerokość do zawartości -->
        <ColumnDefinition Width="*/>       <!-- Reszta -->
    </Grid.ColumnDefinitions>
</Grid>
```

Znaczenie wartości:

- **Liczba** (np. **100**) — stały rozmiar w pikselach
- **Auto** — rozmiar dopasowany do największej zawartości w danym wierszu/kolumnie
- ***** — zajmuje całą pozostałą przestrzeń
- **2***, **3*** — proporcjonalne: **2*** dostaje dwa razy więcej niż *****

UMIESZCZANIE ELEMENTÓW

Używasz **Grid.Row** i **Grid.Column** (numerowanie od zera):

```
<Button Grid.Row="0" Grid.Column="0" Content="0,0"/>
<Button Grid.Row="0" Grid.Column="1" Content="0,1"/>
<Button Grid.Row="1" Grid.Column="0" Content="1,0"/>
```

Kompletny przykład — Grid 2 kolumny, 3 wiersze:

MainWindow.xaml:

```
<Window x:Class="GridPodstawy.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Grid - podstawy" Height="350" Width="500">
    <Grid ShowGridLines="True">
        <Grid.RowDefinitions>
            <RowDefinition Height="60"/>      <!-- Wiersz 0 - stały: nagłówek -->
            <RowDefinition Height="Auto"/>    <!-- Wiersz 1 - auto: treść -->
            <RowDefinition Height="*/>        <!-- Wiersz 2 - reszta -->
        </Grid.RowDefinitions>
```

```

<Grid.ColumnDefinitions>
    <ColumnDefinition Width="150"/> <!-- Kolumna 0 - stała -->
    <ColumnDefinition Width="*/> <!-- Kolumna 1 - reszta -->
</Grid.ColumnDefinitions>

<!-- Wiersz 0 -->
<TextBlock Grid.Row="0" Grid.Column="0"
    Text="Nagłówek Lewy" FontWeight="Bold"
    HorizontalAlignment="Center" VerticalAlignment="Center"/>
<TextBlock Grid.Row="0" Grid.Column="1"
    Text="Nagłówek prawy" FontWeight="Bold"
    HorizontalAlignment="Center" VerticalAlignment="Center"/>

<!-- Wiersz 1 -->
<TextBlock Grid.Row="1" Grid.Column="0"
    Text="Wiersz Auto" Margin="10"/>
<TextBox Grid.Row="1" Grid.Column="1"
    Text="Pole tekstowe" Margin="10"/>

<!-- Wiersz 2 -->
<Button Grid.Row="2" Grid.Column="0"
    Content="Lewy" Margin="10"/>
<Button Grid.Row="2" Grid.Column="1"
    Content="Prawy" Margin="10"/>

</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace GridPodstawy
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Przykład z proporcjonalnym podziałem kolumn:

MainWindow.xaml:

```

<Window x:Class="GridProporcje.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Grid - proporcje" Height="200" Width="600">
    <Grid ShowGridLines="True">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*/> <!-- 1 część -->
            <ColumnDefinition Width="2*/> <!-- 2 części -->
            <ColumnDefinition Width="*/> <!-- 1 część -->
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>
    </Grid>

```

```

<Button Grid.Column="0" Content="1/4 szerokości"
        Margin="5" Background="LightBlue"/>

<Button Grid.Column="1" Content="2/4 szerokości (podwójna)"
        Margin="5" Background="LightGreen"/>

<Button Grid.Column="2" Content="1/4 szerokości"
        Margin="5" Background="LightBlue"/>

</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace GridProporcje
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

7.3. Formularze na Grid

Grid świetnie nadaje się do tworzenia formularzy. Typowy układ: etykieta w lewej kolumnie, pole wejściowe w prawej.

GRID.ROW, GRID.COLUMN, ROWSPAN, COLUMNSPAN

- `Grid.Row="n"` — umieszcza w wierszu n (od 0)
- `Grid.Column="n"` — umieszcza w kolumnie n (od 0)
- `Grid.RowSpan="n"` — element zajmuje n wierszy
- `Grid.ColumnSpan="n"` — element zajmuje n kolumn

Przykład formularza rejestracyjnego:

MainWindow.xaml:

```

<Window x:Class="FormularzGrid.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Formularz - Grid" Height="400" Width="500">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/> <!-- 0: Nagłówek -->
            <RowDefinition Height="10"/> <!-- 1: Odstęp -->
            <RowDefinition Height="Auto"/> <!-- 2: Imię -->
            <RowDefinition Height="Auto"/> <!-- 3: Nazwisko -->
            <RowDefinition Height="Auto"/> <!-- 4: Email -->
            <RowDefinition Height="Auto"/> <!-- 5: Hasło -->
            <RowDefinition Height="Auto"/> <!-- 6: Wiek -->
            <RowDefinition Height="15"/> <!-- 7: Odstęp -->
            <RowDefinition Height="Auto"/> <!-- 8: Przyciski -->
            <RowDefinition Height="Auto"/> <!-- 9: Wynik -->
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="120"/> <!-- Etykiety -->
            <ColumnDefinition Width="*"/> <!-- Pola -->
        </Grid.ColumnDefinitions>
    </Grid>

```



```
</Grid.ColumnDefinitions>

<!-- Nagłówek zajmuje obie kolumny -->
<TextBlock Grid.Row="0" Grid.Column="0" Grid.ColumnSpan="2"
            Text="Formularz rejestracji"
            FontSize="20" FontWeight="Bold"
            HorizontalAlignment="Center"/>

<!-- Imię -->
<Label Grid.Row="2" Grid.Column="0"
        Content="Imię:" VerticalAlignment="Center"/>
<TextBox x:Name="txtImie"
          Grid.Row="2" Grid.Column="1"
          Margin="0,3" Padding="5,3"/>

<!-- Nazwisko -->
<Label Grid.Row="3" Grid.Column="0"
        Content="Nazwisko:" VerticalAlignment="Center"/>
<TextBox x:Name="txtNazwisko"
          Grid.Row="3" Grid.Column="1"
          Margin="0,3" Padding="5,3"/>

<!-- Email -->
<Label Grid.Row="4" Grid.Column="0"
        Content="Email:" VerticalAlignment="Center"/>
<TextBox x:Name="txtEmail"
          Grid.Row="4" Grid.Column="1"
          Margin="0,3" Padding="5,3"/>

<!-- Hasło -->
<Label Grid.Row="5" Grid.Column="0"
        Content="Hasło:" VerticalAlignment="Center"/>
<PasswordBox x:Name="pwdHaslo"
              Grid.Row="5" Grid.Column="1"
              Margin="0,3" Padding="5,3"/>

<!-- Wiek -->
<Label Grid.Row="6" Grid.Column="0"
        Content="Wiek:" VerticalAlignment="Center"/>
<TextBox x:Name="txtWiek"
          Grid.Row="6" Grid.Column="1"
          Margin="0,3" Padding="5,3"/>

<!-- Przyciski - RowSpan = zajmuje obie kolumny -->
<StackPanel Grid.Row="8" Grid.Column="0" Grid.ColumnSpan="2"
            Orientation="Horizontal" HorizontalAlignment="Right">
    <Button x:Name="btnReset"
            Content="Wyczyść"
            Width="90" Height="30"
            Margin="0,0,10,0"
            Click="btnReset_Click"/>
    <Button x:Name="btnZatwierdz"
            Content="Zarejestruj"
            Width="100" Height="30"
            Background="#4CAF50" Foreground="White"
            Click="btnZatwierdz_Click"/>
</StackPanel>

<!-- Wynik -->
<TextBlock x:Name="tbWynik"
            Grid.Row="9" Grid.Column="0" Grid.ColumnSpan="2"
            TextWrapping="Wrap"
```

```

        Margin="0,10,0,0"
        FontSize="13"/>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace FormularzGrid
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnZatwierdz_Click(object sender, RoutedEventArgs e)
        {
            string imie = txtImie.Text.Trim();
            string nazwisko = txtNazwisko.Text.Trim();
            string email = txtEmail.Text.Trim();
            string haslo = pwdHaslo.Password;
            string wiek = txtWiek.Text.Trim();

            if (string.IsNullOrEmpty(imie) || string.IsNullOrEmpty(nazwisko) ||
                string.IsNullOrEmpty(email) || string.IsNullOrEmpty(haslo))
            {
                tbWynik.Text = "Wypełnij wszystkie wymagane pola!";
                tbWynik.Foreground = System.Windows.Media.Brushes.Red;
                return;
            }

            tbWynik.Text = $"Zarejestrowano: {imie} {nazwisko}, email: {email}";
            tbWynik.Foreground = System.Windows.Media.Brushes.Green;
        }

        private void btnReset_Click(object sender, RoutedEventArgs e)
        {
            txtImie.Text = "";
            txtNazwisko.Text = "";
            txtEmail.Text = "";
            pwdHaslo.Clear();
            txtWiek.Text = "";
            tbWynik.Text = "";
        }
    }
}

```

Przykład z Grid.RowSpan — duże pole tekstowe zajmujące kilka wierszy:

MainWindow.xaml:

```

<Window x:Class="GridRowSpan.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Grid.RowSpan i ColumnSpan" Height="350" Width="500">
    <Grid Margin="10" ShowGridLines="True">

```

```

<Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*/>
    <RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<Grid.ColumnDefinitions>
    <ColumnDefinition Width="110"/>
    <ColumnDefinition Width="*/>
    <ColumnDefinition Width="110"/>
</Grid.ColumnDefinitions>

<!-- Nagłówek - 3 kolumny -->
<TextBlock Grid.Row="0" Grid.ColumnSpan="3"
    Text="Dodaj wpis w dzienniku"
    FontSize="16" FontWeight="Bold"
    HorizontalAlignment="Center" Margin="5"/>

<!-- Tytuł -->
<Label Grid.Row="1" Grid.Column="0" Content="Tytuł:" VerticalAlignment="Center"/>
<TextBox Grid.Row="1" Grid.Column="1" Grid.ColumnSpan="2"
    x:Name="txtTytuł" Margin="5,3" Padding="4,3"/>

<!-- Kategoria -->
<Label Grid.Row="2" Grid.Column="0" Content="Kategoria:" VerticalAlignment="Center"/>
<TextBox Grid.Row="2" Grid.Column="1" Grid.ColumnSpan="2"
    x:Name="txtKategoria" Margin="5,3" Padding="4,3"/>

<!-- Treść - zajmuje 1 kolumnę po lewej (etykieta) + RowSpan dla pola -->
<Label Grid.Row="3" Grid.Column="0" Content="Treść:"
    VerticalAlignment="Top" Margin="0,5,0,0"/>
<TextBox Grid.Row="3" Grid.Column="1" Grid.ColumnSpan="2"
    x:Name="txtTresc"
    AcceptsReturn="True"
    TextWrapping="Wrap"
    VerticalScrollBarVisibility="Auto"
    Margin="5,3" Padding="4,3"/>

<!-- Przycisk - 3 kolumny -->
<Button Grid.Row="4" Grid.ColumnSpan="3"
    Content="Zapisz wpis"
    Height="32"
    HorizontalAlignment="Right"
    Width="120"
    Margin="5"
    Click="btnZapisz_Click"/>

</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace GridRowSpan
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {

```

```

        InitializeComponent();
    }

    private void btnZapisz_Click(object sender, RoutedEventArgs e)
    {
        if (string.IsNullOrEmpty(txtTytuł.Text))
        {
            MessageBox.Show("Podaj tytuł wpisu!", "Błąd",
                MessageBoxButton.OK, MessageBoxImage.Warning);

            return;
        }

        MessageBox.Show($"Wpis '{txtTytuł.Text}' został zapisany.",
            "Sukces", MessageBoxButton.OK, MessageBoxImage.Information);
    }
}

```

7.4. StackPanel

StackPanel układa elementy jeden za drugim — pionowo lub poziomo. To najprostrzy panel do szybkiego tworzenia listy elementów.

ORIENTATION

- **Vertical** (domyślna) — elementy jeden pod drugim
- **Horizontal** — elementy obok siebie

MainWindow.xaml:

```

<Window x:Class="StackPanelPrzykład.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="StackPanel – demo" Height="500" Width="450">
    <StackPanel Margin="10">

        <TextBlock Text="StackPanel pionowy (Vertical):"
            FontWeight="Bold" FontSize="14" Margin="0,0,0,5"/>

        <!-- StackPanel pionowy – domyślny -->
        <StackPanel Background="AliceBlue" Margin="0,0,0,15">
            <Button Content="Przycisk 1" Margin="5"/>
            <Button Content="Przycisk 2" Margin="5"/>
            <Button Content="Przycisk 3" Margin="5"/>
        </StackPanel>

        <TextBlock Text="StackPanel poziomy (Horizontal):"
            FontWeight="Bold" FontSize="14" Margin="0,0,0,5"/>

        <!-- StackPanel poziomy -->
        <StackPanel Orientation="Horizontal" Background="LightGoldenrodYellow"
            Margin="0,0,0,15">
            <Button Content="A" Width="60" Margin="5"/>
            <Button Content="B" Width="60" Margin="5"/>
            <Button Content="C" Width="60" Margin="5"/>
            <Button Content="D" Width="60" Margin="5"/>
        </StackPanel>

        <TextBlock Text="Mieszany układ – StackPanel w StackPanel:"
            FontWeight="Bold" FontSize="14" Margin="0,0,0,5"/>

        <!-- Zagnieżdżony StackPanel -->
        <StackPanel Background="Lavender">

```

```
<TextBlock Text="Wiersz 1" Margin="5"/>

<StackPanel Orientation="Horizontal">
    <TextBlock Text="Lewy:" VerticalAlignment="Center" Margin="5"/>
    <TextBox Width="150" Margin="5"/>
    <Button Content="OK" Width="50" Margin="5"/>
</StackPanel>

<StackPanel Orientation="Horizontal">
    <TextBlock Text="Prawy:" VerticalAlignment="Center" Margin="5"/>
    <TextBox Width="150" Margin="5"/>
    <Button Content="Anuluj" Width="70" Margin="5"/>
</StackPanel>

</StackPanel>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace StackPanelPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

7.5. DockPanel

DockPanel pozwala "dokować" elementy do krawędzi panelu. Ostatni element (bez atrybutu Dock) domyślnie wypełnia pozostałą przestrzeń.

DOCKPANEL.DOCK

WARTOŚĆ	OPIS
Top	Przypnij do górnej krawędzi
Bottom	Przypnij do dolnej krawędzi
Left	Przypnij do lewej krawędzi
Right	Przypnij do prawej krawędzi

MainWindow.xaml:

```
<Window x:Class="DockPanelPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="DockPanel - demo" Height="400" Width="550">
    <DockPanel>
        <!-- Pasek menu na górze -->
        <TextBlock DockPanel.Dock="Top"
            Text="Pasek menu (Top)"
            Background="#37474F"
            Foreground="White"
```

```

        Padding="10,8"
        FontSize="14"/>

<!-- Pasek statusu na dole -->
<TextBlock DockPanel.Dock="Bottom"
    Text="Pasek statusu (Bottom)"
    Background="#546E7A"
    Foreground="White"
    Padding="10,5"/>

<!-- Panel boczny po lewej -->
<StackPanel DockPanel.Dock="Left"
    Background="#ECEFF1"
    Width="130">
    <TextBlock Text="MENU" FontWeight="Bold"
        Padding="10,8" Background="#CFD8DC"/>
    <Button Content="Strona główna" Margin="5,3"
        HorizontalAlignment="Stretch"/>
    <Button Content="Ustawienia" Margin="5,3"
        HorizontalAlignment="Stretch"/>
    <Button Content="Pomoc" Margin="5,3"
        HorizontalAlignment="Stretch"/>
</StackPanel>

<!-- Panel boczny po prawej -->
<StackPanel DockPanel.Dock="Right"
    Background="#E8EAF6"
    Width="120">
    <TextBlock Text="INFO" FontWeight="Bold"
        Padding="10,8" Background="#C5CAE9"/>
    <TextBlock Text="Wersja: 1.0" Padding="10,5"/>
    <TextBlock Text="Autor: Jan" Padding="10,5"/>
</StackPanel>

<!-- Środek - wypełnia całą resztę -->
<Grid Background="White">
    <TextBlock Text="Główna treść aplikacji"
        HorizontalAlignment="Center"
        VerticalAlignment="Center"
        FontSize="18"
        Foreground="Gray"/>
</Grid>
</DockPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace DockPanelPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

7.6. WrapPanel

WrapPanel układa elementy poziomo (lub pionowo), a gdy nie ma już miejsca w wierszu — zawija je do kolejnej linii. Działa podobnie do tekstu w edytorze.

MainWindow.xaml:

```
<Window x:Class="WrapPanelPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="WrapPanel – demo" Height="300" Width="420">
    <StackPanel Margin="10">
        <TextBlock Text="WrapPanel – poziomy (domyślny):"
                   FontWeight="Bold" Margin="0,0,0,5"/>

        <WrapPanel Background="AliceBlue" Margin="0,0,0,15">
            <Button Content="Alfa" Width="80" Margin="4"/>
            <Button Content="Beta" Width="80" Margin="4"/>
            <Button Content="Gamma" Width="80" Margin="4"/>
            <Button Content="Delta" Width="80" Margin="4"/>
            <Button Content="Epsilon" Width="80" Margin="4"/>
            <Button Content="Zeta" Width="80" Margin="4"/>
            <Button Content="Eta" Width="80" Margin="4"/>
            <Button Content="Theta" Width="80" Margin="4"/>
        </WrapPanel>

        <TextBlock Text="WrapPanel – pionowy (Vertical):"
                   FontWeight="Bold" Margin="0,0,0,5"/>

        <WrapPanel Orientation="Vertical" Background="LightGoldenrodYellow"
                   Height="120">
            <Button Content="1" Width="70" Height="35" Margin="4"/>
            <Button Content="2" Width="70" Height="35" Margin="4"/>
            <Button Content="3" Width="70" Height="35" Margin="4"/>
            <Button Content="4" Width="70" Height="35" Margin="4"/>
            <Button Content="5" Width="70" Height="35" Margin="4"/>
            <Button Content="6" Width="70" Height="35" Margin="4"/>
        </WrapPanel>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace WrapPanelPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

7.7. Canvas

Canvas to jedyny panel w WPF, który używa stałych pozycji. Umieszczasz kontrolki podając dokładne współrzędne. Używaj go tylko do rysowania i animacji — nie do budowania formularzy.

CANVAS.LEFT, CANVAS.TOP, CANVAS.RIGHT, CANVAS.BOTTOM

- `Canvas.Left` — odległość od lewej krawędzi Canvas
- `Canvas.Top` — odległość od górnej krawędzi Canvas
- `Canvas.Right` — odległość od prawej krawędzi Canvas
- `Canvas.Bottom` — odległość od dolnej krawędzi Canvas

MainWindow.xaml:

```
<Window x:Class="CanvasPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Canvas — pozycjonowanie absolutne" Height="350" Width="500">
    <Canvas Background="LightGray">
        <!-- Tekst w lewym górnym rogu -->
        <TextBlock Canvas.Left="10" Canvas.Top="10"
                    Text="Lewy górny róg (10, 10)"
                    FontWeight="Bold"/>

        <!-- Przycisk na środku -->
        <Button Canvas.Left="180" Canvas.Top="140"
                Content="Środek okna"
                Width="120" Height="35"/>

        <!-- Etykieta w prawym górnym rogu -->
        <TextBlock Canvas.Right="10" Canvas.Top="10"
                    Text="Prawy górny róg"
                    HorizontalAlignment="Right"/>

        <!-- Prostokąt narysowany z Rectangle -->
        <Rectangle Canvas.Left="50" Canvas.Top="60"
                    Width="100" Height="60"
                    Fill="SteelBlue" Stroke="DarkBlue" StrokeThickness="2"/>

        <TextBlock Canvas.Left="60" Canvas.Top="82"
                    Text="Prostokąt" Foreground="White" FontWeight="Bold"/>

        <!-- Elipsa -->
        <Ellipse Canvas.Left="250" Canvas.Top="50"
                 Width="80" Height="80"
                 Fill="OrangeRed" Stroke="DarkRed" StrokeThickness="2"/>

        <!-- Przycisk w lewym dolnym rogu -->
        <Button Canvas.Left="10" Canvas.Bottom="10"
                Content="Lewy dół"
                Width="100" Height="30"/>
    </Canvas>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace CanvasPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```



```
}  
  
}
```

7.8. Marginesy, odstępy i wyrównanie — praktyczne wskazówki

Złota zasada: Stosuj spójne marginesy w całej aplikacji. Najczęściej używa się `Margin="5"` lub `Margin="10"`.

SYTUACJA	ZALECENIE
Odstęp między przyciskami	<code>Margin="0,0,5,0"</code> (prawy)
Wcięcie od krawędzi panelu	<code>Margin="10"</code> lub <code>Margin="15"</code>
Odstęp przed sekcją	<code>Margin="0,15,0,5"</code> (duży góra, mały dół)
Padding TextBox	<code>Padding="8,4"</code>
Padding Button	<code>Padding="12,6"</code>

MainWindow.xaml:

```
<Window x:Class="MarginPraktyka.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Marginesy – praktyczne przykłady" Height="400" Width="450">  
    <Grid Margin="15">  
        <Grid.RowDefinitions>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="*/>  
            <RowDefinition Height="Auto"/>  
        </Grid.RowDefinitions>  
        <Grid.ColumnDefinitions>  
            <ColumnDefinition Width="100"/>  
            <ColumnDefinition Width="*/>  
        </Grid.ColumnDefinitions>  
  
        <TextBlock Grid.Row="0" Grid.ColumnSpan="2"  
            Text="Formularz kontaktowy"  
            FontSize="18" FontWeight="Bold"  
            Margin="0,0,0,15"/>  
  
        <Label Grid.Row="1" Grid.Column="0" Content="Imię:"  
            VerticalAlignment="Center"/>  
        <TextBox Grid.Row="1" Grid.Column="1"  
            Padding="8,4" Margin="0,4"/>  
  
        <Label Grid.Row="2" Grid.Column="0" Content="Email:"  
            VerticalAlignment="Center"/>  
        <TextBox Grid.Row="2" Grid.Column="1"  
            Padding="8,4" Margin="0,4"/>  
  
        <Label Grid.Row="3" Grid.Column="0" Content="Temat:"  
            VerticalAlignment="Center"/>  
        <TextBox Grid.Row="3" Grid.Column="1"  
            Padding="8,4" Margin="0,4"/>  
  
        <Label Grid.Row="4" Grid.Column="0" Content="Wiadomość:"  
            VerticalAlignment="Top" Margin="0,8,0,0"/>  
        <TextBox Grid.Row="4" Grid.Column="1"  
            AcceptsReturn="True" TextWrapping="Wrap"
```

```

        VerticalScrollBarVisibility="Auto"
        Padding="8,4" Margin="0,4" />

<!-- Przyciski z marginesem między nimi -->
<StackPanel Grid.Row="5" Grid.ColumnSpan="2"
    Orientation="Horizontal"
    HorizontalAlignment="Right"
    Margin="0,15,0,0">
    <Button Content="Wyczyść"
        Width="90" Height="32"
        Padding="12,6"
        Margin="0,0,8,0" />
    <Button Content="Wyślij"
        Width="90" Height="32"
        Padding="12,6"
        Background="#1976D2"
        Foreground="White" />
</StackPanel>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace MarginPraktyka
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

7.9. Zagnieżdżanie paneli

Złożone okna aplikacji często wymagają zagnieżdżenia kilku paneli. Łączysz Grid, StackPanel, DockPanel i inne by osiągnąć żądany efekt.

Przykład — kompletne okno aplikacji z nagłówkiem, bocznym menu i obszarem treści:

MainWindow.xaml:

```

<Window x:Class="ZlozonyLayout.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Złożony układ — zagnieżdżone panele" Height="500" Width="700">
<!-- Zewnętrzny: DockPanel — ogólna struktura okna -->
<DockPanel>

    <!-- Nagłówek na górze -->
    <Border DockPanel.Dock="Top" Background="#1565C0" Padding="15,10">
        <StackPanel Orientation="Horizontal">
            <TextBlock Text="☰" FontSize="22" Foreground="White"
                VerticalAlignment="Center" Margin="0,0,10,0" />
            <StackPanel>
                <TextBlock Text="Moja Aplikacja WPF"
                    FontSize="18" FontWeight="Bold" Foreground="White" />
            </StackPanel>
        </StackPanel>
    </Border>
</DockPanel>

```

```

        <TextBlock Text="System zarządzania"
                FontSize="11" Foreground="#90CAF9" />
    </StackPanel>
</StackPanel>
</Border>

<!-- Pasek statusu na dole -->
<Border DockPanel.Dock="Bottom" Background="#E3F2FD" Padding="10,5">
    <TextBlock x:Name="tbStatus" Text="Gotowy"
            FontSize="12" Foreground="#546E7A" />
</Border>

<!-- Lewe menu -->
<Border DockPanel.Dock="Left" Background="#F5F5F5"
        BorderBrush="#E0E0E0" BorderThickness="0,0,1,0" Width="160">
    <StackPanel Margin="8">
        <TextBlock Text="NAWIGACJA" FontSize="11" FontWeight="Bold"
                Foreground="Gray" Margin="5,10,5,5" />
        <Button x:Name="btnStrGl" Content="🏠 Strona główna"
                HorizontalContentAlignment="Left"
                Padding="8,6" Margin="0,2"
                Click="btnNav_Click" />
        <Button x:Name="btnProdukty" Content="📦 Produkty"
                HorizontalContentAlignment="Left"
                Padding="8,6" Margin="0,2"
                Click="btnNav_Click" />
        <Button x:Name="btnKlienci" Content="👤 Klienci"
                HorizontalContentAlignment="Left"
                Padding="8,6" Margin="0,2"
                Click="btnNav_Click" />
        <Button x:Name="btnRaporty" Content="📊 Raporty"
                HorizontalContentAlignment="Left"
                Padding="8,6" Margin="0,2"
                Click="btnNav_Click" />

        <Separator Margin="5,10" />

        <TextBlock Text="OPCJE" FontSize="11" FontWeight="Bold"
                Foreground="Gray" Margin="5,0,5,5" />
        <Button Content="⚙️ Ustawienia"
                HorizontalContentAlignment="Left"
                Padding="8,6" Margin="0,2" />
    </StackPanel>
</Border>

<!-- Główna treść - Grid z zawartością -->
<Grid Background="White" Margin="20">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="*" />
    </Grid.RowDefinitions>

    <!-- Tytuł sekcji -->
    <TextBlock Grid.Row="0" x:Name="tbTytułSekcji"
            Text="Strona główna"
            FontSize="22" FontWeight="Light"
            Foreground="#333" Margin="0,0,0,15" />

    <!-- Pasek narzędzi -->
    <WrapPanel Grid.Row="1" Margin="0,0,0,15">
        <Button Content="Nowy" Background="#43A047" Foreground="White"

```

```

        Padding="12,6" Margin="0,0,5,0"/>

        <Button Content="Edytuj" Padding="12,6" Margin="0,0,5,0"/>
        <Button Content="Usuń" Background="#E53935" Foreground="White"
            Padding="12,6" Margin="0,0,5,0"/>

        <TextBox x:Name="txtSzukaj" Width="150" Padding="6,4"
            Margin="10,0,0,0"/>

        <Button Content="Szukaj" Padding="10,4"/>
    </WrapPanel>

    <!-- Obszar treści -->
    <Border Grid.Row="2" BorderBrush="#E0E0E0" BorderThickness="1"
        Background="#FAFAFA">
        <TextBlock x:Name="tbTresc"
            Text="Wybierz sekcję z menu po lewej stronie."
            HorizontalAlignment="Center"
            VerticalAlignment="Center"
            FontSize="16" Foreground="Gray"
            FontStyle="Italic"/>
    </Border>
</Grid>
</DockPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace ZlozonyLayout
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnNav_Click(object sender, RoutedEventArgs e)
        {
            System.Windows.Controls.Button btn =
                sender as System.Windows.Controls.Button;

            if (btn == null) return;

            // Wyciągamy tekst przycisku (bez emoji)
            string tekst = btn.Content?.ToString()?.Trim() ?? "";

            if (tekst.Contains("główna"))
            {
                tbTytułSekcji.Text = "Strona główna";
                tbTresc.Text = "Witaj w aplikacji! Wybierz moduł z menu.";
                tbStatus.Text = "Strona główna";
            }
            else if (tekst.Contains("Produkty"))
            {
                tbTytułSekcji.Text = "Produkty";
                tbTresc.Text = "Lista produktów będzie wyświetlona tutaj.";
                tbStatus.Text = "Moduł: Produkty";
            }
            else if (tekst.Contains("Klienci"))
            {

```

```
        tbTytułSekcji.Text = "Klienci";
        tbTresc.Text = "Lista klientów będzie wyświetlona tutaj.";
        tbStatus.Text = "Moduł: Klienci";
    }
    else if (tekst.Contains("Raporty"))
    {
        tbTytułSekcji.Text = "Raporty";
        tbTresc.Text = "Raporty i statystyki będą dostępne tutaj.";
        tbStatus.Text = "Moduł: Raporty";
    }
}
}
```

8. Kontrolki tekstowe

Kontrolki tekstowe służą do wyświetlania i wprowadzania tekstu. To fundamentalne elementy każdego interfejsu.

8.1. TextBlock

`TextBlock` to kontrolka do **wyświetlania** tekstu — nie można w nią klikać ani jej edytować. Używasz jej do etykiet, nagłówków, opisów, wyników.

KLUCZOWE WŁAŚCIWOŚCI TEXTBLOCK

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Text</code>	Wyświetlany tekst	<code>Text="Witaj"</code>
<code>FontSize</code>	Rozmiar czcionki	<code>FontSize="16"</code>
<code>FontWeight</code>	Grubość	<code>FontWeight="Bold"</code>
<code>Foreground</code>	Kolor tekstu	<code>Foreground="Red"</code>
<code>TextWrapping</code>	Zawijanie	<code>TextWrapping="Wrap"</code>
<code>TextAlignment</code>	Wyrównanie tekstu	<code>TextAlignment="Center"</code>
<code>TextDecorations</code>	Ozdoby	<code>TextDecorations="UnderLine"</code>

TEXTWRAPPING

Bez `TextWrapping` długi tekst jest obcięty. Z `TextWrapping="Wrap"` tekst przechodzi do nowej linii.

MainWindow.xaml:

```
<Window x:Class="TextBlockPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="TextBlock – właściwości" Height="520" Width="480">
    <StackPanel Margin="15">

        <!-- Prosty tekst -->
        <TextBlock Text="Prosty TextBlock" Margin="0,5"/>

        <!-- Rozmiary -->
        <TextBlock Text="FontSize=10" FontSize="10" Margin="0,3"/>
        <TextBlock Text="FontSize=16" FontSize="16" Margin="0,3"/>
        <TextBlock Text="FontSize=22 Bold" FontSize="22"
                    FontWeight="Bold" Margin="0,3"/>

        <!-- Kolory -->
        <TextBlock Text="Czerwony tekst" Foreground="Red" Margin="0,3"/>
        <TextBlock Text="Zielony tekst" Foreground="ForestGreen" Margin="0,3"/>
        <TextBlock Text="Tekst z tłem" Background="Yellow"
```

```

        Padding="4" Margin="0,3"/>

<!-- TextDecorations -->
<TextBlock Text="Tekst podkreślony"
           TextDecorations="Underline" Margin="0,3"/>
<TextBlock Text="Tekst przekreślony"
           TextDecorations="Strikethrough" Margin="0,3"/>

<!-- TextAlignment -->
<TextBlock Text="Wyśrodkowany" TextAlignment="Center"
           Background="LightGray" Margin="0,3"/>
<TextBlock Text="Do prawej" TextAlignment="Right"
           Background="LightGray" Margin="0,3"/>

<Separator Margin="0,8"/>

<!-- Tekst BEZ zawijania - zostanie obcięty -->
<TextBlock Text="Ten tekst jest bardzo długi i bez zawijania zostanie obcięty po prawej stronie okna"
           Background="LightCoral" Padding="4" Margin="0,3"/>

<!-- Tekst Z zawijaniem -->
<TextBlock Text="Ten tekst jest bardzo długi, ale dzięki TextWrapping=Wrap przechodzi do nowej linii i jest widoczny w całości"
           TextWrapping="Wrap"
           Background="LightGreen" Padding="4" Margin="0,3"/>

<!-- TextBlock wieloliniowy -->
<TextBlock Margin="0,8">
    <Run Text="Pierwsza linia tekstu" FontWeight="Bold"/>
    <LineBreak/>
    <Run Text="Druga linia z kursywą" FontStyle="Italic"/>
    <LineBreak/>
    <Run Text="Trzecia linia w kolorze" Foreground="DarkBlue"/>
</TextBlock>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace TextBlockPrzykład
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

8.2. Label

`Label` to kontrolka podobna do `TextBlock`, ale z kilkoma ważnymi różnicami:

- `Label` używa właściwości `Content` (nie `Text`)
- `Label` obsługuje właściwość `Target` — po naciśnięciu skrótu klawiaturowego (Alt+litera) fokus przechodzi na wskazaną kontrolkę
- `Label` ma domyślny padding i inaczej wygląda

TARGET I SKRÓTY KLAWIATUROWE

Znak  przed literą w Content tworzy skrót klawiaturowy (podkreśla tę literę):

```
<Label Content="_Imię:" Target="{Binding ElementName=txtImię}"/>
```

Po naciśnięciu Alt+I fokus przejdzie do TextBox o nazwie `txtImię`.

MainWindow.xaml:

```
<Window x:Class="LabelPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Label - demo" Height="350" Width="420">
    <StackPanel Margin="15">
        <TextBlock Text="Porównanie Label vs TextBlock:"
                   FontSize="16" FontWeight="Bold" Margin="0,0,0,15"/>

        <!-- Label - ma padding i ramkę z motywu -->
        <Label Content="To jest Label (ma domyślny styl motywu)"/>

        <!-- TextBlock - lżejszy, bez stylu -->
        <TextBlock Text="To jest TextBlock (lżejszy, bez stylu motywu)"
                   Margin="3,0"/>

        <Separator Margin="0,10"/>

        <TextBlock Text="Label z Target (skrót Alt+litera):"
                   FontWeight="Bold" Margin="0,0,0,8"/>

        <Grid>
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
            </Grid.RowDefinitions>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="110"/>
                <ColumnDefinition Width="*/>
            </Grid.ColumnDefinitions>

            <!-- _I = skrót Alt+I, przenosi fokus do txtImię -->
            <Label Grid.Row="0" Grid.Column="0"
                   Content="_Imię:"
                   Target="{Binding ElementName=txtImię}"
                   VerticalAlignment="Center"/>
            <TextBox x:Name="txtImię"
                    Grid.Row="0" Grid.Column="1"
                    Padding="5,3" Margin="0,3"/>

            <!-- _N = skrót Alt+N -->
            <Label Grid.Row="1" Grid.Column="0"
                   Content="_Nazwisko:"
                   Target="{Binding ElementName=txtNazwisko}"
                   VerticalAlignment="Center"/>
            <TextBox x:Name="txtNazwisko"
                    Grid.Row="1" Grid.Column="1"
                    Padding="5,3" Margin="0,3"/>

            <!-- _E = skrót Alt+E -->
            <Label Grid.Row="2" Grid.Column="0">
```

```

        Content="_Email:"
        Target="{Binding ElementName=txtEmail}"
        VerticalAlignment="Center"/>
<TextBox x:Name="txtEmail"
        Grid.Row="2" Grid.Column="1"
        Padding="5,3" Margin="0,3"/>

<!-- Label z niestandardowym wyglądem -->
<Label Grid.Row="3" Grid.Column="0"
        Content="Opis:"
        Foreground="DarkBlue" FontWeight="Bold"
        VerticalAlignment="Center"/>
<TextBox x:Name="txtOpis"
        Grid.Row="3" Grid.Column="1"
        Padding="5,3" Margin="0,3"/>
</Grid>

<TextBlock Text="Naciśnij Alt + podkreśloną literę, by przenieść fokus."
        FontStyle="Italic" Foreground="Gray"
        TextWrapping="Wrap" Margin="0,10,0,0"/>
</StackPanel>
</Window>
```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace LabelPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

8.3. Text

`TextBox` to edytowalne pole, w którym użytkownik wpisuje tekst. To jedna z najważniejszych kontroltek w formularzach.

KLUCZOWE WŁAŚCIWOŚCI TEXTBOX

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Text</code>	Zawartość pola	<code>Text="Witaj"</code>
<code>MaxLength</code>	Maksymalna liczba znaków	<code>MaxLength="50"</code>
<code>IsReadOnly</code>	Tylko do odczytu (nie można edytować)	<code>IsReadOnly="True"</code>
<code>Padding</code>	Wewnętrzny odstęp	<code>Padding="8,4"</code>
<code>TextAlignment</code>	Wyrównanie tekstu	<code>TextAlignment="Right"</code>

METODA FOCUS()

Metoda `Focus()` przenosi fokus klawiatury do `TextBox` — kursor pojawi się w polu.

```
txtImie.Focus(); // kursor wskakuje do txtImie
```


CZYSZCZENIE TEXTBOX

```
txtImie.Text = "";    // przypisanie pustego stringa
txtImie.Clear();      // metoda Clear() – działa tak samo
```

MainWindow.xaml:

```
<Window x:Class="TextBoxPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="TextBox – właściwości" Height="480" Width="480">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="140"/>
            <ColumnDefinition Width="*/>
        </Grid.ColumnDefinitions>

        <!-- Nagłówek -->
        <TextBlock Grid.Row="0" Grid.ColumnSpan="2"
            Text="Właściwości TextBox"
            FontSize="18" FontWeight="Bold" Margin="0,0,0,15"/>

        <!-- Normalny TextBox -->
        <Label Grid.Row="1" Grid.Column="0" Content="Normalny:"
            VerticalAlignment="Center"/>
        <TextBox x:Name="txtNormalny" Grid.Row="1" Grid.Column="1"
            Text="Edytowalny tekst"
            Padding="8,4" Margin="0,4"/>

        <!-- MaxLength -->
        <Label Grid.Row="2" Grid.Column="0" Content="MaxLength=10:"
            VerticalAlignment="Center"/>
        <TextBox x:Name="txtMaxLength" Grid.Row="2" Grid.Column="1"
            MaxLength="10"
            Padding="8,4" Margin="0,4"/>

        <!-- IsReadOnly -->
        <Label Grid.Row="3" Grid.Column="0" Content="IsReadOnly:"
            VerticalAlignment="Center"/>
        <TextBox x:Name="txtReadOnly" Grid.Row="3" Grid.Column="1"
            Text="Nie można edytować"
            IsReadOnly="True"
            Background="#F5F5F5"
            Padding="8,4" Margin="0,4"/>

        <!-- TextAlignment Right -->
        <Label Grid.Row="4" Grid.Column="0" Content="Liczba (prawy):"
            VerticalAlignment="Center"/>
        <TextBox x:Name="txtLiczba" Grid.Row="4" Grid.Column="1"
            Text="0"
```

```

        TextAlignment="Right"
        MaxLength="10"
        Padding="8,4" Margin="0,4"/>

<!-- TextBox z tłem i obramowaniem (styl) -->
<Label Grid.Row="5" Grid.Column="0" Content="Z kolorem tła:"
        VerticalAlignment="Center"/>
<TextBox x:Name="txtKolor" Grid.Row="5" Grid.Column="1"
        Background="#E3F2FD"
        Foreground="#0D47A1"
        Padding="8,4" Margin="0,4"/>

<!-- Przyciski akcji -->
<StackPanel Grid.Row="6" Grid.ColumnSpan="2"
        Orientation="Horizontal" Margin="0,10,0,5">
    <Button Content="Focus na Imię"
        Width="120" Height="30"
        Margin="0,0,8,0"
        Click="btnFocus_Click"/>
    <Button Content="Wyczyść wszystko"
        Width="130" Height="30"
        Margin="0,0,8,0"
        Click="btnWyczysc_Click"/>
    <Button Content="Pokaż dane"
        Width="100" Height="30"
        Click="btnPokaz_Click"/>
</StackPanel>

<!-- Wynik -->
<Border Grid.Row="7" Grid.ColumnSpan="2"
        Background="#F5F5F5" BorderBrush="#E0E0E0"
        BorderThickness="1" Padding="10" Margin="0,5">
    <TextBlock x:Name="tbWynik"
        TextWrapping="Wrap"
        FontFamily="Courier New"/>
</Border>

<TextBlock Grid.Row="8" Grid.ColumnSpan="2"
        Text="*MaxLength ogranicza wpisywanie – spróbuj wpisać więcej niż 10 znaków."
        FontStyle="Italic" Foreground="Gray"
        TextWrapping="Wrap" FontSize="11" Margin="0,5"/>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace TextBoxPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            // Po starcie aplikacji fokus trafia do pierwszego pola
            txtNormalny.Focus();
        }

        private void btnFocus_Click(object sender, RoutedEventArgs e)

```

```

    {
        // Przenosimy fokus do normalnego TextBox
        txtNormalny.Focus();
        // Zaznaczamy cały tekst w polu
        txtNormalny.SelectAll();
    }

    private void btnWyczyszc_Click(object sender, RoutedEventArgs e)
    {
        // Czyszczenie przez Clear()
        txtNormalny.Clear();
        txtMaxLength.Clear();
        txtLiczba.Text = "0";
        txtKolor.Clear();
        tbWynik.Text = "";

        // Po wyczyszczeniu – fokus na pierwsze pole
        txtNormalny.Focus();
    }

    private void btnPokaz_Click(object sender, RoutedEventArgs e)
    {
        // Odczytujemy wartości z pól
        string normalny = txtNormalny.Text;
        string max = txtMaxLength.Text;
        string liczba = txtLiczba.Text;
        string kolor = txtKolor.Text;

        tbWynik.Text =
            $"Normalny: \"{normalny}\"\\n" +
            $"MaxLength: \"{max}\" ({max.Length}/10 znaków)\\n" +
            $"Liczba: \"{liczba}\"\\n" +
            $"Kolor: \"{kolor}\"";
    }
}
}

```

8.4. Wielowierszowy TextBox

Domyślnie TextBox przyjmuje tylko jedną linię. Żeby umożliwić wpisywanie wielu linii, używasz:

- `AcceptsReturn="True"` — Enter tworzy nową linię
- `TextWrapping="Wrap"` — długie linie zawijają się
- `VerticalScrollBarVisibility="Auto"` — pasek przewijania pionowego
- `HorizontalScrollBarVisibility` — pasek poziomy (przy Wrap zbędny)

MainWindow.xaml:

```

<Window x:Class="TextBoxWieloliniowyPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="TextBox wielowierszowy" Height="480" Width="520">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="180"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
    </Grid>

```

```
</Grid.RowDefinitions>

<TextBlock Grid.Row="0"
            Text="Edytor notatek"
            FontSize="20" FontWeight="Bold"
            Margin="0,0,0,10"/>

<!-- Tytuł notatki - jednoliniowy -->
<TextBox x:Name="txtTytuł" Grid.Row="1"
         Padding="8,5"
         Margin="0,0,0,8"
         FontSize="14"
         FontWeight="SemiBold"/>

<!-- Treść notatki - wieloliniowy -->
<TextBox x:Name="txtTresc" Grid.Row="2"
         AcceptsReturn="True"
         TextWrapping="Wrap"
         VerticalScrollBarVisibility="Auto"
         HorizontalScrollBarVisibility="Disabled"
         Padding="8,5"
         FontSize="13"
         VerticalContentAlignment="Top"/>

<!-- Pasek informacyjny -->
<TextBlock x:Name="tbInfo" Grid.Row="3"
            FontSize="11" Foreground="Gray"
            Margin="0,3,0,8"
            HorizontalAlignment="Right"/>

<!-- Przyciski -->
<StackPanel Grid.Row="4" Orientation="Horizontal"
            HorizontalAlignment="Right" Margin="0,0,0,10">

    <Button Content="Zlicz znaki"
            Width="100" Height="30"
            Margin="0,0,8,0"
            Click="btnZlicz_Click"/>

    <Button Content="Wyczyść"
            Width="80" Height="30"
            Margin="0,0,8,0"
            Click="btnWyczysc_Click"/>

    <Button Content="Zapisz"
            Width="80" Height="30"
            Background="#388E3C" Foreground="White"
            Click="btnZapisz_Click"/>

</StackPanel>

<!-- Podgląd wynikowy -->
<Border Grid.Row="5"
        Background="#FAFAFA" BorderBrush="#808080"
        BorderThickness="1" Padding="10">

    <TextBlock x:Name="tbWynik"
            TextWrapping="Wrap"
            FontSize="12"/>

</Border>

<TextBlock Grid.Row="6"
            Text="Wskazówka: Naciśnij Enter by przejść do nowej linii."
            FontStyle="Italic" Foreground="Gray"
            FontSize="11" Margin="0,5"/>
```

```
</Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace TextBoxWieloliniowyPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            txtTytuł.Text = "Moja pierwsza notatka";
            txtTytuł.Focus();

            // Aktualizujemy licznik po każdej zmianie tekstu
            txtTresc.TextChanged += TxtTresc_TextChanged;
        }

        private void TxtTresc_TextChanged(object sender,
            System.Windows.Controls.TextChangedEventArgs e)
        {
            int znaki = txtTresc.Text.Length;
            int linie = txtTresc.LineCount;
            tbInfo.Text = $"Znaków: {znaki} | Linii: {linie}";
        }

        private void btnZlicz_Click(object sender, RoutedEventArgs e)
        {
            string tresc = txtTresc.Text;
            int znaki = tresc.Length;
            int linie = tresc.Split('\n').Length;
            int slowa = string.IsNullOrEmpty(tresc) ? 0 :
                tresc.Split(new char[] { ' ', '\n', '\r' },
                    StringSplitOptions.RemoveEmptyEntries).Length;

            tbWynik.Text = $"Tytuł: {txtTytuł.Text}\n" +
                $"Liczba znaków: {znaki}\n" +
                $"Liczba słów: {slowa}\n" +
                $"Liczba linii: {linie}";
        }

        private void btnWyczyszc_Click(object sender, RoutedEventArgs e)
        {
            txtTytuł.Clear();
            txtTresc.Clear();
            tbWynik.Text = "";
            tbInfo.Text = "";
            txtTytuł.Focus();
        }

        private void btnZapisz_Click(object sender, RoutedEventArgs e)
        {
            if (string.IsNullOrEmpty(txtTytuł.Text))
            {
                MessageBox.Show("Wpisz tytuł notatki!", "Brak tytułu",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
                txtTytuł.Focus();
            }
        }
    }
}
```

```
        return;
    }

    tbWynik.Text = $"✓ Notatka '{txtTytul.Text}' zapisana.\n" +
        $"Zawiera {txtTresc.Text.Length} znaków.";
}
}
}
```

8.5. PasswordBox

PasswordBox to pole tekstowe, w którym wpisywany tekst jest maskowany. Zamiast liter wyświetla się ciąg znaków (domyślnie ).

KLUCZOWE WŁAŚCIWOŚCI PASSWORDBOX

WŁAŚCIWOŚĆ	OPIS
Password	Pobiera wpisane hasło jako string
PasswordChar	Znak maskujący (domyślnie )
MaxLength	Maksymalna liczba znaków

Ważna różnica: **PasswordBox** nie ma właściwości **Text**. Używasz **Password**.

```
string haslo = pwdHaslo.Password;    // odczyt
pwdHaslo.Password = "";              // czyszczenie
pwdHaslo.Clear();                    // też czyści
```

MainWindow.xaml:

```
<Window x:Class="PasswordBoxPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="PasswordBox - demo" Height="400" Width="450">
    <Grid Margin="20">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="130"/>
            <ColumnDefinition Width="*/>
        </Grid.ColumnDefinitions>

        <!-- Nagłówek -->
        <TextBlock Grid.Row="0" Grid.ColumnSpan="2"
            Text="Panel logowania"
            FontSize="20" FontWeight="Bold"
            HorizontalAlignment="Center"
            Margin="0,0,0,20"/>

        <!-- Login -->
        <Label Grid.Row="1" Grid.Column="0"
            Content="Nazwa użytkownika:"
            VerticalAlignment="Center"/>
        <TextBox x:Name="txtLogin"
```

```

        Grid.Row="1" Grid.Column="1"
        Padding="8,5" Margin="0,5" />

<!-- Hasło z domyślnym znakiem maskującym (●) -->
<Label Grid.Row="2" Grid.Column="0"
        Content="Hasło:"
        VerticalAlignment="Center" />
<PasswordBox x:Name="pwdHasło"
        Grid.Row="2" Grid.Column="1"
        MaxLength="32"
        Padding="8,5" Margin="0,5" />

<!-- Hasło z innym znakiem maskującym -->
<Label Grid.Row="3" Grid.Column="0"
        Content="Hasło (*-maski):"
        VerticalAlignment="Center" />
<PasswordBox x:Name="pwdHasłoGwiazda"
        Grid.Row="3" Grid.Column="1"
        PasswordChar="*"
        MaxLength="32"
        Padding="8,5" Margin="0,5" />

<!-- Potwierdzenie hasła -->
<Label Grid.Row="4" Grid.Column="0"
        Content="Potwierdź hasło:"
        VerticalAlignment="Center" />
<PasswordBox x:Name="pwdPotwierdzenie"
        Grid.Row="4" Grid.Column="1"
        MaxLength="32"
        Padding="8,5" Margin="0,5" />

<!-- Przyciski -->
<StackPanel Grid.Row="5" Grid.ColumnSpan="2"
        Orientation="Horizontal"
        HorizontalAlignment="Right"
        Margin="0,15,0,0">
    <Button Content="Wyczyść"
        Width="80" Height="32"
        Margin="0,0,8,0"
        Click="btnWyczysc_Click" />
    <Button Content="Zaloguj"
        Width="90" Height="32"
        Background="#1976D2" Foreground="White"
        Click="btnZaloguj_Click" />
</StackPanel>

<!-- Wynik -->
<TextBlock x:Name="tbWynik"
        Grid.Row="6" Grid.ColumnSpan="2"
        TextWrapping="Wrap"
        FontSize="13"
        Margin="0,10"
        HorizontalAlignment="Center" />

</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

```

```
namespace PasswordBoxPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            txtLogin.Focus();
        }

        private void btnZaloguj_Click(object sender, RoutedEventArgs e)
        {
            string login = txtLogin.Text.Trim();
            string haslo = pwdHaslo.Password;
            string potwierdzenie = pwdPotwierdzenie.Password;

            // Walidacja pustych pól
            if (string.IsNullOrEmpty(login))
            {
                tbWynik.Text = "Podaj nazwę użytkownika!";
                tbWynik.Foreground = System.Windows.Media.Brushes.Red;
                txtLogin.Focus();
                return;
            }

            if (string.IsNullOrEmpty(haslo))
            {
                tbWynik.Text = "Podaj hasło!";
                tbWynik.Foreground = System.Windows.Media.Brushes.Red;
                pwdHaslo.Focus();
                return;
            }

            // Sprawdzenie czy hasła są zgodne
            if (haslo != potwierdzenie)
            {
                tbWynik.Text = "Hasła nie są zgodne!";
                tbWynik.Foreground = System.Windows.Media.Brushes.Red;
                pwdPotwierdzenie.Clear();
                pwdPotwierdzenie.Focus();
                return;
            }

            // Walidacja długości
            if (haslo.Length < 6)
            {
                tbWynik.Text = "Hasło musi mieć co najmniej 6 znaków!";
                tbWynik.Foreground = System.Windows.Media.Brushes.Orange;
                return;
            }

            // Sukces – nie wyświetlamy hasła!
            tbWynik.Text = $"✓ Zalogowano jako: {login}";
            tbWynik.Foreground = System.Windows.Media.Brushes.Green;
        }

        private void btnWyczysc_Click(object sender, RoutedEventArgs e)
        {
            txtLogin.Clear();
            pwdHaslo.Clear();
            pwdHasloGwiazda.Clear();
            pwdPotwierdzenie.Clear();
        }
    }
}
```



```
        tbWynik.Text = "";\n        txtLogin.Focus();\n    }\n}\n}
```

9. Przyciski i akcje użytkownika

Przyciski są podstawowym sposobem interakcji użytkownika z aplikacją. Kliknięcie przycisku wyzwała zdarzenie, które obsługujesz w kodzie C#.

9.1. Button

Button to kontrolka reagująca na kliknięcie. Właściwość **Content** określa co jest wyświetlane na przycisku — może to być tekst, ikona lub dowolna kontrolka.

PODSTAWOWE WŁAŚCIWOŚCI BUTTON

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Content	Zawartość (tekst, kontrolka)	Content ="Kliknij"
Click	Zdarzenie kliknięcia	Click ="btnOk_Click"
Width , Height	Rozmiary	Width ="100" Height ="32"
IsEnabled	Czy aktywny	IsEnabled ="False"
IsDefault	Reaguje na Enter	IsDefault ="True"
IsCancel	Reaguje na Escape	IsCancel ="True"

ISDEFAULT I ISCANCEL

- IsDefault**="True" — przycisk reaguje na naciśnięcie Enter (nawet gdy fokus jest gdzie indziej)
- IsCancel**="True" — przycisk reaguje na naciśnięcie Escape

MainWindow.xaml:

```
<Window x:Class="ButtonPrzykład.MainWindow"\n        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"\n        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"\n        Title="Button – właściwości i style" Height="500" Width="500">\n    <StackPanel Margin="15">\n        <TextBlock Text="Style i rozmiary przycisków"\n                    FontSize="18" FontWeight="Bold" Margin="0,0,0,15"/>\n\n        <!-- Domyślny przycisk -->\n        <Button Content="Domyślny przycisk" Margin="0,5"\n                HorizontalAlignment="Left"/>\n\n        <!-- Różne rozmiary -->\n        <StackPanel Orientation="Horizontal" Margin="0,5">\n            <Button Content="Mały" Width="60" Height="24" Margin="0,0,0,0"/>\n            <Button Content="Normalny" Width="100" Height="30" Margin="0,0,0,0"/>\n            <Button Content="Duży" Width="140" Height="40" FontSize="15"/>\n        </StackPanel>\n\n        <!-- Różne kolory -->\n        <StackPanel Orientation="Horizontal" Margin="0,5">\n            <Button Content="Zielony" Width="90" Height="32" Margin="0,0,0,0"\n                    Background="#4CAF50" Foreground="White"/>\n            <Button Content="Czerwony" Width="90" Height="32" Margin="0,0,0,0"\n                    Background="#F44336" Foreground="White"/>\n            <Button Content="Niebieski" Width="90" Height="32" Margin="0,0,0,0"\n                    Background="#2196F3" Foreground="White"/>\n        </StackPanel>\n    </StackPanel>\n</Window>
```

```

        <Button Content="Szary" Width="90" Height="32"
              Background="#9E9E9E" Foreground="White" />
    </StackPanel>

    <!-- Przycisk wyłączony -->
    <Button Content="Wyłączony (IsEnabled=False)"
          IsEnabled="False" HorizontalAlignment="Left"
          Margin="0,5" Width="220" />

    <!-- IsDefault - reaguje na Enter -->
    <StackPanel Margin="0,10">
        <TextBlock Text="Poniższy przycisk reaguje na Enter (IsDefault):"
              FontStyle="Italic" Margin="0,0,0,5" />
        <StackPanel Orientation="Horizontal">
            <TextBox x:Name="txtTest" Width="200" Padding="5,3"
                  Margin="0,0,8,0" />
            <Button Content="OK (Enter)"
                  IsDefault="True"
                  Height="30"
                  Click="btnDefault_Click" />
        </StackPanel>
    </StackPanel>

    <!-- Przycisk z zawartością złożoną (StackPanel wewnątrz) -->
    <Button HorizontalAlignment="Left" Padding="10,5" Margin="0,10">
        <StackPanel Orientation="Horizontal">
            <TextBlock Text="📁" FontSize="16" VerticalAlignment="Center"
                  Margin="0,0,8,0" />
            <TextBlock Text="Zapisz plik" VerticalAlignment="Center"
                  FontSize="14" />
        </StackPanel>
    </Button>

    <!-- Wynik -->
    <TextBlock x:Name="tbWynik" FontSize="14" Margin="0,10"
          Foreground="DarkBlue" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace ButtonPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnDefault_Click(object sender, RoutedEventArgs e)
        {
            string tekst = txtTest.Text;
            if (string.IsNullOrWhiteSpace(tekst))
                tbWynik.Text = "Wpisz coś i naciśnij Enter lub kliknij OK!";
            else
                tbWynik.Text = $"Wpisałeś: \"{tekst}\"";
        }
    }
}

```

```
}  
}
```

9.2. Zdarzenie Click

Kliknięcie przycisku to **zdarzenie** (event). Musisz je podpiąć — powiedzieć aplikacji, jaka metoda ma zostać wywołana po kliknięciu.

SPOSÓB 1: W XAML (ZALECANY)

```
<Button Content="Kliknij" Click="btnKliknij_Click"/>
```

Visual Studio automatycznie generuje metodę w C# gdy klikniesz dwukrotnie na atrybut Click.

SPOSÓB 2: W C# (PROGRAMOWO)

```
public MainWindow()  
{  
    InitializeComponent();  
    btnKliknij.Click += btnKliknij_Click;  
}
```

PARAMETRY METODY OBSŁUGI ZDARZENIA

Każda metoda obsługi zdarzenia Click ma zawsze dwa parametry:

```
private void btnKliknij_Click(object sender, RoutedEventArgs e)  
{  
    // sender - obiekt który wywołał zdarzenie (np. Button)  
    // e      - informacje o zdarzeniu (RoutedEventArgs)  
}
```

- **sender** — to sam przycisk, który został kliknięty. Możesz go rzutować na **Button**:

```
Button btn = sender as Button;  
string tekst = btn.Content.ToString();
```

- **e** — zawiera szczegóły zdarzenia; dla zwykłego Click nie jest zazwyczaj potrzebny

Kompletny przykład z obsługą zdarzenia Click:

MainWindow.xaml:

```
<Window x:Class="ClickZdarzeniePrzyklad.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Click - zdarzenie" Height="420" Width="480">  
    <Grid Margin="15">  
        <Grid.RowDefinitions>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="*/>  
        </Grid.RowDefinitions>  
        <Grid.ColumnDefinitions>  
            <ColumnDefinition Width="120"/>  
            <ColumnDefinition Width="*/>  
        </Grid.ColumnDefinitions>
```

```

<!-- Nagłówek -->
<TextBlock Grid.Row="0" Grid.ColumnSpan="2"
            Text="Demo zdarzenia Click"
            FontSize="18" FontWeight="Bold"
            Margin="0,0,0,15"/>

<!-- Pola formularza -->
<Label Grid.Row="1" Grid.Column="0" Content="Imię:"
        VerticalAlignment="Center"/>
<TextBox x:Name="txtImie" Grid.Row="1" Grid.Column="1"
         Padding="8,4" Margin="0,4"/>

<Label Grid.Row="2" Grid.Column="0" Content="Wiek:"
        VerticalAlignment="Center"/>
<TextBox x:Name="txtWiek" Grid.Row="2" Grid.Column="1"
         Padding="8,4" Margin="0,4"/>

<Label Grid.Row="3" Grid.Column="0" Content="Miasto:"
        VerticalAlignment="Center"/>
<TextBox x:Name="txtMiasto" Grid.Row="3" Grid.Column="1"
         Padding="8,4" Margin="0,4"/>

<!-- Przyciski – podpięte przez atrybut Click w XAML -->
<StackPanel Grid.Row="4" Grid.ColumnSpan="2"
            Orientation="Horizontal"
            HorizontalAlignment="Right"
            Margin="0,12,0,0">
    <Button x:Name="btnWyczysc"
            Content="Wyczyść"
            Width="90" Height="32"
            Margin="0,0,8,0"
            Click="btnWyczysc_Click"/>
    <Button x:Name="btnZatwierdz"
            Content="Zatwierdź"
            Width="100" Height="32"
            Background="#1565C0"
            Foreground="White"
            IsDefault="True"
            Click="btnZatwierdz_Click"/>
</StackPanel>

<!-- Pole wyniku -->
<Border Grid.Row="5" Grid.ColumnSpan="2"
        Background="#F3F4F6"
        BorderBrush="#D1D5DB"
        BorderThickness="1"
        Padding="12" Margin="0,12,0,0">
    <TextBlock x:Name="tbWynik"
            TextWrapping="Wrap"
            FontSize="13"
            LineHeight="22"/>
</Border>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace ClickZdarzeniePrzyklad

```

```

{
    public partial class MainWindow : Window
    {
        // Licznik kliknięć – pole klasy
        private int licznikKlikniec = 0;

        public MainWindow()
        {
            InitializeComponent();
            txtImie.Focus();
        }

        /// <summary>
        /// Metoda obsługi kliknięcia przycisku "Zatwierdź".
        /// Parametr sender to przycisk który wywołał zdarzenie.
        /// Parametr e zawiera informacje o zdarzeniu.
        /// </summary>
        private void btnZatwierdz_Click(object sender, RoutedEventArgs e)
        {
            licznikKlikniec++;

            // Możemy sprawdzić który przycisk wywołał zdarzenie
            Button kliknietyPrzycisk = sender as Button;
            string nazwaPrzycisku = kliknietyPrzycisk?.Content?.ToString() ?? "Nieznany";

            // Odczytujemy dane z pól
            string imie = txtImie.Text.Trim();
            string wiek = txtWiek.Text.Trim();
            string miasto = txtMiasto.Text.Trim();

            // Walidacja
            if (string.IsNullOrEmpty(imie))
            {
                tbWynik.Text = "❌ Błąd: Podaj imię!";
                tbWynik.Foreground = System.Windows.Media.Brushes.Red;
                txtImie.Focus();
                return;
            }

            // Budujemy wynik
            tbWynik.Text =
                $"✓ Kliknięto: [{nazwaPrzycisku}] (kliknięcie nr {licznikKlikniec})\n" +
                $"Imię: {imie}\n" +
                $"Wiek: {(string.IsNullOrEmpty(wiek) ? "nie podano" : wiek)}\n" +
                $"Miasto: {(string.IsNullOrEmpty(miasto) ? "nie podano" : miasto)}";

            tbWynik.Foreground = System.Windows.Media.Brushes.DarkGreen;
        }

        private void btnWyczysc_Click(object sender, RoutedEventArgs e)
        {
            txtImie.Clear();
            txtWiek.Clear();
            txtMiasto.Clear();
            tbWynik.Text = "";
            licznikKlikniec = 0;
            txtImie.Focus();
        }
    }
}

```

9.3. Jeden przycisk — jedna odpowiedzialność

Dobra praktyka programowania: każdy przycisk powinien mieć **jedną, jasno określoną** odpowiedzialność. Nie tadj całej logiki do jednej metody.

ZŁA PRAKTYKA — WSZYSTKO W JEDNEJ METODZIE:

```
// ŹŁE — zbyt wiele obowiązków w jednej metodzie
private void btnOk_Click(object sender, RoutedEventArgs e)
{
    // walidacja...
    // przeliczanie...
    // formatowanie...
    // wyświetlanie...
    // zapis do pliku...
    // 80 linii kodu...
}
```

DOBRA PRAKTYKA — WYDZIELONE METODY:

```
// DOBRZE — metoda obsługi tylko koordynuje
private void btnOk_Click(object sender, RoutedEventArgs e)
{
    if (!WalidujDane()) return;
    decimal wynik = ObliczWynik();
    WyszwiatlWynik(wynik);
}

private bool WalidujDane()
{
    // logika walidacji
    return true;
}

private decimal ObliczWynik()
{
    // logika obliczeń
    return 0;
}

private void WyszwiatlWynik(decimal wynik)
{
    // logika wyświetlania
}
```

Kompletny przykład z refaktoryzacją:

MainWindow.xaml:

```
<Window x:Class="JednaOdpowiedzialnosc.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kalkulator BMI — zasada jednej odpowiedzialności" Height="380" Width="440">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
```

```

        <ColumnDefinition Width="130"/>
        <ColumnDefinition Width="*/>
    </Grid.ColumnDefinitions>

    <TextBlock Grid.Row="0" Grid.ColumnSpan="2"
        Text="Kalkulator BMI"
        FontSize="20" FontWeight="Bold"
        HorizontalAlignment="Center" Margin="0,0,0,20"/>

    <Label Grid.Row="1" Grid.Column="0" Content="Waga (kg):"
        VerticalAlignment="Center"/>
    <TextBox x:Name="txtWaga" Grid.Row="1" Grid.Column="1"
        Padding="8,4" Margin="0,5" TextAlignment="Right"/>

    <Label Grid.Row="2" Grid.Column="0" Content="Wzrost (cm):"
        VerticalAlignment="Center"/>
    <TextBox x:Name="txtWzrost" Grid.Row="2" Grid.Column="1"
        Padding="8,4" Margin="0,5" TextAlignment="Right"/>

    <Button Grid.Row="3" Grid.ColumnSpan="2"
        Content="Oblicz BMI"
        Height="35"
        Background="#1976D2" Foreground="White"
        FontSize="14"
        Margin="0,10"
        IsDefault="True"
        Click="btnOblicz_Click"/>

    <!-- Wynik BMI -->
    <Border Grid.Row="4" Grid.ColumnSpan="2"
        Background="#F5F5F5" BorderBrush="#E0E0E0"
        BorderThickness="1" Padding="12" Margin="0,5">
        <StackPanel>
            <TextBlock x:Name="tbBMI" FontSize="22"
                FontWeight="Bold" TextAlignment="Center"/>
            <TextBlock x:Name="tbKategoria" FontSize="14"
                TextAlignment="Center" Margin="0,5,0,0"/>
        </StackPanel>
    </Border>

    <!-- Błędy -->
    <TextBlock x:Name="tbBład" Grid.Row="5" Grid.ColumnSpan="2"
        Foreground="Red" TextWrapping="Wrap"
        FontSize="12" Margin="0,5"/>

</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Media;

namespace JednaOdpowiedzialnosc
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            txtWaga.Focus();
        }
    }
}

```

```

}

// Metoda obsługi kliknięcia – TYLKO koordynuje, nie robi wszystkiego
private void btnOblicz_Click(object sender, RoutedEventArgs e)
{
    tbBlad.Text = "";
    tbBMI.Text = "";
    tbKategoria.Text = "";

    if (!WalidujDane(out double waga, out double wzrost))
        return;

    double bmi = ObliczBMI(waga, wzrost);
    string kategoria = OkreslKategorie(bmi);

    WyswietlWynik(bmi, kategoria);
}

// Odpowiada TYLKO za walidację danych wejściowych
private bool WalidujDane(out double waga, out double wzrost)
{
    waga = 0;
    wzrost = 0;

    if (!double.TryParse(txtWaga.Text.Replace(",", "."),
        System.Globalization.NumberStyles.Any,
        System.Globalization.CultureInfo.InvariantCulture,
        out waga) || waga ≤ 0)
    {
        tbBlad.Text = "✗ Podaj poprawną wagę (np. 70 lub 70,5)";
        txtWaga.Focus();
        return false;
    }

    if (!double.TryParse(txtWzrost.Text.Replace(",", "."),
        System.Globalization.NumberStyles.Any,
        System.Globalization.CultureInfo.InvariantCulture,
        out wzrost) || wzrost ≤ 0)
    {
        tbBlad.Text = "✗ Podaj poprawny wzrost w cm (np. 175)";
        txtWzrost.Focus();
        return false;
    }

    return true;
}

// Odpowiada TYLKO za obliczenie BMI
private double ObliczBMI(double waga, double wzrostCm)
{
    double wzrostM = wzrostCm / 100.0;
    return waga / (wzrostM * wzrostM);
}

// Odpowiada TYLKO za określenie kategorii
private string OkreslKategorie(double bmi)
{
    if (bmi < 18.5) return "Niedowaga";
    if (bmi < 25.0) return "Waga prawidłowa";
    if (bmi < 30.0) return "Nadwaga";
    return "Otyłość";
}

```



```

// Odpowiada TYLKO za wyświetlenie wyniku
private void WyświetlWynik(double bmi, string kategoria)
{
    tbBMI.Text = $"BMI = {bmi:F2}";

    // Kolor zależny od kategorii
    tbKategoria.Text = kategoria;
    tbKategoria.Foreground = kategoria switch
    {
        "Waga prawidłowa" => Brushes.Green,
        "Niedowaga" => Brushes.Orange,
        "Nadwaga" => Brushes.DarkOrange,
        _ => Brushes.Red
    };

    tbBMI.Foreground = tbKategoria.Foreground;
}
}
}

```

9.4. Kilka przycisków w jednym oknie

Gdy masz wiele przycisków, każdy powinien mieć **osobną metodę** obsługi. Możesz też użyć jednej metody i sprawdzać `sender`, ale tylko gdy przyciski robią to samo z różnymi wartościami.

MainWindow.xaml:

```

<Window x:Class="KilkaPrzyciskow.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kilka przycisków – osobne metody" Height="520" Width="500">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="110"/>
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>

        <TextBlock Grid.Row="0" Grid.ColumnSpan="2"
            Text="Kalkulator walut"
            FontSize="18" FontWeight="Bold"
            Margin="0,0,0,15"/>

        <!-- Kwota -->
        <Label Grid.Row="1" Grid.Column="0" Content="Kwota (PLN):"
            VerticalAlignment="Center"/>
        <TextBox x:Name="txtKwota" Grid.Row="1" Grid.Column="1"
            Padding="8,4" Margin="0,5" TextAlignment="Right"/>

        <!-- Wynik -->
        <Label Grid.Row="2" Grid.Column="0" Content="Wynik:"
            VerticalAlignment="Center"/>
    </Grid>

```

```
<TextBox x:Name="txtWynik" Grid.Row="2" Grid.Column="1"
        Padding="8,4" Margin="0,5"
        IsReadOnly="True" Background="#F5F5F5"
        TextAlignment="Right"/>

<!-- Informacja o kursie -->
<TextBlock x:Name="tbKurs" Grid.Row="3" Grid.ColumnSpan="2"
        Foreground="Gray" FontSize="11"
        HorizontalAlignment="Right" Margin="0,0,0,10"/>

<!-- Cztery przyciski - każdy z osobną metodą -->
<StackPanel Grid.Row="4" Grid.ColumnSpan="2"
        Orientation="Horizontal"
        HorizontalAlignment="Center"
        Margin="0,5,0,10">

    <Button x:Name="btnEuro"
        Content="→ EUR"
        Width="90" Height="36"
        Background="#1565C0" Foreground="White"
        Margin="4"
        Click="btnEuro_Click"/>

    <Button x:Name="btnDolar"
        Content="→ USD"
        Width="90" Height="36"
        Background="#2E7D32" Foreground="White"
        Margin="4"
        Click="btnDolar_Click"/>

    <Button x:Name="btnFunt"
        Content="→ GBP"
        Width="90" Height="36"
        Background="#6A1B9A" Foreground="White"
        Margin="4"
        Click="btnFunt_Click"/>

    <Button x:Name="btnFrank"
        Content="→ CHF"
        Width="90" Height="36"
        Background="#B71C1C" Foreground="White"
        Margin="4"
        Click="btnFrank_Click"/>

</StackPanel>

<!-- Wyczyść i historia -->
<StackPanel Grid.Row="5" Grid.ColumnSpan="2"
        Orientation="Horizontal"
        HorizontalAlignment="Right"
        Margin="0,0,0,10">

    <Button Content="Wyczyść"
        Width="90" Height="30"
        Margin="0,0,8,0"
        Click="btnWyczysc_Click"/>

</StackPanel>

<!-- Historia obliczeń -->
<Border Grid.Row="6" Grid.ColumnSpan="2"
        Background="#FAFAFA" BorderBrush="#E0E0E0"
        BorderThickness="1" Padding="8">

    <ScrollView VerticalScrollBarVisibility="Auto">

        <TextBlock x:Name="tbHistoria"
            TextWrapping="Wrap"
            FontFamily="Courier New"
            FontSize="11"/>

    </ScrollView>
```

```
</Border>

</Grid>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Text;
using System.Windows;

namespace KilkaPrzyciskow
{
    public partial class MainWindow : Window
    {
        // Kursy walut względem PLN
        private const double KursEuro = 0.2326; // 1 PLN = 0.2326 EUR
        private const double KursDolar = 0.2517;
        private const double KursFunt = 0.1986;
        private const double KursFrank = 0.2260;

        private StringBuilder historia = new StringBuilder();

        public MainWindow()
        {
            InitializeComponent();
            txtKwota.Focus();
        }

        // Osobna metoda dla Euro
        private void btnEuro_Click(object sender, RoutedEventArgs e)
        {
            PrzeliczWalute("EUR", KursEuro, "€");
        }

        // Osobna metoda dla Dolara
        private void btnDolar_Click(object sender, RoutedEventArgs e)
        {
            PrzeliczWalute("USD", KursDolar, "$");
        }

        // Osobna metoda dla Funta
        private void btnFunt_Click(object sender, RoutedEventArgs e)
        {
            PrzeliczWalute("GBP", KursFunt, "£");
        }

        // Osobna metoda dla Franka
        private void btnFrank_Click(object sender, RoutedEventArgs e)
        {
            PrzeliczWalute("CHF", KursFrank, "Fr");
        }

        // Wspólna logika przeliczania – wydzielona do osobnej metody
        private void PrzeliczWalute(string waluta, double kurs, string symbol)
        {
            if (!PobierzKwote(out double kwota)) return;

            double wynik = kwota * kurs;
            txtWynik.Text = $"{wynik:F2} {symbol}";
            tbKurs.Text = $"Kurs: 1 PLN = {kurs:F4} {waluta}";
        }
    }
}
```

```

        // Dodajemy do historii
        string wpis = $"{DateTime.Now:HH:mm:ss} {kwota:F2} PLN → {wynik:F2} {symbol}\n";
        historia.Insert(0, wpis);
        tbHistoria.Text = historia.ToString();
    }

    private bool PobierzKwote(out double kwota)
    {
        string tekst = txtKwota.Text.Trim().Replace(",", ".");
        if (!double.TryParse(tekst,
            System.Globalization.NumberStyles.Any,
            System.Globalization.CultureInfo.InvariantCulture,
            out kwota) || kwota < 0)
        {
            MessageBox.Show("Podaj poprawną kwotę w PLN!",
                "Błąd danych",
                MessageBoxButton.OK,
                MessageBoxImage.Warning);

            txtKwota.Focus();
            return false;
        }
        return true;
    }

    private void btnWyczyszc_Click(object sender, RoutedEventArgs e)
    {
        txtKwota.Clear();
        txtWynik.Clear();
        tbKurs.Text = "";
        historia.Clear();
        tbHistoria.Text = "";
        txtKwota.Focus();
    }
}
}

```

9.5. Włączanie i wyłączanie przycisków: `IsEnabled`

Właściwość `IsEnabled` kontroluje, czy użytkownik może kliknąć przycisk. Wyłączony przycisk (`IsEnabled="False"`) jest wyszarzony i nie reaguje na kliknięcia.

KIEDY WYŁĄCZAĆ PRZYCISKI?

- Gdy brak danych do przetworzenia
- Gdy operacja jest już w toku
- Gdy użytkownik nie spełnił warunków (np. nie wypełnił formularza)
- Po jednorazowej akcji (np. wysłaniu formularza — by uniknąć wielokrotnego wysłania)

```

btnZatwierdz.IsEnabled = false; // wyłącz
btnZatwierdz.IsEnabled = true;  // włącz

```

MainWindow.xaml:

```

<Window x:Class="IsEnabledPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="IsEnabled – włączanie i wyłączanie" Height="460" Width="480">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>

```

```

        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="130" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <TextBlock Grid.Row="0" Grid.ColumnSpan="2"
        Text="Formularz zamówienia"
        FontSize="18" FontWeight="Bold"
        Margin="0,0,0,15" />

    <Label Grid.Row="1" Grid.Column="0" Content="Imię:"
        VerticalAlignment="Center" />
    <TextBox x:Name="txtImie" Grid.Row="1" Grid.Column="1"
        Padding="8,4" Margin="0,5"
        TextChanged="txtPole_TextChanged" />

    <Label Grid.Row="2" Grid.Column="0" Content="Nazwisko:"
        VerticalAlignment="Center" />
    <TextBox x:Name="txtNazwisko" Grid.Row="2" Grid.Column="1"
        Padding="8,4" Margin="0,5"
        TextChanged="txtPole_TextChanged" />

    <Label Grid.Row="3" Grid.Column="0" Content="Adres email:"
        VerticalAlignment="Center" />
    <TextBox x:Name="txtEmail" Grid.Row="3" Grid.Column="1"
        Padding="8,4" Margin="0,5"
        TextChanged="txtPole_TextChanged" />

    <Label Grid.Row="4" Grid.Column="0" Content="Produkt:"
        VerticalAlignment="Center" />
    <TextBox x:Name="txtProdukt" Grid.Row="4" Grid.Column="1"
        Padding="8,4" Margin="0,5"
        TextChanged="txtPole_TextChanged" />

    <!-- Info - jakie pola są wymagane -->
    <TextBlock Grid.Row="5" Grid.ColumnSpan="2"
        Text="* Przycisk 'Zamów' jest aktywny tylko gdy wszystkie pola są wypełnione."
        FontStyle="Italic" Foreground="Gray"
        TextWrapping="Wrap" FontSize="11"
        Margin="0,5,0,10" />

    <!-- Przyciski - Zamów jest domyślnie wyłączony -->
    <StackPanel Grid.Row="6" Grid.ColumnSpan="2"
        Orientation="Horizontal"
        HorizontalAlignment="Right">
        <Button Content="Wyczyść"
            Width="90" Height="32"
            Margin="0,0,8,0"
            Click="btnWyczysc_Click" />
        <Button x:Name="btnZamow"
            Content="Zamów"
            Width="100" Height="32"
            Background="#388E3C" Foreground="White"
            IsEnabled="False"

```

```

        IsDefault="True"
        Click="btnZamow_Click" />
</StackPanel>

<!-- Status -->
<TextBlock x:Name="tbStatus" Grid.Row="7" Grid.ColumnSpan="2"
        Margin="0,12"
        FontSize="13"
        TextWrapping="Wrap" />
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Controls;

namespace IsEnabledPrzyklad
{
    public partial class MainWindow : Window
    {
        private bool zamowioneJuz = false;

        public MainWindow()
        {
            InitializeComponent();
            txtImie.Focus();
            // Sprawdzamy stan od razu po załadowaniu
            AktualizujStanPrzycisku();
        }

        /// <summary>
        /// Wywoływana gdy zmieni się tekst w którymkolwiek polu.
        /// Sprawdza czy przycisk powinien być aktywny.
        /// </summary>
        private void txtPole_TextChanged(object sender, TextChangedEventArgs e)
        {
            AktualizujStanPrzycisku();
        }

        /// <summary>
        /// Włącza przycisk Zamów tylko gdy wszystkie wymagane pola są wypełnione.
        /// </summary>
        private void AktualizujStanPrzycisku()
        {
            if (zamowioneJuz)
            {
                // Po złożeniu zamówienia przycisk pozostaje wyłączony
                btnZamow.IsEnabled = false;
                return;
            }

            bool imieOk = !string.IsNullOrEmpty(txtImie.Text);
            bool nazwiskoOk = !string.IsNullOrEmpty(txtNazwisko.Text);
            bool emailOk = !string.IsNullOrEmpty(txtEmail.Text);
            bool produktOk = !string.IsNullOrEmpty(txtProdukt.Text);

            // Przycisk aktywny TYLKO gdy wszystkie pola wypełnione
            btnZamow.IsEnabled = imieOk && nazwiskoOk && emailOk && produktOk;
        }
    }
}

```

```

// Informacja ile pól brakuje
int wypełnione = (imieOk ? 1 : 0) + (nazwiskoOk ? 1 : 0) +
    (emailOk ? 1 : 0) + (produktOk ? 1 : 0);

if (wypełnione < 4)
    tbStatus.Text = $"Wypełnij {4 - wypełnione} brakujące pole(a) aby aktywować przycisk.";
else
    tbStatus.Text = "Wszystkie pola wypełnione – możesz złożyć zamówienie.";

tbStatus.Foreground = wypełnione == 4
    ? System.Windows.Media.Brushes.DarkGreen
    : System.Windows.Media.Brushes.Gray;
}

private void btnZamow_Click(object sender, RoutedEventArgs e)
{
    // Wyłączamy przycisk natychmiast po kliknięciu
    // – zapobiega podwójnemu kliknięciu
    btnZamow.IsEnabled = false;
    zamowioneJuz = true;

    string podsumowanie =
        $"✓ Zamówienie przyjęte!\n" +
        $"Klient: {txtImie.Text} {txtNazwisko.Text}\n" +
        $"Email: {txtEmail.Text}\n" +
        $"Produkt: {txtProdukt.Text}\n" +
        $"Godzina: {DateTime.Now:HH:mm:ss}";

    tbStatus.Text = podsumowanie;
    tbStatus.Foreground = System.Windows.Media.Brushes.DarkGreen;

    // Blokujemy pola – zamówienie już złożone
    txtImie.IsEnabled = false;
    txtNazwisko.IsEnabled = false;
    txtEmail.IsEnabled = false;
    txtProdukt.IsEnabled = false;
}

private void btnWyczyszc_Click(object sender, RoutedEventArgs e)
{
    // Resetujemy stan
    zamowioneJuz = false;

    txtImie.IsEnabled = true;
    txtNazwisko.IsEnabled = true;
    txtEmail.IsEnabled = true;
    txtProdukt.IsEnabled = true;

    txtImie.Clear();
    txtNazwisko.Clear();
    txtEmail.Clear();
    txtProdukt.Clear();
    tbStatus.Text = "";

    AktualizujStanPrzycisku();
    txtImie.Focus();
}
}
}

```

Drugi przykład — dynamiczne włączanie przycisku "Usuń" tylko gdy coś jest zaznaczone:

MainWindow.xaml:

```
<Window x:Class="IsEnabledLista.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="IsEnabled - lista z akcjami" Height="400" Width="400">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <TextBlock Grid.Row="0" Text="Lista zadań"
            FontSize="18" FontWeight="Bold" Margin="0,0,0,10"/>

        <!-- Dodawanie elementu -->
        <StackPanel Grid.Row="1" Orientation="Horizontal" Margin="0,0,0,8">
            <TextBox x:Name="txtNowy" Width="250" Padding="8,4"
                Margin="0,0,8,0"/>
            <Button Content="Dodaj"
                Width="80" Height="32"
                Background="#1976D2" Foreground="White"
                Click="btnDodaj_Click"/>
        </StackPanel>

        <!-- Lista -->
        <ListBox x:Name="lstZadania"
            Grid.Row="2"
            SelectionChanged="lstZadania_SelectionChanged"
            Margin="0,0,0,8"/>

        <!-- Przyciski - Usun wyłączony gdy nic nie zaznaczono -->
        <StackPanel Grid.Row="3" Orientation="Horizontal"
            HorizontalAlignment="Right">
            <Button Content="Wyczyść listę"
                Width="110" Height="30"
                Margin="0,0,8,0"
                Click="btnWyczyscListe_Click"/>
            <Button x:Name="btnUsun"
                Content="Usun zaznaczone"
                Width="130" Height="30"
                Background="#C62828" Foreground="White"
                IsEnabled="False"
                Click="btnUsun_Click"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;
using System.Windows.Controls;

namespace IsEnabledLista
{
    public partial class MainWindow : Window
    {
        public MainWindow()
```



```

{
    InitializeComponent();

    // Dodajemy kilka przykładowych elementów
    lstZadania.Items.Add("Napisać dokumentację WPF");
    lstZadania.Items.Add("Stworzyć formularz rejestracji");
    lstZadania.Items.Add("Przetestować aplikację");
    lstZadania.Items.Add("Naprawić błędy");

    txtNowy.Focus();
}

private void btnDodaj_Click(object sender, RoutedEventArgs e)
{
    string nowe = txtNowy.Text.Trim();
    if (string.IsNullOrEmpty(nowe))
    {
        MessageBox.Show("Wpisz treść zadania!", "Błąd",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        txtNowy.Focus();
        return;
    }

    lstZadania.Items.Add(nowe);
    txtNowy.Clear();
    txtNowy.Focus();
}

// Gdy zmienia się zaznaczenie – aktualizujemy stan przycisku Usuń
private void lstZadania_SelectionChanged(object sender,
    SelectionChangedEventArgs e)
{
    // Przycisk Usuń aktywny tylko gdy coś zaznaczono
    btnUsun.IsEnabled = lstZadania.SelectedItem != null;
}

private void btnUsun_Click(object sender, RoutedEventArgs e)
{
    if (lstZadania.SelectedItem != null)
    {
        lstZadania.Items.Remove(lstZadania.SelectedItem);
        // Po usunięciu – nic nie jest zaznaczone, więc wyłączamy Usuń
        btnUsun.IsEnabled = false;
    }
}

private void btnWyczyscListe_Click(object sender, RoutedEventArgs e)
{
    if (lstZadania.Items.Count == 0)
    {
        MessageBox.Show("Lista jest już pusta.", "Info",
            MessageBoxButton.OK, MessageBoxImage.Information);
        return;
    }

    MessageBoxResult result = MessageBox.Show(
        "Czy na pewno chcesz usunąć wszystkie zadania?",
        "Potwierdzenie",
        MessageBoxButton.YesNo,
        MessageBoxImage.Question);

    if (result == MessageBoxResult.Yes)

```

```
        {
            lstZadania.Items.Clear();

            btnUsun.IsEnabled = false;
        }
    }
}
```

10. Kontrolki wyboru

Kontrolki wyboru pozwalają użytkownikowi dokonywać wyborów spośród dostępnych opcji. W WPF masz do dyspozycji wiele takich kontrolerek: `CheckBox`, `RadioButton`, `ComboBox`, `ListBox` oraz `DataGrid`. Każda z nich nadaje się do innego scenariusza.

10.1. CheckBox

`CheckBox` to kontrolka, która może mieć trzy stany: zaznaczony (`true`), odznaczony (`false`) lub nieokreślony (`null`). Właściwość `IsChecked` jest typu `bool?` (nullable bool), co właśnie umożliwia ten trzeci stan.

STAN	WARTOŚĆ ISCHECKED	WYGLĄD
Zaznaczony	true	☒
Odznaczony	false	☐
Nieokreślony	null	☐ (szary)

Trzeci stan aktywujesz ustawiając `IsThreeState="True"` na `CheckBox`.

MainWindow.xaml:

```
<Window x:Class="PrzykladCheckBox.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="CheckBox - przykład" Height="300" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Wybierz opcje:" FontWeight="Bold" Margin="0,0,0,10"/>

        <CheckBox x:Name="chkAkceptacja"
            Content="Akceptuję regulamin"
            Margin="0,5"
            Checked="chkAkceptacja_Checked"
            Unchecked="chkAkceptacja_Unchecked" />

        <CheckBox x:Name="chkNewsletter"
            Content="Chcę otrzymywać newsletter"
            IsChecked="True"
            Margin="0,5" />

        <CheckBox x:Name="chkTrzyStany"
            Content="Opcja z trzema stanami (kliknij kilka razy)"
            IsThreeState="True"
            IsChecked="{x:Null}"
            Margin="0,5" />

        <Button Content="Sprawdź stany"
            Margin="0,15,0,0"
            Click="BtnSprawdz_Click"
            Width="150"
            HorizontalAlignment="Left" />

        <TextBlock x:Name="txtWynik"
            Margin="0,10,0,0"
            TextWrapping="Wrap"
```

```
        Foreground="DarkBlue" />

    </StackPanel>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace PrzykladCheckBox
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void chkAkceptacja_Checked(object sender, RoutedEventArgs e)
            {
                // Zdarzenie wywoływane gdy CheckBox zostaje zaznaczony
                MessageBox.Show("Dziękujemy za akceptację regulaminu!");
            }

            private void chkAkceptacja_Unchecked(object sender, RoutedEventArgs e)
            {
                // Zdarzenie wywoływane gdy CheckBox zostaje odznaczony
                MessageBox.Show("Musisz zaakceptować regulamin, aby kontynuować.");
            }

            private void BtnSprawdz_Click(object sender, RoutedEventArgs e)
            {
                // Odczytujemy wartość bool? z każdego CheckBox
                bool? akceptacja = chkAkceptacja.IsChecked;
                bool? newsletter = chkNewsletter.IsChecked;
                bool? trzyStany = chkTrzyStany.IsChecked;

                // Konwertujemy bool? na czytelny tekst
                string stanAkceptacji = akceptacja == true ? "Zaznaczony"
                    : akceptacja == false ? "Odznaczony"
                    : "Nieokreślony";

                string stanTrzechStanow = trzyStany == true ? "Zaznaczony"
                    : trzyStany == false ? "Odznaczony"
                    : "Nieokreślony (null)";

                txtWynik.Text = $"Regulamin: {stanAkceptacji}\n" +
                    $"Newsletter: {(newsletter == true ? "TAK" : "NIE")}\n" +
                    $"Trzy stany: {stanTrzechStanow}";
            }
        }
    }
}
```

Kluczowe elementy:

- `IsChecked` — właściwość typu `bool?` (nullable bool); sprawdzaj ją przez `= true`, `= false` lub `= null`
- `IsThreeState="True"` — włącza trzeci stan (nieokreślony)
- `Checked` / `Unchecked` — zdarzenia wywoływane odpowiednio przy zaznaczeniu i odznaczeniu
- `x:Null` — sposób na ustawienie wartości `null` w XAML

10.2. CheckBox

Klasyczny przykład użycia wielu **CheckBox** to generator haseł. Użytkownik wybiera, jakie znaki mają wchodzić w skład hasła.

MainWindow.xaml:

```
<Window x:Class="GeneratorHasel.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Generator Haseł" Height="420" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Generator Haseł" FontSize="20" FontWeight="Bold" Margin="0,0,0,15"/>

        <TextBlock Text="Skład hasła:" FontWeight="Bold" Margin="0,0,0,8"/>
        <CheckBox x:Name="chkMaleLiterary"
            Content="Małe litery (a-z)"
            IsChecked="True"
            Margin="0,4"/>
        <CheckBox x:Name="chkWielkieLiterary"
            Content="Wielkie litery (A-Z)"
            IsChecked="True"
            Margin="0,4"/>
        <CheckBox x:Name="chkCyfry"
            Content="Cyfry (0-9)"
            IsChecked="True"
            Margin="0,4"/>
        <CheckBox x:Name="chkZnakiSpecjalne"
            Content="Znaki specjalne (!@#$%^&*)"
            IsChecked="False"
            Margin="0,4"/>

        <TextBlock Text="Długość hasła:" FontWeight="Bold" Margin="0,15,0,5"/>
        <StackPanel Orientation="Horizontal">
            <Slider x:Name="sldDlugosc"
                Minimum="4" Maximum="32"
                Value="12"
                Width="250"
                TickFrequency="1"
                IsSnapToTickEnabled="True"
                ValueChanged="sldDlugosc_ValueChanged"/>
            <TextBlock x:Name="txtDlugosc"
                Text="12"
                Margin="10,0,0,0"
                VerticalAlignment="Center"
                FontWeight="Bold"
                Foreground="DarkBlue"/>
        </StackPanel>

        <Button Content="Generuj hasło"
            Margin="0,20,0,0"
            Height="35"
            Click="BtnGeneruj_Click"
            Background="DarkGreen"
            Foreground="White"
            FontSize="14"/>

        <Border BorderBrush="Gray" BorderThickness="1" CornerRadius="4" Padding="10" Margin="0,15,0,0">
            <StackPanel>
                <TextBlock Text="Wygenerowane hasło:" FontWeight="Bold" Margin="0,0,0,5"/>
                <TextBox x:Name="txtHaslo"
                    IsReadOnly="True"
                    FontFamily="Consolas"
                    FontSize="16"
                    Width="250"
                    Height="30"
                    Margin="0,0,0,0"/>
            </StackPanel>
        </Border>
    </StackPanel>
</Window>
```

```

                Padding="5"
                TextWrapping="Wrap" />
            </StackPanel>
        </Border>

        <TextBlock x:Name="txtBladInfo"
            Foreground="Red"
            Margin="0,8,0,0"
            TextWrapping="Wrap" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Text;
using System.Windows;

namespace GeneratorHasel
{
    public partial class MainWindow : Window
    {
        private Random rnd = new Random();

        // Pule znaków
        private const string MaleLiterary = "abcdefghijklmnopqrstuvwxyz";
        private const string WielkieLiterary = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        private const string Cyfry = "0123456789";
        private const string ZnakiSpecjalne = "!@#$$%^&*()_+--[{}|];:,.<>?";

        public MainWindow()
        {
            InitializeComponent();
        }

        private void sldDlugosc_ValueChanged(object sender, RoutedEventArgs<double> e)
        {
            // Aktualizujemy etykietę obok suwaka
            if (txtDlugosc != null)
                txtDlugosc.Text = ((int)e.NewValue).ToString();
        }

        private void BtnGeneruj_Click(object sender, RoutedEventArgs e)
        {
            txtBladInfo.Text = "";

            // Budujemy pulę znaków na podstawie wybranych checkboxów
            StringBuilder pula = new StringBuilder();

            if (chkMaleLiterary.IsChecked == true)
                pula.Append(MaleLiterary);
            if (chkWielkieLiterary.IsChecked == true)
                pula.Append(WielkieLiterary);
            if (chkCyfry.IsChecked == true)
                pula.Append(Cyfry);
            if (chkZnakiSpecjalne.IsChecked == true)
                pula.Append(ZnakiSpecjalne);

            // Sprawdzamy czy wybrano przynajmniej jeden rodzaj znaków
            if (pula.Length == 0)
            {

```

```

        txtBladInfo.Text = "Wybierz przynajmniej jeden rodzaj znaków!";
        txtHaslo.Text = "";
        return;
    }

    // Generujemy hasło
    int dlugosc = (int)sldDlugosc.Value;
    string pulaStr = pula.ToString();
    StringBuilder haslo = new StringBuilder();

    for (int i = 0; i < dlugosc; i++)
    {
        int indeks = rnd.Next(pulaStr.Length);
        haslo.Append(pulaStr[indeks]);
    }

    txtHaslo.Text = haslo.ToString();
}
}
}

```

Kluczowe elementy:

- **Sprawdzanie** `IsChecked == true` — bezpieczne porównanie dla nullable bool
- `StringBuilder pula` — dynamicznie budujemy zestaw dozwolonych znaków
- **Walidacja** — informujemy użytkownika, gdy nie wybrał żadnego zestawu znaków
- `Random.Next(n)` — losujemy indeks z puli znaków

10.3. Button

`RadioButton` służy do wyboru jednej opcji spośród kilku. W odróżnieniu od `CheckBox`, zaznaczenie jednego `RadioButton` w grupie automatycznie odznacza pozostałe.

Właściwość `IsChecked` działa tak samo jak w `CheckBox` — jest typu `bool?`.

MainWindow.xaml:

```

<Window x:Class="PrzykladRadio.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="RadioButton - przykład" Height="300" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Wybierz poziom trudności:" FontWeight="Bold" Margin="0,0,0,10"/>

        <RadioButton x:Name="rbLatwy"
            Content="Łatwy"
            IsChecked="True"
            Margin="0,5"
            Checked="RbPoziom_Checked" />

        <RadioButton x:Name="rbSredni"
            Content="Średni"
            Margin="0,5"
            Checked="RbPoziom_Checked" />

        <RadioButton x:Name="rbTrudny"
            Content="Trudny"
            Margin="0,5"
            Checked="RbPoziom_Checked" />

        <TextBlock x:Name="txtWybrany"
            Text="Wybrany poziom: Łatwy"
            Margin="0,15,0,0"
            FontSize="14"

```

```

        Foreground="DarkBlue" />

        <Button Content="Zatwierdź wybór"
            Margin="0,15,0,0"
            Width="150"
            HorizontalAlignment="Left"
            Click="BtnZatwierdz_Click" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;
using System.Windows.Controls;

namespace PrzykladRadio
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void RbPoziom_Checked(object sender, RoutedEventArgs e)
            {
                // Identyfikujemy który RadioButton został zaznaczony
                if (sender is RadioButton rb && txtWybrany != null)
                {
                    txtWybrany.Text = $"Wybrany poziom: {rb.Content}";
                }
            }

            private void BtnZatwierdz_Click(object sender, RoutedEventArgs e)
            {
                // Sprawdzamy który RadioButton jest zaznaczony
                string poziom = "";
                if (rbLatwy.IsChecked == true) poziom = "Łatwy";
                else if (rbSredni.IsChecked == true) poziom = "Średni";
                else if (rbTrudny.IsChecked == true) poziom = "Trudny";

                MessageBox.Show($"Wybrałeś poziom: {poziom}", "Potwierdzenie");
            }
        }
    }
}

```

Kluczowe elementy:

- `IsChecked="True"` — ustawia domyślnie zaznaczony `RadioButton`
- `sender is RadioButton rb` — pattern matching do bezpiecznego rzutowania
- `rb.Content` — tekst wyświetlany przy `RadioButton`
- Wszystkie `RadioButton` bez `GroupName` w tym samym kontenerze tworzą automatycznie jedną grupę

10.4. Button

Gdy chcesz mieć kilka niezależnych grup `RadioButton` w jednym kontenerze, używasz właściwości `GroupName`. Każda różna wartość `GroupName` tworzy osobną grupę.

MainWindow.xaml:

```

<Window x:Class="WyborPrzesylki.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Wybór przesyłki" Height="450" Width="450">
<StackPanel Margin="20">
    <TextBlock Text="Zamówienie – wybierz opcje dostawy"
        FontSize="16" FontWeight="Bold" Margin="0,0,0,15"/>

    <!-- Grupa 1: Rodzaj przesyłki -->
    <TextBlock Text="Rodzaj przesyłki:" FontWeight="Bold" Margin="0,0,0,5"/>
    <RadioButton x:Name="rbKurier"
        Content="Kurier (1-2 dni robocze) – 15 zł"
        GroupName="Przesylka"
        IsChecked="True"
        Margin="0,4"
        Checked="RbPrzesylka_Checked"/>
    <RadioButton x:Name="rbPaczkomatEkonomy"
        Content="Paczkomat Economy (2-4 dni) – 9 zł"
        GroupName="Przesylka"
        Margin="0,4"
        Checked="RbPrzesylka_Checked"/>
    <RadioButton x:Name="rbPocztaPolska"
        Content="Poczta Polska (3-5 dni) – 7 zł"
        GroupName="Przesylka"
        Margin="0,4"
        Checked="RbPrzesylka_Checked"/>
    <RadioButton x:Name="rbOdbiorOsobisty"
        Content="Odbiór osobisty – 0 zł"
        GroupName="Przesylka"
        Margin="0,4"
        Checked="RbPrzesylka_Checked"/>

    <!-- Grupa 2: Czas dostawy -->
    <TextBlock Text="Preferowana pora dostawy:" FontWeight="Bold" Margin="0,15,0,5"/>
    <RadioButton x:Name="rbRano"
        Content="Rano (8:00 – 12:00)"
        GroupName="Pora"
        IsChecked="True"
        Margin="0,4"/>
    <RadioButton x:Name="rbPoPoludniu"
        Content="Po południu (12:00 – 18:00)"
        GroupName="Pora"
        Margin="0,4"/>
    <RadioButton x:Name="rbWieczorem"
        Content="Wieczorem (18:00 – 22:00)"
        GroupName="Pora"
        Margin="0,4"/>

    <Separator Margin="0,15"/>

    <TextBlock x:Name="txtPodsumowanie"
        Text="Wybrana opcja: Kurier – 15 zł"
        FontSize="13"
        Foreground="DarkGreen"
        TextWrapping="Wrap"/>

    <Button Content="Zamów"
        Margin="0,15,0,0"
        Height="35"
        Background="DarkGreen"
        Foreground="White"

```



```
Click="BtnZamow_Click"/>

</StackPanel>

</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;
using System.Windows.Controls;

namespace WyborPrzesylki
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void RbPrzesylka_Checked(object sender, RoutedEventArgs e)
            {
                if (txtPodsumowanie == null) return;

                if (rbKurier.IsChecked == true)
                    txtPodsumowanie.Text = "Wybrana opcja: Kurier – 15 zł";
                else if (rbPaczkomatEkonomy.IsChecked == true)
                    txtPodsumowanie.Text = "Wybrana opcja: Paczkomat Economy – 9 zł";
                else if (rbPocztaPolska.IsChecked == true)
                    txtPodsumowanie.Text = "Wybrana opcja: Poczta Polska – 7 zł";
                else if (rbOdbiorOsobisty.IsChecked == true)
                    txtPodsumowanie.Text = "Wybrana opcja: Odbiór osobisty – 0 zł";
            }

            private void BtnZamow_Click(object sender, RoutedEventArgs e)
            {
                // Odczytujemy opcje z obu grup niezależnie
                string przesyłka = "";
                decimal cena = 0;

                if (rbKurier.IsChecked == true) { przesyłka = "Kurier"; cena = 15; }
                else if (rbPaczkomatEkonomy.IsChecked == true) { przesyłka = "Paczkomat Economy"; cena = 9; }
                else if (rbPocztaPolska.IsChecked == true) { przesyłka = "Poczta Polska"; cena = 7; }
                else if (rbOdbiorOsobisty.IsChecked == true) { przesyłka = "Odbiór osobisty"; cena = 0; }

                string pora = "";
                if (rbRano.IsChecked == true) pora = "Rano (8:00-12:00)";
                else if (rbPoPoludniu.IsChecked == true) pora = "Po południu (12:00-18:00)";
                else if (rbWieczorem.IsChecked == true) pora = "Wieczorem (18:00-22:00)";

                MessageBox.Show(
                    $"Zamówienie złożone!\n\n" +
                    $"Rodzaj przesyłki: {przesyłka}\n" +
                    $"Koszt dostawy: {cena} zł\n" +
                    $"Preferowana pora: {pora}",
                    "Potwierdzenie zamówienia",
                    MessageBoxButton.OK,
                    MessageBoxImage.Information);
            }
        }
    }
}
```

Kluczowe elementy:

- `GroupName="Przesyłka"` — wszystkie `RadioButton` z tą samą wartością tworzą jedną grupę
- Dwie osobne grupy — `GroupName="Przesyłka"` i `GroupName="Pora"` działają niezależnie
- Bez `GroupName` wszystkie `RadioButton` w tym samym kontenerze tworzą jedną grupę

10.5. ComboBox

`ComboBox` to lista rozwijana, z której użytkownik wybiera jedną pozycję. Możesz wypełniać ją zarówno w XAML jak i dynamicznie w C#.

MainWindow.xaml:

```
<Window x:Class="PrzykladComboBox.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ComboBox - przykład" Height="400" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Wybierz kraj (pozycje w XAML):" FontWeight="Bold" Margin="0,0,0,5"/>
        <ComboBox x:Name="cmbKraj"
                SelectedIndex="0"
                SelectionChanged="cmbKraj_SelectionChanged">
            <ComboBoxItem Content="Polska"/>
            <ComboBoxItem Content="Niemcy"/>
            <ComboBoxItem Content="Francja"/>
            <ComboBoxItem Content="Włochy"/>
            <ComboBoxItem Content="Hiszpania"/>
        </ComboBox>

        <TextBlock Text="Wybierz miasto (ItemsSource z C#):"
                FontWeight="Bold" Margin="0,15,0,5"/>
        <ComboBox x:Name="cmbMiasto"
                SelectionChanged="cmbMiasto_SelectionChanged"
                Width="250"
                HorizontalAlignment="Left"/>

        <TextBlock Text="Wybierz rok urodzenia:"
                FontWeight="Bold" Margin="0,15,0,5"/>
        <ComboBox x:Name="cmbRok"
                Width="150"
                HorizontalAlignment="Left"
                SelectionChanged="cmbRok_SelectionChanged"/>

        <TextBlock x:Name="txtWynik"
                Margin="0,20,0,0"
                FontSize="13"
                Foreground="DarkBlue"
                TextWrapping="Wrap"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Collections.Generic;
using System.Windows;
using System.Windows.Controls;

namespace PrzykladComboBox
{
    public partial class MainWindow : Window
    {
        // ...
    }
}
```

```
public MainWindow()
{
    InitializeComponent();
    WypełnijMiasta();
    WypełnijLata();
}

private void WypełnijMiasta()
{
    // Wypełniamy ComboBox z poziomu C# używając listy stringów
    List<string> miasta = new List<string>
    {
        "Warszawa", "Kraków", "Gdańsk", "Wrocław",
        "Poznań", "Łódź", "Szczecin", "Katowice"
    };

    cmbMiasto.ItemsSource = miasta;
    cmbMiasto.SelectedIndex = 0; // Domyślnie pierwsza pozycja
}

private void WypełnijLata()
{
    // Programowo generujemy listę lat
    List<int> lata = new List<int>();
    for (int rok = 1950; rok ≤ 2010; rok++)
        lata.Add(rok);

    cmbRok.ItemsSource = lata;
    cmbRok.SelectedValue = 1990; // Ustawiamy wartość domyślną
}

private void cmbKraj_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (cmbKraj.SelectedItem is ComboBoxItem item)
    {
        AktualizujWynik();
    }
}

private void cmbMiasto_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    AktualizujWynik();
}

private void cmbRok_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    AktualizujWynik();
}

private void AktualizujWynik()
{
    // Bezpieczne odczytanie wybranych wartości
    string kraj = (cmbKraj.SelectedItem as ComboBoxItem)?.Content?.ToString() ?? "-";
    string miasto = cmbMiasto.SelectedItem?.ToString() ?? "-";
    string rok = cmbRok.SelectedItem?.ToString() ?? "-";

    if (txtWynik ≠ null)
        txtWynik.Text = $"Kraj: {kraj}\nMiasto: {miasto}\nRok: {rok}";
}
}
```

Kluczowe elementy:

- `SelectedIndex="0"` — ustawia domyślnie pierwszą pozycję (indeks od 0)
- `ItemsSource` — przypisujesz kolekcję C# do wyświetlenia
- `SelectedItem` — aktualnie wybrany obiekt (wymaga rzutowania)
- `SelectedValue` — pozwala ustawić domyślną wartość spośród elementów `ItemsSource`
- `as ComboBoxItem` — rzutowanie gdy pozycje dodane w XAML jako `ComboBoxItem`

10.6. List

`ListBox` wyświetla listę pozycji, z których użytkownik może wybrać jedną lub wiele naraz (jeśli ustawisz `SelectionMode`).

MainWindow.xaml:

```
<Window x:Class="PrzykladListBox.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ListBox - przykład" Height="450" Width="500">
    <Grid Margin="15">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>

        <!-- Lewa kolumna: pojedynczy wybór -->
        <TextBlock Grid.Row="0" Grid.Column="0"
            Text="Pojedynczy wybór:"
            FontWeight="Bold" Margin="0,0,0,5" />
        <ListBox Grid.Row="1" Grid.Column="0"
            x:Name="lstJedna"
            Margin="0,0,8,0"
            SelectionMode="Single"
            SelectionChanged="lstJedna_SelectionChanged" />

        <!-- Prawa kolumna: wielokrotny wybór -->
        <TextBlock Grid.Row="0" Grid.Column="1"
            Text="Wielokrotny wybór:"
            FontWeight="Bold" Margin="0,0,0,5" />
        <ListBox Grid.Row="1" Grid.Column="1"
            x:Name="lstWiele"
            SelectionMode="Multiple"
            Margin="8,0,0,0" />

        <!-- Wynik na dole -->
        <StackPanel Grid.Row="2" Grid.Column="0" Grid.ColumnSpan="2" Margin="0,10,0,0">
            <Button Content="Pokaż wybrane"
                Click="BtnPokażWybrane_Click"
                Margin="0,0,0,8" />
            <TextBlock x:Name="txtWybraneWiele"
                TextWrapping="Wrap"
                Foreground="DarkBlue" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```

using System.Collections.Generic;
using System.Text;
using System.Windows;
using System.Windows.Controls;

namespace PrzykladListBox
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            WypełnijListy();
        }

        private void WypełnijListy()
        {
            List<string> jezyki = new List<string>
            {
                "C#", "Java", "Python", "JavaScript",
                "TypeScript", "Go", "Rust", "Kotlin", "Swift", "PHP"
            };

            // Obie listy korzystają z tej samej kolekcji
            lstJedna.ItemsSource = jezyki;
            lstWiele.ItemsSource = new List<string>(jezyki);
        }

        private void lstJedna_SelectionChanged(object sender, SelectionChangedEventArgs e)
        {
            // Pojedynczy wybór – SelectedItem to string
            string wybrany = lstJedna.SelectedItem?.ToString() ?? "nic";
            txtWybraneWiele.Text = $"Wybrany język: {wybrany}";
        }

        private void BtnPokazWybrane_Click(object sender, RoutedEventArgs e)
        {
            // Wielokrotny wybór – SelectedItems to kolekcja
            if (lstWiele.SelectedItems.Count == 0)
            {
                txtWybraneWiele.Text = "Nie wybrano żadnego języka.";
                return;
            }

            StringBuilder sb = new StringBuilder("Wybrane języki:\n");
            foreach (var item in lstWiele.SelectedItems)
            {
                sb.AppendLine($"  • {item}");
            }

            txtWybraneWiele.Text = sb.ToString();
        }
    }
}

```

Kluczowe elementy:

- `SelectionMode="Single"` — tylko jeden element naraz (domyślne)
- `SelectionMode="Multiple"` — wiele elementów przez kliknięcie z Ctrl/Shift
- `SelectedItem` — jeden wybrany obiekt
- `SelectedItems` — kolekcja wszystkich wybranych obiektów (dla Multiple)

- `foreach` po `SelectedItems` — iteracja przez wielokrotne zaznaczenie

10.7. List

`DataGrid` to zaawansowana siatka danych, idealna do wyświetlania list obiektów. Może automatycznie generować kolumny na podstawie właściwości klasy.

MainWindow.xaml:

```
<Window x:Class="PrzykladDataGrid.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="DataGrid - lista uczniów" Height="450" Width="650">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <TextBlock Grid.Row="0"
            Text="Lista uczniów"
            FontSize="16" FontWeight="Bold" Margin="0,0,0,10"/>

        <!-- AutoGenerateColumns=False pozwala ręcznie zdefiniować kolumny -->
        <DataGrid Grid.Row="1"
            x:Name="dgUczniowie"
            AutoGenerateColumns="False"
            IsReadOnly="True"
            AlternatingRowBackground="AliceBlue"
            SelectionChanged="dgUczniowie_SelectionChanged">
            <DataGrid.Columns>
                <DataGridTextColumn Header="ID"
                    Binding="{Binding Id}"
                    Width="50"/>

                <DataGridTextColumn Header="Imię i nazwisko"
                    Binding="{Binding ImieNazwisko}"
                    Width="200"/>

                <DataGridTextColumn Header="Klasa"
                    Binding="{Binding Klasa}"
                    Width="80"/>

                <DataGridTextColumn Header="Średnia"
                    Binding="{Binding Srednia, StringFormat=F2}"
                    Width="80"/>

                <DataGridCheckBoxColumn Header="Obecny"
                    Binding="{Binding Obecny}"
                    Width="70"/>
            </DataGrid.Columns>
        </DataGrid>

        <TextBlock Grid.Row="2"
            x:Name="txtWybranyUczen"
            Margin="0,10,0,0"
            FontSize="12"
            Foreground="DarkBlue"/>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System.Collections.Generic;
using System.Windows;
```

```

using System.Windows.Controls;

namespace PrzykladDataGrid
{
    // Model danych – klasa reprezentująca ucznia
    public class Uczeń
    {
        public int Id { get; set; }
        public string ImieNazwisko { get; set; }
        public string Klasa { get; set; }
        public double Srednia { get; set; }
        public bool Obecny { get; set; }
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            WczytajUczniow();
        }

        private void WczytajUczniow()
        {
            // Tworzymy przykładową listę uczniów
            List<Uczeń> uczniowie = new List<Uczeń>
            {
                new Uczeń { Id=1, ImieNazwisko="Anna Kowalska", Klasa="3A", Srednia=4.8, Obecny=true },
                new Uczeń { Id=2, ImieNazwisko="Piotr Nowak", Klasa="3A", Srednia=3.5, Obecny=true },
                new Uczeń { Id=3, ImieNazwisko="Maria Wiśniewska", Klasa="3B", Srednia=5.0, Obecny=false },
                new Uczeń { Id=4, ImieNazwisko="Tomasz Wójcik", Klasa="3B", Srednia=2.8, Obecny=true },
                new Uczeń { Id=5, ImieNazwisko="Katarzyna Zając", Klasa="3A", Srednia=4.2, Obecny=true },
            };

            dgUczniowie.ItemsSource = uczniowie;
        }

        private void dgUczniowie_SelectionChanged(object sender, SelectionChangedEventArgs e)
        {
            // Odczytujemy wybrany wiersz
            if (dgUczniowie.SelectedItem is Uczeń uczen)
            {
                txtWybranyUczen.Text =
                    $"Wybrany: {uczen.ImieNazwisko} | Klasa: {uczen.Klasa} | Średnia: {uczen.Srednia:F2}";
            }
        }
    }
}

```

Kluczowe elementy:

- `AutoGenerateColumns="False"` — wyłącza automatyczne kolumny, definiujesz je ręcznie
- `Binding="{Binding NazwaWlasciosci}"` — kolumna wiąże się z właściwością obiektu
- `StringFormat=F2` — formatuje liczbę do 2 miejsc po przecinku
- `DataGridCheckBoxColumn` — specjalna kolumna dla wartości bool
- `AlternatingRowBackground` — naprzemienne kolory wierszy
- `IsReadOnly="True"` — zapobiega edycji danych przez użytkownika

11. Suwaki i wartości liczbowe

Slider to kontrolka pozwalająca użytkownikowi wybierać wartość liczbową przez przeciąganie uchwyty. Jest idealna do regulacji głośności, jasności, a w WPF bardzo często używana do wyboru składowych koloru RGB.

11.1. Slider

Suwak ma kilka ważnych właściwości, które kontrolują jego zachowanie:

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Minimum	Minimalna wartość	0
Maximum	Maksymalna wartość	255
Value	Aktualna wartość	128
TickFrequency	Co ile wyświetlać znaczniki	16
IsSnapToTickEnabled	Skaczemy do znaczników	True
SmallChange	Zmiana klawiszami strzałek	1
LargeChange	Zmiana klawiszami Page Up/Down	10
TickPlacement	Położenie znaczników	BottomRight

MainWindow.xaml:

```
<Window x:Class="PrzykladSlider.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Slider - właściwości" Height="350" Width="500">
    <StackPanel Margin="20">
        <TextBlock Text="Suwak podstawowy (0-100):" FontWeight="Bold" Margin="0,0,0,5"/>
        <Slider x:Name="sldPodstawowy"
            Minimum="0" Maximum="100" Value="50"
            SmallChange="1" LargeChange="10"/>

        <TextBlock Text="Suwak ze znacznikami:" FontWeight="Bold" Margin="0,15,0,5"/>
        <Slider x:Name="sldZnaczniki"
            Minimum="0" Maximum="100"
            Value="25"
            TickFrequency="10"
            TickPlacement="BottomRight"
            IsSnapToTickEnabled="True"
            SmallChange="10"/>

        <TextBlock Text="Suwak pionowy:" FontWeight="Bold" Margin="0,15,0,5"/>
        <Slider x:Name="sldPionowy"
            Minimum="0" Maximum="10"
            Value="5"
            Orientation="Vertical"
            Height="100"
            HorizontalAlignment="Left"
            TickFrequency="1"
            TickPlacement="BottomRight"
            IsSnapToTickEnabled="True"/>

        <Button Content="Pokaż wartości" Margin="0,20,0,0"
            Click="BtnPokaz_Click"/>

        <TextBlock x:Name="txtWartosci" Margin="0,10,0,0" Foreground="DarkBlue"/>
    </StackPanel>
</Window>
```


MainWindow.xaml.cs:

```
using System.Windows;

namespace PrzykładSlider
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void BtnPokaz_Click(object sender, RoutedEventArgs e)
            {
                txtWartosci.Text =
                    $"Podstawowy: {sldPodstawowy.Value:F1}\n" +
                    $"Ze znacznikami: {sldZnaczniki.Value}\n" +
                    $"Pionowy: {sldPionowy.Value}";
            }
        }
    }
}
```

11.2. Zdarzenie ValueChanged

Zdarzenie `ValueChanged` wywoływane jest za każdym razem gdy wartość suwaka się zmienia. Parametr `e.NewValue` zawiera nową wartość, a `e.OldValue` poprzednią.

MainWindow.xaml:

```
<Window x:Class="SliderValueChanged.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ValueChanged - przykład" Height="250" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Ustaw głośność:" FontWeight="Bold" Margin="0,0,0,8"/>
        <Slider x:Name="sldGlosnosc"
                Minimum="0" Maximum="100"
                Value="50"
                TickFrequency="10"
                TickPlacement="BottomRight"
                ValueChanged="sldGlosnosc_ValueChanged"/>

        <TextBlock x:Name="txtGlosnosc"
                Text="Głośność: 50%"
                FontSize="14"
                Margin="0,10,0,0"
                HorizontalAlignment="Center"/>

        <TextBlock x:Name="txtStan"
                Margin="0,5,0,0"
                FontSize="12"
                Foreground="Gray"
                HorizontalAlignment="Center"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```

using System.Windows;
using System.Windows.Controls.Primitives;

namespace SliderValueChanged
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void sldGlosnosc_ValueChanged(object sender,
                RoutedPropertyChangedEventArgs<double> e)
            {
                // e.NewValue – nowa wartość
                // e.OldValue – poprzednia wartość
                int nowaWartosc = (int)e.NewValue;

                if (txtGlosnosc == null) return;
                txtGlosnosc.Text = $"Głośność: {nowaWartosc}%";

                // Dynamicznie zmieniamy stan
                if (nowaWartosc == 0)
                    txtStan.Text = "🔇 Wyciszony";
                else if (nowaWartosc ≤ 30)
                    txtStan.Text = "🔊 Cicho";
                else if (nowaWartosc ≤ 70)
                    txtStan.Text = "🔊 Normalnie";
                else
                    txtStan.Text = "🔊 Głośno";
            }
        }
    }
}

```

Kluczowe elementy:

- `RoutedPropertyChangedEventArgs<double>` — argument zdarzenia dla Slider
- `e.NewValue` — nowa wartość suwaka (double)
- `e.OldValue` — poprzednia wartość suwaka
- `(int)e.NewValue` — rzutowanie na int gdy potrzebujemy liczby całkowitej
- Sprawdź czy elementy UI nie są `null` — zdarzenie może być wywołane przed `InitializeComponent`

11.3. TextBlock

Standardowy wzorzec to umieszczenie `Slider` i `TextBlock` obok siebie w `StackPanel` z `Orientatation="Horizontal"`.

MainWindow.xaml:

```

<Window x:Class="SliderZetykieta.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Suwak z etykietą" Height="300" Width="500">
    <StackPanel Margin="20">
        <TextBlock Text="Rozmiar czcionki:" FontWeight="Bold" Margin="0,0,0,8"/>
        <StackPanel Orientation="Horizontal" VerticalAlignment="Center">
            <Slider x:Name="sldRozmiar"
                    Minimum="8" Maximum="72"
                    Value="16"
                    Width="300"

```

```

        SmallChange="1"
        ValueChanged="sldRozmiar_ValueChanged"
        VerticalAlignment="Center" />
<TextBlock x:Name="lblRozmiar"
        Text="16 pt"
        Margin="12,0,0,0"
        FontSize="13"
        FontWeight="Bold"
        VerticalAlignment="Center"
        Foreground="DarkBlue"
        MinWidth="45" />
</StackPanel>

<Border BorderBrush="LightGray" BorderThickness="1"
        Padding="15" Margin="0,20,0,0" CornerRadius="4">
    <TextBlock x:Name="txtPodglad"
        Text="Podgląd tekstu w wybranym rozmiarze"
        TextWrapping="Wrap"
        FontSize="16" />
</Border>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace SlidenZetykieta
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void sldRozmiar_ValueChanged(object sender,
            RoutedPropertyChangedEventArgs<double> e)
        {
            if (lblRozmiar == null || txtPodglad == null) return;

            int rozmiar = (int)e.NewValue;
            lblRozmiar.Text = $"{rozmiar} pt";
            txtPodglad.FontSize = rozmiar;
        }
    }
}

```

11.4. Slider

Poniżej znajdziesz kompletną aplikację do wyboru koloru za pomocą trzech suwaków RGB. Jest to jeden z najczęściej spotykanych przykładów łączący **Slider**, **TextBlock**, **Rectangle** i manipulację kolorami.

MainWindow.xaml:

```

<Window x:Class="KoLoryRGB.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Mieszanie kolorów RGB" Height="520" Width="500">
    <StackPanel Margin="20">

```

```
<TextBlock Text="Mieszanie kolorów RGB"
           FontSize="18" FontWeight="Bold"
           HorizontalAlignment="Center"
           Margin="0,0,0,20"/>

<!-- Suwak R (czerwony) -->
<Grid Margin="0,0,0,10">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="20"/>
        <ColumnDefinition Width="*/>
        <ColumnDefinition Width="50"/>
    </Grid.ColumnDefinitions>
    <TextBlock Grid.Column="0" Text="R" FontWeight="Bold"
               Foreground="Red" VerticalAlignment="Center"/>
    <Slider Grid.Column="1" x:Name="sldR"
            Minimum="0" Maximum="255" Value="128"
            SmallChange="1" LargeChange="16"
            VerticalAlignment="Center" Margin="8,0"
            ValueChanged="sldKolor_ValueChanged"/>
    <TextBlock Grid.Column="2" x:Name="lblR"
               Text="128" FontFamily="Consolas"
               VerticalAlignment="Center"
               HorizontalAlignment="Right"/>
</Grid>

<!-- Suwak G (zielony) -->
<Grid Margin="0,0,0,10">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="20"/>
        <ColumnDefinition Width="*/>
        <ColumnDefinition Width="50"/>
    </Grid.ColumnDefinitions>
    <TextBlock Grid.Column="0" Text="G" FontWeight="Bold"
               Foreground="Green" VerticalAlignment="Center"/>
    <Slider Grid.Column="1" x:Name="sldG"
            Minimum="0" Maximum="255" Value="128"
            SmallChange="1" LargeChange="16"
            VerticalAlignment="Center" Margin="8,0"
            ValueChanged="sldKolor_ValueChanged"/>
    <TextBlock Grid.Column="2" x:Name="lblG"
               Text="128" FontFamily="Consolas"
               VerticalAlignment="Center"
               HorizontalAlignment="Right"/>
</Grid>

<!-- Suwak B (niebieski) -->
<Grid Margin="0,0,0,10">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="20"/>
        <ColumnDefinition Width="*/>
        <ColumnDefinition Width="50"/>
    </Grid.ColumnDefinitions>
    <TextBlock Grid.Column="0" Text="B" FontWeight="Bold"
               Foreground="Blue" VerticalAlignment="Center"/>
    <Slider Grid.Column="1" x:Name="sldB"
            Minimum="0" Maximum="255" Value="128"
            SmallChange="1" LargeChange="16"
            VerticalAlignment="Center" Margin="8,0"
            ValueChanged="sldKolor_ValueChanged"/>
    <TextBlock Grid.Column="2" x:Name="lblB"
               Text="128" FontFamily="Consolas"
               VerticalAlignment="Center"/>
```

```

                HorizontalAlignment="Right"/>

</Grid>

<!-- Kod HEX -->
<TextBlock x:Name="LblHex"
    Text="#808080"
    FontFamily="Consolas"
    FontSize="18"
    HorizontalAlignment="Center"
    Margin="0,10"/>

<!-- Podgląd bieżącego koloru -->
<TextBlock Text="Bieżący kolor:" FontWeight="Bold" Margin="0,10,0,5"/>
<Rectangle x:Name="rectBiezacy"
    Height="80"
    Fill="#808080"
    RadiusX="6" RadiusY="6"
    Stroke="Gray" StrokeThickness="1"/>

<!-- Przyciski -->
<StackPanel Orientation="Horizontal" Margin="0,12,0,0" HorizontalAlignment="Center">
    <Button Content="Zapisz kolor"
        Click="BtnZapisz_Click"
        Padding="16,6" Margin="0,0,10,0"
        Background="DarkGreen" Foreground="White"/>
    <Button Content="Losuj kolor"
        Click="BtnLosuj_Click"
        Padding="16,6"
        Background="DarkBlue" Foreground="White"/>
</StackPanel>

<!-- Zapisany kolor -->
<TextBlock Text="Zapisany kolor:" FontWeight="Bold" Margin="0,12,0,5"/>
<Rectangle x:Name="rectZapisany"
    Height="80"
    Fill="LightGray"
    RadiusX="6" RadiusY="6"
    Stroke="Gray" StrokeThickness="1"/>

</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Media;

namespace KoLoryRGB
{
    public partial class MainWindow : Window
    {
        private Random rnd = new Random();

        public MainWindow()
        {
            InitializeComponent();
            // Przy starcie ustawiamy kolor domyślny
            AktualizujKolor();
        }

        // Jedno zdarzenie obsługuje wszystkie trzy suwaki

```

```

private void sldKolor_ValueChanged(object sender,
    RoutedPropertyChangedEventArgs<double> e)
{
    AktualizujKolor();
}

private void AktualizujKolor()
{
    // Zabezpieczenie przed wywołaniem przed inicjalizacją
    if (lblR == null) return;

    // Pobieramy wartości suwaków jako bajty (0-255)
    byte r = (byte)sldR.Value;
    byte g = (byte)sldG.Value;
    byte b = (byte)sldB.Value;

    // Aktualizujemy etykiety numeryczne
    lblR.Text = r.ToString();
    lblG.Text = g.ToString();
    lblB.Text = b.ToString();

    // Tworzymy kolor i pędzel
    Color kolor = Color.FromRgb(r, g, b);
    SolidColorBrush pedzel = new SolidColorBrush(kolor);

    // Aktualizujemy podgląd koloru
    rectBiezacy.Fill = pedzel;

    // Wyświetlamy kod HEX
    lblHex.Text = $"#{r:X2}-{g:X2}-{b:X2}";

    // Dostosowujemy kolor tekstu HEX dla czytelności
    double jasnoc = (r * 0.299 + g * 0.587 + b * 0.114);
    lblHex.Foreground = jasnoc > 128
        ? new SolidColorBrush(Colors.Black)
        : new SolidColorBrush(Colors.White);
}

private void BtnZapisz_Click(object sender, RoutedEventArgs e)
{
    // Kopiujemy bieżący kolor do "zapisanego"
    rectZapisany.Fill = rectBiezacy.Fill;
    MessageBox.Show($"Zapisano kolor: {lblHex.Text}", "Zapisano");
}

private void BtnLosuj_Click(object sender, RoutedEventArgs e)
{
    // Losujemy wartości RGB i ustawiamy suwaki
    sldR.Value = rnd.Next(0, 256);
    sldG.Value = rnd.Next(0, 256);
    sldB.Value = rnd.Next(0, 256);

    // AktualizujKolor() zostanie wywołana automatycznie przez ValueChanged
}
}
}

```

Kluczowe elementy:

- Jeden handler dla trzech suwaków — wszystkie `Slider` wskazują na `sldKolor_ValueChanged`
- `(byte)sldR.Value` — rzutowanie double na byte (0-255)
- `Color.FromRgb(r, g, b)` — tworzy kolor z trzech składowych
- `new SolidColorBrush(kolor)` — tworzy pędzel jednokolorowy

- `rectBieżący.Fill = pedzel` — ustawia tło prostokąta
- `{r:X2}` — formatowanie liczby jako dwucyfrowy hex (np. `0A`, `FF`)

11.5. SolidColorBrush

Żeby ustawić kolor elementu WPF z poziomu C#, musisz:

1. Stworzyć obiekt `Color` za pomocą `Color.FromRgb(r, g, b)` lub `Color.FromArgb(a, r, g, b)`
2. Owinąć go w `SolidColorBrush`
3. Przypisać do właściwości `Fill` (kształty) lub `Background` (panele, przyciski)

```
using System.Windows.Media; // Color, SolidColorBrush, Colors, Brushes

// Tworzenie koloru z RGB
Color mojaBarwa = Color.FromRgb(255, 128, 0); // pomarańczowy

// Tworzenie koloru z ARGB (alpha = przezroczystość, 0-255)
Color poloprze = Color.FromArgb(128, 255, 0, 0); // półprzezroczysty czerwony

// Tworzenie pędzla
SolidColorBrush pedzel = new SolidColorBrush(mojaBarwa);

// Użycie gotowych kolorów
SolidColorBrush czerwony = new SolidColorBrush(Colors.Red);
SolidColorBrush niebieski = Brushes.Blue; // skrót

// Ustawianie tła panelu/okna
mojaStackPanel.Background = pedzel;

// Ustawianie wypełnienia kształtu
mojaRectangle.Fill = pedzel;

// Ustawianie tła przycisku
mojButton.Background = new SolidColorBrush(Color.FromRgb(0, 128, 64));
```

11.6. Rectangle

Ten wzorec jest bardzo przydatny: użytkownik manipuluje suwakami, a wybrany kolor można "zapamiętać". Poniżej uproszczony wariant skupiony na dwóch `Rectangle`.

MainWindow.xaml:

```
<Window x:Class="ZapisanyKolor.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Bieżący i zapisany kolor" Height="380" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Sterowanie kolorem" FontSize="16" FontWeight="Bold" Margin="0,0,15"/>

        <StackPanel Orientation="Horizontal" Margin="0,0,5">
            <TextBlock Text="R:" Width="20" VerticalAlignment="Center"/>
            <Slider x:Name="sldR" Minimum="0" Maximum="255" Value="200" Width="280"
                ValueChanged="Suwak_ValueChanged"/>
            <TextBlock x:Name="lblR" Text="200" Width="35" Margin="8,0,0" VerticalAlignment="Center"/>
        </StackPanel>

        <StackPanel Orientation="Horizontal" Margin="0,0,5">
            <TextBlock Text="G:" Width="20" VerticalAlignment="Center"/>
            <Slider x:Name="sldG" Minimum="0" Maximum="255" Value="100" Width="280"
                ValueChanged="Suwak_ValueChanged"/>
            <TextBlock x:Name="lblG" Text="100" Width="35" Margin="8,0,0" VerticalAlignment="Center"/>
        </StackPanel>
    </StackPanel>
</Window>
```

```

</StackPanel>
<StackPanel Orientation="Horizontal" Margin="0,0,0,15">
    <TextBlock Text="B:" Width="20" VerticalAlignment="Center"/>
    <Slider x:Name="sldB" Minimum="0" Maximum="255" Value="50" Width="280"
        ValueChanged="Suwak_ValueChanged"/>
    <TextBlock x:Name="lblB" Text="50" Width="35" Margin="8,0,0,0" VerticalAlignment="Center"/>
</StackPanel>

<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
    <StackPanel Grid.Column="0" Margin="0,0,5,0">
        <TextBlock Text="Bieżący:" FontWeight="Bold" Margin="0,0,0,4"/>
        <Rectangle x:Name="rectBiezacy" Height="80" RadiusX="5" RadiusY="5"
            Stroke="Gray" StrokeThickness="1"/>
    </StackPanel>
    <StackPanel Grid.Column="1" Margin="5,0,0,0">
        <TextBlock Text="Zapisany:" FontWeight="Bold" Margin="0,0,0,4"/>
        <Rectangle x:Name="rectZapisany" Height="80" Fill="LightGray"
            RadiusX="5" RadiusY="5" Stroke="Gray" StrokeThickness="1"/>
    </StackPanel>
</Grid>

<Button Content="Pobierz (zapisz bieżący)"
    Click="BtnPobierz_Click"
    Margin="0,15,0,0"
    Height="35"
    Background="SteelBlue" Foreground="White"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;
using System.Windows.Media;

namespace ZapisanyKolor
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            OdswiezKolor();
        }

        private void Suwak_ValueChanged(object sender,
            RoutedPropertyChangedEventArgs<double> e)
        {
            OdswiezKolor();
        }

        private void OdswiezKolor()
        {
            if (lblR == null) return;
            byte r = (byte)sldR.Value;
            byte g = (byte)sldG.Value;
            byte b = (byte)sldB.Value;
            lblR.Text = r.ToString();
        }
    }
}

```



```
        lblG.Text = g.ToString();
        lblB.Text = b.ToString();
        rectBiezacy.Fill = new SolidColorBrush(Color.FromRgb(r, g, b));
    }

    private void BtnPobierz_Click(object sender, RoutedEventArgs e)
    {
        // Kopiujemy pędzel z bieżącego do zapisanego
        rectZapisany.Fill = rectBiezacy.Fill;
    }
}
```

12. Obrazy, grafiki i zasoby

Kontrolka `Image` wyświetla grafikę. W WPF obrazy możesz ładować z zasobów projektu (skompilowanych do EXE) lub ze ścieżek na dysku. Każdy z tych scenariuszy omówimy osobno.

12.1. Kontrolka Image

Kontrolka `Image` wyświetla grafikę. Kluczowe właściwości:

WŁAŚCIWOŚĆ	OPIS
<code>Source</code>	Źródło obrazu (plik, zasób, URL)
<code>Stretch="None"</code>	Obraz wyświetlany w oryginalnym rozmiarze
<code>Stretch="Fill"</code>	Rozciągnięty do wypełnienia całego obszaru
<code>Stretch="Uniform"</code>	Skalowany proporcjonalnie, bez przycinania
<code>Stretch="UniformToFill"</code>	Skalowany proporcjonalnie, wypełnia cały obszar
<code>Width</code> / <code>Height</code>	Rozmiar kontrolki Image

MainWindow.xaml:

```
<Window x:Class="PrzykladImage.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Stretch obrazu" Height="400" Width="600">
    <Grid Margin="10">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <TextBlock Grid.Row="0" Grid.Column="0" Text="None" HorizontalAlignment="Center" FontWeight="Bold" />
        <TextBlock Grid.Row="0" Grid.Column="1" Text="Fill" HorizontalAlignment="Center" FontWeight="Bold" />
        <TextBlock Grid.Row="0" Grid.Column="2" Text="Uniform" HorizontalAlignment="Center" FontWeight="Bold" />
        <TextBlock Grid.Row="0" Grid.Column="3" Text="UniformToFill" HorizontalAlignment="Center" FontWeight="Bold" />

        <!-- Wszystkie cztery tryby Stretch -->
        <Border Grid.Row="1" Grid.Column="0" BorderBrush="Gray" BorderThickness="1" Margin="2">
            <Image Source="pack://application:,,,/Images/foto.jpg" Stretch="None" />
        </Border>
    </Grid>
</Window>
```

```

</Border>
<Border Grid.Row="1" Grid.Column="1" BorderBrush="Gray" BorderThickness="1" Margin="2">
    <Image Source="pack://application:,,,/Images/foto.jpg" Stretch="Fill"/>
</Border>
<Border Grid.Row="1" Grid.Column="2" BorderBrush="Gray" BorderThickness="1" Margin="2">
    <Image Source="pack://application:,,,/Images/foto.jpg" Stretch="Uniform"/>
</Border>
<Border Grid.Row="1" Grid.Column="3" BorderBrush="Gray" BorderThickness="1" Margin="2">
    <Image Source="pack://application:,,,/Images/foto.jpg" Stretch="UniformToFill"/>
</Border>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace PrzykładImage
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

12.2. Build Action: Resource

Żeby obraz był częścią aplikacji (wkompilowany do pliku EXE), musisz go poprawnie dodać do projektu:

1. W **Solution Explorer** kliknij prawym przyciskiem na projekt → **Add** → **New Folder** → nazwij go **Images**
2. Kliknij prawym na folder **Images** → **Add** → **Existing Item** → wybierz plik obrazu
3. Kliknij na dodany obraz w Solution Explorer
4. W oknie **Properties** (F4) ustaw:
 - **Build Action:** **Resource**
 - **Copy to Output Directory:** **Do not copy** (bo obraz jest wkompilowany)

Po tych krokach obraz jest dostępny pod adresem:

```
pack://application:,,,/Images/nazwaPliku.jpg
```

Jeśli chcesz ładować obraz z dysku w runtime (nie z zasobów), ustaw:

- **Build Action:** **Content**
- **Copy to Output Directory:** **Copy if newer**

12.3. Wyświetlanie obrazu w XAML — BitmapImage, pack://application

Gdy obraz jest zasobem projektu (Build Action: Resource), wyświetlasz go w XAML tak:

MainWindow.xaml:

```

<Window x:Class="WyświetlanieObrazow.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Obrazy z zasobów" Height="350" Width="400">
    <StackPanel Margin="15">

```

```

<TextBlock Text="Obraz z zasobu projektu:" FontWeight="Bold" Margin="0,0,0,8"/>

<!-- Sposób 1: Uproszczony (WPF automatycznie tworzy BitmapImage) -->
<Image Source="Images/Logo.png"
        Width="200" Height="150"
        Stretch="Uniform"
        HorizontalAlignment="Center"/>

<!-- Sposób 2: Jawny zapis z pack URI -->
<Image Width="200" Height="150" Stretch="Uniform"
        HorizontalAlignment="Center" Margin="0,10,0,0">
    <Image.Source>
        <BitmapImage UriSource="pack://application:,,,/Images/Logo.png"/>
    </Image.Source>
</Image>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace WyświetlanieObrazow
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

12.4. Dynamiczne ładowanie obrazu z obsługą błędów — kompletny przykład

Poniższy przykład pokazuje jak dynamicznie zmieniać obraz w trakcie działania aplikacji, budować ścieżkę do pliku na podstawie danych użytkownika oraz obsługiwać sytuację, gdy plik nie istnieje.

MainWindow.xaml:

```

<Window x:Class="LadowanieObrazow.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Dynamiczne ładowanie obrazów" Height="500" Width="480">
    <StackPanel Margin="20">
        <TextBlock Text="System weryfikacji dokumentów"
                FontSize="16" FontWeight="Bold" Margin="0,0,0,15"/>

        <TextBlock Text="Wprowadź numer dokumentu (np. AB123456):"
                FontWeight="Bold" Margin="0,0,0,5"/>
        <StackPanel Orientation="Horizontal" Margin="0,0,0,5">
            <TextBox x:Name="txtNumerDok"
                    Width="200"
                    Padding="5,3"
                    FontSize="14"
                    LostFocus="txtNumerDok_LostFocus"/>
            <Button Content="Wczytaj zdjęcie"
                    Margin="10,0,0,0"
                    Padding="10,3"
                    Click="BtnWczytaj_Click"/>
        </StackPanel>
    </StackPanel>

```

```

</StackPanel>

<!-- Informacja o ścieżce -->
<TextBlock x:Name="txtSciezka"
    FontSize="10"
    Foreground="Gray"
    Margin="0,3,0,10"
    TextWrapping="Wrap" />

<!-- Podgląd zdjęcia -->
<Border BorderBrush="LightGray" BorderThickness="1"
    CornerRadius="4" Padding="5" Height="220">
    <Grid>
        <!-- Obraz dokumentu -->
        <Image x:Name="imgDokument"
            Stretch="Uniform" />
        <!-- Komunikat gdy brak obrazu -->
        <TextBlock x:Name="txtBrakObr"
            Text="🖼️ Brak zdjęcia"
            FontSize="18"
            Foreground="LightGray"
            HorizontalAlignment="Center"
            VerticalAlignment="Center" />
    </Grid>
</Border>

<!-- Status -->
<TextBlock x:Name="txtStatus"
    Margin="0,10,0,0"
    TextWrapping="Wrap"
    FontSize="12" />

<!-- Przyciski testowe -->
<WrapPanel Margin="0,10,0,0">
    <Button Content="Istniejący plik" Click="BtnTestOk_Click"
        Margin="0,0,5,0" Padding="8,4"
        Background="DarkGreen" Foreground="White" />
    <Button Content="Nieistniejący plik" Click="BtnTestBrak_Click"
        Margin="0,0,5,0" Padding="8,4"
        Background="DarkRed" Foreground="White" />
    <Button Content="Wyczyść" Click="BtnWyczysc_Click"
        Padding="8,4" />
</WrapPanel>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.IO;
using System.Windows;
using System.Windows.Media.Imaging;

namespace LadowanieObrazow
{
    public partial class MainWindow : Window
    {
        // Ścieżka do folderu ze zdjęciami
        // W rzeczywistej aplikacji może to być AppDomain.CurrentDomain.BaseDirectory
        private string folderZdjec = Path.Combine(
            AppDomain.CurrentDomain.BaseDirectory, "Images");
    }
}

```

```
public MainWindow()
{
    InitializeComponent();
}

private void txtNumerDok_LostFocus(object sender, RoutedEventArgs e)
{
    // Zdarzenie opuszczenia pola – automatycznie ładujemy obraz
    if (!string.IsNullOrWhiteSpace(txtNumerDok.Text))
    {
        WczytajObrazDlaNumeruDokumentu(txtNumerDok.Text.Trim());
    }
}

private void BtnWczytaj_Click(object sender, RoutedEventArgs e)
{
    string numer = txtNumerDok.Text.Trim();
    if (string.IsNullOrEmpty(numer))
    {
        PokażBłąd("Podaj numer dokumentu.");
        return;
    }
    WczytajObrazDlaNumeruDokumentu(numer);
}

private void WczytajObrazDlaNumeruDokumentu(string numerDokumentu)
{
    // Budowanie nazwy pliku na podstawie numeru dokumentu
    // Np. "AB123456" → "AB123456.jpg"
    string nazwaPliku = $"{numerDokumentu}.jpg";
    string pełnaŚciezka = Path.Combine(folderZdjec, nazwaPliku);

    txtSciezka.Text = $"Ścieżka: {pełnaŚciezka}";

    // Sposób 1: Sprawdzamy czy plik istnieje przed załadowaniem
    if (!File.Exists(pełnaŚciezka))
    {
        PokażBrakObrazka($"Brak zdjęcia dla dokumentu: {numerDokumentu}");
        return;
    }

    // Sposób 2: Zabezpieczamy ładowanie przez try/catch
    ZaładujObrazZDysku(pełnaŚciezka);
}

private void ZaładujObrazZDysku(string sciezka)
{
    try
    {
        // Tworzymy BitmapImage z pliku na dysku
        BitmapImage bitmapImage = new BitmapImage();

        // Używamy BeginInit/EndInit dla poprawnej inicjalizacji
        bitmapImage.BeginInit();
        bitmapImage.UriSource = new Uri(sciezka, UriKind.Absolute);

        // CacheOption.OnLoad – ładuje cały obraz do pamięci,
        // dzięki temu możemy potem usunąć plik z dysku
        bitmapImage.CacheOption = BitmapCacheOption.OnLoad;
        bitmapImage.EndInit();
    }
}
```

```

        // Przypisujemy do kontrolki Image
        imgDokument.Source = bitmapImage;
        txtBrakObr.Visibility = Visibility.Collapsed;
        UstawStatus($"✓ Załadowano: {Path.GetFileName(sciezka)}", "DarkGreen");
    }
    catch (Exception ex)
    {
        // Obsługa błędu ładowania (uszkodzony plik, brak dostępu itp.)
        PokażBrakObrazka($"Błąd ładowania: {ex.Message}");
    }
}

private void PokażBrakObrazka(string komunikat)
{
    imgDokument.Source = null;
    txtBrakObr.Visibility = Visibility.Visible;
    UstawStatus($"✗ {komunikat}", "DarkRed");
}

private void PokażBłąd(string komunikat)
{
    UstawStatus($"⚠ {komunikat}", "DarkOrange");
}

private void UstawStatus(string tekst, string kolor)
{
    txtStatus.Text = tekst;
    txtStatus.Foreground = new System.Windows.Media.SolidColorBrush(
        (System.Windows.Media.Color)System.Windows.Media.ColorConverter.ConvertFromString(kolor));
}

// Przyciski testowe
private void BtnTestOk_Click(object sender, RoutedEventArgs e)
{
    txtNumerDok.Text = "AB123456";
    // Symulujemy istniejący plik - w teście użyj własnej ścieżki
    WczytajObrazDlaNumeruDokumentu("AB123456");
}

private void BtnTestBrak_Click(object sender, RoutedEventArgs e)
{
    txtNumerDok.Text = "XY999999";
    WczytajObrazDlaNumeruDokumentu("XY999999");
}

private void BtnWyczysc_Click(object sender, RoutedEventArgs e)
{
    txtNumerDok.Text = "";
    imgDokument.Source = null;
    txtBrakObr.Visibility = Visibility.Visible;
    txtStatus.Text = "";
    txtSciezka.Text = "";
}
}
}

```

Kluczowe elementy:

- `File.Exists(pełnaSciezka)` — sprawdzamy przed próbą ładowania
- `try/catch` — zabezpieczamy przed uszkodzonym plikiem lub brakiem dostępu
- `BitmapImage.BeginInit() / EndInit()` — poprawny sposób inicjalizacji
- `UriKind.Absolute` — pełna ścieżka dyskowa to URI absolutne

- `BitmapCacheOption.OnLoad` — ładuje cały obraz do pamięci od razu
- `imgDokument.Source = null` — czyści kontrolkę Image
- `Path.Combine()` — bezpieczne łączenie ścieżek (bez ręcznego dodawania `\`)

12.5. Button

Bardzo często używa się obrazów do wizualnego potwierdzenia wyboru. Poniższy przykład zmienia obraz w zależności od wybranego `RadioButton`.

MainWindow.xaml:

```
<Window x:Class="ObrazyStanu.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Rodzaj dokumentu" Height="380" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Wybierz rodzaj dokumentu:"
                   FontWeight="Bold" Margin="0,0,0,10" />

        <RadioButton x:Name="rbDowod"
                     Content="Dowód osobisty"
                     GroupName="Dokument"
                     IsChecked="True"
                     Margin="0,4"
                     Checked="RbDokument_Checked" />

        <RadioButton x:Name="rbPaszport"
                     Content="Paszport"
                     GroupName="Dokument"
                     Margin="0,4"
                     Checked="RbDokument_Checked" />

        <RadioButton x:Name="rbPrawoJazdy"
                     Content="Prawo jazdy"
                     GroupName="Dokument"
                     Margin="0,4"
                     Checked="RbDokument_Checked" />

        <Border BorderBrush="LightGray" BorderThickness="1"
                CornerRadius="5" Margin="0,15,0,0" Padding="10">
            <StackPanel>
                <TextBlock x:Name="txtNazwaDok"
                           Text="Dowód osobisty"
                           FontSize="15" FontWeight="Bold"
                           HorizontalAlignment="Center"
                           Margin="0,0,0,8" />

                <Image x:Name="imgDokument"
                       Height="180"
                       Stretch="Uniform"
                       HorizontalAlignment="Center" />
            </StackPanel>
        </Border>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;
using System.Windows.Media.Imaging;

namespace ObrazyStanu
{
    public partial class MainWindow : Window
```

```

{
    public MainWindow()
    {
        InitializeComponent();
        // Ustawiamy obraz dla domyślnie wybranej opcji
        UstawObraz("dowod_osobisty.png", "Dowód osobisty");
    }

    private void RbDokument_Checked(object sender, RoutedEventArgs e)
    {
        if (imgDokument == null) return;

        if (rbDowod.IsChecked == true)
            UstawObraz("dowod_osobisty.png", "Dowód osobisty");
        else if (rbPaszport.IsChecked == true)
            UstawObraz("paszport.png", "Paszport");
        else if (rbPrawoJazdy.IsChecked == true)
            UstawObraz("prawo_jazdy.png", "Prawo jazdy");
    }

    private void UstawObraz(string nazwaPliku, string opis)
    {
        txtNazwaDok.Text = opis;
        try
        {
            // Próbujemy załadować obraz z zasobów projektu
            Uri uri = new Uri($"pack://application:,,,/Images/{nazwaPliku}");
            imgDokument.Source = new BitmapImage(uri);
        }
        catch
        {
            // Jeśli plik nie istnieje, czyścimy obraz
            imgDokument.Source = null;
        }
    }
}
}

```

13. Kolory, kształty i elementy wizualne

WPF oferuje bogaty zestaw kontrolki do budowania interfejsu wizualnego. `Rectangle`, `Border`, `ProgressBar` i inne elementy pozwalają budować estetyczne aplikacje bez osobnych bibliotek graficznych.

13.1. Rectangle

`Rectangle` to kształt prostokąta. Najczęściej używasz go jako wizualny podgląd wartości lub koloru.

MainWindow.xaml:

```

<Window x:Class="PrzykładRectangle.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Rectangle - przykłady" Height="350" Width="500">
    <StackPanel Margin="20">
        <TextBlock Text="Różne style prostokąta:" FontWeight="Bold" Margin="0,0,0,10"/>

        <!-- Prosty wypełniony prostokąt -->
        <Rectangle Fill="SteelBlue" Width="200" Height="50"
            HorizontalAlignment="Left" Margin="0,5"/>
    </StackPanel>
</Window>

```



```

<!-- Prostokąt z obrysem -->
<Rectangle Fill="White" Stroke="DarkRed" StrokeThickness="3"
    Width="200" Height="50"
    HorizontalAlignment="Left" Margin="0,5"/>

<!-- Prostokąt z zaokrąglonymi rogami (tylko przez Border, Rectangle nie ma RadiusX/Y w starszych wersjach) -->
<Rectangle Fill="DarkGreen" Width="200" Height="50"
    RadiusX="10" RadiusY="10"
    HorizontalAlignment="Left" Margin="0,5"/>

<!-- Podgląd koloru RGB – prostokąt zmieniany z C# -->
<TextBlock Text="Podgląd wybranego koloru:" FontWeight="Bold" Margin="0,15,0,5"/>
<Rectangle x:Name="rectPodglad"
    Width="300" Height="80"
    Fill="#FF8000"
    RadiusX="6" RadiusY="6"
    Stroke="Black" StrokeThickness="1"
    HorizontalAlignment="Left"/>

<Button Content="Zmień na losowy kolor"
    Margin="0,10,0,0" Width="200"
    HorizontalAlignment="Left"
    Click="BtnLosuj_Click"/>

</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Media;

namespace PrzykladRectangle
{
    public partial class MainWindow : Window
    {
        private Random rnd = new Random();

        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnLosuj_Click(object sender, RoutedEventArgs e)
        {
            byte r = (byte)rnd.Next(256);
            byte g = (byte)rnd.Next(256);
            byte b = (byte)rnd.Next(256);

            Color kolor = Color.FromRgb(r, g, b);
            rectPodglad.Fill = new SolidColorBrush(kolor);
        }
    }
}

```

Kluczowe właściwości Rectangle:

- **Fill** — kolor wypełnienia (Brush lub nazwa koloru)
- **Stroke** — kolor obrysu
- **StrokeThickness** — grubość obrysu w pikselach
- **RadiusX** / **RadiusY** — zaokrąglenie rogów

- **Width** / **Height** — rozmiar prostokąta

13.2. Border

Border to kontrolka-kontener, która dodaje obramowanie, zaokrąglone rogi i tło do dowolnej innej kontrolki. Może zawierać dokładnie jeden element potomny.

MainWindow.xaml:

```
<Window x:Class="PrzykladBorder.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Border - kontener" Height="420" Width="480">
    <StackPanel Margin="20">
        <TextBlock Text="Border jako karta informacyjna:" FontWeight="Bold" Margin="0,0,0,10" />

        <!-- Karta ucznia jako Border z zawartością -->
        <Border BorderBrush="DodgerBlue"
                BorderThickness="2"
                CornerRadius="8"
                Padding="15"
                Margin="0,0,0,15"
                Background="AliceBlue">
            <StackPanel>
                <TextBlock Text="Anna Kowalska"
                           FontSize="18" FontWeight="Bold"
                           Foreground="DodgerBlue" />
                <TextBlock Text="Klasa: 3A" Margin="0,4,0,2" />
                <TextBlock Text="Numer ucznia: 15" />
                <TextBlock Text="Średnia: 4.82"
                           FontWeight="Bold" Foreground="DarkGreen" Margin="0,4,0,0" />
            </StackPanel>
        </Border>

        <!-- Border z innym stylem - alertowy -->
        <Border BorderBrush="DarkRed"
                BorderThickness="3"
                CornerRadius="6"
                Padding="12"
                Background="#FFF0F0"
                Margin="0,0,0,15">
            <StackPanel Orientation="Horizontal">
                <TextBlock Text="▲" FontSize="20" Margin="0,0,10,0" VerticalAlignment="Center" />
                <TextBlock Text="Uwaga: brakujące dane dokumentu!"
                           Foreground="DarkRed" VerticalAlignment="Center"
                           FontWeight="Bold" />
            </StackPanel>
        </Border>

        <!-- Border z różną grubością obramowania po stronach -->
        <Border BorderBrush="Orange"
                BorderThickness="0,0,0,4"
                Padding="10"
                Background="LightYellow">
            <TextBlock Text="Linia tylko na dole (BorderThickness="0,0,0,4")"
                           FontStyle="Italic" />
        </Border>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace PrzykladBorder
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Kluczowe właściwości Border:

- CornerRadius** — zaokrąglenie rogów; możesz podać jedną wartość lub cztery: "8,8,0,0" (lewy-górny, prawy-górny, prawy-dolny, lewy-dolny)
- BorderThickness** — grubość; jedna wartość = wszystkie strony; cztery = Lewo, Góra, Prawo, Dół
- BorderBrush** — kolor obramowania
- Padding** — wewnętrzny odstęp od krawędzi do zawartości

13.3. SolidColorBrush

W WPF kolory przekazujesz jako **Brush** (pędzel), a nie jako **Color**. Różnica jest ważna:

TYP	OPIS	UŻYCIE
Color	Struktura ARGB w przestrzeni System.Windows.Media	Pośredni — do tworzenia Brush
Brush	Abstrakcyjna klasa bazowa pędzla	Właściwości Background/Fill/etc.
SolidColorBrush	Jednokolorowy pędzel	Najpopularniejszy typ
Colors.Red	Gotowy kolor z klasy Colors	new SolidColorBrush(Colors.Red)
Brushes.Red	Gotowy pędzel z klasy Brushes	Bezpośrednio do właściwości

MainWindow.xaml:

```
<Window x:Class="KoloryIPedzle.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kolory i pędzle" Height="350" Width="450">
    <StackPanel Margin="20" x:Name="panelGlowny">
        <TextBlock x:Name="txtInfo"
            Text="Kliknij przycisk, aby zmienić kolory"
            FontSize="14"
            Margin="0,0,0,15"
            HorizontalAlignment="Center"/>

        <Button x:Name="btnKolor"
            Content="Zmień kolory panelu"
            Height="40"
            FontSize="14"
            Click="BtnKolor_Click"/>

        <TextBlock Text="Objaśnienie:"
            FontWeight="Bold" Margin="0,15,0,5"/>
        <TextBlock TextWrapping="Wrap"
            Text="Background to tło kontrolki. Foreground to kolor tekstu. BorderBrush to kolor obramowania. Wszystkie przyjmują wartość Brush."/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;
using System.Windows.Media;

namespace KoloryIPedzle
{
    public partial class MainWindow : Window
    {
        private int stan = 0;

        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnKolor_Click(object sender, RoutedEventArgs e)
        {
            stan = (stan + 1) % 3;

            switch (stan)
            {
                case 0:
                    // Gotowe pędzle z klasy Brushes
                    panelGlowny.Background = Brushes.White;
                    txtInfo.Foreground = Brushes.Black;
                    txtInfo.Text = "Stan 1: Domyślny (Brushes.White)";
                    break;

                case 1:
                    // Kolor z Color.FromRgb + SolidColorBrush
                    Color ciemnyGranat = Color.FromRgb(25, 42, 86);
                    panelGlowny.Background = new SolidColorBrush(ciemnyGranat);
                    txtInfo.Foreground = new SolidColorBrush(Colors.LightYellow);
                    txtInfo.Text = "Stan 2: Color.FromRgb(25, 42, 86)";
                    break;

                case 2:
                    // Kolor z hex stringa
                    Color zolty = (Color)ColorConverter.ConvertFromString("#FFD700");
                    panelGlowny.Background = new SolidColorBrush(zolty);
                    txtInfo.Foreground = new SolidColorBrush(Colors.DarkRed);
                    txtInfo.Text = "Stan 3: ColorConverter.ConvertFromString(\"#FFD700\")";
                    break;
            }
        }
    }
}
```

13.4. Text

Zmienianie wyglądu kontrolki to prosty i skuteczny sposób na sygnalizowanie błędów walidacji.

MainWindow.xaml:

```
<Window x:Class="WalidacjaKolorem.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Walidacja z kolorem" Height="350" Width="430">
    <StackPanel Margin="20">
```

```

<TextBlock Text="Formularz rejestracji" FontSize="16"
           FontWeight="Bold" Margin="0,0,0,15"/>

<TextBlock Text="Imię:" FontWeight="Bold" Margin="0,0,0,3"/>
<TextBox x:Name="txtImie"
         Padding="5,4"
         LostFocus="Pole_LostFocus"/>

<TextBlock Text="Email:" FontWeight="Bold" Margin="0,10,0,3"/>
<TextBox x:Name="txtEmail"
         Padding="5,4"
         LostFocus="Pole_LostFocus"/>

<TextBlock Text="Wiek (18-99):" FontWeight="Bold" Margin="0,10,0,3"/>
<TextBox x:Name="txtWiek"
         Padding="5,4"
         LostFocus="Pole_LostFocus"/>

<TextBlock x:Name="txtKomunikat"
           Margin="0,12,0,0"
           Foreground="DarkRed"
           TextWrapping="Wrap"
           FontSize="12"/>

<Button Content="Wyślij formularz"
        Margin="0,15,0,0" Height="35"
        Click="BtnWyslij_Click"
        Background="SteelBlue" Foreground="White"/>

</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;

namespace WalidacjaKoloRem
{
    public partial class MainWindow : Window
    {
        // Kolory tła dla stanu pola
        private static readonly SolidColorBrush TloBledne =
            new SolidColorBrush(Color.FromRgb(255, 200, 200)); // jasna czerwień
        private static readonly SolidColorBrush TloPoprawne =
            new SolidColorBrush(Color.FromRgb(200, 255, 200)); // jasna zieleń
        private static readonly SolidColorBrush TloDomyślne = Brushes.White;

        public MainWindow()
        {
            InitializeComponent();
        }

        private void Pole_LostFocus(object sender, RoutedEventArgs e)
        {
            // Walidujemy pole po opuszczeniu
            if (sender is TextBox tb)
                WalidujPole(tb);
        }
    }
}

```

```

private bool WalidujPole(TextBox pole)
{
    string wartosc = pole.Text.Trim();
    bool poprawne = true;
    string blad = "";

    if (pole == txtImie)
    {
        if (string.IsNullOrEmpty(wartosc))
            blad = "Imię jest wymagane.";
        else if (wartosc.Length < 2)
            blad = "Imię musi mieć przynajmniej 2 znaki.";
    }
    else if (pole == txtEmail)
    {
        if (!wartosc.Contains("@") || !wartosc.Contains("."))
            blad = "Nieprawidłowy format adresu email.";
    }
    else if (pole == txtWiek)
    {
        if (!int.TryParse(wartosc, out int wiek) || wiek < 18 || wiek > 99)
            blad = "Wiek musi być liczbą całkowitą od 18 do 99.";
    }

    if (!string.IsNullOrEmpty(blad))
    {
        pole.Background = TloBledne;
        poprawne = false;
    }
    else if (!string.IsNullOrEmpty(wartosc))
    {
        pole.Background = TloPoprawne;
    }
    else
    {
        pole.Background = TloDomyślne;
    }

    return poprawne;
}

private void BtnWyslij_Click(object sender, RoutedEventArgs e)
{
    bool ok = WalidujPole(txtImie) & WalidujPole(txtEmail) & WalidujPole(txtWiek);

    if (ok)
    {
        txtKomunikat.Foreground = Brushes.DarkGreen;
        txtKomunikat.Text = "✓ Formularz wysłany pomyślnie!";
    }
    else
    {
        txtKomunikat.Foreground = Brushes.DarkRed;
        txtKomunikat.Text = "✗ Popraw błędy przed wysłaniem.";
    }
}
}
}

```

Kluczowe elementy:

- `TloBledne`, `TloPoprawne` — predefiniowane pędzle jako pola statyczne

- `pole.Background = TloBledne` — zmiana tła pola na sygnał błędu
- `& (nie &&)` — nieprzerwywające AND — walidujemy wszystkie pola naraz
- `sender is TextBox tb` — pattern matching identyfikuje źródło zdarzenia

13.5. Rectangle

MainWindow.xaml:

```
<Window x:Class="PodgladDanych.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Karty produktów" Height="400" Width="500">
    <StackPanel Margin="15">
        <TextBlock Text="Wybierz produkt, aby zobaczyć kartę:"
                   FontWeight="Bold" Margin="0,0,0,10" />
        <ComboBox x:Name="cmbProdukt"
                  SelectionChanged="cmbProdukt_SelectionChanged" />

        <!-- Karta produktu jako Border -->
        <Border x:Name="borderKarta"
                BorderThickness="2"
                CornerRadius="8"
                Padding="15"
                Margin="0,15,0,0"
                Visibility="Collapsed">
            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="Auto" />
                    <ColumnDefinition Width="*" />
                </Grid.ColumnDefinitions>

                <!-- Kolorowy wskaźnik kategorii -->
                <Rectangle x:Name="rectKategoria"
                           Grid.Column="0"
                           Width="12" Height="80"
                           RadiusX="3" RadiusY="3"
                           Margin="0,0,12,0"
                           VerticalAlignment="Center" />

                <StackPanel Grid.Column="1">
                    <TextBlock x:Name="txtNazwa" FontSize="17" FontWeight="Bold" />
                    <TextBlock x:Name="txtKategoriaTxt" Foreground="Gray" Margin="0,2,0,6" />
                    <TextBlock x:Name="txtCena" FontSize="15" FontWeight="Bold" Foreground="DarkGreen" />
                </StackPanel>
            </Grid>
        </Border>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Collections.Generic;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;

namespace PodgladDanych
{
    public class Produkt
    {
```

```

        public string Nazwa { get; set; }
        public string Kategoria { get; set; }
        public decimal Cena { get; set; }
        public Color KolorKategorii { get; set; }
    }

    public partial class MainWindow : Window
    {
        private List<Produkt> produkty;

        public MainWindow()
        {
            InitializeComponent();
            ZaładujProdukty();
        }

        private void ZaładujProdukty()
        {
            produkty = new List<Produkt>
            {
                new Produkt { Nazwa="Laptop Pro 15", Kategoria="Elektronika", Cena=3499m, KolorKategorii=Colors.DodgerBlue },
                new Produkt { Nazwa="Krzesło ergonomiczne", Kategoria="Meble", Cena=899m, KolorKategorii=Colors.ForestGreen },
                new Produkt { Nazwa="Kawa ziarnista 1kg", Kategoria="Spożywcze", Cena=79m, KolorKategorii=Colors.SaddleBrown },
            };

            cmbProdukt.ItemsSource = produkty;
            cmbProdukt.DisplayMemberPath = "Nazwa";
        }

        private void cmbProdukt_SelectionChanged(object sender, SelectionChangedEventArgs e)
        {
            if (cmbProdukt.SelectedItem is Produkt p)
            {
                borderKarta.Visibility = Visibility.Visible;
                borderKarta.BorderBrush = new SolidColorBrush(p.KolorKategorii);
                rectKategoria.Fill = new SolidColorBrush(p.KolorKategorii);
                txtNazwa.Text = p.Nazwa;
                txtKategoriaTxt.Text = p.Kategoria;
                txtCena.Text = $"{p.Cena:C}";
            }
        }
    }
}

```

13.6. ProgressBar

ProgressBar wizualnie pokazuje postęp operacji.

MainWindow.xaml:

```

<Window x:Class="PrzykladProgressBar.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ProgressBar" Height="300" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Postęp operacji:" FontWeight="Bold" Margin="0,0,0,8"/>

        <ProgressBar x:Name="progressBar"
                    Minimum="0" Maximum="100"
                    Value="0"
                    Height="25"

```



```

        Margin="0,0,0,8"/>

        <TextBlock x:Name="txtProcent"
            Text="0%"
            HorizontalAlignment="Center"
            FontSize="13"
            Margin="0,0,0,15"/>

        <!-- Nieokreślony postęp (IsIndeterminate) -->
        <TextBlock Text="Nieokreślony postęp (ładowanie...):"
            FontWeight="Bold" Margin="0,0,0,8"/>
        <ProgressBar IsIndeterminate="True" Height="20" Margin="0,0,0,20"/>

        <StackPanel Orientation="Horizontal">
            <Button Content="Dodaj 10%" Click="BtnDodaj_Click" Margin="0,0,8,0" Padding="10,4"/>
            <Button Content="Resetuj" Click="BtnReset_Click" Padding="10,4"/>
        </StackPanel>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace PrzykladProgressBar
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnDodaj_Click(object sender, RoutedEventArgs e)
        {
            if (progressBar.Value < progressBar.Maximum)
            {
                progressBar.Value += 10;
                txtProcent.Text = $"{progressBar.Value}%";

                if (progressBar.Value ≥ progressBar.Maximum)
                    MessageBox.Show("Operacja zakończona!", "Gotowe");
            }
        }

        private void BtnReset_Click(object sender, RoutedEventArgs e)
        {
            progressBar.Value = 0;
            txtProcent.Text = "0%";
        }
    }
}

```

Kluczowe elementy:

- `Value` — aktualna wartość (między Minimum a Maximum)
- `IsIndeterminate="True"` — animowany pasek bez określonego postępu (np. ładowanie)
- `Minimum` / `Maximum` — zakres wartości (domyślnie 0–100)

13.7. Color, Brush i SolidColorBrush

Kolor w WPF może być zapisany jako nazwa, zapis hex albo obiekt `Color`. Większość właściwości wizualnych, takich jak `Background`, `Foreground`, `Fill`, `Stroke`, nie przyjmuje bezpośrednio `Color`, tylko `Brush`, czyli pędzel. Najczęściej jest to `SolidColorBrush`.

RÓŻNICA MIĘDZY COLOR I BRUSH

POJĘCIE	CO OZNACZA	PRZYKŁAD
<code>Color</code>	sama wartość koloru	<code>Color.FromRgb(255, 0, 0)</code>
<code>Brush</code>	sposób malowania powierzchni	<code>new SolidColorBrush(Colors.Red)</code>
<code>SolidColorBrush</code>	jednolity pędzel	<code>Background="Red"</code>
<code>LinearGradientBrush</code>	gradient liniowy	przejście od jednego koloru do drugiego
<code>ImageBrush</code>	pędzel z obrazem	tło elementu jako obraz

```
using System.Windows.Media;

Color kolor = Color.FromRgb(255, 0, 0);
SolidColorBrush pedzel = new SolidColorBrush(kolor);
rectPodglad.Fill = pedzel;
```

KOLORY NAZWANE W XAML

Najprostszy zapis koloru w XAML to nazwa:

```
<TextBlock Text="Komunikat" Foreground="DarkBlue"/>
<Border Background="LightYellow" BorderBrush="Goldenrod" BorderThickness="1"/>
<Rectangle Fill="Tomato" Stroke="DarkRed" StrokeThickness="2"/>
```

Nazwy kolorów są wygodne, bo łatwo je zapamiętać i szybko odczytać w kodzie XAML. Do precyzyjnego koloru lepszy jest zapis hex, np. `#FF336699`.

13.8. Pełna tabela kolorów nazwanych

KOLOR	PRÓBKA XAML	TYPOWE UŻYCIE
AliceBlue	Background="AliceBlue"	informacja, link, akcent
AntiqueWhite	Background="AntiqueWhite"	kolor dekoracyjny lub tło
Aqua	Background="Aqua"	kolor dekoracyjny lub tło
Aquamarine	Background="Aquamarine"	kolor dekoracyjny lub tło
Azure	Background="Azure"	kolor dekoracyjny lub tło
Beige	Background="Beige"	kolor dekoracyjny lub tło
Bisque	Background="Bisque"	kolor dekoracyjny lub tło
Black	Background="Black"	kolor dekoracyjny lub tło
BlanchedAlmond	Background="BlanchedAlmond"	kolor dekoracyjny lub tło
Blue	Background="Blue"	informacja, link, akcent
BlueViolet	Background="BlueViolet"	informacja, link, akcent
Brown	Background="Brown"	kolor dekoracyjny lub tło
BurlyWood	Background="BurlyWood"	kolor dekoracyjny lub tło
CadetBlue	Background="CadetBlue"	informacja, link, akcent
Chartreuse	Background="Chartreuse"	kolor dekoracyjny lub tło
Chocolate	Background="Chocolate"	kolor dekoracyjny lub tło
Coral	Background="Coral"	kolor dekoracyjny lub tło
CornflowerBlue	Background="CornflowerBlue"	informacja, link, akcent
Cornsilk	Background="Cornsilk"	kolor dekoracyjny lub tło
Crimson	Background="Crimson"	błąd, ostrzeżenie, usuwanie
Cyan	Background="Cyan"	kolor dekoracyjny lub tło
DarkBlue	Background="DarkBlue"	informacja, link, akcent
DarkCyan	Background="DarkCyan"	kolor dekoracyjny lub tło
DarkGoldenrod	Background="DarkGoldenrod"	kolor dekoracyjny lub tło
DarkGray	Background="DarkGray"	tło pomocnicze, obramowanie
DarkGreen	Background="DarkGreen"	sukces, poprawny stan
DarkKhaki	Background="DarkKhaki"	kolor dekoracyjny lub tło
DarkMagenta	Background="DarkMagenta"	kolor dekoracyjny lub tło
DarkOliveGreen	Background="DarkOliveGreen"	sukces, poprawny stan
DarkOrange	Background="DarkOrange"	kolor dekoracyjny lub tło
DarkOrchid	Background="DarkOrchid"	kolor dekoracyjny lub tło
DarkRed	Background="DarkRed"	błąd, ostrzeżenie, usuwanie
DarkSalmon	Background="DarkSalmon"	kolor dekoracyjny lub tło
DarkSeaGreen	Background="DarkSeaGreen"	sukces, poprawny stan
DarkSlateBlue	Background="DarkSlateBlue"	informacja, link, akcent
DarkSlateGray	Background="DarkSlateGray"	tło pomocnicze, obramowanie
DarkTurquoise	Background="DarkTurquoise"	kolor dekoracyjny lub tło
DarkViolet	Background="DarkViolet"	kolor dekoracyjny lub tło

KOLOR	PRÓBKA XAML	TYPOWE UŻYCIE
DeepPink	Background="DeepPink"	kolor dekoracyjny lub tło
DeepSkyBlue	Background="DeepSkyBlue"	informacja, link, akcent
DimGray	Background="DimGray"	tło pomocnicze, obramowanie
DodgerBlue	Background="DodgerBlue"	informacja, link, akcent
Firebrick	Background="Firebrick"	kolor dekoracyjny lub tło
FloralWhite	Background="FloralWhite"	kolor dekoracyjny lub tło
ForestGreen	Background="ForestGreen"	sukces, poprawny stan
Fuchsia	Background="Fuchsia"	kolor dekoracyjny lub tło
Gainsboro	Background="Gainsboro"	kolor dekoracyjny lub tło
GhostWhite	Background="GhostWhite"	kolor dekoracyjny lub tło
Gold	Background="Gold"	kolor dekoracyjny lub tło
Goldenrod	Background="Goldenrod"	kolor dekoracyjny lub tło
Gray	Background="Gray"	tło pomocnicze, obramowanie
Green	Background="Green"	sukces, poprawny stan
GreenYellow	Background="GreenYellow"	sukces, poprawny stan
Honeydew	Background="Honeydew"	kolor dekoracyjny lub tło
HotPink	Background="HotPink"	kolor dekoracyjny lub tło
IndianRed	Background="IndianRed"	błąd, ostrzeżenie, usuwanie
Indigo	Background="Indigo"	kolor dekoracyjny lub tło
Ivory	Background="Ivory"	kolor dekoracyjny lub tło
Khaki	Background="Khaki"	kolor dekoracyjny lub tło
Lavender	Background="Lavender"	kolor dekoracyjny lub tło
LavenderBlush	Background="LavenderBlush"	kolor dekoracyjny lub tło
LawnGreen	Background="LawnGreen"	sukces, poprawny stan
LemonChiffon	Background="LemonChiffon"	kolor dekoracyjny lub tło
LightBlue	Background="LightBlue"	informacja, link, akcent
LightCoral	Background="LightCoral"	kolor dekoracyjny lub tło
LightCyan	Background="LightCyan"	kolor dekoracyjny lub tło
LightGoldenrodYellow	Background="LightGoldenrodYellow"	kolor dekoracyjny lub tło
LightGray	Background="LightGray"	tło pomocnicze, obramowanie
LightGreen	Background="LightGreen"	sukces, poprawny stan
LightPink	Background="LightPink"	kolor dekoracyjny lub tło
LightSalmon	Background="LightSalmon"	kolor dekoracyjny lub tło
LightSeaGreen	Background="LightSeaGreen"	sukces, poprawny stan
LightSkyBlue	Background="LightSkyBlue"	informacja, link, akcent
LightSlateGray	Background="LightSlateGray"	tło pomocnicze, obramowanie
LightSteelBlue	Background="LightSteelBlue"	informacja, link, akcent
LightYellow	Background="LightYellow"	kolor dekoracyjny lub tło

KOLOR	PRÓBKA XAML	TYPOWE UŻYCIE
Lime	Background="Lime"	sukces, poprawny stan
LimeGreen	Background="LimeGreen"	sukces, poprawny stan
Linen	Background="Linen"	kolor dekoracyjny lub tło
Magenta	Background="Magenta"	kolor dekoracyjny lub tło
Maroon	Background="Maroon"	kolor dekoracyjny lub tło
MediumAquamarine	Background="MediumAquamarine"	kolor dekoracyjny lub tło
MediumBlue	Background="MediumBlue"	informacja, link, akcent
MediumOrchid	Background="MediumOrchid"	kolor dekoracyjny lub tło
MediumPurple	Background="MediumPurple"	kolor dekoracyjny lub tło
MediumSeaGreen	Background="MediumSeaGreen"	sukces, poprawny stan
MediumSlateBlue	Background="MediumSlateBlue"	informacja, link, akcent
MediumSpringGreen	Background="MediumSpringGreen"	sukces, poprawny stan
MediumTurquoise	Background="MediumTurquoise"	kolor dekoracyjny lub tło
MediumVioletRed	Background="MediumVioletRed"	błąd, ostrzeżenie, usuwanie
MidnightBlue	Background="MidnightBlue"	informacja, link, akcent
MintCream	Background="MintCream"	kolor dekoracyjny lub tło
MistyRose	Background="MistyRose"	kolor dekoracyjny lub tło
Moccasin	Background="Moccasin"	kolor dekoracyjny lub tło
NavajoWhite	Background="NavajoWhite"	kolor dekoracyjny lub tło
Navy	Background="Navy"	informacja, link, akcent
OldLace	Background="OldLace"	kolor dekoracyjny lub tło
Olive	Background="Olive"	kolor dekoracyjny lub tło
OliveDrab	Background="OliveDrab"	kolor dekoracyjny lub tło
Orange	Background="Orange"	kolor dekoracyjny lub tło
OrangeRed	Background="OrangeRed"	błąd, ostrzeżenie, usuwanie
Orchid	Background="Orchid"	kolor dekoracyjny lub tło
PaleGoldenrod	Background="PaleGoldenrod"	kolor dekoracyjny lub tło
PaleGreen	Background="PaleGreen"	sukces, poprawny stan
PaleTurquoise	Background="PaleTurquoise"	kolor dekoracyjny lub tło
PaleVioletRed	Background="PaleVioletRed"	błąd, ostrzeżenie, usuwanie
PapayaWhip	Background="PapayaWhip"	kolor dekoracyjny lub tło
PeachPuff	Background="PeachPuff"	kolor dekoracyjny lub tło
Peru	Background="Peru"	kolor dekoracyjny lub tło
Pink	Background="Pink"	kolor dekoracyjny lub tło
Plum	Background="Plum"	kolor dekoracyjny lub tło
PowderBlue	Background="PowderBlue"	informacja, link, akcent
Purple	Background="Purple"	kolor dekoracyjny lub tło
Red	Background="Red"	błąd, ostrzeżenie, usuwanie

KOLOR	PRÓBKA XAML	TYPOWE UŻYCIE
RosyBrown	Background="RosyBrown"	kolor dekoracyjny lub tło
RoyalBlue	Background="RoyalBlue"	informacja, link, akcent
SaddleBrown	Background="SaddleBrown"	kolor dekoracyjny lub tło
Salmon	Background="Salmon"	kolor dekoracyjny lub tło
SandyBrown	Background="SandyBrown"	kolor dekoracyjny lub tło
SeaGreen	Background="SeaGreen"	sukces, poprawny stan
SeaShell	Background="SeaShell"	kolor dekoracyjny lub tło
Sienna	Background="Sienna"	kolor dekoracyjny lub tło
Silver	Background="Silver"	tło pomocnicze, obramowanie
SkyBlue	Background="SkyBlue"	informacja, link, akcent
SlateBlue	Background="SlateBlue"	informacja, link, akcent
SlateGray	Background="SlateGray"	tło pomocnicze, obramowanie
Snow	Background="Snow"	kolor dekoracyjny lub tło
SpringGreen	Background="SpringGreen"	sukces, poprawny stan
SteelBlue	Background="SteelBlue"	informacja, link, akcent
Tan	Background="Tan"	kolor dekoracyjny lub tło
Teal	Background="Teal"	kolor dekoracyjny lub tło
Thistle	Background="Thistle"	kolor dekoracyjny lub tło
Tomato	Background="Tomato"	kolor dekoracyjny lub tło
Transparent	Background="Transparent"	przezroczyste tło
Turquoise	Background="Turquoise"	kolor dekoracyjny lub tło
Violet	Background="Violet"	kolor dekoracyjny lub tło
Wheat	Background="Wheat"	kolor dekoracyjny lub tło
White	Background="White"	kolor dekoracyjny lub tło
WhiteSmoke	Background="WhiteSmoke"	kolor dekoracyjny lub tło
Yellow	Background="Yellow"	kolor dekoracyjny lub tło
YellowGreen	Background="YellowGreen"	sukces, poprawny stan

13.9. Kolory hex, RGB i ARGB

ZAPIS	ZNACZENIE	PRZYKŁAD
#RRGGBB	czerwony, zielony, niebieski	#336699
#AARRGGBB	alfa + RGB	#80336699
CoLoR.FromRgb(r,g,b)	kolor z C# bez przezroczystości	CoLoR.FromRgb(51,102,153)
CoLoR.FromArgb(a,r,g,b)	kolor z C# z przezroczystością	CoLoR.FromArgb(128,51,102,153)

```
<Border Background="#FF336699" />
<Border Background="#80336699" />
```

```
byte r = 51;
byte g = 102;
```

```
byte b = 153;

rect.Fill = new SolidColorBrush(Color.FromRgb(r, g, b));
```

13.10. Gradienty i pędzle obrazkowe

```
<Border Width="240" Height="80">
    <Border.Background>
        <LinearGradientBrush StartPoint="0,0" EndPoint="1,0">
            <GradientStop Color="DarkBlue" Offset="0"/>
            <GradientStop Color="LightBlue" Offset="1"/>
        </LinearGradientBrush>
    </Border.Background>
</Border>
```

Gradienty są efektowne, ale warto ich używać wtedy, gdy poprawiają czytelność interfejsu, a nie tylko dekorują ekran. Do zwykłego sygnalizowania stanu lepszy jest prosty `SolidColorBrush`.

14. Zdarzenia w WPF

Zdarzenia to mechanizm informowania aplikacji o akcjach użytkownika lub zmianach stanu. WPF używa rozszerzonego systemu zdarzeń zwanego **Routed Events** (zdarzenia trasowane), który różni się od klasycznych zdarzeń .NET.

14.1. Czym są zdarzenia

Zdarzenie (event) to sygnał wysyłany przez kontrolkę, gdy coś się dzieje. Np. kliknięcie przycisku to zdarzenie `Click`. Aby zareagować na zdarzenie, piszesz **metodę obsługi** (event handler).

Delegate to "szablon" metody — określa jakie parametry i zwracany typ musi mieć handler:

```
// Delegate dla typowych zdarzeń UI w WPF:
// void NazwaMetody(object sender, RoutedEventArgs e)

// Delegate dla zdarzeń Slider:
// void NazwaMetody(object sender, RoutedEventArgs<double> e)

// Delegate dla zdarzeń zamknięcia okna:
// void NazwaMetody(object sender, EventArgs e)
```

Parametry:

- `sender` — obiekt, który wywołał zdarzenie (np. przycisk)
- `e` — dodatkowe informacje o zdarzeniu (zależne od typu zdarzenia)

14.2. Podpinanie zdarzeń w XAML

Możesz podpiąć zdarzenie na dwa sposoby:

Sposób 1: W XAML (prostszy)

```
<Button Content="Kliknij" Click="PrzyciskKlikniecia_Click"/>
```

Sposób 2: W C# (bardziej elastyczny)

```
// W konstruktorze lub metodzie inicjalizującej:
mojPrzycisk.Click += PrzyciskKlikniecia_Click;

// Odpięcie zdarzenia (ważne przy dynamicznych kontrolkach):
mojPrzycisk.Click -= PrzyciskKlikniecia_Click;
```

```
// Lambda (wyrażenie anonimowe) – gdy handler jest prosty:
mojPrzycisk.Click += (s, e) => { MessageBox.Show("Kliknięto!"); };
```

MainWindow.xaml:

```
<Window x:Class="PodpiecieZdarzen.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Podpięcie zdarzeń" Height="300" Width="400">
    <StackPanel Margin="20">
        <!-- Zdarzenie podpięte w XAML -->
        <Button x:Name="btnXAML"
                Content="Zdarzenie z XAML"
                Margin="0,0,0,10"
                Click="btnXAML_Click"/>

        <!-- Zdarzenie podpięte w C# (bez atrybutu Click w XAML) -->
        <Button x:Name="btnCS"
                Content="Zdarzenie z C#"
                Margin="0,0,0,10"/>

        <!-- Zdarzenie przez lambda -->
        <Button x:Name="btnLambda"
                Content="Zdarzenie przez lambda"
                Margin="0,0,0,10"/>

        <TextBlock x:Name="txtInfo" TextWrapping="Wrap" Foreground="DarkBlue"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace PodpiecieZdarzen
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // Podpięcie zdarzenia w C#
            btnCS.Click += BtnCS_Click;

            // Podpięcie zdarzenia przez lambda
            btnLambda.Click += (sender, e) =>
            {
                txtInfo.Text = "Kliknięto przycisk przez wyrażenie lambda!";
            };
        }

        private void btnXAML_Click(object sender, RoutedEventArgs e)
        {
            txtInfo.Text = "Kliknięto przycisk z zdarzeniem w XAML.";
        }

        private void BtnCS_Click(object sender, RoutedEventArgs e)
        {

```



```

        txtInfo.Text = "Kliknięto przycisk z zdarzeniem podpiętym w C#.";
    }
}
}

```

14.3. Parametry sender i RoutedEventArgs

Parametr `sender` to obiekt który wywołał zdarzenie. Możesz go użyć by zidentyfikować które pole lub przycisk spowodowało zdarzenie — szczególnie gdy kilka kontrolki współdzieli jeden handler.

MainWindow.xaml:

```

<Window x:Class="IdentyfikacjaSendera.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Identyfikacja sender" Height="320" Width="420">
    <StackPanel Margin="20">
        <TextBlock Text="Kliknij dowolny przycisk:" FontWeight="Bold" Margin="0,0,0,10"/>

        <!-- Trzy przyciski z jednym handlerem -->
        <Button x:Name="btnCzerwony" Content="Czerwony"
                Tag="Czerwony" Margin="0,5"
                Background="Red" Foreground="White"
                Click="Btn_Click"/>
        <Button x:Name="btnZielony" Content="Zielony"
                Tag="Zielony" Margin="0,5"
                Background="Green" Foreground="White"
                Click="Btn_Click"/>
        <Button x:Name="btnNiebieski" Content="Niebieski"
                Tag="Niebieski" Margin="0,5"
                Background="Blue" Foreground="White"
                Click="Btn_Click"/>

        <TextBlock x:Name="txtWynik"
                Margin="0,15,0,0"
                FontSize="14"
                TextWrapping="Wrap"
                Foreground="DarkBlue"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;
using System.Windows.Controls;

namespace IdentyfikacjaSendera
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        // Jeden handler dla trzech przycisków
        private void Btn_Click(object sender, RoutedEventArgs e)
        {
            // Rzutowanie sender na Button
            if (sender is Button klikniety)

```

```

{
    string nazwa = klikniety.Name;        // x:Name przycisku
    string tresc = klikniety.Content?.ToString() ?? "?"; // Treść
    string tag = klikniety.Tag?.ToString() ?? "brak";    // Tag

    // Identyfikacja przez referencję
    string identyfikacja;
    if (klikniety == btnCzerwony)
        identyfikacja = "To jest przycisk CZERWONY";
    else if (klikniety == btnZielony)
        identyfikacja = "To jest przycisk ZIELONY";
    else
        identyfikacja = "To jest przycisk NIEBIESKI";

    txtWynik.Text = $"Name: {nazwa}\nContent: {tresc}\nTag: {tag}\n{identyfikacja}";
}
}
}
}

```

Kluczowe elementy:

- `sender is Button klikniety` — bezpieczne rzutowanie + przypisanie
- `klikniety.Name` — wartość `x:Name` kontrolki
- `klikniety.Tag` — własność `Tag` to dowolny obiekt który możesz przypisać w XAML
- `klikniety == btnCzerwony` — porównanie referencji do konkretnej kontrolki

14.4. Click

MainWindow.xaml:

```

<Window x:Class="PełnyClick.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kalkulator BMI" Height="380" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Kalkulator BMI" FontSize="18" FontWeight="Bold" Margin="0,0,0,15"/>

        <TextBlock Text="Waga (kg):" FontWeight="Bold" Margin="0,0,0,4"/>
        <TextBox x:Name="txtWaga" Padding="5,4" Margin="0,0,0,10"/>

        <TextBlock Text="Wzrost (cm):" FontWeight="Bold" Margin="0,0,0,4"/>
        <TextBox x:Name="txtWzrost" Padding="5,4" Margin="0,0,0,15"/>

        <Button Content="Oblicz BMI"
                Height="38" FontSize="14"
                Background="SteelBlue" Foreground="White"
                Click="BtnOblicz_Click"/>

        <Border x:Name="borderWynik"
                Visibility="Collapsed"
                BorderThickness="2" CornerRadius="6"
                Padding="15" Margin="0,15,0,0">
            <StackPanel>
                <TextBlock x:Name="txtBMI" FontSize="22" FontWeight="Bold"
                            HorizontalAlignment="Center"/>
                <TextBlock x:Name="txtKategoria" FontSize="14"
                            HorizontalAlignment="Center" Margin="0,5,0,0"/>
            </StackPanel>
        </Border>
    </StackPanel>
</Window>

```

```
</StackPanel>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;
using System.Windows.Media;

namespace PelnyClick
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnOblicz_Click(object sender, RoutedEventArgs e)
        {
            // Odczyt i walidacja danych wejściowych
            if (!double.TryParse(txtWaga.Text.Replace(',', '.'),
                System.Globalization.NumberStyles.Float,
                System.Globalization.CultureInfo.InvariantCulture, out double waga)
                || waga ≤ 0)
            {
                MessageBox.Show("Podaj poprawną wagę (np. 70.5).");
                return;
            }

            if (!double.TryParse(txtWzrost.Text.Replace(',', '.'),
                System.Globalization.NumberStyles.Float,
                System.Globalization.CultureInfo.InvariantCulture, out double wzrost)
                || wzrost ≤ 0)
            {
                MessageBox.Show("Podaj poprawny wzrost (np. 175).");
                return;
            }

            // Obliczenie BMI
            double wzrostM = wzrost / 100.0;
            double bmi = waga / (wzrostM * wzrostM);

            // Wyznaczanie kategorii i koloru
            string kategoria;
            Color kolor;

            if (bmi < 18.5) { kategoria = "Niedowaga"; kolor = Colors.DodgerBlue; }
            else if (bmi < 25.0) { kategoria = "Waga prawidłowa ✓"; kolor = Colors.ForestGreen; }
            else if (bmi < 30.0) { kategoria = "Nadwaga"; kolor = Colors.Orange; }
            else { kategoria = "Otyłość"; kolor = Colors.Crimson; }

            // Aktualizacja UI
            txtBMI.Text = $"BMI = {bmi:F2}";
            txtKategoria.Text = kategoria;
            txtKategoria.Foreground = new SolidColorBrush(kolor);
            borderWynik.BorderBrush = new SolidColorBrush(kolor);
            borderWynik.Background = new SolidColorBrush(Color.FromArgb(30, kolor.R, kolor.G, kolor.B));
            borderWynik.Visibility = Visibility.Visible;
        }
    }
}
```

```
}  
  
}
```

14.5. TextChanged

Zdarzenie `TextChanged` jest wywoływane za każdym razem gdy tekst w `TextBox` się zmienia. Przydatne do wyszukiwania w czasie rzeczywistym lub liczenia znaków.

MainWindow.xaml:

```
<Window x:Class="TextChangedPrzyklad.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="TextChanged" Height="300" Width="420">  
    <StackPanel Margin="20">  
        <TextBlock Text="Wpisz tekst (max 100 znaków):"  
                   FontWeight="Bold" Margin="0,0,0,5" />  
        <TextBox x:Name="txtWpis"  
                 MaxLength="100"  
                 TextWrapping="Wrap"  
                 MinHeight="80"  
                 AcceptsReturn="True"  
                 Padding="5"  
                 TextChanged="txtWpis_TextChanged" />  
  
        <TextBlock x:Name="txtLicznik"  
                   Text="0 / 100 znaków"  
                   HorizontalAlignment="Right"  
                   Margin="0,5,0,15"  
                   Foreground="Gray" />  
  
        <!-- Wyszukiwanie na żywo -->  
        <TextBlock Text="Wyszukaj kraj:" FontWeight="Bold" Margin="0,0,0,5" />  
        <TextBox x:Name="txtSzukaj"  
                 Padding="5,4"  
                 TextChanged="txtSzukaj_TextChanged" />  
  
        <ListBox x:Name="lstWyniki" Height="100" Margin="0,5,0,0" />  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System.Collections.Generic;  
using System.Linq;  
using System.Windows;  
using System.Windows.Controls;  
using System.Windows.Media;  
  
namespace TextChangedPrzyklad  
{  
    public partial class MainWindow : Window  
    {  
        private List<string> kraje = new List<string>  
        {  
            "Polska", "Niemcy", "Francja", "Włochy", "Hiszpania",  
            "Portugalia", "Holandia", "Belgia", "Czechy", "Austria"  
        };  
  
        public MainWindow()  
        {
```

```

        InitializeComponent();
        lstWyniki.ItemsSource = kraje;
    }

    private void txtWpis_TextChanged(object sender, TextChangedEventArgs e)
    {
        int dlugosc = txtWpis.Text.Length;
        txtLicznik.Text = $"{dlugosc} / 100 znaków";
        // Zmiana koloru gdy zbliżamy się do limitu
        txtLicznik.Foreground = dlugosc > 85 ? Brushes.DarkRed : Brushes.Gray;
    }

    private void txtSzukaj_TextChanged(object sender, TextChangedEventArgs e)
    {
        string fraza = txtSzukaj.Text.ToLower();

        if (string.IsNullOrEmpty(fraza))
        {
            lstWyniki.ItemsSource = kraje;
            return;
        }

        // Filtrujemy listę na żywo
        var wyniki = kraje.Where(k => k.ToLower().Contains(fraza)).ToList();
        lstWyniki.ItemsSource = wyniki;
    }
}

```

Kluczowe elementy:

- `TextChangedEventArgs` — argument zdarzenia (zawiera `Changes` z informacją o zmianach)
- `txtWpis.Text.Length` — odczytujemy długość tekstu w każdej zmianie
- `MaxLength="100"` — XAML ogranicza ilość znaków
- `LINQ Where` — wygodne filtrowanie listy

14.6. LostFocus

`LostFocus` jest wywoływany gdy kontrolka traci fokus (użytkownik przechodzi do następnego pola Tab lub kliknął gdzie indziej). Idealny do walidacji i ładowania danych.

MainWindow.xaml:

```

<Window x:Class="LostFocusPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="LostFocus - ładowanie zdjęcia" Height="420" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Podaj numer ID pracownika:"
                  FontWeight="Bold" Margin="0,0,0,5"/>

        <!-- LostFocus wywoła ładowanie zdjęcia po opuszczeniu pola -->
        <TextBox x:Name="txtIdPracownika"
                 Padding="6,5"
                 FontSize="15"
                 MaxLength="6"
                 LostFocus="txtIdPracownika_LostFocus"/>

        <TextBlock x:Name="txtPodpowiedz"
                  Text="(opusć pole Tab-em lub klikając dalej)"
                  Foreground="Gray" FontSize="11"
                  Margin="0,4,0,15"/>
    </StackPanel>
</Window>

```

```

<!-- Karta pracownika -->
<Border x:Name="borderPracownik"
        BorderBrush="LightGray" BorderThickness="1"
        CornerRadius="6" Padding="15"
        Visibility="Collapsed">

    <Grid>

        <Grid.ColumnDefinitions>

            <ColumnDefinition Width="Auto"/>

            <ColumnDefinition Width="*/>

        </Grid.ColumnDefinitions>

        <Border Grid.Column="0" Width="80" Height="80"
                BorderBrush="Gray" BorderThickness="1"
                CornerRadius="40" Margin="0,0,15,0"
                ClipToBounds="True">

            <Image x:Name="imgPracownik" Stretch="UniformToFill"/>

        </Border>

        <StackPanel Grid.Column="1" VerticalAlignment="Center">

            <TextBlock x:Name="txtNazwiskoPrac"
                      FontSize="16" FontWeight="Bold"/>

            <TextBlock x:Name="txtDzial"
                      Foreground="Gray" Margin="0,3,0,0"/>

            <TextBlock x:Name="txtStanowisko"
                      FontWeight="Bold" Foreground="SteelBlue"
                      Margin="0,3,0,0"/>

        </StackPanel>

    </Grid>

</Border>

<TextBlock x:Name="txtBladPrac"
            Foreground="DarkRed"
            Margin="0,10,0,0"
            TextWrapping="Wrap"/>

</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Windows;
using System.Windows.Media.Imaging;

namespace LostFocusPrzyklad
{
    // Prosta klasa danych pracownika
    public class Pracownik
    {
        public string Nazwisko { get; set; }
        public string Dzial { get; set; }
        public string Stanowisko { get; set; }
        public string ZdjeciePlik { get; set; }
    }

    public partial class MainWindow : Window
    {
        // Słownik: ID → dane pracownika (symulacja bazy danych)
        private Dictionary<string, Pracownik> pracownicy = new Dictionary<string, Pracownik>

```

```
{
    { "001", new Pracownik { Nazwisko="Jan Kowalski", Dzial="IT", Stanowisko="Programista", ZdjeciePlik="001.jpg" } },
    { "002", new Pracownik { Nazwisko="Anna Nowak", Dzial="HR", Stanowisko="Kierownik", ZdjeciePlik="002.jpg" } },
    { "003", new Pracownik { Nazwisko="Piotr Zając", Dzial="Finanse", Stanowisko="Księgowy", ZdjeciePlik="003.jpg" } },
};

public MainWindow()
{
    InitializeComponent();
}

private void txtIdPracownika_LostFocus(object sender, RoutedEventArgs e)
{
    string id = txtIdPracownika.Text.Trim();
    txtBladPrac.Text = "";

    if (string.IsNullOrEmpty(id))
    {
        borderPracownik.Visibility = Visibility.Collapsed;
        return;
    }

    // Wyszukujemy pracownika
    if (pracownicy.TryGetValue(id, out Pracownik prac))
    {
        // Wypełniamy dane
        txtNazwiskoPrac.Text = prac.Nazwisko;
        txtDzial.Text = prac.Dzial;
        txtStanowisko.Text = prac.Stanowisko;

        // Ładujemy zdjęcie
        ZaladujZdjeciePracownika(prac.ZdjeciePlik);
        borderPracownik.Visibility = Visibility.Visible;
    }
    else
    {
        borderPracownik.Visibility = Visibility.Collapsed;
        txtBladPrac.Text = $"Nie znaleziono pracownika o ID: {id}";
    }
}

private void ZaladujZdjeciePracownika(string nazwaPliku)
{
    string sciezka = Path.Combine(
        AppDomain.CurrentDomain.BaseDirectory, "Images", nazwaPliku);

    if (!File.Exists(sciezka))
    {
        imgPracownik.Source = null;
        return;
    }

    try
    {
        BitmapImage bmp = new BitmapImage();
        bmp.BeginInit();
        bmp.UriSource = new Uri(sciezka, UriKind.Absolute);
        bmp.CacheOption = BitmapCacheOption.OnLoad;
        bmp.EndInit();
        imgPracownik.Source = bmp;
    }
    catch
```

```

        {
            imgPracownik.Source = null;
        }
    }
}
}

```

14.7. List

`SelectionChanged` jest wywoływane gdy użytkownik zmienia zaznaczenie w `ComboBox` lub `ListBox`.

MainWindow.xaml:

```

<Window x:Class="SelectionChangedPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="SelectionChanged" Height="380" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Kraj:" FontWeight="Bold" Margin="0,0,0,4"/>
        <ComboBox x:Name="cmbKraj"
                SelectionChanged="cmbKraj_SelectionChanged">
            <ComboBoxItem Content="Polska"/>
            <ComboBoxItem Content="Niemcy"/>
            <ComboBoxItem Content="Francja"/>
        </ComboBox>

        <TextBlock Text="Wybrane języki (kliknij kilka z Ctrl):"
                FontWeight="Bold" Margin="0,15,0,4"/>
        <ListBox x:Name="lstJęzyki"
                Height="130"
                SelectionMode="Extended"
                SelectionChanged="lstJęzyki_SelectionChanged"/>

        <TextBlock x:Name="txtWynik"
                Margin="0,12,0,0"
                TextWrapping="Wrap"
                Foreground="DarkBlue"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Collections.Generic;
using System.Text;
using System.Windows;
using System.Windows.Controls;

namespace SelectionChangedPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            lstJęzyki.ItemsSource = new List<string>
            { "C#", "Java", "Python", "JavaScript", "Go", "Rust" };
        }

        private void cmbKraj_SelectionChanged(object sender, SelectionChangedEventArgs e)
        {

```



```

        // e.AddedItems – nowo zaznaczone
        // e.RemovedItems – odznaczone
        if (cmbKraj.SelectedItem is ComboBoxItem item)
        {
            txtWynik.Text = $"Wybrany kraj: {item.Content}";
        }
    }

    private void lstJęzyki_SelectionChanged(object sender, SelectionChangedEventArgs e)
    {
        var sb = new StringBuilder("Wybrane języki: ");
        foreach (var item in lstJęzyki.SelectedItems)
            sb.Append($"{item}, ");

        txtWynik.Text = sb.ToString().TrimEnd(' ', ',');
    }
}

```

Kluczowe elementy:

- `SelectionChangedEventArgs` — zawiera `AddedItems` i `RemovedItems`
- `e.AddedItems` — kolekcja nowo zaznaczonych elementów
- `e.RemovedItems` — kolekcja odznaczonych elementów
- `SelectionMode="Extended"` — zaznaczanie z Ctrl i Shift

14.8. Button

Dzdarzenia `Checked` i `Unchecked` są wywoływane odpowiednio gdy kontrolka jest zaznaczana i odznaczana. Działają dla `CheckBox` i `RadioButton`.

MainWindow.xaml:

```

<Window x:Class="CheckedUnchecked.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Checked / Unchecked" Height="320" Width="420">
    <StackPanel Margin="20">
        <TextBlock Text="Opcje powiadomień:" FontWeight="Bold" Margin="0,0,0,8"/>

        <CheckBox x:Name="chkEmail"
            Content="Powiadomienia Email"
            Margin="0,4"
            Checked="Powiadomienie_Checked"
            Unchecked="Powiadomienie_Unchecked"/>

        <CheckBox x:Name="chkSMS"
            Content="Powiadomienia SMS"
            Margin="0,4"
            Checked="Powiadomienie_Checked"
            Unchecked="Powiadomienie_Unchecked"/>

        <!-- Indeterminate działa gdy IsThreeState=True -->
        <CheckBox x:Name="chkPush"
            Content="Powiadomienia PUSH (trzy stany)"
            IsThreeState="True"
            Margin="0,4"
            Checked="Powiadomienie_Checked"
            Unchecked="Powiadomienie_Unchecked"
            Indeterminate="chkPush_Indeterminate"/>

        <Separator Margin="0,12"/>
    
```

```

<TextBlock Text="Log zdarzeń:" FontWeight="Bold" Margin="0,0,0,5"/>
<TextBlock x:Name="txtLog"
            TextWrapping="Wrap"
            Foreground="DarkBlue"
            FontFamily="Consolas"
            FontSize="11"/>

<Button Content="Wyczyść log"
        Click="BtnWyczysc_Click"
        Margin="0,10,0,0"
        HorizontalAlignment="Left"
        Padding="10,4"/>

</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;
using System.Windows.Controls;

namespace CheckedUnchecked
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void Powiadomienie_Checked(object sender, RoutedEventArgs e)
        {
            if (sender is CheckBox cb && txtLog != null)
            {
                txtLog.Text = $"[ZAZNACZONO] {cb.Content}\n" + txtLog.Text;
            }
        }

        private void Powiadomienie_Unchecked(object sender, RoutedEventArgs e)
        {
            if (sender is CheckBox cb && txtLog != null)
            {
                txtLog.Text = $"[ODZNACZONO] {cb.Content}\n" + txtLog.Text;
            }
        }

        private void chkPush_Indeterminate(object sender, RoutedEventArgs e)
        {
            if (txtLog != null)
                txtLog.Text = "[NIEOKREŚLONY] Powiadomienia PUSH\n" + txtLog.Text;
        }

        private void BtnWyczysc_Click(object sender, RoutedEventArgs e)
        {
            txtLog.Text = "";
        }
    }
}

```

Kluczowe elementy:

- Checked** — wywoływane przy zaznaczeniu

- **Unchecked** — wywoływane przy odznaczeniu
- **Indeterminate** — wywoływane gdy ustawia się stan **null** (trzeci stan)
- Jeden handler może obsługiwać wiele kontroltek — identyfikujesz przez **sender**

14.9. Slider

Zdarzenie **ValueChanged** Slidera zostało omówione w rozdziale 11.2. Tutaj pokazujemy je w kontekście zdarzeń WPF.

MainWindow.xaml:

```
<Window x:Class="ValueChangedPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ValueChanged - Slider" Height="280" Width="430">
    <StackPanel Margin="20">
        <TextBlock Text="Regulacja temperatury:" FontWeight="Bold" Margin="0,0,0,8"/>
        <StackPanel Orientation="Horizontal">
            <Slider x:Name="sldTemperatura"
                    Minimum="16" Maximum="30"
                    Value="21"
                    Width="280"
                    SmallChange="0.5"
                    TickFrequency="1"
                    IsSnapToTickEnabled="True"
                    ValueChanged="sldTemperatura_ValueChanged"/>
            <TextBlock x:Name="lblTemp"
                    Text="21°C"
                    Margin="10,0,0,0"
                    FontSize="16"
                    FontWeight="Bold"
                    VerticalAlignment="Center"/>
        </StackPanel>

        <TextBlock x:Name="txtKomfort"
                    Margin="0,12,0,0"
                    FontSize="14"
                    TextWrapping="Wrap"/>

        <TextBlock Text="Poprzednia wartość:"
                    FontWeight="Bold" Margin="0,15,0,4"/>
        <TextBlock x:Name="txtPoprzednia" Foreground="Gray"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;
using System.Windows.Controls.Primitives;

namespace ValueChangedPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void sldTemperatura_ValueChanged(object sender,
            RoutedPropertyChangedEventArgs<double> e)
        {
            // ...
        }
    }
}
```

```

        if (lblTemp == null) return;

        double nowa = e.NewValue;
        double poprzednia = e.OldValue;

        lblTemp.Text = $"{nowa:F0}°C";

        // Korzystamy z e.NewValue do reakcji
        if (nowa < 18)
            txtKomfort.Text = "❄ Zbyt zimno – poniżej strefy komfortu";
        else if (nowa ≤ 23)
            txtKomfort.Text = "✓ Optymalna temperatura";
        else if (nowa ≤ 26)
            txtKomfort.Text = "🌡 Ciepło – powyżej optymalnej";
        else
            txtKomfort.Text = "🔥 Bardzo ciepło!";

        txtPoprzednia.Text = $"e.OldValue = {poprzednia:F0}°C → e.NewValue = {nowa:F0}°C";
    }
}
}

```

14.10. Loaded

Zdarzenie `Loaded` okna jest wywoływane gdy okno zostało załadowane i wszystkie kontrolki są gotowe. To odpowiednie miejsce na ładowanie danych z pliku lub bazy, inicjalizację list itp.

MainWindow.xaml:

```

<Window x:Class="LoadedPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Loaded - inicjalizacja" Height="350" Width="450"
        Loaded="Window_Loaded">
    <StackPanel Margin="20">
        <TextBlock x:Name="txtPowitanie"
            FontSize="16" FontWeight="Bold"
            Margin="0,0,0,10"/>

        <TextBlock Text="Ostatnie dokumenty:" FontWeight="Bold" Margin="0,0,0,5"/>
        <ListBox x:Name="lstOstatnie" Height="120" Margin="0,0,0,10"/>

        <TextBlock x:Name="txtCzas"
            FontSize="12" Foreground="Gray"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.Generic;
using System.Windows;

namespace LoadedPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

```

```

        // Uwaga: tutaj kontrolki istnieją, ale okno nie jest jeszcze widoczne
    }

    private void Window_Loaded(object sender, RoutedEventArgs e)
    {
        // Tu okno jest w pełni załadowane i widoczne – idealne miejsce na inicjalizację

        // Powitanie z godziną
        string godzina = DateTime.Now.Hour;
        string powitanie = int.Parse(godzina.ToString()) < 12 ? "Dzień dobry" : "Dobry wieczór";
        txtPowitanie.Text = $"{powitanie}! Aplikacja gotowa.";

        // Ładowanie danych (np. z rejestru, pliku konfiguracyjnego)
        ZaładujOstatnieDokumenty();

        // Aktualna data i czas
        txtCzas.Text = $"Czas uruchomienia: {DateTime.Now:yyyy-MM-dd HH:mm:ss}";
    }

    private void ZaładujOstatnieDokumenty()
    {
        // W prawdziwej aplikacji czytasz z pliku lub rejestru Windows
        List<string> dokumenty = new List<string>
        {
            "Faktura_2024_001.pdf",
            "Raport_roczny.xlsx",
            "Umowa_kowalski.docx",
            "Zdjecia_paszportowe.zip"
        };

        lstOstatnie.ItemsSource = dokumenty;
    }
}

```

14.11. Zdarzenia klawiatury: KeyDown, KeyUp

KeyDown jest wywoływane gdy użytkownik naciśnie klawisz. Możesz sprawdzić który klawisz za pomocą **e.Key**.

MainWindow.xaml:

```

<Window x:Class="KlawiaturaPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Obsługa klawiatury" Height="320" Width="430">
    <StackPanel Margin="20">
        <TextBlock Text="Wyszukaj (naciśnij Enter):"
                   FontWeight="Bold" Margin="0,0,0,5" />

        <!-- KeyDown na TextBox -->
        <TextBox x:Name="txtSzukaj"
                 Padding="6,5"
                 FontSize="14"
                 KeyDown="txtSzukaj_KeyDown" />

        <!-- KeyDown na całym oknie (IsEnabled na Window) -->
        <TextBlock Text="Naciśnij dowolny klawisz w dowolnym miejscu:"
                   FontWeight="Bold" Margin="0,15,0,5" />
        <TextBlock x:Name="txtKlawisz"
                   FontFamily="Consolas"
                   FontSize="16"

```

```

        Foreground="DarkBlue"
        Text="(czekam na klawisz...)" />

        <TextBlock x:Name="txtWynikSzukania"
            Margin="0,15,0,0"
            TextWrapping="Wrap"
            Foreground="DarkGreen"
            FontSize="13" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;
using System.Windows.Input;

namespace KlawiaturaPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            // Podpinamy zdarzenie klawiatury dla całego okna
            this.KeyDown += Window_KeyDown;
        }

        private void txtSzukaj_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.Key == Key.Enter)
            {
                // Obsługa klawisza Enter w polu wyszukiwania
                string fraza = txtSzukaj.Text.Trim();

                if (string.IsNullOrEmpty(fraza))
                {
                    txtWynikSzukania.Text = "Wpisz frazę do wyszukania.";
                    return;
                }

                txtWynikSzukania.Text = $"🔍 Szukam: \"{faza}\"...";

                // Oznaczamy zdarzenie jako obsłużone (nie propaguje dalej)
                e.Handled = true;
            }
            else if (e.Key == Key.Escape)
            {
                txtSzukaj.Clear();
                txtWynikSzukania.Text = "";
            }
        }

        private void Window_KeyDown(object sender, KeyEventArgs e)
        {
            // Wyświetlamy wciśnięty klawisz
            txtKlawisz.Text = $"Wciśnięto: {e.Key}";

            // Sprawdzanie klawiszy funkcyjnych
            if (e.Key == Key.F1)
            {
                MessageBox.Show("Pomoc aplikacji – F1", "Pomoc");
            }
        }
    }
}

```

```

        e.Handled = true;
    }
    else if (e.Key == Key.F5)
    {
        txtWynikSzukania.Text = "Odświeżono (F5)";
        e.Handled = true;
    }
    }
}
}
}

```

Kluczowe elementy:

- `KeyEventArgs e` — zawiera informacje o zdarzeniu klawiatury
- `e.Key` — enumeracja `Key` z wartościami jak `Key.Enter`, `Key.Escape`, `Key.F1`, `Key.A` itp.
- `e.Handled = true` — zatrzymuje propagację zdarzenia do kontrolek nadrzędnych
- `this.KeyDown` — zdarzenie na poziomie okna przechwytuje wszystkie klawisze

14.12. Closing

Zdarzenie `Closing` jest wywoływane gdy użytkownik próbuje zamknąć okno (klikając X lub przez Alt+F4). Możesz anulować zamknięcie przez `e.Cancel = true`.

MainWindow.xaml:

```

<Window x:Class="ZdarzeniaOkna.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zdarzenia okna" Height="350" Width="450"
        Loaded="Window_Loaded"
        Closing="Window_Closing">
    <StackPanel Margin="20">
        <TextBlock Text="Aplikacja z potwierdzeniem zamknięcia"
            FontSize="15" FontWeight="Bold" Margin="0,0,0,15" />

        <CheckBox x:Name="chkNiezapisane"
            Content="Symuluj niezapisane zmiany"
            IsChecked="True"
            Margin="0,0,0,15" />

        <TextBlock Text="Status:" FontWeight="Bold" Margin="0,0,0,5" />
        <TextBlock x:Name="txtStatus"
            Text="Aplikacja uruchomiona."
            TextWrapping="Wrap"
            Foreground="DarkBlue" />

        <StackPanel Orientation="Horizontal" Margin="0,20,0,0">
            <Button Content="Zapisz"
                Click="BtnZapisz_Click"
                Padding="12,5" Margin="0,0,10,0"
                Background="DarkGreen" Foreground="White" />
            <Button Content="Zamknij aplikację"
                Click="BtnZamknij_Click"
                Padding="12,5"
                Background="DarkRed" Foreground="White" />
        </StackPanel>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```
using System.ComponentModel;
using System.Windows;

namespace ZdarzeniaOkna
{
    public partial class MainWindow : Window
    {
        private bool maNiezapisaneZmiany = true;

        public MainWindow()
        {
            InitializeComponent();

            private void Window_Loaded(object sender, RoutedEventArgs e)
            {
                txtStatus.Text = "Aplikacja załadowana i gotowa do pracy.";
            }

            private void Window_Closing(object sender, CancelEventArgs e)
            {
                // Sprawdzamy czy są niezapisane zmiany
                bool symulowaNeizapisane = chkNiezapisane.IsChecked == true;

                if (symulowaNeizapisane)
                {
                    MessageBoxResult wynik = MessageBox.Show(
                        "Masz niezapisane zmiany.\nCzy chcesz wyjść bez zapisywania?",
                        "Potwierdzenie wyjścia",
                        MessageBoxButton.YesNo,
                        MessageBoxImage.Warning);

                    if (wynik == MessageBoxResult.No)
                    {
                        // Anulujemy zamknięcie okna
                        e.Cancel = true;
                        txtStatus.Text = "Zamknięcie anulowane przez użytkownika.";
                    }
                    else
                    {
                        txtStatus.Text = "Zamykanie aplikacji...";
                        // e.Cancel pozostaje false, okno się zamknie
                    }
                }

                // Jeśli brak niezapisanych zmian – okno zamknie się bez pytania
            }

            private void BtnZapisz_Click(object sender, RoutedEventArgs e)
            {
                maNiezapisaneZmiany = false;
                chkNiezapisane.IsChecked = false;
                txtStatus.Text = "✓ Zapisano. Możesz bezpiecznie zamknąć aplikację.";
            }

            private void BtnZamknij_Click(object sender, RoutedEventArgs e)
            {
                // Wywołanie Close() wywoła zdarzenie Closing
                this.Close();
            }
        }
    }
}
```



```
}  
  
}
```

Kluczowe elementy:

- `Closing` — wywoływane przed zamknięciem okna (jeszcze można anulować)
- `Closed` — wywoływane po zamknięciu okna (za późno na anulowanie)
- `CancelEventArgs e` — argument z właściwością `Cancel`
- `e.Cancel = true` — anuluje zamknięcie okna
- `MessageBoxButton.YesNo` — okno dialogowe z przyciskami Tak/Nie
- `MessageBoxResult.No` — sprawdzamy który przycisk użytkownik kliknął

Podsumowanie rozdziałów 10–14

Poznałeś kompletny zestaw kontrolki wyboru (`CheckBox`, `RadioButton`, `ComboBox`, `ListBox`, `DataGrid`), naukę manipulacji suwakami i mieszania kolorów RGB, pracę z obrazami i obsługę błędów przy ładowaniu plików, kształty i elementy wizualne do budowania bogatego interfejsu, oraz pełny przegląd systemu zdarzeń WPF — od kliknięcia przycisku przez zdarzenia klawiatury aż po potwierdzenie zamknięcia okna.

15. Odczytywanie i ustawianie wartości kontrolki w C#

Każda kontrolka WPF przechowuje dane w swoich właściwościach. Żeby odczytać co wpisał użytkownik albo ustawić wartość programowo, musisz znać nazwy tych właściwości i sposób ich używania w C#. W tej sekcji zobaczysz jak to robić dla każdej popularnej kontrolki.

15.1. Text

`TextBox` przechowuje tekst w właściwości `Text`. Możesz ją odczytać, ustawić albo wyczyścić.

- Odczyt: `string tekst = MojTextBox.Text;`
- Zapis: `MojTextBox.Text = "Nowa wartość";`
- Czyszczenie: `MojTextBox.Text = "";` lub `MojTextBox.Clear();`
- `Trim()`: usuwa spacje z początku i końca — zawsze używaj przy odczycie

Poniższy przykład pokazuje formularz z polem tekstowym, gdzie po kliknięciu przycisku wyświetlasz wprowadzony tekst w Label, a drugi przycisk czyści pole.

MainWindow.xaml:

```
<Window x:Class="KontrolkiDemo.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="TextBox - odczyt i zapis" Height="300" Width="450">  
    <StackPanel Margin="20" VerticalAlignment="Center">  
        <Label Content="Wpisz swoje imię:" FontSize="14" />  
        <TextBox x:Name="TxtImie"  
            FontSize="14"  
            Padding="5"  
            Margin="0,5,0,10" />  
        <StackPanel Orientation="Horizontal">  
            <Button x:Name="BtnPokaz"  
                Content="Pokaż"  
                Width="100"  
                Height="35"  
                Margin="0,0,10,0"  
                Click="BtnPokaz_Click" />  
            <Button x:Name="BtnCzysc"  
                Content="Wyczyść"  
                Width="100"  
                Height="35"  
                Click="BtnCzysc_Click" />  
        </StackPanel>  
        <Label x:Name="LblWynik"  
            FontSize="14"  
            Foreground="DarkBlue"
```

```
        Margin="0,15,0,0"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace KontrolkiDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void BtnPokaz_Click(object sender, RoutedEventArgs e)
            {
                // Odczyt tekstu – zawsze używaj Trim() żeby usunąć spacje
                string imie = TxtImie.Text.Trim();

                if (imie == "")
                {
                    LblWynik.Content = "Nie wpisałeś imienia!";
                }
                else
                {
                    // Zapis do innej kontrolki
                    LblWynik.Content = "Cześć, " + imie + "!";
                }
            }

            private void BtnCzyszc_Click(object sender, RoutedEventArgs e)
            {
                // Czyszczenie pola
                TxtImie.Text = "";
                LblWynik.Content = "";

                // Przenieś fokus z powrotem na pole tekstowe
                TxtImie.Focus();
            }
        }
    }
}
```

Kluczowe elementy:

- `TxtImie.Text` — odczyt tekstu wpisanego przez użytkownika
- `.Trim()` — usuwa wiodące i końcowe spacje, zawsze stosuj przy walidacji
- `TxtImie.Text = ""` — ustawia puste pole
- `TxtImie.Clear()` — alternatywna metoda czyszczenia (robi to samo co przypisanie `""`)
- `TxtImie.Focus()` — przenosi kursor do pola po wyczyszczeniu

15.2. CheckBox

Właściwość `IsChecked` jest typu `bool?` (nullable bool), co oznacza że może mieć trzy wartości: `true`, `false`, lub `null`. Dzieje się tak bo CheckBox może być w stanie nieokreślonym (gdy `IsThreeState="True"`).

Do bezpiecznego odczytu używasz operatora `??` (null-coalescing) albo rzutowania `(bool)`.

MainWindow.xaml:

```
<Window x:Class="KontrolkiDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="CheckBox - odczyt" Height="350" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Wybierz opcje zamówienia:" FontSize="14" FontWeight="Bold"/>
        <CheckBox x:Name="ChkSos"
            Content="Dodatek: sos"
            FontSize="13"
            Margin="0,10,0,5"/>
        <CheckBox x:Name="ChkSalata"
            Content="Dodatek: sałata"
            FontSize="13"
            Margin="0,5,0,5"/>
        <CheckBox x:Name="ChkSer"
            Content="Dodatek: ser"
            FontSize="13"
            Margin="0,5,0,15"/>
        <Button Content="Zamów"
            Width="120"
            Height="35"
            HorizontalAlignment="Left"
            Click="BtnZamow_Click"/>
        <TextBlock x:Name="TxtWynik"
            FontSize="13"
            Foreground="DarkGreen"
            TextWrapping="Wrap"
            Margin="0,15,0,0"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace KontrolkiDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnZamow_Click(object sender, RoutedEventArgs e)
        {
            // Sposób 1: operator ?? (bezpieczny, zalecany)
            bool masSos = ChkSos.IsChecked ?? false;
            bool masSalata = ChkSalata.IsChecked ?? false;
            bool masSer = ChkSer.IsChecked ?? false;

            // Sposób 2: sprawdzenie == true (też poprawny)
            // bool masSos = ChkSos.IsChecked == true;

            // Sposób 3: rzutowanie (może rzucić wyjątek gdy null!)
            // bool masSos = (bool)ChkSos.IsChecked;

            string dodatki = "";
        }
    }
}
```

```

        if (masSos) dodatki += "sos, ";
        if (masSalata) dodatki += "sałata, ";
        if (masSer) dodatki += "ser, ";

        if (dodatki == "")
        {
            TxtWynik.Text = "Zamówiłeś burgera bez dodatków.";
        }
        else
        {
            // Usuń ostatni przecinek
            dodatki = dodatki.TrimEnd(',', ' ');
            TxtWynik.Text = "Zamówiłeś burgera z: " + dodatki + ".";
        }
    }
}
}
}

```

Kluczowe elementy:

- `ChkSos.IsChecked ?? false` — jeśli `IsChecked` jest `null`, zwraca `false`
- `ChkSos.IsChecked == true` — alternatywa, działa tak samo w praktyce
- `(bool)ChkSos.IsChecked` — działa tylko gdy wiesz, że nie jest null
- `IsChecked = true` — możesz też programowo zaznaczyć CheckBox

15.3. Button

RadioButton nie ma jednej wspólnej właściwości dla grupy. Musisz sprawdzić `IsChecked` każdego przycisku z osobna. Grupowanie odbywa się przez `GroupName` lub przez umieszczenie w tym samym panelu.

MainWindow.xaml:

```

<Window x:Class="KontrolkiDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="RadioButton - odczyt" Height="400" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Wybierz rozmiar pizzy:" FontSize="14" FontWeight="Bold"/>

        <RadioButton x:Name="RbMała"
            Content="Mała (25 cm) – 25 zł"
            FontSize="13"
            GroupName="Rozmiar"
            Margin="0,8,0,4"
            IsChecked="True"/>

        <RadioButton x:Name="RbSrednia"
            Content="Średnia (30 cm) – 35 zł"
            FontSize="13"
            GroupName="Rozmiar"
            Margin="0,4,0,4"/>

        <RadioButton x:Name="RbDuża"
            Content="Duża (40 cm) – 45 zł"
            FontSize="13"
            GroupName="Rozmiar"
            Margin="0,4,0,15"/>

        <Button Content="Zamów pizzę"
            Width="130"
            Height="35"
            HorizontalAlignment="Left"
            Click="BtnZamow_Click"/>
    </StackPanel>
</Window>

```

```

        <Label x:Name="LblWynik"
               FontSize="14"
               Foreground="DarkRed"
               Margin="0,15,0,0" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace KontrolkiDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void BtnZamow_Click(object sender, RoutedEventArgs e)
            {
                // Sprawdzasz każdy RadioButton z osobna
                string rozmiar = "";
                int cena = 0;

                if (RbMała.IsChecked == true)
                {
                    rozmiar = "Mała (25 cm)";
                    cena = 25;
                }
                else if (RbŚrednia.IsChecked == true)
                {
                    rozmiar = "Średnia (30 cm)";
                    cena = 35;
                }
                else if (RbDuża.IsChecked == true)
                {
                    rozmiar = "Duża (40 cm)";
                    cena = 45;
                }

                LblWynik.Content = $"Zamówiono: {rozmiar} – {cena} zł";
            }
        }
    }
}

```

Kluczowe elementy:

- `GroupName="Rozmiar"` — łączy RadioButtony w grupę, tylko jeden może być zaznaczony
- `IsChecked == true` — sprawdzasz każdy przycisk z osobna w if/else if
- `IsChecked="True"` w XAML — ustawia domyślnie zaznaczony przycisk
- Kolejność `if/else if` — gdy żaden nie jest zaznaczony, `rozmiar` zostaje pusty

15.4. ComboBox

`ComboBox` udostępnia dwie główne właściwości do odczytu wyboru:

- `SelectedItem` — zwraca wybrany obiekt (ComboBoxItem lub string) — wymaga rzutowania
- `SelectedIndex` — zwraca numer (indeks) wybranego elementu, od zera; `-1` gdy nic nie wybrano

MainWindow.xaml:

```
<Window x:Class="KontrolkiDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ComboBox - odczyt" Height="380" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Wybierz województwo:" FontSize="14" FontWeight="Bold"/>
        <ComboBox x:Name="CmbWojewodztwo"
                FontSize="13"
                Height="32"
                Margin="0,5,0,10">
            <ComboBoxItem Content="Małopolskie"/>
            <ComboBoxItem Content="Mazowieckie"/>
            <ComboBoxItem Content="Śląskie"/>
            <ComboBoxItem Content="Wielkopolskie"/>
            <ComboBoxItem Content="Dolnośląskie"/>
        </ComboBox>

        <Label Content="Wybierz rok:" FontSize="14" FontWeight="Bold" Margin="0,10,0,0"/>
        <ComboBox x:Name="CmbRok"
                FontSize="13"
                Height="32"
                Margin="0,5,0,15"
                SelectedIndex="0">
            <ComboBoxItem Content="2023"/>
            <ComboBoxItem Content="2024"/>
            <ComboBoxItem Content="2025"/>
            <ComboBoxItem Content="2026"/>
        </ComboBox>

        <Button Content="Potwierdź"
                Width="120"
                Height="35"
                HorizontalAlignment="Left"
                Click="BtnPotwierdz_Click"/>

        <TextBlock x:Name="TxtWynik"
                FontSize="13"
                Foreground="Navy"
                Margin="0,15,0,0"
                TextWrapping="Wrap"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;
using System.Windows.Controls;

namespace KontrolkiDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnPotwierdz_Click(object sender, RoutedEventArgs e)
```

```

{
    // Odczyt przez SelectedItem – trzeba sprawdzić null i rzutować
    string wojewodztwo = "";
    if (CmbWojewodztwo.SelectedItem != null)
    {
        // SelectedItem zwraca ComboBoxItem, pobieramy Content
        ComboBoxItem wybrany = (ComboBoxItem)CmbWojewodztwo.SelectedItem;
        wojewodztwo = wybrany.Content.ToString();
    }
    else
    {
        TxtWynik.Text = "Wybierz województwo!";
        return;
    }

    // Odczyt przez SelectedIndex – prostszy, zwraca int
    int indeksRoku = CmbRok.SelectedIndex;
    string rok = "";
    if (indeksRoku >= 0)
    {
        ComboBoxItem wybranyRok = (ComboBoxItem)CmbRok.SelectedItem;
        rok = wybranyRok.Content.ToString();
    }

    TxtWynik.Text = $"Wybrano: {wojewodztwo}, rok {rok}\n" +
        $"Indeks województwa: {CmbWojewodztwo.SelectedIndex}\n" +
        $"Indeks roku: {indeksRoku}";

    // Ustawianie programowe – przez indeks
    // CmbRok.SelectedIndex = 2; // wybierze "2025"

    // Ustawianie przez wartość (dla ItemsSource z listą stringów):
    // CmbWojewodztwo.SelectedItem = "Śląskie";
}
}
}

```

Kluczowe elementy:

- `SelectedItem != null` — zawsze sprawdzaj przed rzutowaniem
- `(ComboBoxItem)SelectedItem` — rzutowanie gdy elementy to ComboBoxItem
- `wybrany.Content.ToString()` — pobiera tekst z ComboBoxItem
- `SelectedIndex` — prostszy sposób, zwraca `-1` gdy brak wyboru, `0` dla pierwszego
- `SelectedIndex = 0` w XAML — ustawi domyślny wybór

15.5. Slider

`Slider` zwraca wartość jako `double`. Często potrzebujesz liczby całkowitej — używasz rzutowania `(int)` lub `Math.Round()`.

MainWindow.xaml:

```

<Window x:Class="KontrolkiDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Slider - odczyt wartości" Height="320" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Wybierz wiek (1-100):" FontSize="14" FontWeight="Bold"/>
        <Slider x:Name="SliderWiek"
              Minimum="1"
              Maximum="100"
              Value="25"
              TickFrequency="10"

```

```

        IsSnapToTickEnabled="False"
        AutoToolTipPlacement="TopLeft"
        Margin="0,5,0,5"
        ValueChanged="SliderWiek_ValueChanged" />
<Label x:Name="LblWiek"
        Content="Wiek: 25"
        FontSize="16"
        Foreground="DarkBlue"
        HorizontalAlignment="Center"
        Margin="0,0,0,15" />

<Label Content="Wybierz głośność (0-100):" FontSize="14" FontWeight="Bold" />
<Slider x:Name="SliderGlosnosc"
        Minimum="0"
        Maximum="100"
        Value="50"
        Margin="0,5,0,5" />
<Label x:Name="LblGlosnosc"
        Content="Głośność: 50%"
        FontSize="14"
        HorizontalAlignment="Center"
        Margin="0,0,0,15" />

<Button Content="Zatwierdź"
        Width="120"
        Height="35"
        HorizontalAlignment="Left"
        Click="BtnZatwierdz_Click" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace KontrolkiDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // Aktualizacja etykiety podczas przesuwania suwaka
            private void SliderWiek_ValueChanged(object sender,
                System.Windows.RoutedPropertyChangedEventArgs<double> e)
            {
                // Konwersja double na int - rzutowanie uciną część ułamkową
                int wiek = (int)SliderWiek.Value;
                LblWiek.Content = "Wiek: " + wiek;
            }

            private void BtnZatwierdz_Click(object sender, RoutedEventArgs e)
            {
                // Sposób 1: rzutowanie (int) - uciną część ułamkową (nie zaokrągla)
                int wiek = (int)SliderWiek.Value;

                // Sposób 2: Math.Round - zaokrągla do najbliższej liczby całkowitej
                int glosnosc = (int)Math.Round(SliderGlosnosc.Value);
            }
        }
    }
}

```



```

// Sposób 3: Convert.ToInt32 – zaokrągla (jak Math.Round)
// int wiek = Convert.ToInt32(SliderWiek.Value);

MessageBox.Show(
    $"Wiek: {wiek} lat\nGłośność: {glosnosc}%",
    "Wybrane wartości",
    MessageBoxButton.OK,
    MessageBoxImage.Information
);

// Ustawianie wartości Slider programowo
// SliderWiek.Value = 30.0;
}
}
}

```

Kluczowe elementy:

- `SliderWiek.Value` — zwraca `double`
- `(int)SliderWiek.Value` — rzutowanie, ucina część ułamkową (np. 4.9 → 4)
- `Math.Round()` — zaokrągla matematycznie (4.5 → 5, 4.4 → 4)
- `ValueChanged` — zdarzenie wywoływane przy każdej zmianie wartości suwaka
- `AutoToolTipPlacement="TopLeft"` — pokazuje aktualną wartość nad suwakiem

15.6. TextBlock

`Label` i `TextBlock` mają podobne zastosowanie — wyświetlają tekst. Różnica:

- `Label` używa właściwości `Content` (typ `object`)
- `TextBlock` używa właściwości `Text` (typ `string`)

MainWindow.xaml:

```

<Window x:Class="KontrolkiDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Label i TextBlock" Height="350" Width="500">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <TextBox x:Name="TxtWejscie"
                FontSize="13"
                Padding="5"
                Margin="0,0,0,10" />

        <Button Content="Aktualizuj"
                Height="35"
                Click="BtnAktualizuj_Click"
                Margin="0,0,0,20" />

        <!-- Label - właściwość Content -->
        <Label Content="Tutaj pojawi się tekst w Label:"
                FontWeight="Bold"
                FontSize="13" />

        <Label x:Name="LblWynik"
                Content="(brak)"
                FontSize="14"
                Foreground="DarkBlue"
                Background="LightCyan"
                Padding="8"
                Margin="0,0,0,15" />

        <!-- TextBlock - właściwość Text -->
    
```

```

        <TextBlock Text="Tutaj pojawi się tekst w TextBlock:"
                FontWeight="Bold"
                FontSize="13" />

        <TextBlock x:Name="TbWynik"
                Text="(brak)"
                FontSize="14"
                Foreground="DarkGreen"
                Background="LightYellow"
                Padding="8"
                TextWrapping="Wrap" />

    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace KontrolkiDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void BtnAktualizuj_Click(object sender, RoutedEventArgs e)
            {
                string tekst = TxtWejscie.Text.Trim();

                if (tekst == "")
                {
                    // Czyszczenie
                    LblWynik.Content = "(brak)";
                    TbWynik.Text = "(brak)";
                }
                else
                {
                    // Ustawianie Label.Content – przyjmuje object
                    LblWynik.Content = "Label: " + tekst;

                    // Ustawianie TextBlock.Text – przyjmuje string
                    TbWynik.Text = "TextBlock: " + tekst;

                    // Można też ustawiać inne właściwości
                    LblWynik.FontSize = tekst.Length > 10 ? 12 : 16;
                    TbWynik.FontWeight = tekst.Length > 10
                        ? System.Windows.FontWeights.Bold
                        : System.Windows.FontWeights.Normal;
                }
            }
        }
    }
}

```

Kluczowe elementy:

- **Label.Content** — typ **object**, przyjmuje string, liczbę, a nawet inne kontrolki
- **TextBlock.Text** — typ **string**, tylko tekst
- **TextWrapping="Wrap"** — TextBlock może zawiązać długi tekst, Label nie
- **Label** ma margines wewnętrzny i obsługę akceleratorów klawiszowych (podkreślony znak)

15.7. Source

Kontrolka `Image` wyświetla obrazek. Właściwość `Source` przyjmuje obiekt `BitmapImage`. Możesz ustawiać zdjęcie programowo lub sprawdzić czy jest załadowane.

MainWindow.xaml:

```
<Window x:Class="KontrolkiDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Image - ustawianie Source" Height="400" Width="500">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Image x:Name="ImgPodglad"
              Width="200"
              Height="150"
              Stretch="Uniform"
              Margin="0,0,0,15"
              HorizontalAlignment="Left" />

        <TextBox x:Name="TxtSciezka"
                Text="C:\Users\Public\Pictures\Sample Pictures\Tulips.jpg"
                FontSize="12"
                Padding="5"
                Margin="0,0,0,10" />

        <StackPanel Orientation="Horizontal">
            <Button Content="Załaduj"
                    Width="100"
                    Height="35"
                    Margin="0,0,10,0"
                    Click="BtnZaladuj_Click" />
            <Button Content="Wyczyść"
                    Width="100"
                    Height="35"
                    Click="BtnWyczysc_Click" />
        </StackPanel>

        <Label x:Name="LblStatus"
               FontSize="13"
               Foreground="DarkRed"
               Margin="0,10,0,0" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.IO;
using System.Windows;
using System.Windows.Media.Imaging;

namespace KontrolkiDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnZaladuj_Click(object sender, RoutedEventArgs e)
```

```

{
    string sciezka = TxtSciezka.Text.Trim();

    if (sciezka == "")
    {
        LblStatus.Content = "Podaj ścieżkę do pliku!";
        return;
    }

    if (!File.Exists(sciezka))
    {
        LblStatus.Content = "Plik nie istnieje!";
        return;
    }

    try
    {
        // Tworzenie BitmapImage ze ścieżki
        BitmapImage bmp = new BitmapImage();
        bmp.BeginInit();
        bmp.UriSource = new Uri(sciezka, UriKind.Absolute);
        bmp.CacheOption = BitmapCacheOption.OnLoad;
        bmp.EndInit();

        // Ustawianie Source obrazka
        ImgPodglad.Source = bmp;
        LblStatus.Content = "Załadowano: " + Path.GetFileName(sciezka);
    }
    catch (Exception ex)
    {
        LblStatus.Content = "Błąd: " + ex.Message;
        ImgPodglad.Source = null;
    }
}

private void BtnWyczyszc_Click(object sender, RoutedEventArgs e)
{
    // Czyszczenie obrazka – ustawiasz Source na null
    ImgPodglad.Source = null;
    LblStatus.Content = "Wyczyszczono.";

    // Sprawdzanie czy obrazek jest załadowany
    // if (ImgPodglad.Source != null) { ... }
}
}
}

```

Kluczowe elementy:

- `BitmapImage` — klasa reprezentująca obrazek bitmapowy
- `bmp.UriSource` — ścieżka do pliku jako `Uri`
- `CacheOption = BitmapCacheOption.OnLoad` — zwalnia plik po załadowaniu
- `ImgPodglad.Source = null` — usuwa obrazek
- `try/catch` — plik może być uszkodzony lub nieobsługiwany format

15.8. Background

Możesz zmieniać kolory tła i tekstu kontrolki programowo. Używasz klasy `SolidColorBrush` i struktury `Color`.

MainWindow.xaml:

```

<Window x:Class="KontrolkiDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zmiana kolorów" Height="420" Width="500">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Podaj wartości RGB (0-255):" FontSize="14" FontWeight="Bold"/>

        <StackPanel Orientation="Horizontal" Margin="0,10,0,5">
            <Label Content="R:" Width="25" FontSize="13"/>
            <TextBox x:Name="TxtR" Text="255" Width="60" FontSize="13" Padding="3"/>
            <Label Content="G:" Width="25" FontSize="13" Margin="10,0,0,0"/>
            <TextBox x:Name="TxtG" Text="165" Width="60" FontSize="13" Padding="3"/>
            <Label Content="B:" Width="25" FontSize="13" Margin="10,0,0,0"/>
            <TextBox x:Name="TxtB" Text="0" Width="60" FontSize="13" Padding="3"/>
        </StackPanel>

        <Button Content="Zastosuj kolor"
                Height="35"
                Margin="0,10,0,15"
                HorizontalAlignment="Left"
                Width="150"
                Click="BtnKolor_Click"/>

        <Border x:Name="BrdPodglad"
                Height="80"
                Background="White"
                BorderBrush="Gray"
                BorderThickness="1"
                CornerRadius="5"
                Margin="0,0,0,10">
            <Label x:Name="LblPodglad"
                    Content="Podglad koloru"
                    HorizontalAlignment="Center"
                    VerticalAlignment="Center"
                    FontSize="16"
                    FontWeight="Bold"/>
        </Border>

        <Label x:Name="LblKodHex"
                Content="HEX: #FFA500"
                FontSize="13"
                Foreground="Gray"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Media;

namespace KontrolkiDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

```

private void BtnKolor_Click(object sender, RoutedEventArgs e)
{
    // Parsowanie wartości RGB
    if (!byte.TryParse(TxtR.Text, out byte r) ||
        !byte.TryParse(TxtG.Text, out byte g) ||
        !byte.TryParse(TxtB.Text, out byte b))
    {
        MessageBox.Show("Podaj liczby całkowite od 0 do 255!",
            "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // Tworzenie koloru z komponentów RGB
    Color kolor = Color.FromRgb(r, g, b);

    // Tworzenie pędzla jednokolorowego
    SolidColorBrush pedzel = new SolidColorBrush(kolor);

    // Ustawianie tła kontrolki
    BrdPodglad.Background = pedzel;

    // Automatyczny kontrast tekstu (ciemny/jasny)
    double jasnoc = 0.299 * r + 0.587 * g + 0.114 * b;
    LblPodglad.Foreground = jasnoc > 128
        ? new SolidColorBrush(Colors.Black)
        : new SolidColorBrush(Colors.White);

    // Wyświetl kod HEX
    LblKodHex.Content = $"HEX: #{r:X2}-{g:X2}-{b:X2}";

    // Inne predefiniowane kolory:
    // BrdPodglad.Background = new SolidColorBrush(Colors.Red);
    // BrdPodglad.Background = Brushes.Blue;
}
}
}

```

Kluczowe elementy:

- `Color.FromRgb(r, g, b)` — tworzy kolor z trzech składowych (bajty 0–255)
- `new SolidColorBrush(kolor)` — tworzy pędzel jednokolorowy
- `kontrolka.Background = pedzel` — ustawia tło
- `Colors.Red` — predefiniowane kolory w klasie `Colors`
- `Brushes.Blue` — gotowe pędzle w klasie `Brushes`
- `{r:X2}` — formatowanie do hex, zawsze dwie cyfry

16. Formularze i pobieranie danych od użytkownika

Formularz to zestaw kontroltek, który pozwala użytkownikowi wprowadzić dane. W WPF budujesz formularze używając Grid z etykietami i polami wejściowymi. W tej sekcji zobaczysz jak zbudować kompletny formularz i poprawnie zebrać dane ze wszystkich typów kontroltek.

16.1. Budowa prostego formularza

Dobrze zbudowany formularz ma:

- Etykiety opisujące każde pole (kolumna lewa)
- Kontrolki do wprowadzania danych (kolumna prawa)
- Przyciski akcji (Zapisz, Anuluj, Wyczyść)
- Miejsce na komunikat wyniku

Poniżej zobaczysz kompletny formularz pracownika z wieloma rodzajami kontroltek.

MainWindow.xaml:

```
<Window x:Class="FormularzDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Formularz Pracownika" Height="620" Width="580">
    <ScrollViewer VerticalScrollBarVisibility="Auto">
        <StackPanel Margin="25">

            <!-- Nagłówek -->
            <TextBlock Text="Formularz danych pracownika"
                      FontSize="18"
                      FontWeight="Bold"
                      Margin="0,0,0,20"
                      HorizontalAlignment="Center" />

            <!-- Formularz jako Grid -->
            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="150" />
                    <ColumnDefinition Width="*" />
                </Grid.ColumnDefinitions>
                <Grid.RowDefinitions>
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                </Grid.RowDefinitions>

                <!-- Imię -->
                <Label Grid.Row="0" Grid.Column="0"
                      Content="Imię:"
                      VerticalAlignment="Center"
                      FontSize="13" />
                <TextBox x:Name="TxtImie"
                        Grid.Row="0" Grid.Column="1"
                        FontSize="13"
                        Padding="5"
                        Margin="0,4,0,4" />

                <!-- Nazwisko -->
                <Label Grid.Row="1" Grid.Column="0"
                      Content="Nazwisko:"
                      VerticalAlignment="Center"
                      FontSize="13" />
                <TextBox x:Name="TxtNazwisko"
                        Grid.Row="1" Grid.Column="1"
                        FontSize="13"
                        Padding="5"
                        Margin="0,4,0,4" />

                <!-- Wiek (Slider) -->
                <Label Grid.Row="2" Grid.Column="0"
                      Content="Wiek:"
                      VerticalAlignment="Center"
                      FontSize="13" />
                <StackPanel Grid.Row="2" Grid.Column="1">
```

```

        Orientation="Horizontal"
        Margin="0,4,0,4">
<Slider x:Name="SliderWiek"
        Minimum="18"
        Maximum="70"
        Value="30"
        Width="220"
        VerticalAlignment="Center"
        ValueChanged="SliderWiek_ValueChanged" />
<Label x:Name="LblWiekWartosc"
        Content="30"
        Width="40"
        FontSize="13"
        VerticalAlignment="Center" />
</StackPanel>

```

←!— Dział (ComboBox) →

```

<Label Grid.Row="3" Grid.Column="0"
        Content="Dział:"
        VerticalAlignment="Center"
        FontSize="13" />
<ComboBox x:Name="CmbDzial"
        Grid.Row="3" Grid.Column="1"
        FontSize="13"
        Height="32"
        Margin="0,4,0,4">
    <ComboBoxItem Content="IT" />
    <ComboBoxItem Content="Sprzedaz" />
    <ComboBoxItem Content="HR" />
    <ComboBoxItem Content="Finanse" />
    <ComboBoxItem Content="Marketing" />
</ComboBox>

```

←!— Stanowisko (RadioButton) →

```

<Label Grid.Row="4" Grid.Column="0"
        Content="Stanowisko:"
        VerticalAlignment="Top"
        Margin="0,8,0,0"
        FontSize="13" />
<StackPanel Grid.Row="4" Grid.Column="1"
        Margin="0,6,0,4">
    <RadioButton x:Name="RbJunior"
        Content="Junior"
        FontSize="13"
        GroupName="Stanowisko"
        IsChecked="True"
        Margin="0,2" />
    <RadioButton x:Name="RbMid"
        Content="Mid"
        FontSize="13"
        GroupName="Stanowisko"
        Margin="0,2" />
    <RadioButton x:Name="RbSenior"
        Content="Senior"
        FontSize="13"
        GroupName="Stanowisko"
        Margin="0,2" />
</StackPanel>

```

←!— Opcje dodatkowe (CheckBox) →

```

<Label Grid.Row="5" Grid.Column="0"
        Content="Opcje:"

```



```

        VerticalAlignment="Top"
        Margin="0,8,0,0"
        FontSize="13" />
<StackPanel Grid.Row="5" Grid.Column="1"
    Margin="0,6,0,4">
    <CheckBox x:Name="ChkZdalny"
        Content="Praca zdalna"
        FontSize="13"
        Margin="0,2" />
    <CheckBox x:Name="ChkUbezpieczenie"
        Content="Ubezpieczenie grupowe"
        FontSize="13"
        IsChecked="True"
        Margin="0,2" />
    <CheckBox x:Name="ChkSamochod"
        Content="Samochód służbowy"
        FontSize="13"
        Margin="0,2" />
</StackPanel>

<!-- Uwagi (TextBox wieloliniowy) -->
<Label Grid.Row="6" Grid.Column="0"
    Content="Uwagi:"
    VerticalAlignment="Top"
    Margin="0,6,0,0"
    FontSize="13" />
<TextBox x:Name="TxtUwagi"
    Grid.Row="6" Grid.Column="1"
    FontSize="12"
    Padding="5"
    Height="70"
    TextWrapping="Wrap"
    AcceptsReturn="True"
    VerticalScrollBarVisibility="Auto"
    Margin="0,4,0,4" />

<!-- Przyciski -->
<StackPanel Grid.Row="7" Grid.Column="1"
    Orientation="Horizontal"
    Margin="0,15,0,0">
    <Button x:Name="BtnZapisz"
        Content="Zapisz"
        Width="100"
        Height="35"
        Margin="0,0,10,0"
        Background="DarkGreen"
        Foreground="White"
        FontSize="13"
        Click="BtnZapisz_Click" />
    <Button x:Name="BtnCzysc"
        Content="Wyczyść"
        Width="100"
        Height="35"
        Background="DarkOrange"
        Foreground="White"
        FontSize="13"
        Click="BtnCzysc_Click" />
</StackPanel>

<!-- Wynik -->
<TextBlock x:Name="TbWynik"
    Grid.Row="8" Grid.Column="0"

```

```

                Grid.ColumnSpan="2"
                FontSize="12"
                Foreground="DarkBlue"
                TextWrapping="Wrap"
                Margin="0,15,0,0"
                Background="LightBlue"
                Padding="10"/>
            </Grid>
        </StackPanel>
    </ScrollViewer>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Text;
using System.Windows;
using System.Windows.Controls;

namespace FormularzDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void SliderWiek_ValueChanged(object sender,
                RoutedPropertyChangedEventArgs<double> e)
            {
                // Aktualizuj etykietę obok suwaka (sprawdź czy kontrolka już istnieje)
                if (LblWiekWartosc != null)
                {
                    LblWiekWartosc.Content = ((int)SliderWiek.Value).ToString();
                }

                private void BtnZapisz_Click(object sender, RoutedEventArgs e)
                {
                    // 16.2 Pobieranie tekstu z TextBox
                    string imie = TxtImie.Text.Trim();
                    string nazwisko = TxtNazwisko.Text.Trim();

                    if (imie == "" || nazwisko == "")
                    {
                        MessageBox.Show("Imię i nazwisko są wymagane!",
                            "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
                        return;
                    }

                    // 16.6 Pobieranie wartości ze Slider
                    int wiek = (int)SliderWiek.Value;

                    // 16.5 Pobieranie wyboru z ComboBox
                    string dzial = "";
                    if (CmbDzial.SelectedItem != null)
                    {
                        ComboBoxItem wybranyDzial = (ComboBoxItem)CmbDzial.SelectedItem;
                        dzial = wybranyDzial.Content.ToString();
                    }
                    else
                    {

```

```

        MessageBox.Show("Wybierz dział!",
            "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // 16.4 Pobieranie wyboru z RadioButton
    string stanowisko = "";
    if (RbJunior.IsChecked == true) stanowisko = "Junior";
    else if (RbMid.IsChecked == true) stanowisko = "Mid";
    else if (RbSenior.IsChecked == true) stanowisko = "Senior";

    // 16.3 Pobieranie zaznaczenia z CheckBox
    bool pracaZdalna = ChkZdalny.IsChecked ?? false;
    bool ubezpieczenie = ChkUbezpieczenie.IsChecked ?? false;
    bool samochod = ChkSamochod.IsChecked ?? false;

    // Notatki
    string uwagi = TxtUwagi.Text.Trim();

    // Budowanie podsumowania
    StringBuilder sb = new StringBuilder();
    sb.AppendLine($"Pracownik: {imie} {nazwisko}");
    sb.AppendLine($"Wiek: {wiek} lat");
    sb.AppendLine($"Dział: {dzial}, Stanowisko: {stanowisko}");
    sb.Append("Benefity: ");
    if (pracaZdalna) sb.Append("praca zdalna, ");
    if (ubezpieczenie) sb.Append("ubezpieczenie, ");
    if (samochod) sb.Append("samochód, ");
    sb.AppendLine();
    if (uwagi != "") sb.AppendLine($"Uwagi: {uwagi}");

    TbWynik.Text = sb.ToString();
}

private void BtnCzysc_Click(object sender, RoutedEventArgs e)
{
    // 16.7 Czyszczenie formularza
    TxtImie.Text = "";
    TxtNazwisko.Text = "";
    SliderWiek.Value = 30;
    CmbDzial.SelectedIndex = -1;
    RbJunior.IsChecked = true;
    ChkZdalny.IsChecked = false;
    ChkUbezpieczenie.IsChecked = true;
    ChkSamochod.IsChecked = false;
    TxtUwagi.Text = "";
    TbWynik.Text = "";

    TxtImie.Focus();
}
}
}

```

16.2. Text

Odczyt tekstu z `TextBox` to podstawa każdego formularza. Zawsze stosuj `Trim()` i sprawdzaj puste pole.

```

// Odczyt i oczyszczenie
string imie = TxtImie.Text.Trim();

```

```
// Sprawdzenie pustego pola
if (imie == "")
{
    MessageBox.Show("Pole Imię jest wymagane!");
    TxtImie.Focus(); // przenieś kursor do pola
    return;
}

// Alternatywnie: string.IsNullOrWhiteSpace (bezpieczniejsze)
if (string.IsNullOrWhiteSpace(TxtImie.Text))
{
    MessageBox.Show("Imię nie może być puste!");
    return;
}
```

Kiedy używać którego sposobu:

SPOSÓB	KIEDY
<code>Text.Trim() == ""</code>	Standardowe sprawdzenie
<code>string.IsNullOrWhiteSpace(Text)</code>	Gdy Text może być null
<code>Text.Length == 0</code>	Gdy nie chcesz usuwać spacji

16.3. CheckBox

```
// Sposób podstawowy – operator ??
bool pracaZdalna = ChkZdalny.IsChecked ?? false;

// Sposób alternatywny – porównanie == true
bool ubezpieczenie = (ChkUbezpieczenie.IsChecked == true);

// Obsługa gdy żadna opcja nie jest wymagana
if (!pracaZdalna && !ubezpieczenie)
{
    // Żaden benefit nie wybrany – to jest OK, nie błąd
    MessageBox.Show("Nie wybrano żadnych benefitów.");
}

// Ustawianie programowo
ChkZdalny.IsChecked = true; // zaznacz
ChkZdalny.IsChecked = false; // odznacz
```

16.4. Button

```
// Sprawdzasz każdy RadioButton z osobna
string stanowisko = "";
if (RbJunior.IsChecked == true)
    stanowisko = "Junior";
else if (RbMid.IsChecked == true)
    stanowisko = "Mid";
else if (RbSenior.IsChecked == true)
    stanowisko = "Senior";

// Walidacja gdy żaden nie zaznaczony (rzadkie, bo zazwyczaj jeden jest domyślny)
if (stanowisko == "")
{
    MessageBox.Show("Wybierz stanowisko!");
}
```

```
    return;  
}
```

16.5. ComboBox

```
// Sprawdź null przed rzutowaniem  
if (CmbDzial.SelectedItem == null)  
{  
    MessageBox.Show("Wybierz dział!");  
    return;  
}  
  
// Rzutowanie na ComboBoxItem  
ComboBoxItem wybrany = (ComboBoxItem)CmbDzial.SelectedItem;  
string dzial = wybrany.Content.ToString();  
  
// Alternatywnie przez SelectedIndex  
int indeks = CmbDzial.SelectedIndex;  
if (indeks < 0)  
{  
    MessageBox.Show("Wybierz dział!");  
    return;  
}  
  
// Dla ComboBox wypełnionego stringami (ItemsSource = lista):  
// string dzial = CmbDzial.SelectedItem as string ?? "";
```

16.6. Slider

```
// Odczyt jako double  
double wartoscDouble = SliderWiek.Value;  
  
// Konwersja na int – rzutowanie (ucina część ułamkową)  
int wiek = (int)SliderWiek.Value;  
  
// Konwersja na int – zaokrąglenie matematyczne  
int glosnosc = (int)Math.Round(SliderGlosnosc.Value);  
  
// Slider zawsze ma wartość (między Minimum a Maximum), nie trzeba sprawdzać null  
// Ale warto sprawdzić zakres jeśli Minimum/Maximum były zmieniane dynamicznie  
if (wiek < 18 || wiek > 70)  
{  
    MessageBox.Show("Wiek poza zakresem!");  
}
```

16.7. Czyszczenie formularza po akcji

Czyszczenie formularza po zapisaniu danych to dobra praktyka. Resetujesz każdą kontrolkę do wartości domyślnej.

MainWindow.xaml:

```
<Window x:Class="FormularzDemo.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Reset formularza" Height="380" Width="450">  
    <StackPanel Margin="20">
```

```

<TextBox x:Name="TxtNazwa"
        FontSize="13" Padding="5" Margin="0,5"/>

<ComboBox x:Name="CmbKategoria" Height="30" FontSize="13" Margin="0,5">
    <ComboBoxItem Content="Elektronika"/>
    <ComboBoxItem Content="Odzież"/>
    <ComboBoxItem Content="Jedzenie"/>
</ComboBox>

<CheckBox x:Name="ChkDostepny" Content="Dostępny w sklepie"
        FontSize="13" Margin="0,8"/>

<Slider x:Name="SliderCena" Minimum="0" Maximum="1000"
        Value="100" Margin="0,5"/>

<Label x:Name="LblCena" Content="Cena: 100 zł" FontSize="13"/>

<StackPanel Orientation="Horizontal" Margin="0,15">
    <Button Content="Dodaj produkt" Width="130" Height="35"
        Margin="0,0,10,0" Click="BtnDodaj_Click"/>
    <Button Content="Wyczyść" Width="100" Height="35"
        Click="BtnReset_Click"/>
</StackPanel>

<TextBlock x:Name="TbPotwierdzenie" FontSize="13"
        Foreground="DarkGreen" TextWrapping="Wrap"/>

</StackPanel>

</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Controls;

namespace FormularzDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            SliderCena.ValueChanged += (s, e) =>
            {
                if (LblCena != null)
                    LblCena.Content = $"Cena: {(int)SliderCena.Value} zł";
            };
        }

        private void BtnDodaj_Click(object sender, RoutedEventArgs e)
        {
            string nazwa = TxtNazwa.Text.Trim();
            if (nazwa == "")
            {
                MessageBox.Show("Podaj nazwę produktu!");
                return;
            }

            string kategoria = CmbKategoria.SelectedItem != null
                ? ((ComboBoxItem)CmbKategoria.SelectedItem).Content.ToString()
                : "bez kategorii";

            int cena = (int)SliderCena.Value;
            bool dostepny = ChkDostepny.IsChecked ?? false;

            TbPotwierdzenie.Text =
                $"Dodano: {nazwa} | Kat: {kategoria} | Cena: {cena} zł" +

```

```

        $" | {(dostepny ? "Dostępny" : "Niedostępny")}";

        // Po dodaniu – wyczyść formularz
        WyczyszcFormularz();
    }

    private void BtnReset_Click(object sender, RoutedEventArgs e)
    {
        WyczyszcFormularz();
        TbPotwierdzenie.Text = "";
    }

    private void WyczyszcFormularz()
    {
        // Resetuj każdą kontrolkę do wartości domyślnej
        TxtNazwa.Text = "";
        CmbKategoria.SelectedIndex = -1; // brak wyboru
        ChkDostepny.IsChecked = false;
        SliderCena.Value = 100; // wartość domyślna
        TxtNazwa.Focus(); // fokus na pierwsze pole
    }
}

```

Kluczowe elementy:

- `WyczyszcFormularz()` — osobna metoda czyszczenia, którą możesz wywołać z wielu miejsc
- `SelectedIndex = -1` — dla ComboBox: brak wybranego elementu
- `IsChecked = false` — odznaczyć CheckBox
- `Slider.Value = 100` — przywróć wartość domyślną
- `TxtNazwa.Focus()` — przenieś kursor do pierwszego pola po resecie

17. Walidacja danych

Walidacja to sprawdzanie czy dane wprowadzone przez użytkownika są poprawne przed ich użyciem. Nigdy nie zakładaj, że użytkownik poda właściwe dane — może wpisać litery zamiast cyfr, zostawić puste pola, albo podać wartość poza zakresem. W tej sekcji nauczysz się jak chronić program przed takimi sytuacjami.

17.1. Czym jest walidacja

Walidacja to zestaw sprawdzeń wykonywanych zanim program przetworzy dane. Bez niej program może się wysypać lub pokazać błędne wyniki.

Przykłady sytuacji bez walidacji:

- Pole "wiek" jest puste → `int.Parse("")` rzuca wyjątek
- Pole "kod pocztowy" zawiera "abcde" → błędny format
- Slider ustawiony na 0 → dzielenie przez zero

Kiedy walidować:

- Przed każdą operacją na danych z kontroltek
- Przy każdym kliknięciu przycisku "Zapisz", "Oblicz", "Wyślij"
- Możesz też walidować na bieżąco (`TextChanged` , `LostFocus`)

17.2. Sprawdzanie pustych pól

Najczęstsza walidacja — czy pole nie jest puste.

MainWindow.xaml:

```

<Window x:Class="WalidacjaDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

```

```

        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Walidacja pustych pól" Height="350" Width="450">
<StackPanel Margin="20" VerticalAlignment="Center">
    <Label Content="Imię:" FontSize="13"/>
    <TextBox x:Name="TxtImie" FontSize="13" Padding="5" Margin="0,3,0,10"/>

    <Label Content="Email:" FontSize="13"/>
    <TextBox x:Name="TxtEmail" FontSize="13" Padding="5" Margin="0,3,0,10"/>

    <Label Content="Opis:" FontSize="13"/>
    <TextBox x:Name="TxtOpis" FontSize="13" Padding="5"
        Height="60" TextWrapping="Wrap" AcceptsReturn="True"
        Margin="0,3,0,15"/>

    <Button Content="Sprawdź"
        Height="35"
        Click="BtnSprawdz_Click"/>

    <TextBlock x:Name="TbBled"
        FontSize="13"
        Foreground="Red"
        TextWrapping="Wrap"
        Margin="0,10,0,0"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WalidacjaDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnSprawdz_Click(object sender, RoutedEventArgs e)
        {
            TbBled.Text = ""; // wyczyść poprzednie błędy

            // Sposób 1: porównanie z pustym stringiem (po Trim)
            string imie = TxtImie.Text.Trim();
            if (imie == "")
            {
                TbBled.Text = "Błąd: Pole Imię jest wymagane!";
                TxtImie.Focus();
                return;
            }

            // Sposób 2: string.IsNullOrEmpty – najlepszy do walidacji
            // Obsługuje null, "", " " (same spacje)
            if (string.IsNullOrEmpty(TxtEmail.Text))
            {
                TbBled.Text = "Błąd: Pole Email jest wymagane!";
                TxtEmail.Focus();
                return;
            }
        }
    }
}

```



```

        // Sposób 3: sprawdzenie Length
        string opis = TxtOpis.Text.Trim();
        if (opis.Length == 0)
        {
            TbBled.Text = "Błąd: Opis jest wymagany!";
            TxtOpis.Focus();
            return;
        }

        // Jeśli wszystko OK
        TbBled.Foreground = System.Windows.Media.Brushes.DarkGreen;
        TbBled.Text = "✓ Wszystkie pola są poprawnie wypełnione!";
    }
}
}

```

Kluczowe elementy:

- `string.IsNullOrEmpty()` — sprawdza null, pusty string i same spacje
- `return` po znalezieniu błędu — zatrzymuje walidację, nie sprawdza dalej
- `TxtImie.Focus()` — przenosi kursor do błędnego pola

17.3. Sprawdzanie długości tekstu

Możesz sprawdzać minimalną i maksymalną liczbę znaków.

MainWindow.xaml:

```

<Window x:Class="WalidacjaDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Walidacja długości" Height="350" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Login (4-16 znaków):" FontSize="13"/>
        <TextBox x:Name="TxtLogin"
                FontSize="13"
                Padding="5"
                Margin="0,3,0,10"/>

        <Label Content="Hasło (min. 8 znaków):" FontSize="13"/>
        <PasswordBox x:Name="PboxHaslo"
                   FontSize="13"
                   Padding="5"
                   Margin="0,3,0,15"/>

        <Button Content="Zarejestruj"
               Height="35"
               Click="BtnZapisz_Click"/>

        <TextBlock x:Name="TbBled"
                  FontSize="13"
                  TextWrapping="Wrap"
                  Margin="0,10,0,0"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

```

```
using System.Windows.Media;

namespace WalidacjaDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnZapisz_Click(object sender, RoutedEventArgs e)
        {
            TbBled.Foreground = Brushes.Red;
            TbBled.Text = "";

            string login = TxtLogin.Text.Trim();
            string haslo = PboxHaslo.Password;

            // Sprawdzenie pustego pola
            if (login == "")
            {
                TbBled.Text = "Login nie może być pusty!";
                return;
            }

            // Sprawdzenie minimalnej długości
            if (login.Length < 4)
            {
                TbBled.Text = $"Login jest za krótki ({login.Length} znaki). Minimum to 4 znaki.";
                return;
            }

            // Sprawdzenie maksymalnej długości
            if (login.Length > 16)
            {
                TbBled.Text = $"Login jest za długi ({login.Length} znaków). Maksimum to 16 znaków.";
                return;
            }

            // Walidacja hasła – minimum 8 znaków
            if (haslo.Length < 8)
            {
                TbBled.Text = $"Hasło jest za krótkie ({haslo.Length} znaków). Wymagane minimum 8 znaków.";
                return;
            }

            // Wszystko OK
            TbBled.Foreground = Brushes.DarkGreen;
            TbBled.Text = $"✓ Konto dla '{login}' zostało zarejestrowane!";
        }
    }
}
```

Kluczowe elementy:

- `.Length` — liczba znaków w stringu
- `login.Length < 4` — zbyt krótki
- `login.Length > 16` — zbyt długi
- Komunikaty podają aktualną liczbę znaków — to pomaga użytkownikowi

17.4. Sprawdzanie, czy tekst zawiera tylko cyfry

Możesz sprawdzić czy każdy znak to cyfra.

MainWindow.xaml:

```
<Window x:Class="WalidacjaDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Walidacja cyfr" Height="300" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Numer telefonu (9 cyfr):" FontSize="13"/>
        <TextBox x:Name="TxtTelefon"
                FontSize="13"
                Padding="5"
                Margin="0,3,0,10"/>

        <Label Content="Numer PIN (4 cyfry):" FontSize="13"/>
        <TextBox x:Name="TxtPin"
                FontSize="13"
                Padding="5"
                Margin="0,3,0,15"/>

        <Button Content="Sprawdź"
                Height="35"
                Click="BtnSprawdz_Click"/>

        <TextBlock x:Name="TbBled"
                FontSize="13"
                TextWrapping="Wrap"
                Margin="0,10,0,0"
                Foreground="Red"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Linq;
using System.Windows;

namespace WalidacjaDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        // Metoda pomocnicza – sprawdza czy tekst zawiera tylko cyfry
        private bool ZawieraTylkoCyfry(string tekst)
        {
            if (tekst == "") return false;

            // Sposób 1: pętla foreach
            foreach (char c in tekst)
            {
                if (!char.IsDigit(c))
                    return false;
            }
            return true;
        }
    }
}
```

```

        // Sposób 2: LINQ (krócej)
        // return tekst.All(char.IsDigit);
    }

    private void BtnSprawdz_Click(object sender, RoutedEventArgs e)
    {
        TbBled.Text = "";
        TbBled.Foreground = System.Windows.Media.Brushes.Red;

        string telefon = TxtTelefon.Text.Trim();
        string pin = TxtPin.Text.Trim();

        // Walidacja telefonu
        if (string.IsNullOrEmpty(telefon))
        {
            TbBled.Text = "Podaj numer telefonu!";
            return;
        }

        if (!ZawieraTylkoCyfry(telefon))
        {
            TbBled.Text = "Numer telefonu może zawierać tylko cyfry!";
            return;
        }

        if (telefon.Length != 9)
        {
            TbBled.Text = $"Numer telefonu musi mieć 9 cyfr (wpisano {telefon.Length})!";
            return;
        }

        // Walidacja PIN – użycie LINQ
        if (string.IsNullOrEmpty(pin))
        {
            TbBled.Text = "Podaj PIN!";
            return;
        }

        if (!pin.All(char.IsDigit))
        {
            TbBled.Text = "PIN może zawierać tylko cyfry!";
            return;
        }

        if (pin.Length != 4)
        {
            TbBled.Text = $"PIN musi mieć dokładnie 4 cyfry (wpisano {pin.Length})!";
            return;
        }

        TbBled.Foreground = System.Windows.Media.Brushes.DarkGreen;
        TbBled.Text = "✓ Dane poprawne!";
    }
}

```

Kluczowe elementy:

- `char.IsDigit(c)` — sprawdza czy jeden znak to cyfra
- Pętla `foreach` — sprawdza każdy znak po kolei
- `tekst.All(char.IsDigit)` — LINQ, sprawdza czy wszystkie znaki są cyframi

- `using System.Linq` — wymagane dla `.All()`

17.5. Sprawdzanie poprawności liczby

`int.TryParse()` próbuje zamienić string na liczbę całkowitą. Zwraca `true` jeśli się udało i `false` jeśli tekst nie jest liczbą. Jest bezpieczniejszy niż `int.Parse()`, który rzuca wyjątek.

MainWindow.xaml:

```
<Window x:Class="WalidacjaDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="TryParse - walidacja liczb" Height="350" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Wiek:" FontSize="13"/>
        <TextBox x:Name="TxtWiek" FontSize="13" Padding="5" Margin="0,3,0,10"/>

        <Label Content="Cena (zł):" FontSize="13"/>
        <TextBox x:Name="TxtCena" FontSize="13" Padding="5" Margin="0,3,0,15"/>

        <Button Content="Oblicz" Height="35" Click="BtnOblicz_Click"/>

        <TextBlock x:Name="TbWynik" FontSize="13"
                TextWrapping="Wrap" Margin="0,10,0,0"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace WalidacjaDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnOblicz_Click(object sender, RoutedEventArgs e)
        {
            TbWynik.Foreground = System.Windows.Media.Brushes.Red;
            TbWynik.Text = "";

            // int.TryParse - bezpieczna konwersja
            // Zwraca true/false i wynik przez parametr out
            if (!int.TryParse(TxtWiek.Text.Trim(), out int wiek))
            {
                TbWynik.Text = "Wiek musi być liczbą całkowitą!";
                TxtWiek.Focus();
                return;
            }

            // double.TryParse - dla liczb zmiennoprzecinkowych
            // Uwaga: w polskich ustawieniach separator to przecinek!
            if (!double.TryParse(TxtCena.Text.Trim(),
                                System.Globalization.NumberStyles.Any,
                                System.Globalization.CultureInfo.InvariantCulture,
                                out double cena))
```

```

    {
        TbWynik.Text = "Cena musi być liczbą (użyj kropki jako separatora)!";
        TxtCena.Focus();
        return;
    }

    // Jeśli dotarliśmy tutaj – obie wartości są prawidłowe
    TbWynik.Foreground = System.Windows.Media.Brushes.DarkGreen;
    TbWynik.Text = $"✓ Wiek: {wiek} lat, Cena: {cena:F2} zł";
}
}
}

```

Kluczowe elementy:

- `int.TryParse(tekst, out int wynik)` — próbuje parsować, zwraca bool
- `out int wiek` — jeśli parsowanie się uda, `wiek` ma wartość
- `double.TryParse` — dla liczb zmiennoprzecinkowych
- `CultureInfo.InvariantCulture` — używa kropki jako separatora dziesiętnego

17.6. Walidacja zakresu liczbowego

Po sprawdzeniu czy wartość jest liczbą, sprawdzasz czy mieści się w dozwolonym zakresie.

MainWindow.xaml:

```

<Window x:Class="WalidacjaDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Walidacja zakresu" Height="320" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Temperatura (-50 do 60 °C):" FontSize="13"/>
        <TextBox x:Name="TxtTemp" FontSize="13" Padding="5" Margin="0,3,0,10"/>

        <Label Content="Ocena (1-6):" FontSize="13"/>
        <TextBox x:Name="TxtOcena" FontSize="13" Padding="5" Margin="0,3,0,15"/>

        <Button Content="Sprawdź" Height="35" Click="BtnSprawdz_Click"/>

        <TextBlock x:Name="TbWynik" FontSize="13"
            TextWrapping="Wrap" Margin="0,10,0,0"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WalidacjaDemo
{
    public partial class MainWindow : Window
    {
        // Stałe zakresu – łatwiej zmienić w jednym miejscu
        private const int TEMP_MIN = -50;
        private const int TEMP_MAX = 60;
        private const int OCENA_MIN = 1;
        private const int OCENA_MAX = 6;

        public MainWindow()

```

```

{
    InitializeComponent();
}

private void BtnSprawdz_Click(object sender, RoutedEventArgs e)
{
    TbWynik.Foreground = System.Windows.Media.Brushes.Red;
    TbWynik.Text = "";

    // Sprawdzenie temperatury
    if (!int.TryParse(TxtTemp.Text.Trim(), out int temperatura))
    {
        TbWynik.Text = "Temperatura musi być liczbą całkowitą!";
        return;
    }

    // Walidacja zakresu temperatury
    if (temperatura < TEMP_MIN || temperatura > TEMP_MAX)
    {
        TbWynik.Text =
            $"Temperatura poza zakresem! Dozwolone: {TEMP_MIN}°C do {TEMP_MAX}°C." +
            $" Wpisano: {temperatura}°C";
        return;
    }

    // Sprawdzenie oceny
    if (!int.TryParse(TxtOcena.Text.Trim(), out int ocena))
    {
        TbWynik.Text = "Ocena musi być liczbą całkowitą!";
        return;
    }

    // Walidacja zakresu oceny
    if (ocena < OCENA_MIN || ocena > OCENA_MAX)
    {
        TbWynik.Text =
            $"Ocena musi być między {OCENA_MIN} a {OCENA_MAX}!";
        return;
    }

    TbWynik.Foreground = System.Windows.Media.Brushes.DarkGreen;
    TbWynik.Text = $"✓ Temperatura: {temperatura}°C, Ocena: {ocena}";
    }
}
}

```

Kluczowe elementy:

- `temperatura < TEMP_MIN || temperatura > TEMP_MAX` — sprawdzenie zakresu
- Stałe `const` — trzymaj granice zakresu w stałych, nie w "magicznych liczbach"
- Komunikat błędu powinien podawać dozwolony zakres i wpisaną wartość

17.7. Walidacja formatu przez Regex

Regex (wyrażenie regularne) to wzorzec opisujący format tekstu. Możesz nim sprawdzić np. kod pocztowy, numer PESEL, adres email.

Wzorzec kodu pocztowego: `^\d{2}-\d{3}$`

- `^` — początek
- `\d{2}` — dokładnie dwie cyfry
- `-` — myślnik
- `\d{3}` — dokładnie trzy cyfry

- 💰 — koniec

MainWindow.xaml:

```
<Window x:Class="WalidacjaDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Walidacja formatu - Regex" Height="380" Width="480">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Kod pocztowy (format: XX-XXX):" FontSize="13"/>
        <TextBox x:Name="TxtKod"
                FontSize="13"
                Padding="5"
                Margin="0,3,0,10"
                MaxLength="6"/>

        <Label Content="Adres email:" FontSize="13"/>
        <TextBox x:Name="TxtEmail"
                FontSize="13"
                Padding="5"
                Margin="0,3,0,10"/>

        <Label Content="Numer PESEL (11 cyfr):" FontSize="13"/>
        <TextBox x:Name="TxtPesel"
                FontSize="13"
                Padding="5"
                Margin="0,3,0,15"
                MaxLength="11"/>

        <Button Content="Waliduj"
                Height="35"
                Click="BtnWaliduj_Click"/>

        <TextBlock x:Name="TbWynik"
                FontSize="13"
                TextWrapping="Wrap"
                Margin="0,10,0,0"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Text.RegularExpressions;
using System.Windows;

namespace WalidacjaDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // Metoda walidacji kodu pocztowego – format XX-XXX
            private bool WalidujKodPocztowy(string kod)
            {
                if (string.IsNullOrEmpty(kod))
                    return false;
            }
        }
    }
}
```



```

        // Wzorzec: dwie cyfry, myślnik, trzy cyfry
        Regex wzorzec = new Regex(@"^d{2}-d{3}$");
        return wzorzec.IsMatch(kod);
    }

    // Metoda walidacji email – podstawowy wzorzec
    private bool WalidujEmail(string email)
    {
        if (string.IsNullOrEmpty(email))
            return false;

        // Wzorzec: coś@coś.coś
        Regex wzorzec = new Regex(@"^[^@\s]+@[^\s]+\.[^\s]+$");
        return wzorzec.IsMatch(email);
    }

    // Metoda walidacji PESEL – 11 cyfr
    private bool WalidujPesel(string pesel)
    {
        if (string.IsNullOrEmpty(pesel))
            return false;

        // Wzorzec: dokładnie 11 cyfr
        Regex wzorzec = new Regex(@"^d{11}$");
        return wzorzec.IsMatch(pesel);
    }

    private void BtnWaliduj_Click(object sender, RoutedEventArgs e)
    {
        TbWynik.Text = "";
        string wynik = "";
        bool wszystkieOK = true;

        // Kod pocztowy
        string kod = TxtKod.Text.Trim();
        if (!WalidujKodPocztowy(kod))
        {
            wynik += "x Kod pocztowy jest niepoprawny! Format: XX-XXX (np. 12-345)\n";
            wszystkieOK = false;
        }
        else
        {
            wynik += $"✓ Kod pocztowy: {kod}\n";
        }

        // Email
        string email = TxtEmail.Text.Trim();
        if (!WalidujEmail(email))
        {
            wynik += "x Adres email jest niepoprawny!\n";
            wszystkieOK = false;
        }
        else
        {
            wynik += $"✓ Email: {email}\n";
        }

        // PESEL
        string pesel = TxtPesel.Text.Trim();
        if (!WalidujPesel(pesel))
        {
            wynik += "x Numer PESEL musi mieć dokładnie 11 cyfr!\n";
        }
    }

```

```

        wszystkieOK = false;
    }
    else
    {
        wynik += $"✓ PESEL: {pesel}\n";
    }

    TbWynik.Foreground = wszystkieOK
        ? System.Windows.Media.Brushes.DarkGreen
        : System.Windows.Media.Brushes.Red;
    TbWynik.Text = wynik;
    }
}
}
}

```

Kluczowe elementy:

- `new Regex(@"wzorec")` — tworzenie obiektu wyrażenia regularnego
- `wzorec.IsMatch(tekst)` — sprawdza czy tekst pasuje do wzorca
- `@"..."` — string dosłowny, odwrotny ukośnik `\` traktowany literalnie
- `^\d{2}-\d{3}$` — dokładnie dwie cyfry, myślnik, trzy cyfry
- `using System.Text.RegularExpressions` — wymagany namespace

17.8. TextBlock

Masz dwa sposoby wyświetlania błędów. Każdy ma swoje zastosowanie.

SPOSÓB	KIEDY UŻYWAĆ
<code>MessageBox.Show()</code>	Błąd krytyczny, wymagający potwierdzenia, przed niebezpieczną akcją
<code>TextBlock</code> (czerwony)	Błędy walidacji formularza — widoczne bez klikania

MessageBox — blokuje działanie:

```

// Użytkownik musi kliknąć OK żeby kontynuować
MessageBox.Show(
    "Pole 'Imię' jest wymagane!",
    "Błąd walidacji",
    MessageBoxButton.OK,
    MessageBoxImage.Warning
);

```

TextBlock — lepszy dla formularzy:

MainWindow.xaml:

```

<Window x:Class="WalidacjaDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Komunikaty błędów" Height="380" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <TextBox x:Name="TxtImie"
                FontSize="13" Padding="5" Margin="0,3,0,3"/>

        <!-- TextBlock błędu przy polu -->
        <TextBlock x:Name="TbBladImie"
                FontSize="11"
                Foreground="Red"
                Margin="0,0,0,10"/>

        <TextBox x:Name="TxtWiek"

```

```

        FontSize="13" Padding="5" Margin="0,3,0,3"/>
<TextBlock x:Name="TbBładWiek"
        FontSize="11"
        Foreground="Red"
        Margin="0,0,0,15"/>

<Button Content="Zapisz" Height="35" Click="BtnZapisz_Click"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WalidacjaDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnZapisz_Click(object sender, RoutedEventArgs e)
        {
            // Wyczyść wszystkie komunikaty błędów na początku
            TbBładImie.Text = "";
            TbBładWiek.Text = "";

            bool jestBład = false;

            // Walidacja imienia
            if (string.IsNullOrWhiteSpace(TxtImie.Text))
            {
                TbBładImie.Text = "⚠ Imię jest wymagane";
                jestBład = true;
            }

            // Walidacja wieku
            string wiekTekst = TxtWiek.Text.Trim();
            if (!int.TryParse(wiekTekst, out int wiek))
            {
                TbBładWiek.Text = "⚠ Wiek musi być liczbą";
                jestBład = true;
            }
            else if (wiek < 1 || wiek > 120)
            {
                TbBładWiek.Text = "⚠ Wiek musi być między 1 a 120";
                jestBład = true;
            }

            // Jeśli są błędy – nie kontynuuj
            if (jestBład) return;

            // Sukces – tutaj możesz pokazać MessageBox
            MessageBox.Show(
                $"Zapisano: {TxtImie.Text.Trim()}, wiek {wiek}",
                "Sukces",
                MessageBoxButton.OK,
                MessageBoxImage.Information
            );
        }
    }
}

```

```
    }  
    }  
}
```

Kluczowe elementy:

- Przy wielu polach: **zbieraj wszystkie błędy** zamiast zatrzymywać się na pierwszym
- `jestBład = true` — flaga informująca że jest bład
- Sprawdzaj **wszystkie** pola, wyświetl **wszystkie** błędy naraz
- `MessageBox` używaj tylko przy potwierdzeniu sukcesu lub krytycznym błędzie

17.9. Walidacja z fallbackiem

Fallback to wartość domyślna używana gdy dane są niepoprawne — zamiast blokować działanie, program używa bezpiecznej wartości zastępczej.

Przykład: szyfr Cezara — gdy użytkownik nie wpisze klucza, używaj 0 (brak przesunięcia).

MainWindow.xaml:

```
<Window x:Class="WalidacjaDemo.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Fallback - wartość domyślna" Height="380" Width="500">  
    <StackPanel Margin="20" VerticalAlignment="Center">  
        <Label Content="Tekst do zaszyfrowania:" FontSize="13"/>  
        <TextBox x:Name="TxtTekst"  
            FontSize="13"  
            Padding="5"  
            Margin="0,3,0,10" />  
  
        <Label Content="Klucz (liczba całkowita, domyślnie 3):" FontSize="13"/>  
        <TextBox x:Name="TxtKlucz"  
            FontSize="13"  
            Padding="5"  
            Margin="0,3,0,15" />  
  
        <Button Content="Zaszyfruj"  
            Height="35"  
            Click="BtnSzyfruj_Click" />  
  
        <Label x:Name="LblWynik"  
            FontSize="13"  
            Foreground="DarkBlue"  
            Margin="0,10,0,0"  
            FontFamily="Courier New" />  
  
        <TextBlock x:Name="TbInfo"  
            FontSize="11"  
            Foreground="Gray"  
            Margin="0,5,0,0" />  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.Windows;  
  
namespace WalidacjaDemo  
{
```

```

public partial class MainWindow : Window
{
    // Domyślna wartość klucza – fallback
    private const int DOMYSLNY_KLUCZ = 3;

    public MainWindow()
    {
        InitializeComponent();
    }

    private void BtnSzyfruj_Click(object sender, RoutedEventArgs e)
    {
        string tekst = TxtTekst.Text;
        if (string.IsNullOrEmpty(tekst))
        {
            MessageBox.Show("Wpisz tekst do zaszyfrowania!",
                "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        // Walidacja z fallbackiem – błędny klucz = wartość domyślna
        int klucz;
        string infoKlucz;

        if (!int.TryParse(TxtKlucz.Text.Trim(), out klucz))
        {
            // Tekst nie jest liczbą – użyj fallbacku
            klucz = DOMYSLNY_KLUCZ;
            infoKlucz = $"(użyto domyślnego klucza = {DOMYSLNY_KLUCZ}, bo wpisano nieprawidłową wartość)";
        }
        else
        {
            infoKlucz = $"(klucz = {klucz})";
        }

        // Szyfr Cezara
        string zaszyfrowany = SzyfrCezara(tekst, klucz);
        LblWynik.Content = "Wynik: " + zaszyfrowany;
        TbInfo.Text = infoKlucz;
    }

    private string SzyfrCezara(string tekst, int klucz)
    {
        char[] wynik = new char[tekst.Length];
        for (int i = 0; i < tekst.Length; i++)
        {
            char c = tekst[i];
            if (char.IsLetter(c))
            {
                char baza = char.IsUpper(c) ? 'A' : 'a';
                wynik[i] = (char)((c - baza + klucz % 26 + 26) % 26 + baza);
            }
            else
            {
                wynik[i] = c; // znaki niebędące literami bez zmian
            }
        }
        return new string(wynik);
    }
}

```

Kluczowe elementy:

- **Fallback** — program nie zatrzymuje się na błędnym wejściu, używa wartości domyślnej
- Informujesz użytkownika że użyto wartości domyślnej (`TbInfo`)
- `const int DOMYSLNY_KLUCZ = 3` — wartość domyślna jako stała (łatwiej zmienić)
- Fallback jest dobry dla opcjonalnych parametrów, nie dla danych krytycznych

18. Komunikaty dla użytkownika

`MessageBox` to okno dialogowe wyświetlające wiadomość i oczekujące odpowiedzi użytkownika. Służy do informowania o błędach, sukcesach i potwierdzania akcji. W tej sekcji poznasz wszystkie opcje `MessageBox` i nauczysz się kiedy i jak go używać.

18.1. `MessageBox.Show()`

`MessageBox.Show()` ma kilka przeciążeń (wersji z różną liczbą parametrów).

```
// Wersja 1 – tylko tekst
MessageBox.Show("Witaj w programie!");

// Wersja 2 – tekst i tytuł okna
MessageBox.Show("Operacja zakończona.", "Informacja");

// Wersja 3 – tekst, tytuł, przyciski
MessageBox.Show("Czy chcesz kontynuować?", "Pytanie",
    MessageBoxButton.YesNo);

// Wersja 4 – tekst, tytuł, przyciski, ikona
MessageBox.Show("Nie znaleziono pliku!", "Błąd",
    MessageBoxButton.OK, MessageBoxImage.Error);
```

MainWindow.xaml:

```
<Window x:Class="MessageBoxDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="MessageBox - podstawy" Height="300" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Button Content="Tylko tekst"
                Height="35"
                Margin="0,5"
                Click="BtnTylkoTekst_Click" />
        <Button Content="Tekst i tytuł"
                Height="35"
                Margin="0,5"
                Click="BtnTytuł_Click" />
        <Button Content="Pełny komunikat"
                Height="35"
                Margin="0,5"
                Click="BtnPełny_Click" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace MessageBoxDemo
```

```
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnTylkoTekst_Click(object sender, RoutedEventArgs e)
        {
            MessageBox.Show("To jest najprostszy komunikat.");
        }

        private void BtnTytuł_Click(object sender, RoutedEventArgs e)
        {
            MessageBox.Show("Dane zostały zapisane.", "Informacja");
        }

        private void BtnPełny_Click(object sender, RoutedEventArgs e)
        {
            MessageBox.Show(
                "Nie można otworzyć pliku.\nSprawdź czy plik istnieje.",
                "Błąd odczytu",
                MessageBoxButton.OK,
                MessageBoxImage.Error
            );
        }
    }
}
```

Kluczowe elementy:

- Parametr 1 — treść komunikatu (string)
- Parametr 2 — tytuł okna (string)
- Parametr 3 — zestaw przycisków (MessageBoxButton)
- Parametr 4 — ikona (MessageBoxImage)

18.2. Button

Możesz wybrać jeden z gotowych zestawów przycisków:

WARTOŚĆ	WYŚWIETLANE PRZYCISKI
MessageBoxButton.OK	OK
MessageBoxButton.OKCancel	OK, Anuluj
MessageBoxButton.YesNo	Tak, Nie
MessageBoxButton.YesNoCancel	Tak, Nie, Anuluj

MainWindow.xaml:

```
<Window x:Class="MessageBoxDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Typy przycisków" Height="350" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Button Content="MessageBoxButton.OK"
                Height="35" Margin="0,5"
                Click="BtnOK_Click" />
        <Button Content="MessageBoxButton.OKCancel"
                Height="35" Margin="0,5"
                Click="BtnOKCancel_Click" />
    
```

```

        <Button Content="MessageBoxButton.YesNo"
            Height="35" Margin="0,5"
            Click="BtnYesNo_Click" />

        <Button Content="MessageBoxButton.YesNoCancel"
            Height="35" Margin="0,5"
            Click="BtnYesNoCancel_Click" />

        <Label x:Name="LblWynik" FontSize="13" Margin="0,10,0,0" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace MessageBoxDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnOK_Click(object sender, RoutedEventArgs e)
        {
            MessageBoxResult wynik = MessageBox.Show(
                "Operacja zakończona pomyślnie.",
                "Informacja",
                MessageBoxButton.OK,
                MessageBoxImage.Information
            );
            LblWynik.Content = "Kliknięto: " + wynik.ToString();
        }

        private void BtnOKCancel_Click(object sender, RoutedEventArgs e)
        {
            MessageBoxResult wynik = MessageBox.Show(
                "Czy chcesz zapisać zmiany?",
                "Zapis",
                MessageBoxButton.OKCancel,
                MessageBoxImage.Question
            );
            LblWynik.Content = "Kliknięto: " + wynik.ToString();
        }

        private void BtnYesNo_Click(object sender, RoutedEventArgs e)
        {
            MessageBoxResult wynik = MessageBox.Show(
                "Czy na pewno chcesz usunąć ten rekord?",
                "Potwierdzenie",
                MessageBoxButton.YesNo,
                MessageBoxImage.Warning
            );
            LblWynik.Content = "Kliknięto: " + wynik.ToString();
        }

        private void BtnYesNoCancel_Click(object sender, RoutedEventArgs e)
        {
            MessageBoxResult wynik = MessageBox.Show(
                "Czy zapisać przed zamknięciem?\n• Tak – zapisz i zamknij\n• Nie – zamknij bez zapisu\n• Anuluj – wróć",

```



```

        "Zamykanie aplikacji",
        MessageBoxButton.YesNoCancel,
        MessageBoxImage.Question
    );
    LblWynik.Content = "Kliknięto: " + wynik.ToString();
}
}
}
}

```

18.3. MessageBoxResult

WARTOŚĆ	IKONA	KIEDY UŻYWAĆ
<code>MessageBoxImage.Information</code>	 niebieskie	Informacje, potwierdzenia
<code>MessageBoxImage.Question</code>	 niebieskie	Pytania do użytkownika
<code>MessageBoxImage.Warning</code>	 żółte	Ostrzeżenia, potencjalny problem
<code>MessageBoxImage.Error</code>	 czerwone	Błędy, awarie
<code>MessageBoxImage.None</code>	brak	Bez ikony

MainWindow.xaml:

```

<Window x:Class="MessageBoxDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Ikony MessageBox" Height="350" Width="450">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Button Content="Information – informacja"
                Height="35" Margin="0,5"
                Click="BtnInfo_Click"/>
        <Button Content="Question – pytanie"
                Height="35" Margin="0,5"
                Click="BtnPytanie_Click"/>
        <Button Content="Warning – ostrzeżenie"
                Height="35" Margin="0,5"
                Click="BtnOstrzezenie_Click"/>
        <Button Content="Error – błąd"
                Height="35" Margin="0,5"
                Click="BtnBled_Click"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace MessageBoxDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnInfo_Click(object sender, RoutedEventArgs e)
        {

```

```
        MessageBox.Show(
            "Dane zostały zapisane do bazy.",
            "Sukces",
            MessageBoxButton.OK,
            MessageBoxImage.Information // niebieska ikona 'i'
        );
    }

    private void BtnPytanie_Click(object sender, RoutedEventArgs e)
    {
        MessageBox.Show(
            "Czy chcesz załadować ostatni zapis?",
            "Pytanie",
            MessageBoxButton.YesNo,
            MessageBoxImage.Question // znak zapytania
        );
    }

    private void BtnOstrzezenie_Click(object sender, RoutedEventArgs e)
    {
        MessageBox.Show(
            "Uwaga! Ta operacja jest nieodwracalna.",
            "Ostrzeżenie",
            MessageBoxButton.OKCancel,
            MessageBoxImage.Warning // żółty trójkąt z wykrzyknikiem
        );
    }

    private void BtnBled_Click(object sender, RoutedEventArgs e)
    {
        MessageBox.Show(
            "Wystąpił błąd podczas otwierania pliku.\nPlik może być uszkodzony.",
            "Błąd",
            MessageBoxButton.OK,
            MessageBoxImage.Error // czerwony X
        );
    }
}
```

18.4. MessageBoxResult

`MessageBox.Show()` zwraca wartość `MessageBoxResult` wskazującą który przycisk kliknął użytkownik. Możesz na tej podstawie podjąć decyzję w programie.

WARTOŚĆ MESSAGEBOXRESULT	ZNACZENIE
<code>MessageBoxResult.OK</code>	Kliknięto OK
<code>MessageBoxResult.Cancel</code>	Kliknięto Anuluj
<code>MessageBoxResult.Yes</code>	Kliknięto Tak
<code>MessageBoxResult.No</code>	Kliknięto Nie
<code>MessageBoxResult.None</code>	Zamknięto X lub naciśnięto Escape

MainWindow.xaml:

```
<Window x:Class="MessageBoxDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Odczyt odpowiedzi" Height="320" Width="450">
```

```

<StackPanel Margin="20" VerticalAlignment="Center">
    <TextBox x:Name="TxtDane"
        FontSize="13"
        Padding="5"
        Margin="0,0,0,10" />

    <Button Content="Zapisz z potwierdzeniem"
        Height="35"
        Margin="0,0,0,10"
        Click="BtnZapisz_Click" />

    <Button Content="Usuń z potwierdzeniem"
        Height="35"
        Click="BtnUsun_Click" />

    <TextBlock x:Name="TbLog"
        FontSize="13"
        Foreground="Navy"
        Margin="0,15,0,0"
        TextWrapping="Wrap" />

</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace MessageBoxDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnZapisz_Click(object sender, RoutedEventArgs e)
        {
            // Pytanie z przyciskami OK / Anuluj
            MessageBoxResult odpowiedz = MessageBox.Show(
                $"Czy zapisać dane:\n\"{TxtDane.Text}\"?",
                "Zapis danych",
                MessageBoxButton.OKCancel,
                MessageBoxImage.Question
            );

            // Sprawdzenie odpowiedzi
            if (odpowiedz == MessageBoxResult.OK)
            {
                TbLog.Text = $"Zapisano: \"{TxtDane.Text}\"";
                TxtDane.Text = "";
            }
            else
            {
                TbLog.Text = "Zapis anulowany.";
            }
        }

        private void BtnUsun_Click(object sender, RoutedEventArgs e)
        {
            // Pytanie Tak/Nie/Anuluj
            MessageBoxResult odpowiedz = MessageBox.Show(
                "Czy na pewno chcesz usunąć wpis?\nTej operacji nie można cofnąć.",

```

```

        "Usuwanie",
        MessageBoxButton.YesNoCancel,
        MessageBoxImage.Warning
    );

    if (odpowiedz == MessageBoxResult.Yes)
    {
        TbLog.Text = "Wpis został usunięty!";
        TxtDane.Text = "";
    }
    else if (odpowiedz == MessageBoxResult.No)
    {
        TbLog.Text = "Usuwanie pominięte.";
    }
    else // Cancel lub None (zamknięcie X)
    {
        TbLog.Text = "Operacja anulowana.";
    }
    }
}
}
}

```

Kluczowe elementy:

- `MessageBoxResult odpowiedz = MessageBox.Show(...)` — zapisujesz wynik
- `if (odpowiedz == MessageBoxResult.Yes)` — sprawdzasz który przycisk
- `MessageBoxResult.None` — gdy użytkownik zamknął okno klawiszem X lub Escape

18.5. Komunikaty błędów

Poniżej widzisz konkretne komunikaty błędów dla typowych sytuacji walidacji.

MainWindow.xaml:

```

<Window x:Class="MessageBoxDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Komunikaty błędów walidacji" Height="420" Width="500">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <TextBox x:Name="TxtImie"
                FontSize="13" Padding="5" Margin="0,3"/>
        <TextBox x:Name="TxtWiek"
                FontSize="13" Padding="5" Margin="0,3"/>
        <TextBox x:Name="TxtEmail"
                FontSize="13" Padding="5" Margin="0,3"/>
        <TextBox x:Name="TxtKod"
                FontSize="13" Padding="5" Margin="0,3"/>
        <Button Content="Waliduj i zapisz"
                Height="35"
                Margin="0,15,0,0"
                Click="BtnWaliduj_Click"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Text.RegularExpressions;
using System.Windows;

namespace MessageBoxDemo

```

```
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnWaliduj_Click(object sender, RoutedEventArgs e)
        {
            // Walidacja imienia
            if (string.IsNullOrEmpty(TxtImie.Text))
            {
                MessageBox.Show(
                    "Pole 'Imię' jest wymagane.\nWpisz imię przed zapisaniem.",
                    "Brakujące dane",
                    MessageBoxButton.OK,
                    MessageBoxImage.Warning
                );
                TxtImie.Focus();
                return;
            }

            // Walidacja wieku – czy jest liczbą
            if (!int.TryParse(TxtWiek.Text.Trim(), out int wiek))
            {
                MessageBox.Show(
                    "Pole 'Wiek' zawiera nieprawidłową wartość.\n" +
                    "Wpisz liczbę całkowitą (np. 25).",
                    "Nieprawidłowe dane",
                    MessageBoxButton.OK,
                    MessageBoxImage.Warning
                );
                TxtWiek.Focus();
                return;
            }

            // Walidacja zakresu wieku
            if (wiek < 18 || wiek > 100)
            {
                MessageBox.Show(
                    $"Wiek '{wiek}' jest poza dopuszczalnym zakresem.\n" +
                    "Dopuszczalny wiek: 18-100 lat.",
                    "Zakres",
                    MessageBoxButton.OK,
                    MessageBoxImage.Warning
                );
                TxtWiek.Focus();
                return;
            }

            // Walidacja emaila
            string email = TxtEmail.Text.Trim();
            if (!new Regex(@"^[^@\s]+@[^@\s]+\.[^@\s]+$").IsMatch(email))
            {
                MessageBox.Show(
                    $"Adres email '{email}' jest nieprawidłowy.\n" +
                    "Przykład poprawnego emaila: jan@example.pl",
                    "Nieprawidłowy email",
                    MessageBoxButton.OK,
                    MessageBoxImage.Warning
                );
            }
        }
    }
}
```

```

        TxtEmail.Focus();
        return;
    }

    // Walidacja kodu pocztowego
    string kod = TxtKod.Text.Trim();
    if (!new Regex(@"^\d{2}-\d{3}$").IsMatch(kod))
    {
        MessageBox.Show(
            $"Kod pocztowy '{kod}' jest nieprawidłowy.\n" +
            "Wymagany format: XX-XXX, np. 12-345.",
            "Nieprawidłowy kod pocztowy",
            MessageBoxButton.OK,
            MessageBoxImage.Warning
        );
        TxtKod.Focus();
        return;
    }

    // Wszystko OK
    MessageBox.Show(
        $"Dane zapisane!\n\n" +
        $"Imię: {TxtImie.Text.Trim()}\n" +
        $"Wiek: {wiek} lat\n" +
        $"Email: {email}\n" +
        $"Kod: {kod}",
        "Sukces",
        MessageBoxButton.OK,
        MessageBoxImage.Information
    );
}
}
}

```

18.6. Komunikaty wyniku działania

Po wykonaniu operacji pokazujesz podsumowanie wyników.

MainWindow.xaml:

```

<Window x:Class="MessageBoxDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Komunikat z wynikiem" Height="350" Width="480">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Wpisz liczby oddzielone spacją:" FontSize="13"/>
        <TextBox x:Name="TxtLiczby"
            Text="10 20 30 40 50"
            FontSize="13"
            Padding="5"
            Margin="0,5,0,15"/>
        <Button Content="Oblicz statystyki"
            Height="35"
            Click="BtnOblicz_Click"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```
using System;
using System.Collections.Generic;
using System.Windows;

namespace MessageBoxDemo
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnOblicz_Click(object sender, RoutedEventArgs e)
        {
            string wejscie = TxtLiczby.Text.Trim();
            string[] czesci = wejscie.Split(' ');

            List<double> liczby = new List<double>();
            int bledy = 0;

            foreach (string s in czesci)
            {
                if (double.TryParse(s.Trim(), out double l))
                {
                    liczby.Add(l);
                }
                else if (!string.IsNullOrEmpty(s))
                {
                    bledy++;
                }
            }

            if (liczby.Count == 0)
            {
                MessageBox.Show("Nie znaleziono żadnych poprawnych liczb!",
                                "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
                return;
            }

            double suma = 0;
            double min = liczby[0];
            double max = liczby[0];
            foreach (double l in liczby)
            {
                suma += l;
                if (l < min) min = l;
                if (l > max) max = l;
            }

            double srednia = suma / liczby.Count;

            // Podsumowanie z wynikiem
            string komunikat =
                $"Przetworzono {liczby.Count} liczb.\n" +
                (bledy > 0 ? $"Pominięto {bledy} nieprawidłowych wartości.\n\n" : "\n") +
                $"Suma: {suma}\n" +
                $"Średnia: {srednia:F2}\n" +
                $"Minimum: {min}\n" +
                $"Maksimum: {max}";

            MessageBox.Show(komunikat, "Wyniki obliczeń",
                            MessageBoxButton.OK, MessageBoxImage.Information);
        }
    }
}
```

```
}  
  
}
```

18.7. Potwierdzanie akcji — dialog Tak/Nie przed niebezpieczną operacją

Przed nieodwracalną operacją (usunięcie, reset, zamknięcie) zawsze pytaj użytkownika o potwierdzenie.

MainWindow.xaml:

```
<Window x:Class="MessageBoxDemo.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Potwierdzanie akcji" Height="380" Width="480"  
    Closing="Window_Closing">  
    <StackPanel Margin="20" VerticalAlignment="Center">  
        <TextBox x:Name="TxtNotatka"  
            Height="120"  
            FontSize="13"  
            TextWrapping="Wrap"  
            AcceptsReturn="True"  
            Padding="5"  
            Margin="0,0,0,15"/>  
  
        <StackPanel Orientation="Horizontal">  
            <Button Content="Wyczyść wszystko"  
                Height="35"  
                Width="140"  
                Margin="0,0,10,0"  
                Background="DarkOrange"  
                Foreground="White"  
                Click="BtnWyczysc_Click"/>  
            <Button Content="Usuń dane"  
                Height="35"  
                Width="100"  
                Background="DarkRed"  
                Foreground="White"  
                Click="BtnUsun_Click"/>  
        </StackPanel>  
  
        <TextBlock x:Name="TbStatus"  
            FontSize="13"  
            Foreground="Gray"  
            Margin="0,15,0,0"/>  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.ComponentModel;  
using System.Windows;  
  
namespace MessageBoxDemo  
{  
    public partial class MainWindow : Window  
    {  
        private bool _manieZapisaneZmiany = false;  
  
        public MainWindow()  
        {
```



```

InitializeComponent();
TxtNotatka.TextChanged += (s, e) => _manieZapisaneZmiany = true;
}

private void BtnWyczysc_Click(object sender, RoutedEventArgs e)
{
    if (TxtNotatka.Text.Trim() == "")
    {
        TbStatus.Text = "Pole jest już puste.";
        return;
    }

    // Potwierdzenie przed wyczyszczeniem
    MessageBoxResult odpowiedz = MessageBox.Show(
        "Czy na pewno chcesz wyczyścić wszystkie notatki?\n" +
        "Dane zostaną utracone.",
        "Potwierdzenie",
        MessageBoxButton.YesNo,
        MessageBoxImage.Warning
    );

    if (odpowiedz == MessageBoxResult.Yes)
    {
        TxtNotatka.Text = "";
        _manieZapisaneZmiany = false;
        TbStatus.Text = "Notatki wyczyszczone.";
    }
    else
    {
        TbStatus.Text = "Akcja anulowana.";
    }
}

private void BtnUsun_Click(object sender, RoutedEventArgs e)
{
    // Potwierdzenie z dokładniejszym opisem skutków
    MessageBoxResult odpowiedz = MessageBox.Show(
        "UWAGA: Usunięcie danych jest nieodwracalne!\n\n" +
        "Zostaną usunięte:\n" +
        "• Wszystkie notatki\n" +
        "• Ustawienia aplikacji\n\n" +
        "Czy na pewno kontynuować?",
        "Potwierdzenie usunięcia",
        MessageBoxButton.YesNo,
        MessageBoxImage.Error
    );

    if (odpowiedz == MessageBoxResult.Yes)
    {
        TxtNotatka.Text = "";
        _manieZapisaneZmiany = false;
        TbStatus.Text = "Wszystkie dane usunięte.";
    }
    else
    {
        TbStatus.Text = "Usunięcie anulowane.";
    }
}

// Potwierdzenie przy zamykaniu okna
private void Window_Closing(object sender, CancelEventArgs e)
{

```

```

        if (!_manieZapisaneZmiany)
        {
            MessageBoxResult odpowiedz = MessageBox.Show(
                "Masz niezapisane zmiany.\nCzy chcesz zamknąć bez zapisywania?",
                "Niezapisane zmiany",
                MessageBoxButton.YesNo,
                MessageBoxImage.Question
            );

            if (odpowiedz == MessageBoxResult.No)
            {
                e.Cancel = true; // zapobiega zamknięciu okna
            }
        }
    }
}
}

```

Kluczowe elementy:

- `MessageBoxButton.YesNo` — przyciski Tak/Nie
- `odpowiedz == MessageBoxResult.Yes` — sprawdzasz czy użytkownik potwierdził
- `e.Cancel = true` — w zdarzeniu `Closing` zapobiega zamknięciu okna
- Komunikat powinien jasno opisywać **co zostanie utracone**
- Używaj `MessageBoxImage.Warning` lub `Error` dla niebezpiecznych akcji

19. Stan aplikacji i aktualizacja widoku

Stan aplikacji to zestaw danych, które program pamięta podczas działania. Bez stanu każde kliknięcie przycisku byłoby niezależne — program nie pamiętałby co było wcześniej. W tej sekcji nauczysz się jak prawidłowo przechowywać i aktualizować stan aplikacji w WPF.

19.1. Czym jest stan aplikacji

Stan to wszystkie dane, które program utrzymuje między akcjami użytkownika. Przykłady stanu:

- Wygenerowane hasło — żeby móc je skopiować po wygenerowaniu
- Aktualny indeks — żeby wiedzieć który element listy jest aktywny
- Wybrany kolor — żeby podgląd był zawsze aktualny
- Historia operacji — żeby móc cofnąć ostatnią akcję

Bez stanu — każde kliknięcie jest niezależne, program nic nie pamięta. **Ze stanem** — program kumuluje informacje i działa na podstawie historii.

19.2. Zmienne przechowujące stan — pola klasy okna

Stan przechowujesz w polach (fields) klasy `MainWindow`. Nie w metodach — zmienne wewnątrz metod znikają po jej zakończeniu.

MainWindow.xaml:

```

<Window x:Class="StanAplikacji.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Stan aplikacji - pola klasy" Height="420" Width="500">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Generator haseł" FontSize="18" FontWeight="Bold"
            HorizontalAlignment="Center" Margin="0,0,0,15" />

        <Slider x:Name="SliderDlugosc"
            Minimum="4"
            Maximum="20"
            Value="8"
            Margin="0,0,0,5" />
    </StackPanel>
</Window>

```

```

<Label x:Name="LblDlugosc"
        Content="Długość: 8"
        FontSize="13"
        HorizontalAlignment="Center" />

<CheckBox x:Name="ChkCyfry"
        Content="Dodaj cyfry"
        FontSize="13"
        IsChecked="True"
        Margin="0,10,0,3" />
<CheckBox x:Name="ChkZnaki"
        Content="Dodaj znaki specjalne"
        FontSize="13"
        Margin="0,3,0,15" />

<StackPanel Orientation="Horizontal">
    <Button x:Name="BtnGeneruj"
            Content="Generuj hasło"
            Height="35"
            Width="130"
            Margin="0,0,10,0"
            Click="BtnGeneruj_Click" />
    <Button x:Name="BtnKopiuuj"
            Content="Kopiuuj"
            Height="35"
            Width="100"
            Click="BtnKopiuuj_Click"
            IsEnabled="False" />
</StackPanel>

<TextBox x:Name="TxtHaslo"
        FontSize="16"
        FontFamily="Courier New"
        IsReadOnly="True"
        Padding="8"
        Margin="0,15,0,5"
        Background="LightGray" />

<Label x:Name="LblStatus"
        FontSize="12"
        Foreground="Gray"
        HorizontalAlignment="Center" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace StanAplikacji
{
    public partial class MainWindow : Window
    {
        // POLA KLASY – przechowują stan aplikacji
        // Dostępne przez całe życie okna, we wszystkich metodach
        private string _wygenerowaneHaslo = "";
        private int _liczbaGeneracji = 0;
        private Random _losowy = new Random();

        public MainWindow()

```

```

{
    InitializeComponent();
    SliderDlugosc.ValueChanged += (s, e) =>
    {
        if (LblDlugosc != null)
            LblDlugosc.Content = $"Długość: {(int)SliderDlugosc.Value}";
    };
}

private void BtnGeneruj_Click(object sender, RoutedEventArgs e)
{
    int dlugosc = (int)SliderDlugosc.Value;
    bool dodajCyfry = ChkCyfry.IsChecked ?? false;
    bool dodajZnaki = ChkZnaki.IsChecked ?? false;

    // Generowanie nowego hasła
    _wygenerowaneHaslo = GenerujHaslo(dlugosc, dodajCyfry, dodajZnaki);
    _liczbaGeneracji++;

    // Aktualizacja widoku na podstawie stanu
    TxtHaslo.Text = _wygenerowaneHaslo;
    BtnKopiuuj.IsEnabled = true; // teraz można kopiować
    LblStatus.Content = $"Wygenerowano hasło #{_liczbaGeneracji}";
}

private void BtnKopiuuj_Click(object sender, RoutedEventArgs e)
{
    // Używamy przechowywanego stanu - _wygenerowaneHaslo
    if (_wygenerowaneHaslo != "")
    {
        Clipboard.SetText(_wygenerowaneHaslo);
        LblStatus.Content = "Hasło skopiowane do schowka!";
    }
}

private string GenerujHaslo(int dlugosc, bool cyfry, bool znaki)
{
    string litery = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    string numerki = "0123456789";
    string specjalne = "!@#$%^&*";

    string pula = litery;
    if (cyfry) pula += numerki;
    if (znaki) pula += specjalne;

    char[] haslo = new char[dlugosc];
    for (int i = 0; i < dlugosc; i++)
        haslo[i] = pula[_losowy.Next(pula.Length)];

    return new string(haslo);
}
}
}

```

Kluczowe elementy:

- `private string _wygenerowaneHaslo` — pole klasy, żyje przez cały czas działania okna
- `private int _liczbaGeneracji` — licznik, rośnie z każdym wygenerowaniem
- `private Random _losowy` — jeden obiekt Random dla całego okna (lepszą losowość)
- Konwencja: pola prywatne zaczynają się od `_` (podkreślenie)

19.3. Stan lokalny i globalny

To kluczowa różnica! Zmienna wewnątrz metody (lokalna) żyje tylko podczas jej wykonania. Pole klasy (stan okna) żyje przez cały czas.

MainWindow.xaml:

```
<Window x:Class="StanAplikacji.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Stan lokalny vs globalny" Height="380" Width="500">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <Label Content="Licznik kliknięć:" FontSize="14" FontWeight="Bold"/>

        <StackPanel Orientation="Horizontal" Margin="0,10,0,0">
            <Button Content="Lokalny (zły!)"
                    Height="35"
                    Width="150"
                    Margin="0,0,10,0"
                    Click="BtnLokalny_Click" />
            <Button Content="Globalny (dobry)"
                    Height="35"
                    Width="150"
                    Click="BtnGlobalny_Click" />
        </StackPanel>

        <StackPanel Orientation="Horizontal" Margin="0,15,0,0">
            <Label Content="Lokalny licznik:"
                    FontSize="13"
                    Width="150"
                    VerticalAlignment="Center" />
            <Label x:Name="LblLokalny"
                    Content="0"
                    FontSize="20"
                    Foreground="DarkRed"
                    FontWeight="Bold" />
        </StackPanel>

        <StackPanel Orientation="Horizontal" Margin="0,5,0,0">
            <Label Content="Globalny licznik:"
                    FontSize="13"
                    Width="150"
                    VerticalAlignment="Center" />
            <Label x:Name="LblGlobalny"
                    Content="0"
                    FontSize="20"
                    Foreground="DarkGreen"
                    FontWeight="Bold" />
        </StackPanel>

        <TextBlock FontSize="12"
                    Foreground="DimGray"
                    TextWrapping="Wrap"
                    Margin="0,20,0,0">
            Lokalny zawsze pokazuje "1" – bo zmienna jest tworzona od nowa przy każdym kliknięciu.
            Globalny rośnie – bo pole klasy jest pamiętane między kliknięciami.
        </TextBlock>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace StanAplikacji
{
    public partial class MainWindow : Window
    {
        // POLE KLASY – stan globalny okna
        // Jest tutaj: poza metodami, ale w klasie
        private int _licznikGlobalny = 0;

        public MainWindow()
        {
            InitializeComponent();
        }

        private void BtnLokalny_Click(object sender, RoutedEventArgs e)
        {
            // BŁĄD! Zmienna lokalna – tworzona od nowa przy każdym kliknięciu
            int licznikLokalny = 0; // zawsze zaczyna od 0!
            licznikLokalny++;        // zawsze będzie 1
            LblLokalny.Content = licznikLokalny.ToString();
        }

        private void BtnGlobalny_Click(object sender, RoutedEventArgs e)
        {
            // POPRAWNIE! Pole klasy – pamiętane między kliknięciami
            _licznikGlobalny++;
            LblGlobalny.Content = _licznikGlobalny.ToString();
        }
    }
}

```

Kluczowe elementy:

- Zmienna lokalna (`int licznikLokalny = 0;` wewnątrz metody) — żyje tylko podczas wywołania metody
- Pole klasy (`private int _licznikGlobalny = 0;` na poziomie klasy) — żyje przez całe działanie okna
- Typowy błąd: deklarowanie zmiennej wewnątrz handlera zdarzenia zamiast jako pola klasy

19.4. Stan bieżący i zatwierdzony

Czasem masz dwa rodzaje stanu: **bieżący** (to co aktualnie widać w kontrolkach) i **zatwierdzony** (to co zostało zatwierdzone przez użytkownika). Przykład: mieszanie kolorów RGB.

MainWindow.xaml:

```

<Window x:Class="StanAplikacji.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Stan bieżący vs zatwierdzony" Height="500" Width="520">
    <StackPanel Margin="20">
        <Label Content="Mieszalnik kolorów RGB" FontSize="16" FontWeight="Bold"
               HorizontalAlignment="Center" Margin="0,0,0,15"/>

        <!-- Suwaki RGB -->
        <StackPanel Orientation="Horizontal" Margin="0,0,0,5">
            <Label Content="R:" Width="20" FontSize="13" VerticalAlignment="Center"/>
            <Slider x:Name="SliderR" Minimum="0" Maximum="255" Value="255"
                   Width="300" ValueChanged="Slider_ValueChanged"/>
            <Label x:Name="LblR" Content="255" Width="35" FontSize="13"
                   VerticalAlignment="Center"/>

```

```

</StackPanel>
<StackPanel Orientation="Horizontal" Margin="0,5,0,5">
    <Label Content="G:" Width="20" FontSize="13" VerticalAlignment="Center"/>
    <Slider x:Name="SliderG" Minimum="0" Maximum="255" Value="0"
        Width="300" ValueChanged="Slider_ValueChanged"/>
    <Label x:Name="LblG" Content="0" Width="35" FontSize="13"
        VerticalAlignment="Center"/>
</StackPanel>
<StackPanel Orientation="Horizontal" Margin="0,5,0,10">
    <Label Content="B:" Width="20" FontSize="13" VerticalAlignment="Center"/>
    <Slider x:Name="SliderB" Minimum="0" Maximum="255" Value="0"
        Width="300" ValueChanged="Slider_ValueChanged"/>
    <Label x:Name="LblB" Content="0" Width="35" FontSize="13"
        VerticalAlignment="Center"/>
</StackPanel>

<!-- Podgląd bieżący -->
<Label Content="Podgląd bieżący:" FontSize="13" FontWeight="Bold"/>
<Border x:Name="BrdBiezacy"
    Height="60"
    Background="Red"
    CornerRadius="5"
    Margin="0,5,0,15"/>

<!-- Przyciski -->
<StackPanel Orientation="Horizontal">
    <Button Content="Zatwierdź kolor"
        Height="35"
        Width="130"
        Margin="0,0,10,0"
        Click="BtnZatwierdz_Click"/>
    <Button Content="Przywróć zatwierdzony"
        Height="35"
        Width="160"
        Click="BtnPrzywroc_Click"/>
</StackPanel>

<!-- Stan zatwierdzony -->
<Label Content="Kolor zatwierdzony:" FontSize="13"
    FontWeight="Bold" Margin="0,15,0,0"/>
<Border x:Name="BrdZatwierdzony"
    Height="60"
    Background="Red"
    CornerRadius="5"
    Margin="0,5,0,5"/>
<Label x:Name="LblZatwierdzonyHex"
    Content="Zatwierdzony: #FF0000"
    FontSize="12"
    Foreground="Gray"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Media;

namespace StanAplikacji
{
    public partial class MainWindow : Window

```

```

{
    // STAN BIEŻĄCY – aktualizowany przy każdym ruchu suwaka
    private byte _biezacyR = 255;
    private byte _biezacyG = 0;
    private byte _biezacyB = 0;

    // STAN ZATWIERDZONY – aktualizowany tylko po kliknięciu "Zatwierdź"
    private byte _zatwierdzonR = 255;
    private byte _zatwierdzonG = 0;
    private byte _zatwierdzonB = 0;

    public MainWindow()
    {
        InitializeComponent();

    }

    // Zdarzenie zmiana suwaka – aktualizuje STAN BIEŻĄCY
    private void Slider_ValueChanged(object sender,
        RoutedPropertyChangedEventArgs<double> e)
    {
        if (BrdBiezacy == null) return; // ostrożność przy inicjalizacji

        // Pobierz wartości ze sliderów
        _biezacyR = (byte)SliderR.Value;
        _biezacyG = (byte)SliderG.Value;
        _biezacyB = (byte)SliderB.Value;

        // Aktualizuj etykiety
        LblR.Content = _biezacyR.ToString();
        LblG.Content = _biezacyG.ToString();
        LblB.Content = _biezacyB.ToString();

        // Aktualizuj podgląd bieżący (ale NIE zatwierdzonego!)
        Color kolor = Color.FromRgb(_biezacyR, _biezacyG, _biezacyB);
        BrdBiezacy.Background = new SolidColorBrush(kolor);
    }

    // Zatwierdzenie – przeniesienie stanu bieżącego do zatwierdzonego
    private void BtnZatwierdz_Click(object sender, RoutedEventArgs e)
    {
        _zatwierdzonR = _biezacyR;
        _zatwierdzonG = _biezacyG;
        _zatwierdzonB = _biezacyB;

        // Aktualizuj podgląd zatwierdzonego
        Color kolor = Color.FromRgb(_zatwierdzonR, _zatwierdzonG, _zatwierdzonB);
        BrdZatwierdzony.Background = new SolidColorBrush(kolor);
        LblZatwierdzonyHex.Content =
            $"Zatwierdzony: #{_zatwierdzonR:X2}{_zatwierdzonG:X2}{_zatwierdzonB:X2}";

        MessageBox.Show(
            $"Kolor #{_zatwierdzonR:X2}{_zatwierdzonG:X2}{_zatwierdzonB:X2} zatwierdzony!",
            "Zatwierdzono",
            MessageBoxButton.OK,
            MessageBoxImage.Information
        );
    }

    // Przywrócenie zatwierdzonego – odwrót do stanu zatwierdzonego
    private void BtnPrzywroc_Click(object sender, RoutedEventArgs e)
    {
        // Ustaw suwaki na zatwierdzony kolor
    }
}

```



```

        SliderR.Value = _zatwierdzonR;
        SliderG.Value = _zatwierdzonG;
        SliderB.Value = _zatwierdzonB;
        // Zdarzenie ValueChanged zostanie wywołane automatycznie
    }
}
}
}

```

Kluczowe elementy:

- **Stan bieżący** (`_biezacyR/G/B`) — zmienia się przy każdym ruchu suwaka
- **Stan zatwierdzony** (`_zatwierdzonR/G/B`) — zmienia się tylko po kliknięciu przycisku
- To wzorzec jak "preview" vs "apply" w edytorach graficznych
- "Przywróć" ustawia suwaki z powrotem — zdarzenie `ValueChanged` aktualizuje resztę

19.5. Aktualizacja danych tylko w pamięci

Wzorzec aktualizacji stanu po każdej akcji jest zawsze taki sam:

1. **Pobierz** dane z kontrolek
2. **Oblicz** nowy stan
3. **Zapisz** do pola klasy
4. **Odśwież** widok (wywołaj metodę aktualizującą UI)

MainWindow.xaml:

```

<Window x:Class="StanAplikacji.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Wzorzec: pobierz → oblicz → zapisz → odśwież" Height="480" Width="520">
    <StackPanel Margin="20">
        <Label Content="Kalkulator punktów" FontSize="16"
            FontWeight="Bold" HorizontalAlignment="Center" Margin="0,0,0,15" />

        <!-- Wejście -->
        <StackPanel Orientation="Horizontal" Margin="0,5">
            <Label Content="Punkty:" Width="80" FontSize="13" VerticalAlignment="Center" />
            <TextBox x:Name="TxtPunkty" Width="100" FontSize="13" Padding="5" />
            <Button Content="Dodaj"
                Width="70"
                Height="32"
                Margin="10,0,0,0"
                Click="BtnDodaj_Click" />
            <Button Content="Oddejmij"
                Width="70"
                Height="32"
                Margin="5,0,0,0"
                Click="BtnOddejmij_Click" />
        </StackPanel>

        <!-- Mnożnik (Slider) -->
        <StackPanel Orientation="Horizontal" Margin="0,10">
            <Label Content="Mnożnik:" Width="80" FontSize="13" VerticalAlignment="Center" />
            <Slider x:Name="SliderMnoznik"
                Minimum="1"
                Maximum="5"
                Value="1"
                Width="200"
                TickFrequency="1"
                IsSnapToTickEnabled="True"
                ValueChanged="SliderMnoznik_Changed" />
            <Label x:Name="LblMnoznik"

```

```

        Content="x1"
        Width="30"
        FontSize="13"
        VerticalAlignment="Center"/>
</StackPanel>

<Button Content="Pomnóż aktualne punkty"
        Height="35"
        Margin="0,5,0,15"
        Click="BtnPomnoz_Click"/>

<!-- Wyświetlanie stanu -->
<Border BorderBrush="LightGray" BorderThickness="1"
        Padding="15" CornerRadius="5">
    <StackPanel>
        <Label x:Name="LblSuma"
            Content="Suma punktów: 0"
            FontSize="20"
            FontWeight="Bold"
            Foreground="DarkBlue"
            HorizontalAlignment="Center"/>
        <Label x:Name="LblLiczbaOperacji"
            Content="Liczba operacji: 0"
            FontSize="13"
            Foreground="Gray"
            HorizontalAlignment="Center"/>
        <Label x:Name="LblOstatnia"
            Content="Ostatnia operacja: -"
            FontSize="12"
            Foreground="Gray"
            HorizontalAlignment="Center"/>
    </StackPanel>
</Border>

<Button Content="Reset"
        Height="35"
        Margin="0,15,0,0"
        Background="Gray"
        Foreground="White"
        Click="BtnReset_Click"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace StanAplikacji
{
    public partial class MainWindow : Window
    {
        // STAN APLIKACJI – wszystkie pola klasy
        private int _sumaPunktow = 0;
        private int _liczbaOperacji = 0;
        private string _ostatniaOperacja = "-";
        private int _aktualneMnoznik = 1;

        public MainWindow()
        {
            InitializeComponent();

```

```

}

private void SliderMnoznic_Changed(object sender,
    RoutedPropertyChangedEventArgs<double> e)
{
    _aktualneMnoznic = (int)SliderMnoznic.Value;
    if (LblMnoznic != null)
        LblMnoznic.Content = $"x{_aktualneMnoznic}";
}

private void BtnDodaj_Click(object sender, RoutedEventArgs e)
{
    // KROK 1: Pobierz dane z kontrolek
    if (!int.TryParse(TxtPunkty.Text.Trim(), out int punkty))
    {
        MessageBox.Show("Wpisz poprawną liczbę punktów!",
            "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // KROK 2: Oblicz nowy stan
    int dodane = punkty * _aktualneMnoznic;

    // KROK 3: Zapisz do pól klasy
    _sumaPunktow += dodane;
    _liczbaOperacji++;
    _ostatniaOperacja = $"Dodano +{dodane} (x{_aktualneMnoznic})";

    // KROK 4: Odśwież widok
    OdswiezWidok();
    TxtPunkty.Text = "";
}

private void BtnOdejmij_Click(object sender, RoutedEventArgs e)
{
    // KROK 1: Pobierz
    if (!int.TryParse(TxtPunkty.Text.Trim(), out int punkty))
    {
        MessageBox.Show("Wpisz poprawną liczbę punktów!",
            "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // KROK 2: Oblicz
    int odjete = punkty * _aktualneMnoznic;

    // KROK 3: Zapisz
    _sumaPunktow -= odjete;
    _liczbaOperacji++;
    _ostatniaOperacja = $"Odjęto -{odjete} (x{_aktualneMnoznic})";

    // KROK 4: Odśwież
    OdswiezWidok();
    TxtPunkty.Text = "";
}

private void BtnPomnoz_Click(object sender, RoutedEventArgs e)
{
    // KROK 2: Oblicz
    int staraWartosc = _sumaPunktow;
    _sumaPunktow *= _aktualneMnoznic;
}

```

```

// KROK 3: Zapisz
_liczbaOperacji++;
_ostatniaOperacja = $"Pomnożono {staraWartosc} x {_aktualneMnozник} = {_sumaPunktow}";

// KROK 4: Odśwież
OdswiezWidok();
}

private void BtnReset_Click(object sender, RoutedEventArgs e)
{
    MessageBoxResult odp = MessageBox.Show(
        "Czy zresetować licznik?",
        "Reset",
        MessageBoxButton.YesNo,
        MessageBoxImage.Question
    );

    if (odp == MessageBoxResult.Yes)
    {
        // Resetuj stan
        _sumaPunktow = 0;
        _liczbaOperacji = 0;
        _ostatniaOperacja = "Reset";
        OdswiezWidok();
    }
}

// JEDNA METODA aktualizuje cały widok na podstawie stanu
private void OdswiezWidok()
{
    LblSuma.Content = $"Suma punktów: {_sumaPunktow}";
    LblLiczbaOperacji.Content = $"Liczba operacji: {_liczbaOperacji}";
    LblOstatnia.Content = $"Ostatnia operacja: {_ostatniaOperacja}";

    // Kolor punktów zależny od wartości
    if (_sumaPunktow > 0)
        LblSuma.Foreground = System.Windows.Media.Brushes.DarkBlue;
    else if (_sumaPunktow < 0)
        LblSuma.Foreground = System.Windows.Media.Brushes.DarkRed;
    else
        LblSuma.Foreground = System.Windows.Media.Brushes.Gray;
}
}
}

```

Kluczowe elementy:

- **4 kroki:** Pobierz → Oblicz → Zapisz → Odśwież — zawsze w tej kolejności
- `OdswiezWidok()` — jedna metoda aktualizująca całe UI; wołana po każdej zmianie stanu
- Każdy handler (`BtnDodaj_Click`, `BtnOdejmij_Click`) robi to samo: pobiera, oblicza, zapisuje, odświeża

19.6. Aktualizacja widoku na podstawie stanu — metoda `OdswiezWidok()`

Zamiast aktualizować każdą etykietę osobno w wielu miejscach, tworzysz jedną metodę `OdswiezWidok()`. Wywołujesz ją zawsze gdy stan się zmieni.

MainWindow.xaml:

```

<Window x:Class="StanAplikacji.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="OdswiezWidok - wzorzec" Height="560" Width="520">
    <StackPanel Margin="20">

```

```

<Label Content="Panel sterowania pojazdem" FontSize="16"
        FontWeight="Bold" HorizontalAlignment="Center" Margin="0,0,0,15"/>

<!-- Przyciski sterowania -->
<StackPanel Orientation="Horizontal" HorizontalAlignment="Center" Margin="0,0,0,15">
    <Button Content="▲ Przyspieszenie"
            Height="40"
            Width="130"
            Margin="0,0,10,0"
            Click="BtnPrzyspieszenie_Click"/>
    <Button Content="▼ Hamuj"
            Height="40"
            Width="100"
            Margin="0,0,10,0"
            Click="BtnHamuj_Click"/>
    <Button Content="○ Reset"
            Height="40"
            Width="80"
            Click="BtnReset_Click"/>
</StackPanel>

<StackPanel Orientation="Horizontal" Margin="0,5">
    <Button Content="Włącz silnik"
            Height="35"
            Width="120"
            Margin="0,0,10,0"
            Click="BtnSilnik_Click"/>
    <Button Content="Zatankuj"
            Height="35"
            Width="100"
            Click="BtnTankuj_Click"/>
</StackPanel>

<!-- Panel stanu pojazdu -->
<Border BorderBrush="LightGray" BorderThickness="1"
        Padding="15" CornerRadius="5" Margin="0,15,0,0">
    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <Label Grid.Row="0" Grid.Column="0"
            Content="Stan silnika:"
            FontSize="13" FontWeight="Bold"/>
        <Label x:Name="LblSilnik"
            Grid.Row="0" Grid.Column="1"
            Content="WYŁĄCZONY"
            FontSize="13"
            Foreground="Red"/>

        <Label Grid.Row="1" Grid.Column="0"
            Content="Prędkość:"
            FontSize="13" FontWeight="Bold"/>
        <Label x:Name="LblPredkosc"

```

```

        Grid.Row="1" Grid.Column="1"
        Content="0 km/h"
        FontSize="13"/>

<Label Grid.Row="2" Grid.Column="0"
        Content="Paliwo:"
        FontSize="13" FontWeight="Bold"/>

<Label x:Name="LblPaliwo"
        Grid.Row="2" Grid.Column="1"
        Content="100%"
        FontSize="13"
        Foreground="DarkGreen"/>

<Label Grid.Row="3" Grid.Column="0"
        Content="Przebieg:"
        FontSize="13" FontWeight="Bold"/>

<Label x:Name="LblPrzebieg"
        Grid.Row="3" Grid.Column="1"
        Content="0 km"
        FontSize="13"/>

<Label Grid.Row="4" Grid.Column="0"
        Content="Status:"
        FontSize="13" FontWeight="Bold"/>

<Label x:Name="LblStatusPojazd"
        Grid.Row="4" Grid.Column="1"
        Content="Gotowy"
        FontSize="13"
        Foreground="DarkBlue"/>

</Grid>
</Border>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Media;

namespace StanAplikacji
{
    public partial class MainWindow : Window
    {
        // CAŁY STAN pojazdu jako pola klasy
        private bool _silnikWlaczony = false;
        private int _predkosc = 0;
        private int _paliwo = 100;
        private int _przebieg = 0;

        public MainWindow()
        {
            InitializeComponent();
            OdswiezWidok(); // Inicjalny stan widoku
        }

        private void BtnSilnik_Click(object sender, RoutedEventArgs e)
        {
            if (_paliwo ≤ 0)
            {
                MessageBox.Show("Brak paliwa! Zatankuj najpierw.",

```

```

        "Uwaga", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // Przełącz stan silnika
    _silnikWlaczony = !_silnikWlaczony;
    if (!_silnikWlaczony)
        _predkosc = 0; // wyłączenie silnika → stop

    OdswiezWidok(); // jedyna aktualizacja widoku
}

private void BtnPrzyspieszenie_Click(object sender, RoutedEventArgs e)
{
    if (!_silnikWlaczony)
    {
        MessageBox.Show("Włącz silnik!", "Uwaga",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    if (_paliwo ≤ 0)
    {
        MessageBox.Show("Brak paliwa!", "Uwaga",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    _predkosc = Math.Min(_predkosc + 10, 200); // max 200 km/h
    _paliwo = Math.Max(_paliwo - 5, 0);        // zużycie paliwa
    _przebieg += 1;                            // przebyty km

    OdswiezWidok();
}

private void BtnHamuj_Click(object sender, RoutedEventArgs e)
{
    _predkosc = Math.Max(_predkosc - 10, 0); // nie poniżej 0
    OdswiezWidok();
}

private void BtnTankuj_Click(object sender, RoutedEventArgs e)
{
    _paliwo = 100;
    OdswiezWidok();
    MessageBox.Show("Zatankowano do pełna (100%)!",
        "Tankowanie", MessageBoxButton.OK, MessageBoxImage.Information);
}

private void BtnReset_Click(object sender, RoutedEventArgs e)
{
    _silnikWlaczony = false;
    _predkosc = 0;
    _paliwo = 100;
    _przebieg = 0;
    OdswiezWidok();
}

// JEDNA METODA – aktualizuje WSZYSTKIE etykiety na podstawie stanu
private void OdswiezWidok()
{
    // Silnik

```

```

        LblSilnik.Content = _silnikWlaczony ? "WŁĄCZONY" : "WYŁĄCZONY";
        LblSilnik.Foreground = _silnikWlaczony
            ? Brushes.DarkGreen
            : Brushes.Red;

        // Prędkość
        LblPredkosc.Content = $"{_predkosc} km/h";
        LblPredkosc.Foreground = _predkosc > 120
            ? Brushes.Red
            : (_predkosc > 0 ? Brushes.DarkOrange : Brushes.Black);

        // Paliwo
        LblPaliwo.Content = $"{_paliwo}%";
        if (_paliwo > 30)
            LblPaliwo.Foreground = Brushes.DarkGreen;
        else if (_paliwo > 10)
            LblPaliwo.Foreground = Brushes.Orange;
        else
            LblPaliwo.Foreground = Brushes.Red;

        // Przebieg
        LblPrzebieg.Content = $"{_przebieg} km";

        // Status ogólny
        if (_paliwo ≤ 0)
            LblStatusPojazd.Content = "BRAK PALIWA!";
        else if (!_silnikWlaczony)
            LblStatusPojazd.Content = "Silnik wyłączony";
        else if (_predkosc == 0)
            LblStatusPojazd.Content = "Na biegu jałowym";
        else if (_predkosc > 120)
            LblStatusPojazd.Content = "Przekroczona prędkość!";
        else
            LblStatusPojazd.Content = "Jedzie";

        LblStatusPojazd.Foreground = _paliwo ≤ 0 || _predkosc > 120
            ? Brushes.Red
            : Brushes.DarkBlue;
    }
}
}

```

Kluczowe elementy:

- `OdswiezWidok()` — jedna metoda aktualizująca wszystkie etykiety i kolory
- Wywoływana po każdej zmianie stanu — jeden punkt prawdy
- **Spójność widoku** — gdy zmienia się wiele rzeczy, nie możesz przegapić aktualizacji etykiety
- Kolory zależne od stanu (zielony=OK, pomarańczowy=uwaga, czerwony=błąd)
- Wzorzec: każdy handler kliknięcia kończy się `OdswiezWidok()`

20. Dynamiczna aktualizacja interfejsu

Dynamiczna aktualizacja interfejsu to zmiana wyglądu i zawartości okna w trakcie działania programu — bez ponownego uruchamiania aplikacji. WPF daje Ci pełną kontrolę nad każdym elementem okna z poziomu kodu C#. Możesz zmieniać teksty, kolory, widoczność i obrazy — wszystko w odpowiedzi na akcje użytkownika.

20.1. TextBlock

Każda kontrolka tekstowa w WPF ma właściwość, przez którą ustawiasz jej zawartość:

KONTROLKA	WŁAŚCIWOŚĆ	TYP WARTOŚCI
TextBlock	.Text	string
Label	.Content	object (zwykle string)
Button	.Content	object (zwykle string)
TextBox	.Text	string

Poniższy przykład pokazuje przycisk, który zmienia tekst trzech różnych kontroltek jednocześnie.

MainWindow.xaml:

```
<Window x:Class="DynamicznaAktualizacja.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zmiana tekstu" Height="300" Width="400">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <TextBlock x:Name="tbInformacja"
            Text="Kliknij przycisk, aby zmienić teksty."
            FontSize="14"
            Margin="0,0,0,10"/>

        <Label x:Name="lblStatus"
            Content="Status: oczekiwanie"
            Background="LightGray"
            Margin="0,0,0,10"/>

        <Button x:Name="btnAkcja"
            Content="Uruchom akcję"
            Width="150"
            Height="35"
            Margin="0,0,0,10"
            Click="btnAkcja_Click"/>

        <TextBlock x:Name="tbWynik"
            Text=""
            FontSize="16"
            FontWeight="Bold"
            Foreground="DarkGreen"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace DynamicznaAktualizacja
{
    public partial class MainWindow : Window
    {
        private int _liczbaKlikniec = 0;

        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnAkcja_Click(object sender, RoutedEventArgs e)
        {
            _liczbaKlikniec++;
        }
    }
}
```

```

// Zmiana tekstu w TextBlock
tbInformacja.Text = $"Akcja została wykonana {_liczbaKlikniec} raz(y).";

// Zmiana zawartości Label
lblStatus.Content = $"Status: aktywny (kliknięcie #{_liczbaKlikniec})";

// Zmiana tekstu przycisku
btnAkcja.Content = $"Kliknij ponownie ({_liczbaKlikniec})";

// Wyświetlenie wyniku
tbWynik.Text = $"Łącznie kliknięć: {_liczbaKlikniec}";
    }
}
}

```

Kluczowe elementy:

- `tbInformacja.Text = "..."` — przypisanie nowej wartości bezpośrednio do właściwości
- `lblStatus.Content = "..."` — Label używa `Content` zamiast `Text`
- `btnAkcja.Content = "..."` — Button też używa `Content`
- `$"tekst {zmienna}"` — interpolacja stringów, wygodny sposób łączenia tekstu z wartościami

20.2. Pokazywanie wyniku dopiero po kliknięciu — wynik ukryty (Collapsed), pokazany po akcji

Często chcesz ukryć sekcję wyników i pokazać ją dopiero gdy użytkownik kliknie przycisk. Służy do tego właściwość `Visibility` ustawiona domyślnie na `Collapsed`.

MainWindow.xaml:

```

<Window x:Class="UkrytyWynik.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Ukryty wynik" Height="350" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Podaj dwie liczby:" FontSize="14" Margin="0,0,0,5"/>

        <TextBox x:Name="txtLiczba1"
                Margin="0,0,0,5"/>

        <TextBox x:Name="txtLiczba2"
                Margin="0,0,0,10"/>

        <Button Content="Oblicz sumę"
                Click="btnOblicz_Click"
                Width="120"
                Height="35"
                HorizontalAlignment="Left"/>

        <!-- Panel wyników - domyślnie niewidoczny -->
        <Border x:Name="paneLWynik"
                Visibility="Collapsed"
                Background="LightGreen"
                CornerRadius="5"
                Padding="10"
                Margin="0,15,0,0">
            <StackPanel>
                <TextBlock Text="Wynik obliczenia:"
                        FontWeight="Bold"
                        FontSize="14"/>

                <TextBlock x:Name="tbWynik"
                        FontSize="20"

```

```

                FontWeight="Bold"
                Foreground="DarkGreen" />
            </StackPanel>
        </Border>

        <!-- Komunikat błędu – domyślnie niewidoczny -->
        <TextBlock x:Name="tbBład"
            Visibility="Collapsed"
            Text=""
            Foreground="Red"
            Margin="0,10,0,0" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace UkrytyWynik
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void btnOblicz_Click(object sender, RoutedEventArgs e)
            {
                // Ukryj poprzednie wyniki i błędy
                panelWynik.Visibility = Visibility.Collapsed;
                tbBład.Visibility = Visibility.Collapsed;

                bool ok1 = double.TryParse(txtLiczba1.Text, out double liczba1);
                bool ok2 = double.TryParse(txtLiczba2.Text, out double liczba2);

                if (!ok1 || !ok2)
                {
                    // Pokaż komunikat błędu
                    tbBład.Text = "Błąd: Podaj poprawne liczby w obu polach.";
                    tbBład.Visibility = Visibility.Visible;
                    return;
                }

                // Oblicz i pokaż wynik
                double suma = liczba1 + liczba2;
                tbWynik.Text = $"{liczba1} + {liczba2} = {suma}";
                panelWynik.Visibility = Visibility.Visible;
            }
        }
    }
}

```

Kluczowe elementy:

- `Visibility="Collapsed"` w XAML — element domyślnie niewidoczny i nie zajmuje miejsca
- `panelWynik.Visibility = Visibility.Visible` — pokazuje element po kliknięciu
- `panelWynik.Visibility = Visibility.Collapsed` — chowanie przed nowym obliczeniem (reset)

20.3. Ukrywanie i pokazywanie elementów — Visibility.Visible, Hidden, Collapsed — różnice

WPF oferuje trzy stany widoczności. Każdy działa inaczej:

WARTOŚĆ	ELEMENT WIDOCZNY?	ZAJMUJE MIEJSCE W UKŁADZIE?
Visibility.Visible	TAK	TAK
Visibility.Hidden	NIE	TAK
Visibility.Collapsed	NIE	NIE

MainWindow.xaml:

```
<Window x:Class="VisibilityDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Visibility – demo" Height="400" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Demonstracja trzech stanów Visibility"
                   FontSize="14" FontWeight="Bold" Margin="0,0,0,15"/>

        <!-- Trzy przyciski sterujące -->
        <StackPanel Orientation="Horizontal" Margin="0,0,0,15">
            <Button Content="Visible" Width="100" Margin="0,0,5,0"
                    Click="btnVisible_Click"/>
            <Button Content="Hidden" Width="100" Margin="0,0,5,0"
                    Click="btnHidden_Click"/>
            <Button Content="Collapsed" Width="100"
                    Click="btnCollapsed_Click"/>
        </StackPanel>

        <!-- Element, którego widoczność zmieniamy -->
        <Border Background="SkyBlue" Height="60" CornerRadius="5"
                Margin="0,0,0,5">
            <TextBlock x:Name="elementDoUkrycia"
                       Text="TEN ELEMENT SIĘ CHOWA / POJAWIA"
                       HorizontalAlignment="Center"
                       VerticalAlignment="Center"
                       FontWeight="Bold"/>
        </Border>

        <!-- Ten element jest zawsze widoczny – pokazuje efekt -->
        <Border Background="LightCoral" Height="60" CornerRadius="5">
            <TextBlock Text="TEN ELEMENT JEST ZAWSZE WIDOCZNY"
                       HorizontalAlignment="Center"
                       VerticalAlignment="Center"
                       FontWeight="Bold"/>
        </Border>

        <TextBlock x:Name="tbOpis" Text="" Margin="0,15,0,0"
                   TextWrapping="Wrap" FontSize="13"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace VisibilityDemo
{
    public partial class MainWindow : Window
    {
        // ...
    }
}
```

```

public MainWindow()
{
    InitializeComponent();
}

private void btnVisible_Click(object sender, RoutedEventArgs e)
{
    elementDoUkrycia.Visibility = Visibility.Visible;
    tbOpis.Text = "Visible: element jest widoczny i zajmuje miejsce w układzie.";
}

private void btnHidden_Click(object sender, RoutedEventArgs e)
{
    elementDoUkrycia.Visibility = Visibility.Hidden;
    tbOpis.Text = "Hidden: element jest NIEWIDOCZNY, ale nadal ZAJMUJE MIEJSCE. " +
        "Zobaczysz pustą przestrzeń tam, gdzie był.";
}

private void btnCollapsed_Click(object sender, RoutedEventArgs e)
{
    elementDoUkrycia.Visibility = Visibility.Collapsed;
    tbOpis.Text = "Collapsed: element jest NIEWIDOCZNY i NIE ZAJMUJE miejsca. " +
        "Dolny element przesunął się w górę.";
}
}
}

```

Kiedy używać którego stanu:

- **Visible** — standardowy, element widoczny i aktywny
- **Hidden** — gdy chcesz zachować układ (zarezerwować miejsce), ale element ma nie być widoczny
- **Collapsed** — najczęściej używany przy ukrywaniu wyników, formularzy, paneli — element znika całkowicie

20.4. Source

Kontrolka **Image** może wyświetlać różne obrazki w zależności od akcji użytkownika. Zmieniasz właściwość **Source** przypisując nowy obiekt **BitmapImage**.

MainWindow.xaml:

```

<Window x:Class="ZmianaObrazka.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Galeria obrazków" Height="450" Width="500">
    <StackPanel Margin="20">
        <TextBlock Text="Galeria – kliknij przycisk aby zmienić obrazek"
            FontSize="13" Margin="0,0,0,10"/>

        <Image x:Name="imgGaleria"
            Width="400" Height="250"
            Stretch="Uniform"
            Source="obrazki/kot.jpg"
            Margin="0,0,0,10"/>

        <TextBlock x:Name="tbNazwaObrazka"
            Text="Aktualny obrazek: kot.jpg"
            HorizontalAlignment="Center"
            Margin="0,0,0,10"/>

        <StackPanel Orientation="Horizontal" HorizontalAlignment="Center">
            <Button Content="Kot" Width="80" Margin="5"
                Click="btnKot_Click"/>

```

```

        <Button Content="Pies" Width="80" Margin="5"
                Click="btnPies_Click" />

        <Button Content="Ptak" Width="80" Margin="5"
                Click="btnPtak_Click" />

    </StackPanel>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Media.Imaging;

namespace ZmianaObrazka
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void ZmienObrazek(string nazwaPliku)
        {
            try
            {
                // Buduj ścieżkę względną do katalogu aplikacji
                string katalogAplikacji = AppDomain.CurrentDomain.BaseDirectory;
                string sciezka = System.IO.Path.Combine(katalogAplikacji, "obrazki", nazwaPliku);

                // Utwórz nowy BitmapImage i przypisz do Image.Source
                BitmapImage bmp = new BitmapImage();
                bmp.BeginInit();
                bmp.UriSource = new Uri(sciezka, UriKind.Absolute);
                bmp.EndInit();

                imgGaleria.Source = bmp;
                tbNazwaObrazka.Text = $"Aktualny obrazek: {nazwaPliku}";
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Nie można wczytać obrazka: {ex.Message}", "Błąd");
            }
        }

        private void btnKot_Click(object sender, RoutedEventArgs e)
        {
            ZmienObrazek("kot.jpg");
        }

        private void btnPies_Click(object sender, RoutedEventArgs e)
        {
            ZmienObrazek("pies.jpg");
        }

        private void btnPtak_Click(object sender, RoutedEventArgs e)
        {
            ZmienObrazek("ptak.jpg");
        }
    }
}

```

```
}  
  
}
```

Kluczowe elementy:

- `BitmapImage` — klasa do ładowania obrazków z pliku lub zasobu
- `bmp.BeginInit()` / `bmp.EndInit()` — wymagane przy ręcznym tworzeniu `BitmapImage`
- `UriKind.Absolute` — podajesz pełną ścieżkę do pliku
- `AppDomain.CurrentDomain.BaseDirectory` — katalog gdzie znajduje się plik `.exe` aplikacji

20.5. TextBlock

Kolory w WPF ustawiasz przez właściwości takie jak `Fill`, `Background` i `Foreground`. W C# używasz klasy `Brushes` lub tworzysz `SolidColorBrush`.

MainWindow.xaml:

```
<Window x:Class="ZmianaKoloru.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Zmiana kolorów" Height="400" Width="450">  
    <StackPanel Margin="20">  
        <TextBlock Text="Zmiana kolorów elementów WPF"  
            FontSize="15" FontWeight="Bold" Margin="0,0,0,15"/>  
  
        <!-- Rectangle - zmiana Fill -->  
        <Rectangle x:Name="rect"  
            Width="200" Height="80"  
            Fill="LightGray"  
            Margin="0,0,0,10"/>  
  
        <!-- Border - zmiana Background -->  
        <Border x:Name="brd"  
            Width="200" Height="60"  
            Background="LightGray"  
            CornerRadius="8"  
            Margin="0,0,0,10">  
            <TextBlock Text="Ramka z kolorem"  
                HorizontalAlignment="Center"  
                VerticalAlignment="Center"/>  
        </Border>  
  
        <!-- TextBlock - zmiana Foreground -->  
        <TextBlock x:Name="tbKolorowy"  
            Text="Ten tekst zmienia kolor"  
            FontSize="18"  
            FontWeight="Bold"  
            HorizontalAlignment="Center"  
            Margin="0,0,0,15"/>  
  
        <!-- Przyciski kolorów -->  
        <StackPanel Orientation="Horizontal" HorizontalAlignment="Center">  
            <Button Content="Czerwony" Width="90" Margin="4"  
                Background="LightCoral" Click="btnCzerwony_Click"/>  
            <Button Content="Zielony" Width="90" Margin="4"  
                Background="LightGreen" Click="btnZielony_Click"/>  
            <Button Content="Niebieski" Width="90" Margin="4"  
                Background="LightBlue" Click="btnNiebieski_Click"/>  
        </StackPanel>  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;
using System.Windows.Media;

namespace ZmianaKoloru
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void UstawKolory(Color kolor)
            {
                SolidColorBrush pedzel = new SolidColorBrush(kolor);

                // Zmiana koloru wypełnienia Rectangle
                rect.Fill = pedzel;

                // Zmiana tła Border
                brd.Background = new SolidColorBrush(
                    Color.FromArgb(80, kolor.R, kolor.G, kolor.B) // półprzezroczysty
                );

                // Zmiana koloru tekstu TextBlock
                tbKolorowy.Foreground = pedzel;
            }

            private void btnCzerwony_Click(object sender, RoutedEventArgs e)
            {
                UstawKolory(Colors.Crimson);
            }

            private void btnZielony_Click(object sender, RoutedEventArgs e)
            {
                UstawKolory(Colors.ForestGreen);
            }

            private void btnNiebieski_Click(object sender, RoutedEventArgs e)
            {
                UstawKolory(Colors.SteelBlue);
            }
        }
    }
}
```

Kluczowe elementy:

- `Brushes.Red` — gotowe, statyczne pędzle z klasy `Brushes`
- `new SolidColorBrush(kolor)` — tworzysz pędzel z konkretnego koloru
- `Colors.Crimson` — kolory z klasy `Colors` (typ `Color`)
- `Color.FromArgb(a, r, g, b)` — kolor z kanałem przezroczystości (0–255)

20.6. Metoda `OdswiezWidok()` — centralna metoda aktualizująca wiele elementów naraz

Kiedy aplikacja ma wiele elementów do aktualizacji, dobrą praktyką jest stworzenie jednej metody `OdswiezWidok()`, która aktualizuje wszystko naraz. Dzięki temu zawsze masz pewność, że interfejs jest spójny.

Poniższy przykład symuluje przeglądarkę albumów muzycznych.

MainWindow.xaml:


```

<Window x:Class="PrzeglądarkaAlbumow.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Przeglądarka albumów" Height="500" Width="550">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <!-- Nagłówek -->
        <Border Grid.Row="0" Background="DarkSlateBlue" Padding="15">
            <TextBlock Text="🎵 Przeglądarka Albumów"
                Foreground="White" FontSize="18" FontWeight="Bold"/>
        </Border>

        <!-- Szczegóły albumu -->
        <StackPanel Grid.Row="1" Margin="20">
            <TextBlock Text="Informacje o albumie:" FontSize="13"
                Foreground="Gray" Margin="0,0,0,5"/>

            <TextBlock x:Name="tbTytuł" FontSize="22" FontWeight="Bold"
                Foreground="DarkSlateBlue" Margin="0,0,0,3"/>
            <TextBlock x:Name="tbArtysta" FontSize="16"
                Foreground="SlateGray" Margin="0,0,0,3"/>
            <TextBlock x:Name="tbRok" FontSize="14"
                Foreground="Gray" Margin="0,0,0,10"/>

            <Separator Margin="0,0,0,10"/>

            <StackPanel Orientation="Horizontal" Margin="0,0,0,5">
                <TextBlock Text="Liczba utworów: " FontSize="14" FontWeight="Bold"/>
                <TextBlock x:Name="tbUtworów" FontSize="14"/>
            </StackPanel>

            <StackPanel Orientation="Horizontal">
                <TextBlock Text="Pobrań: " FontSize="14" FontWeight="Bold"/>
                <TextBlock x:Name="tbPobran" FontSize="14" Foreground="ForestGreen"/>
            </StackPanel>

            <!-- Wskaźnik popularności -->
            <TextBlock Text="Popularność:" FontSize="13" Margin="0,15,0,5"/>
            <ProgressBar x:Name="pbPopularnosc"
                Height="20" Minimum="0" Maximum="1000"/>

            <TextBlock x:Name="tbNumer"
                HorizontalAlignment="Center"
                FontSize="13" Foreground="Gray"
                Margin="0,15,0,0"/>
        </StackPanel>

        <!-- Nawigacja -->
        <StackPanel Grid.Row="2" Orientation="Horizontal"
            HorizontalAlignment="Center" Margin="10">
            <Button Content="◀ Poprzedni" Width="110" Height="35"
                Margin="5" Click="btnPoprzedni_Click"/>
            <Button Content="Następny ▶" Width="110" Height="35"
                Margin="5" Click="btnNastepny_Click"/>
        </StackPanel>
    </Grid>

```

```
</Grid>

</Window>
```

MainWindow.xaml.cs:

```
using System.Collections.Generic;
using System.Windows;

namespace PrzeglądarkaAlbumow
{
    // Prosta klasa danych
    public class Album
    {
        public string Tytuł { get; set; }
        public string Artysta { get; set; }
        public int Rok { get; set; }
        public int LiczbaUtworow { get; set; }
        public int LiczbaPobran { get; set; }
    }

    public partial class MainWindow : Window
    {
        private List<Album> _albumy;
        private int _aktualnyIndeks = 0;

        public MainWindow()
        {
            InitializeComponent();
            ZaładujAlbumy();
            OdświeżWidok();
        }

        private void ZaładujAlbumy()
        {
            _albumy = new List<Album>
            {
                new Album { Tytuł = "Nevermind", Artysta = "Nirvana",
                    Rok = 1991, LiczbaUtworow = 13, LiczbaPobran = 854 },
                new Album { Tytuł = "Abbey Road", Artysta = "The Beatles",
                    Rok = 1969, LiczbaUtworow = 17, LiczbaPobran = 972 },
                new Album { Tytuł = "Thriller", Artysta = "Michael Jackson",
                    Rok = 1982, LiczbaUtworow = 9, LiczbaPobran = 1000 },
                new Album { Tytuł = "Dark Side of the Moon", Artysta = "Pink Floyd",
                    Rok = 1973, LiczbaUtworow = 10, LiczbaPobran = 931 }
            };
        }

        // =====
        // CENTRALNA METODA AKTUALIZACJI WIDOKU
        // =====
        private void OdświeżWidok()
        {
            Album album = _albumy[_aktualnyIndeks];

            // Aktualizacja wszystkich kontroltek tekstowych
            tbTytuł.Text = album.Tytuł;
            tbArtysta.Text = album.Artysta;
            tbRok.Text = $"Rok wydania: {album.Rok}";
            tbUtworow.Text = album.LiczbaUtworow.ToString();
            tbPobran.Text = $"{album.LiczbaPobran:N0}";
            tbNumer.Text = $"Album {_aktualnyIndeks + 1} z {_albumy.Count}";
        }
    }
}
```

```
// Aktualizacja paska postępu
pbPopularnosc.Value = album.LiczbaPobran;
}

private void btnPoprzedni_Click(object sender, RoutedEventArgs e)
{
    if (_aktualnyIndeks > 0)
    {
        _aktualnyIndeks--;
        OdswiezWidok(); // jedyne wywołanie
    }
}

private void btnNastepny_Click(object sender, RoutedEventArgs e)
{
    if (_aktualnyIndeks < _albumy.Count - 1)
    {
        _aktualnyIndeks++;
        OdswiezWidok(); // jedyne wywołanie
    }
}
}
```

Kluczowe elementy:

- `OdswiezWidok()` — jedna metoda odpowiedzialna za cały widok; wywołujesz ją w każdym miejscu gdzie dane się zmieniają
- Dzięki tej metodzie** — nie musisz pamiętać o aktualizacji każdego elementu osobno, co eliminuje błędy
- `{album.LiczbaPobran:N0}` — formatowanie liczby z separatorem tysięcy

21. Pliki i dane zewnętrzne

Aplikacje WPF często potrzebują wczytać dane z pliku tekstowego (np. lista albumów, pytania quizu, produkty) lub zapisać wyniki do pliku. W tej sekcji zobaczysz wszystkie podstawowe operacje na plikach.

21.1. Text

Klasa `File` z przestrzeni nazw `System.IO` oferuje proste metody do odczytu pliku:

METODA	ZWRACA	KIEDY UŻYWAĆ
<code>File.ReadAllLines(ścieżka)</code>	<code>string[]</code>	Gdy każda linia to osobny rekord
<code>File.ReadAllText(ścieżka)</code>	<code>string</code>	Gdy chcesz cały tekst naraz
<code>File.ReadAllLines(ścieżka, Encoding.UTF8)</code>	<code>string[]</code>	Gdy plik zawiera polskie znaki

MainWindow.xaml:

```
<Window x:Class="OdczytPliku.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Odczyt pliku" Height="400" Width="500">
<StackPanel Margin="20">
    <TextBlock Text="Odczyt pliku tekstowego" FontSize="15"
        FontWeight="Bold" Margin="0,0,0,10"/>

    <Button Content="Wczytaj plik linia po linii (ReadAllLines)"
        Click="btnReadLines_Click" Height="35" Margin="0,0,0,5"/>

    <Button Content="Wczytaj cały tekst (ReadAllText)"
```

```

Click="btnReadText_Click" Height="35" Margin="0,0,0,10"/>

<TextBlock Text="Zawartość pliku:" FontWeight="Bold" Margin="0,0,0,5"/>
<ScrollView Height="200" VerticalScrollBarVisibility="Auto">
    <TextBlock x:Name="tbZawartosc"
        TextWrapping="Wrap"
        FontFamily="Consolas"
        FontSize="12"/>
</ScrollView>

<TextBlock x:Name="tbInfo" Text="" Foreground="Gray"
    Margin="0,10,0,0"/>

</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.IO;
using System.Text;
using System.Windows;

namespace OdczytPliku
{
    public partial class MainWindow : Window
    {
        // Ścieżka do pliku względem katalogu aplikacji
        private readonly string _sciezkaPliku;

        public MainWindow()
        {
            InitializeComponent();

            string katalog = AppDomain.CurrentDomain.BaseDirectory;
            _sciezkaPliku = Path.Combine(katalog, "dane.txt");

            // Utwórz przykładowy plik jeśli nie istnieje
            if (!File.Exists(_sciezkaPliku))
            {
                File.WriteAllText(_sciezkaPliku,
                    "Nevermind;Nirvana;1991;13\n" +
                    "Abbey Road;The Beatles;1969;17\n" +
                    "Thriller;Michael Jackson;1982;9\n",
                    Encoding.UTF8);
            }
        }

        private void btnReadLines_Click(object sender, RoutedEventArgs e)
        {
            try
            {
                // Odczyt linii – każda linia jako osobny string w tablicy
                string[] linie = File.ReadAllLines(_sciezkaPliku, Encoding.UTF8);

                StringBuilder sb = new StringBuilder();
                for (int i = 0; i < linie.Length; i++)
                {
                    sb.AppendLine($"Linia {i + 1}: {linie[i]}");
                }

                tbZawartosc.Text = sb.ToString();
                tbInfo.Text = $"Wczytano {linie.Length} linii za pomocą ReadAllLines.";
            }
            catch { }
        }
    }
}

```

```

    }
    catch (FileNotFoundException)
    {
        tbZawartosc.Text = "Plik nie został znaleziony.";
    }
    catch (IOException ex)
    {
        tbZawartosc.Text = $"Błąd odczytu: {ex.Message}";
    }
}

private void btnReadText_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // Odczyt całego tekstu naraz
        string caLyTekst = File.ReadAllText(_sciezkaPliku, Encoding.UTF8);
        tbZawartosc.Text = caLyTekst;
        tbInfo.Text = $"Wczytano {caLyTekst.Length} znaków za pomocą ReadAllText.";
    }
    catch (Exception ex)
    {
        tbZawartosc.Text = $"Błąd: {ex.Message}";
    }
}
}
}

```

Kluczowe elementy:

- `Encoding.UTF8` — zawsze podawaj kodowanie gdy plik może zawierać polskie litery (ą, ę, ó itp.)
- `File.ReadAllLines` — zwraca tablicę `string[]`, każda linia to jeden element tablicy
- `File.ReadAllText` — zwraca jeden długi `string` z całą zawartością pliku

21.2. Format danych w pliku — parsowanie struktury (Split, TryParse, kolejność linii)

Pliki tekstowe często przechowują dane w określonym formacie. Najczęściej używasz separatora (np. `;` lub `,`) do oddzielenia pól.

Przykład formatu pliku `albumy.txt`:

```

Nevermind;Nirvana;1991;13
Abbey Road;The Beatles;1969;17
Thriller;Michael Jackson;1982;9

```

MainWindow.xaml:

```

<Window x:Class="ParseowaniePliku.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Parsowanie danych z pliku" Height="380" Width="500">
    <StackPanel Margin="20">
        <TextBlock Text="Parsowanie danych z pliku CSV" FontSize="15"
                    FontWeight="Bold" Margin="0,0,0,10"/>
        <Button Content="Wczytaj i parsuj albumy.txt"
                Click="btnWczytaj_Click" Height="35" Margin="0,0,0,10"/>
        <ListBox x:Name="lstAlbumy" Height="200"/>
        <TextBlock x:Name="tbStatus" Text="" Margin="0,10,0,0"
                    Foreground="Gray"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.IO;
using System.Text;
using System.Windows;

namespace ParseowaniePliku
{
    public partial class MainWindow : Window
    {
        private readonly string _sciezka;

        public MainWindow()
        {
            InitializeComponent();
            _sciezka = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "albumy.txt");
        }

        private void btnWczytaj_Click(object sender, RoutedEventArgs e)
        {
            lstAlbumy.Items.Clear();

            if (!File.Exists(_sciezka))
            {
                tbStatus.Text = "Plik albumy.txt nie istnieje.";
                return;
            }

            string[] linie = File.ReadAllLines(_sciezka, Encoding.UTF8);
            int poprawnych = 0;

            foreach (string linia in linie)
            {
                // Pomijaj puste linie
                if (string.IsNullOrEmpty(linia)) continue;

                // Podziel linię po separatorze ';'
                string[] czesci = linia.Split(';');

                // Sprawdź czy linia ma wymagane 4 pola
                if (czesci.Length < 4) continue;

                string tytuł = czesci[0].Trim();
                string artysta = czesci[1].Trim();
            }
        }
    }
}

```

```

        // TryParse – bezpieczna konwersja stringa na int
        if (!int.TryParse(czesci[2].Trim(), out int rok)) rok = 0;
        if (!int.TryParse(czesci[3].Trim(), out int liczbaUtworow)) liczbaUtworow = 0;

        string pozycja = $"{tytul} – {artysta} ({rok}), {liczbaUtworow} utworów";
        lstAlbumy.Items.Add(pozycja);
        poprawnych++;
    }

    tbStatus.Text = $"Wczytano {poprawnych} albumów z {linii.Length} linii.";
}
}
}
}
}

```

Kluczowe elementy:

- `linia.Split(';')` — dzieli string na tablicę po znaku separatora
- `czesci[0].Trim()` — usuwa spacje z początku i końca każdego pola
- `int.TryParse(..., out int rok)` — bezpieczna konwersja; zwraca `false` jeśli string nie jest liczbą, nie rzuca wyjątku

21.3. Tworzenie obiektów z danych tekstowych

Po parsowaniu linii tworzysz obiekty klasy modelu i dodajesz je do listy.

MainWindow.xaml:

```

<Window x:Class="ObiektyZPliku.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Obiekty z pliku" Height="420" Width="520">
    <StackPanel Margin="20">
        <Button Content="Wczytaj albumy z pliku"
                Click="btnWczytaj_Click" Height="35" Margin="0,0,0,10" />
        <ListBox x:Name="lstAlbumy" Height="180"
                SelectionChanged="lstAlbumy_SelectionChanged" />
        <Border Background="AliceBlue" CornerRadius="5"
                Padding="10" Margin="0,10,0,0">
            <StackPanel>
                <TextBlock Text="Szczegóły wybranego albumu:" FontWeight="Bold" />
                <TextBlock x:Name="tbSzczegoly" Text="(wybierz album z listy)"
                        Margin="0,5,0,0" TextWrapping="Wrap" />
            </StackPanel>
        </Border>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Windows;
using System.Windows.Controls;

namespace ObiektyZPliku
{
    public class Album
    {
        public string Tytul { get; set; }
    }
}

```

```

public string Artysta { get; set; }
public int Rok { get; set; }
public int LiczbaUtworow { get; set; }

public override string ToString()
{
    return $"{Tytuł} – {Artysta}";
}
}

public partial class MainWindow : Window
{
    private List<Album> _albumy = new List<Album>();
    private readonly string _sciezka;

    public MainWindow()
    {
        InitializeComponent();
        _sciezka = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "albumy.txt");
    }

    private List<Album> WczytajAlbumyZPliku(string sciezka)
    {
        var lista = new List<Album>();
        string[] linie = File.ReadAllLines(sciezka, Encoding.UTF8);

        foreach (string linia in linie)
        {
            if (string.IsNullOrEmpty(linia)) continue;
            string[] p = linia.Split(';');
            if (p.Length < 4) continue;

            int.TryParse(p[2].Trim(), out int rok);
            int.TryParse(p[3].Trim(), out int liczba);

            // Tworzenie nowego obiektu Album z danych z pliku
            Album album = new Album
            {
                Tytuł = p[0].Trim(),
                Artysta = p[1].Trim(),
                Rok = rok,
                LiczbaUtworow = liczba
            };

            lista.Add(album);
        }

        return lista;
    }

    private void btnWczytaj_Click(object sender, RoutedEventArgs e)
    {
        if (!File.Exists(_sciezka))
        {
            MessageBox.Show("Brak pliku albumy.txt!", "Błąd");
            return;
        }

        _albumy = WczytajAlbumyZPliku(_sciezka);
        lstAlbumy.ItemsSource = null;
        lstAlbumy.ItemsSource = _albumy; // ToString() wywoływane automatycznie
    }
}

```



```

private void lstAlbumy_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (lstAlbumy.SelectedItem is Album wybrany)
    {
        tbSzczegoly.Text =
            $"Tytuł: {wybrany.Tytuł}\n" +
            $"Artysta: {wybrany.Artysta}\n" +
            $"Rok: {wybrany.Rok}\n" +
            $"Liczba utworów: {wybrany.LiczbaUtworow}";
    }
}
}
}
}

```

21.4. Text

METODA	DZIAŁANIE
<code>File.WriteAllText(ścieżka, tekst)</code>	Nadpisuje plik (tworzy jeśli nie istnieje)
<code>File.AppendAllText(ścieżka, tekst)</code>	Dodaje tekst na koniec pliku

MainWindow.xaml:

```

<Window x:Class="ZapisPliku.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zapis do pliku" Height="380" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Notatnik – zapis do pliku" FontSize="15"
            FontWeight="Bold" Margin="0,0,0,10"/>
        <TextBox x:Name="txtNotatka"
            Height="150"
            TextWrapping="Wrap"
            AcceptsReturn="True"
            VerticalScrollBarVisibility="Auto"
            Margin="0,0,0,10"/>
        <StackPanel Orientation="Horizontal">
            <Button Content="Zapisz (nadpisz)" Width="130" Height="35"
                Margin="0,0,5,0" Click="btnZapisz_Click"/>
            <Button Content="Dołącz na koniec" Width="130" Height="35"
                Click="btnDolacz_Click"/>
        </StackPanel>
        <TextBlock x:Name="tbStatus" Text="" Foreground="Green"
            Margin="0,10,0,0"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.IO;
using System.Text;
using System.Windows;

namespace ZapisPliku
{
    public partial class MainWindow : Window
    {

```

```

private readonly string _sciezka;

public MainWindow()
{
    InitializeComponent();
    _sciezka = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "notatka.txt");
}

private void btnZapisz_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // Nadpisuje istniejącą zawartość pliku
        File.WriteAllText(_sciezka, txtNotatka.Text, Encoding.UTF8);
        tbStatus.Text = $"Zapisano pomyślnie: {_sciezka}";
    }
    catch (IOException ex)
    {
        tbStatus.Text = $"Błąd zapisu: {ex.Message}";
    }
}

private void btnDolacz_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // Dołącza tekst na koniec pliku (nie nadpisuje)
        File.AppendAllText(_sciezka, txtNotatka.Text + Environment.NewLine, Encoding.UTF8);
        tbStatus.Text = "Tekst dodany na koniec pliku.";
    }
    catch (IOException ex)
    {
        tbStatus.Text = $"Błąd: {ex.Message}";
    }
}
}
}

```

21.5. SaveFileDialog

`SaveFileDialog` to standardowe okno systemowe "Zapisz jako". Użytkownik wybiera lokalizację i nazwę pliku, a Ty dostajesz ścieżkę.

MainWindow.xaml:

```

<Window x:Class="DialogZapisu.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zapis z dialogiem" Height="420" Width="500">
    <StackPanel Margin="20">
        <TextBlock Text="Edytor z zapisem przez dialog" FontSize="15"
                  FontWeight="Bold" Margin="0,0,0,10"/>

        <TextBox x:Name="txtEdytor"
                 Height="200"
                 TextWrapping="Wrap"
                 AcceptsReturn="True"
                 VerticalScrollBarVisibility="Auto"
                 FontFamily="Consolas"
                 Text="Tu wpisz tekst do zapisania..."
                 Margin="0,0,0,10"/>
    
```

```

<StackPanel Orientation="Horizontal" Margin="0,0,0,10">
    <Button Content="💾 Zapisz jako..."
        Width="130" Height="35" Margin="0,0,0,10"
        Click="btnZapiszJako_Click" />
    <Button Content="📄 Otwórz..."
        Width="130" Height="35"
        Click="btnOtworz_Click" />
</StackPanel>

<TextBlock x:Name="tbSciezka"
    Text="(brak wybranego pliku)"
    TextWrapping="Wrap"
    Foreground="Gray"
    FontSize="12" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.IO;
using System.Text;
using System.Windows;
using Microsoft.Win32; // SaveFileDialog i OpenFileDialog są tu

namespace DialogZapisu
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnZapiszJako_Click(object sender, RoutedEventArgs e)
        {
            // Tworzenie obiektu SaveFileDialog
            SaveFileDialog dialog = new SaveFileDialog();

            // Konfiguracja dialogu
            dialog.Title = "Zapisz plik tekstowy";
            dialog.Filter = "Pliki tekstowe (*.txt)|*.txt|Wszystkie pliki (*.*)|*.*";
            dialog.FilterIndex = 1; // domyślnie pierwszy filtr
            dialog.DefaultExt = "txt"; // domyślne rozszerzenie
            dialog.InitialDirectory = Environment.GetFolderPath(Environment.SpecialFolder.Desktop);

            // Wyświetlenie dialogu – ShowDialog() zwraca true jeśli użytkownik kliknął OK
            bool? wynik = dialog.ShowDialog();

            if (wynik == true)
            {
                // dialog.FileName zawiera pełną ścieżkę do pliku
                string sciezka = dialog.FileName;

                try
                {
                    File.WriteAllText(sciezka, txtEdytor.Text, Encoding.UTF8);
                    tbSciezka.Text = $"Zapisano do: {sciezka}";
                    tbSciezka.Foreground = System.Windows.Media.Brushes.DarkGreen;
                }
                catch (IOException ex)

```

```

        {
            MessageBox.Show($"Błąd zapisu: {ex.Message}", "Błąd",
                MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
    else
    {
        {
            tbSciezka.Text = "AnuLowano zapis.";
        }
    }
}

private void btnOtworz_Click(object sender, RoutedEventArgs e)
{
    OpenFileDialog dialog = new OpenFileDialog();
    dialog.Title = "Otwórz plik tekstowy";
    dialog.Filter = "Pliki tekstowe (*.txt)|*.txt|Wszystkie pliki (*.*)|*.*";
    dialog.InitialDirectory = Environment.GetFolderPath(Environment.SpecialFolder.Desktop);

    bool? wynik = dialog.ShowDialog();

    if (wynik == true)
    {
        try
        {
            txtEdytor.Text = File.ReadAllText(dialog.FileName, Encoding.UTF8);
            tbSciezka.Text = $"Otwarto: {dialog.FileName}";
            tbSciezka.Foreground = System.Windows.Media.Brushes.SteelBlue;
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Błąd odczytu: {ex.Message}", "Błąd");
        }
    }
}
}
}
}

```

Kluczowe elementy SaveFileDialog:

- `dialog.Filter` — format `"Opis (*.ext)|*.ext"`, możesz łączyć kilka filtrów pipem `|`
- `dialog.FilterIndex` — który filtr jest domyślnie wybrany (od 1)
- `dialog.InitialDirectory` — katalog który otworzy się domyślnie
- `dialog.ShowDialog()` — wyświetla okno; zwraca `true` gdy użytkownik kliknął "Zapisz"
- `dialog.FileName` — pełna ścieżka wybrana przez użytkownika (dostępna po ShowDialog)

21.6. OpenFileDialog

`OpenFileDialog` działa podobnie do `SaveFileDialog`, ale służy do wyboru istniejącego pliku. Szczegółowy przykład pokazano już w poprzednim podrozdziale (metoda `btnOtworz_Click`). Poniżej widzisz kluczowe różnice:

```

using Microsoft.Win32; // OpenFileDialog i SaveFileDialog są w tym namespace, nie w System.Windows.Forms!

// Tylko odczyt istniejącego pliku
OpenFileDialog dialog = new OpenFileDialog();
dialog.Filter = "Obrazy (*.jpg;*.png)|*.jpg;*.png|Wszystkie|*.*";
dialog.Multiselect = false; // czy można wybrać wiele plików

if (dialog.ShowDialog() == true)
{
    string wybranyPlik = dialog.FileName;
}

```

```
// działaj na pliku...  
}
```

21.7. Sprawdzanie istnienia pliku: File.Exists

Przed próbą odczytu zawsze sprawdzaj czy plik istnieje. Inaczej program rzuci wyjątek.

MainWindow.xaml:

```
<Window x:Class="SprawdzaniePliku.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Sprawdzanie pliku" Height="300" Width="450">  
    <StackPanel Margin="20">  
        <TextBlock Text="Wpisz ścieżkę do pliku:" Margin="0,0,0,5"/>  
        <TextBox x:Name="txtSciezka"  
                Text="C:\Users\Publiczny\test.txt"  
                Margin="0,0,0,10"/>  
        <Button Content="Sprawdź czy plik istnieje"  
                Click="btnSprawdz_Click" Height="35" Margin="0,0,0,10"/>  
        <TextBlock x:Name="tbWynik" Text="" FontSize="14" TextWrapping="Wrap"/>  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System.IO;  
using System.Windows;  
  
namespace SprawdzaniePliku  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
  
        private void btnSprawdz_Click(object sender, RoutedEventArgs e)  
        {  
            string sciezka = txtSciezka.Text.Trim();  
  
            // File.Exists zwraca true lub false – nigdy nie rzuca wyjątku  
            if (File.Exists(sciezka))  
            {  
                FileInfo info = new FileInfo(sciezka);  
                tbWynik.Text = $"✅ Plik ISTNIEJE\n" +  
                    $"Rozmiar: {info.Length} bajtów\n" +  
                    $"Ostatnia zmiana: {info.LastWriteTime}";  
                tbWynik.Foreground = System.Windows.Media.Brushes.DarkGreen;  
            }  
            else  
            {  
                tbWynik.Text = $"❌ Plik NIE ISTNIEJE pod ścieżką:\n{sciezka}";  
                tbWynik.Foreground = System.Windows.Media.Brushes.Crimson;  
            }  
        }  
    }  
}
```

21.8. Ścieżki do plików — relatywne i absolutne, AppDomain.CurrentDomain.BaseDirectory

W aplikacjach WPF musisz wiedzieć gdzie szukać pliku. Są dwa rodzaje ścieżek:

RODZAJ	PRZYKŁAD	UWAGI
Absolutna	C:\Aplikacja\dane.txt	Pełna ścieżka, działa zawsze
Relatywna	dane.txt	Względem katalogu roboczego

Najbezpieczniejszym sposobem jest AppDomain.CurrentDomain.BaseDirectory — zawsze wskazuje na katalog gdzie znajduje się plik .exe .

```
using System;
using System.IO;

// Sposób 1: AppDomain.CurrentDomain.BaseDirectory
string katalogExe = AppDomain.CurrentDomain.BaseDirectory;
string sciezka1 = Path.Combine(katalogExe, "dane.txt");

// Sposób 2: Environment.CurrentDirectory (może się zmieniać!)
string sciezka2 = Path.Combine(Environment.CurrentDirectory, "dane.txt");

// Sposób 3: Ścieżka absolutna (twarda — nie przenaszalna)
string sciezka3 = @"C:\MojaAplikacja\dane.txt";

// Path.Combine bezpiecznie łączy ścieżki (nie musisz myśleć o ukośnikach)
string podkatalog = Path.Combine(katalogExe, "dane", "albumy.txt");
```

21.9. Obsługa błędów przy plikach

Operacje na plikach mogą się nie udać z wielu powodów. Zawsze otaczaj je blokiem try/catch .

MainWindow.xaml:

```
<Window x:Class="ObslugaBledow.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Obsługa błędów pliku" Height="350" Width="450">
    <StackPanel Margin="20">
        <TextBlock Text="Wpisz ścieżkę do pliku:" Margin="0,0,0,5"/>
        <TextBox x:Name="txtSciezka" Text="C:\nieistniejacy.txt" Margin="0,0,0,10"/>
        <Button Content="Wczytaj plik" Click="btnWczytaj_Click" Height="35"
            Margin="0,0,0,10"/>
        <ScrollView Height="150" VerticalScrollBarVisibility="Auto">
            <TextBlock x:Name="tbZawartosc" TextWrapping="Wrap"/>
        </ScrollView>
        <TextBlock x:Name="tbBlad" Text="" Foreground="Red"
            TextWrapping="Wrap" Margin="0,10,0,0"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.IO;
using System.Text;
using System.Windows;

namespace ObslugaBledow
{
    public partial class MainWindow : Window
```

```

{
    public MainWindow()
    {
        InitializeComponent();
    }

    private void btnWczytaj_Click(object sender, RoutedEventArgs e)
    {
        tbBlad.Text = "";
        tbZawartosc.Text = "";

        try
        {
            string zawartosc = File.ReadAllText(txtSciezka.Text.Trim(), Encoding.UTF8);
            tbZawartosc.Text = zawartosc;
        }
        catch (FileNotFoundException ex)
        {
            // Plik nie istnieje
            tbBlad.Text = $"Plik nie znaleziony:\n{ex.FileName}";
        }
        catch (UnauthorizedAccessException)
        {
            // Brak uprawnień do odczytu
            tbBlad.Text = "Brak uprawnień do odczytu pliku.";
        }
        catch (IOException ex)
        {
            // Inny błąd I/O (np. plik używany przez inny program)
            tbBlad.Text = $"Błąd odczytu pliku:\n{ex.Message}";
        }
        catch (Exception ex)
        {
            // Każdy inny nieoczekiwany błąd
            tbBlad.Text = $"Nieoczekiwany błąd:\n{ex.Message}";
        }
    }
}

```

Hierarchia wyjątków plików:

- `FileNotFoundException` — plik nie istnieje (najbardziej konkretny)
- `UnauthorizedAccessException` — brak uprawnień
- `IOException` — ogólny błąd I/O, obejmuje wiele sytuacji
- `Exception` — złap wszystko co nie pasuje do powyższych

22. Algorytmy w aplikacjach WPF

Dobra aplikacja WPF oddziela logikę (algorytmy) od interfejsu użytkownika. W tej sekcji zobaczysz kompletne implementacje popularnych algorytmów zintegrowanych z WPF.

22.1. Oddzielenie algorytmu od interfejsu — metoda `ZaszyfrujTekst(string tekst, int klucz)` osobno

Zamiast umieszczać logikę bezpośrednio w event handlerach, tworzysz osobne metody. Dzięki temu możesz je łatwo testować i ponownie używać.

Zła praktyka (logika w handlerze):

```

private void btnSzyfruj_Click(object sender, RoutedEventArgs e)
{
    string tekst = txtWejscie.Text;

```

```

int klucz = 3;
string wynik = "";
foreach (char c in tekst)
{
    if (char.IsLetter(c))
        wynik += (char)(c + klucz);
    else
        wynik += c;
}
txtWyscie.Text = wynik;
}

```

Dobra praktyka (algorytm w osobnej metodzie):

```

// Metoda algorytmu – nie wie nic o interfejsie
private string ZaszyfrujTekst(string tekst, int klucz)
{
    string wynik = "";
    foreach (char c in tekst)
    {
        if (char.IsLetter(c))
            wynik += (char)(c + klucz);
        else
            wynik += c;
    }
    return wynik;
}

// Handler tylko zbiera dane z UI i wyświetla wynik
private void btnSzyfruj_Click(object sender, RoutedEventArgs e)
{
    int klucz = int.TryParse(txtKlucz.Text, out int k) ? k : 0;
    txtWyscie.Text = ZaszyfrujTekst(txtWyscie.Text, klucz);
}

```

22.2. Generator haseł

Poniższy przykład to kompletny generator haseł. Użytkownik wybiera długość hasła i zestawy znaków za pomocą CheckBox. Program losuje hasło.

MainWindow.xaml:

```

<Window x:Class="GeneratorHasel.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Generator Haseł" Height="480" Width="480">
    <StackPanel Margin="20">
        <TextBlock Text="🔑 Generator Haseł" FontSize="20"
                  FontWeight="Bold" Margin="0,0,0,15"/>

        <!-- Długość hasła -->
        <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
            <TextBlock Text="Długość hasła:" VerticalAlignment="Center"
                      Width="120"/>
            <Slider x:Name="sldDlugosc"
                    Minimum="4" Maximum="32" Value="12"
                    Width="180" VerticalAlignment="Center"
                    ValueChanged="sldDlugosc_ValueChanged"
                    IsSnapToTickEnabled="True" TickFrequency="1"/>
            <TextBlock x:Name="tbDlugosc" Text="12"
                      Width="30" VerticalAlignment="Center"

```



```

                Margin="8,0,0,0" FontWeight="Bold"/>
</StackPanel>

<!-- Zestawy znaków -->
<TextBlock Text="Zestawy znaków:" FontWeight="Bold" Margin="0,0,0,8"/>
<CheckBox x:Name="chkMaleLiterary"
    Content="Małe litery (a-z)"
    IsChecked="True" Margin="0,0,0,5"/>
<CheckBox x:Name="chkDuzeliterary"
    Content="Duże litery (A-Z)"
    IsChecked="True" Margin="0,0,0,5"/>
<CheckBox x:Name="chkCyfry"
    Content="Cyfry (0-9)"
    IsChecked="True" Margin="0,0,0,5"/>
<CheckBox x:Name="chkSpecjalne"
    Content="Znaki specjalne (!@#$%^&*)"
    IsChecked="False" Margin="0,0,0,15"/>

<!-- Przycisk generowania -->
<Button Content="🎲 Generuj hasło"
    Click="btnGeneruj_Click"
    Height="40" FontSize="14"
    Margin="0,0,0,15"/>

<!-- Wynik -->
<Border Background="#F0F0F0" CornerRadius="6" Padding="12"
    Margin="0,0,0,10">
    <TextBlock x:Name="tbHaslo"
        Text="(tutaj pojawi się hasło)"
        FontFamily="Consolas"
        FontSize="18"
        FontWeight="Bold"
        TextWrapping="Wrap"
        HorizontalAlignment="Center"/>

</Border>

<TextBlock x:Name="tbSila" Text="" HorizontalAlignment="Center"
    FontSize="13"/>
<TextBlock x:Name="tbBlad" Text="" Foreground="Red"
    HorizontalAlignment="Center" Margin="0,5,0,0"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Text;
using System.Windows;

namespace GeneratorHasel
{
    public partial class MainWindow : Window
    {
        private readonly Random _rng = new Random();

        // Stałe zestawy znaków
        private const string MALE_LITERY = "abcdefghijklmnopqrstuvwxyz";
        private const string DUZE_LITERY = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        private const string CYFRY = "0123456789";
        private const string SPECJALNE = "!@#$%^&*()-_+=[]{}|;:,.<?";
    }
}

```

```

public MainWindow()
{
    InitializeComponent();
}

private void sldDlugosc_ValueChanged(object sender,
    System.Windows.RoutedPropertyChangedEventArgs<double> e)
{
    if (tbDlugosc != null)
        tbDlugosc.Text = ((int)sldDlugosc.Value).ToString();
}

// =====
// GŁÓWNA METODA ALGORYTMU – oddzielona od UI
// =====
private string GenerujHaslo(int dlugosc, string zestawZnakow)
{
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < dlugosc; i++)
    {
        int indeks = _rng.Next(zestawZnakow.Length);
        sb.Append(zestawZnakow[indeks]);
    }
    return sb.ToString();
}

private string ZbudujZestawZnakow()
{
    StringBuilder zestaw = new StringBuilder();

    if (chkMaleLitary.IsChecked == true) zestaw.Append(MALE_LITERY);
    if (chkDuzeLitary.IsChecked == true) zestaw.Append(DUZE_LITERY);
    if (chkCyfry.IsChecked == true) zestaw.Append(CYFRY);
    if (chkSpecjalne.IsChecked == true) zestaw.Append(SPECJALNE);

    return zestaw.ToString();
}

private string OcenSileHasla(int dlugosc, string zestawZnakow)
{
    int punkty = 0;
    if (dlugosc ≥ 8) punkty++;
    if (dlugosc ≥ 12) punkty++;
    if (dlugosc ≥ 20) punkty++;
    if (zestawZnakow.Contains("A")) punkty++; // duże litery
    if (zestawZnakow.Contains("0")) punkty++; // cyfry
    if (zestawZnakow.Contains("!")) punkty++; // specjalne

    if (punkty ≤ 2) return "🔴 Słabe hasło";
    if (punkty ≤ 4) return "🟡 Średnie hasło";
    return "🟢 Silne hasło";
}

private void btnGeneruj_Click(object sender, RoutedEventArgs e)
{
    tbBlad.Text = "";

    string zestaw = ZbudujZestawZnakow();

    // Walidacja – sprawdź czy wybrany jest przynajmniej jeden zestaw
    if (string.IsNullOrEmpty(zestaw))
    {

```

```

        tbBlad.Text = "Wybierz przynajmniej jeden zestaw znaków!";
        tbHaslo.Text = "(brak hasła)";
        tbSila.Text = "";
        return;
    }

    int dlugosc = (int)sldDlugosc.Value;

    // Wywołanie algorytmu
    string haslo = GenerujHaslo(dlugosc, zestaw);

    // Aktualizacja widoku
    tbHaslo.Text = haslo;
    tbSila.Text = OcenSileHasla(dlugosc, zestaw);
}
}
}

```

Kluczowe elementy:

- `ZbudujZestawZnakow()` — buduje string ze znaków z zaznaczonych CheckBoxów; sprawdza `IsChecked == true`
- `string.IsNullOrEmpty(zestaw)` — walidacja: jeśli żaden checkbox nie zaznaczony, zestaw będzie pusty
- `GenerujHaslo(dlugosc, zestawZnakow)` — algorytm losowania oddzielony od UI
- `_rng.Next(zestawZnakow.Length)` — losuje indeks w zakresie 0 do długości zestawu - 1
- `sb.Append(zestawZnakow[indeks])` — pobiera znak z zestawu po indeksie

22.3. Szyfr Cezara

Szyfr Cezara przesuwa każdą literę w alfabecie o określoną liczbę pozycji. Działa na literach (a–z, A–Z), pomijając wszystkie inne znaki.

MainWindow.xaml:

```

<Window x:Class="SzyfrCezara.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Szyfr Cezara" Height="440" Width="500">
    <StackPanel Margin="20">
        <TextBlock Text="🔒 Szyfr Cezara" FontSize="20"
                  FontWeight="Bold" Margin="0,0,0,15"/>

        <!-- Tekst wejściowy -->
        <TextBlock Text="Tekst do zaszyfrowania:" Margin="0,0,0,5"/>
        <TextBox x:Name="txtWejscie"
                Text="Ala ma kota"
                Height="70"
                TextWrapping="Wrap"
                AcceptsReturn="True"
                Margin="0,0,0,10"/>

        <!-- Klucz -->
        <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
            <TextBlock Text="Klucz (przesunięcie):" VerticalAlignment="Center"
                      Width="170"/>
            <TextBox x:Name="txtKlucz" Text="3" Width="60"
                    HorizontalAlignment="Left"/>
            <TextBlock Text="(liczba od 1 do 25)" Foreground="Gray"
                      Margin="10,0,0,0" VerticalAlignment="Center"/>
        </StackPanel>

        <!-- Przyciski -->
        <StackPanel Orientation="Horizontal" Margin="0,0,0,15">
            <Button Content="Zaszyfruj" Width="100" Height="35"

```

```

        Margin="0,0,10,0" Click="btnSzyfruj_Click"/>
        <Button Content="Odszyfruj" Width="100" Height="35"
            Click="btnOdszyfruj_Click"/>
    </StackPanel>

    <!-- Wynik -->
    <TextBlock Text="Wynik:" FontWeight="Bold" Margin="0,0,0,5"/>
    <Border Background="AliceBlue" CornerRadius="5" Padding="10"
        Margin="0,0,0,10">
        <TextBlock x:Name="tbWynik"
            Text="(wynik pojawi się tutaj)"
            FontFamily="Consolas"
            FontSize="15"
            TextWrapping="Wrap"/>
    </Border>

    <TextBlock x:Name="tbInfo" Text="" Foreground="Gray" FontSize="12"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace SzyfrCezara
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        // =====
        // ALGORYTM SZYFRU CEZARA – oddzielony od UI
        // =====
        private string ZaszyfrujCezar(string tekst, int klucz)
        {
            // Normalizacja klucza – zawsze w zakresie 0-25
            klucz = ((klucz % 26) + 26) % 26;

            char[] znaki = tekst.ToCharArray();

            for (int i = 0; i < znaki.Length; i++)
            {
                char c = znaki[i];

                if (char.IsUpper(c))
                {
                    // Duże litery: A=65, Z=90
                    // Odejmujemy 'A', dodajemy klucz, bierzemy modulo 26, dodajemy 'A' z powrotem
                    znaki[i] = (char)((((c - 'A' + klucz) % 26) + 'A'));
                }
                else if (char.IsLower(c))
                {
                    // Małe litery: a=97, z=122
                    znaki[i] = (char)((((c - 'a' + klucz) % 26) + 'a'));
                }

                // Wszystkie inne znaki (spacje, cyfry, znaki interpunkcyjne) zostawiamy bez zmian
            }
        }
    }
}

```

```

        return new string(znaki);
    }

    private int PobraKlucz()
    {
        // TryParse – jeśli użytkownik wpisał coś złego, używamy klucza 0 (brak szyfrowania)
        if (!int.TryParse(txtKlucz.Text.Trim(), out int klucz))
        {
            klucz = 0;
            tbInfo.Text = "Niepoprawny klucz – użyto wartości 0 (brak szyfrowania).";
        }
        else
        {
            tbInfo.Text = $"Użyty klucz: {klucz} (przesunięcie o {(klucz % 26) + 26} % 26} pozycji)";
        }
        return klucz;
    }

    private void btnSzyfruj_Click(object sender, RoutedEventArgs e)
    {
        int klucz = PobraKlucz();
        string wynik = ZaszyfrujCezar(txtWejscie.Text, klucz);
        tbWynik.Text = wynik;
    }

    private void btnOdszyfruj_Click(object sender, RoutedEventArgs e)
    {
        // Odszyfrowanie = zaszyfrowanie z kluczem odwrotnym (26 - klucz)
        int klucz = PobraKlucz();
        string wynik = ZaszyfrujCezar(txtWejscie.Text, -klucz);
        tbWynik.Text = wynik;
    }
}
}

```

Kluczowe elementy:

- `c - 'A'` — konwersja litery na liczbę (A=0, B=1, ..., Z=25)
- `% 26` — zawijanie w zakresie 0–25 (po Z wracamy do A)
- `+ 'A'` — konwersja liczby z powrotem na literę
- `((klucz % 26) + 26) % 26` — obsługa ujemnych kluczy (dla odszyfrowywania)
- `char.IsUpper(c)` / `char.IsLower(c)` — rozróżnianie wielkich i małych liter

22.4. Walidator kodu pocztowego

Kod pocztowy w Polsce ma format `XX-XXX` (6 znaków: 2 cyfry, myślnik, 3 cyfry). Walidator sprawdza każdy warunek osobno i wyświetla konkretny komunikat błędu.

MainWindow.xaml:

```

<Window x:Class="WalidatorKodu.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Walidator kodu pocztowego" Height="380" Width="430">
    <StackPanel Margin="20">
        <TextBlock Text="🔍 Walidator kodu pocztowego" FontSize="17"
                  FontWeight="Bold" Margin="0,0,0,15" />

        <TextBlock Text="Format: XX-XXX (np. 00-001)" Foreground="Gray"
                  Margin="0,0,0,5" />

        <TextBox x:Name="txtKod"

```

```

        FontSize="20"
        Width="160"
        HorizontalAlignment="Left"
        MaxLength="6"
        Margin="0,0,0,10" />

<Button Content="Sprawdź kod"
        Click="btnSprawdz_Click"
        Width="130" Height="35"
        HorizontalAlignment="Left"
        Margin="0,0,0,15" />

<!-- Wynik walidacji -->
<Border x:Name="borderWynik"
        Visibility="Collapsed"
        CornerRadius="6"
        Padding="12"
        Margin="0,0,0,0">
    <StackPanel>
        <TextBlock x:Name="tbWynikGłówny"
            FontSize="16" FontWeight="Bold"
            Margin="0,0,0,8" />

        <TextBlock x:Name="tbWynikSzczegół"
            TextWrapping="Wrap"
            FontSize="13" />

    </StackPanel>
</Border>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;
using System.Windows.Media;

namespace WalidatorKodu
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        // =====
        // ALGORYTM WALIDACJI – oddzielony od UI
        // =====
        private string WalidujKodPocztowy(string kod)
        {
            // Warunek 1: Długość musi być dokładnie 6 znaków
            if (kod.Length != 6)
                return $"Błąd: Kod musi mieć dokładnie 6 znaków (ma {kod.Length}).";

            // Warunek 2: Pozycja 2 (indeks 2) musi być myślnikiem
            if (kod[2] != '-')
                return $"Błąd: Na pozycji 3 musi być myślnik '-' (jest '{kod[2]}').";

            // Warunek 3: Pozycje 0 i 1 muszą być cyframi
            if (!char.IsDigit(kod[0]))
                return $"Błąd: Pierwsza pozycja musi być cyfrą (jest '{kod[0]}').";
        }
    }
}

```

```

        if (!char.IsDigit(kod[1]))
            return $"Błąd: Druga pozycja musi być cyfrą (jest '{kod[1]}').";

        // Warunek 4: Pozycje 3, 4, 5 muszą być cyframi
        if (!char.IsDigit(kod[3]))
            return $"Błąd: Czwarta pozycja musi być cyfrą (jest '{kod[3]}').";

        if (!char.IsDigit(kod[4]))
            return $"Błąd: Piąta pozycja musi być cyfrą (jest '{kod[4]}').";

        if (!char.IsDigit(kod[5]))
            return $"Błąd: Szósta pozycja musi być cyfrą (jest '{kod[5]}').";

        // Wszystkie warunki spełnione
        return null; // null oznacza brak błędu
    }

    private void btnSprawdz_Click(object sender, RoutedEventArgs e)
    {
        string kod = txtKod.Text.Trim();

        // Walidacja pustego pola
        if (string.IsNullOrEmpty(kod))
        {
            PokażWynik(false, "Brak danych", "Wpisz kod pocztowy przed sprawdzeniem.");
            return;
        }

        string bladWalidacji = WalidujKodPocztowy(kod);

        if (bladWalidacji == null)
        {
            // Kod poprawny
            PokażWynik(true, $"✅ Kod '{kod}' jest POPRAWNY",
                "Format XX-XXX został zachowany. Kod może być użyty.");
        }
        else
        {
            // Kod niepoprawny – pokaż konkretny komunikat
            PokażWynik(false, $"❌ Kod '{kod}' jest NIEPOPRAWNY", bladWalidacji);
        }
    }

    private void PokażWynik(bool poprawny, string naglowek, string szczegoly)
    {
        borderWynik.Visibility = Visibility.Visible;
        tbWynikGlowny.Text = naglowek;
        tbWynikSzczegoly.Text = szczegoly;

        if (poprawny)
        {
            borderWynik.Background = new SolidColorBrush(Color.FromRgb(200, 240, 200));
            tbWynikGlowny.Foreground = Brushes.DarkGreen;
        }
        else
        {
            borderWynik.Background = new SolidColorBrush(Color.FromRgb(255, 220, 220));
            tbWynikGlowny.Foreground = Brushes.DarkRed;
        }
    }
}

```

```
}  
  
}
```

Kluczowe elementy:

- `kod.Length != 6` — sprawdzenie długości (pierwsze sprawdzenie)
- `kod[2] != '-'` — sprawdzenie myślnika na konkretnej pozycji (indeks 2 = trzeci znak)
- `char.IsDigit(kod[0])` — sprawdzenie czy znak jest cyfrą
- `return null` — konwencja: `null` = brak błędu, `string` = komunikat błędu
- Osobne komunikaty dla każdego warunku — użytkownik wie dokładnie co jest nie tak

22.5. Przeliczanie ceny lub wyniku

MainWindow.xaml:

```
<Window x:Class="PrzeliczanieCeny.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Kalkulator ceny" Height="380" Width="420">  
    <StackPanel Margin="20">  
        <TextBlock Text="🛒 Kalkulator ceny" FontSize="17"  
            FontWeight="Bold" Margin="0,0,0,15"/>  
  
        <StackPanel Orientation="Horizontal" Margin="0,0,0,10">  
            <TextBlock Text="Cena netto (zł):" Width="140" VerticalAlignment="Center"/>  
            <TextBox x:Name="txtCenaNetto" Text="100" Width="100"/>  
        </StackPanel>  
  
        <TextBlock Text="Stawka VAT:" FontWeight="Bold" Margin="0,5,0,5"/>  
        <RadioButton x:Name="rbVat5" Content="5% (np. żywność)" Margin="0,2"/>  
        <RadioButton x:Name="rbVat8" Content="8% (np. usługi budowlane)" Margin="0,2"/>  
        <RadioButton x:Name="rbVat23" Content="23% (standardowa)" IsChecked="True" Margin="0,2"/>  
  
        <Button Content="Oblicz cenę brutto"  
            Click="btnOblicz_Click" Height="35"  
            Width="160" HorizontalAlignment="Left"  
            Margin="0,15,0,10"/>  
  
        <Border Background="AliceBlue" CornerRadius="5" Padding="10">  
            <StackPanel>  
                <TextBlock x:Name="tbCenaNet" Text="" Margin="0,2"/>  
                <TextBlock x:Name="tbVat" Text="" Margin="0,2"/>  
                <TextBlock x:Name="tbCenaBrut" Text="" FontSize="16"  
                    FontWeight="Bold" Margin="0,5,0,0"  
                    Foreground="DarkBlue"/>  
            </StackPanel>  
        </Border>  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;  
  
namespace PrzeliczanieCeny  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  

```



```

        InitializeComponent();
    }

    private double PobierzStawkeVat()
    {
        if (rbVat5.IsChecked == true) return 0.05;
        if (rbVat8.IsChecked == true) return 0.08;
        return 0.23; // domyślnie 23%
    }

    private void btnOblicz_Click(object sender, RoutedEventArgs e)
    {
        if (!double.TryParse(txtCenaNetto.Text.Replace(",", "."),
            System.Globalization.NumberStyles.Any,
            System.Globalization.CultureInfo.InvariantCulture,
            out double cenaNetto))
        {
            MessageBox.Show("Podaj poprawną cenę netto.", "Błąd");
            return;
        }

        double stawkaVat = PobierzStawkeVat();
        double kwotaVat = cenaNetto * stawkaVat;
        double cenaBrutto = cenaNetto + kwotaVat;

        tbCenaNet.Text = $"Cena netto: {cenaNetto:F2} zł";
        tbVat.Text = $"VAT ({stawkaVat * 100:F0}%): {kwotaVat:F2} zł";
        tbCenaBrut.Text = $"Cena brutto: {cenaBrutto:F2} zł";
    }
}

```

22.6. Aktualizacja licznika — `int_liczbaPobran++`, wyświetlanie w etykiecie

Liczniki to jeden z najczęstszych wzorców w aplikacjach. Przechowujesz wartość w polu klasy i aktualizujesz etykietę po każdej zmianie.

MainWindow.xaml:

```

<Window x:Class="Liczniki.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Liczniki" Height="300" Width="380">
    <StackPanel Margin="20" VerticalAlignment="Center">
        <TextBlock Text="Panel liczników" FontSize="16"
            FontWeight="Bold" HorizontalAlignment="Center"
            Margin="0,0,0,20" />

        <StackPanel Orientation="Horizontal" HorizontalAlignment="Center"
            Margin="0,0,0,10">
            <Button Content="↓ Pobierz" Width="100" Height="35"
                Margin="5" Click="btnPobierz_Click" />
            <TextBlock x:Name="tbPobran"
                Text="Pobrań: 0"
                VerticalAlignment="Center"
                FontSize="14" Width="100" />
        </StackPanel>

        <StackPanel Orientation="Horizontal" HorizontalAlignment="Center"
            Margin="0,0,0,10">
            <Button Content="★ PoLub" Width="100" Height="35"
                Margin="5" Click="btnPoLub_Click" />
        </StackPanel>
    </StackPanel>

```

```

        <TextBlock x:Name="tbPunktow"
            Text="Punktów: 0"
            VerticalAlignment="Center"
            FontSize="14" Width="100"/>
    </StackPanel>

    <Button Content="Resetuj" Width="100" Height="30"
        HorizontalAlignment="Center" Margin="0,15,0,0"
        Click="btnResetuj_Click"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace Liczniki
{
    public partial class MainWindow : Window
    {
        // Pola klasy przechowują stan liczników
        private int _liczbaPobran = 0;
        private int _punkty = 0;

        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnPobierz_Click(object sender, RoutedEventArgs e)
        {
            _liczbaPobran++; // zwiększ licznik
            tbPobran.Text = $"Pobrań: {_liczbaPobran}"; // odśwież etykietę
        }

        private void btnPolub_Click(object sender, RoutedEventArgs e)
        {
            _punkty += 10; // dodaj 10 punktów
            tbPunktow.Text = $"Punktów: {_punkty}";
        }

        private void btnResetuj_Click(object sender, RoutedEventArgs e)
        {
            _liczbaPobran = 0;
            _punkty = 0;
            tbPobran.Text = "Pobrań: 0";
            tbPunktow.Text = "Punktów: 0";
        }
    }
}

```

22.7. StringBuilder

Gdy budujesz długi string z wielu części, nie używaj operatora `+` w pętli — jest wolny. Używaj klasy `StringBuilder`.

Dlaczego StringBuilder?

```

// ŹLE – tworzy nowy string w każdej iteracji, wolne dla dużych pętli
string wynik = "";
for (int i = 0; i < 10000; i++)

```

```
wynik += i.ToString() + ", ";

// DOBRZE – StringBuilder modyfikuje swój bufor wewnętrzny, szybkie
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++)
{
    sb.Append(i);
    sb.Append(", ");
}
string wynik = sb.ToString();
```

MainWindow.xaml:

```
<Window x:Class="StringBuilderDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="StringBuilder – demo" Height="400" Width="480">
    <StackPanel Margin="20">
        <TextBlock Text="Generator raportu z StringBuilder"
                   FontSize="14" FontWeight="Bold" Margin="0,0,0,10"/>
        <StackPanel Orientation="Horizontal" Margin="0,0,0,5">
            <TextBlock Text="Liczba pozycji:" VerticalAlignment="Center" Width="120"/>
            <TextBox x:Name="txtLiczba" Text="20" Width="60"/>
        </StackPanel>
        <Button Content="Generuj raport" Click="btnGeneruj_Click"
                Height="35" Width="140" HorizontalAlignment="Left"
                Margin="0,10,0,10"/>
        <ScrollView Height="220" VerticalScrollBarVisibility="Auto">
            <TextBlock x:Name="tbRaport" FontFamily="Consolas"
                       FontSize="12" TextWrapping="Wrap"/>
        </ScrollView>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Text;
using System.Windows;

namespace StringBuilderDemo
{
    public partial class MainWindow : Window
    {
        private readonly Random _rng = new Random();

        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnGeneruj_Click(object sender, RoutedEventArgs e)
        {
            if (!int.TryParse(txtLiczba.Text, out int n) || n ≤ 0)
            {
                MessageBox.Show("Podaj dodatnią liczbę całkowitą.", "Błąd");
                return;
            }

            // Użyj StringBuilder do budowania długiego raportu
        }
    }
}
```

```

        StringBuilder sb = new StringBuilder();
        sb.AppendLine("=== RAPORT PRODUKTÓW ===");
        sb.AppendLine($"Data: {DateTime.Now:dd.MM.yyyy HH:mm}");
        sb.AppendLine(new string('-', 40));

        string[] nazwy = { "Monitor", "Klawiatura", "Mysz", "Słuchawki", "Kamera" };

        double suma = 0;
        for (int i = 1; i ≤ n; i++)
        {
            string nazwa = nazwy[_rng.Next(nazwy.Length)];
            double cena = Math.Round(_rng.NextDouble() * 500 + 50, 2);
            int ilosc = _rng.Next(1, 10);
            double wartosc = cena * ilosc;
            suma += wartosc;

            // Każda linia to Append + AppendLine
            sb.Append($"{i,3}. ");
            sb.Append($"{nazwa,-12}");
            sb.Append($" {cena,8:F2} zł");
            sb.Append($" x{ilosc}");
            sb.AppendLine($" = {wartosc:F2} zł");
        }

        sb.AppendLine(new string('-', 40));
        sb.AppendLine($"SUMA: {suma:F2} zł");

        // Konwersja do string jednym wywołaniem ToString()
        tbRaport.Text = sb.ToString();
    }
}

```

Metody StringBuilder:

- `sb.Append(tekst)` — dodaje tekst bez nowej linii
- `sb.AppendLine(tekst)` — dodaje tekst z nową linią (`\n`)
- `sb.ToString()` — konwertuje cały bufor do `string`
- `new string('-', 40)` — tworzy string 40 myślników (przydatne do separatorów)

23. Generowanie losowych danych

Losowość jest potrzebna w wielu aplikacjach: generatory haseł, losowanie pytań, symulacje, gry. W C# do generowania liczb losowych służy klasa `Random`.

23.1. Klasa Random

```

// Tworzenie generatora
Random rng = new Random();

// Losowa liczba całkowita: 0, 1, 2, ..., 9 (max jest WYKLUCZONY)
int liczba = rng.Next(0, 10);

// Losowa liczba całkowita: 1, 2, 3, ..., 100
int od1do100 = rng.Next(1, 101);

// Losowa liczba zmiennoprzecinkowa: 0.0 ≤ x < 1.0
double ułamek = rng.NextDouble();

// Losowa liczba zmiennoprzecinkowa w zakresie 0.0-100.0

```

```
double zakres = rng.NextDouble() * 100.0;

// Losowy element tablicy
string[] owoce = { "jabłko", "gruszką", "śliwka" };
string losowy = owoce[rng.Next(owoce.Length)];
```

Zasady użycia Random:

- **Twórz jeden obiekt** `Random` jako pole klasy (nie w pętli — wielokrotne `new Random()` w pętli da te same liczby)
- `Next(min, max)` — max jest **wykluczony**, więc `Next(1, 7)` losuje 1-6 (jak kostka)
- `NextDouble()` — zawsze zwraca wartość z zakresu `[0.0, 1.0)`

MainWindow.xaml:

```
<Window x:Class="RandomDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Random - demo" Height="350" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Generator liczb losowych" FontSize="15"
                   FontWeight="Bold" Margin="0,0,0,15"/>

        <StackPanel Orientation="Horizontal" Margin="0,0,0,8">
            <TextBlock Text="Min:" Width="40" VerticalAlignment="Center"/>
            <TextBox x:Name="txtMin" Text="1" Width="60" Margin="5,0"/>
            <TextBlock Text="Max:" Width="40" VerticalAlignment="Center"/>
            <TextBox x:Name="txtMax" Text="100" Width="60" Margin="5,0"/>
        </StackPanel>

        <Button Content="Losuj liczbę całkowitą"
                Click="btnLosujInt_Click" Height="35" Margin="0,5"/>
        <Button Content="Losuj ułamek (0.0-1.0)"
                Click="btnLosujDouble_Click" Height="35" Margin="0,5"/>
        <Button Content="Rzut kostką (1-6)"
                Click="btnKostka_Click" Height="35" Margin="0,5"/>

        <TextBlock x:Name="tbWynik" Text="" FontSize="24"
                   FontWeight="Bold" HorizontalAlignment="Center"
                   Margin="0,15,0,0" Foreground="DarkBlue"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace RandomDemo
{
    public partial class MainWindow : Window
    {
        // Jeden obiekt Random jako pole klasy
        private readonly Random _rng = new Random();

        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnLosujInt_Click(object sender, RoutedEventArgs e)
        {

```

```

        int.TryParse(txtMin.Text, out int min);
        int.TryParse(txtMax.Text, out int max);

        if (min ≥ max)
        {
            tbWynik.Text = "Min musi być mniejsze niż Max!";
            return;
        }

        // Next(min, max) – max jest wykluczony, więc dodajemy 1
        int wynik = _rng.Next(min, max + 1);
        tbWynik.Text = wynik.ToString();
    }

    private void btnLosujDouble_Click(object sender, RoutedEventArgs e)
    {
        double wynik = _rng.NextDouble();
        tbWynik.Text = wynik.ToString("F6");
    }

    private void btnKostka_Click(object sender, RoutedEventArgs e)
    {
        int oczka = _rng.Next(1, 7); // 1-6
        tbWynik.Text = $"🎲 {oczka}";
    }
}
}

```

23.2. StringBuilder

Poniższy przykład to uproszczona wersja generatora haseł pokazująca sam algorytm losowania.

MainWindow.xaml:

```

<Window x:Class="GeneratorHaslaProste.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Generator hasła – prosty" Height="300" Width="420">
    <StackPanel Margin="20">
        <TextBlock Text="Prosty generator hasła" FontSize="15"
            FontWeight="Bold" Margin="0,0,0,15"/>

        <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
            <TextBlock Text="Długość:" Width="80" VerticalAlignment="Center"/>
            <TextBox x:Name="txtDlugosc" Text="12" Width="60"/>
        </StackPanel>

        <Button Content="Generuj" Click="btnGeneruj_Click"
            Width="100" Height="35" HorizontalAlignment="Left"
            Margin="0,0,0,15"/>

        <Border Background="#EFEFEF" CornerRadius="5" Padding="10">
            <TextBlock x:Name="tbHaslo"
                Text="(brak hasła)"
                FontFamily="Consolas"
                FontSize="18"
                FontWeight="Bold"/>
        </Border>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```
using System;
using System.Text;
using System.Windows;

namespace GeneratorHaslaProste
{
    public partial class MainWindow : Window
    {
        private readonly Random _rng = new Random();

        // Pełny zestaw znaków
        private const string ZNAKI =
            "abcdefghijklmnopqrstuvwxyz" +
            "ABCDEFGHIJKLMNOPQRSTUVWXYZ" +
            "0123456789" +
            "!@#$$%^&*";

        public MainWindow()
        {
            InitializeComponent();
        }

        private string GenerujHaslo(int dlugosc, string zestawZnakow)
        {
            StringBuilder sb = new StringBuilder(dlugosc);

            for (int i = 0; i < dlugosc; i++)
            {
                // Losuj indeks w zestawie znaków
                int indeks = _rng.Next(zestawZnakow.Length);

                // Pobierz znak z zestawu i dodaj do buildera
                sb.Append(zestawZnakow[indeks]);
            }

            return sb.ToString();
        }

        private void btnGeneruj_Click(object sender, RoutedEventArgs e)
        {
            if (!int.TryParse(txtDlugosc.Text, out int dlugosc) || dlugosc < 1)
            {
                MessageBox.Show("Podaj poprawną długość (liczba > 0).", "Błąd");
                return;
            }

            tbHaslo.Text = GenerujHaslo(dlugosc, ZNAKI);
        }
    }
}
```

23.3. CheckBox

Szczegółowy przykład z CheckBoxami pokazano w rozdziale 22.2 (Generator haseł). Poniżej widoczny jest sam wzorec budowania zestawu:

```
private const string MALE = "abcdefghijklmnopqrstuvwxyz";
private const string DUZE = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
private const string CYFRY = "0123456789";
```

```
private const string SPEC = "!@#$$%";

private string ZbudujZestaw()
{
    StringBuilder sb = new StringBuilder();

    // Każdy blok if sprawdza jeden CheckBox
    if (chkMale.IsChecked == true) sb.Append(MALE);
    if (chkDuze.IsChecked == true) sb.Append(DUZE);
    if (chkCyfry.IsChecked == true) sb.Append(CYFRY);
    if (chkSpec.IsChecked == true) sb.Append(SPEC);

    return sb.ToString(); // pusty string = żaden checkbox nie zaznaczony
}
```

Po wywołaniu `ZbudujZestaw()` zawsze sprawdzaj czy wynik nie jest pusty:

```
string zestaw = ZbudujZestaw();
if (string.IsNullOrEmpty(zestaw))
{
    tbBlad.Text = "Zaznacz przynajmniej jeden zestaw znaków!";
    return;
}

// dopiero teraz generuj hasło...
```

23.4. List

MainWindow.xaml:

```
<Window x:Class="LosowaniePytan.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Quiz – losowanie pytań" Height="420" Width="520">
    <StackPanel Margin="20">
        <TextBlock Text="🎲 Quiz – losowe pytania" FontSize="17"
                  FontWeight="Bold" Margin="0,0,0,15" />

        <!-- Pytanie -->
        <Border Background="SteelBlue" CornerRadius="6" Padding="12"
                Margin="0,0,0,10">
            <TextBlock x:Name="tbPytanie"
                       Text="Kliknij 'Losuj pytanie' aby zacząć"
                       Foreground="White" FontSize="14"
                       TextWrapping="Wrap" />
        </Border>

        <!-- Odpowiedź użytkownika -->
        <TextBox x:Name="txtOdpowiedz"
                 Height="35" Margin="0,0,0,10" />

        <StackPanel Orientation="Horizontal" Margin="0,0,0,15">
            <Button Content="🎲 Losuj pytanie" Width="130" Height="35"
                    Margin="0,0,10,0" Click="btnLosuj_Click" />
            <Button Content="✅ Sprawdź" Width="100" Height="35"
                    Click="btnSprawdz_Click" />
        </StackPanel>

        <TextBlock x:Name="tbWynik" Text="" FontSize="14"
                  TextWrapping="Wrap" />
    </StackPanel>
```



```

        <TextBlock x:Name="tbStat" Text="Pytań: 0 | Poprawnych: 0"
                Foreground="Gray" Margin="0,10,0,0"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.Generic;
using System.Windows;

namespace LosowaniePytan
{
    public class Pytanie
    {
        public string Tresc { get; set; }
        public string PoprawnaOdpowiedz { get; set; }
    }

    public partial class MainWindow : Window
    {
        private readonly Random _rng = new Random();
        private List<Pytanie> _pytania;
        private Pytanie _aktualne;
        private int _liczbaPytan = 0;
        private int _liczbaPoprawnych = 0;

        public MainWindow()
        {
            InitializeComponent();
            ZaladujPytania();
        }

        private void ZaladujPytania()
        {
            _pytania = new List<Pytanie>
            {
                new Pytanie { Tresc = "Jaka jest stolica Polski?",
                              PoprawnaOdpowiedz = "Warszawa" },
                new Pytanie { Tresc = "Ile wynosi 7 x 8?",
                              PoprawnaOdpowiedz = "56" },
                new Pytanie { Tresc = "Jaki jest symbol chemiczny zlota?",
                              PoprawnaOdpowiedz = "Au" },
                new Pytanie { Tresc = "W którym roku zakończył się II wojna światowa?",
                              PoprawnaOdpowiedz = "1945" },
                new Pytanie { Tresc = "Ile liter ma alfabet angielski?",
                              PoprawnaOdpowiedz = "26" }
            };
        }

        private void btnLosuj_Click(object sender, RoutedEventArgs e)
        {
            // Losowanie elementu z listy
            int indeks = _rng.Next(_pytania.Count);
            _aktualne = _pytania[indeks];

            tbPytanie.Text = _aktualne.Tresc;
            txtOdpowiedz.Text = "";
            tbWynik.Text = "";
            txtOdpowiedz.Focus();
        }
    }
}

```

```

    }

    private void btnSprawdz_Click(object sender, RoutedEventArgs e)
    {
        if (_aktualne == null)
        {
            tbWynik.Text = "Najpierw wylosuj pytanie!";
            return;
        }

        string odpowiedz = txtOdpowiedz.Text.Trim();
        _liczbaPytan++;

        bool poprawna = string.Equals(odpowiedz, _aktualne.PoprawnaOdpowiedz,
            StringComparison.OrdinalIgnoreCase);

        if (poprawna)
        {
            _liczbaPoprawnych++;
            tbWynik.Text = "✅ Poprawna odpowiedź!";
            tbWynik.Foreground = System.Windows.Media.Brushes.DarkGreen;
        }
        else
        {
            tbWynik.Text = $"❌ Błąd! Poprawna odpowiedź: {_aktualne.PoprawnaOdpowiedz}";
            tbWynik.Foreground = System.Windows.Media.Brushes.Crimson;
        }

        tbStat.Text = $"Pytań: {_liczbaPytan} | Poprawnych: {_liczbaPoprawnych}";
    }
}

```

Kluczowe elementy:

- `_rng.Next(_pytania.Count)` — losuje indeks od 0 do `Count - 1`
- `_pytania[indeks]` — pobiera element z listy po wylosowanym indeksie
- `StringComparison.OrdinalIgnoreCase` — porównanie ignorujące wielkość liter

24. Programowanie obiektowe w WPF

Programowanie obiektowe (OOP) to fundament C#. W WPF używasz go do oddzielenia danych (modele) od logiki (klasy usługowe) i od interfejsu (XAML + code-behind). Ta sekcja pokazuje jak stosować OOP w praktyce.

24.1. Klasy i obiekty — tworzenie klasy Album z właściwościami

Klasa to wzorzec (szablon) dla obiektów. Obiekt to konkretny egzemplarz klasy. Klasa `Album` opisuje jeden album muzyczny.

MainWindow.xaml:

```

<Window x:Class="OOPDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Klasy i obiekty" Height="450" Width="520">
    <StackPanel Margin="20">
        <TextBlock Text="Tworzenie obiektów klasy Album" FontSize="15"
            FontWeight="Bold" Margin="0,0,0,15" />

        <Grid Margin="0,0,0,10">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="120" />

```

```

        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition Height="35" />
        <RowDefinition Height="35" />
        <RowDefinition Height="35" />
        <RowDefinition Height="35" />
        <RowDefinition Height="35" />
    </Grid.RowDefinitions>

    <TextBlock Grid.Row="0" Grid.Column="0" Text="Tytuł:"
        VerticalAlignment="Center" />
    <TextBox x:Name="txtTytuł" Grid.Row="0" Grid.Column="1"
        Margin="0,2" />

    <TextBlock Grid.Row="1" Grid.Column="0" Text="Artysta:"
        VerticalAlignment="Center" />
    <TextBox x:Name="txtArtysta" Grid.Row="1" Grid.Column="1"
        Margin="0,2" />

    <TextBlock Grid.Row="2" Grid.Column="0" Text="Rok:"
        VerticalAlignment="Center" />
    <TextBox x:Name="txtRok" Grid.Row="2" Grid.Column="1"
        Margin="0,2" />

    <TextBlock Grid.Row="3" Grid.Column="0" Text="Liczba utworów:"
        VerticalAlignment="Center" />
    <TextBox x:Name="txtLiczbaUtworow" Grid.Row="3" Grid.Column="1"
        Margin="0,2" />

    <TextBlock Grid.Row="4" Grid.Column="0" Text="Pobrań:"
        VerticalAlignment="Center" />
    <TextBox x:Name="txtPobran" Grid.Row="4" Grid.Column="1"
        Margin="0,2" />
</Grid>

<Button Content="Stwórz obiekt Album"
    Click="btnStworzAlbum_Click" Height="35"
    Width="180" HorizontalAlignment="Left"
    Margin="0,0,0,10" />

<Border Background="AliceBlue" CornerRadius="5" Padding="10">
    <TextBlock x:Name="tbWynik" TextWrapping="Wrap"
        FontFamily="Consolas" FontSize="12" />
</Border>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace OOPDemo
{
    // =====
    // KLASA MODELU – oddzielona od UI
    // =====
    public class Album
    {
        // Właściwości (Properties) – publiczne dane albumu
        public string Tytuł { get; set; }
    }
}

```

```

public string Artysta { get; set; }
public int Rok { get; set; }
public int LiczbaUtworow { get; set; }
public int LiczbaPobran { get; set; }

// Konstruktor domyślny (bez parametrów)
public Album()
{
    Tytuł = "Nieznany";
    Artysta = "Nieznany";
}

// Konstruktor z parametrami – wygodne tworzenie obiektu
public Album(string tytuł, string artysta, int rok,
             int liczbaUtworow, int liczbaPobran)
{
    Tytuł = tytuł;
    Artysta = artysta;
    Rok = rok;
    LiczbaUtworow = liczbaUtworow;
    LiczbaPobran = liczbaPobran;
}

// Metoda ToString – wywoływana automatycznie gdy obiekt trafia do ListBox itp.
public override string ToString()
{
    return $"{Tytuł} – {Artysta} ({Rok})";
}

// Metoda obliczająca (logika wewnątrz klasy)
public string OcenaPopularnosc()
{
    if (LiczbaPobran ≥ 900) return "Mega hit";
    if (LiczbaPobran ≥ 500) return "Popularny";
    return "Mało znany";
}
}

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    private void btnStworzAlbum_Click(object sender, RoutedEventArgs e)
    {
        int.TryParse(txtRok.Text, out int rok);
        int.TryParse(txtLiczbaUtworow.Text, out int liczbaU);
        int.TryParse(txtPobran.Text, out int pobran);

        // Tworzenie obiektu przez konstruktor z parametrami
        Album album = new Album(
            tytuł:      txtTytuł.Text,
            artysta:    txtArtysta.Text,
            rok:        rok,
            liczbaUtworow: liczbaU,
            liczbaPobran: pobran
        );

        // Wyświetlenie danych obiektu
        tbWynik.Text =

```

```

        $"Tytuł:      {album.Tytuł}\n" +
        $"Artysta:    {album.Artysta}\n" +
        $"Rok:         {album.Rok}\n" +
        $"Utworów:     {album.LiczbaUtworow}\n" +
        $"Pobrań:      {album.LiczbaPobran}\n" +
        $"ToString():   {album}\n" +
        $"Popularność:  {album.OcenPopularnosc()}";
    }
}
}

```

Kluczowe elementy:

- **Klasa `Album`** — definicja poza klasą `MainWindow`, w tym samym pliku lub osobnym
- **Właściwości `{ get; set; }`** — publiczne pola dostępne zarówno do odczytu jak i zapisu
- **Konstruktor z parametrami** — wygodne tworzenie obiektu w jednej linii
- **`override ToString()`** — metoda wywoływana automatycznie gdy obiekt jest konwertowany na string

24.2. Właściwości z `get` i `set`

```

public class Album
{
    // Auto-implemented property – najprostsza forma, pełny dostęp
    public string Tytuł { get; set; }

    // Private set – można ustawić tylko wewnątrz klasy
    public int LiczbaPobran { get; private set; }

    // Właściwość tylko do odczytu – ustawiana w konstruktorze
    public string KodAlbumu { get; }

    // Właściwość z logiką w get
    public string Opis => $"{Tytuł} ({Rok})";

    public int Rok { get; set; }

    public Album(string tytuł, int rok, string kod)
    {
        Tytuł = tytuł;
        Rok = rok;
        KodAlbumu = kod; // można ustawić tylko w konstruktorze
        LiczbaPobran = 0;
    }

    // Metoda modyfikująca private set
    public void Pobierz()
    {
        LiczbaPobran++; // OK – jesteśmy wewnątrz klasy
    }
}

// Użycie:
// Album a = new Album("Nevermind", 1991, "ALB-001");
// a.Tytuł = "Inny";           // OK – public set
// a.LiczbaPobran = 5;         // BŁĄD – private set
// a.Pobierz();                // OK – metoda wewnątrz klasy zmienia LiczbaPobran

```

24.3. Hermetyzacja: private i public

Hermetyzacja (encapsulation) to ukrywanie szczegółów implementacji. Używasz jej aby:

- Chronić dane przed niepoprawną modyfikacją z zewnątrz
- Mieć jedno miejsce gdzie kontrolujesz logikę

MODYFIKATOR	DOSTĘPNOŚĆ
public	Dostępny wszędzie
private	Tylko wewnątrz klasy
protected	W klasie i klasach dziedziczących
internal	W ramach projektu (assembly)

```
public class GeneratorHasel
{
    // private – szczegół implementacji, nikt z zewnątrz nie musi tego wiedzieć
    private readonly Random _rng = new Random();
    private string _ostatnieHaslo = "";

    // public – to jest API klasy – to co pokazujesz na zewnątrz
    public int MinimalnaDlugosc { get; set; } = 8;

    // public metoda – główny punkt wejścia
    public string Generuj(int dlugosc, string zestawZnakow)
    {
        if (dlugosc < MinimalnaDlugosc)
            dlugosc = MinimalnaDlugosc;

        _ostatnieHaslo = WylosujZnaki(dlugosc, zestawZnakow); // wywołanie private
        return _ostatnieHaslo;
    }

    public string OstatnieHaslo => _ostatnieHaslo; // tylko odczyt z zewnątrz

    // private – szczegół, nie musi być widoczna na zewnątrz
    private string WylosujZnaki(int dlugosc, string zestaw)
    {
        System.Text.StringBuilder sb = new System.Text.StringBuilder();
        for (int i = 0; i < dlugosc; i++)
            sb.Append(zestaw[_rng.Next(zestaw.Length)]);
        return sb.ToString();
    }
}
```

24.4. Generator hasel

Poniższy przykład pokazuje kompletną aplikację gdzie logika jest całkowicie oddzielona od interfejsu.

MainWindow.xaml:

```
<Window x:Class="OddzieloneKlasy.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Oddzielone klasy logiki" Height="460" Width="520">
    <StackPanel Margin="20">
        <TextBlock Text="Klasy logiki oddzielone od UI" FontSize="15"
            FontWeight="Bold" Margin="0,0,0,15" />

        <!-- Sekcja generatora hasel -->
        <Border Background="#F5F5F5" CornerRadius="6" Padding="12">
```

```

        Margin="0,0,0,10">
<StackPanel>
    <TextBlock Text="Generator hasel" FontWeight="Bold"
        Margin="0,0,0,8" />
    <StackPanel Orientation="Horizontal">
        <TextBox x:Name="txtDlugoscHasla" Text="12" Width="50"
            Margin="0,0,10,0" />
        <Button Content="Generuj" Width="80" Height="28"
            Click="btnGenerujHaslo_Click" />
        <TextBlock x:Name="tbHaslo" Margin="10,0,0,0"
            VerticalAlignment="Center"
            FontFamily="Consolas" FontWeight="Bold"
            FontSize="14" />
    </StackPanel>
</StackPanel>
</Border>

<!-- Sekcja szyfru Cezara -->
<Border Background="#F5F5F5" CornerRadius="6" Padding="12">
    <StackPanel>
        <TextBlock Text="Szyfr Cezara" FontWeight="Bold"
            Margin="0,0,0,8" />
        <TextBox x:Name="txtTekstDoSzyfrowania"
            Text="Witaj świecie"
            Margin="0,0,0,5" />
        <StackPanel Orientation="Horizontal" Margin="0,0,0,5">
            <TextBlock Text="Klucz: " VerticalAlignment="Center" />
            <TextBox x:Name="txtKluczSzyfrowania" Text="3"
                Width="50" Margin="5,0,10,0" />
            <Button Content="Zaszyfruj" Width="80" Height="28"
                Click="btnZaszyfruj_Click" />
        </StackPanel>
        <TextBlock x:Name="tbZaszyfrowany"
            Text="" FontFamily="Consolas"
            FontSize="14" FontWeight="Bold"
            Foreground="DarkBlue" />
    </StackPanel>
</Border>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Text;
using System.Windows;

namespace OddzieloneKlasy
{
    // =====
    // KLASA LOGIKI 1 – Generator hasel
    // Nie wie nic o WPF ani o UI
    // =====
    public class GeneratorHasel
    {
        private readonly Random _rng = new Random();

        private const string WSZYSTKIE =
            "abcdefghijklmnopqrstuvwxyz" +
            "ABCDEFGHIJKLMNOPQRSTUVWXYZ" +
            "0123456789!@#$%";
    }
}

```

```

        public string Generuj(int dlugosc)
        {
            if (dlugosc < 1) dlugosc = 8;
            StringBuilder sb = new StringBuilder(dlugosc);
            for (int i = 0; i < dlugosc; i++)
                sb.Append(WSZYSTKIE[_rng.Next(WSZYSTKIE.Length)]);
            return sb.ToString();
        }
    }

// =====
// KLASA LOGIKI 2 – Szyfr Cezara
// Nie wie nic o WPF ani o UI
// =====
public class SzyfrCezara
{
    public string Zaszyfruj(string tekst, int klucz)
    {
        klucz = ((klucz % 26) + 26) % 26;
        char[] znaki = tekst.ToCharArray();

        for (int i = 0; i < znaki.Length; i++)
        {
            char c = znaki[i];
            if (char.IsUpper(c))
                znaki[i] = (char)(((c - 'A' + klucz) % 26) + 'A');
            else if (char.IsLower(c))
                znaki[i] = (char)(((c - 'a' + klucz) % 26) + 'a');
        }

        return new string(znaki);
    }

    public string Odszyfruj(string tekst, int klucz)
    {
        return Zaszyfruj(tekst, -klucz);
    }
}

// =====
// KLASA UI – tylko zbiera dane i wyświetla wyniki
// =====
public partial class MainWindow : Window
{
    // Obiekty klas logiki – tworzone raz
    private readonly GeneratorHasel _generator = new GeneratorHasel();
    private readonly SzyfrCezara _szyfr = new SzyfrCezara();

    public MainWindow()
    {
        InitializeComponent();
    }

    private void btnGenerujHaslo_Click(object sender, RoutedEventArgs e)
    {
        int.TryParse(txtDlugoscHasla.Text, out int dl);
        // UI przekazuje dane do klasy logiki, a wynik wyświetla
        tbHaslo.Text = _generator.Generuj(dl);
    }

    private void btnZaszyfruj_Click(object sender, RoutedEventArgs e)

```



```

    {
        int.TryParse(txtKluczSzyfrowania.Text, out int klucz);
        // UI przekazuje dane do klasy logiki, a wynik wyświetla
        tbZaszyfrowany.Text = _szyfr.Zaszyfruj(txtTekstDoSzyfrowania.Text, klucz);
    }
}
}

```

Kluczowe elementy:

- Klasa `GeneratorHasel` — nie zawiera żadnych odwołań do WPF (`using System.Windows` brak)
- Klasa `SzyfrCezara` — czysty algorytm, można go przetestować bez uruchamiania WPF
- `MainWindow` — tylko zbiera dane z pól tekstowych i wyświetla wyniki; nie zawiera logiki algorytmu
- `private readonly` — pola klas logiki tworzone raz w konstruktorze okna

24.5. List

Poniższy przykład to kompletna aplikacja: odczyt pliku → parsowanie → lista obiektów → wyświetlenie w `ListBox` → szczegóły wybranego elementu.

MainWindow.xaml:

```

<Window x:Class="ListaObiektow.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Plik → Lista&lt;Album&gt;" Height="520" Width="600">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>

        <!-- Nagłówek -->
        <Border Grid.Row="0" Grid.ColumnSpan="2"
                Background="DarkSlateBlue" Padding="12">
            <StackPanel Orientation="Horizontal">
                <TextBlock Text="📁 Kolekcja albumów"
                           Foreground="White" FontSize="16" FontWeight="Bold"
                           VerticalAlignment="Center" />
                <Button Content="Wczytaj plik" Margin="20,0,0,0"
                        Height="28" Width="100"
                        Click="btnWczytaj_Click" />
            </StackPanel>
        </Border>

        <!-- Lista albumów -->
        <StackPanel Grid.Row="1" Grid.Column="0" Margin="10">
            <TextBlock Text="Lista albumów:" FontWeight="Bold" Margin="0,0,0,5" />
            <ListBox x:Name="lstAlbumy"
                   Height="300"
                   SelectionChanged="lstAlbumy_SelectionChanged" />
            <TextBlock x:Name="tbLicznik" Text="(brak danych)"
                   Foreground="Gray" Margin="0,5,0,0" FontSize="12" />
        </StackPanel>

        <!-- Szczegóły wybranego albumu -->
        <StackPanel Grid.Row="1" Grid.Column="1" Margin="10">
            <TextBlock Text="Szczegóły:" FontWeight="Bold" Margin="0,0,0,5" />

```

```

        <Border Background="AliceBlue" CornerRadius="5"
            Padding="12" MinHeight="200">
            <StackPanel>
                <TextBlock x:Name="tbDetailTytul" FontSize="16"
                    FontWeight="Bold" TextWrapping="Wrap"
                    Margin="0,0,0,5"/>
                <TextBlock x:Name="tbDetailArtysta" FontSize="13"
                    Foreground="SlateGray" Margin="0,0,0,3"/>
                <TextBlock x:Name="tbDetailRok" FontSize="13"
                    Margin="0,0,0,3"/>
                <TextBlock x:Name="tbDetailUtworow" FontSize="13"
                    Margin="0,0,0,3"/>
                <TextBlock x:Name="tbDetailPobran" FontSize="13"
                    Margin="0,0,0,8"/>
                <Separator/>
                <TextBlock x:Name="tbDetailPopularnosc"
                    FontSize="14" FontWeight="Bold"
                    Margin="0,8,0,0"/>
            </StackPanel>
        </Border>
    </StackPanel>

    <!-- Pasek statusu -->
    <Border Grid.Row="2" Grid.ColumnSpan="2"
        Background="#E0E0E0" Padding="8">
        <TextBlock x:Name="tbStatus" Text="Gotowy." FontSize="11"/>
    </Border>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Windows;
using System.Windows.Controls;

namespace ListaObiektow
{
    // =====
    // KLASA MODELU
    // =====
    public class Album
    {
        public string Tytul { get; set; }
        public string Artysta { get; set; }
        public int Rok { get; set; }
        public int LiczbaUtworow { get; set; }
        public int LiczbaPobran { get; set; }

        public Album(string tytul, string artysta, int rok,
            int liczbaUtworow, int liczbaPobran)
        {
            Tytul = tytul;
            Artysta = artysta;
            Rok = rok;
            LiczbaUtworow = liczbaUtworow;
            LiczbaPobran = liczbaPobran;
        }
    }
}

```

```

    public string OcenPopularnosc()
    {
        if (LiczbaPobran ≥ 900) return "☆☆☆ Mega hit";
        if (LiczbaPobran ≥ 500) return "☆☆ Popularny";
        return "☆ Mało znany";
    }

    // ToString wywoływane automatycznie przez ListBox
    public override string ToString() ⇒ $"{Tytuł} – {Artysta}";
}

// =====
// KLASA USŁUGI – wczytywanie danych
// =====
public static class AlbumReader
{
    public static List<Album> WczytajZPliku(string sciezka)
    {
        var lista = new List<Album>();
        string[] linie = File.ReadAllLines(sciezka, Encoding.UTF8);

        foreach (string linia in linie)
        {
            if (string.IsNullOrEmpty(linia)) continue;

            string[] p = linia.Split(';');
            if (p.Length < 5) continue;

            int.TryParse(p[2].Trim(), out int rok);
            int.TryParse(p[3].Trim(), out int liczbaU);
            int.TryParse(p[4].Trim(), out int pobran);

            // Tworzenie nowego obiektu z danych z pliku
            lista.Add(new Album(
                tytuł:      p[0].Trim(),
                artysta:    p[1].Trim(),
                rok:        rok,
                liczbaUtworow:  liczbaU,
                liczbaPobran: pobran
            ));
        }

        return lista;
    }
}

// =====
// KLASA OKNA – tylko UI
// =====
public partial class MainWindow : Window
{
    private List<Album> _albumy = new List<Album>();
    private readonly string _sciezkaPliku;

    public MainWindow()
    {
        InitializeComponent();
        _sciezkaPliku = Path.Combine(
            AppDomain.CurrentDomain.BaseDirectory, "albumy.txt");
        UtworzPrzykładowyPlik();
    }
}

```

```

private void UtworzPrzykladowyPlik()
{
    if (!File.Exists(_sciezkaPliku))
    {
        string zawartosc =
            "Nevermind;Nirvana;1991;13;854\n" +
            "Abbey Road;The Beatles;1969;17;972\n" +
            "Thriller;Michael Jackson;1982;9;1000\n" +
            "Dark Side of the Moon;Pink Floyd;1973;10;931\n" +
            "Back in Black;AC/DC;1980;10;312\n";
        File.WriteAllText(_sciezkaPliku, zawartosc, Encoding.UTF8);
        tbStatus.Text = "Utworzono przykładowy plik albumy.txt";
    }
}

private void btnWczytaj_Click(object sender, RoutedEventArgs e)
{
    if (!File.Exists(_sciezkaPliku))
    {
        tbStatus.Text = "Nie znaleziono pliku albumy.txt";
        return;
    }

    try
    {
        // Wczytanie danych przez klasę usługi
        _albumy = AlbumReader.WczytajZPliku(_sciezkaPliku);

        // Aktualizacja ListBox
        lstAlbumy.ItemsSource = null;
        lstAlbumy.ItemsSource = _albumy;

        tbLicznik.Text = $"Wczytano {_albumy.Count} albumów.";
        tbStatus.Text = $"Plik: {_sciezkaPliku}";
    }
    catch (Exception ex)
    {
        tbStatus.Text = $"Błąd wczytywania: {ex.Message}";
    }
}

private void lstAlbumy_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    // Sprawdź czy wybrany element jest obiektem Album (operator 'is')
    if (lstAlbumy.SelectedItem is Album wybrany)
    {
        // Aktualizacja panelu szczegółów
        tbDetailTytul.Text = wybrany.Tytul;
        tbDetailArtysta.Text = $"Artysta: {wybrany.Artysta}";
        tbDetailRok.Text = $"Rok wydania: {wybrany.Rok}";
        tbDetailUtworow.Text = $"Liczba utworów: {wybrany.LiczbaUtworow}";
        tbDetailPobran.Text = $"Pobrań: {wybrany.LiczbaPobran:N0}";
        tbDetailPopularnosc.Text = wybrany.OcenPopularnosc();
    }
}
}
}

```

Kluczowe elementy:

- Klasa `Album` — model danych, pełna hermetyzacja przez właściwości

- Klasa `AlbumReader` — statyczna klasa usługowa odpowiedzialna wyłącznie za wczytywanie
- `static class` — nie musisz tworzyć obiektu, wywołujesz `AlbumReader.WczytajZPliku(...)`
- `lstAlbumy.ItemsSource = _albumy` — przypisanie listy do ListBox; kontrolka wywołuje `ToString()` każdego obiektu
- `if (lstAlbumy.SelectedItem is Album wybrany)` — bezpieczne rzutowanie z pattern matching (C# 7+)
- `{wybrany.LiczbaPobran:N0}` — formatowanie liczby z separatorem tysięcy

Schemat przepływu danych:

```

Plik albumy.txt
    ↓
AlbumReader.WczytajZPliku() ← Klasa usługi
    ↓
List<Album>                  ← lista obiektów modelu
    ↓
lstAlbumy.ItemsSource        ← ListBox (UI)
    ↓
Kliknięcie w ListBox
    ↓
lstAlbumy_SelectionChanged  ← event handler (UI)
    ↓
Wybrany Album               ← obiekt modelu
    ↓
Aktualizacja TextBlocków    ← widok szczegółów
  
```

25. Binding — podstawy

25.1. Czym jest binding

W tej sekcji zobaczysz, jak działa mechanizm wiązania danych (ang. binding) w WPF. Binding pozwala na automatyczne synchronizowanie danych między kodem C# a interfejsem użytkownika w XAML. Zamiast ręcznie przypisywać wartości (np. `MojePoLe.Text = zmienna;`), deklarujesz w XAML, że dana kontrolka "nasłuchuje" zmian konkretnej właściwości.

Dzięki temu kod staje się dużo bardziej przejrzysty, a logika aplikacji zostaje oddzielona od wyglądu. Binding to podstawa zaawansowanych wzorców architektonicznych, które poznasz później.

MainWindow.xaml:

```

<Window x:Class="NaukaBindingu.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Czym jest binding" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <Label Content="Wpisz tekst poniżej:" />
            <TextBox x:Name="ZrodloDanychTextBox" Width="300" Height="30" Margin="0,0,0,20"/>

            <Label Content="Wynik bindingu:" />
            <TextBlock Text="{Binding ElementName=ZrodloDanychTextBox, Path=Text}"
                      FontSize="20" Foreground="Blue" />
        </StackPanel>
    </Grid>
</Window>
  
```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace NaukaBindingu
{
    public partial class MainWindow : Window
  
```

```

    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy w tym przykładzie:

- `{Binding ...}` — specjalna składnia w XAML, która mówi WPF, że ta właściwość będzie dynamicznie pobierać wartość.
- `ElementName=ZrodloDanychTextBox` — wskazuje nazwę kontrolki, z której chcemy pobrać dane.
- `Path=Text` — określa, którą dokładnie właściwość ze źródłowej kontrolki chcemy czytać. Tutaj czytamy wpisany tekst.

25.2. Text

W tej sekcji zobaczysz, jak określić globalne źródło danych dla całego okna za pomocą właściwości `DataContext`. Zamiast pisać za każdym razem `ElementName`, możemy przypisać obiekt (np. samo okno) jako główne źródło danych dla całego XAML.

Najprostszym sposobem na start jest ustawienie w kodzie C# `this.DataContext = this;`. Dzięki temu XAML wie, że ma szukać właściwości bezpośrednio w klasie okna.

MainWindow.xaml:

```

<Window x:Class="DataContextPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="DataContext Przykład" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <!-- Nie podajemy ElementName, bo DataContext to całe okno -->
            <TextBlock Text="{Binding Powitanie}" FontSize="24" FontWeight="Bold" />
            <TextBlock Text="{Binding Informacja}" FontSize="16" Margin="0,10,0,0" />
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace DataContextPrzyklad
{
    public partial class MainWindow : Window
    {
        // Właściwości publiczne, do których będziemy się wiązać
        public string Powitanie { get; set; }
        public string Informacja { get; set; }

        public MainWindow()
        {
            InitializeComponent();

            // Ustawiamy wartości właściwości
            Powitanie = "Witaj w świecie DataContext!";
            Informacja = "To okno używa this jako DataContext.";

            // Ustawiamy DataContext na instancję tego okna
            this.DataContext = this;
        }
    }
}

```

```
}  
  
}
```

Kluczowe elementy:

- `this.DataContext = this;` — mówi oknu XAML: "szukaj danych w moich własnych właściwościach C#".
- **Właściwości publiczne** (`{ get; set; }`) — tylko do takich właściwości można się wiązać. Pola prywatne ani zwykłe zmienne nie zadziałają w bindingu.
- **Brak `ElementName`** — w XAML piszesz tylko `{Binding Powitanie}`. WPF automatycznie sprawdza DataContext, znajduje klasę MainWindow i wyciąga z niej `Powitanie`.

25.3. Binding do właściwości

W tej sekcji zobaczysz, jak wiązać dane z niestandardowym obiektem klasy, a nie tylko z właściwościami okna. Tworzysz osobną klasę reprezentującą dane i przypisujesz jej instancję jako `DataContext`.

MainWindow.xaml:

```
<Window x:Class="WlasciwosciBinding.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Binding do Obiektu" Height="450" Width="800">  
    <Grid>  
        <StackPanel Margin="20">  
            <StackPanel Orientation="Horizontal" Margin="5">  
                <TextBlock Text="Imię: " Width="100"/>  
                <TextBlock Text="{Binding Imie}" FontWeight="Bold" />  
            </StackPanel>  
  
            <StackPanel Orientation="Horizontal" Margin="5">  
                <TextBlock Text="Wiek: " Width="100"/>  
                <TextBlock Text="{Binding Wiek}" FontWeight="Bold" />  
            </StackPanel>  
  
            <StackPanel Orientation="Horizontal" Margin="5">  
                <TextBlock Text="Zawód: " Width="100"/>  
                <TextBlock Text="{Binding Zawod}" FontWeight="Bold" />  
            </StackPanel>  
        </StackPanel>  
    </Grid>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.Windows;  
  
namespace WlasciwosciBinding  
{  
    // Osobna klasa na dane  
    public class Pracownik  
    {  
        public string Imie { get; set; }  
        public int Wiek { get; set; }  
        public string Zawod { get; set; }  
    }  
  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
    }  
}
```

```

// Tworzysz obiekt z danymi
Pracownik nowyPracownik = new Pracownik
{
    Imie = "Jan Kowalski",
    Wiek = 35,
    Zawod = "Programista WPF"
};

// Przypisujesz obiekt jako źródło danych
this.DataContext = nowyPracownik;
}
}
}

```

Kluczowe elementy:

- **Klasa Pracownik** — prosta klasa z trzema publicznymi właściwościami. Muszą to być właściwości (get; set;), a nie pola klasy.
- `this.DataContext = nowyPracownik;` — całe okno otrzymuje jeden obiekt pracownika, z którego XAML pobiera wartości dla `Imie`, `Wiek` i `Zawod`.
- **Czysty XAML** — interfejs po prostu wyświetla to, co dostanie, nie mając pojęcia o szczegółach tworzenia pracownika w kodzie.

25.4. Mode

W tej sekcji zobaczysz, jak umożliwić przepływ danych w obu kierunkach. Standardowy binding często jest jednokierunkowy (od danych do kontrolki). Gdy użytkownik wpisuje coś w TextBox, chcemy, by ta wartość wracała do naszego obiektu C#. W tym celu używasz parametru `Mode=TwoWay`.

MainWindow.xaml:

```

<Window x:Class="TwoWayBinding.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="TwoWay Binding" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <Label Content="Edytuj dane:" />
            <!-- Mode=TwoWay i UpdateSourceTrigger=PropertyChanged sprawia, że wpisywany tekst od razu aktualizuje obiekt -->
            <TextBox Text="{Binding TekstUzytkownika, Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}"
                    Width="300" Height="30" Margin="0,0,0,20"/>

            <Label Content="Podgląd wartości w obiekcie:" />
            <TextBlock Text="{Binding TekstUzytkownika}" FontSize="20" Foreground="Green" />

            <Button Content="Pokaż w MessageBoxie wartość obiektu"
                    Width="250" Height="30" Margin="0,20,0,0"
                    Click="SprawdzButton_Click"/>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace TwoWayBinding
{
    // Klasa przechowująca dane
    public class DaneAplikacji
    {
        public string TekstUzytkownika { get; set; }
    }
}

```



```

public partial class MainWindow : Window
{
    private DaneAplikacji mojeDane;

    public MainWindow()
    {
        InitializeComponent();

        mojeDane = new DaneAplikacji();
        mojeDane.TekstUzytkownika = "Początkowy tekst";

        this.DataContext = mojeDane;
    }

    private void SprawdzButton_Click(object sender, RoutedEventArgs e)
    {
        // Sprawdzamy czy zmiana w TextBoxie faktycznie zaktualizowała obiekt
        MessageBox.Show("Obecna wartość w obiekcie to: " + mojeDane.TekstUzytkownika);
    }
}

```

Kluczowe elementy:

- **Mode=TwoWay** — informuje WPF, że gdy użytkownik edytuje TextBox, wartość ma zostać z powrotem wpisana do właściwości **TekstUzytkownika**.
- **UpdateSourceTrigger=PropertyChanged** — wymusza aktualizację obiektu po każdym naciśnięciu klawisza (natychmiast). Bez tego właściwość zaktualizowałaby się dopiero, gdy TextBox straci fokus (np. klikniesz poza nim).
- **Button Click** — w metodzie przycisku odczytujemy wartość bezpośrednio z obiektu C#, a nie z TextBoxa. To potwierdza, że binding dwukierunkowy zadziałał.

25.5. INotifyPropertyChanged — interfejs, PropertyChanged event, OnPropertyChanged(), kompletna implementacja

W tej sekcji zobaczysz, jak poinformować interfejs graficzny o tym, że właściwość w kodzie C# zmieniła swoją wartość. Zwykłe klasy nie informują automatycznie okna o zmianach. Aby to zrobić, klasa musi zaimplementować interfejs **INotifyPropertyChanged**. Bez tego, jeśli zmienisz dane w C#, XAML tego nie zauważy i nie zaktualizuje ekranu.

Poniżej znajduje się KOMPLETNA implementacja tego kluczowego interfejsu.

MainWindow.xaml:

```

<Window x:Class="INotifyExample.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="INotifyPropertyChanged" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <TextBlock Text="{Binding Licznik}" FontSize="50" HorizontalAlignment="Center" Margin="0,0,0,20"/>
            <Button Content="Zwiększ licznik w kodzie C#" Width="200" Height="40" Click="ZwiekszButton_Click"/>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.ComponentModel; // Potrzebne dla INotifyPropertyChanged
using System.Runtime.CompilerServices; // Potrzebne dla CallerMemberName
using System.Windows;

namespace INotifyExample
{
    // Klasa wdrażająca INotifyPropertyChanged
    public class ModelDanych : INotifyPropertyChanged
    {

```

```

{
    private int licznik;

    // Właściwość ze specjalnym setterem
    public int Licznik
    {
        get { return licznik; }
        set
        {
            if (licznik != value)
            {
                licznik = value;
                OnPropertyChanged(); // Wywołujemy metodę informującą UI
            }
        }
    }

    // 1. Deklaracja zdarzenia z interfejsu
    public event PropertyChangedEventHandler PropertyChanged;

    // 2. Metoda pomocnicza do wywoływania zdarzenia
    // Zastosowanie [CallerMemberName] sprawia, że kompilator sam wstawia nazwę właściwości
    protected virtual void OnPropertyChanged([CallerMemberName] string propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}

public partial class MainWindow : Window
{
    private ModelDanych mojModel;

    public MainWindow()
    {
        InitializeComponent();
        mojModel = new ModelDanych { Licznik = 0 };
        this.DataContext = mojModel;
    }

    private void ZwiększButton_Click(object sender, RoutedEventArgs e)
    {
        // Zmieniamy tylko właściwość w obiekcie, a dzięki INotifyPropertyChanged
        // XAML automatycznie zauważy zmianę i odświeży TextBlock
        mojModel.Licznik++;
    }
}
}

```

Kluczowe elementy:

- `INotifyPropertyChanged` — interfejs wymagający posiadania zdarzenia `PropertyChanged`.
- **Pełna właściwość (get i set)** — zamiast automatycznej właściwości `public int Licznik { get; set; }` musimy użyć pola prywatnego `licznik` i rozbudowanego settera, który wywołuje zdarzenie przy każdej zmianie.
- `[CallerMemberName]` — to atrybut sprawiający, że nie musimy ręcznie wpisywać nazwy właściwości w stringu (np. `OnPropertyChanged("Licznik")`). Kompilator zrobi to za nas.
- **Reaktywność** — gdy klikasz przycisk, zmienia się wyłącznie liczba w obiekcie C#, a interfejs sam reaguje i podmienia widoczny na ekranie napis.

25.6. List

W tej sekcji zobaczysz, jak połączyć listę danych z kontrolką typu ListBox lub DataGrid. Standardowa klasa `List<T>` nie informuje interfejsu o dodaniu lub usunięciu elementów. Dlatego w WPF dla kolekcji używa się specjalnej klasy `ObservableCollection<T>`.

MainWindow.xaml:

```

<Window x:Class="ObservableColl.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Binding Kolekcji" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <!-- ItemsSource mówi liście skąd ma brać dane -->
            <ListBox ItemsSource="{Binding Nazwiska}" Height="150" Margin="0,0,0,20"/>

            <StackPanel Orientation="Horizontal" HorizontalAlignment="Center">
                <TextBox x:Name="NoweNazwiskoTextBox" Width="150" Height="30" Margin="0,0,10,0"/>
                <Button Content="Dodaj do listy" Width="100" Height="30" Click="DodajButton_Click"/>
            </StackPanel>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.ObjectModel; // Przestrzeń nazw dla ObservableCollection
using System.Windows;

namespace ObservableColl
{
    public class MojaAplikacjaDane
    {
        // Kolekcja, która sama powiadamia UI o dodaniu lub usunięciu elementu
        public ObservableCollection<string> Nazwiska { get; set; }

        public MojaAplikacjaDane()
        {
            Nazwiska = new ObservableCollection<string>();
            Nazwiska.Add("Kowalski");
            Nazwiska.Add("Nowak");
        }
    }

    public partial class MainWindow : Window
    {
        private MojaAplikacjaDane dane;

        public MainWindow()
        {
            InitializeComponent();
            dane = new MojaAplikacjaDane();
            this.DataContext = dane;
        }

        private void DodajButton_Click(object sender, RoutedEventArgs e)
        {
            string nowe = NoweNazwiskoTextBox.Text;
            if (!string.IsNullOrEmpty(nowe))
            {
                // Dodajemy element tylko do obiektu ObservableCollection.
                // ListBox automatycznie zaktualizuje widok!
                dane.Nazwiska.Add(nowe);
                NoweNazwiskoTextBox.Clear();
            }
        }
    }
}

```

```
}  
  
}
```

Kluczowe elementy:

- `ObservableCollection<T>` — klasa podobna do `List<T>`, ale posiadająca wbudowany mechanizm powiadamiania XAML o zmianach w liście (dodanie, usunięcie, wyczyszczenie).
- `ItemsSource="{Binding Nazwiska}"` — atrybut w `ListBox`, który przyjmuje całą kolekcję danych.
- **Auto-odświeżanie** — kod C# w metodzie kliknięcia nic nie wie o `ListBoxie`. Modyfikuje tylko strukturę danych, a WPF automatycznie aktualizuje elementy na ekranie.

25.7. Mode

W tej sekcji zobaczysz, jak wygląda podział aplikacji na warstwy za pomocą wzorca MVVM (Model-View-ViewModel). Jest to standard w tworzeniu profesjonalnych aplikacji WPF. Polega on na oddzieleniu interfejsu (View) od danych (Model) poprzez specjalną klasę pośredniczącą (ViewModel).

W tym przykładzie połączymy wiedzę o bindingu i `INotifyPropertyChanged`.

MainWindow.xaml:

```
<Window x:Class="WzorzecMVVM.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Wprowadzenie do MVVM" Height="450" Width="800">  
    <Grid>  
        <StackPanel Margin="20">  
            <TextBlock Text="Kalkulator MVVM" FontSize="20" FontWeight="Bold" Margin="0,0,0,20"/>  
  
            <StackPanel Orientation="Horizontal" Margin="0,0,0,10">  
                <TextBlock Text="Liczba A: " VerticalAlignment="Center" Width="70"/>  
                <TextBox Text="{Binding LiczbaA, UpdateSourceTrigger=PropertyChanged}" Width="100"/>  
            </StackPanel>  
  
            <StackPanel Orientation="Horizontal" Margin="0,0,0,10">  
                <TextBlock Text="Liczba B: " VerticalAlignment="Center" Width="70"/>  
                <TextBox Text="{Binding LiczbaB, UpdateSourceTrigger=PropertyChanged}" Width="100"/>  
            </StackPanel>  
  
            <StackPanel Orientation="Horizontal" Margin="0,20,0,0">  
                <TextBlock Text="Suma: " FontSize="18" VerticalAlignment="Center" Width="70"/>  
                <TextBlock Text="{Binding Suma}" FontSize="18" FontWeight="Bold" Foreground="Red"/>  
            </StackPanel>  
        </StackPanel>  
    </Grid>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.ComponentModel;  
using System.Runtime.CompilerServices;  
using System.Windows;  
  
namespace WzorzecMVVM  
{  
    // MODEL - czyste dane lub prosta logika (tutaj pominięty dla prostoty)  
  
    // VIEWMODEL - logika powiązana z widokiem  
    public class KalkulatorViewModel : INotifyPropertyChanged  
    {  
        private int liczbaA;  
        public int LiczbaA  
        {  

```

```

        get { return liczbaA; }
        set
        {
            liczbaA = value;
            OnPropertyChanged();
            OnPropertyChanged("Suma"); // Kiedy zmienia się A, wymuszamy aktualizację Sumy
        }
    }

    private int liczbaB;
    public int LiczbaB
    {
        get { return liczbaB; }
        set
        {
            liczbaB = value;
            OnPropertyChanged();
            OnPropertyChanged("Suma"); // Kiedy zmienia się B, wymuszamy aktualizację Sumy
        }
    }

    // Właściwość tylko do odczytu - nie ma settera, jej wartość jest obliczana z innych zmiennych
    public int Suma
    {
        get { return LiczbaA + LiczbaB; }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected virtual void OnPropertyChanged([CallerMemberName] string propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}

// VIEW - plik okna (Code-Behind jest prawie pusty)
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        // Przypisanie ViewModelu do Widoku
        this.DataContext = new KalkulatorViewModel();
    }
}

```

Kluczowe elementy:

- **View (Widok)** — to plik XAML. Jego zadaniem jest tylko ładnie wyświetlać dane. Nie wykonuje żadnych obliczeń.
- **ViewModel** — to klasa C# zawierająca logikę dla danego okna. W niej wykonują się obliczenia i w niej znajdują się właściwości obsługujące powiadomianie.
- **Brak zdarzeń (Events)** — zauważ, że w C# nie ma żadnych metod takich jak `TextChanged`. ViewModel sam wylicza wynik dzięki setterom i przekazuje go do widoku.

25.8. ICommand

W tej sekcji zobaczysz, jak całkowicie pozbyć się zdarzeń typu `Button_Click` w pliku Code-Behind (C# widoku) dzięki użyciu komend i interfejsu `ICommand`. Komendy pozwalają dodatkowo automatycznie sterować włączaniem i wyłączeniem przycisków za pomocą metody `CanExecute`.

Poniżej znajduje się KOMPLETNA implementacja RelayCommand i jej użycie w MVVM.

MainWindow.xaml:

```

<Window x:Class="ICommandPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

```

```

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
Title="Komendy MVVM" Height="450" Width="800">
<Grid>
    <StackPanel Margin="20">
        <TextBox Text="{Binding Wiadomosc, UpdateSourceTrigger=PropertyChanged}"
            Width="300" Height="30" Margin="0,0,0,10"/>

        <!-- Używamy Command zamiast Click! -->
        <Button Content="Zapisz do pliku (udawane)"
            Command="{Binding ZapiszCommand}"
            Width="200" Height="40" />

        <TextBlock Text="Zauważ, że przycisk staje się nieaktywny, jeśli pole tekstowe jest puste!"
            Margin="0,20,0,0" FontStyle="Italic" TextWrapping="Wrap"/>
    </StackPanel>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;
using System.Windows.Input; // Wymagane dla ICommand

namespace ICommandPrzyklad
{
    // KOMPLETNA IMPLEMENTACJA RelayCommand (uniwersalnej klasy dla komend)
    public class RelayCommand : ICommand
    {
        private readonly Action<object> execute;
        private readonly Predicate<object> canExecute;

        // Konstruktor
        public RelayCommand(Action<object> execute, Predicate<object> canExecute = null)
        {
            this.execute = execute ?? throw new ArgumentNullException(nameof(execute));
            this.canExecute = canExecute;
        }

        // Wydarzenie wymagane przez ICommand - odpala się, gdy zmieniają się warunki (np. wpisano tekst)
        public event EventHandler CanExecuteChanged
        {
            add { CommandManager.RequerySuggested += value; }
            remove { CommandManager.RequerySuggested -= value; }
        }

        // Czy komenda może się wykonać?
        public bool CanExecute(object parameter)
        {
            return canExecute == null || canExecute(parameter);
        }

        // Główna logika do wykonania
        public void Execute(object parameter)
        {
            execute(parameter);
        }
    }
}

```

```

// ViewModel
public class GlownyViewModel : INotifyPropertyChanged
{
    private string wiadomosc;
    public string Wiadomosc
    {
        get { return wiadomosc; }
        set
        {
            wiadomosc = value;
            OnPropertyChanged();
        }
    }

    // Właściwość przechowująca komendę
    public ICommand ZapiszCommand { get; private set; }

    public GlownyViewModel()
    {
        // Inicjalizacja komendy: przekaż co ma się stać (Execute) i pod jakim warunkiem (CanExecute)
        ZapiszCommand = new RelayCommand(
            execute: _ => ZapiszDane(),
            canExecute: _ => CzyMoznaZapisac()
        );
    }

    private void ZapiszDane()
    {
        MessageBox.Show("Zapisano wiadomość: " + Wiadomosc);
        Wiadomosc = string.Empty; // Czyszczenie pola po akcji
    }

    private bool CzyMoznaZapisac()
    {
        // Przycisk jest włączony (IsEnabled = true) tylko wtedy, gdy Wiadomosc nie jest pusta
        return !string.IsNullOrEmpty(Wiadomosc);
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected virtual void OnPropertyChanged([CallerMemberName] string propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}

// Code-Behind (Widok)
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        this.DataContext = new GlownyViewModel();
    }
}

```

Kluczowe elementy:

- **RelayCommand** — klasa, która implementuje `ICommand`. Możesz jej używać w każdym projekcie MVVM. Przyjmuje ona akcję do wykonania i ewentualny warunek uruchomienia.
- `Command="{Binding ZapiszCommand}"` — przypisujemy komendę do przycisku. Nie ma tutaj atrybutu `Click`.

- **CanExecute** — ta metoda decyduje o tym, czy przycisk jest klikalny. Gdy funkcja `CzyMoznaZapisac()` zwróci fałsz, WPF automatycznie zablokuje przycisk, zmieniając go na wyszarzony.

26. Style i wygląd aplikacji

26.1. Background

W tej sekcji zobaczysz, jak zmieniać wygląd pojedynczych kontroltek ręcznie, bezpośrednio w pliku XAML. Jest to najszybszy sposób stylizacji, chociaż przy dużej ilości kontroltek prowadzi do powtarzania kodu.

MainWindow.xaml:

```
<Window x:Class="FormatowanieReczne.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Formatowanie" Height="450" Width="800">
    <Grid Background="LightGray">
        <StackPanel Margin="30">
            <!-- Formatowanie bezpośrednie -->
            <TextBlock Text="Zwykły nagłówek"
                      FontSize="30"
                      FontWeight="Bold"
                      Foreground="DarkBlue"
                      Margin="0,0,0,10" />

            <Button Content="Czerwony przycisk"
                   Background="DarkRed"
                   Foreground="White"
                   FontSize="16"
                   FontWeight="DemiBold"
                   Padding="10,5"
                   Width="200"
                   HorizontalAlignment="Left" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace FormatowanieReczne
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Kluczowe elementy:

- **Foreground i Background** — kontrolują kolor tekstu i kolor tła.
- **FontSize i FontWeight** — zmieniają wielkość czcionki oraz jej grubość (np. Bold).
- **Padding** — wewnętrzny margines w kontrolkach typu Button, odpychający tekst od krawędzi przycisku.

26.2. Source

W tej sekcji zobaczysz, jak zapobiegać powtarzaniu tego samego kodu. Wykorzystujesz zasoby okna, by zdefiniować styl raz i zastosować go w wielu miejscach jednocześnie.

MainWindow.xaml:

```
<Window x:Class="StyleLokalne.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Style Lokalne" Height="450" Width="800">

    <!-- Definicja zasobów lokalnych dla tego okna -->
    <Window.Resources>
        <Style TargetType="Button" x:Key="MojStylPrzycisku">
            <Setter Property="Background" Value="LightSeaGreen"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="FontSize" Value="18"/>
            <Setter Property="Margin" Value="0,5,0,5"/>
            <Setter Property="Height" Value="40"/>
        </Style>
    </Window.Resources>

    <Grid>
        <StackPanel Margin="50">
            <!-- Każdy przycisk odwołuje się do stylu za pomocą StaticResource -->
            <Button Content="Opcja 1" Style="{StaticResource MojStylPrzycisku}"/>
            <Button Content="Opcja 2" Style="{StaticResource MojStylPrzycisku}"/>
            <Button Content="Inny przycisk" /> <!-- Ten przycisk nie ma zdefiniowanego stylu, będzie wyglądał domyślnie -->
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace StyleLokalne
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Kluczowe elementy:

- `Window.Resources` — specjalna sekcja u góry XAML, gdzie przechowujesz narzędzia i style dla tego okna.
- `x:Key` — unikalna nazwa przypisana do stylu. Bez niej nie mógłbyś się odwołać do tego konkretnego wyglądu.
- `TargetType="Button"` — mówi o tym, do jakiej kontrolki ten styl może zostać przypisany.
- `StaticResource` — sposób odwołania się z kontrolki do zasobu stworzonego wyżej w pliku.

26.3. Source

W tej sekcji zobaczysz, jak ustalić domyślny wygląd dla wszystkich kontrolki danego typu w całej aplikacji. Zamiast tworzyć styl w jednym oknie, umieszczasz go w pliku `App.xaml`. Poniższy przykład pokazuje również tworzenie stylu bez klucza (`x:Key`), co sprawia, że jest on stosowany automatycznie.

App.xaml:

```
<Application x:Class="StyleGlobalne.App"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
```

```

        StartupUri="MainWindow.xaml">
<Application.Resources>
    <!-- Styl bez x:Key! Zaaplikuje się do każdego TextBoxa w aplikacji -->
    <Style TargetType="TextBox">
        <Setter Property="Margin" Value="0,5,0,5" />
        <Setter Property="FontSize" Value="16" />
        <Setter Property="BorderBrush" Value="CornflowerBlue" />
        <Setter Property="BorderThickness" Value="2" />
    </Style>
</Application.Resources>
</Application>

```

MainWindow.xaml:

```

<Window x:Class="StyleGlobalne.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Style Globalne" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30">
            <!-- Oba pola tekstowe z automatu otrzymają pogrubioną niebieską ramkę z App.xaml -->
            <TextBox Text="Pierwsze pole..." />
            <TextBox Text="Drugie pole..." />
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace StyleGlobalne
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **App.xaml** — plik konfiguracyjny uruchamiający się z całą aplikacją. Idealne miejsce na kolory, czcionki i style używane na wielu ekranach.
- **Brak x:Key w Stylu** — tworzy to tak zwany "styl domyślny" (implicit style). Każda kontrolka podanego typu (**TargetType**) na każdym ekranie dostanie ten styl, chyba że ustawisz jej wygląd ręcznie.

26.4. Button

W tej sekcji omówimy dokładniej działanie Settera w stylach. Styl to w WPF zbiór przypisań. Setter składa się z podania właściwości (**Property**) i wartości (**Value**), jaką ta właściwość ma przyjąć.

MainWindow.xaml:

```

<Window x:Class="ZaawansowanySetter.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Setter w Stylach" Height="450" Width="800">
    <Window.Resources>

```

```

        <Style TargetType="Button">
            <Setter Property="FontFamily" Value="Consolas" />
            <Setter Property="FontSize" Value="20" />
            <Setter Property="Width" Value="250" />
            <Setter Property="Height" Value="50" />
            <Setter Property="Background" Value="Coral" />
            <Setter Property="Cursor" Value="Hand" />
        </Style>
    </Window.Resources>

    <Grid>
        <StackPanel Margin="20" HorizontalAlignment="Center">
            <Button Content="START APLIKACJI" Margin="0,0,0,10" />
            <Button Content="USTAWIENIA" Margin="0,0,0,10" />
            <!-- Ten przycisk nadpisuje Setter Width ręcznie -->
            <Button Content="WYJŚCIE" Width="100" />
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace ZaawansowanySetter
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Setter** — dosłownie "ustawiacz". To instrukcja "ustaw tę właściwość na tę wartość".
- **Kolejność nadpisywania** — jeśli ustawisz coś w stylu, a potem zmienisz tę samą właściwość bezpośrednio w kontrolce (np. `Width="100"` w ostatnim przycisku), WPF uzna lokalną wartość za ważniejszą.

26.5. Trigger — PropertyTrigger, zmiana wyglądu gdy IsMouseOver=True, IsEnabled=False

W tej sekcji zobaczysz, jak tworzyć reaktywne interfejsy bez pisania logiki C#. Wyzwalacze (Triggers) pozwalają zmieniać wartości właściwości, gdy spełniony jest określony warunek (np. kursor najechał na przycisk).

MainWindow.xaml:

```

<Window x:Class="TriggeryPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Triggery" Height="450" Width="800">
    <Window.Resources>
        <Style TargetType="Button">
            <Setter Property="Background" Value="LightBlue" />
            <Setter Property="Foreground" Value="Black" />
            <Setter Property="FontSize" Value="18" />

            <Style.Triggers>
                <!-- Trigger sprawdzający czy kursor myszy najechał na kontrolkę -->
                <Trigger Property="IsMouseOver" Value="True">

```

```

        <Setter Property="Background" Value="RoyalBlue" />
        <Setter Property="Foreground" Value="White" />
        <!-- Możemy zmienić dowolną właściwość przy najechaniu myszką -->
        <Setter Property="Cursor" Value="Hand" />
    </Trigger>

    <!-- Trigger na wypadek gdy przycisk jest zablokowany -->
    <Trigger Property="IsEnabled" Value="False">
        <Setter Property="Background" Value="LightGray" />
        <Setter Property="Foreground" Value="Gray" />
    </Trigger>
</Style.Triggers>
</Style>
</Window.Resources>

<Grid>
    <StackPanel Margin="30">
        <Button Content="Najedź na mnie!" Width="200" Height="40" Margin="0,0,0,20" />
        <Button Content="Jestem wyłączony" Width="200" Height="40" IsEnabled="False" />
    </StackPanel>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace TriggeryPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- `Style.Triggers` — blok definiujący listę wyzwalaczy reagujących na stany kontrolki.
- `Trigger Property="..." Value="..."` — określa, jakiego warunku szukamy. Na przykład `Property="IsMouseOver" Value="True"` oznacza moment najechania myszką.
- Tymczasowe Settery** — właściwości wewnątrz triggera obowiązują tylko wtedy, gdy warunek jest prawdziwy. Kiedy myszka opuści przycisk, wróci on do swoich podstawowych ustawień.

26.6. List

W tej sekcji zobaczysz, jak kontrolować wygląd elementów prezentujących dane w kontrolkach list. Kiedy wrzucisz obiekty np. klasy `Album` do `ListBoxa`, WPF po prostu wywoła metodę `ToString()`, co wygląda źle. Aby ułożyć właściwości ładnie na ekranie (np. ze zdjęciem, grubym tekstem itp.), musisz stworzyć kompletny `DataTemplate`.

MainWindow.xaml:

```

<Window x:Class="DataTemplateAlbum.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Szablony Danych" Height="450" Width="800">
    <Grid>
        <ListBox Margin="20" ItemsSource="{Binding ListaAlbumow}">
            <ListBox.ItemTemplate>

```

```

<!-- KOMPLETNY DataTemplate dla pojedynczego elementu -->
<DataTemplate>
    <Border BorderBrush="Gray" BorderThickness="1" CornerRadius="5" Margin="5" Padding="10">
        <StackPanel Orientation="Horizontal">
            <!-- Zakładamy, że obiekty posiadają podane niżej właściwości -->
            <StackPanel Width="250">
                <TextBlock Text="{Binding Tytuł}" FontWeight="Bold" FontSize="18" Foreground="DarkSlateBlue"/>
                <TextBlock Text="{Binding Wykonawca}" FontStyle="Italic" FontSize="14" Margin="0,5,0,0"/>
            </StackPanel>
            <StackPanel VerticalAlignment="Center">
                <TextBlock Text="{Binding RokWydania, StringFormat='Rok: {0}'}" />
                <TextBlock Text="{Binding LiczbaPobran, StringFormat='Pobrania: {0}'}" Foreground="Gray"/>
            </StackPanel>
        </StackPanel>
    </Border>
</DataTemplate>
</ListBox.ItemTemplate>
</ListBox>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.ObjectModel;
using System.Windows;

namespace DataTemplateAlbum
{
    // Klasa danych, na której zbudowany jest DataTemplate
    public class Album
    {
        public string Tytuł { get; set; }
        public string Wykonawca { get; set; }
        public int RokWydania { get; set; }
        public int LiczbaPobran { get; set; }
    }

    public partial class MainWindow : Window
    {
        public ObservableCollection<Album> ListaAlbumow { get; set; }

        public MainWindow()
        {
            InitializeComponent();

            ListaAlbumow = new ObservableCollection<Album>();
            ListaAlbumow.Add(new Album { Tytuł = "Dark Side of the Moon", Wykonawca = "Pink Floyd", RokWydania = 1973, LiczbaPobran = 500000 });
            ListaAlbumow.Add(new Album { Tytuł = "Thriller", Wykonawca = "Michael Jackson", RokWydania = 1982, LiczbaPobran = 850000 });
            ListaAlbumow.Add(new Album { Tytuł = "Back in Black", Wykonawca = "AC/DC", RokWydania = 1980, LiczbaPobran = 400000 });

            this.DataContext = this;
        }
    }
}

```

Kluczowe elementy:

- `ListBox.ItemTemplate` — właściwość określająca, w jaki sposób zinterpretować wizualnie i narysować jeden element kolekcji.
- `DataTemplate` — pojemnik, w którym budujesz standardowy widok XAML (StackPanel, Grid, kontrolki textowe), ale stosujesz do nich binding z klasą, która trafi do elementu kolekcji (np. z klasą `Album`).

- `StringFormat` — wygodna funkcja bindingu, pozwalająca dokleić tekst przed lub po pobranej zmiennej (np. z formatowaniem `{0}'`).

26.7. SolidColorBrush

W tej sekcji zobaczysz, jak zdefiniować globalne zmienne dla kolorów za pomocą pędzli (Brush) i stworzyć jednolity motyw kolorystyczny, by móc łatwo zmienić kolor całej aplikacji w jednym miejscu.

App.xaml:

```
<Application x:Class="MotywKolorystyczny.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    StartupUri="MainWindow.xaml">

    <Application.Resources>
        <!-- Definiujemy pędzle, które będzie można używać w całej aplikacji -->
        <SolidColorBrush x:Key="KolorGłówny" Color="#2E86C1" />
        <SolidColorBrush x:Key="KolorAkcentu" Color="#F39C12" />
        <SolidColorBrush x:Key="KolorTła" Color="#F4F6F6" />
        <SolidColorBrush x:Key="KolorTekstu" Color="#212F3D" />
    </Application.Resources>
</Application>
```

MainWindow.xaml:

```
<Window x:Class="MotywKolorystyczny.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Aplikacja z motywem" Height="450" Width="800"
    Background="{StaticResource KolorTła}">

    <Grid>
        <StackPanel Margin="30">
            <TextBlock Text="System Zarządzania"
                Foreground="{StaticResource KolorGłówny}"
                FontSize="36" FontWeight="Bold" Margin="0,0,0,20" />

            <TextBlock Text="Wprowadź dane logowania:"
                Foreground="{StaticResource KolorTekstu}"
                FontSize="16" Margin="0,0,0,10" />

            <Button Content="ZALOGUJ SIĘ"
                Background="{StaticResource KolorAkcentu}"
                Foreground="White"
                Width="200" Height="45" HorizontalAlignment="Left"
                FontSize="18" FontWeight="Bold" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace MotywKolorystyczny
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

```
}  
}  
}
```

Kluczowe elementy:

- **SolidColorBrush** — struktura reprezentująca kolor wypełnienia w WPF. To ona jest potrzebna dla właściwości **Background** czy **Foreground** kontrolerek.
- **Kolory Hex** — wartość w **Color** wpisujemy w formacie heksadecymalnym z użyciem zanków RGB (np. **#2E86C1**).
- **Zmiana w jednym miejscu** — jeśli zmienisz kod koloru w **App.xaml**, natychmiast wszystkie przyciski i teksty korzystające z tego klucza w aplikacji zmieniają swój kolor.

27. Nawigacja i wiele widoków

27.1. Wiele okien

W tej sekcji zobaczysz, jak nawigować między tradycyjnymi oknami w WPF. Omówione zostaną dwa sposoby otwierania okien (modalny i niemodalny) oraz sposób na przesłanie danych z okna A do okna B przez konstruktor klasy.

WindowSzczegoly.xaml: (Drugie okno)

```
<Window x:Class="WieleOkien.WindowSzczegoly"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Szczegóły" Height="300" Width="400">  
    <Grid>  
        <StackPanel Margin="20" HorizontalAlignment="Center" VerticalAlignment="Center">  
            <TextBlock x:Name="WiadomoscTextBlock" FontSize="20" FontWeight="Medium" Margin="0,0,0,20"/>  
            <Button Content="Zamknij to okno" Width="150" Height="30" Click="ZamknijButton_Click"/>  
        </StackPanel>  
    </Grid>  
</Window>
```

WindowSzczegoly.xaml.cs: (Drugie okno C#)

```
using System.Windows;  
  
namespace WieleOkien  
{  
    public partial class WindowSzczegoly : Window  
    {  
        // Przekazujemy dane przez konstruktor!  
        public WindowSzczegoly(string odebranaWiadomosc)  
        {  
            InitializeComponent();  
            WiadomoscTextBlock.Text = "Otrzymano: " + odebranaWiadomosc;  
        }  
  
        private void ZamknijButton_Click(object sender, RoutedEventArgs e)  
        {  
            this.Close(); // Zamknięcie bieżącego okna  
        }  
    }  
}
```

MainWindow.xaml: (Główne okno)

```
<Window x:Class="WieleOkien.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

```

        Title="Główne Okno" Height="450" Width="800">
<Grid>
    <StackPanel Margin="20">
        <TextBox x:Name="InputTextBox" Width="250" Height="30" Margin="0,0,0,20" Text="Przykładowy tekst" />

        <Button Content="Otwórz okno normalnie (Show)" Height="40" Margin="0,0,0,10" Click="ShowButton_Click" />
        <Button Content="Otwórz okno blokując obecne (ShowDialog)" Height="40" Click="ShowDialogButton_Click" />
    </StackPanel>
</Grid>
</Window>

```

MainWindow.xaml.cs: (Główne okno C#)

```

using System;
using System.Windows;

namespace WieleOkien
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void ShowButton_Click(object sender, RoutedEventArgs e)
            {
                string tekst = InputTextBox.Text;
                WindowSzczegoly okno = new WindowSzczegoly(tekst);
                // Show() pozwala wciąż klikać i używać okna głównego
                okno.Show();
            }

            private void ShowDialogButton_Click(object sender, RoutedEventArgs e)
            {
                string tekst = InputTextBox.Text;
                WindowSzczegoly okno = new WindowSzczegoly(tekst);
                // ShowDialog() blokuje całą aplikację do czasu zamknięcia nowego okna
                okno.ShowDialog();
            }
        }
    }
}

```

Kluczowe elementy:

- **Przekazywanie danych przez konstruktor** — w klasie drugiego okna deklarujesz konstruktor przyjmujący parametry, dzięki czemu po użyciu słowa **new** możesz tam przestać dane z MainWindow.
- **Show()** — otwiera drugie okno obok głównego. Użytkownik może wracać z powrotem do okna głównego.
- **ShowDialog()** — otwiera okno typu "Dialog". Użytkownik nie może kliknąć w okno główne, dopóki nie zamknie okna dialogowego. Używane w formularzach lub komunikatach o błędach.

27.2. UserControl

W tej sekcji zobaczysz, jak tworzyć mniejsze ekrany komponentów (tzw. kontrolki użytkownika), by uprościć kod w głównym oknie. Zamiast pisać 1000 linii w MainWindow, robisz mniejsze klocki z użyciem klasy **UserControl**.

MojaKontrolka.xaml: (To jest plik UserControl, a nie Window)

```

<UserControl x:Class="NawigacjaUC.MojaKontrolka"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    DesignHeight="200" DesignWidth="300">

```



```

<Grid Background="LightYellow">
    <StackPanel Margin="10">
        <TextBlock Text="Jestem komponentem!" FontSize="18" FontWeight="Bold"/>
        <TextBox Margin="0,10" Text="Można mnie osadzić w innym miejscu"/>
        <Button Content="Kliknij" Click="AkcjaButton_Click"/>
    </StackPanel>
</Grid>
</UserControl>

```

MojaKontrolka.xaml.cs:

```

using System.Windows;
using System.Windows.Controls;

namespace NawigacjaUC
{
    public partial class MojaKontrolka : UserControl
    {
        public MojaKontrolka()
        {
            InitializeComponent();
        }

        private void AkcjaButton_Click(object sender, RoutedEventArgs e)
        {
            MessageBox.Show("Kliknięto przycisk wewnątrz UserControl!");
        }
    }
}

```

MainWindow.xaml:

```

<Window x:Class="NawigacjaUC.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:lokalne="clr-namespace:NawigacjaUC"
        Title="Osadzanie UserControl" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <TextBlock Text="Główne okno aplikacji" FontSize="24" Margin="0,0,0,20"/>

            <!-- Osadzenie stworzonego komponentu! Musimy dodać przestrzeń nazw 'lokalne' -->
            <lokalne:MojaKontrolka />

        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace NawigacjaUC
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {

```

```

        InitializeComponent();
    }
}
}

```

Kluczowe elementy:

- **UserControl** — element podobny do okna, ale nie mający ramki. Służy do osadzania w oknach i łączenia wielu kontroltek w jeden logiczny moduł.
- **Dodatkowa przestrzeń nazw** `xmlns:lokalne="clr-namespace:NazwaProjektu"` — by móc dodać element w XAML, kompilator musi wiedzieć, gdzie znajdzie definicję własnego komponentu.
- **Osadzenie** `<lokalne:MojaKontrolka/>` — nasz skomplikowany komponent używamy w innych plikach XAML tak samo prosto jak zwykły TextBox.

27.3. Content

W tej sekcji potoczmy UserControl i mechanizm **ContentControl**, by stworzyć nowoczesną, jednoonkieniową nawigację, spotykaną chociażby w aplikacjach wizualnych (zmiana widoku bez otwierania nowych okien).

Założmy, że mamy stworzone dwa **UserControl**: **WidokLogowania** i **WidokDashboard**.

MainWindow.xaml:

```

<Window x:Class="NawigacjaContentControl.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Nawigacja na jednym oknie" Height="450" Width="800">
    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="200" /> <!-- Pasek boczny menu -->
            <ColumnDefinition Width="*" /> <!-- Główny obszar contentu -->
        </Grid.ColumnDefinitions>

        <!-- Pasek boczny -->
        <StackPanel Grid.Column="0" Background="DarkGray">
            <Button Content="Pokaż logowanie" Height="40" Margin="10" Click="LogowanieButton_Click"/>
            <Button Content="Pokaż panel" Height="40" Margin="10" Click="PanelButton_Click"/>
        </StackPanel>

        <!-- Tutaj będziemy wstrzykiwać widoki! -->
        <ContentControl x:Name="GłównyObszar" Grid.Column="1" Margin="20" />
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace NawigacjaContentControl
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            // Ustawienie początkowego widoku (założmy że posiadamy ten UserControl)
            GłównyObszar.Content = new WidokLogowania();
        }

        private void LogowanieButton_Click(object sender, RoutedEventArgs e)
        {
            // Podmiana zawartości ekranu na kontrolkę logowania
            GłównyObszar.Content = new WidokLogowania();
        }
    }
}

```

```

    }

    private void PanelButton_Click(object sender, RoutedEventArgs e)
    {
        // Podmiana zawartości ekranu na kontrolkę dashboardu
        GłównyObszar.Content = new WidokDashboard();
    }
}

// Proste fałszywe klasy by kod był bez błędów w kompilacji
// W praktyce to byłyby osobne pliki XAML z UserControl
public class WidokLogowania : System.Windows.Controls.UserControl { }
public class WidokDashboard : System.Windows.Controls.UserControl { }
}

```

Kluczowe elementy:

- **ContentControl** — dynamiczny kontener potrafiący wyświetlić wszystko. To ramka na inne kontrolki użytkownika.
- **Podmiana wartości z kodu C#** — zmieniamy właściwość **Content** odwołując się do nowych obiektów, przez co cały widok zmienia się w mgnieniu oka, mimo że nie zamknęliśmy ani nie otworzyliśmy żadnego okna systemu Windows.

27.4. Widok startowy, roboczy i wynikowy

W tej sekcji zobaczysz alternatywną metodę nawigacji wykorzystywaną najczęściej w prostych i jednorazowych aplikacjach (np. quiz, instalator) - chowanie i wyświetlanie całych grup interfejsu (StackPanel) za pomocą właściwości **Visibility**.

MainWindow.xaml:

```

<Window x:Class="NawigacjaVisibility.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Stan aplikacji" Height="450" Width="800">
    <Grid>
        <!-- EKRAN STARTOWY -->
        <StackPanel x:Name="EkranStartowy" VerticalAlignment="Center" HorizontalAlignment="Center" Visibility="Visible">
            <TextBlock Text="Witaj w Quizie!" FontSize="30" Margin="0,0,0,20"/>
            <Button Content="Rozpocznij" Width="200" Height="50" FontSize="18" Click="RozpocznijButton_Click"/>
        </StackPanel>

        <!-- EKRAN ROBOCZY -->
        <StackPanel x:Name="EkranRoboczy" VerticalAlignment="Center" HorizontalAlignment="Center" Visibility="Collapsed">
            <TextBlock Text="Pytanie: Jak nazywa się język stylowania okien w WPF?" FontSize="20" Margin="0,0,0,20"/>
            <Button Content="HTML" Width="200" Height="40" Margin="0,5" Click="ZlaOdpowiedz_Click"/>
            <Button Content="XAML" Width="200" Height="40" Margin="0,5" Click="DobraOdpowiedz_Click"/>
        </StackPanel>

        <!-- EKRAN WYNIKOWY -->
        <StackPanel x:Name="EkranWynikowy" VerticalAlignment="Center" HorizontalAlignment="Center" Visibility="Collapsed">
            <TextBlock Text="Koniec gry!" FontSize="30" Margin="0,0,0,20"/>
            <TextBlock x:Name="TekstWyniku" FontSize="20" Margin="0,0,0,20"/>
            <Button Content="Graj ponownie" Width="200" Height="50" Click="ZagrajPonownie_Click"/>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace NawigacjaVisibility
{

```

```

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();

        // Metoda pomocnicza do ukrywania wszystkich ekranów
        private void UkryjWszystko()
        {
            EkranStartowy.Visibility = Visibility.Collapsed;
            EkranRoboczy.Visibility = Visibility.Collapsed;
            EkranWynikowy.Visibility = Visibility.Collapsed;
        }

        private void RozpocznijButton_Click(object sender, RoutedEventArgs e)
        {
            UkryjWszystko();
            EkranRoboczy.Visibility = Visibility.Visible;
        }

        private void ZlaOdpowiedz_Click(object sender, RoutedEventArgs e)
        {
            PokazWynik(false);
        }

        private void DobraOdpowiedz_Click(object sender, RoutedEventArgs e)
        {
            PokazWynik(true);
        }

        private void PokazWynik(bool czyDobra)
        {
            UkryjWszystko();
            EkranWynikowy.Visibility = Visibility.Visible;

            if (czyDobra)
                TekstWyniku.Text = "Super! Dobra odpowiedź.";
            else
                TekstWyniku.Text = "Niestety, to zła odpowiedź.";
        }

        private void ZagrajPonownie_Click(object sender, RoutedEventArgs e)
        {
            UkryjWszystko();
            EkranStartowy.Visibility = Visibility.Visible;
        }
    }
}

```

Kluczowe elementy:

- **Nakładające się ekrany w XAML** — wszystkie trzy widoki znajdują się fizycznie w tym samym oknie na tej samej pozycji w obiekcie Grid.
- `Visibility.Collapsed` — w przeciwieństwie do `Hidden`, kontrolka zwinięta jako `Collapsed` nie zajmuje w ogóle miejsca, co pozwala na pełne nałożenie nowego ekranu na stary.
- **Zarządzanie stanem w C#** — manipulując widocznością paneli w zdarzeniach tworzysz wrażenie nawigacji krok po kroku.

28. Algorytmy w aplikacjach WPF

28.1. Wzorec: odczyt kontrolki → logika → aktualizacja UI — schemat każdej obsługi zdarzenia

W tej sekcji zaprezentujemy uniwersalny sposób pisania kodu obsługi każdego zdarzenia (np. kliknięcia). Częstym błędem jest robienie skomplikowanych obliczeń bezpośrednio na tekście z kontrolki w jednej linii. Dobrą praktyką jest rygorystyczny podział na trzy logiczne bloki.

MainWindow.xaml:

```
<Window x:Class="WzorceDobrePraktyki.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Trójetapowy schemat" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <TextBlock Text="Podaj promień koła:" />
            <TextBox x:Name="PromienTextBox" Width="200" Margin="0,5,0,20"/>

            <Button Content="Oblicz pole" Click="ObliczButton_Click" Width="200" Height="30"/>

            <TextBlock x:Name="WynikTextBlock" FontSize="20" FontWeight="Bold" Margin="0,20,0,0"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace WzorceDobrePraktyki
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void ObliczButton_Click(object sender, RoutedEventArgs e)
            {
                // ETAP 1: Odczyt i parsowanie danych z UI do zmiennych C#
                string wpisanyTekst = PromienTextBox.Text;
                if (string.IsNullOrEmpty(wpisanyTekst))
                {
                    MessageBox.Show("Pole jest puste!");
                    return;
                }

                if (!double.TryParse(wpisanyTekst, out double promien))
                {
                    MessageBox.Show("Podano nieprawidłową liczbę!");
                    return;
                }

                // ETAP 2: Logika obliczeniowa operująca wyłącznie na czystych zmiennych w C#
                double poleKoła = Math.PI * Math.Pow(promien, 2);
                double zaokrąglonePole = Math.Round(poleKoła, 2);

                // ETAP 3: Aktualizacja interfejsu (wypchnięcie gotowych danych do kontrolki UI)
                WynikTextBlock.Text = $"Pole koła wynosi: {zaokrąglonePole} cm²";
            }
        }
    }
}
```

Kluczowe elementy:

- **Etap 1: Pobranie** — czytujesz `.Text` do zmiennych, walidujesz dane (np. `TryParse`). Robisz to najwcześniej jak to możliwe.
- **Etap 2: Logika** — właściwe liczenie, korzystające tylko z wyodrębnionych przed chwilą zmiennych (takich jak `double promien`). Żadnych odwołań do interfejsu (UI).
- **Etap 3: Zwrócenie wyniku** — na samym końcu wpisujesz obliczony i sformatowany wynik do wyznaczonej kontrolki (tutaj `WynikTextBlock`).

28.2. Przechowywanie stanu w polach klasy okna — `private fields`, kiedy i jak

W tej sekcji zobaczysz, jak utrzymać stan aplikacji w pamięci. Czasami wynik jednej operacji musimy zapamiętać, by móc skorzystać z niego w kolejnym zdarzeniu kliknięcia. Służą do tego pola prywatne tworzone na poziomie całej klasy okna (tzw. "globalne" dla danej klasy).

MainWindow.xaml:

```
<Window x:Class="StanAplikacji.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Przechowywanie stanu" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <TextBlock x:Name="KomunikatTextBlock" Text="Stan konta: 0 PLN" FontSize="24"/>
            <Button Content="Wpłać 10 PLN" Click="WplacButton_Click" Width="150" Height="40" Margin="0,20,0,10"/>
            <Button Content="Wypłać wszystko" Click="WyplacButton_Click" Width="150" Height="40"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace StanAplikacji
{
    public partial class MainWindow : Window
    {
        // Pole klasy - przechowuje stan (pamięta wartość pomiędzy kliknięciami)
        private decimal stanKonta = 0;

        public MainWindow()
        {
            InitializeComponent();
        }

        private void WplacButton_Click(object sender, RoutedEventArgs e)
        {
            // Modyfikujemy pole, które nie znika po zakończeniu tej metody
            stanKonta += 10m;
            OdswiezWidok();
        }

        private void WyplacButton_Click(object sender, RoutedEventArgs e)
        {
            MessageBox.Show($"Wypłacono {stanKonta} PLN!");
            stanKonta = 0;
            OdswiezWidok();
        }

        private void OdswiezWidok()
        {
            KomunikatTextBlock.Text = $"Stan konta: {stanKonta} PLN";
        }
    }
}
```

```
}  
  
}
```

Kluczowe elementy:

- **Zasięg pola prywatnego** — pole `stankonta` zdefiniowano na samym początku klasy. Dzięki temu zmienna "żyje" tak długo jak otwarte jest okno.
- **Niebezpieczeństwo** — zmienne tworzone lokalnie wewnątrz metody `Button_Click` giną w momencie zakończenia się metody. Nie zachowują wyniku na przyszłość.

28.3. Loaded

W tej sekcji zobaczysz miejsce do wykonywania czasochłonnego przygotowywania danych (wczytywanie z pliku, połączenie z bazą). Najlepszym do tego miejscem jest wbudowane w każde okno zdarzenie `Loaded`, które wykonuje się, gdy okno z powodzeniem wyświetli się użytkownikowi.

MainWindow.xaml:

```
<!-- Wpisujemy zdarzenie Loaded bezpośrednio do znacznika Window -->  
<Window x:Class="ZdarzenieLoaded.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Loaded="Window_Loaded"  
    Title="Inicjalizacja" Height="450" Width="800">  
    <Grid>  
        <StackPanel Margin="20">  
            <ListBox x:Name="WczytaneElementyListBox" Height="200" />  
            <TextBlock x:Name="StatusTextBlock" Margin="0,10,0,0" FontStyle="Italic"/>  
        </StackPanel>  
    </Grid>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.Collections.Generic;  
using System.Windows;  
using System.IO;  
  
namespace ZdarzenieLoaded  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
            // UWAGA: Tu w konstruktorze unikajmy otwierania plików!  
            // Jeśli coś wybuchnie, aplikacja nie włączy się wcale.  
        }  
  
        // To zdarzenie wywoła się na ułamek sekundy po tym jak użytkownik zobaczy puste okno.  
        private void Window_Loaded(object sender, RoutedEventArgs e)  
        {  
            StatusTextBlock.Text = "Rozpoczynam ładowanie danych...";  
  
            try  
            {  
                // Symulacja np. długotrwałego pobierania zawartości pliku tekstowego  
                List<string> linie = new List<string> { "Plik wczytany 1", "Plik wczytany 2", "Plik wczytany 3" };  
  
                // Przypisanie danych po wczytaniu  
                WczytaneElementyListBox.ItemsSource = linie;  
                StatusTextBlock.Text = "Załadowano pomyślnie.";  
            }  
        }  
    }  
}
```

```

        catch (Exception ex)
        {
            StatusTextBlock.Text = "Błąd: " + ex.Message;
        }
    }
}
}
}

```

Kluczowe elementy:

- **Zdarzenie `Loaded` okna** — jest dedykowanym i najbardziej odpowiednim momentem z życia okna dla wszelkich zapytań o dane początkowe.
- **Separacja konstruktora** — w konstruktorze `MainWindow()` pozostawiaj tylko `InitializeComponent()` lub proste ustawienia startowe. Długie operacje mogą spowodować, że białe, niezbudowane okno pojawi się dopiero po kilku sekundach.

28.4. Obsługa wyjątków — try/catch w zdarzeniach, co łapać, co logować

W tej sekcji zobaczysz, jak poprawnie łapać sytuacje, w których coś idzie nie tak, aby aplikacja z niebieskim oknem "program przestał odpowiadać" się nie zamykała awaryjnie. Wykorzystamy bloki `try..catch` bezpośrednio w punktach, które mogą generować błędy.

MainWindow.xaml:

```

<Window x:Class="TryCatchWpf.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Bezpieczny kod" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <TextBox x:Name="SciezkaTextBox" Text="C:\WymyslonaSciezka\plik.txt" Width="300" Margin="0,0,0,10"/>
            <Button Content="Przeczytaj i wyświetl zawartość" Click="CzytajPlik_Click" Width="200" Height="40"/>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.IO;
using System.Windows;

namespace TryCatchWpf
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void CzytajPlik_Click(object sender, RoutedEventArgs e)
        {
            string path = SciezkaTextBox.Text;

            // Używamy bloku try, jeśli kod uderza do zewnętrznego systemu, dysku, pliku lub sieci.
            try
            {
                string zawartosc = File.ReadAllText(path);
                MessageBox.Show("Udało się odczytać: " + zawartosc);
            }
            catch (FileNotFoundException ex)
            {
                // łapanie specjalistyczne: obsługa faktu, że brakuje pliku
                MessageBox.Show("Wybrany plik nie istnieje!\nSzczegóły: " + ex.Message, "Błąd z plikiem", MessageBoxButton.OK, MessageBoxImage.Warning);
            }
        }
    }
}

```



```

    }
    catch (Exception ex)
    {
        // łapanie uniwersalne: wyłapie np. brak uprawnień (UnauthorizedAccessException) itp.
        MessageBox.Show("Wystąpił niespodziewany błąd: " + ex.Message, "Błąd Krytyczny", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
}
}
}
}

```

Kluczowe elementy:

- **Bezpieczeństwo operacji zewnętrznych** — każde użycie systemowych plików IO i konwersji należy obudować instrukcjami `try/catch`.
- **Użytkownikowi błąd przyjazny** — aplikacja wyświetla `MessageBox` z ikonką i zrozumiałą informacją (zamiast pokazywać angielski "Stack Trace" albo po prostu wyłączać się w mgnieniu oka).

28.5. Blokowanie przycisku — `IsEnabled = false` przed operacją, `true` po zakończeniu

W tej sekcji zapobiegiesz "klikalności". Kiedy aplikacja wykonuje obliczenia lub zapisuje plik, użytkownik często może z niecierpliwości kliknąć dany przycisk 5 razy, doprowadzając do zacięcia logiki czy nadpisania danych. Pokażemy rozwiązanie z odcinaniem interfejsu z użyciem asynchroniczności (`await Task.Delay`).

MainWindow.xaml:

```

<Window x:Class="BlokowanieInterfejsu.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Blokowanie UI" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20" HorizontalAlignment="Center">
            <Button x:Name="ZapiszButton" Content="Zapisz ogromną konfigurację"
                    Width="250" Height="50" Click="ZapiszButton_Click"/>
            <TextBlock x:Name="StatusTekst" Margin="0,10,0,0" FontSize="16"/>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Threading.Tasks; // Potrzebne do opóźnienia
using System.Windows;

namespace BlokowanieInterfejsu
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // Użycie słowa async, aby móc symulować długą pracę asynchronicznie (bez zamrożenia widoku)
            private async void ZapiszButton_Click(object sender, RoutedEventArgs e)
            {
                // ODCINAMY INTERFEJS
                ZapiszButton.IsEnabled = false;
                ZapiszButton.Content = "Przetwarzanie...";
                StatusTekst.Text = "Proszę czekać, trwa operacja zapisu...";

                try
                {

```

```

        // Symulacja długiej operacji wykonującej się 3 sekundy
        await Task.Delay(3000);

        // Gdy operacja się powiedzie
        StatusTekst.Text = "Zakończono powodzeniem!";
    }
    finally
    {
        // Blok finally gwarantuje wykonanie tego kodu nawet jak kod wywali błąd wyżej
        // ODBLOKOWUJEMY INTERFEJS
        ZapiszButton.IsEnabled = true;
        ZapiszButton.Content = "Zapisz ogromną konfigurację";
    }
}
}
}
}
}

```

Kluczowe elementy:

- **Blokada** `IsEnabled = false` — użytkownik widzi wizualnie "wyszarzony" przycisk i po prostu fizycznie nie może w niego naklikać.
- **Blok** `finally` — zawsze w tym bloku odblokowuj stan, by aplikacja nie utknęła ze zablokowanym widokiem w nieskończoność w wypadku wczesnego `return` lub napotkania błędu (wyjątku).

28.6. Czyszczenie formularza po akcji

W tej sekcji zobaczysz ostatnią dobrą praktykę, jaką jest poprawianie wrażeń z interfejsu (UX). Gdy np. formularz jest skutecznie przetworzony, wyodrębniamy metodę czyszczącą poszczególne elementy (przywracając ich pierwotny stan zerowy), aby użytkownik łatwiej wpisał kolejne dane.

MainWindow.xaml:

```

<Window x:Class="CzyszczenieFormularza.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Resetowanie formularza" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20" Width="300">
            <Label Content="Imię:" />
            <TextBox x:Name="ImieTextBox" Margin="0,0,0,10" />

            <Label Content="Nazwisko:" />
            <TextBox x:Name="NazwiskoTextBox" Margin="0,0,0,10" />

            <CheckBox x:Name="ZgodaCheckBox" Content="Akceptuję regulamin" Margin="0,0,0,20"/>

            <Button Content="Zarejestruj" Click="Zarejestruj_Click" Height="40" Margin="0,0,0,10"/>
            <Button Content="Wyczyść ręcznie" Click="WyczyscButton_Click" Height="30" Background="Salmon"/>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace CzyszczenieFormularza
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

```

    }

    private void Zarejestruj_Click(object sender, RoutedEventArgs e)
    {
        // Zwykła walidacja (pominięta dla czytelności)
        MessageBox.Show("Użytkownik zarejestrowany!");

        // Po pomyślnej rejestracji nie chcemy by dane dalej widniały na ekranie.
        ResetujFormularz();
    }

    private void WyczyszcButton_Click(object sender, RoutedEventArgs e)
    {
        // Możemy do tej metody wywołać wielokrotnie.
        ResetujFormularz();
    }

    // Metoda która przywraca wszystkie kontrolki do ustawień domyślnych
    private void ResetujFormularz()
    {
        ImieTextBox.Text = string.Empty; // lub pole.Clear();
        NazwiskoTextBox.Text = string.Empty;
        ZgodaCheckBox.IsChecked = false;
    }
}
}

```

Kluczowe elementy:

- **Użycie** `string.Empty` — bezpieczny i elegancki sposób wyczyszczenia pól tekstowych z zawartości. Ustawia na `""`.
- **Centralizacja kodu** — zamknięcie czyszczenia w dedykowanej prywatnej metodzie oznacza, że możesz wyczyścić formularz przy każdym anulowaniu, ponownej rejestracji i po poprawnym dodaniu – bez powtarzania kodu.

29. Porządkowanie kodu

29.1. TextBlock

W tej sekcji poznasz profesjonalną konwencję nazewnictwa dla okien WPF, która uchroni cię przed bałaganem. Częstym błędem nowicjuszy jest zostawianie kontroltek jak `textBox1`, `button3`, co przy większej aplikacji doprowadza do zgubienia się we własnym kodzie. Złota zasada to dopisywanie typu w postfixie (na końcu nazwy).

MainWindow.xaml:

```

<Window x:Class="NazewnictwoWPF.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Konwencje nazw" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20">
            <!-- Właściwe podejście: CoToRobi + TypKontrolki -->
            <TextBox x:Name="ImieWpisTextBox" Width="200" Margin="5"/>
            <TextBox x:Name="WiekWpisTextBox" Width="200" Margin="5"/>

            <Button x:Name="ZapiszDaneButton" Content="Zapisz" Width="200" Margin="5" Click="ZapiszDaneButton_Click"/>

            <!-- Konwencja działa w podpowiadaniu kodu.
                 Wiesz z góry, że to kontrolka etykiety (TextBlock), a nie wpisywania. -->
            <TextBlock x:Name="PodsumowanieWynikTextBlock" Foreground="Green"/>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace NazewnictwoWPF
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void ZapiszDaneButton_Click(object sender, RoutedEventArgs e)
            {
                // Kod używający wspólnych nazw zmiennych
                string imie = ImieWpisTextBox.Text;
                PodsumowanieWynikTextBlock.Text = $"Zapisano: {imie}";
            }
        }
    }
}
```

Kluczowe elementy:

- **PascalCase (Duże Litery)** — standardowy sposób nadawania nazw polom `x:Name`. Początek nazwy musi mówić, do czego to służy.
- **Typ kontrolki (Suffix)** — słowo `TextBox`, `Button`, `ListView` przyklejamy ZAWSZE na koniec nazwy. Zapewnia to natychmiastowe rozpoznanie właściwości tego pola z poziomu kodowania w C#.

29.2. Nazewnictwo metod

W tej sekcji omówimy standardy pisania samych metod dla C#. C# i platforma .NET posiadają sztywny standard określania funkcji za pomocą czasowników z wielkiej litery (PascalCase), co gwarantuje łatwe odróżnienie metody od pola lub lokalnej zmiennej.

MainWindow.xaml:

```
<Window x:Class="MetodyPrzyklad.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Metody" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="20" HorizontalAlignment="Center">
            <Button Content="Wykonaj Złożoną Logikę" Click="OdpalKalkulacjeButton_Click" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace MetodyPrzyklad
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void OdpalKalkulacjeButton_Click(object sender, RoutedEventArgs e)
```

```

{
    // Nazwy metod mówią wyraźnie, jakie wykonują zadanie (czasownik)
    PrzygotujKolekcjeStartowa();
    int wartoscZobliczen = PrzeliczZarobkiDlaZadanegoMiesiaca(5);
    ZaktualizujEkranPoObliczeniach(wartoscZobliczen);
}

// PascalCase w Metodach
private void PrzygotujKolekcjeStartowa()
{
    // Logika przygotowania
}

private int PrzeliczZarobkiDlaZadanegoMiesiaca(int indeksMiesiaca)
{
    // Parametry w nazwach używają formatu camelCase!
    int wynikowyBudzet = indeksMiesiaca * 1000;
    return wynikowyBudzet;
}

private void ZaktualizujEkranPoObliczeniach(int koncowyWynik)
{
    // Kod ui
    MessageBox.Show("Wynik końcowy: " + koncowyWynik);
}
}
}

```

Kluczowe elementy:

- **Czasowniki** — Metody opisują czynności. "Policz", "Odswiez", "Utworz", "Uruchom".
- **PascalCase w C#** — metody i nazwy klas w technologii C# zaczynają się z Dużej litery, inaczej niż np. w Java czy JavaScript.
- **Nazwy parametrów i lokalnych zmiennych** — wewnątrz metod zachowujemy camelCase (zaczynamy małą literą, kolejne słowa dużą) - `koncowyWynik`, `wartoscZobliczen`.

29.3. Button

W tej sekcji zajmiemy się bardzo niebezpiecznym nawykiem i zaprezentujemy jego rozwiązanie. Często metody `Button_Click` rosną do rozmiarów 150 linii i wykonują wszystko jednocześnie (zapytania bazodanowe, ukrywanie kontrolek, formatowanie liczb). Zdarzenie kliknięcia powinno z definicji jedynie "koordynować", a nie pracować fizycznie na logice.

MainWindow.xaml:

```

<Window x:Class="LogikaSeparacja.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Oddzielanie logiki" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30">
            <TextBox x:Name="WprowadzonyCiagTextBox" Text="ABBA123" Margin="0,0,0,10"/>
            <Button Content="Analizuj Tekst" Click="AnalizujButton_Click" Height="40"/>
            <TextBlock x:Name="WyjscieAnalizyTextBlock" Margin="0,20,0,0" FontWeight="Bold"/>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Linq;
using System.Windows;

```

```

namespace LogikaSeparacja
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // ZDARZENIE (Event Handler) - SŁUŻY TYLKO DO KOORDYNACJI
            private void AnalizujButton_Click(object sender, RoutedEventArgs e)
            {
                string tekst = WprowadzonyCiągTextBox.Text;

                // Delegowanie ciężkiego kodu do wydzielonej funkcji poniżej
                string rezultat = AnalizatorZnakowy.ZnajdzWszystkieSamogloskiICyfr(tekst);

                WyjścieAnalizyTextBlock.Text = rezultat;
            }
        }

        // Dodatkowa klasa wewnątrz tego samego namespace trzymająca zawitą logikę aplikacji.
        // Dzięki temu Window jest czyste, a my możemy użyć ponownie tej samej metody z dowolnego miejsca.
        public static class AnalizatorZnakowy
        {
            // Ta metoda nic nie wie o przyciskach i oknach, jest czystym kodem C#.
            public static string ZnajdzWszystkieSamogloskiICyfr(string wejście)
            {
                char[] samogloskiZestaw = { 'A', 'E', 'I', 'O', 'U', 'Y', 'a', 'e', 'i', 'o', 'u', 'y' };
                int ileSamoglosek = wejście.Count(c => samogloskiZestaw.Contains(c));
                int ileCyfr = wejście.Count(char.IsDigit);

                return $"Znaleziono samogłosek: {ileSamoglosek}, Cyfr: {ileCyfr}";
            }
        }
    }
}

```

Kluczowe elementy:

- **Delegowanie zadania** — Button_Click pobiera ciąg, woła metodę i przypisuje wynik. O niczym nie wie, jest głupią warstwą widoku. To świetna zaleta.
- **Reużywalność** — wydzielona logika i wsadzona do innej metody jest odcięta od widoku. Jeśli stworzysz nową sekcję, w której analizę tekstu uruchamiasz po wciśnięciu klawisza "Enter" z klawiatury (bez klikania w przycisk), odwołasz się dokładnie do tego samego miejsca bez dublowania logiki.

29.4. Generator haseł

W tej sekcji zobaczysz, jak umieszczać poszczególne zadania systemu w wyspecjalizowanych, osobnych klasach pomocniczych. Tworzenie klas z końcówkami np. `Service`, `Manager`, `Helper`, czy `Generator` to serce inżynierii C#, nazywane od strony architektonicznej "Rozdziałem Obowiązków" (Separation of Concerns).

Plik SzyfrCezaraHelper.cs: (Osobny plik .cs utworzony w projekcie)

```

using System.Text;

namespace StrukturyKlasowe
{
    // Czysta klasa narzędziowa C#.
    // Odpowiada za robienie wyłącznie JEDNEJ RZECZY: szyfrowania algorytmem Cezara.
    public class SzyfrCezaraHelper
    {
        public static string Zakoduj(string wpisaneSłowo, int przesuniecie = 3)
        {
            StringBuilder wynik = new StringBuilder();
            foreach (char literka in wpisaneSłowo)
            {
                // Logika szyfrowania
            }
        }
    }
}

```

```

        {
            if (char.IsLetter(literka))
            {
                char baza = char.IsUpper(literka) ? 'A' : 'a';
                char nowaLitera = (char)((((literka + przesuniecie) - baza) % 26) + baza);
                wynik.Append(nowaLitera);
            }
            else
            {
                wynik.Append(literka);
            }
        }
        return wynik.ToString();
    }
}
}

```

MainWindow.xaml:

```

<Window x:Class="StrukturyKlasowe.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Klasy Pomocnicze" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30">
            <TextBox x:Name="OryginalTextBox" Text="TajnaWiadomosc!" Width="300" Margin="0,0,0,10"/>
            <Button Content="Wykonaj Szyfr" Click="SzyfrujButton_Click" Width="200" Height="40"/>
            <TextBox x:Name="SzyfrTextBox" Width="300" Margin="0,10,0,0" IsReadOnly="True" />
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace StrukturyKlasowe
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void SzyfrujButton_Click(object sender, RoutedEventArgs e)
            {
                string danyWpis = OryginalTextBox.Text;

                // Okno nie zawiera w sobie logiki liczenia modulo 26 na tablicy znaków ASCII.
                // Okno bierze w pełni działający klocek i korzysta z niego wpisując go poniżej.
                string tajneHaslo = SzyfrCezaraHelper.Zakoduj(danyWpis);

                SzyfrTextBox.Text = tajneHaslo;
            }
        }
    }
}

```

Kluczowe elementy:

- **Tworzenie klas w osobnych plikach** — aby utrzymać porządek na drzewie projektu.
- **Metody i Klasy Statyczne** — ponieważ klasa jest tylko zbiorem narzędzi ułatwiających liczenie wzorów, zazwyczaj możemy określić funkcje słowem `static`. Umożliwia to zjawisko, gdzie w oknie głównym odwołujemy się do niej `SzyfrCezaraHelper.Zakoduj()`, bez robienia przed każdym wywołaniem `new SzyfrCezaraHelper()`.

29.5. Komentarze i czytelność kodu — kiedy pisać komentarz, kiedy kod mówi sam za siebie

W tej sekcji omówimy komentowanie. Częstym błędem jest komentowanie absolutnie każdej linii kodu. Powoduje to chaos wizualny (znany pod zjawiskiem zanieczyszczenia kodu). Pokażemy, co znaczy, że poprawnie zaprojektowany i nazwany kod stanowi najlepszy komentarz.

MainWindow.xaml:

```
<Window x:Class="CzytelNoscKodu.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Komentowanie kodu" Height="450" Width="800">
    <Grid>
        <TextBlock Text="Przykład tylko do odczytu kodu C#" Margin="20"/>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace CzytelNoscKodu
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // PRZYKŁAD ZŁY: Kod brzydki obłożony banalnymi komentarzami.
            // Komentarz wyjaśnia CO się dzieje, zamiast DLACZEGO.
            private void ZlePodejscie()
            {
                // tworzy zmienna n
                int n = 18;

                // sprawdza czy n wieksze rowne od osiemnastu
                if (n ≥ 18)
                {
                    // zwraca true
                    bool res = true;
                }
            }

            // PRZYKŁAD DOBRY: Kod napisany poprawnie (Zmienna to nie "n", tylko "wiekUzytkownika").
            // Zła nazwa metody zmieniona. Taki kod absolutnie nie wymaga dokumentacji.
            private void DobrePodejscie()
            {
                int wiekUzytkownika = 18;
                int minimalnyWiek = 18;

                bool czyJestPelnoletni = wiekUzytkownika ≥ minimalnyWiek;
            }

            // PRZYKŁAD ŚWIETNY: Jedyny prawidłowy sposób użycia komentarzy jest w miejscach "zawitych" algorytmów
            // by wyjaśnić nietypowe zagrania lub zasady biznesowe ("DLACZEGO", a nie "CO")
        }
    }
}
```



```
private decimal ObliczRabatySystemowe(decimal kosztCalkowity, int ileLatSledziFirme)
{
    decimal wynikowyKoszt = kosztCalkowity;

    // UWAGA DLA NOWYCH PRACOWNIKÓW: Poniższe opóźnienie i specyficzne 2.5 raza mnożnika
    // wynika ze statutu prawnego narzuconego na spółki w maju 2021 roku.
    // Proszę pod żadnym pozorem nie zmieniać tej logiki.
    if (ileLatSledziFirme > 5)
    {
        wynikowyKoszt -= (kosztCalkowity * 0.1m) * 2.5m;
    }

    return wynikowyKoszt;
}
}
```

Kluczowe elementy:

- **Unikaj komentarzy typu "inicjalizacja zmiennej"** — dobry kod języka czyta się podobnie do angielskiego artykułu. Uczciwie stosowanie dobrych nazw niszczy w 90% potrzebę używania znaków `//`.
- **Używaj komentarzy do celów zarządczych** — komentarze są idealnym miejscem dla zostawienia sobie i innym członkom projektu znaczników `//TODO:` (do zrobienia na później), czy `//FIXME:` (do naprawienia później).
- **Zostawiaj wyjaśnienie powodów** — jeśli z użyciem regexów obchodzisz dziwny błąd kontrolki zewnętrznej — opisz w komentarzu ten nietypowy, omijający problem proces — bo na pierwszy rzut oka nikt nie będzie wiedział, o co chodzi.

30. Debugowanie i typowe błędy

30.1. Podgląd zmiennych

W tej sekcji poznasz narzędzia Visual Studio służące do kontrolowania i wstrzymywania kodu krok po kroku w trakcie jego prawdziwego życia. Pozwala to zaglądać do środka pętli i czytać stany kontroltek bez seryjnego "używania MessageBox" jako testera wartości.

MainWindow.xaml:

```
<Window x:Class="DebugowanieAplikacji.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Jak debugować w C#" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30">
            <Button Content="Uruchom skomplikowaną logikę" Click="OdpalKrok_Click" Height="40" Width="300" />
            <TextBlock x:Name="WynikTestuTextBlock" Margin="0,10,0,0" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace DebugowanieAplikacji
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

```

private void OdpalKrok_Click(object sender, RoutedEventArgs e)
{
    int wynikA = 5;
    int wynikB = 10;

    // Aby wstrzymać program w tym miejscu podczas pracy, wciśnij F9, ustawiając czerwoną kropkę (Breakpoint).
    // Uruchamiając przyciskiem Start lub klawiszem F5 i klikając przycisk na okienku, program zatrzyma się właśnie w linii z wywołaniem metody.
    int sumaPoObliczeniach = MojeTajemniczeDodawanie(wynikA, wynikB);

    WynikTestuTextBlock.Text = sumaPoObliczeniach.ToString();
}

private int MojeTajemniczeDodawanie(int a, int b)
{
    // Będąc wstrzymanym u góry, jeśli wciśniesz F11 (Krok z Wejściem - Step Into), wejdiesz fizycznie i wizualnie do tej metody.
    // Z kolei jeśli wciśnąłbyś F10, przeszedłbyś do kolejnej linii na gorze, całkowicie omijając wizualne wchodzenie tutaj w dół.
    int tymczasowa = a + b;
    tymczasowa += 100; // Po zatrzymaniu się tutaj i najechaniu myszką na słowo "tymczasowa", zobaczysz aktualną wartość.
    return tymczasowa;
}
}
}

```

Kluczowe elementy:

- **Breakpoint (F9)** — zwany też punktem przerwania. To czerwona kulka tworzona zazwyczaj poprzez kliknięcie na lewy skraj marginesu kodu w Visual Studio, określająca "gdzie ma się zatrzymać symulacja".
- **Rozpoczęcie debugowania (F5)** — Start, który aktywnie kompiluje kod i łączy go do maszyny analizującej, sprawiając, że czerwone kropki zadziałają.
- **Krok i Krok z wejściem (F10 / F11)** — gdy kod "wisi" wstrzymany na czerwonej kropce i chcemy przenieść się powoli na absolutnie kolejną linię i patrzeć co psuje wynik – używamy F10 (Step Over). Jeśli jest to złożona metoda i chcemy "zagłębić" się do jej wnętrza, wybieramy F11 (Step Into).

30.2. Debugowanie XAML — Output window, brak zamknięcia znacznika, literówka w x:Name, brak metody zdarzenia

W tej sekcji omówimy debugowanie XAML. O ile C# wyrzuca czytelny komunikat na czerwono po złej kompilacji, błędy w języku XAML bywają skryte, albo wybuchają niepokazując linii pochodzenia (bo XAML parsuje się i tworzy obiekty na podstawie węzłów). Poniżej poznasz trzy główne usterki.

MainWindow.xaml:

```

<!-- BŁĄD XAML 1: literówki lub zapomnienie o zamknięciu wewnątrz bloków.
Jeżeli usuniesz symbol zamykający elementu na samym dole Grid'a </Grid>,
VS rzuci błąd: "End tag is missing." -->

<Window x:Class="BledyXAML.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Jak debugować w XAML" Height="450" Width="800">
    <Grid>
        <!-- BŁĄD XAML 2: Literówka w x:Name.
        Jeśli piszesz po XAML z wciśniętym CapsLockiem i wpiszesz <TextBox x:nAME="Test"/>
        kod w XAML wskaże czerwonym "wężykiem", że taki element we wbudowanych schematach przestrzeni nie istnieje! -->

        <!-- BŁĄD XAML 3: Zdarzenie w XAML, którego fizycznie nie ma w pliku .cs.
        Najczęstszy błąd początkujących. Jeśli poniżej dopiszesz Click="JakisMojKlik",
        a potem usuniesz tę metodę w pliku MainWindow.xaml.cs,
        system nie przepuści cię przez etap kompilacji okna i wywali srogim wierszem błędu. -->

        <Button x:Name="BezpiecznyButton" Content="Nic nie robię" Margin="20"/>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace BledyXAML
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // Zauważ że jeśli ktoś stąd usunął wcześniej przypięte do xaml'owego okna zdarzenie kliknięcia,
            // to cała kompilacja rzuci okropnym komunikatem "Does not contain a definition for...".
        }
    }
}
```

Kluczowe elementy:

- **Output Window / Error List** — narzędzie (często na samym dole środowiska), do którego należy od razu zerknąć, gdy zobaczysz czerwoną linię wężyka pod jakąkolwiek z form znaczników w okienku graficznym XAML.
- **Rozbieżność podpisów do zdarzeń (Events)** — jest to relacja podtrzymywana ze strachu w plikach WPF. Gdy do kontrolki Button powiązałeś `Click="Button_Kliknij_Go_Raz"`, nie wolno ci w klasie C# swobodnie decydować o zmianie tej nazwy. Obie te rzeczy powiązane są mocno węzłem. Zmiana jednej strony wymusza absolutnie dostosowanie po drugiej.

30.3. Brak x:Name

W tej sekcji zobaczysz błąd w kodzie działający od strony systemu, gdy nie podłączysz poprawnego odwołania x:Name kontrolki. Słynny przypadek to brak nadania nazwy interfejsowi w XAML po czym na beczelnego uderzanie w nią w kodzie C#, czego skutkiem jest najsłynniejszy i najgorszy z błędów - zrzut z komunikatem "Object reference not set to an instance of an object".

MainWindow.xaml:

```
<Window x:Class="BladZReferencja.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Błędy Reference" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30">
            <!-- Gdy na początku zapomniemy po chamsku dodać atrybutu x:Name i zostawimy:
            <TextBlock Text="Czekam na wynik"/>
            To aplikacja w C# po prostu nam wyrzuci potężny błąd gdy będziemy próbowali się odwołać. -->

            <TextBlock x:Name="WielkiEkranTextBlock" Text="Dobry znacznik posiada x:Name" Foreground="Green"/>
            <Button Content="Uderzaj w TextBlock" Click="Aktywator_Click" Margin="0,20,0,0" Height="40"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace BladZReferencja
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

```

        // Pamiętaj! Próba ustawienia wartości zanim zdąży się wykonać InitializeComponent
        // natychmiast wyrzuci słynne wysmienite NullReferenceException na twarz!
        // Bo XAML nie miał kiedy przetworzyć wszystkich "cegieł".
    }

    private void Aktywator_Click(object sender, RoutedEventArgs e)
    {
        // Ponieważ u góry nazwaliśmy <TextBlock x:Name="WielkiEkranTextBlock" /> obiekt prawidłowo,
        // to po wpisaniu poniższego C# połączy sobie powiązania pamięci.
        // Gdyby nazwa ta nie istniała w XAML to kompilator od razu by nam to podkreślił że "WielkiEkranTextBlock nie istnieje w tym kontekście".
        WielkiEkranTextBlock.Text = "Oto zmieniłem tekst prawidłowo - bez rzucania Null Reference i wieszania aplikacji na czerwono!";
    }
}
}
}

```

Kluczowe elementy:

- **Błąd** `NullReferenceException` — w najprostszych słowach: "Zmienna, w której chciałeś wywołać jakąś właściwość (np. `.Text`) albo modyfikację, wynosi dosłownie pustkę (null) i jej nie ma, więc nie możesz po niej wejść jak do pudełka!".
- **Rola** `InitializeComponent()` — ta jedna jedyna linijka pod maską odpowiada w WPF za stworzenie całego drzewa elementów XAML. Jak tylko ją usuniesz lub skasujesz — żadna kontrolka okna nie zostanie powołana do życia fizycznego, a na kliknięcie czegokolwiek aplikacja wystrzeli potężnym błędem o niedostępności obiektów.

30.4. Błędy z plikami — `FileNotFoundException`, ścieżka względna vs absolutna, Debug vs Release

W tej sekcji zajmiemy się omówieniem poszukiwania plików. Ścieżki na dysku to klasyczne punkty załamania dla niedoświadczonych w aplikacji WPF, doprowadzające do niekontrolowanego komunikatu `FileNotFoundException`. Pokażemy różnicę między plikiem zbadanym bezwzględnie w Windows a sposobem podróżywania w katalogach "od pliku egzekucyjnego" (względnie).

MainWindow.xaml:

```

<Window x:Class="CzytanieBledy.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Poszukiwania Plików" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30">
            <Button Content="Uruchom Wczytywanie Ścieżka Absolutna" Click="ŚcieżkaTwarda_Click" Height="40" Margin="5"/>
            <Button Content="Uruchom Wczytywanie Ścieżka Względna" Click="ŚcieżkaOdniesienia_Click" Height="40" Margin="5"/>
            <TextBlock x:Name="RaportZPliku" Margin="0,20,0,0" FontWeight="Bold"/>
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.IO;
using System.Windows;

namespace CzytanieBledy
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void ŚcieżkaTwarda_Click(object sender, RoutedEventArgs e)
        {

```

```

// TO JEST ZŁA PRAKTYKA (Zabójstwo w projekcie grupowym)
// Używamy TWARDEJ i SZTYWNEJ (Absolutnej) ścieżki.
// Spowoduje to u innej osoby natychmiastowe wyrzucenie błędu FileNotFoundException, ponieważ ona nie ma na swoim komputerze pendrive'a K:\ !
string absolutna = @"K:\ProjektWPF\mojDokumentTajnaBaza.txt";

try
{
    string tresc = File.ReadAllText(absolutna);
    RaportZPliku.Text = tresc;
}
catch (Exception)
{
    MessageBox.Show("Wywaliło aplikację na innym komputerze z racji użycia absolutnego 'K:' na wejściu dysku!");
}
}

private void SciezkaOdniesienia_Click(object sender, RoutedEventArgs e)
{
    // TO JEST DOBRA PRAKTYKA - Ścieżka WZGLĘDNA
    // Pobierana jest tak zwana ścieżka od miejsca, z którego Uruchomiono aplikację exe!
    // Gdziekolwiek by projekt nie wędrował po komputerach, folder wyjściowy programu wskaże to dynamicznie.
    string sciezkaOkołoBieznegoFolderu = "mojDokumentWypadowy.txt";

    try
    {
        // To wywoła się na lokalnym folderze np: bin/Debug/net6.0-windows/mojDokumentWypadowy.txt
        string tresc = File.ReadAllText(sciezkaOkołoBieznegoFolderu);
        RaportZPliku.Text = tresc;
    }
    catch (FileNotFoundException)
    {
        MessageBox.Show("Szukałem tylko w lokalnym folderze z aplikacją .exe, ale też nie znalazłem pliku. Ale tym razem nikomu nie zepsujemy ścieżki literą K! Upewnij się czy we właściwościach pliku pliku ustawiłeś 'Copy if newer / Copy to output directory' w środowisku.");
    }
}
}
}

```

Kluczowe elementy:

- **Ścieżka Względna** — odniesienie do czegoś poprzez określanie lokalizacji np. `folder0bok\dane.json`. Sposób polegający na tym, że plik ma towarzyszyć naszej budowanej maszynie lub aplikacji exe. Pamiętaj tylko o funkcji we właściwościach na boku menu w VS — "Copy To Output Directory" by pliki były przenoszone automatycznie podczas naciśnięcia klawisza kompilacji F5.
- **Ścieżka Absolutna** — sztywne zapisanie całego schematu dysku na piechotę `C:\Katalog\Gra\Zapis.txt`. Taki sposób jest groźny dla przenoszalności oprogramowania. Zawsze kończy się w mrozącym trybie FileNotFoundException i awarią środowiska po wymianie repozytorium do znajomych, czy oddania produktu na płycie do klienta.

30.5. List

W tej sekcji powiemy sobie o błędach w logice kodu C#. Są najtrudniejszą formą błędu – nie wywalają się podczas kompilacji na czerwono. Kod wpisano syntaktycznie, program działa, po czym podczas wciskania akcji i generowania logiki przez maszynę okazuje się, że obliczył źle pozycje, tablice lub podał kosmicznie nieprzewidzianą kombinację wyjścia poza skalę danych, wywalając się do pulpitu z hukiem!

MainWindow.xaml:

```

<Window x:Class="LogiczneTypowe.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Logiczne Problemy" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30">
            <Button Content="Zniszcz aplikację po wyjściu poza zakres tablicy" Click="Niszczyciel_Click" Height="40" Margin="10"/>
            <Button Content="Uruchom bezpieczne i poprawnie" Click="Obronca_Click" Height="40" Margin="10"/>

```

```
</StackPanel>

</Grid>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Collections.Generic;
using System.Windows;

namespace LogiczneTypowe
{
    public partial class MainWindow : Window
    {
        private List<string> owoce = new List<string>() { "Jabłko", "Banan", "Gruszka" };

        public MainWindow()
        {
            InitializeComponent();
        }

        private void Niszczyciel_Click(object sender, RoutedEventArgs e)
        {
            // Typowy Błąd z Logiki dla młodych twórców (tzw. zły indeks - IndexOutOfRangeException)
            // Ustaliliśmy, że u góry utworzyliśmy element o długości = 3 elementom (Indeksy to: 0, 1 i 2).

            // Program w momencie naciśnięcia przycisku natychmiastowo wejdzie pod próbę wyświetlenia pola pod indeksem 3 (czyli dla ludzkiego oka
            // pozycja czwarta!), a takowa po prostu nie istnieje. System odpowie krytycznym zamknięciem gry.

            try
            {
                MessageBox.Show(owoce[3]);
            }
            catch (Exception ex)
            {
                MessageBox.Show("Boom, spaliłeś układ bo wyszedłeś w nieokreśloną przestrzeń obok!\n " + ex.Message);
            }
        }

        private void Obronca_Click(object sender, RoutedEventArgs e)
        {
            // Poprawne zapytanie i logika uwzględniająca długość zestawów, by zawsze być gotowym by wyświetlić coś na spokojnie bez usterki.
            int potencjalnyIndeks = 2; // (Indeks Gruszki)

            if (potencjalnyIndeks >= 0 && potencjalnyIndeks < owoce.Count)
            {
                MessageBox.Show("Wybrano bezpiecznie i pomyślnie owoc z zestawu to: " + owoce[potencjalnyIndeks]);
            }
            else
            {
                MessageBox.Show("Ups! Logiczne wezwanie pokazało że wybrany obiekt uderzyłby poza listę - zatrzymałem wykonanie by bronić cię!");
            }
        }
    }
}
```

Kluczowe elementy:

- **Problemy z zakresem list (IndexOutOfRangeException)** — zawsze i absolutnie pilnuj, by nie wzywać elementów po numerach (indeksach), które są równe lub przewyższają liczbę elementów zestawu, do którego sięga dana pętla albo wybór.
- **Problem nieskończonych cykli/Pustych znaków** — powszechny błąd logiczny "Zawieszenia w nieskończoności", gdy stworzona pętla `while` nigdy nie dojdzie do zakończenia. To błąd logiczny, system go skompiluje. Dopiero po użyciu w WPF wciśniesz guzik, system zatnie się, i zniknie. Zawsze pisz kod

na miarę – testuj stany wyjścia i bezpieczniki.

31. Testowanie logiki aplikacji

31.1. Debug i Release

W tej sekcji zajmiemy się finałowym trybem przygotowywania i optymalizacji aplikacji po długotrwałym kodowaniu i testowaniu, używając dostępnych trybów na samej górze środowiska IDE – zazwyczaj znanym trójkątem pod przyciskiem budowy obok paska z nazwą (np. "Debug", "Any CPU").

MainWindow.xaml:

```
<Window x:Class="TrybyKompilacji.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Tryby Debug i Release" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30" HorizontalAlignment="Center" VerticalAlignment="Center">
            <TextBlock Text="Tryb pod którym aktualnie skompilowano widać w zmiennych warunkowych środowiska, oto przykład poniżej C#:"
                      FontSize="18" TextWrapping="Wrap" TextAlignment="Center"/>
            <Button Content="Uruchom kod i pokaż Tryb z logiką systemu pre-procesorów"
                    Width="400" Height="50" Margin="0,30,0,0" Click="Zbadaj_Click"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace TrybyKompilacji
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void Zbadaj_Click(object sender, RoutedEventArgs e)
        {
            // #if DEBUG to specjalny pre-procesor dyrektyw w C#,
            // Który decyduje co znajdzie się w wypuszczonej finalnie maszynowej budowie aplikacji
            // w zależności od wybranego trybu rozwijanej belki w Visual Studio na samej górze (tam przy guziku F5 do gry)!

            #if DEBUG
                MessageBox.Show("Jesteś na trybie DEBUG!\nPlik exe i biblioteki mają nieobcięty wielki rozmiar, masz dostęp do śledzenia kropkami kodu, i aplikacja chodzi nieco wolniej po to, żebyś mógł znaleźć ukryte usterki podczas bycia twórcą.", "Analiza", MessageBoxButton.OK, MessageBoxImage.Information);
            #else
                MessageBox.Show("Jesteś na trybie RELEASE (Publikacyjnym).\nTwój plik wejściowy .exe został ostro zoptymalizowany przez kompilator, działa najszybciej jak może u klienta, a ty by go zobaczyć odpiąłeś debugowanie ze strony okienkowych środowisk! Dajemy ci szybką, szczupłą aplikację w prezencie dla klienta docelowego.", "Analiza", MessageBoxButton.OK, MessageBoxImage.Warning);
            #endif
        }
    }
}
```

Kluczowe elementy:

- **Tryb Debug** — zawiera ciężkie symbole (tzw. pliki `.pdb`), nie ucina wolnego i nieużywanego kodu, działa zauważalnie wolniej – bo nastawiony jest na współpracę ze środowiskiem byś mógł z pomocą F9 szukać usterek "w podziemnym prześwicie z prześwietleniem pacjenta". Tworzony podczas procesu ciągłej edycji.

- **Tryb Release** — agresywny silnik kompilacji okrawa wszystko, wycisza środowisko testowe i wypłuka odcięty szybki twór finalnego EXE, który wysyłasz uciśniętemu na rynkach zewnętrznych. Powinieneś korzystać w chwili eksportu całości do gotowego produktu.

31.2. Budowanie aplikacji — Build Solution (Ctrl+Shift+B), gdzie powstają pliki .exe

W tej sekcji dowiemy się, w którym momencie kod zamienia się na fizyczny plik w formacie "programu wykonywalnego" (tzw. exe w systemach windows) oraz jak odnaleźć uciekającą po zbudowaniu do folderów zawartość z włączonym logiem C#.

MainWindow.xaml:

```
<Window x:Class="BudowanieApki.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Budowanie" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30">
            <TextBlock Text="Zbadanie zawartości o tym Gdzie znajduje się program egzekucyjny w środowisku:"
                       FontSize="20" FontWeight="Black"/>
            <Button Content="Ustal Lokację na twardo" Click="Lokacja_Click" Width="300" Height="50" Margin="0,20,0,0"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.IO;
using System.Reflection;
using System.Windows;

namespace BudowanieApki
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void Lokacja_Click(object sender, RoutedEventArgs e)
        {
            // Sposób na sprawdzenie, do którego folderu zbudował się główny rdzeń twojego wpisywanego kodu.
            string gdzieJestemNaDyskuWKomputerze = Assembly.GetEntryAssembly().Location;
            MessageBox.Show("Środowisko wrzuciło plik wykonywalny zbudowanego WPF w miejsce:\n\n" + gdzieJestemNaDyskuWKomputerze);
        }
    }
}
```

Kluczowe elementy:

- **Opcja Build (Ctrl+Shift+B)** — kompiluje okna w tle do zaplecza bez okna "uruchamiania z F5 z wielkim wczytywaniem". System w dolnym panelu wpisze na pomarańczowo, czy program przeszedł przez parsowanie.
- **Miejsce tworzenia wynikowego pliku** — zazwyczaj na dysku docelowo pliki wyjściowe dla wprawnego C# to foldery typu `<KatalogProjektu>\bin\Debug\net6.0-windows\` albo w przypadku wyżej omawianego i gotowego release `<KatalogProjektu>\bin\Release\net6.0-windows\`. Tam właśnie znajduje się aplikacja pod nazwą "TwojaApka.exe" wypłuta na dysk przez kowala od okien.

31.3. Publikowanie folderowe

W tej sekcji przeprowadzisz ostateczny proces pakowania dla odbiorców zewnętrznych, wykorzystując do tego opcję masową – moduł "Publish". Jest to procedura, w której nie kopiujemy ręcznie plików z podziemi (jak pokazano wyżej w folderze bin), a tworzymy profesjonalną, docelową i odciętą ścieżkę generowanego ostatecznego produktu.

MainWindow.xaml:


```
<Window x:Class="PublikowanieBindow.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zdalne Publikowanie" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30" HorizontalAlignment="Center">
            <TextBlock Text="Aby spublishować wejdź prawym w projekt Solution i podaj opcję Publish..."
                        FontSize="16" Foreground="Teal" FontWeight="Bold"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace PublikowanieBindow
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Kluczowe elementy:

- **Publish - Folder** — wybierając w narzędziach projektowych opcję rzutu masz do wyboru tryb stworzenia kompletnej i odizolowanej od deweloperskich plików konfiguracji wersji na docelowy, obcy folder.
- **Czystość wydania z Publish** — w porównaniu do prostego ręcznego przenoszenia `.exe` – tutaj aplikacja kompiluje i sprawdza absolutnie puste wyeliminowanie wszystkich pliczeków .pdb (jeśli na Release ustawimy), czy też dociąga wszystkie potrzebne paczki zewnętrzne by wszystko ruszyło gładko i bez upadków dla klienta po rozpakowaniu.

31.4. Aplikacja zależna od runtime

W ostatniej sekcji zajmiemy się najnowszą z plag od Microsoftu – doбором ciężkości. Musimy dokonać finalnego, mistrzowskiego wyboru konfiguracji publikacji. Obojętnie jaki proces podjąłeś, C# stawia Cię zawsze na wielkiej drodze – co chcesz zrobić z frameworkiem na którym zbudowano środowisko i w jaki sposób plik uruchomi klient obcy.

MainWindow.xaml:

```
<Window x:Class="FinalTrybRelease.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Wersja Zależności C#" Height="450" Width="800">
    <Grid>
        <StackPanel Margin="30">
            <TextBlock Text="Różnice w rozwijanej konfiguracji Publish Settings przed zatwierdzeniem." FontWeight="Bold"/>

            <TextBlock Text="Wbudowane Self-Contained:" Margin="0,15,0,0" Foreground="Red" FontSize="18"/>
            <TextBlock Text="Użytkownik nie musi pobierać ani wiedzieć co to jest 'platforma .NET' po swojej stronie systemu na PC! Pakujesz we wielkim rozmiarze cały silnik maszyny .NETa w sam folder z grą, wysyłając im gigantyczne ilości MB w paczce .ZIP. Idealne gdy piszesz w szkole do sprawdzenia po sieci, dla osób o nieznannej specyfice komputerowej środowiska, np w zamkniętym systemie księgowości." TextWrapping="Wrap" />

            <TextBlock Text="Framework-Dependent (Zależne od Runtime):" Margin="0,15,0,0" Foreground="Blue" FontSize="18"/>
            <TextBlock Text="Cieszysz się z małej objętości (aplikacja waży 300 Kilobajtów). Wypływasz tylko goły szkielet, by okno C# się odpaliło... ale to wymaga, aby pan X, którego namierzasz zainstalować z przeglądarki oficjalny moduł instalacyjny np: '.NET 6.0 Desktop Runtime'. Bez tego ujrzy puste i żałosne pole do pobierania modułów zewnętrznych." TextWrapping="Wrap" />
        </StackPanel>
    </Grid>
</Window>
```

```
</StackPanel>

</Grid>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace FinalTrybRelease
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Kluczowe elementy:

- **Framework-Dependent** (Zależne) — mały rozmiar, ufa, że obcy system Windows ma wgrane biblioteki środowiska .NET.
- **Self-Contained** (Samowystarczalne) — duży ciężar w MB, dołącza tysiące podziemnych ddl-ek Microsoftowych, przez co aplikacja uruchomi się na pustym, totalnie czystym i zresetowanym dzisiaj Windowsie u każdego. To pewniak kompilacji na produkcję o dużym rozmiarze, ale bez proszenia o wejście w tryb pobierania silników.

32. INotifyPropertyChanged i ObservableCollection

32.1. Po co widokowi powiadomienia o zmianach

W aplikacjach WPF interfejs użytkownika (czyli widok) jest połączony z danymi za pomocą mechanizmu Bindingu. Domyślnie, jeśli zmienisz wartość właściwości w kodzie C#, widok w ogóle o tym nie wie. Mechanizm Bindingu potrzebuje wyraźnego sygnału, że dane uległy zmianie i trzeba odświeżyć ekran. Bez powiadomień aplikacja będzie wyglądała na zamrożoną lub ignorującą akcje użytkownika. Poniższy przykład pokazuje problem – kliknięcie przycisku zmienia dane, ale tekst na ekranie pozostaje bez zmian.

MainWindow.xaml:

```
<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Brak powiadomień" Height="200" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="{Binding UserName}" FontSize="20" Margin="0,0,0,10"/>
        <Button Content="Zmień imię" Click="Button_Click" Padding="10"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace WpfApp
{
    public partial class MainWindow : Window
    {
        public string UserName { get; set; } = "Jan Kowalski";

        public MainWindow()
```

```

    {
        InitializeComponent();
        DataContext = this;
    }

    private void Button_Click(object sender, RoutedEventArgs e)
    {
        // Zmieniamy wartość w pamięci, ale widok o tym nie wie!
        UserName = "Anna Nowak";
    }
}
}

```

- **UserName** – Zwykła właściwość, która przechowuje tekst.
- **DataContext** – Kontekst danych ustawiony na klasę okna.
- **Brak odświeżenia** – Przycisk zmienia właściwość, ale TextBlock nie aktualizuje swojego tekstu.

32.2. Interfejs INotifyPropertyChanged

Aby rozwiązać problem braku odświeżania, musisz zaimplementować interfejs `INotifyPropertyChanged`. Pochodzi on z przestrzeni nazw `System.ComponentModel`. Posiada on tylko jeden element – zdarzenie `PropertyChanged`. Kiedy właściwość się zmienia, wywołujesz to zdarzenie, a WPF automatycznie reaguje i odświeża widok.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Interfejs INotifyPropertyChanged" Height="200" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="{Binding UserName}" FontSize="20" Margin="0,0,0,10"/>
        <Button Content="Zmień imię" Click="Button_Click" Padding="10"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.ComponentModel;
using System.Windows;

namespace WpfApp
{
    public partial class MainWindow : Window, INotifyPropertyChanged
    {
        private string _userName = "Jan Kowalski";

        public string UserName
        {
            get { return _userName; }
            set
            {
                _userName = value;
                // Zgłaszamy zmianę!
                PropertyChanged?.Invoke(this, new PropertyChangedEventArgs("UserName"));
            }
        }

        public event PropertyChangedEventHandler PropertyChanged;

        public MainWindow()

```

```

    {
        InitializeComponent();
        DataContext = this;
    }

    private void Button_Click(object sender, RoutedEventArgs e)
    {
        UserName = "Anna Nowak"; // Teraz widok się zaktualizuje
    }
}
}

```

- **INotifyPropertyChanged** – Interfejs dodany do klasy `MainWindow`.
- **PropertyChanged** – Zdarzenie wymagane przez interfejs.
- **PropertyChangedEventArgs** – Obiekt przenoszący nazwę właściwości, która uległa zmianie.

32.3. Metoda OnPropertyChanged

Wpisywanie nazwy właściwości jako tekstu (np. `"UserName"`) jest podatne na błędy (literówki). Lepiej jest utworzyć uniwersalną metodę np. `OnPropertyChanged`, która wykorzystuje atrybut `[CallerMemberName]`. Dzięki niemu kompilator sam domyśli się, która właściwość wywołała zmianę.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Metoda OnPropertyChanged" Height="250" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="{Binding Status}" FontSize="18" Foreground="Blue"/>
        <Button Content="Rozpocznij" Click="Button_Click" Margin="0,10,0,0" Padding="5"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;

namespace WpfApp
{
    public partial class MainWindow : Window, INotifyPropertyChanged
    {
        private string _status = "Gotowy";

        public string Status
        {
            get { return _status; }
            set
            {
                _status = value;
                OnPropertyChanged(); // Nie podajemy nazwy ręcznie
            }
        }

        public event PropertyChangedEventHandler PropertyChanged;

        protected void OnPropertyChanged([CallerMemberName] string propertyName = null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
        }
    }
}

```

```

    }

    public MainWindow()
    {
        InitializeComponent();
        DataContext = this;
    }

    private void Button_Click(object sender, RoutedEventArgs e)
    {
        Status = "Przetwarzanie danych...";
    }
}
}

```

- **[CallerMemberName]** – Magiczny atrybut z `System.Runtime.CompilerServices`.
- **OnPropertyChanged** – Hermetyzuje wywołanie zdarzenia `PropertyChanged`.
- **Brak stringów** – Unikasz problemów w przypadku zmiany nazwy właściwości.

32.4. Aktualizacja pojedynczych właściwości

Kiedy stworzysz osobne klasy dla swoich danych, każda z nich powinna implementować `INotifyPropertyChanged`. W ten sposób oddzielasz kod interfejsu (widoku) od logiki danych. Tworzysz klasę np. `Person`, a w oknie jedynie podpinasz jej instancję do `DataContext`.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Aktualizacja pojedynczych właściwości" Height="250" Width="400">
    <StackPanel Margin="20">
        <Label Content="Imię:" />
        <TextBox Text="{Binding FirstName, UpdateSourceTrigger=PropertyChanged}" />

        <Label Content="Nazwisko:" Margin="0,10,0,0" />
        <TextBox Text="{Binding LastName, UpdateSourceTrigger=PropertyChanged}" />

        <TextBlock Text="{Binding FullName}" FontSize="20" FontWeight="Bold" Margin="0,20,0,0"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;

namespace WpfApp
{
    public class Person : INotifyPropertyChanged
    {
        private string _firstName = "Adam";
        private string _lastName = "Kowalski";

        public string FirstName
        {
            get { return _firstName; }
            set
            {
                _firstName = value;
            }
        }
    }
}

```

```

        OnPropertyChanged();
        OnPropertyChanged(nameof(FullName)); // Aktualizacja zależnej właściwości
    }
}

public string LastName
{
    get { return _lastName; }
    set
    {
        _lastName = value;
        OnPropertyChanged();
        OnPropertyChanged(nameof(FullName)); // Aktualizacja zależnej właściwości
    }
}

public string FullName => $"{FirstName} {LastName}";

public event PropertyChangedEventHandler PropertyChanged;
protected void OnPropertyChanged([CallerMemberName] string name = null)
{
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}
}

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        DataContext = new Person();
    }
}
}

```

- **Zależne właściwości** – Kiedy zmieniasz `FirstName`, musisz powiadomić też o zmianie `FullName`.
- **UpdateSourceTrigger=PropertyChanged** – Formularz odświeża właściwość wraz z każdym wciśniętym klawiszem w TextBox.
- **nameof** – Operator języka C#, który zwraca nazwę właściwości jako bezpieczny tekst.

32.5. Aktualizacja list przez ObservableCollection

Standardowa lista `List<T>` nie potrafi powiadomić widoku o tym, że dodano lub usunięto jej element. Oznacza to, że po dodaniu obiektu do Listy w kodzie C#, kontrolka np. `ListBox` czy `DataGrid` go nie wyświetli. Aby rozwiązać ten problem, używaj kolekcji `ObservableCollection<T>`.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ObservableCollection" Height="300" Width="400">
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>

        <ListBox ItemsSource="{Binding Users}" DisplayMemberPath="Name" />

        <Button Grid.Row="1" Content="Dodaj użytkownika" Click="Button_Click" Margin="0,10,0,0"/>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```
using System;
using System.Collections.ObjectModel;
using System.Windows;

namespace WpfApp
{
    public class User
    {
        public string Name { get; set; }
    }

    public partial class MainWindow : Window
    {
        public ObservableCollection<User> Users { get; set; }

        public MainWindow()
        {
            InitializeComponent();
            Users = new ObservableCollection<User>
            {
                new User { Name = "Marek" },
                new User { Name = "Zofia" }
            };
            DataContext = this;
        }

        private void Button_Click(object sender, RoutedEventArgs e)
        {
            // ListBox od razu zobaczy ten nowy element
            Users.Add(new User { Name = "Nowy użytkownik " + DateTime.Now.Second });
        }
    }
}
```

- **ObservableCollection** – Kolekcja, która automatycznie powiadamia widok o modyfikacjach swoich elementów.
- **Brak INotifyPropertyChanged w User** – Dodawanie do kolekcji działa, ale jeśli zmienisz imię pojedynczego użytkownika (np. `Users[0].Name = "Test"`), widok się nie zaktualizuje, dopóki klasa `User` nie wdroży `INotifyPropertyChanged`.
- **Wyświetlanie** – Mechanizm `DisplayMemberPath="Name"` w `ListBox` mówi, którą właściwość obiektu wyświetlić.

32.6. Typowe błędy przy braku powiadomień

Początkujący programiści często przypisują do właściwości zupełnie nową referencję do kolekcji i dziwią się, że widok nie odświeża listy. Należy zawsze modyfikować (dodawać, usuwać) istniejącą kolekcję `ObservableCollection`, albo zaimplementować `INotifyPropertyChanged` dla samej kolekcji. Poniższy przykład pokazuje, jak naprawić błąd z całkowitym zastępowaniem listy.

MainWindow.xaml:

```
<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Typowe błędy" Height="300" Width="400">
    <StackPanel Margin="10">
        <ListBox ItemsSource="{Binding Items}" Height="150" />
        <Button Content="Błędne załadowanie danych" Click="Wrong_Click" Margin="0,10,0,0"/>
        <Button Content="Poprawne załadowanie danych" Click="Correct_Click" Margin="0,10,0,0"/>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```

using System;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;

namespace WpfApp
{
    public partial class MainWindow : Window, INotifyPropertyChanged
    {
        private ObservableCollection<string> _items = new ObservableCollection<string>();

        public ObservableCollection<string> Items
        {
            get { return _items; }
            set
            {
                _items = value;
                OnPropertyChanged(); // Konieczne przy tworzeniu nowego obiektu ObservableCollection!
            }
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string name = null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
        }

        public MainWindow()
        {
            InitializeComponent();
            DataContext = this;
            Items.Add("Start");
        }

        private void Wrong_Click(object sender, RoutedEventArgs e)
        {
            // BŁĄD LOGICZNY: Jeśli usuniesz implementację INotifyPropertyChanged, to nowa referencja nie zostanie zauważona przez widok.
            // Items = new ObservableCollection<string> { "Jeden", "Dwa" };

            // Poprawne podejście jeśli nie masz modyfikatora set powiadamiającego o zmianie:
            Items.Clear();
            Items.Add("Jeden z Wrong_Click");
        }

        private void Correct_Click(object sender, RoutedEventArgs e)
        {
            // Gdy mamy zaimplementowany setter z OnPropertyChanged(), możemy bez obaw przypisać całkiem nową kolekcję.
            Items = new ObservableCollection<string> { "Nowy 1", "Nowy 2", "Nowy 3" };
        }
    }
}

```

- **Nadpisywanie kolekcji** – Przypisanie `new ObservableCollection` zadziała tylko wtedy, gdy cała właściwość posiada mechanizm powiadamiania o zmianie.
- **Clear() i Add()** – Bezpieczniejszym podejściem dla instancji widoku jest czyszczenie bieżącej listy i uzupełnianie jej nowymi elementami.
- **Ukryte błędy** – Zapomnienie dodania `DataContext = this` to również częsty powód, dla którego powiadomienia nie dochodzą do elementów interfejsu.

33. Konwertery danych

33.1. Po co stosować konwertery

Często typ danych, który posiadasz w C#, nie odpowiada typowi potrzebnemu przez kontrolki WPF. Przykładowo, w kodzie posiadasz zmienną logiczną `bool IsActive`, a kontrolka na ekranie potrzebuje typu `Visibility` (np. `Visibility.Visible` lub `Visibility.Collapsed`), aby się ukryć. Zamiast zanieczyszczać model danych właściwościami WPF, stosujesz Konwertery (Converters). Tłumaczą one w locie jeden typ danych na inny.

MainWindow.xaml:

```
<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Po co konwertery" Height="200" Width="400">
    <StackPanel Margin="20">
        <!-- Zwykły checkbox powiązany z typem logicznym -->
        <CheckBox x:Name="SettingsCheckbox" Content="Pokaż opcje zaawansowane" IsChecked="True" />

        <!-- Tu przydałby się konwerter, bo IsChecked (bool) nie pasuje bezpośrednio do Visibility! -->
        <Button Content="Ustawienia bazy danych" Margin="0,20,0,0" Height="40" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace WpfApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            // Ten kod uświadamia problem: interfejs potrzebuje widoczności, a my dysponujemy true/false.
            // Konwerter rozwiąże to w czystym XAMLu.
        }
    }
}
```

- **Rozdział logiki** – Dane pozostają niezależne od wizualizacji.
- **Brak zaśmiecania klas** – Twoje obiekty używają standardowych typów jak `bool`, `int`, a konwertery zajmują się wizualną otoczką na poziomie widoku.

33.2. IValueConverter

Podstawowym interfejsem dla konwerterów w WPF jest `IValueConverter`. Wymaga on implementacji dwóch metod: `Convert` (z kodu do widoku) i `ConvertBack` (z widoku do kodu, wykorzystywane przy bindingu dwukierunkowym). Często metoda `ConvertBack` wyrzuca wyjątek `NotImplementedException`, jeśli nie potrzebujesz edycji.

MainWindow.xaml:

```
<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:WpfApp"
        Title="IValueConverter" Height="200" Width="400">

    <Window.Resources>
        <!-- Rejestracja konwertera w zasobach okna -->
        <local:YesNoConverter x:Key="YesNoConverter" />
    </Window.Resources>
```

```

<StackPanel Margin="20">
    <CheckBox IsChecked="{Binding IsMarried}" Content="Stan cywilny" Margin="0,0,0,10"/>

    <StackPanel Orientation="Horizontal">
        <TextBlock Text="Czy jest w związku małżeńskim: " />
        <!-- Użycie konwertera -->
        <TextBlock Text="{Binding IsMarried, Converter={StaticResource YesNoConverter}}" FontWeight="Bold" />
    </StackPanel>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Globalization;
using System.Windows;
using System.Windows.Data;

namespace WpfApp
{
    // Nasz własny konwerter bool → string
    public class YesNoConverter : IValueConverter
    {
        public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
        {
            if (value is bool isMarried)
            {
                return isMarried ? "Tak" : "Nie";
            }
            return "Brak danych";
        }

        public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
        {
            throw new NotImplementedException();
        }
    }

    public partial class MainWindow : Window
    {
        public bool IsMarried { get; set; } = true;

        public MainWindow()
        {
            InitializeComponent();
            DataContext = this;
        }
    }
}

```

- **Zasoby okna** – Aby użyć konwertera w XAML, musisz go dodać do bloku `Window.Resources` z odpowiednim kluczem `x:Key`.
- **Metoda Convert** – Odbiera obiekt typu `bool` i zwraca obiekt typu `string` ("Tak" / "Nie").
- **Binding parametru** – Słowo kluczowe `Converter={StaticResource Klucz}` wykonuje podmianę danych.

33.3. IMultiValueConverter

Gdy musisz powiązać jedną właściwość w kontrolce z wieloma zmiennymi z kodu, użyj `IMultiValueConverter`. Na przykład w sytuacji, gdy chcesz aktywować przycisk tylko wtedy, gdy imię i nazwisko zostały wpisane. Wykorzystuje się do tego `MultiBinding`.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:WpfApp"
        Title="IMultiValueConverter" Height="250" Width="400">
    <Window.Resources>
        <local:FullNameConverter x:Key="FullNameConverter" />
    </Window.Resources>

    <StackPanel Margin="20">
        <TextBox x:Name="FirstNameBox" Text="Jan" Margin="0,0,0,10" />
        <TextBox x:Name="LastNameBox" Text="Kowalski" Margin="0,0,0,10" />

        <TextBlock FontWeight="Bold" FontSize="18">
            <TextBlock.Text>
                <!-- MultiBinding przesyła kilka wartości do konwertera -->
                <MultiBinding Converter="{StaticResource FullNameConverter}">
                    <Binding ElementName="FirstNameBox" Path="Text" />
                    <Binding ElementName="LastNameBox" Path="Text" />
                </MultiBinding>
            </TextBlock.Text>
        </TextBlock>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Globalization;
using System.Windows;
using System.Windows.Data;

namespace WpfApp
{
    public class FullNameConverter : IMultiValueConverter
    {
        public object Convert(object[] values, Type targetType, object parameter, CultureInfo culture)
        {
            if (values.Length == 2)
            {
                string first = values[0]?.ToString() ?? "";
                string last = values[1]?.ToString() ?? "";
                return $"Witaj użytkowniku: {first} {last}";
            }
            return string.Empty;
        }

        public object[] ConvertBack(object value, Type[] targetTypes, object parameter, CultureInfo culture)
        {
            throw new NotImplementedException();
        }
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

```
}  
  
}
```

- **Tablica values** – Metoda `Convert` odbiera wszystkie powiązane zmienne pod postacią tablicy `object[]`.
- **MultiBinding** – Wymaga zastosowania specyficznego bloku kodu w XAML wewnątrz tagu np. `<TextBlock.Text>`.
- **Zabezpieczenia** – Zawsze weryfikuj długość tablicy `values` i dokonuj sprawdzenia `null`.

33.4. Konwersja bool -> Visibility

To najpopularniejszy konwerter, który często używa się podczas programowania interfejsów w WPF. WPF ma nawet gotowy konwerter o nazwie `BooleanToVisibilityConverter`, ale samodzielne jego napisanie daje możliwość tworzenia wariantów odwrotnych (odwrócona logika: `true` ukrywa element).

MainWindow.xaml:

```
<Window x:Class="WpfApp.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    xmlns:local="clr-namespace:WpfApp"  
    Title="Bool to Visibility" Height="250" Width="400">  
    <Window.Resources>  
        <local:InverseBoolToVisibilityConverter x:Key="InverseVisibilityConverter" />  
        <BooleanToVisibilityConverter x:Key="StandardVisibilityConverter" />  
    </Window.Resources>  
  
    <StackPanel Margin="20">  
        <CheckBox x:Name="LoadingCheck" Content="Trwa ładowanie" IsChecked="True" Margin="0,0,0,20"/>  
  
        <TextBlock Text="Pobieranie danych..." Foreground="Blue" FontWeight="Bold"  
            Visibility="{Binding ElementName=LoadingCheck, Path=IsChecked, Converter={StaticResource StandardVisibilityConverter}}" />  
  
        <Button Content="Główny panel aplikacji" Height="50" Margin="0,10,0,0"  
            Visibility="{Binding ElementName=LoadingCheck, Path=IsChecked, Converter={StaticResource InverseVisibilityConverter}}" />  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.Globalization;  
using System.Windows;  
using System.Windows.Data;  
  
namespace WpfApp  
{  
    public class InverseBoolToVisibilityConverter : IValueConverter  
    {  
        public object Convert(object value, Type targetType, object parameter, CultureInfo culture)  
        {  
            // Jeśli jest zaznaczone (true) - chowamy przycisk (Collapsed)  
            if (value is bool b && b)  
            {  
                return Visibility.Collapsed;  
            }  
            return Visibility.Visible;  
        }  
  
        public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)  
        {  
            throw new NotImplementedException();  
        }  
    }  
}
```

```

    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

- **BooleanToVisibilityConverter** – Wbudowany konwerter WPF. Trzeba go zadeklarować w **Resources**.
- **Inverse** – Odwrotny konwerter jest niezbędny, gdy dany stan powoduje zniknięcie kontrolki z ekranu.
- **Visibility.Collapsed** – Ukrywa element całkowicie, sprawiając, że przestaje on zajmować miejsce na ekranie (w przeciwieństwie do **Visibility.Hidden**).

33.5. Konwersja liczby na kolor

Konwertery to genialne narzędzie do wizualizacji progów punktowych (np. stan magazynowy poniżej zera ma być czerwony, a powyżej – zielony). Konwerter odbiera wartość liczbową i zwraca obiekt **SolidColorBrush**, z którego chętnie korzystają kontrolki WPF.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:WpfApp"
        Title="Number to Color" Height="250" Width="400">
    <Window.Resources>
        <local:StockLevelToColorConverter x:Key="StockColorConverter" />
    </Window.Resources>

    <StackPanel Margin="20">
        <Slider x:Name="StockSlider" Minimum="-50" Maximum="150" Value="20" Margin="0,0,0,20" />

        <TextBlock Text="Stan magazynu:" />
        <TextBlock Text="{Binding ElementName=StockSlider, Path=Value}"
                   FontSize="40" FontWeight="Bold"
                   Foreground="{Binding ElementName=StockSlider, Path=Value, Converter={StaticResource StockColorConverter}}" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Globalization;
using System.Windows;
using System.Windows.Data;
using System.Windows.Media;

namespace WpfApp
{
    public class StockLevelToColorConverter : IValueConverter
    {
        public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
        {
            if (value is double stock)
            {
                if (stock < 0)
                {
                    return Brushes.Red;
                }
                if (stock < 50)
                {
                    return Brushes.Orange;
                }
            }
        }
    }
}

```

```

        return Brushes.Green;
    }

    return Brushes.Black;
}

public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
{
    throw new NotImplementedException();
}
}

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
}
}

```

- **Brushes** – Gotowe palety kolorów udostępniane przez klasę `System.Windows.Media.Brushes`.
- **Skalowalność** – Logika progów kolorów i wartości ukryta jest w jednej małej klasie.
- **Powiązanie właściwości** – Tekst oraz jego kolor czytają z tego samego źródła (`Value` suwaka `Slider`).

33.6. Konwersja enum na tekst

Typy wyliczeniowe `enum` w C# najczęściej używają słów kluczowych (np. `OrderState.InProgress`), które nie nadają się do zaprezentowania użytkownikowi na ekranie. Konwerter enum→tekst rozwiązuje sprawę, przekształcając anglojęzyczne zmienne na czytelne dla polskiego użytkownika.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:WpfApp"
        Title="Enum to String" Height="200" Width="400">
    <Window.Resources>
        <local:OrderStateConverter x:Key="StateConverter" />
    </Window.Resources>

    <StackPanel Margin="20">
        <TextBlock Text="Status bieżącego zamówienia:" FontSize="16" />
        <TextBlock Text="{Binding CurrentState, Converter={StaticResource StateConverter}}"
                    FontSize="24" FontWeight="Bold" Margin="0,10,0,0" />

        <Button Content="Zmień status" Click="Button_Click" Margin="0,20,0,0" Padding="5"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.ComponentModel;
using System.Globalization;
using System.Runtime.CompilerServices;
using System.Windows;
using System.Windows.Data;

namespace WpfApp
{

```

```

public enum OrderState { Pending, InProgress, Completed }

public class OrderStateConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is OrderState state)
        {
            switch (state)
            {
                case OrderState.Pending: return "Oczekujące na płatność";
                case OrderState.InProgress: return "W trakcie realizacji";
                case OrderState.Completed: return "Zakończone";
            }
        }
        return "Nieznany status";
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}

public partial class MainWindow : Window, INotifyPropertyChanged
{
    private OrderState _currentState = OrderState.Pending;
    public OrderState CurrentState
    {
        get => _currentState;
        set { _currentState = value; OnPropertyChanged(); }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string name = null) =>
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));

    public MainWindow()
    {
        InitializeComponent();
        DataContext = this;
    }

    private void Button_Click(object sender, RoutedEventArgs e)
    {
        CurrentState = OrderState.InProgress; // Aktualizacja w wyliczeniu
    }
}

```

- **Tłumaczenie** – Konwerter skutecznie maskuje techniczne nazewnictwo wartości typów wyliczeniowych.
- **Rozszerzalność** – Dodanie nowego stanu zamówienia oznacza modyfikację zaledwie jednej metody `Convert` przy użyciu mechanizmu `switch`.
- **Czysty XAML** – Kontrolki na widoku nie zajmują się logiką sprawdzania warunków.

33.7. Parametry konwertera

Czasami chcesz sterować tym, jak konwerter zadziała na obiekty. Z pomocą przychodzi właściwość `ConverterParameter` z poziomu tagu `Binding`. Przesłany tekst lub zmienna trafiają jako obiekty do argumentu `parameter` w metodzie `Convert`.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

```

```

    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="clr-namespace:WpfApp"
    Title="Parametry konwertera" Height="250" Width="400">
<Window.Resources>
    <local:TextLengthConverter x:Key="LengthConverter" />
</Window.Resources>

<StackPanel Margin="20">
    <TextBox x:Name="InputBox" Text="Programowanie w WPF" Margin="0,0,0,20" />

    <TextBlock Text="Zwykły limit:" />
    <TextBlock Text="{Binding ElementName=InputBox, Path=Text, Converter={StaticResource LengthConverter}, ConverterParameter=10}"
        FontWeight="Bold" Margin="0,0,0,10" />

    <TextBlock Text="Duży limit:" />
    <TextBlock Text="{Binding ElementName=InputBox, Path=Text, Converter={StaticResource LengthConverter}, ConverterParameter=20}"
        FontWeight="Bold" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Globalization;
using System.Windows;
using System.Windows.Data;

namespace WpfApp
{
    public class TextLengthConverter : IValueConverter
    {
        public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
        {
            string text = value as string ?? "";

            // Pobieramy parametr konwertera
            if (parameter != null && int.TryParse(parameter.ToString(), out int maxLength))
            {
                if (text.Length > maxLength)
                {
                    return text.Substring(0, maxLength) + "...";
                }
            }
            return text;
        }

        public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
        {
            throw new NotImplementedException();
        }
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```


- **ConverterParameter** – Dodatkowy stały argument przesyłany z poziomu XAMLa do C#.
- **Reużywalność** – Otrzymujesz jeden konwerter ucinający teksty, ale potrafi on mieć różne zachowania dla różnych kontroltek w zależności od limitu.
- **Parsowanie** – Pamiętaj, by w kodzie sprawdzić lub rzutować typ `parameter`, zazwyczaj przesyłany jest on jako zwykły tekst (`string`).

34. Walidacja w bindingu

34.1. Walidacja ręczna a walidacja bindingowa

Walidacja ręczna polega na sprawdzaniu warunków przy kliknięciu w przycisk zapisu (np. `if (TextBox.Text == "")`). To starsza technika, która prowadzi do dużego chaosu w pliku okna i miesza logikę z wyglądem. Z kolei w WPF powiązania (Binding) posiadają wbudowane techniki walidacji odrzucające błędne wpisy i wizualizujące problemy od razu wokół pola tekstowego (czerwona ramka).

MainWindow.xaml:

```
<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Walidacja wstęp" Height="200" Width="400">
    <StackPanel Margin="20">
        <!-- Zwykła, ręczna weryfikacja pod przyciskiem - bez korzyści ze strony WPF -->
        <TextBox x:Name="ManualTextBox" Margin="0,0,0,10" />
        <Button Content="Sprawdź" Click="Button_Click" />
        <TextBlock x:Name="ErrorText" Foreground="Red" Margin="0,10,0,0" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace WpfApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void Button_Click(object sender, RoutedEventArgs e)
        {
            if (string.IsNullOrWhiteSpace(ManualTextBox.Text))
            {
                ErrorText.Text = "Pole nie może być puste!";
            }
            else
            {
                ErrorText.Text = "Wszystko w porządku.";
            }
        }
    }
}
```

- **Proceduralne sprawdzanie** – Powyższy styl pracy nie skaluje się przy wielkich, kilkunastopolowych formularzach.
- **Oczekiwanie na kliknięcie** – Użytkownik dowiaduje się o błędzie na samym końcu procesu zamiast w momencie opuszczania kontrolki.

34.2. ValidationRule

Mechanizm `ValidationRule` wykonuje sprawdzenia podczas przekazywania danych do zmiennych z poziomu widoku w XAML. Klasę sprawdzającą podpinasz bezpośrednio w sekcji `Binding.ValidationRules`. Jest to proste do użycia, kiedy sprawdzanie nie ma związku z modelem danych, a weryfikuje np. czystą zawartość znakową.

MainWindow.xaml:

```
<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:WpfApp"
        Title="Validation Rule" Height="250" Width="400">
    <StackPanel Margin="20">
        <Label Content="Wprowadź wiek:" />
        <TextBox>
            <TextBox.Text>
                <!-- Zastosowanie ValidationRule z parametrem natychmiastowego zgłaszania na starcie -->
                <Binding Path="Age" UpdateSourceTrigger="PropertyChanged" ValidatesOnDataErrors="True">
                    <Binding.ValidationRules>
                        <local:AgeValidationRule />
                    </Binding.ValidationRules>
                </Binding>
            </TextBox.Text>
        </TextBox>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Globalization;
using System.Windows;
using System.Windows.Controls;

namespace WpfApp
{
    // Reguła dziedziczy po ValidationRule
    public class AgeValidationRule : ValidationRule
    {
        public override ValidationResult Validate(object value, CultureInfo cultureInfo)
        {
            if (int.TryParse(value?.ToString(), out int age))
            {
                if (age < 0 || age > 120)
                {
                    return new ValidationResult(false, "Wiek musi być z przedziału 0-120.");
                }

                return ValidationResult.ValidResult; // Zwracamy ValidResult w razie sukcesu
            }

            return new ValidationResult(false, "To nie jest poprawna liczba.");
        }
    }

    public partial class MainWindow : Window
    {
        public int Age { get; set; } = 0;

        public MainWindow()
        {
            InitializeComponent();
            DataContext = this;
        }
    }
}
```

- **ValidationResult** – Wynik zawiera status przejścia weryfikacji i (opcjonalnie) wiadomość błędu.

- **ValidationRules** – Specjalna sekcja w definicji Bindingu nałożona w XAML.
- **Automatyczne wizualizacje** – WPF przytępuje `false` i domyślnie podświetla ramkę TextBoxu na czerwono.

34.3. IDataErrorInfo

Starszy, jednak nadal potężny interfejs modelujący walidację ze strony klas z danymi. Kiedy klasa wdroży `IDataErrorInfo`, sama o sobie decyduje, co jest błędem w jej polach, co świetnie wpisuje się w standardy programowania zorientowanego obiektowo. Trzeba uruchomić właściwość `ValidatesOnDataErrors="True"` w obiekcie widoku.

MainWindow.xaml:

```
<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="IDataErrorInfo" Height="250" Width="400">
    <StackPanel Margin="20">
        <Label Content="Nazwa użytkownika (wymagana):" />
        <TextBox Text="{Binding Username, UpdateSourceTrigger=PropertyChanged, ValidatesOnDataErrors=True}"
                Margin="0,0,0,10" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.ComponentModel;
using System.Windows;

namespace WpfApp
{
    public class UserAccount : IDataErrorInfo, INotifyPropertyChanged
    {
        private string _username;
        public string Username
        {
            get => _username;
            set { _username = value; OnPropertyChanged(nameof(Username)); }
        }

        // Metoda interfejsu (zwracająca podsumowanie całości klasy)
        public string Error => null;

        // Metoda interfejsu weryfikująca po nazwie kolumny (właściwości)
        public string this[string columnName]
        {
            get
            {
                if (columnName == nameof(Username))
                {
                    if (string.IsNullOrEmpty(Username))
                        return "Nazwa użytkownika nie może być pusta.";
                }
                return null; // Zwracamy null jeśli jest ok
            }
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged(string name) => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }

    public partial class MainWindow : Window
    {

```

```

public MainWindow()
{
    InitializeComponent();
    DataContext = new UserAccount();
}
}
}

```

- **Indeksator** – Pętla sprawdzania pyta obiekt `this["NazwaPola"]` czy kryje on jakieś komunikaty o błędach.
- **Centralizacja** – Obiekt klasy zawiera całą logikę obronną u siebie w metodach.
- **Ważność INotifyPropertyChanged** – Obiekt musi sygnalizować widokowi zmianę danych, by wyzwać walidację w czasie przepisywania znaków.

34.4. INotifyDataErrorInfo

Został wprowadzony w późniejszych wersjach C# i WPF, aby zastąpić sztywne zasady `IDataErrorInfo`. Jest nowoczesny, potrafi zwracać serię błędów dla jednego pola i obsługuje asynchroniczną walidację (np. sprawdzanie na serwerze czy login już istnieje). Korzysta z

`ValidatesOnNotifyDataErrors="True"`.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="INotifyDataErrorInfo" Height="250" Width="400">
    <StackPanel Margin="20">
        <Label Content="Adres e-mail:" />
        <TextBox Text="{Binding Email, UpdateSourceTrigger=PropertyChanged, ValidatesOnNotifyDataErrors=True}"
                Margin="0,0,0,10" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;
using System.Windows;

namespace WpfApp
{
    public class ContactForm : INotifyPropertyChanged, INotifyDataErrorInfo
    {
        private string _email = "";
        private readonly Dictionary<string, List<string>> _errors = new Dictionary<string, List<string>>();

        public string Email
        {
            get => _email;
            set
            {
                _email = value;
                ValidateEmail();
                OnPropertyChanged(nameof(Email));
            }
        }

        // Ważne: Sprawdzenie poprawności przypisuje tekst na listę i strzela zdarzeniem
        private void ValidateEmail()
        {

```

```

        ClearErrors(nameof(Email));
        if (string.IsNullOrEmpty(Email) || !Email.Contains("@"))
        {
            AddError(nameof(Email), "Podaj prawidłowy adres e-mail (musi zawierać @).");
        }
    }

    public bool HasErrors => _errors.Any();
    public event EventHandler<DataErrorsChangedEventArgs> ErrorsChanged;

    public IEnumerable GetErrors(string propertyName)
    {
        if (string.IsNullOrEmpty(propertyName) || !_errors.ContainsKey(propertyName))
            return null;
        return _errors[propertyName];
    }

    private void AddError(string propertyName, string error)
    {
        if (!_errors.ContainsKey(propertyName))
            _errors[propertyName] = new List<string>();

        if (!_errors[propertyName].Contains(error))
        {
            _errors[propertyName].Add(error);
            ErrorsChanged?.Invoke(this, new DataErrorsChangedEventArgs(propertyName));
        }
    }

    private void ClearErrors(string propertyName)
    {
        if (_errors.ContainsKey(propertyName))
        {
            _errors.Remove(propertyName);
            ErrorsChanged?.Invoke(this, new DataErrorsChangedEventArgs(propertyName));
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged(string name) => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        DataContext = new ContactForm();
    }
}
}

```

- **Złożoność** – Konfiguracja wymaga utworzenia małego zarządcy (słownika `Dictionary`), który magazynuje zbiór błędów dla podanej właściwości.
- **ErrorsChanged** – Zamiast polegać na natychmiastowym sprawdzaniu podczas indeksowania, rzucasz wydarzenie, że stan błędów uległ zmianie.
- **Wiele usterek na raz** – Funkcja `GetErrors` może zwrócić kolekcję błędów zgrupowanych np. przy hasle.

34.5. Komunikaty błędów przy kontroli

Sama czerwona ramka nie poinformuje użytkownika, co konkretnie zepsuł. Na szczęście kontrolki `TextBox` utrzymują dostęp do błędów poprzez ukrytą właściwość `Validation.Errors`. Przekierowujemy treść tego błędu bezpośrednio na ekran.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:WpfApp"
        Title="Komunikaty błędów" Height="250" Width="400">
    <StackPanel Margin="20">
        <Label Content="Imię (minimum 3 znaki):" />

        <!-- Pole z przypiętym DataErrorInfo -->
        <TextBox x:Name="NameTextBox"
                Text="{Binding FirstName, UpdateSourceTrigger=PropertyChanged, ValidatesOnDataErrors=True}" />

        <!-- Pobieramy komunikat korzystając ze składni ElementName oraz odniesienia do kolekcji Errors -->
        <TextBlock Foreground="Red" FontSize="12" Margin="0,5,0,0"
                Text="{Binding ElementName=NameTextBox, Path=(Validation.Errors)[0].ErrorContent}" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.ComponentModel;
using System.Windows;

namespace WpfApp
{
    public class Student : IDataErrorInfo, INotifyPropertyChanged
    {
        private string _firstName = "";
        public string FirstName
        {
            get => _firstName;
            set { _firstName = value; OnPropertyChanged(nameof(FirstName)); }
        }

        public string Error => null;
        public string this[string columnName]
        {
            get
            {
                if (columnName == nameof(FirstName))
                {
                    if (FirstName.Length < 3)
                        return "Imię jest za krótkie!";
                }
                return null;
            }
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged(string name) => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            DataContext = new Student();
        }
    }
}

```

```
}  
  
}
```

- **Validation.Errors[0]** – Kolekcja błędów przechowywanych w specjalnej puli w WPF na bazie pierwszej wygenerowanej pomyłki z modelu.
- **ErrorContent** – Sam tekst, który wygenerowała walidacja np. za pomocą `IDataErrorInfo`.
- **Czytelność interfejsu** – Informacja wyświetla się dynamicznie pod polem po wykryciu nieprawidłowości.

34.6. Style dla błędów walidacji

Jeśli nie chcesz dublować tagów `TextBlock` pod każdym `TextBoxem`, możesz włączyć styl z `Validation.ErrorTemplate` u podstaw okna. W ten sposób całkowicie odmieniasz wygląd zgłaszanych błędów. Zastępujesz domyślną, czerwoną ramkę własną grafiką na przykład dodając do kontrolki czerwony wykrzyknik informacyjny lub bąbelek z pop-upem (Tooltip).

MainWindow.xaml:

```
<Window x:Class="WpfApp.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Style walidacji" Height="250" Width="400">  
  
    <Window.Resources>  
        <!-- Zmieniamy zachowanie dla wszystkich TextBoxów posiadających błędy -->  
        <Style TargetType="TextBox">  
            <Setter Property="Validation.ErrorTemplate">  
                <Setter.Value>  
                    <ControlTemplate>  
                        <StackPanel Orientation="Horizontal">  
                            <!-- Pole tekstowe właściwe -->  
                            <AdornedElementPlaceholder x:Name="placeholder" />  
                            <!-- Element dodawany - ikonka i Tooltip -->  
                            <TextBlock Text=" ✖ " Foreground="Red" FontWeight="Bold" FontSize="14"  
                                Tooltip="{Binding ElementName=placeholder, Path=AdornedElement.(Validation.Errors)[0].ErrorContent}"/>  
                        </StackPanel>  
                    </ControlTemplate>  
                </Setter.Value>  
            </Setter>  
            <Style.Triggers>  
                <Trigger Property="Validation.HasError" Value="True">  
                    <Setter Property="Background" Value="#FFE8E8" />  
                </Trigger>  
            </Style.Triggers>  
        </Style>  
    </Window.Resources>  
  
    <StackPanel Margin="30">  
        <Label Content="Nazwa produktu (wymagana):" />  
        <TextBox Text="{Binding ProductName, UpdateSourceTrigger=PropertyChanged, ValidatesOnDataErrors=True}" Width="200" HorizontalAlignment="Left" />  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System.ComponentModel;  
using System.Windows;  
  
namespace WpfApp  
{  
    public class ProductItem : IDataErrorInfo, INotifyPropertyChanged  
    {  
        private string _productName = "";  
        public string ProductName
```

```

{
    get => _productName;
    set { _productName = value; OnPropertyChanged(nameof(ProductName)); }
}

public string Error => null;
public string this[string columnName]
{
    get
    {
        if (columnName == nameof(ProductName))
        {
            if (string.IsNullOrEmpty(ProductName))
                return "Brak podanej nazwy produktu!";
        }
        return null;
    }
}

public event PropertyChangedEventHandler PropertyChanged;
protected void OnPropertyChanged(string name) => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        DataContext = new ProductItem();
    }
}
}

```

- **ErrorTemplate** – Oszukuje renderowanie WPF dodając do obrysu kontrolki całkowicie nowy układ elementów.
- **AdornedElementPlaceholder** – Specjalny tag oznaczający: "Tutaj wstaw po prostu tę kontrolkę (TextBox), dla której wyzwolono styl".
- **Trigger HasError** – Użyty w przykładzie, aby podświetlić tło elementu na błąd czerwien w przypadku negatywnej walidacji.

35. MVVM w praktyce

35.1. Model, View, ViewModel

Wzorec projektowy MVVM (Model-View-ViewModel) narzuca ścisłą izolację między trzema częściami projektu. Jest rekomendowany do tworzenia profesjonalnych systemów WPF.

1. **Model** – Twoje dane bazodanowe, klasy domenowe (np. `Klucz`, `Zamek`).
2. **View (Widok)** – Kod XAML (np. `MainWindow.xaml`), kontrolki. Tutaj z reguły nie piszesz w ogóle językiem C#. Zostawiasz okno tylko do formatowania.
3. **ViewModel** – Tłumacz, silnik scalający i stan okna. Kopia robocza modelu, do której binduje się wprost interfejs.

MainWindow.xaml (View):

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="MVVM Start" Height="200" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="{Binding InfoMessage}" FontSize="20"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs oraz ViewModel.cs (C#):


```

using System.Windows;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace WpfApp
{
    // 1. MODEL
    public class ApplicationInfo
    {
        public string ApplicationName { get; set; } = "WPF Sklep";
        public string Version { get; set; } = "1.0.0";
    }

    // 2. VIEWMODEL
    public class MainViewModel : INotifyPropertyChanged
    {
        private ApplicationInfo _model;

        public MainViewModel()
        {
            _model = new ApplicationInfo();
        }

        // ViewModel "tłumaczy" dane surowe pod wyśrodkowany tekst w Widoku
        public string InfoMessage => $"Witaj w {_model.ApplicationName} v. {_model.Version}";

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string name = null) =>
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }

    // 3. POŁĄCZENIE VIEW Z VIEWMODEL (tzw. Code-Behind jest skrócony do minimum)
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            DataContext = new MainViewModel(); // View wie o ViewModelu
        }
    }
}

```

- **Zmniejszenie Code-Behind** – Plik `MainWindow.xaml.cs` prawie znika, zajmując się wyłącznie przełączeniem konstruktora pod klasę z ViewModelu.
- **Wysoka testowalność** – ViewModel i Model nie wiedzą, że operują w strukturze okienkowej, więc nie wymagają uruchamiania interfejsu (dają się testować maszynowo w testach jednostkowych).

35.2. Kiedy warto używać MVVM

Praca bez wzorca powoduje tzw. "Spaghetti Code" w zdarzeniach, gdzie przyciski wpływają na formularze z wykorzystaniem zagnieżdżonych `if/else`. W środowiskach firmowych praca grupowa na XAML z wieloma ukrytymi nazwami, które potem odwołują się do C#, nie działa dobrze. W MVVM architektura projektowania jest dużo czystsza, a designerzy modyfikujący warstwy w programie Blend pracują tylko na plikach XAML. Poniżej zaprezentowano typowy zły nawyk (bez MVVM) oraz krótką opozycję XAML.

Błędne (Bez MVVM) - MainWindow.xaml / .cs:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zły nawyk" Height="200" Width="400">
    <StackPanel>
        <TextBox x:Name="CalcInput" />
        <Button Click="Btn_Click" />
    </StackPanel>

```

```

        <TextBlock x:Name="CalcResult" />
    </StackPanel>
</Window>

```

```

using System;
using System.Windows;

namespace WpfApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // W XAML musi istnieć sztywna nazwa "CalcInput" oraz "CalcResult"
            private void Btn_Click(object sender, RoutedEventArgs e)
            {
                // Mieszasz kod działania z manipulowaniem elementami
                int val = int.Parse(CalcInput.Text);
                CalcResult.Text = (val * 2).ToString();
            }
        }
    }
}

```

- **Podejście klasyczne** – Ścisłe związanie przycisku ze zdarzeniem, a pól tekstowych z ich unikalnymi kontrolkami widoku.
- **Problemy w klasycznym podejściu** – Kiedy usuniesz element tekstowy z okienka XAML, twój plik w C# się zatrzyma z błędami i aplikacja wyrzuci okno awaryjne kompilacji. Przy zastosowaniu elastycznego wzorca MVVM brak kontrolki wyłącza zwyczajnie binding i aplikacja działa dalej bez zarzutów.

35.3. ViewModel jako stan widoku

Wiele osób myli ViewModel z samym zbiorem właściwości. To nie jest po prostu plik magazynowy. Kontroluje on widoczność bloków, informuje co otworzyć, kiedy kręcić kursorem postępu (ang. *Spinner*) i tak dalej. Definiuje on pełen "Stan Aplikacji".

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ViewModel jako Stan" Height="250" Width="400">
    <StackPanel Margin="20">
        <!-- Wiążemy Enabled na podstawie ViewModelu -->
        <TextBox Text="Dane..." IsEnabled="{Binding IsInputEnabled}" Margin="0,0,0,10"/>
        <Button Content="Uruchom skrypt" IsEnabled="{Binding IsInputEnabled}" Margin="0,0,0,20"/>

        <!-- Tu przydałby się konwerter na Visibility, ale używamy wbudowanego Bindingu ze sprawdzaniem stanu. -->
        <TextBlock Text="Aplikacja jest aktualnie zablokowana i pracuje w tle."
                  Foreground="Red" FontWeight="Bold"
                  Visibility="{Binding LoadingMessageVisibility}" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;

namespace WpfApp

```

```

{
    public class ProcessViewModel : INotifyPropertyChanged
    {
        private bool _isProcessing = true; // Zaczynamy symulację blokady formularza

        public bool IsInputEnabled => !_isProcessing; // Jeśli przetwarza, zablokuj wejście

        public Visibility LoadingMessageVisibility => _isProcessing ? Visibility.Visible : Visibility.Collapsed;

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string name = null) =>
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));

        // W normalnej aplikacji ta metoda wyzwalalaby się po ukończeniu asynchronicznych zdań
        public void FinishProcessing()
        {
            _isProcessing = false;
            OnPropertyChanged(nameof(IsInputEnabled));
            OnPropertyChanged(nameof>LoadingMessageVisibility));
        }
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            var vm = new ProcessViewModel();
            DataContext = vm;

            // W demonstracji szybko przerywamy przetwarzanie, okno by się oblokowało.
            // vm.FinishProcessing();
        }
    }
}

```

- **Brak nadmiarowości widoku** – Nie potrzebujesz konwertera `BoolToVisibilityConverter`, bo w ViewModelu wystawiasz konkretny typ np. `Visibility`. Puryści architektoniczni radzą tego unikać i stawiać na konwertery, ale to wygodna opcja.
- **Strumieniowanie aktualizacji** – Zmiana ukrytego stanu (zmiennej pola np. `_isProcessing`) emituje zgłoszenia do wszystkich właściwości odczytujących z niej informacje na ekranie.

35.4. ICommand

W MVVM unikasz subskrybowania się pod zdarzenia `Click="Method"`. Przyciski dysponują zaawansowanym systemem wiązania z interfejsem `ICommand`. Metody w ViewModel implementujące komendy mają wielką moc i są znacznie sprytniejsze od eventów.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ICommand z MVVM" Height="200" Width="400">
    <StackPanel Margin="20">
        <!-- Zauważ brak słowa Click, zamiast tego Command Binding -->
        <Button Content="Uruchom" Command="{Binding RunCommand}" Height="40" FontSize="16"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Input;

namespace WpfApp
{
    // Prosta klasa wykonująca, która przyjmuje polecenia
    public class SimpleCommand : ICommand
    {
        public event EventHandler CanExecuteChanged;

        public bool CanExecute(object parameter)
        {
            return true; // Komenda jest zawsze gotowa do kliknięcia
        }

        public void Execute(object parameter)
        {
            MessageBox.Show("Wykonano komendę!");
        }
    }

    public class CommandViewModel
    {
        public ICommand RunCommand { get; }

        public CommandViewModel()
        {
            RunCommand = new SimpleCommand();
        }
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            DataContext = new CommandViewModel();
        }
    }
}

```

- **ICommand** – Składa się z metody akcyjnej (**Execute**) i metody sprawdzającej poprawność warunków (**CanExecute**).
- **Command Property** – Kontrolka **Button** wspiera powiązanie akcji z tak zdefiniowanymi typami za pomocą jej wbudowanej właściwości **Command**.

35.5. RelayCommand

Pisanie osobnej klasy dziedziczącej z **ICommand** dla każdego przycisku w oknie było by uciążliwe (zamiast np. 10 metod, otrzymujemy 10 plików klas). Społeczność stworzyła popularną implementację generyczną **RelayCommand** (niekiedy zwaną **DelegateCommand**), która pozwala na przekazanie wskaźników do metod (tzw. delegatów), bezpośrednio wewnątrz konstruktora w klasie.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Relay Command" Height="200" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="{Binding Counter}" FontSize="24" Margin="0,0,0,10" HorizontalAlignment="Center"/>
        <Button Content="Zwiększ licznik" Command="{Binding IncrementCommand}" Height="30" />
    </StackPanel>
</Window>

```

```
</StackPanel>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;
using System.Windows.Input;

namespace WpfApp
{
    // Wielorazowa, globalna klasa narzędziowa RelayCommand
    public class RelayCommand : ICommand
    {
        private readonly Action<object> _execute;
        private readonly Func<object, bool> _canExecute;

        public RelayCommand(Action<object> execute, Func<object, bool> canExecute = null)
        {
            _execute = execute;
            _canExecute = canExecute;
        }

        public event EventHandler CanExecuteChanged
        {
            add { CommandManager.RequerySuggested += value; }
            remove { CommandManager.RequerySuggested -= value; }
        }

        public bool CanExecute(object parameter) => _canExecute == null || _canExecute(parameter);

        public void Execute(object parameter) => _execute(parameter);
    }

    // Twoja klasa z danymi (ViewModel)
    public class CounterViewModel : INotifyPropertyChanged
    {
        private int _counter;
        public int Counter
        {
            get => _counter;
            set { _counter = value; OnPropertyChanged(); }
        }

        public ICommand IncrementCommand { get; }

        public CounterViewModel()
        {
            // Przekazanie metody do delegata:
            IncrementCommand = new RelayCommand(ExecuteIncrement);

            private void ExecuteIncrement(object parameter)
            {
                Counter++;
            }

            public event PropertyChangedEventHandler PropertyChanged;
            protected void OnPropertyChanged([CallerMemberName] string name = null) =>
```

```

        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            DataContext = new CounterViewModel();
        }
    }
}

```

- **Action / Func** – Specyficzne typy metod, w których zawierasz własny kod akcji i zapytań o wykonanie (Delegaty i wyrażenia Lambda to twój przyjaciel z języka C#).
- **RequerySuggested** – Podpięty globalny sygnał, odświeżający zapytanie stanu do wszystkich przycisków na widoku przy każdej zmianie interfejsu klawiatury.
- **Jedna deklaracja** – Raz zakodowaną klasę `RelayCommand` możesz swobodnie kopiować między wirtualnie każdym projektem WPF jaki utworzysz.

35.6. CanExecute

Jedną z najpotężniejszych funkcji wzorca MVVM po stronie obsługi interfejsu poprzez zaimplementowane ICommandy, jest fakt, że przyciski kontrolują się same. Zwykle zjawisko uaktywniania lub gaszenia kontrolki np. przy wyczyszczonym polu wpisywania jest kodowane dziesiątkami reguł typu `Zapisz.IsEnabled = true`. A jeśli przekażesz swoją logikę odczytywania w delegacie funkcji `CanExecute`, to cały problem znika, bowiem szary wygląd dezaktywowanego przycisku nałoży aplikacja.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="CanExecute w Akcji" Height="250" Width="400">
    <StackPanel Margin="20">
        <CheckBox x:Name="AgreementCheck" IsChecked="{Binding HasAgreed}" Content="Zgadzam się na warunki" Margin="0,0,0,20"/>

        <Button Content="Zarejestruj się" Command="{Binding RegisterCommand}" Height="40" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;
using System.Windows.Input;

namespace WpfApp
{
    // Minimalistyczna wersja RelayCommand
    public class RelayCommand : ICommand
    {
        private Action _execute;
        private Func<bool> _canExecute;

        public RelayCommand(Action execute, Func<bool> canExecute)
        {
            _execute = execute;
            _canExecute = canExecute;
        }

        public event EventHandler CanExecuteChanged

```

```

    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }

    public bool CanExecute(object parameter) => _canExecute();
    public void Execute(object parameter) => _execute();
}

public class RegistrationViewModel : INotifyPropertyChanged
{
    private bool _hasAgreed;
    public bool HasAgreed
    {
        get => _hasAgreed;
        set { _hasAgreed = value; OnPropertyChanged(); }
    }

    public ICommand RegisterCommand { get; }

    public RegistrationViewModel()
    {
        // Przycisk kliknie się tylko wtedy, gdy HasAgreed powróci z wartością TRUE
        RegisterCommand = new RelayCommand(
            execute: () => MessageBox.Show("Rejestracja pomyślna!"),
            canExecute: () => HasAgreed
        );
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string name = null) =>
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        DataContext = new RegistrationViewModel();
    }
}
}

```

- **Zablokowanie interfejsu z automatu** – Zgoda użytkownika ustawiona na `false` spowoduje wyłączenie opcji interakcji.
- **Reaktywacja powiązań** – Klikając w opcje zgód uruchamiasz `CommandManager` do przeszukania przycisków w celu skanowania zgody i ponownej ich aktywacji, gdy zapalą się właściwości z danymi.

35.7. Parametry komend

Często jeden uniwersalny przycisk wywołuje to samo zdarzenie na podstawie określonych zmiennych danych (np. dodawanie argumentu produktu). Binding obsługuje zjawisko dostarczania modyfikatora (zmiennej) wyzwalanego akcją użytkownika i przechwytywanego we wnętrzu klasy dziedziczącej logikę akcji. Tak podpinasz np. parametry numeryczne. Używaj właściwości `CommandParameter` na poziomie przycisku.

MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Command Parameter" Height="250" Width="400">
    <StackPanel Margin="20" HorizontalAlignment="Center">
        <TextBlock Text="{Binding Score}" FontSize="30" Margin="0,0,0,20" HorizontalAlignment="Center"/>
    </StackPanel>
</Window>

```

```

<StackPanel Orientation="Horizontal">
    <!-- Pośyłamy parametr +10 -->
    <Button Content="+10" Command="{Binding ChangeScoreCommand}" CommandParameter="10" Width="60" Margin="5"/>
    <!-- Pośyłamy parametr -5 -->
    <Button Content="-5" Command="{Binding ChangeScoreCommand}" CommandParameter="-5" Width="60" Margin="5"/>
</StackPanel>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;
using System.Windows.Input;

namespace WpfApp
{
    public class RelayCommand : ICommand
    {
        private Action<object> _execute;

        public RelayCommand(Action<object> execute)
        {
            _execute = execute;
        }

        public event EventHandler CanExecuteChanged;
        public bool CanExecute(object parameter) => true;
        public void Execute(object parameter) => _execute(parameter);
    }

    public class GameViewModel : INotifyPropertyChanged
    {
        private int _score;
        public int Score
        {
            get => _score;
            set { _score = value; OnPropertyChanged(); }
        }

        public ICommand ChangeScoreCommand { get; }

        public GameViewModel()
        {
            ChangeScoreCommand = new RelayCommand(ChangeScore);
        }

        private void ChangeScore(object parameter)
        {
            if (parameter != null && int.TryParse(parameter.ToString(), out int changeValue))
            {
                Score += changeValue;
            }
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string name = null) =>
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }
}

```



```

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        DataContext = new GameViewModel();
    }
}
}

```

- **Jeden kod na wszystko** – Korzystasz tylko z jednego pola `ChangeScoreCommand` dla dziesiątek klawiszy funkcyjnych.
- **Niejawne posyłanie tekstu** – WPF standardowo przesyła to co odnajdzie jako parametry np. po tagu zadeklarowanym w kodzie jako ciąg formatowany do stringa.
- **Obiekty dynamiczne** – Przesyłanym wskaźnikiem nie musi być ciąg tekstowy. Parametr komendy to swobodne powiązanie za pomocą tagów `{Binding MójZłożonyObiekt}` kierujące cały zasób instancji ujętej klasy na stół operacyjny polecenia przy jego wywołaniu.

35.8. Przenoszenie logiki z code-behind

Największym testem zrozumienia wzorca projektowego Model View ViewModel jest moment refaktoringu logiki skompresowanej za zdarzeniami do klas zewnętrznych powiązanych danymi. Operację tę nazywamy "odchudzaniem". Prezentowany układ konwersji demonstruje ten proces krok po kroku włączając odseparowanie interakcji pod akcje i zamianę zmiennych operacyjnych w zdarzenia odświeżenia.

Przed refaktoryzacją (zła praktyka):

```

<!-- Plik przed modyfikacjami -->
<Window ...>
    <StackPanel>
        <TextBox x:Name="InputA" />
        <TextBox x:Name="InputB" />
        <Button Content="Połącz" Click="JoinBtn_Click" />
        <TextBlock x:Name="Result" />
    </StackPanel>
</Window>

```

```

// Wnętrze pliku okna MainWindow.cs
private void JoinBtn_Click(object sender, RoutedEventArgs e)
{
    Result.Text = InputA.Text + " " + InputB.Text;
}

```

Po refaktoryzacji na pełne MVVM – MainWindow.xaml:

```

<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Czyste środowisko" Height="250" Width="400">
    <StackPanel Margin="20">
        <!-- Zniknęły nazwy kluczowe (x:Name). Pozostały same wiązania -->
        <TextBox Text="{Binding PartA, UpdateSourceTrigger=PropertyChanged}" Margin="0,0,0,10"/>
        <TextBox Text="{Binding PartB, UpdateSourceTrigger=PropertyChanged}" Margin="0,0,0,10"/>

        <Button Content="Połącz" Command="{Binding JoinCommand}" Margin="0,0,0,20"/>

        <TextBlock Text="{Binding FullResult}" FontWeight="Bold" />
    </StackPanel>
</Window>

```

Po refaktoryzacji na pełne MVVM – MainWindow.xaml.cs (oraz reszta struktury):

```

using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;
using System.Windows.Input;

namespace WpfApp
{
    public class SimpleRelayCommand : ICommand
    {
        private Action _execute;

        public SimpleRelayCommand(Action execute) ⇒ _execute = execute;

        public event EventHandler CanExecuteChanged;

        public bool CanExecute(object parameter) ⇒ true;

        public void Execute(object parameter) ⇒ _execute();
    }

    public class TextJoinerViewModel : INotifyPropertyChanged
    {
        private string _partA;
        private string _partB;
        private string _fullResult;

        public string PartA
        {
            get ⇒ _partA;
            set { _partA = value; OnPropertyChanged(); }
        }

        public string PartB
        {
            get ⇒ _partB;
            set { _partB = value; OnPropertyChanged(); }
        }

        public string FullResult
        {
            get ⇒ _fullResult;
            set { _fullResult = value; OnPropertyChanged(); }
        }

        public ICommand JoinCommand { get; }

        public TextJoinerViewModel()
        {
            JoinCommand = new SimpleRelayCommand(PerformJoin);
        }

        private void PerformJoin()
        {
            FullResult = $"{PartA} {PartB}";
        }

        public event PropertyChangedEventHandler PropertyChanged;

        protected void OnPropertyChanged([CallerMemberName] string name = null) ⇒
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
    }
}

```

```

    {
        InitializeComponent();

        // Code-behind sprowadzone do jednego nakierowania instancji!
        DataContext = new TextJoinerViewModel();
    }
}
}

```

- **Puste x:Name** – Kontrolki na widoku stają się całkowicie anonimowe w układzie hierarchicznym XAML. Nie potrzebujemy wiedzieć jak się nazywają, a tylko pożądaną jest do nich wniknięcie by dostarczyć pakiet danych po przez `Binding`.
- **Uniezależnienie widoku** – Nawet usunięcie przycisku "Połącz" ze szkieletu aplikacji nie wywoła paraliżu pozostałych części i logiki biznesowej z nim związanej.
- **Modularność** – Możemy stworzyć 10 odrębnych widoków podpinających się pod identycznie ten sam interfejs silnika logiki aplikacji, zmniejszając wymiar napisanego kodu C#.

36. Zasoby i style

W tej sekcji zobaczysz, w jaki sposób organizować wygląd aplikacji za pomocą zasobów i stylów. Dowiesz się, jak tworzyć powtarzalne schematy kolorów, czcionek czy zachowań kontrolki i używać ich w wielu miejscach jednocześnie. Jest to kluczowe, aby interfejs był spójny i łatwy w utrzymaniu.

36.1. Resources

Każdy element w WPF, od całej aplikacji (`App.xaml`), przez okno (`Window`), aż po pojedynczą kontrolkę (np. `Grid` lub `StackPanel`), posiada właściwość `Resources`. Służy ona do przechowywania wielokrotnie używanych obiektów, takich jak kolory, pędzle (Brushes), style czy szablony. Definiując coś raz w `Resources`, możesz odwoływać się do tego wielokrotnie.

MainWindow.xaml:

```

<Window x:Class="WpfResources.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zasoby" Height="450" Width="800">

    <Window.Resources>

        <SolidColorBrush x:Key="MyCustomColor" Color="#FF336699" />

        <FontFamily x:Key="MyCustomFont">Segoe UI</FontFamily>

    </Window.Resources>

    <StackPanel Margin="20">

        <Button Background="{StaticResource MyCustomColor}"
                FontFamily="{StaticResource MyCustomFont}"
                Content="Przycisk 1" Height="40" Margin="5" />

        <Button Background="{StaticResource MyCustomColor}"
                FontFamily="{StaticResource MyCustomFont}"
                Content="Przycisk 2" Height="40" Margin="5" />

    </StackPanel>

</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfResources
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **x:Key** – Każdy zasób musi mieć unikalny klucz, po którym będziesz się do niego odwoływać.
- **Window.Resources** – Kolekcja zasobów dostępna dla okna i wszystkich jego elementów potomnych.
- **Wielokrotne użycie** – Przycisk 1 i Przycisk 2 używają tego samego zasobu koloru i czcionki.

36.2. StaticResource

StaticResource to najczęstszy sposób odwoływania się do zasobów w XAML. Kiedy używasz **StaticResource**, WPF szuka zasobu tylko raz – podczas wczytywania okna. Jeśli później zmienisz wartość tego zasobu w kodzie, zmiana ta **nie zostanie** zaktualizowana w widoku. Używasz go dla wartości, które są stałe podczas działania programu.

MainWindow.xaml:

```

<Window x:Class="WpfStaticResource.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="StaticResource" Height="450" Width="800">
    <Window.Resources>
        <System:String x:Key="MyGreeting" xmlns:System="clr-namespace:System;assembly=mscorlib">Witaj w systemie!</System:String>
    </Window.Resources>
    <StackPanel Margin="20">
        <TextBlock Text="{StaticResource MyGreeting}" FontSize="24" FontWeight="Bold" />
        <Button Content="Zmień tekst (nie zadziała na ekranie)" Click="ChangeText_Click" Margin="0,20,0,0" Padding="10"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfStaticResource
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void ChangeText_Click(object sender, RoutedEventArgs e)
        {
            // Mimo że zmieniamy wartość w zasobach, TextBlock się nie odświeży
            // ponieważ użył StaticResource (pobrał wartość raz na starcie).
            this.Resources["MyGreeting"] = "Tekst po zmianie";
        }
    }
}

```

```
}  
  
}
```

Kluczowe elementy:

- **StaticResource** – Oszczędza pamięć i procesor, ponieważ nie nasłuchuje zmian.
- **Jednorazowe bindowanie** – Pobranie wartości ma miejsce w momencie ładowania interfejsu.
- **Użycie** – Idealne do stałych kolorów, rozmiarów czcionek i marginesów.

36.3. DynamicResource

W przeciwieństwie do **StaticResource**, **DynamicResource** nasłuchuje zmian w zasobach przez cały czas życia aplikacji. Jeśli zmienisz wartość zasobu w kodzie C#, wszystkie elementy używające **DynamicResource** natychmiast się zaktualizują. Używasz go głównie wtedy, gdy tworzysz motywy (jasny/ciemny) i zmieniasz je "w locie".

MainWindow.xaml:

```
<Window x:Class="WpfDynamicResource.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="DynamicResource" Height="450" Width="800">  
    <Window.Resources>  
        <SolidColorBrush x:Key="AppBackground" Color="LightGray" />  
    </Window.Resources>  
    <Grid Background="{DynamicResource AppBackground}">  
        <StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">  
            <TextBlock Text="Ten kolor można zmienić w locie" FontSize="20" Margin="10" />  
            <Button Content="Zmień na ciemny motyw" Click="ChangeTheme_Click" Padding="10" />  
        </StackPanel>  
    </Grid>  
</Window>
```

MainWindow.xaml.cs:

```
using System;  
using System.Windows;  
using System.Windows.Media;  
  
namespace WpfDynamicResource  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
  
        private void ChangeTheme_Click(object sender, RoutedEventArgs e)  
        {  
            // Aktualizujemy zasób, a widok automatycznie się odświeża  
            this.Resources["AppBackground"] = new SolidColorBrush(Colors.DarkSlateGray);  
        }  
    }  
}
```

Kluczowe elementy:

- **Aktualizacja w czasie rzeczywistym** – Element interfejsu reaguje na podmiannę zasobu.
- **Koszt wydajności** – Jest nieco wolniejszy i zużywa więcej zasobów niż **StaticResource**.
- **Zastosowanie** – Zmiana języka (lokalizacja), zmiana motywów graficznych.

36.4. Style lokalne

Style pozwalają zgrupować wiele właściwości (np. tło, czcionka, marginesy) i zastosować je do wielu kontroltek naraz. Styl lokalny to taki, który umieszczasz np. w `Window.Resources` lub `Grid.Resources` i odwołujesz się do niego w obrębie danego pliku.

MainWindow.xaml:

```
<Window x:Class="WpfLocalStyles.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Styl lokalne" Height="450" Width="800">
    <Window.Resources>
        <Style x:Key="MyButtonStyle" TargetType="Button">
            <Setter Property="Background" Value="CornflowerBlue" />
            <Setter Property="Foreground" Value="White" />
            <Setter Property="FontSize" Value="16" />
            <Setter Property="Margin" Value="10" />
            <Setter Property="Padding" Value="15,5" />
            <Setter Property="Cursor" Value="Hand" />
        </Style>
    </Window.Resources>
    <StackPanel>
        <Button Content="Zastosowany styl 1" Style="{StaticResource MyButtonStyle}" />
        <Button Content="Zastosowany styl 2" Style="{StaticResource MyButtonStyle}" />
        <Button Content="Zwykły przycisk bez stylu" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace WpfLocalStyles
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Kluczowe elementy:

- **Style** – Tag definiujący zestaw właściwości.
- **TargetType** – Określa, dla jakiej kontrolki ten styl jest przeznaczony.
- **Użycie** `Style="{StaticResource ...}"` – Przypisanie stylu do konkretnej kontrolki.

36.5. Style globalne w App.xaml

Kiedy stworzysz dużą aplikację, chcesz, aby przyciski w każdym oknie wyglądały tak samo. Zamiast kopiować style do każdego okna, umieszczasz je w pliku `App.xaml`. Zasoby z `App.xaml` są dostępne we wszystkich oknach i widokach aplikacji.

App.xaml:

```
<Application x:Class="WpfGlobalStyles.App"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        StartupUri="MainWindow.xaml">
    <Application.Resources>
        <!-- Brak x:Key oznacza, że to styl domyślny dla wszystkich TextBoxów w aplikacji -->
```

```

        <Style TargetType="TextBox">
            <Setter Property="Margin" Value="5" />
            <Setter Property="Padding" Value="5" />
            <Setter Property="BorderBrush" Value="DodgerBlue" />
            <Setter Property="BorderThickness" Value="2" />
        </Style>
    </Application.Resources>
</Application>

```

MainWindow.xaml:

```

<Window x:Class="WpfGlobalStyles.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Style globalne" Height="450" Width="800">
    <StackPanel Margin="20">
        <TextBlock Text="Imię:" />
        <TextBox />
        <TextBlock Text="Nazwisko:" />
        <TextBox />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfGlobalStyles
{
    public partial class MainWindow : Window
    {
        {
            public MainWindow()
            {
                InitializeComponent();
            }
        }
    }
}

```

Kluczowe elementy:

- **Application.Resources** – Globalne zasoby dostępne w całej aplikacji.
- **Styl domyślny** – Gdy podasz **TargetType**, ale pominiesz **x:Key**, styl zostanie nałożony na wszystkie kontrolki danego typu automatycznie.
- **Spójność** – Zmiana w jednym miejscu w **App.xaml** wpływa na setki okien naraz.

36.6. Setter

Wewnątrz tagu **<Style>** umieszczasz elementy **<Setter>**. Setter to instrukcja mówiąca: "ustaw tę właściwość na tę wartość". Bez Setterów style nie robiłyby nic.

MainWindow.xaml:

```

<Window x:Class="WpfSetters.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Setter" Height="450" Width="800">
    <Window.Resources>
        <Style x:Key="BigLabelStyle" TargetType="Label">
            <Setter Property="FontSize" Value="30" />
            <Setter Property="FontWeight" Value="Bold" />
            <Setter Property="HorizontalContentAlignment" Value="Center" />
        </Style>
    </Window.Resources>

```

```

        <!-- Możemy też ustawiać złożone wartości używając zagnieżdżenia -->
        <Setter Property="Foreground">
            <Setter.Value>
                <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
                    <GradientStop Color="Red" Offset="0.0" />
                    <GradientStop Color="Yellow" Offset="1.0" />
                </LinearGradientBrush>
            </Setter.Value>
        </Setter>
    </Style>
</Window.Resources>
<Grid>
    <Label Content="Używamy Settera!" Style="{StaticResource BigLabelStyle}" />
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfSetters
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Property** – Nazwa właściwości, którą chcesz zmienić (np. **FontSize**).
- **Value** – Prosta wartość podana jako tekst (np. **"30"**).
- **Setter.Value** – Znacznik pozwalający na użycie złożonych obiektów jako wartości (np. gradientów).

36.7. TargetType

Zrozumienie **TargetType** jest kluczowe, kiedy tworzysz style. Definiuje on z góry, z jaką klasą (jaką kontrolką) masz do czynienia. Dzięki temu edytor kodu wie, jakie właściwości może ci podpowiedzieć w atrybucie **Property** wewnątrz Settera.

MainWindow.xaml:

```

<Window x:Class="WpfTargetType.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="TargetType" Height="450" Width="800">
    <Window.Resources>
        <!-- Styl bazowy dla wszystkich kontroltek dziedziczących z Control -->
        <Style x:Key="BaseControlStyle" TargetType="Control">
            <Setter Property="Margin" Value="10" />
            <Setter Property="FontSize" Value="20" />
        </Style>

        <!-- Styl dziedziczący po BaseControlStyle -->
        <Style x:Key="SpecialButtonStyle" TargetType="Button" BasedOn="{StaticResource BaseControlStyle}">
            <Setter Property="Background" Value="Green" />
            <Setter Property="Foreground" Value="White" />
        </Style>
    </Window.Resources>

```



```

<StackPanel>
    <!-- TextBox dziedziczy po Control, więc pasują mu właściwości z BaseControlStyle -->
    <TextBox Style="{StaticResource BaseControlStyle}" Text="Wpisz coś..." />

    <!-- Button używa stylu SpecialButtonStyle, który sam z kolei dziedziczy z BaseControlStyle -->
    <Button Style="{StaticResource SpecialButtonStyle}" Content="Dziedziczenie stylów" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfTargetType
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Klasa bazowa** – Możesz określić `TargetType="Control"`, dzięki czemu styl zadziała na Button, TextBox, Label itd.
- `BasedOn` – Pozwala tworzyć hierarchię stylów (dziedziczenie). Styl pochodny otrzymuje wszystkie Settery ze stylu bazowego.

36.8. Zasoby kolorów

W profesjonalnych aplikacjach nigdy nie wpisuje się kolorów na sztywno ("na twardo") bezpośrednio do właściwości (np. `Background="Red"`). Zamiast tego definiuje się tak zwaną paletę kolorów w sekcji Resources, używając obiektów `Color` lub `SolidColorBrush`.

MainWindow.xaml:

```

<Window x:Class="WpfColorResources.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zasoby kolorów" Height="450" Width="800">
    <Window.Resources>
        <!-- Definiujemy sam kolor -->
        <Color x:Key="PrimaryColor">#FF6200EE</Color>
        <Color x:Key="SecondaryColor">#FF03DAC6</Color>

        <!-- Definiujemy pędzle oparte na kolorach -->
        <SolidColorBrush x:Key="PrimaryBrush" Color="{StaticResource PrimaryColor}" />
        <SolidColorBrush x:Key="SecondaryBrush" Color="{StaticResource SecondaryColor}" />
        <SolidColorBrush x:Key="TextBrush" Color="White" />
    </Window.Resources>
    <Grid Background="#FFF5F5F5">
        <StackPanel Margin="50">
            <Border Background="{StaticResource PrimaryBrush}" CornerRadius="10" Padding="20" Margin="10">
                <TextBlock Text="Główny element interfejsu" Foreground="{StaticResource TextBrush}" FontSize="24" />
            </Border>
            <Button Background="{StaticResource SecondaryBrush}" Foreground="Black"
                    Content="Akcja poboczna" Padding="15" Margin="10" FontSize="18" BorderThickness="0" />
        </StackPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace WpfColorResources
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Kluczowe elementy:

- **Separacja** – Trzymanie kolorów w jednym miejscu ułatwia ewentualną zmianę designu (rebranding).
- **Brush a Color** – Właściwość `Background` wymaga obiektu typu `Brush`, a nie samego koloru. Dlatego definiujemy `SolidColorBrush`.
- **Czytelność** – Nazywając pędzle logicznie (np. `PrimaryBrush`), kod XAML staje się bardzo łatwy do czytania.

36.9. Prosty motyw aplikacji

Połączenie dynamicznych zasobów, pędzli i kodu C# pozwala na utworzenie prostego systemu zmiany motywu aplikacji, na przykład trybu jasnego i ciemnego. Używamy `DynamicResource`, aby kontrolki natychmiast załapały zmianę i zareagowały na nadpisanie kluczy w zasobach.

MainWindow.xaml:

```
<Window x:Class="WpfSimpleTheme.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Motyw" Height="450" Width="800">
    <!-- Deklarujemy domyślne (jasne) pędzle jako DynamicResource -->
    <Window.Resources>
        <SolidColorBrush x:Key="WindowBg" Color="White" />
        <SolidColorBrush x:Key="TextFg" Color="Black" />
        <SolidColorBrush x:Key="PanelBg" Color="#EEEEEE" />
    </Window.Resources>

    <Grid Background="{DynamicResource WindowBg}">
        <Border Background="{DynamicResource PanelBg}" Margin="50" Padding="20" CornerRadius="15">
            <StackPanel>
                <TextBlock Text="Przełącznik motywu" Foreground="{DynamicResource TextFg}"
                           FontSize="28" HorizontalAlignment="Center" Margin="0,0,0,20"/>

                <Button Content="Uruchom tryb ciemny" Click="DarkMode_Click" Margin="10" Padding="10"/>
                <Button Content="Uruchom tryb jasny" Click="LightMode_Click" Margin="10" Padding="10"/>
            </StackPanel>
        </Border>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;
using System.Windows.Media;

namespace WpfSimpleTheme
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

```

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    private void DarkMode_Click(object sender, RoutedEventArgs e)
    {
        // Nadpisujemy zasoby nowymi wartościami
        this.Resources["WindowBg"] = new SolidColorBrush(Color.FromRgb(30, 30, 30));
        this.Resources["TextFg"] = new SolidColorBrush(Colors.White);
        this.Resources["PanelBg"] = new SolidColorBrush(Color.FromRgb(45, 45, 45));
    }

    private void LightMode_Click(object sender, RoutedEventArgs e)
    {
        // Przywracamy jasne kolory
        this.Resources["WindowBg"] = new SolidColorBrush(Colors.White);
        this.Resources["TextFg"] = new SolidColorBrush(Colors.Black);
        this.Resources["PanelBg"] = new SolidColorBrush(Color.FromRgb(238, 238, 238));
    }
}

```

Kluczowe elementy:

- **DynamicResource** **wszędzie** – Każdy element, który ma zmienić wygląd, korzysta z dynamicznego zasobu.
- **Nadpisywanie słownika** – W kodzie wystarczy zmienić wartość w kolekcji **Resources**, a WPF sam przerysuje interfejs.
- **Natychmiastowy efekt** – Motyw zmienia się bez konieczności ponownego ładowania okna.

37. Szablony i wygląd kontroltek

W tej sekcji zobaczysz, jak całkowicie odmienić wygląd kontroltek bez utraty ich funkcjonalności. Dowiesz się również, jak prezentować dane w listach za pomocą niestandardowych układów.

37.1. DataTemplate

DataTemplate (Szablon Danych) służy do określenia, w jaki sposób dany obiekt C# (np. klasa **Osoba**) powinien zostać wyświetlony na ekranie. Zamiast widzieć suchą nazwę obiektu wypisaną przez metodę **ToString()**, tworzysz strukturę z przyciskami, obrazkami i tekstami, w które wstrzykujesz właściwości obiektu za pomocą Bindingu.

MainWindow.xaml:

```

<Window x:Class="WpfDataTemplate.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="DataTemplate" Height="450" Width="800">
    <Window.Resources>
        <!-- DataTemplate określa wygląd każdego elementu na liście -->
        <DataTemplate x:Key="PersonTemplate">
            <Border BorderBrush="Gray" BorderThickness="1" Margin="5" Padding="10" CornerRadius="5">
                <StackPanel Orientation="Horizontal">
                    <Ellipse Width="40" Height="40" Fill="LightBlue" Margin="0,0,10,0"/>
                    <StackPanel>
                        <TextBlock Text="{Binding Name}" FontWeight="Bold" FontSize="16"/>
                        <TextBlock Text="{Binding Role}" FontStyle="Italic" Foreground="DarkGray"/>
                    </StackPanel>
                </StackPanel>
            </Border>
        </DataTemplate>
    </Window.Resources>

```

```

<Grid>
    <!-- Przypinamy szablon do właściwości ItemTemplate listy -->
    <ListBox x:Name="PersonsList" ItemTemplate="{StaticResource PersonTemplate}" Margin="10" />
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.Generic;
using System.Windows;

namespace WpfDataTemplate
{
    public class Person
    {
        public string Name { get; set; }
        public string Role { get; set; }
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // Przygotowujemy dane do wyświetlenia
            var people = new List<Person>
            {
                new Person { Name = "Jan Kowalski", Role = "Programista" },
                new Person { Name = "Anna Nowak", Role = "Grafik" },
                new Person { Name = "Tomasz Kot", Role = "Kierownik" }
            };

            PersonsList.ItemsSource = people;
        }
    }
}

```

Kluczowe elementy:

- **ItemTemplate** – Właściwość kontrolki **ListBox** i **ComboBox**, która przyjmuje obiekt **DataTemplate**.
- **Układ wewnątrz szablonu** – Możesz użyć Gridów, StackPanelów, obrazków, czegokolwiek potrzebujesz, aby obiekt wyglądał dobrze.
- **Binding w szablonie** – Odwołuje się bezpośrednio do właściwości pojedynczego obiektu z listy.

37.2. ControlTemplate

O ile **DataTemplate** zmienia wygląd wyświetlanych *danych*, o tyle **ControlTemplate** (Szablon Kontrolki) pozwala na całkowitą zmianę wizualnej reprezentacji samej *kontrolki*. Np. z prostokątnego, szarego przycisku (Button) możesz zrobić okrągły, mieniący się kolorami kształt, który wciąż zachowuje się jak przycisk (można w niego kliknąć).

MainWindow.xaml:

```

<Window x:Class="WpfControlTemplate.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="ControlTemplate" Height="450" Width="800">
    <Window.Resources>
        <Style x:Key="RoundButtonStyle" TargetType="Button">
            <Setter Property="Template">
                <Setter.Value>
                    <!-- Całkowicie wymieniamy to, co kontrolka ma w środku -->

```

```

        <ControlTemplate TargetType="Button">
            <Grid>
                <Ellipse Fill="Orange" Stroke="DarkOrange" StrokeThickness="3" />
                <!-- ContentPresenter to specjalny tag, który wyświetla zawartość (np. napis na przycisku) -->
                <ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center" />
            </Grid>
        </ControlTemplate>
    </Setter.Value>
</Setter>
</Style>
</Window.Resources>

<StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
    <Button Content="Zwykły" Width="100" Height="40" Margin="10" />
    <Button Content="Okrągły" Width="100" Height="100" Style="{StaticResource RoundButtonStyle}" Margin="10" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfControlTemplate
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Wymiana wnętrza** – Definiując `ControlTemplate`, wyrzucasz stary wygląd i wstawiasz własny kod XAML.
- `ContentPresenter` – Bez tego elementu przycisk straciłby napis podany we właściwości `Content`. Prezentuje on zawartość, którą do kontrolki przypisał użytkownik.
- **Funkcjonalność pozostaje** – Kontrolka nadal reaguje na zdarzenie `Click`.

37.3. TemplateBinding

Gdy tworzysz `ControlTemplate`, nowe wnętrze kontrolki (np. okrąg) z definicji nie ma dostępu do właściwości samej kontrolki (np. jej tła, ramki). Aby połączyć właściwość kształtu z właściwością oryginalnej kontrolki (np. `Background` przycisku), używasz wyrażenia `TemplateBinding`.

MainWindow.xaml:

```

<Window x:Class="WpfTemplateBinding.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="TemplateBinding" Height="450" Width="800">
    <Window.Resources>
        <ControlTemplate x:Key="ColoredButtonTemplate" TargetType="Button">
            <Border CornerRadius="10"
                    Background="{TemplateBinding Background}"
                    BorderBrush="{TemplateBinding BorderBrush}"
                    BorderThickness="{TemplateBinding BorderThickness}">
                <ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center" Margin="{TemplateBinding Padding}" />
            </Border>
        </ControlTemplate>
    </Window.Resources>

```

```

<StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
    <!-- Używamy tego samego szablonu, ale przypisujemy różne tła i ramki -->
    <Button Content="Czerwony" Template="{StaticResource ColoredButtonTemplate}"
        Background="Red" BorderBrush="DarkRed" BorderThickness="2" Padding="20" Margin="10"/>

    <Button Content="Zielony" Template="{StaticResource ColoredButtonTemplate}"
        Background="LightGreen" BorderBrush="DarkGreen" BorderThickness="4" Padding="15" Margin="10"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfTemplateBinding
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Uproszczony Binding** – **TemplateBinding** to bardzo szybka, odchudzona wersja zwykłego Bindingu, zaprojektowana tylko do użycia w szablonach kontrolki.
- **Elastyczność** – Dzięki **TemplateBinding** szablony stają się uniwersalne. Jeden szablon reaguje na różne ustawienia koloru zdefiniowane z zewnątrz kontrolki.

37.4. Triggery w stylach

Triggery (Wyzwalacze) pozwalają zmieniać właściwości kontrolki automatycznie, gdy zajdzie jakaś zmiana w jej stanie, np. gdy najedziesz na nią myszką (**IsMouseOver="True"**) albo gdy przycisk zostanie wciśnięty (**IsPressed="True"**). Dzięki temu ożywasz interfejs bez pisania kodu w C#.

MainWindow.xaml:

```

<Window x:Class="WpfTriggers.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Triggery" Height="450" Width="800">
    <Window.Resources>
        <Style x:Key="HoverButtonStyle" TargetType="Button">
            <!-- Wartości domyślne -->
            <Setter Property="Background" Value="LightBlue" />
            <Setter Property="Foreground" Value="Black" />
            <Setter Property="FontSize" Value="16" />

            <Style.Triggers>
                <!-- Zmiana koloru, gdy mysz znajdzie się nad przyciskiem -->
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Background" Value="Blue" />
                    <Setter Property="Foreground" Value="White" />
                </Trigger>

                <!-- Reakcja na kliknięcie (wciśnięcie przycisku) -->
                <Trigger Property="IsPressed" Value="True">
                    <Setter Property="Background" Value="DarkBlue" />

```

```

        <!-- Dodatkowy efekt, np. powiększenie testu podczas kliku -->
        <Setter Property="FontSize" Value="18" />
    </Trigger>
</Style.Triggers>
</Style>
</Window.Resources>

<Grid>
    <Button Content="Najedź na mnie i kliknij!"
        Style="{StaticResource HoverButtonStyle}"
        Width="250" Height="60" />
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfTriggers
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Czysty XAML** – Efekty wizualne wywoływane w pełni w kodzie XAML.
- **Stan kontrolki** – Odpytujesz kontrolkę o jej aktualny stan właściwości (np. `IsMouseOver`, `IsFocused`).
- **Kolejność ma znaczenie** – Jeśli dwa triggerzy się wykluczają (np. zmieniają to samo tło), ten zapisany niżej w kodzie ma wyższy priorytet.

37.5. Proste motywy kontroltek

Łącząc Style, ControlTemplate, TemplateBinding i Triggerzy w jedną całość, jesteś w stanie stworzyć pełnoprawny, profesjonalny wygląd dla kontroltek we własnym systemie wizualnym (tzw. "Theme" lub Motyw). Możesz całkowicie zastąpić standardowy, szary wygląd Windows.

MainWindow.xaml:

```

<Window x:Class="WpfControlTheme.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Motywy kontroltek" Height="450" Width="800">
    <Window.Resources>
        <Style TargetType="Button" x:Key="ModernButton">
            <Setter Property="Background" Value="#FF4CAF50" />
            <Setter Property="Foreground" Value="White" />
            <Setter Property="Padding" Value="15,10" />
            <Setter Property="BorderThickness" Value="0" />
            <Setter Property="Cursor" Value="Hand" />
            <Setter Property="Template">
                <Setter.Value>
                    <ControlTemplate TargetType="Button">
                        <Border x:Name="btnBorder" Background="{TemplateBinding Background}" CornerRadius="20">
                            <ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center" Margin="{TemplateBinding Padding}" />
                        </Border>
                    <ControlTemplate.Triggers>
                        <Trigger Property="IsMouseOver" Value="True">

```

```

        <Setter TargetName="btnBorder" Property="Background" Value="#FF45A049" />
    </Trigger>
    <Trigger Property="IsPressed" Value="True">
        <Setter TargetName="btnBorder" Property="Background" Value="#FF2E7D32" />
    </Trigger>
    <!-- Jeśli wyłączymy przycisk, traci na wyrazistości -->
    <Trigger Property="IsEnabled" Value="False">
        <Setter TargetName="btnBorder" Property="Background" Value="#FFBDBDBD" />
    </Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
</Window.Resources>

<StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
    <Button Content="Zaloguj się" Style="{StaticResource ModernButton}" Margin="10" />
    <Button Content="Anuluj akcję" Style="{StaticResource ModernButton}" Margin="10" />
    <Button Content="Niedostępny" Style="{StaticResource ModernButton}" IsEnabled="False" Margin="10" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfControlTheme
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **TargetName** w triggerach – Wewnątrz szablonu (ControlTemplate) mamy własne nazwane elementy (np. `x:Name="btnBorder"`). Trigger używa **TargetName**, by wiedzieć, który wewnętrzny element zaktualizować.
- **Kompletność** – Rozwiązanie uwzględnia tło, krągłości, stany najechania, naciśnięcia i wyłączenia.

37.6. Różnica między stylem, szablonem i zasobem

Bardzo często te trzy pojęcia się mieszają, ponieważ są ściśle powiązane i współpracują w tym samym fragmencie kodu. Warto raz na zawsze ustalić, za co konkretnie odpowiada każde z nich.

1. **Zasób (Resource)** To worek, magazynek na powtarzalne obiekty. Jeśli stworzysz pędzel (`Brush`) i wpiszesz mu `x:Key`, staje się on zasobem. Sam zasób nic nie zmienia na ekranie, dopóki ktoś go nie wywoła za pomocą `{StaticResource ...}`.
2. **Styl (Style)** To zbiór właściwości przypisanych naraz. Styl to instrukcja mówiąca: "Zmień kolor czcionki na biały, a tło na niebieskie". Styl może korzystać z zasobów.
3. **Szablon (Template)** Szablon to fizyczna architektura, "kości" i struktura elementów potomnych. Zmienia całkowicie schemat wewnętrzny kontrolki (np. rzuca kółko zamiast kwadratu). Szablon bardzo często definiuje się wewnątrz Stylu (jako `Setter` właściwości `Template`), żeby móc go nakładać na wiele elementów na raz.

MainWindow.xaml:

```

<Window x:Class="WpfDifferences.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

```



```

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
Title="Różnice" Height="450" Width="800">
<Window.Resources>
    <!-- 1. TO JEST ZASÓB (Color i Brush) -->
    <SolidColorBrush x:Key="DangerBrush" Color="Red" />

    <!-- 2. TO JEST STYL (Grupowanie właściwości, wywołanie zasobu w Setterze) -->
    <Style x:Key="WarningLabelStyle" TargetType="Label">
        <Setter Property="Foreground" Value="{StaticResource DangerBrush}" />
        <Setter Property="FontSize" Value="24" />
        <Setter Property="FontWeight" Value="Bold" />

    <!-- 3. TO JEST SZABLON (zmienia fizyczną budowę kontrolki, zagnieżdżony w Stylu) -->
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="Label">
                <Border BorderBrush="{TemplateBinding Foreground}" BorderThickness="2" Padding="5">
                    <StackPanel Orientation="Horizontal">
                        <TextBlock Text="! UWAGA !" Foreground="{TemplateBinding Foreground}" />
                        <ContentPresenter />
                    </StackPanel>
                </Border>
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>
</Window.Resources>

<Grid>
    <!-- Zastosowanie całego pakietu do konkretnej kontrolki -->
    <Label Content="Błąd serwera bazy danych"
        Style="{StaticResource WarningLabelStyle}"
        HorizontalAlignment="Center" VerticalAlignment="Center" />
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfDifferences
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Czytelna separacja** – Używanie tych trzech pojęć zgodnie z ich przeznaczeniem sprawia, że kod XAML, choć jest długi, staje się bardzo logiczny i zorganizowany.
- **Hermetyzacja** – Styl ukrywa całą złożoność kontrolki, a widok końcowy z punktu widzenia programisty pozostaje krótki (`<Label ... Style="..." />`).

38. Drzewo logiczne i wizualne

W tej sekcji zobaczysz, jak platforma WPF zarządza elementami pod spodem interfejsu. Zrozumienie struktury "drzewa" w WPF pozwala pojąć, dlaczego właściwości są dziedziczone oraz w jaki sposób program szuka stylów i zasobów.

38.1. Logical tree

Drzewo logiczne (Logical Tree) to struktura, którą piszesz w swoim kodzie XAML. Przedstawia ono widok z lotu ptaka – główne kontrolki aplikacji. Każdy węzeł to po prostu element, który dodajesz na ekran, np. `Window` zawiera `Grid`, a w nim znajduje się `Button`.

MainWindow.xaml:

```
<Window x:Class="WpfLogicalTree.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Logical Tree" Height="450" Width="800">
    <!-- Drzewo logiczne: Window -> Grid -> StackPanel -> TextBox i Button -->
    <Grid>
        <StackPanel Margin="50">
            <TextBox x:Name="MyTextBox" Text="Drzewo logiczne to kod XAML" FontSize="20" />
            <Button Content="Pokaż rodzica logicznego" Click="ShowParent_Click" Margin="0,10,0,0" Padding="10"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;
using System.Windows.Controls;

namespace WpfLogicalTree
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void ShowParent_Click(object sender, RoutedEventArgs e)
        {
            Button btn = sender as Button;

            // LogicalTreeHelper pozwala przemieszczać się w górę i w dół drzewa
            DependencyObject parent = LogicalTreeHelper.GetParent(btn);

            if (parent != null)
            {
                MessageBox.Show("Rodzicem logicznym przycisku jest: " + parent.GetType().Name);
            }
        }
    }
}
```

Kluczowe elementy:

- `LogicalTreeHelper` – Klasa wbudowana w WPF pozwalająca nawigować po strukturze logicznej.
- Prostota** – Drzewo logiczne odzwierciedla to, co zadeklarował programista. Rodzicem przycisku jest `StackPanel`.

38.2. Visual tree

Drzewo wizualne (Visual Tree) to rozwinięcie drzewa logicznego o elementy najniższego poziomu. O ile przycisk w XAML to tylko jedno słowo `<Button>`, o tyle w drzewie wizualnym zostaje on rozłożony na mniejsze części rysunkowe (granice, teksty, obrysy, ContentPresenter). Każdy element wyświetlany na

ekranie pochodzi od klasy `Visual`.

MainWindow.xaml:

```
<Window x:Class="WpfVisualTree.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Visual Tree" Height="450" Width="800">
    <StackPanel Margin="50">
        <Button x:Name="MyButton" Content="Drzewo wizualne" FontSize="20" Padding="10" Click="AnalyzeVisualTree_Click" />
        <TextBlock x:Name="ResultText" Margin="0,20,0,0" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;

namespace WpfVisualTree
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void AnalyzeVisualTree_Click(object sender, RoutedEventArgs e)
            {
                string results = "Pod spodem przycisku kryją się m.in.:\n";

                // VisualTreeHelper pozwala zbadać ukryte wnętrza kontrolki
                int childCount = VisualTreeHelper.GetChildrenCount(MyButton);

                for (int i = 0; i < childCount; i++)
                {
                    var child = VisualTreeHelper.GetChild(MyButton, i);
                    results += $"- {child.GetType().Name}\n";

                    // MyButton domyślnie posiada element typu ButtonChrome lub Border,
                    // a w środku niego ukryty jest ContentPresenter.
                }

                ResultText.Text = results;
            }
        }
    }
}
```

Kluczowe elementy:

- `VisualTreeHelper` – Narzędzie badające wyrenderowaną, pełną konstrukcję kontrolki na ekranie.
- **Głębokość** – Drzewo wizualne jest kilkukrotnie większe i głębsze od logicznego, ponieważ ujawnia całą mechanikę rysowania (obramowania, efekty cieni itp.).
- **Wydajność** – Skomplikowane drzewo wizualne zużywa więcej pamięci, dlatego czasami optymalizuje się układy.

38.3. Dziedziczenie właściwości

Niektóre właściwości w WPF przenoszą się w dół na elementy potomne. To mechanizm zwany dziedziczeniem właściwości (Property Value Inheritance). Jeśli ustawisz powiększoną czcionkę głównemu elementowi okna (np. `Window`), wszystkie przyciski i etykiety w tym oknie również się powiększą, chociaż ich nie

konfigurowałeś.

MainWindow.xaml:

```
<Window x:Class="WpfPropertyInheritance.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Dziedziczenie właściwości" Height="450" Width="800"
        FontFamily="Courier New" FontSize="18" Foreground="DarkBlue">
    <!-- Grid oraz wszystkie dzieci dziedziczą czcionkę i kolor z elementu Window -->
    <Grid>
        <StackPanel Margin="20">
            <TextBlock Text="Dziedziczę czcionkę z głównego okna!" />
            <Button Content="Ja też dziedziczę!" Margin="0,10,0,0" Padding="5"/>

            <!-- Tu łamiemy dziedziczenie, nadpisując właściwość lokalnie -->
            <TextBlock Text="Ja nadpisuję domyślny rozmiar" FontSize="30" Foreground="Red" Margin="0,20,0,0"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace WpfPropertyInheritance
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Kluczowe elementy:

- **Rodzaj drzewa** – Dziedziczenie odbywa się na bazie Drzewa Logicznego (w dół).
- **Właściwości tekstowe** – Najczęściej dziedziczone są: `FontSize`, `FontFamily`, `FontWeight`, `Foreground`. Ale uwaga: `Background` (tło) nie dziedziczy w dół.
- **Priorytety** – Jeśli zdefiniujesz lokalną wartość np. na `TextBox`, jest ona ważniejsza niż wartość spływająca od rodzica (lokalne nadpisanie).

38.4. Szukanie zasobów

Gdy deklarujesz użycie zasobu kluczem np. `{StaticResource MyBrush}`, WPF musi wiedzieć, gdzie ten `MyBrush` się znajduje. Robi to, przeszukując w górę drzewo logiczne, by znaleźć zadeklarowany styl czy pędzel. Przeszukanie idzie w kierunku od dziecka do korzenia i kończy się na zasobach aplikacji (`App.xaml`).

MainWindow.xaml:

```
<Window x:Class="WpfResourceLookup.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Szukanie zasobów" Height="450" Width="800">
    <Window.Resources>
        <SolidColorBrush x:Key="AppColor" Color="Red" />
    </Window.Resources>

    <StackPanel>
        <StackPanel.Resources>
```

```

        <!-- Tutaj nadpisujemy klucz AppColor lokalnie w StackPanel -->
        <SolidColorBrush x:Key="AppColor" Color="Green" />
    </StackPanel.Resources>

    <!-- Poniższy element szuka klucza AppColor. Znajdzie go w najbliższym
    element-rodzicu, czyli w StackPanel, więc przycisk będzie zielony, a nie czerwony. -->
    <Button Background="{StaticResource AppColor}" Content="Kliknij" Margin="50" Padding="10"/>
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfResourceLookup
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Algorytm szukania** – 1. Element lokalny, 2. Rodzic kontrolki (np. StackPanel), 3. Okno (Window), 4. Plik główny App.xaml.
- **Zatrzymanie na pierwszym znalezieniu** – WPF zawsze wykorzysta najbliższy znaleziony słownik zasobów. Jeśli znajdzie klucz w StackPanel, w ogóle nie sprawdzi w zasobach okna.

38.5. Wpływ drzewa na style

Podobnie jak zasoby, również i style wyszukiwane są wzdłuż struktury drzewa. Dotyczy to w szczególności tzw. "stylów ukrytych" (bez podanego klucza `x:Key`), które same z siebie poszukują kontrolki w drzewie o podanym `TargetType`.

MainWindow.xaml:

```

<Window x:Class="WpfTreeStyles.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Drzewo a style" Height="450" Width="800">
    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>

        <!-- Kolumna lewa (czerwone przyciski) -->
        <StackPanel Grid.Column="0" Margin="10">
            <StackPanel.Resources>
                <Style TargetType="Button">
                    <Setter Property="Background" Value="LightCoral" />
                    <Setter Property="Margin" Value="5" />
                </Style>
            </StackPanel.Resources>

            <Button Content="Lewy 1" />
            <Button Content="Lewy 2" />
        </StackPanel>
    </Grid>
</Window>

```

```

<!-- Kolumna prawa (niebieskie przyciski) -->
<StackPanel Grid.Column="1" Margin="10">
    <StackPanel.Resources>
        <Style TargetType="Button">
            <Setter Property="Background" Value="LightBlue" />
            <Setter Property="Margin" Value="5" />
        </Style>
    </StackPanel.Resources>

    <Button Content="Prawy 1" />
    <Button Content="Prawy 2" />
</StackPanel>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfTreeStyles
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Zasięg stylów** – Styl zdefiniowany w prawym StackPanelu dotyczy tylko elementów znajdujących się w jego drzewie. Lewy panel o nim nie wie.
- **Organizacja UI** – Jeśli aplikacja dzieli się na pasek narzędzi (ToolBar) i panel główny, możesz zdefiniować różne domyślne zachowania przycisków na podstawie tego, w którym gałęzi drzewa leżą.

38.6. Wpływ drzewa na binding

Głównym zadaniem Drzewa Logicznego z perspektywy mechanizmu Bindingu jest przekazywanie obiektu bindowania (DataContext). Kiedy nadajesz **DataContext** na oknie aplikacji, wędruje on w dół do każdego elementu w interfejsie. Z kolei Drzewo Wizualne przydaje się, by w złożonych kontrolkach w górę lub w dół namierzyć kontrolkę.

MainWindow.xaml:

```

<Window x:Class="WpfTreeBinding.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Wpływ drzewa na binding" Height="450" Width="800">
    <StackPanel Margin="20">
        <!-- Zwykły bindowanie szuka źródła danych, które wpada przez DataContext -->
        <TextBlock Text="Dziedziczenie DataContext w drzewie:" FontWeight="Bold" />
        <TextBlock Text="{Binding SystemName}" FontSize="20" Foreground="Purple" Margin="0,5,0,20"/>

        <TextBlock Text="Szukanie elementu po drzewie wizualnym (RelativeSource):" FontWeight="Bold" />
        <!-- Szukamy w górę drzewa pierwszego rodzica, który jest typu Window i bierzemy od niego Title -->
        <TextBlock Text="{Binding RelativeSource={RelativeSource AncestorType=Window}, Path=Title}"
            FontSize="20" Foreground="Orange" Margin="0,5,0,0"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfTreeBinding
{
    public class SystemData
    {
        public string SystemName { get; set; } = "Aplikacja Sprzedażowa v1.0";
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // Przypisanie obiektu do okna - wędruje automatycznie do TextBlocków
            this.DataContext = new SystemData();
        }
    }
}

```

Kluczowe elementy:

- **Przekazywanie `DataContext`** – Po przypisaniu go do korzenia drzewa (okna), całe drzewo dziedziczy z niego właściwości i pozwala robić krótkie bindingi bez deklarowania źródeł w każdym przycisku.
- **`RelativeSource AncestorType`** – Służy do celowego podróżowania po drzewie w górę (do rodziców), aby wydobyć stamtąd potrzebną wartość podczas pisania kodu XAML.

39. DependencyProperty i właściwości WPF

W tej sekcji zobaczysz serce architektury WPF, czyli Właściwości Zależności (Dependency Properties). Dzięki nim możliwe są zaawansowane funkcje, takie jak wiązanie danych (Binding), style, szablony, czy powiadamianie o zmianach.

39.1. Czym jest DependencyProperty

W WPF większość właściwości w kontrolkach wizualnych to nie są zwykłe zmienne w C#. Kiedy ustawiasz tło w przycisku (`Background="Red"`), WPF zapisuje to w specjalnym systemie Dependency Properties (właściwości zależnych). System ten analizuje, skąd ma brać wartość: czy z kodu, czy z Bindingu, a może z nałożonej animacji albo lokalnego stylu. Przez to jedna "właściwość" staje się systemem reagującym na zależności z zewnątrz.

MainWindow.xaml:

```

<Window x:Class="WpfDependencyBasic.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Dependency Property" Height="450" Width="800">
    <StackPanel Margin="50">
        <!-- Wszystkie pokazane tu atrybuty to de facto Właściwości Zależności -->
        <TextBox x:Name="MyInput" Text="Uczymy się Dependency Properties!" FontSize="20" Background="Yellow" />
        <Button Content="Sprawdź" Margin="0,20,0,0" Padding="10" Click="CheckValue_Click"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Controls;

namespace WpfDependencyBasic
{

```

```

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    private void CheckValue_Click(object sender, RoutedEventArgs e)
    {
        // Możemy odczytać wartość z Dependency Property bezpośrednio poprzez metodę klasową
        object backgroundValue = MyInput.GetValue(TextBox.BackgroundProperty);

        MessageBox.Show("Aktualne tło pochodzące z Dependency Property to: " + backgroundValue.ToString());
    }
}

```

Kluczowe elementy:

- **Statyczne pole** – Każda właściwość zależności (np. `.Text` w `TextBox`) ma odpowiadające jej twarde, statyczne pole w klasie z przyrostkiem "Property" (np. `TextBox.TextProperty`).
- **Słownik zamiast pamięci obiektu** – `DependencyProperty` nie zajmuje pamięci dopóki jej nie przypiszesz. Są to po prostu słowniki klucz-wartość, gdzie kluczem jest właściwość, a wartością – konkretna wartość (np. kolor lub tekst).

39.2. Zwykła właściwość C# a właściwość zależności

Dlaczego po prostu nie użyjemy zwykłej właściwości (`public string Text { get; set; }`) w kontrolkach WPF? Zwykła właściwość może być wywołana tylko z C#, musiałbyś przy każdej zmianie ręcznie odpalać powiadomienia (np. `INotifyPropertyChanged`). System nie potrafiłby też rozpoznać, czy wartość w danej chwili ma być zmieniana przez styl (Style), animację czy wiązanie. Właściwość zależności "obsługuje" te wszystkie warstwy za programistę.

MainWindow.xaml:

```

<Window x:Class="WpfDiffProps.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Porównanie właściwości" Height="450" Width="800">
    <StackPanel Margin="50">
        <TextBlock x:Name="StatusText" FontSize="18" Text="Gotowe." />
        <Button Content="Zmień tło animacją" Click="RunAnimation_Click" Margin="0,20,0,0" Padding="10" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Media;
using System.Windows.Media.Animation;

namespace WpfDiffProps
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void RunAnimation_Click(object sender, RoutedEventArgs e)
        {
            // Animacja jest możliwa TYLKO dlatego, że Background to Dependency Property!
            // Zwykła właściwość C# z { get; set; } rzuciłaby błąd, bo animacja nie wie, jak nią sterować.
        }
    }
}

```



```

        var animation = new ColorAnimation(Colors.White, Colors.Red, new Duration(TimeSpan.FromSeconds(1)));

        SolidColorBrush brush = new SolidColorBrush(Colors.White);
        StatusText.Background = brush;

        brush.BeginAnimation(SolidColorBrush.ColorProperty, animation);
    }
}
}

```

Kluczowe elementy:

- **Rozszerzona funkcjonalność** – To właśnie to "rozszerzenie" natywnych właściwości pozwala modułowi animacji kontrolować zmiany wartości z płynnymi przejściami.
- **Rozróżnianie klas** – Standardowe własności są dobre dla warstwy modelu (np. klasa *Osoba*), podczas gdy `DependencyProperty` to wymóg niezbędny dla warstwy widoku (kontrolki XAML).

39.3. Dependency properties w kontrolkach WPF

Gdy przeglądasz dokumentację Microsoftu dla dowolnej kontrolki np. przycisku `Button`, każda z jej wizualnych cech jest odnotowana jako "Dependency property". Pozwala to platformie łączyć właściwości między wieloma elementami bez pisania kodu C#.

MainWindow.xaml:

```

<Window x:Class="WpfControlDepProps.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Właściwości kontrolki" Height="450" Width="800">
    <StackPanel Margin="50">
        <!-- Zwykły suwak -->
        <Slider x:Name="MySlider" Minimum="10" Maximum="100" Value="25" />

        <!-- TextBlock wykorzystuje DependencyProperty (Właściwość: FontSize), aby
             przyjąć wartość od innej DependencyProperty suwaka (Właściwość: Value). -->
        <TextBlock Text="Ten tekst powiększy się razem z suwakiem"
                   FontSize="{Binding ElementName=MySlider, Path=Value}"
                   Margin="0,30,0,0" HorizontalAlignment="Center"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace WpfControlDepProps
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

Kluczowe elementy:

- **Mechanizm Bindingu** – Cały binding w WPF opiera się o fakt, że zarówno element wysyłający wartość, jak i odbierający to Dependency Property.
- **Brak kodu "Behind"** – Otrzymujesz bezpośrednią interakcję użytkownika na UI, nie musisz korzystać ze zdarzenia `ValueChanged` w C# w celu powiększenia tekstu.

39.4. Wartości lokalne, style i binding

System obliczający finalną wartość danej właściwości posługuje się zbiorem priorytetów. Np. przycisk może mieć kolor nałożony z 4 różnych miejsc naraz (lokalnie jako tło, ze stylu w aplikacji, w triggerze animacji, lub poprzez system operacyjny Windows). Która wartość wygra?

MainWindow.xaml:

```
<Window x:Class="WpfPrecedence.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kolejność priorytetów" Height="450" Width="800">

    <Window.Resources>

        <!-- PRIORYTET 3: Styl globalny dla przycisku ustawia tło na Niebieski -->

        <Style TargetType="Button">

            <Setter Property="Background" Value="Blue" />

        </Style>

    </Window.Resources>

    <StackPanel Margin="20">

        <!-- Zastosuje się kolor ze stylu (Niebieski), bo nic innego nie walczy o to tło -->

        <Button Content="Przycisk oparty o styl" Height="40" Margin="10" />

        <!-- PRIORYTET 1: Wartość lokalna. Wygrywa ze Stylem! -->
        <!-- Ten przycisk będzie Czerwony, mimo zdefiniowanego Niebieskiego stylu -->

        <Button Content="Przycisk oparty o wartość lokalną" Background="Red" Height="40" Margin="10" />

        <!-- PRIORYTET 2: Animacja/Trigger. Wygra z wartością lokalną! -->

        <Button Content="Test Triggerów" Background="Green" Height="40" Margin="10">

            <Button.Style>

                <Style TargetType="Button">

                    <Style.Triggers>

                        <!-- Po najechaniu tło stanie się Żółte (najwyższy priorytet podczas najechania myszy) -->

                        <Trigger Property="IsMouseOver" Value="True">

                            <Setter Property="Background" Value="Yellow" />

                        </Trigger>

                    </Style.Triggers>

                </Style>

            </Button.Style>

        </Button>

    </StackPanel>

</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace WpfPrecedence
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Kluczowe elementy:

- **System priorytetów** – Kolejność od najważniejszych: 1. Animacje (Active animations), 2. Lokalne modyfikatory (np. tło wpisane wprost w tagu Button), 3. Triggery stylu, 4. Style z resources.

- **Nie tam stylizacji twardym zapisem** – Właśnie dlatego tak groźne jest przypisanie koloru w kodzie HTML/XAML bezpośrednio (np. `Background="Red"`). Wtedy zablokujesz możliwość jego zmiany globalnym stylem!

39.5. Tworzenie własnej prostej właściwości zależności

Gdy piszesz własną, niestandardową kontrolkę (np. powiadomienie-baner, własny przełącznik on/off), do obsługi jej unikalnych cech potrzebujesz stworzyć własną Właściwość Zależności. Tylko w ten sposób umożliwić użytkownikom kontrolki swobodne bindowanie do jej zmiennych w kodzie XAML.

CustomLabel.cs:

```
using System.Windows;
using System.Windows.Controls;

namespace WpfCustomDependency
{
    // Budujemy własną etykietę, która odziedziczy po TextBox, ale będzie miała nową właściwość: IsImportant
    public class CustomLabel : TextBox
    {
        // KROK 1: Rejestrujemy właściwość zależności. Nazwa konwencji MUSI kończyć się na "Property".
        public static readonly DependencyProperty IsImportantProperty =
            DependencyProperty.Register(
                "IsImportant",           // Nazwa w XAML
                typeof(bool),           // Typ danych
                typeof(CustomLabel),     // Klasa właściciela
                new PropertyMetadata(false) // Wartość domyślna
            );

        // KROK 2: Tworzymy zwykłą nakładkę właściwości C# (tzw. Wrapper), ułatwiającą odczyt.
        public bool IsImportant
        {
            get { return (bool)GetValue(IsImportantProperty); }
            set { SetValue(IsImportantProperty, value); }
        }
    }
}
```

MainWindow.xaml:

```
<Window x:Class="WpfCustomDependency.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:WpfCustomDependency"
        Title="Własne Dependency Property" Height="450" Width="800">
    <StackPanel Margin="50">
        <!-- Sprawdzenie Bindingu: do własnej właściwości "IsImportant" przepinamy CheckBox -->
        <CheckBox x:Name="ImportanceCheck" Content="Oznacz jako ważne" Margin="0,0,0,10" />

        <!-- Własna kontrolka -->
        <local:CustomLabel Text="Jestem niestandardowym komponentem!"
                        IsImportant="{Binding ElementName=ImportanceCheck, Path=IsChecked}"
                        FontSize="20" Padding="5" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace WpfCustomDependency
{
    // ...
```

```

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
}
}

```

Kluczowe elementy:

- **DependencyProperty.Register** – Metoda fabrykująca Właściwość Zależności, z którą jądro WPF będzie "rozmawiać".
- **Wrapper GetValue/SetValue** – Podstawowy szkielet, bez żadnego dodawania własnej logiki obok wywołań bazowych (inaczej WPF to zignoruje i przeskoczy przez XAML).

40. CollectionView i prezentacja kolekcji

W tej sekcji poznasz potężny mechanizm zarządzania sposobem wyświetlania kolekcji w WPF. Zobaczysz, jak w czasie rzeczywistym filtrować, sortować i grupować dane bez dotykania oryginalnej listy w kodzie. Omówimy to na przykładzie mini-aplikacji do zarządzania kontaktami w firmie.

40.1. Po co używać widoku kolekcji

Kiedy przekazujesz kontrolce **ListBox** lub **DataGrid** listę elementów (np. pobraną z bazy danych w **List<T>** lub **ObservableCollection<T>**), wyświetla ona po prostu dane w takiej kolejności, w jakiej je dodałeś. Co jeśli użytkownik chce je wyszukać, posortować alfabetycznie lub ukryć nieaktywnych pracowników? Mógłbyś oczywiście stworzyć nową listę, na niej posortować elementy i podpiąć ją z powrotem do kontrolki. To jednak niepotrzebnie dubluje pamięć i potrafi "zerwać" synchronizację przy aktualizacji danych. WPF rozwiązuje to poprzez tworzenie przezroczystej "szyby" pomiędzy prawdziwymi danymi, a interfejsem graficznym. Ta "szyba" to właśnie Widok Kolekcji (Collection View). Patrząc przez szybę, możesz odwracać, ukrywać i grupować elementy – podczas gdy pod spodem oryginalne dane się nie zmieniają.

40.2. ICollectionView

Interfejs **ICollectionView** to mechanizm zaszyty w WPF, który pozwala na modyfikację strumienia danych. Kiedy podpinasz **ObservableCollection<T>** do listy **ItemsSource**, WPF i tak niejawnie owija te dane w widok. Możesz wyciągnąć referencję do tego automatycznego widoku z użyciem klasy **CollectionViewSource** i zacząć go programowo obsługiwać.

MainWindow.xaml:

```

<Window x:Class="WpfCollectionView.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Pracownicy firmy" Height="450" Width="800">
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <StackPanel Orientation="Horizontal" Grid.Row="0" Margin="0,0,10,10">
            <Button Content="Sortuj Alfabetycznie" Click="Sort_Click" Padding="10" Margin="0,0,10,0"/>
            <Button Content="Pokaż tylko IT" Click="Filter_Click" Padding="10" Margin="0,0,10,0"/>
            <Button Content="Zresetuj widok" Click="Reset_Click" Padding="10" />
        </StackPanel>

        <!-- Lista bindowana do właściwości kodu -->
        <ListBox x:Name="EmployeesList" Grid.Row="1" DisplayMemberPath="Name" FontSize="16" />
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.ObjectModel;
using System.ComponentModel;

```

```

using System.Windows;
using System.Windows.Data;

namespace WpfCollectionView
{
    public class Employee
    {
        public string Name { get; set; }
        public string Department { get; set; }
    }

    public partial class MainWindow : Window
    {
        private ObservableCollection<Employee> _employees;
        private ICollectionView _view;

        public MainWindow()
        {
            InitializeComponent();

            // 1. To jest nasza PRAWDZIWA i niezmienna kolekcja
            _employees = new ObservableCollection<Employee>
            {
                new Employee { Name = "Zuzanna Kowalska", Department = "HR" },
                new Employee { Name = "Adam Nowak", Department = "IT" },
                new Employee { Name = "Bartosz Wójcik", Department = "IT" },
                new Employee { Name = "Marta Wiśniewska", Department = "Finanse" }
            };

            // 2. Podpinamy twarde dane do kontrolki
            EmployeesList.ItemsSource = _employees;

            // 3. Wyciągamy DOMYŚLNY widok przypisany do tej samej kolekcji
            _view = CollectionViewSource.GetDefaultView(_employees);
        }

        private void Sort_Click(object sender, RoutedEventArgs e)
        {
            _view.SortDescriptions.Clear();
            _view.SortDescriptions.Add(new SortDescription("Name", ListSortDirection.Ascending));
        }

        private void Filter_Click(object sender, RoutedEventArgs e)
        {
            _view.Filter = (obj) =>
            {
                Employee emp = obj as Employee;
                return emp.Department == "IT"; // True oznacza: Pokaż element
            };
        }

        private void Reset_Click(object sender, RoutedEventArgs e)
        {
            _view.SortDescriptions.Clear();
            _view.Filter = null; // Ściągnięcie filtrów
        }
    }
}

```

Kluczowe elementy:

- `CollectionViewSource.GetDefaultView` – Metoda pobierająca jedyny wspólny widok dla podanej kolekcji danych w systemie.

- **Jedno źródło prawdy** – W każdej metodzie nie ruszasz obiektu `_employees`, używasz jedynie `_view`.

40.3. CollectionViewSource

Aby nie pisać zarządzania sortowaniem w C# z użyciem `ICollectionView`, platforma WPF udostępnia obiekt `CollectionViewSource`. Pozwala on zadeklarować Widok Kolekcji bezpośrednio w kodzie XAML jako zasób (Resource). Daje to potężną możliwość bindowania kontrolki interfejsu (np. pole wyszukiwarki) bezpośrednio do widoku, minimalizując logikę kontrolera (code-behind).

MainWindow.xaml:

```
<Window x:Class="WpfCVS.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:scm="clr-namespace:System.ComponentModel;assembly=WindowsBase"
        Title="CollectionViewSource" Height="450" Width="800">

    <Window.Resources>

        <!-- Deklarujemy obiekt widoku i jako źródło bierzemy "RawData" (przygotowane w kodzie) -->
        <CollectionViewSource x:Key="SortedView" Source="{Binding RawData}">

            <!-- Tu definiujemy sortowanie bez jednej linijki C#! -->
            <CollectionViewSource.SortDescriptions>

                <scm:SortDescription PropertyName="Department" Direction="Ascending" />

                <scm:SortDescription PropertyName="Name" Direction="Descending" />

            </CollectionViewSource.SortDescriptions>

        </CollectionViewSource>

    </Window.Resources>

    <Grid>

        <!-- Kontrolka zamiast do surowych danych wpina się do Sformatowanego Widoku -->
        <ListBox ItemsSource="{Binding Source={StaticResource SortedView}}"
                DisplayMemberPath="Name" Margin="20" />

    </Grid>

</Window>
```

MainWindow.xaml.cs:

```
using System.Collections.ObjectModel;
using System.Windows;

namespace WpfCVS
{
    public partial class MainWindow : Window
    {
        public ObservableCollection<Employee> RawData { get; set; }

        public MainWindow()
        {
            InitializeComponent();

            RawData = new ObservableCollection<Employee>
            {
                new Employee { Name = "Zuzanna", Department = "IT" },
                new Employee { Name = "Adam", Department = "HR" },
                new Employee { Name = "Bartosz", Department = "IT" }
            };

            this.DataContext = this;
        }
    }

    public class Employee
    {
        public string Name { get; set; }
    }
}
```

```

        public string Department { get; set; }
    }
}

```

Kluczowe elementy:

- **Przeniesienie ciężaru na XAML** – Definiowanie widoku oraz reguł w tagu powiększa czytelność i czystość w pliku z kodem C#.
- **Source** zamiast **ItemsSource** – Zauważ, że **ListBox** przyjmuje podłączony zasób widoku przez **Binding Source**.

40.4. Sortowanie danych w widoku

Sortowanie opiera się na klasie **SortDescription**. Dodajesz ją do kolekcji **SortDescriptions** w Twoim obiekcie widoku. Zobaczysz natychmiastowe przemodelowanie elementów w interfejsie. Jeśli dodasz nowe osoby do **ObservableCollection**, nowy element zostanie tam umieszczony, ale interfejs wpisze go od razu do dobrego sortowania w widoku!

MainWindow.xaml:

```

<Window x:Class="WpfSorting.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Sortowanie" Height="450" Width="800">
    <StackPanel Margin="20">
        <Button Content="Posortuj" Click="ApplySort_Click" Width="150" Margin="10"/>
        <ListBox x:Name="MyList" DisplayMemberPath="Name" Height="150" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Windows;
using System.Windows.Data;

namespace WpfSorting
{
    public class Item
    {
        public string Name { get; set; }
        public int Price { get; set; }
    }

    public partial class MainWindow : Window
    {
        public ObservableCollection<Item> Products { get; set; }
        private ICollectionView view;

        public MainWindow()
        {
            InitializeComponent();
            Products = new ObservableCollection<Item>
            {
                new Item { Name = "Mysz", Price = 50 },
                new Item { Name = "Klawiatura", Price = 150 },
                new Item { Name = "Słuchawki", Price = 250 }
            };

            MyList.ItemsSource = Products;
            view = CollectionViewSource.GetDefaultView(Products);
        }
    }
}

```

```

private void ApplySort_Click(object sender, RoutedEventArgs e)
{
    // Można podpiąć wiele reguł: Najpierw sortuj po cenie, potem po nazwie
    view.SortDescriptions.Clear();
    view.SortDescriptions.Add(new SortDescription("Price", ListSortDirection.Descending));
    view.SortDescriptions.Add(new SortDescription("Name", ListSortDirection.Ascending));
}
}
}

```

Kluczowe elementy:

- **Wielokrotne sortowanie** – Do listy `SortDescriptions` można łądować setki warunków (tzw. sortowanie po wielu kolumnach).
- **Resetowanie przed sortowaniem** – Przed nałożeniem nowego sortowania upewnij się, że użyłeś metody `.Clear()`, inaczej dodasz kolidujące parametry z poprzednim kliknięciem.

40.5. Filtrowanie danych w widoku

Najczęstszym miejscem użycia `CollectionView` jest pole tekstowe do wyszukiwania zjawisk zwane potocznie "Lupką". Filtr działa na zasadzie wywoływania zdefiniowanego na zewnątrz "predykatu" (czyli funkcji, która sprawdza element i odpowiada True – Pokaż lub False – Ukryj) za każdym razem, kiedy wywołasz odświeżenie.

MainWindow.xaml:

```

<Window x:Class="WpfFiltering.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Wyszukiwanie" Height="450" Width="800">
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <!-- Pole wpisywania -->
        <TextBox x:Name="SearchBox" TextChanged="SearchBox_TextChanged" Grid.Row="0" Margin="0,0,0,10"
                FontSize="16" ToolTip="Wpisz szukany tekst" />

        <ListBox x:Name="ResultList" Grid.Row="1" DisplayMemberPath="Name" FontSize="16" />
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;

namespace WpfFiltering
{
    public class SimpleItem
    {
        public string Name { get; set; }
    }

    public partial class MainWindow : Window
    {
        private ObservableCollection<SimpleItem> Data;
        private ICollectionView View;
    }
}

```



```
public MainWindow()
{
    InitializeComponent();

    Data = new ObservableCollection<SimpleItem>
    {
        new SimpleItem { Name = "Telefon" },
        new SimpleItem { Name = "Telewizor" },
        new SimpleItem { Name = "Radio" },
        new SimpleItem { Name = "Monitor" }
    };

    ResultList.ItemsSource = Data;
    View = CollectionViewSource.GetDefaultView(Data);

    // Przypisanie metody filtrującej do akcji CollectionView
    View.Filter = SearchPredicate;
}

private bool SearchPredicate(object obj)
{
    var item = obj as SimpleItem;
    if (item == null) return false;

    string searchText = SearchBox.Text.ToLower();

    // Jeśli element zawiera wpisany tekst (ignorując wielkość liter) to się wyświetli
    return item.Name.ToLower().Contains(searchText);
}

private void SearchBox_TextChanged(object sender, TextChangedEventArgs e)
{
    // Gdy wpisujemy znak do TextBox, wymuszamy odświeżenie widoku
    View.Refresh();
}
}
```

Kluczowe elementy:

- **Delegat do metody Filter** – Przekazujesz nazwę metody oceniającej obiekt (np. `SearchPredicate`)
- `View.Refresh()` – Jeśli zmieni się tekst wpisany w wyszukiwarce, musisz poprosić o ponowne sprawdzenie i przejście każdego elementu na liście. To powoduje aktualizację ListBoxa.

40.6. Grupowanie danych

Jeżeli chcesz np. połączyć pracowników w drzewiaste działy (HR podział 1, IT podział 2), Widok Kolekcji używa grup (GroupDescriptions). Do pełnego wizualnego pogrupowania musisz również dopisać XAML-ową strukturę (`GroupStyle`), aby kontrolka wiedziała, jak dorysować podziały i nagłówki przed daną grupą.

MainWindow.xaml:

```
<Window x:Class="WpfGrouping.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Grupowanie" Height="450" Width="800">
    <Grid>
        <ListBox x:Name="GroupedList" Margin="20">
            ◀— Określamy wygląd grupy —▶
            <ListBox.GroupStyle>
                <GroupStyle>
                    ◀— Nagłówek grupy to tekst zawierający pole 'Name' (nazwę tejże grupy) —▶
                    <GroupStyle.HeaderTemplate>
```

```

        <DataTemplate>
            <TextBlock Text="{Binding Name}" FontWeight="Bold" FontSize="18" Foreground="DarkBlue" Margin="0,10,0,5" />
        </DataTemplate>
    </GroupStyle.HeaderTemplate>
</GroupStyle>
</ListBox.GroupStyle>

<!-- Jak wyglądają elementy wewnątrz grupy -->
<ListBox.ItemTemplate>
    <DataTemplate>
        <TextBlock Text="{Binding FullName}" Margin="15,0,0,0" />
    </DataTemplate>
</ListBox.ItemTemplate>
</ListBox>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Collections.ObjectModel;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Windows;
using System.Windows.Data;

namespace Wpf6Grouping
{
    public class Person
    {
        public string FullName { get; set; }
        public string Category { get; set; }
    }

    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            var people = new ObservableCollection<Person>
            {
                new Person { FullName = "Janek", Category = "Zarząd" },
                new Person { FullName = "Anna", Category = "Zarząd" },
                new Person { FullName = "Tomek", Category = "Pracownicy" },
                new Person { FullName = "Magda", Category = "Pracownicy" }
            };

            GroupedList.ItemsSource = people;
            ICollectionView view = CollectionViewSource.GetDefaultView(people);

            // Aktywacja grupowania według pola 'Category'
            view.GroupDescriptions.Add(new PropertyGroupDescription("Category"));
        }
    }
}

```

Kluczowe elementy:

- **PropertyGroupDescription** – Systemowy podział po właściwości. Obiekty z tą samą wartością trafią do "jednego koszyka".
- **GroupStyle** w XAML – Obiekty grupy zostaną ukryte jako "jeden płaski ciąg", jeśli brakuje nagłówka tłumaczącego użytkownikowi, że zaszło tu rozdzielenie. Słowo **Name** w Bindingu nagłówka odpowiada systemowej nazwie zbudowanej grupy (w naszym wypadku "Zarząd" lub "Pracownicy").

40.7. Aktualny element kolekcji

Jeśli masz kontrolkę, która modyfikuje dane osoby z listy (np. obok listy masz TextBox), przydatne okazuje się zjawisko `CurrentItem`. Widok z `ICollectionView` oprócz układania danych, potrafi wskazać swoim "kursorem" zaznaczony aktualnie wiersz. Zsynchronizujesz interfejs, aby pokazywał obok te same dane, co aktywnie zaznaczone nazwisko.

MainWindow.xaml:

```
<Window x:Class="WpfCurrentItem.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Aktualny element" Height="450" Width="800">
    <Grid Margin="20">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="200" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>

        <!-- IsSynchronizedWithCurrentItem każe liście wpływać na pozycję kursora widoku! -->
        <ListBox x:Name="SelectorList" Grid.Column="0"
                DisplayMemberPath="Name"
                IsSynchronizedWithCurrentItem="True" />

        <StackPanel Grid.Column="1" Margin="20">
            <TextBlock Text="Edycja Imienia:" />
            <TextBox x:Name="EditorBox" Text="{Binding Name, UpdateSourceTrigger=PropertyChanged}" FontSize="18" Margin="0,10,0,0"/>
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Windows;
using System.Windows.Data;

namespace WpfCurrentItem
{
    public class SimplePerson : INotifyPropertyChanged
    {
        private string _name;
        public string Name
        {
            get { return _name; }
            set
            {
                _name = value;
                PropertyChanged?.Invoke(this, new PropertyChangedEventArgs("Name"));
            }
        }

        public event PropertyChangedEventHandler PropertyChanged;
    }

    public partial class MainWindow : Window
    {
        public ObservableCollection<SimplePerson> Data { get; set; }

        public MainWindow()
        {

```

```

InitializeComponent();
Data = new ObservableCollection<SimplePerson>
{
    new SimplePerson { Name = "Osoba 1" },
    new SimplePerson { Name = "Osoba 2" }
};

SelectorList.ItemsSource = Data;

// Ustawiamy powiązanie do EditorBoxa na podstawie kursora
ICollectionView view = CollectionViewSource.GetDefaultView(Data);

// Główny stos okna "skupia się" na bieżącym elemencie, więc TextBox wyłapie z niego "Name"
this.DataContext = view;
}
}
}

```

Kluczowe elementy:

- `IsSynchronizedWithCurrentItem="True"` – Właściwość na ListBoxie. Powoduje, że zaznaczenie myszką aktualizuje wewnętrzny suwak `ICollectionView`.
- **Binding w TextBox do kolekcji** – Przypinając jako DataContext cały obiekt widoku `ICollectionView`, Binding `{Binding Name}` odszukuje właściwość w tym jednym obiekcie, na który spogląda kursor (zaznaczony wiersz ListBoxa).

41. Nawigacja i wiele widoków

W tej sekcji zobaczysz, jak zbudować program, który nie gnieździ się w jednym "okienku". Nauczysz się przekazywać informacje między panelami, przełączać kontrolki w zależności od etapu procesu, a także wywoływać osobne, wbudowane systemy dialogowe.

41.1. Wiele okien

Z czasem każda aplikacja rośnie. Jedno `MainWindow` przestaje wystarczać i pojawia się potrzeba stworzenia np. Okna Ustawień (`SettingsWindow`). W WPF, każde nowe okno jest oddzielnym plikiem `.xaml` posiadającym dziedziczenie z klasy `Window`.

Aby stworzyć nowe okno w edytorze: "Prawy przycisk myszy na projekt → Dodaj → Okno (WPF)".

SettingsWindow.xaml:

```

<Window x:Class="WpfMultiWindow.SettingsWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Ustawienia Aplikacji" Height="300" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Oto osobne okno ustawień" FontSize="18"/>
        <Button Content="Zamknij" Click="Close_Click" Margin="0,20,0,0" Padding="10"/>
    </StackPanel>
</Window>

```

SettingsWindow.xaml.cs:

```

using System.Windows;

namespace WpfMultiWindow
{
    public partial class SettingsWindow : Window
    {
        public SettingsWindow()
        {
            InitializeComponent();
        }

        private void Close_Click(object sender, RoutedEventArgs e)

```

```

    {
        // Metoda wbudowana zamykająca to konkretne okno
        this.Close();
    }
}
}

```

Kluczowe elementy:

- **Separacja kodów** – `SettingsWindow` to osobna klasa w oddzielnej przestrzeni nazw. Posiada swoją niezależną od `MainWindow` metodę `InitializeComponent()`.
- **Cykl życia** – Zamykając okno podrzędne (tutaj settings), pamięć zostanie po nim zwolniona (zlikwidowane ze stosu w C#). Zamykając okno główne, cała aplikacja wygasa.

41.2. Show()

Posiadając definicję drugiego okna, należy je wywołać z pierwszego (głównego). Użycie metody `.Show()` spowoduje otwarcie nowego okna w trybie "Niemodalnym" (modeless). Oznacza to, że użytkownik może swobodnie przełączać się pomiędzy oboma oknami i np. pisać w nich na zmianę. Główne okno w ogóle na niego nie czeka i kod przechodzi dalej od razu po otwarciu okna.

MainWindow.xaml:

```

<Window x:Class="WpfShowMethod.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Otwieranie niemodalne" Height="450" Width="800">
    <Grid>
        <Button Content="Otwórz ustawienia równolegle" Click="OpenSettings_Click" Width="200" Height="50" />
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace WpfShowMethod
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void OpenSettings_Click(object sender, RoutedEventArgs e)
        {
            // 1. Inicjacja obiektu
            var window = new SettingsWindow();

            // 2. Wyświetlenie na ekran (Niemodalnie)
            window.Show();

            // 3. Kod wykona się NATYCHMIAST bez czekania na reakcję użytkownika
            MessageBox.Show("Okno zostało otwarte, i program działa sobie dalej.");
        }
    }
}

```

Kluczowe elementy:

- **Niemodalność** – Obie warstwy aplikacji żyją równolegle. Nadaje się do odpiętych paneli narzędzi (np. panel przybornika farb w programie graficznym).

- **Zarządzanie instancją** – Musisz tworzyć obiekt operatorem `new`. Przy każdym kliknięciu otwiera się nowy panel. Jeśli klikniesz 5 razy, otworzy się 5 osobnych okien!

41.3. ShowDialog()

Prawdziwa praca biurowa (np. wyskakujące powiadomienie "Czy na pewno chcesz usunąć plik?") opiera się o modalność. Używasz do tego metody `.ShowDialog()`. Blokuję ona użytkownika – zmusza go do interakcji w tym nowym oknie (wybranie OK lub Cancel), w czasie gdy całe okno główne "zawiesza się", czekając na powrót wartości.

MainWindow.xaml:

```
<Window x:Class="WpfShowDialog.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Otwieranie modalne" Height="450" Width="800">
    <StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
        <Button Content="Wybierz opcję" Click="OpenDialog_Click" Padding="15" Margin="10"/>
        <TextBlock x:Name="ResultText" FontSize="18" HorizontalAlignment="Center" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace WpfShowDialog
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void OpenDialog_Click(object sender, RoutedEventArgs e)
        {
            var dialog = new DialogWindow();

            // Wykonanie zamraża kod XAML głównego okna. Nic poniżej tej linii nie wystartuje
            // dopóki ktoś nie zamknie okna 'dialog'.
            bool? result = dialog.ShowDialog();

            // Tu jesteśmy po powrocie z zamkniętego okienka
            if (result == true)
            {
                ResultText.Text = "Zaakceptowano opcję w oknie.";
            }
            else
            {
                ResultText.Text = "Odrzucono opcję lub zamknięto na X.";
            }
        }
    }
}
```

Kluczowe elementy:

- **Zawieszona pętla** – Metoda `ShowDialog()` zwraca typ nullable bool (`bool?`).
- **Akceptacja w oknie potomnym** – We wnętrzu takiego "DialogWindow" trzeba wpisać kod `this.DialogResult = true;`, aby zakończyć działanie z sukcesem. Program sam wywoła zamykanie i wróci do macierzy.

41.4. Przekazywanie danych między oknami

Najczęstszym problemem początkujących deweloperów jest: "Jak podać z głównego okna informacje o logowaniu do drugiego okna, a potem z niego wrócić ze zmienionym hasłem?". Sposobem na to jest modyfikowanie konstruktora (Constructor Injection) przy wywoływaniu i publiczne właściwości (Properties) przy powrocie.

DataWindow.xaml.cs:

```
using System.Windows;

namespace WpfDataTransfer
{
    public partial class DataWindow : Window
    {
        // Publiczna protercja przygotowana do odebrania zmodyfikowanych danych po zamknięciu
        public string ModifiedText { get; private set; }

        // Konstruktor odbiera "msg" wysłany z zewnątrz
        public DataWindow(string initialMsg)
        {
            InitializeComponent();

            // Wypełniamy okienko danymi przychodzącymi
            InputBox.Text = initialMsg;
        }

        private void Save_Click(object sender, RoutedEventArgs e)
        {
            // Przed powrotem zapisujemy publiczną protercję na to co wpisał użytkownik
            ModifiedText = InputBox.Text;
            this.DialogResult = true; // Koniec
        }
    }
}
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace WpfDataTransfer
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void Edit_Click(object sender, RoutedEventArgs e)
        {
            // 1. WYSYŁAMY dane w konstruktorze
            var window = new DataWindow("Moje startowe dane");

            // 2. Otwieramy wymuszając blokadę (Modalnie)
            if (window.ShowDialog() == true)
            {
                // 3. ODBIERAMY dane po powrocie, używając własnej zmiennej "ModifiedText"
                string gotValue = window.ModifiedText;
                MessageBox.Show("Zmieniono na: " + gotValue);
            }
        }
    }
}
```

```
}  
  
}
```

Kluczowe elementy:

- **Parametr w konstruktorze** – Najlepszy i "najszczelniejszy" sposób ładowania paczki na start, okno narzuca wymóg, żeby o niej pamiętać (w wymogu obiektu).
- **Stan po ShowDialog** – Mimo że okno jest wygaszone u użytkownika i jego wynik jest zwrócony, obiekt w pamięci z C# (`window`) cały czas w tamtej linijce istnieje! Możesz stamtąd doczytywać wszystkie niezbędne cechy i wyniki wyliczeń wprost z właściwości `.ModifiedText`.

41.5. UserControl

Otwieranie 15 okien staje się koszmarem dla pamięci komputera. Nowoczesne programy mają wbudowany jeden ekran, na którym "wstrzykują" różne bloki bez mnożenia paneli, wykorzystując tzw. System Kontrolki Użytkownika. `UserControl` tworzysz analogicznie do klasycznego przycisku `Button` – z własnym kodem XAML – i zamykasz go w spójną całość do wrzucenia gdziekolwiek w aplikacji.

MyHeaderControl.xaml:

```
<UserControl x:Class="WpfUserControls.MyHeaderControl"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">  
    <Border Background="Navy" Padding="15">  
        <StackPanel Orientation="Horizontal" HorizontalAlignment="Center">  
            <TextBlock Text="ZARZĄDZANIE" Foreground="White" FontSize="20" FontWeight="Bold" Margin="0,0,10,0"/>  
            <Button Content="Wyloguj" Padding="5" />  
        </StackPanel>  
    </Border>  
</UserControl>
```

MainWindow.xaml:

```
<Window x:Class="WpfUserControls.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    xmlns:local="clr-namespace:WpfUserControls"  
    Title="Wiele kontrolkek" Height="450" Width="800">  
    <Grid>  
        <Grid.RowDefinitions>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="*" />  
        </Grid.RowDefinitions>  
  
        <!-- Tu osadzamy napisaną w osobnym pliku UserControl! -->  
        <local:MyHeaderControl Grid.Row="0" />  
  
        <TextBlock Text="Treść główna programu..." Grid.Row="1" FontSize="20" Margin="20"/>  
    </Grid>  
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;  
  
namespace WpfUserControls  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
    }  
}
```



```
}  
  
}
```

Kluczowe elementy:

- **Przestrzeń nazw** `xmlns:local` – Potrzebna na górze pliku z oknem, ponieważ okno musi wiedzieć, z jakiego folderu projektu z C# wyciąga nową wtyczkę `MyHeaderControl`.
- **Modularyzacja** – Jeżeli napiszesz własną belkę tytułową czy panel powiadomień w ten sposób, załączysz go do każdego nowego projektu i zapomnisz o kopiowaniu kilkuset linii XAML.

41.6. ContentControl

`ContentControl` to ramka, "pojemnik", do którego wrzucamy treść w czasie działania programu. To doskonałe miejsce, aby umieszczać zdefiniowane przed chwilą `UserControl` podczas przeskakiwania po różnych widokach w jednym oknie (np. przełączając pasek z napisem Krok 1, Krok 2, Krok 3).

MainWindow.xaml:

```
<Window x:Class="WpfContentControl.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="ContentControl - SPA" Height="450" Width="800">  
    <Grid>  
        <Grid.ColumnDefinitions>  
            <ColumnDefinition Width="200"/>  
            <ColumnDefinition Width="*" />  
        </Grid.ColumnDefinitions>  
  
        <StackPanel Grid.Column="0" Background="LightGray">  
            <Button Content="Panel Główny" Click="ShowHome_Click" Height="40" Margin="10"/>  
            <Button Content="Ustawienia" Click="ShowSettings_Click" Height="40" Margin="10"/>  
        </StackPanel>  
  
        
```

```

// MainViewFrame.Content = new SettingsUserControl();
MainViewFrame.Content = new TextBlock { Text = "Tu będą ustawienia...", FontSize = 24, Foreground = System.Windows.Media.Brushes.Red };
    }
}
}

```

Kluczowe elementy:

- **Single Page Application (SPA)** – Aplikacja, która nie gubi środowiska (nie ucieka wzrokowo). Przyciska się lewy pasek i miga tylko połówka prawa.
- **Typowanie w locie** – We właściwość `Content` od kontrolki `ContentControl` możesz wrzucać tekst, przyciski, gotowe całe panele. Cokolwiek przypiszesz do tej zdefiniowanej ramki XAML, pojawi się w tym precyzyjnym prostokącie!

41.7. Widok startowy, roboczy i wynikowy

Aby podsumować system przełączania wielu widoków w ramach `ContentControl`, zbudujemy mechanizm typowego procesowania (Wizard). Użytkownik widzi po kolei etapy: Start (Wstęp), Praca (Wypełnianie ankiety), Wynik (Podsumowanie podane z parametrami), nie wymieniając przy tym otwartego pierwotnie pliku z Okna (Window).

MainWindow.xaml:

```

<Window x:Class="WpfWizard.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kreator Aplikacji" Height="450" Width="800">
    <Grid>
        <!-- Kontener dla etapu -->
        <ContentControl x:Name="WizardHost" />
    </Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;
using System.Windows.Controls;

namespace WpfWizard
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            // Inicjacja kroku pierwszego
            ShowStartView();
        }

        public void ShowStartView()
        {
            StackPanel view = new StackPanel { VerticalAlignment = VerticalAlignment.Center };
            view.Children.Add(new TextBlock { Text = "Witaj w kreatorze!", FontSize = 30, HorizontalAlignment = HorizontalAlignment.Center });

            Button btnNext = new Button { Content = "Rozpocznij proces", Width = 200, Margin = new Thickness(20) };
            btnNext.Click += (s, e) => ShowWorkView(); // Następny etap z eventu

            view.Children.Add(btnNext);
            WizardHost.Content = view;
        }

        public void ShowWorkView()
        {
            StackPanel view = new StackPanel { VerticalAlignment = VerticalAlignment.Center };
            view.Children.Add(new TextBlock { Text = "Podaj swój wiek:", HorizontalAlignment = HorizontalAlignment.Center });

```

```

        TextBox tbAge = new TextBox { Width = 100, Margin = new Thickness(10) };
        view.Children.Add(tbAge);

        Button btnSubmit = new Button { Content = "Zakończ", Width = 150 };
        btnSubmit.Click += (s, e) =>
        {
            // Zebranie danych i przekazanie na ostatni widok
            ShowResultView(tbAge.Text);
        };
        view.Children.Add(btnSubmit);

        WizardHost.Content = view;
    }

    public void ShowResultView(string resultAge)
    {
        StackPanel view = new StackPanel { VerticalAlignment = VerticalAlignment.Center };
        view.Children.Add(new TextBlock { Text = $"Dziękujemy, masz: {resultAge} lat.", FontSize = 24, HorizontalAlignment =
        HorizontalAlignment.Center });

        Button btnClose = new Button { Content = "Zamknij system", Width = 150, Margin = new Thickness(20) };
        btnClose.Click += (s, e) => this.Close();
        view.Children.Add(btnClose);

        WizardHost.Content = view;
    }
}
}
}

```

Kluczowe elementy:

- **Logika maszyny stanów** – C# wymienia w pełni kontent od początku do końca, ale sam stan i krawędzie wokół "środką" pozostają zamrożone na jednym, głównym Oknie startowym programu. To daje bardzo ładny i szybki wizualny efekt.
- **Przekazywanie wyników pomiędzy stronami** – Jak widzisz z parametru (`string resultAge`), przepychając użytkownika od metody do metody, można rzucić wynikiem pobranym wcześniej w formularzu, nic nie zapisując do bazy danych lub zewnętrznych struktur.

42. Asynchroniczność i responsywność UI

W tej sekcji zrozumiesz, dlaczego i kiedy aplikacje "zamrażają się" oraz jak temu zaradzić. Odkryjesz narzędzia takie jak `async` i `await`, które gwarantują płynne działanie interfejsu (np. reagowanie paska przewijania lub klikania okien), podczas gdy skomplikowane algorytmy przetwarzają się pod spodem. Zostanie to przedstawione na przykładzie mini aplikacji generującej duży raport.

42.1. Dlaczego interfejs może się zawieszać

Zasada budowy Windowsa mówi jasno: istnieje tylko jeden główny strumień zdarzeń graficznych nazywany "Wątkiem UI". Jeśli użytkownik klika Przycisk, na ten czas "Wątek UI" przejmuje funkcję z kodem `Click`. Jeśli wpiszesz w nim pętlę trwającą 10 sekund (np. symulowaną przez uśpienie `Thread.Sleep`), "Wątek UI" przez 10 sekund utknie na wykonywaniu tej operacji. Ponieważ utknął, nie może obsłużyć kliknięcia w krzyżyk, pomniejszenia okna czy odświeżenia grafiki powiadamiającej o uciekającym czasie! Program wydaje się być martwy.

MainWindow.xaml:

```

<Window x:Class="WpfFreezing.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zawieszenie UI" Height="450" Width="800">
    <StackPanel Margin="50">
        <Button Content="Uruchom skomplikowany raport" Click="StartReport_Click" Padding="15" />
        <TextBlock x:Name="StatusText" Text="Status: Czekam na start..." FontSize="18" Margin="0,20,0,0"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Threading;
using System.Windows;

namespace WpfFreezing
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void StartReport_Click(object sender, RoutedEventArgs e)
            {
                StatusText.Text = "Status: Rozpoczynam (teraz aplikacja umrze na 5 sek)...";

                // To na twardo zamraza Wątek UI - użytkownik uważa, że apka wisi ("Brak odpowiedzi")
                Thread.Sleep(5000);

                StatusText.Text = "Status: Skończone!";
            }
        }
    }
}

```

Kluczowe elementy:

- **Thread.Sleep** – Symuluje tu odpytanie bazy danych, ściąganie pliku, obróbkę ciężkiego obrazu itp. Zamrożenie jednego wątku paraliżuje całe okno graficzne.
- **Brak odpowiedzi** – Brak komunikacji zwrotnej sprawia, że po kliknięciu użytkownik nie uświadczy nowej aktualizacji napisu "Rozpoczynam", bo procesor natychmiast się wybudzi przy "Skończone!".

42.2. async i await

Aby rozwiązać ten problem, nowoczesny C# wprowadził mechanizm **async** (oznacza, że funkcja może zostać wstrzymana) i **await** (czeka na "odpędzone" zadanie). Jeśli wstrzymasz wykonanie poprzez **await**, WPF nie wyrzuca Wątku UI w nicość, lecz "uwalnia go", by wracał obsługiwać przyciski w systemie, kiedy baza danych powoli miele wiersze z tyłu na pobocznym torze.

MainWindow.xaml:

```

<Window x:Class="WpfAsyncAwait.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Async i Await" Height="450" Width="800">
    <StackPanel Margin="50">
        <Button Content="Uruchom raport asynchronicznie" Click="StartReportAsync_Click" Padding="15" />
        <TextBlock x:Name="StatusText" Text="Status: Czekam na start..." FontSize="18" Margin="0,20,0,0" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Threading.Tasks;
using System.Windows;

namespace WpfAsyncAwait
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

```

    }

    // Deklarujemy 'async', ponieważ musimy uwolnić UI w środku
    private async void StartReportAsync_Click(object sender, RoutedEventArgs e)
    {
        StatusText.Text = "Status: Rozpoczynam (spróbuj teraz poruszyć oknem, zadziała!)...";

        // Używamy Task.Delay (asynchronicznego zamiennika Thread.Sleep).
        // Await oddaje kontrolę WPF, by obsługiwał interfejs aż do momentu, kiedy upłynie te 5 sekund.
        await Task.Delay(5000);

        // Kod po "await" jest znów pod kontrolą Wątku UI, więc można bezpiecznie zaktualizować TextBlock
        StatusText.Text = "Status: Skończone, powrócono do UI!";
    }
}
}

```

Kluczowe elementy:

- **Brak zamrożenia** – Kiedy wykonanie osiąga `await`, metoda jest porzucana i następuje reanimacja UI (pasek postępu ładuje się, okno jest klikalne).
- **Modyfikator w metodzie zdarzenia** – `async void` w C# stosuje się zazwyczaj tylko do sygnatury zdarzeń (Event Handlers np. `OnClick`).

42.3. Zadania długotrwałe

`Task.Delay` służyło tylko do celów demonstracyjnych. Jeśli chcesz faktycznie przeprowadzić np. "100 obliczeń matematycznych w pętli", musisz wymusić przeniesienie pracy z "Wątku UI" na "Wątek w tle" (tzw. `ThreadPool`). Mechanizmem delegującym taką żmudną robotę jest `Task.Run()`.

MainWindow.xaml:

```

<Window x:Class="WpfTaskRun.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zadania poboczne" Height="450" Width="800">
    <StackPanel Margin="50">
        <Button Content="Uruchom ogromne obliczenia" Click="Calc_Click" Padding="15" />
        <TextBlock x:Name="ResultText" FontSize="18" Margin="0,20,0,0" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Threading.Tasks;
using System.Windows;

namespace WpfTaskRun
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private async void Calc_Click(object sender, RoutedEventArgs e)
        {
            ResultText.Text = "Obliczam gigantyczny problem...";

            // Przenosimy brudną robotę na wątek poboczny (północny serwer)
            // i AWAIT by UI mogło żyć własnym życiem!
            long finalValue = await Task.Run(() => PerformHeavyCalculations());

            // Kiedy operacja skończy (wracamy tu), podajemy wynik

```

```

        ResultText.Text = $"Gotowe! Wynik to: {finalValue}";
    }

    private long PerformHeavyCalculations()
    {
        long sum = 0;
        for (long i = 0; i < 500_000_000; i++)
        {
            sum += i; // Symulowanie wielkiego zużycia zasobów
        }
        return sum;
    }
}

```

Kluczowe elementy:

- **Separacja logiki CPU-heavy** – Operacje oparte o wyliczenia algorytmu czy parsowanie tysięcy linii tekstu powinno się pakować do bloku wewnątrz `Task.Run()`.
- **Wyłączne blokady na poboczu** – Proces w tle może liczyć to 30 sekund i zjadać 100% procesora w tym jednym wątku, lecz użytkownik w ogóle tego nie odczuje na polu przewijania okna (będzie masetko).

42.4. Dispatcher

Gdy odpędzasz logikę od "Wątku UI" za pomocą `Task.Run()`, pracujesz w tzw. "obcym terytorium". Jeśli poboczny wątek zapragnie w połowie długiego pętlenia zaktualizować pasek postępu ("Jestem na 50%"), spotka się z błędem w konsoli "Ten wątek nie należy do tego elementu". Żeby poboczny wątek zapukał do jedynego autoryzowanego głównego wątku UI, używamy narzędzia `Dispatcher`.

MainWindow.xaml:

```

<Window x:Class="WpfDispatcher.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Dispatcher" Height="450" Width="800">
    <StackPanel Margin="50">
        <Button Content="Start z progresem" Click="StartProg_Click" Padding="15" />
        ←— Pasek postępu →
        <ProgressBar x:Name="MyProgBar" Maximum="100" Height="30" Margin="0,20,0,0"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Threading;
using System.Threading.Tasks;
using System.Windows;

namespace WpfDispatcher
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private async void StartProg_Click(object sender, RoutedEventArgs e)
        {
            await Task.Run(() =>
            {
                // Jesteśmy obcym wątkiem pracującym ciężko
                for (int i = 1; i ≤ 100; i++)

```

```

        {
            Thread.Sleep(50); // Udajemy żmudną czynność (5 sekund łącznie)

            // Odśwież UI z obcego terytorium za pomocą Dispatchera okna
            Application.Current.Dispatcher.Invoke(() =>
            {
                MyProgBar.Value = i;
            });
        }
    });

    MessageBox.Show("100% Osiągnięte!");
}
}
}
}
}

```

Kluczowe elementy:

- **Błąd wielu wątków w Windows** – Wszystkie elementy dziedziczące po `DispatcherObject` (ProgressBar, TextBox) żądają bezwzględnie operowania na nich w obrębie wątku matki.
- `Dispatcher.Invoke()` – Pozwala to wyizolować pojedyncze kawałki w XAML i uderzyć tylko we fragment kodu tak krótki, żeby zaktualizować cyfrę.

42.5. Aktualizacja UI z innego wątku

Użycie `Dispatcher` czasami sprawia kłopoty wydajnościowe na bardzo szybkiej pętli liczącej setki uderzeń na ułamek sekundy (zatyka kolejkę interfejsu). Nowoczesnym wzorcem jest obiekt `IProgress<T>`, na który wpisuje się zadania a on dba, by podnieść zdarzenie na Wątku UI bez jawnego, uciążliwego stukania "Invoke".

MainWindow.xaml:

```

<Window x:Class="WpfIProgress.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Pasek IProgress" Height="450" Width="800">
    <StackPanel Margin="50">
        <Button Content="Uruchom czyste zadanie" Click="RunTask_Click" Padding="15" />
        <TextBlock x:Name="ProgressText" FontSize="20" FontWeight="Bold" Margin="0,20,0,0" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Threading.Tasks;
using System.Windows;

namespace WpfIProgress
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private async void RunTask_Click(object sender, RoutedEventArgs e)
        {
            // Deklarujemy kogo poinformować, gdy w padnie zgłoszenie Progressu
            var progressHandler = new Progress<int>(percent =>
            {
                // Kod ten będzie wołany ZAWSZE na poprawnym wątku UI!
                ProgressText.Text = $"Aktualny stan: {percent}%";
            });
        }
    }
}

```

```

});

// Odpalamy algorytm w obcym wątku, wręczając mu dzwonek Progressu
await Task.Run(() => LongAlgorithm(progressHandler));
}

private void LongAlgorithm(IProgress<int> progress)
{
    for (int i = 1; i <= 100; i++)
    {
        System.Threading.Thread.Sleep(30); // 30ms przerwy
        // Wybijamy dzwonek. Resztę (w tym wątek UI) obsłuży system zdarzeń
        progress?.Report(i);
    }
}
}
}
}

```

Kluczowe elementy:

- **Separacja architektoniczna** – `LongAlgorithm` nie posiada żadnych słów i odniesień do elementów platformy WPF, nie musi znać ani Dispatchera, ani textblocka, więc staje się uniwersalny dla reszty aplikacji.
- `IProgress.Report` – Asynchroniczne wyzwalanie eventu, które od razu układa zadanie w kolejce w wątku bazowym i aktualizuje widok okna.

42.6. CancellationToken

Zadania `Task.Run` po uruchomieniu są porzucane. Jeśli użytkownik zmieni zadanie na pasku albo zamknie aplikację, pętla może żyć "pod spodem" i obciążać system. Musimy mieć bat pozwalający przerwać to zadanie w trakcie jego liczenia. Do takich akcji wykorzystuje się w C# pojęcie `CancellationTokenSource` (sygnał wywołania o przerwaniu) oraz `CancellationToken` (token kontrolny w pętli zadania).

MainWindow.xaml:

```

<Window x:Class="WpfCancellation.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Przerwanie zadania" Height="450" Width="800">
    <StackPanel Margin="50">
        <Button Content="Uruchom 10-sekundową robotę" Click="Start_Click" Margin="10"/>
        <Button Content="Anuluj przymusowo!" Click="Cancel_Click" Background="Salmon" Margin="10"/>
        <TextBlock x:Name="OutputText" FontSize="18" Margin="10" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Threading;
using System.Threading.Tasks;
using System.Windows;

namespace WpfCancellation
{
    public partial class MainWindow : Window
    {
        private CancellationTokenSource _cts;

        public MainWindow()
        {
            InitializeComponent();
        }

        private async void Start_Click(object sender, RoutedEventArgs e)

```



```

{
    // Nowa pigułka z przyciskiem Anuluj
    _cts = new CancellationTokenSource();
    CancellationToken token = _cts.Token;
    OutputText.Text = "Rozpoczęto zadanie...";

    try
    {
        await Task.Run(() => LongAlgorithmWithCancelCheck(token), token);
        OutputText.Text = "Operacja zakończona sukcesem.";
    }
    catch (OperationCanceledException)
    {
        OutputText.Text = "Przerwano w połowie!";
    }
}

private void Cancel_Click(object sender, RoutedEventArgs e)
{
    // Naciśnięcie tego wywołuje flagę we wszystkich tokenach z tego źródła
    _cts?.Cancel();
}

private void LongAlgorithmWithCancelCheck(CancellationToken ct)
{
    for (int i = 0; i < 100; i++)
    {
        // Rzucenie twardego błędu, który przerwie tę metodę jeśli użytkownik nacisnął "Cancel" na UI.
        ct.ThrowIfCancellationRequested();
        Thread.Sleep(100);
    }
}
}
}

```

Kluczowe elementy:

- **Globalne powiadomienie z `Cancel()`** – Jeśli masz 3 potężne zadania spięte pod jeden wspólny token, jedno zwołanie przerwie wykonanie ich wszystkich naraz.
- **`ThrowIfCancellationRequested`** – Bez ciągłego pukania do tokena (np. co 1 pętlę), wstawiony z tyłu silnik na procesorze nigdy nie dowie się o chęciach bycia wstrzymanym. C# nie zatrzymuje kodu siłowo z urzędu.

42.7. Blokowanie przycisku podczas operacji

Podczas wywoływania długich skryptów trzeba zabezpieczyć się przed tzw. double-clickiem lub klikaniem spamu przez zdezorientowanych użytkowników. Jeśli tego nie wymusisz, jedno kliknięcie przycisku odpali Taska, po 3 sekundach drugie kliknięcie stworzy nową bazę i wyrzuci wyjątki blokady systemu plików na dysku serwera.

MainWindow.xaml:

```

<Window x:Class="WpfButtonLock.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Blokada UI" Height="450" Width="800">
    <StackPanel Margin="50">
        <Button x:Name="ExecuteButton" Content="Zapisz do bazy danych" Click="Execute_Click" Padding="20" FontSize="16" />
        <TextBlock x:Name="InfoText" Margin="0,20,0,0" FontSize="18" Foreground="Green" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Threading.Tasks;
using System.Windows;

namespace WpfButtonLock
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private async void Execute_Click(object sender, RoutedEventArgs e)
        {
            // 1. Zablokuj przycisk NATYCHMIAST żeby go nie kliknąć drugi raz
            ExecuteButton.IsEnabled = false;
            ExecuteButton.Content = "Zapisuję... Proszę czekać.";
            InfoText.Text = "";

            try
            {
                // 2. Oddajemy się długiej pracy z siecią / dyskiem
                await SimulateDatabaseSaveAsync();

                InfoText.Text = "Zapis wykonany poprawnie.";
            }
            catch (Exception ex)
            {
                InfoText.Text = "Błąd zapisu: " + ex.Message;
                InfoText.Foreground = System.Windows.Media.Brushes.Red;
            }
            finally
            {
                // 3. Wracamy z zaświatów. Odblokuj przycisk, żeby ktoś mógł poprawić błędy
                // Zapewniamy bezpieczeństwo kodem z sekcji "finally"
                ExecuteButton.IsEnabled = true;
                ExecuteButton.Content = "Zapisz do bazy danych";
            }
        }

        private async Task SimulateDatabaseSaveAsync()
        {
            // Symulacja ciężkiej wrzuty po internecie na zewn. serwer np. Azure.
            await Task.Delay(3000);
        }
    }
}

```

Kluczowe elementy:

- **Niezawodność bloku `finally`** – Dlaczego nie odblokowano go od razu na dole pod taskiem? Ponieważ wybuch wewnętrznego błędu o nazwie np. "Brak internetu", uziemiłby twój przycisk trwale aż do wyłączenia aplikacji. "Finally" wykona się zawsze i wycofa wszystkie restrykcje zablokowanych przed startem kontrolek.
- **Informowanie użytkownika** – Zamiana tekstu z "Zapisz" na "Zapisuję..." wyśmienicie działa na ludzką psychologię i daje poczucie płynnej aplikacji mimo długiej blokady.

43. Testowanie logiki aplikacji

W tej sekcji zobaczysz, jak sprawdzać poprawność działania Twojego kodu. Testowanie pozwala upewnić się, że aplikacja zachowuje się zgodnie z oczekiwaniami, zanim trafi do użytkownika końcowego.

43.1. Co testować w aplikacji WPF

Podczas tworzenia aplikacji WPF nie testujemy bezpośrednio interfejsu graficznego (np. tego, czy przycisk zmieni kolor po najechaniu myszką). Zamiast tego skupiasz się na logice biznesowej. Testujesz algorytmy, obliczenia, sprawdzanie warunków oraz modyfikację danych.

MainWindow.xaml:

```
<Window x:Class="TestApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Kalkulator" Height="450" Width="800">
    <StackPanel Margin="20">
        <TextBox x:Name="InputA" Width="100" Margin="5" />
        <TextBox x:Name="InputB" Width="100" Margin="5" />
        <Button Content="Dodaj" Click="AddButton_Click" Width="100" Margin="5" />
        <TextBlock x:Name="ResultText" Margin="5" FontWeight="Bold" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace TestApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void AddButton_Click(object sender, RoutedEventArgs e)
        {
            // Zła praktyka: logika uwięziona w interfejsie.
            // Zamiast tego powinniśmy korzystać z zewnętrznych klas.
            if (int.TryParse(InputA.Text, out int a) && int.TryParse(InputB.Text, out int b))
            {
                ResultText.Text = (a + b).ToString();
            }
        }
    }
}
```

- **Logika interfejsu:** Kod wewnątrz `AddButton_Click` to logika, którą bardzo trudno przetestować bez uruchamiania okna.
- **Odseparowanie:** Najważniejszym krokiem do testowania jest wydzielenie kodu (np. dodawania) do osobnych metod, a najlepiej do osobnych klas.

43.2. Testy jednostkowe algorytmów

Testy jednostkowe (Unit Tests) polegają na sprawdzaniu pojedynczych metod. Tworzysz projekt testowy i wywołujesz w nim metody z Twojej aplikacji WPF, sprawdzając, czy wynik jest poprawny.

Calculator.cs:

```
using System;

namespace TestApp.Logic
{
    public class Calculator
    {
        public int Add(int a, int b)
        {
            // ...
        }
    }
}
```

```

        return a + b;
    }
}
}

```

CalculatorTests.cs:

```

using NUnit.Framework;
using TestApp.Logic;

namespace TestApp.Tests
{
    [TestFixture]
    public class CalculatorTests
    {
        [Test]
        public void Add_WhenCalled_ReturnsSumOfArguments()
        {
            // Arrange - przygotowanie danych
            var calculator = new Calculator();
            int a = 5;
            int b = 10;

            // Act - wywołanie metody
            int result = calculator.Add(a, b);

            // Assert - sprawdzenie wyniku
            Assert.AreEqual(15, result);
        }
    }
}

```

- **Arrange (Przygotowanie):** Tworzysz obiekty i zmienne niezbędne do przeprowadzenia testu.
- **Act (Działanie):** Wykonujesz metodę, którą chcesz przetestować.
- **Assert (Sprawdzenie):** Weryfikujesz, czy wynik pokrywa się z oczekiwaniami.

43.3. Testowanie klas pomocniczych

Klasy pomocnicze (np. służące do formatowania tekstu lub generowania identyfikatorów) to idealni kandydaci do testów jednostkowych, ponieważ zazwyczaj nie polegają na interfejsie graficznym.

StringUtils.cs:

```

using System;

namespace TestApp.Helpers
{
    public static class StringUtils
    {
        public static string Capitalize(string input)
        {
            if (string.IsNullOrEmpty(input)) return input;
            return char.ToUpper(input[0]) + input.Substring(1).ToLower();
        }
    }
}

```

StringUtilsTests.cs:

```

using NUnit.Framework;
using TestApp.Helpers;

namespace TestApp.Tests
{
    [TestFixture]
    public class StringUtilsTests
    {
        [Test]
        public void Capitalize_LowercaseWord_ReturnsCapitalized()
        {
            // Act
            var result = StringUtils.Capitalize("warszawa");

            // Assert
            Assert.AreEqual("Warszawa", result);
        }

        [Test]
        public void Capitalize_EmptyString_ReturnsEmpty()
        {
            // Act
            var result = StringUtils.Capitalize("");

            // Assert
            Assert.AreEqual("", result);
        }
    }
}

```

- **Przypadki brzegowe:** Testujesz nie tylko idealne warunki (np. "warszawa"), ale też puste ciągi znaków.
- **Brak zależności:** Metody statyczne są proste w testowaniu, bo nie wymagają tworzenia skomplikowanych obiektów.

43.4. Testowanie ViewModeli

Jeśli używasz wzorca MVVM, testowanie staje się dużo prostsze. ViewModel to po prostu klasa C#, która nie wie o istnieniu okna. Zawiera za to właściwości, które są bindowane do XAML.

MainViewModel.cs:

```

using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace TestApp.ViewModels
{
    public class MainViewModel : INotifyPropertyChanged
    {
        private string _userName;
        public string UserName
        {
            get => _userName;
            set
            {
                _userName = value;
                OnPropertyChanged();
                Greeting = $"Witaj, {_userName}!";
            }
        }

        private string _greeting;
        public string Greeting

```

```

    {
        get => _greeting;
        private set
        {
            _greeting = value;
            OnPropertyChanged();
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string name = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }
}
}

```

MainViewModelTests.cs:

```

using NUnit.Framework;
using TestApp.ViewModels;

namespace TestApp.Tests
{
    [TestFixture]
    public class MainViewModelTests
    {
        [Test]
        public void UserName_WhenSet_UpdatesGreeting()
        {
            // Arrange
            var viewModel = new MainViewModel();

            // Act
            viewModel.UserName = "Jan";

            // Assert
            Assert.AreEqual("Witaj, Jan!", viewModel.Greeting);
        }
    }
}

```

- **Brak XAML:** Zauważ, że w teście w ogóle nie uruchamiasz okna WPF.
- **Symulacja zachowania:** Zmiana `UserName` działa dokładnie tak samo, jakby użytkownik wpisał tekst do TextBoxa powiązanego z tą właściwością.

43.5. Oddzielanie logiki od UI pod testy

Aby kod można było testować, nie możesz operować bezpośrednio na kontrolkach w metodach zdarzeniowych. Zamiast pisać logikę w `Click`, wywołuj metodę, która znajduje się w oddzielnej klasie.

MainWindow.xaml:

```

<Window x:Class="TestApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Rabat" Height="450" Width="800">
    <StackPanel Margin="20">
        <TextBox x:Name="PriceTextBox" />
        <Button Content="Oblicz" Click="Calculate_Click" />
        <TextBlock x:Name="ResultTextBlock" />
    </StackPanel>
</Window>

```

```
</StackPanel>

</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;
using TestApp.Logic;

namespace TestApp
{
    public partial class MainWindow : Window
    {
        private readonly DiscountService _discountService;

        public MainWindow()
        {
            InitializeComponent();
            _discountService = new DiscountService();
        }

        private void Calculate_Click(object sender, RoutedEventArgs e)
        {
            if (decimal.TryParse(PriceTextBox.Text, out decimal price))
            {
                // Oddzielenie - MainWindow nie wie, JAK liczony jest rabat.
                decimal newPrice = _discountService.CalculateDiscount(price);
                ResultTextBlock.Text = newPrice.ToString("C");
            }
        }
    }
}
```

- **Izolacja:** `DiscountService` może być przetestowane całkowicie niezależnie od `MainWindow`.
- **Zależności:** Okno korzysta z klasy biznesowej, ale reguły są zamknięte w serwisie.

43.6. Przykładowe przypadki testowe dla walidatorów

Walidacja to sprawdzanie, czy dane wprowadzone przez użytkownika są poprawne. Jest to krytyczny element logiki, który powinien być dokładnie pokryty testami.

EmailValidator.cs:

```
using System;
using System.Text.RegularExpressions;

namespace TestApp.Validators
{
    public class EmailValidator
    {
        public bool IsValid(string email)
        {
            if (string.IsNullOrWhiteSpace(email)) return false;
            return Regex.IsMatch(email, @"^[^@\s]+@[^@\s]+\.[^@\s]+$");
        }
    }
}
```

EmailValidatorTests.cs:

```

using NUnit.Framework;
using TestApp.Validators;

namespace TestApp.Tests
{
    [TestFixture]
    public class EmailValidatorTests
    {
        private EmailValidator _validator;

        [SetUp]
        public void Setup()
        {
            _validator = new EmailValidator();
        }

        [Test]
        public void IsValid_CorrectEmail_ReturnsTrue()
        {
            bool result = _validator.IsValid("test@example.com");
            Assert.IsTrue(result);
        }

        [Test]
        public void IsValid_MissingAtSign_ReturnsFalse()
        {
            bool result = _validator.IsValid("testexample.com");
            Assert.IsFalse(result);
        }

        [Test]
        public void IsValid_EmptyString_ReturnsFalse()
        {
            bool result = _validator.IsValid("");
            Assert.IsFalse(result);
        }
    }
}

```

- **Metoda SetUp:** Pozwala na uruchomienie kodu przed każdym testem.
- **Różne scenariusze:** Testujesz przypadek poprawny oraz błędne, aby upewnić się, że walidator reaguje na różne dane.

44. Debugowanie i typowe błędy

W tej sekcji zobaczysz, w jaki sposób diagnozować problemy w kodzie, dlaczego aplikacja czasem się zamyka oraz co robić, gdy widok nie chce się zaktualizować.

44.1. Breakpointy

Breakpointy (punkty wstrzymania) to najważniejsze narzędzie do analizowania działania aplikacji. Pozwalają one na wstrzymanie wykonywania programu i analizę jego stanu.

MainWindow.xaml:

```

<Window x:Class="DebugApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Debugowanie" Height="450" Width="800">
    <Button Content="Uruchom proces" Click="Process_Click" />
</Window>

```


MainWindow.xaml.cs:

```
using System.Windows;

namespace DebugApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void Process_Click(object sender, RoutedEventArgs e)
            {
                int limit = 5;
                int sum = 0;

                // Ustawiasz czerwoną kropkę obok tej linii, aby rozpocząć podgląd.
                for (int i = 0; i < limit; i++)
                {
                    sum += i;
                }

                MessageBox.Show($"Suma wynosi: {sum}");
            }
        }
    }
}
```

- **Wstrzymanie:** Gdy aplikacja dotrze do czerwonej kropki, zatrzyma się, a Ty zobaczysz aktualny stan kodu.
- **Wykonanie krokowe:** Używając skrótów (np. F10) możesz przeskakiwać linijką po linijce.

44.2. Podgląd zmiennych

Gdy zatrzymasz kod przy użyciu breakpointu, możesz najechać myszką na zmienną, aby sprawdzić jej obecną wartość.

MainWindow.xaml:

```
<Window x:Class="DebugApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Podgląd" Height="450" Width="800">
    <Button Content="Podgląd Zmiennych" Click="ShowVariables_Click" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Collections.Generic;
using System.Windows;

namespace DebugApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void ShowVariables_Click(object sender, RoutedEventArgs e)
            {

```

```

        List<string> users = new List<string> { "Jan", "Anna", "Piotr" };

        // Postaw breakpoint na poniższej linii
        string message = "Liczba użytkowników: " + users.Count;

        MessageBox.Show(message);
    }
}
}

```

- **Lista Locals:** W środowisku programistycznym w zakładce "Locals" znajdziesz wszystkie zmienne.
- **Watch:** Możesz wpisywać wyrażenia ręcznie, aby na bieżąco monitorować ich zmiany w specjalnym panelu Watch.

44.3. Błędy składni XAML

Jednym z najczęstszych problemów są niezamknięte tagi w pliku XAML. Często powodują one dziwne komunikaty o błędach.

MainWindow.xaml (Błędny):

```

<Window x:Class="DebugApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Błędy XAML" Height="450" Width="800">
    <!-- Błąd: brak tagu zamykającego Button -->
    <StackPanel>
        <Button Content="Kliknij"
        <TextBlock Text="Witaj!" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace DebugApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

- **Sprawdzanie błędów:** Błąd w XAML często oznacza po prostu literówkę lub brak zamknięcia nawiasu kątownego `>`.
- **Zamykanie tagów:** Używaj nawiasów `</>` na końcu krótkich kontrolkek.

44.4. Brak x:Name

Gdy chcesz odwołać się do kontrolki w pliku C#, musi ona posiadać atrybut `x:Name`.

MainWindow.xaml:

```

<Window x:Class="DebugApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Brak x:Name" Height="450" Width="800">
    <StackPanel>
        <!-- Ta kontrolka musi mieć x:Name, aby C# ją widział -->
        <TextBlock x:Name="StatusText" Text="Czekam na akcję..." />
        <Button Content="Zmień" Click="Change_Click" />
    </StackPanel>
</Window>

```

```
</StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace DebugApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void Change_Click(object sender, RoutedEventArgs e)
            {
                StatusText.Text = "Akcja wykonana!";
            }
        }
    }
}
```

- **Łączenie światów:** Atrybut `x:Name` działa jak zmienna obiektowa wiążąca C# i XAML.
- **Błędy kompilacji:** Edytor kodu podkreślił użycie `StatusText` jeśli usuniesz `x:Name` w widoku.

44.5. Niezgodne nazwy kontroltek

Zmiana nazwy kontrolki tylko w jednym miejscu spowoduje problemy z kompilacją i błędy środowiska.

MainWindow.xaml:

```
<Window x:Class="DebugApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zmiana Nazwy" Height="450" Width="800">
    <StackPanel>
        <TextBox x:Name="UserLoginTextBox" Width="200" />
        <Button Content="Zaloguj" Click="Login_Click" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace DebugApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void Login_Click(object sender, RoutedEventArgs e)
            {
                // Błąd powstanie gdy w XAML nazwa będzie inna
                string login = UserLoginTextBox.Text;
                MessageBox.Show($"Witaj, {login}");
            }
        }
    }
}
```

```
}  
}  
}
```

- **Automatyczna poprawa:** Korzystaj z narzędzi refaktoryzacji, by przy zmianie nazwy aktualizować cały projekt naraz.

44.6. Błędne ścieżki obrazów

Jeśli ścieżka do obrazu jest nieprawidłowa lub plik nie został poprawnie dodany, po prostu się nie wyświetli.

MainWindow.xaml:

```
<Window x:Class="DebugApp.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Obrazy" Height="450" Width="800">  
    <StackPanel>  
        <Image Source="Images/logo.png" Width="200" Height="200" />  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;  
  
namespace DebugApp  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
    }  
}
```

- **Właściwości pliku:** Upewnij się, że opcja **Build Action** obrazka jest ustawiona na **Resource**.
- **Ścieżki:** W WPF stosuj prawy ukośnik **/** rozdzielając katalogi we właściwości **Source**.

44.7. Błędy z listą i indeksem

Wyjątek **ArgumentOutOfRangeException** pojawia się często przy obsłudze list po odczycie złego indeksu.

MainWindow.xaml:

```
<Window x:Class="DebugApp.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Wyjątki z Listą" Height="450" Width="800">  
    <StackPanel>  
        <ListBox x:Name="ItemsList" SelectionChanged="ItemsList_SelectionChanged" />  
        <TextBlock x:Name="DetailsText" />  
    </StackPanel>  
</Window>
```

MainWindow.xaml.cs:

```
using System.Collections.Generic;  
using System.Windows;  
  
namespace DebugApp
```

```

{
    public partial class MainWindow : Window
    {
        private List<string> _elements = new List<string> { "Jeden", "Dwa" };

        public MainWindow()
        {
            InitializeComponent();
            ItemsList.ItemsSource = _elements;
        }

        private void ItemsList_SelectionChanged(object sender, System.Windows.Controls.SelectionChangedEventArgs e)
        {
            int index = ItemsList.SelectedIndex;

            // Konieczne jest upewnienie się, że element został faktycznie wybrany
            if (index ≥ 0 && index < _elements.Count)
            {
                DetailsText.Text = "Wybrano: " + _elements[index];
            }
        }
    }
}

```

- **Weryfikacja:** Zawsze sprawdzaj `index ≥ 0` przed użyciem nawiasów klamrowych tablicy `[]`.

44.8. Brak odświeżenia widoku

Gdy w kodzie zmieniasz wartość, ale okno nie reaguje, brakuje najprawdopodobniej `INotifyPropertyChanged`.

MainWindow.xaml:

```

<Window x:Class="DebugApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Odświeżanie" Height="450" Width="800">
    <StackPanel>
        <TextBlock Text="{Binding Info}" FontSize="20" Margin="10" />
        <Button Content="Zaktualizuj" Click="Update_Click" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.ComponentModel;
using System.Windows;

namespace DebugApp
{
    public partial class MainWindow : Window, INotifyPropertyChanged
    {
        private string _info;
        public string Info
        {
            get => _info;
            set
            {
                _info = value;
                PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(nameof(Info)));
            }
        }
    }
}

```

```

        public event PropertyChangedEventHandler PropertyChanged;

        public MainWindow()
        {
            InitializeComponent();
            DataContext = this;
            Info = "Tekst startowy";
        }

        private void Update_Click(object sender, RoutedEventArgs e)
        {
            Info = "Nowy tekst!";
        }
    }
}

```

- **Zdarzenie:** `PropertyChanged` informuje strukturę WPF, że właściwość uległa zmianie.
- **Kontekst danych:** Bez `DataContext = this` okno nie wie, do jakich właściwości się odwołuje.

45. Porządkowanie kodu

W tej sekcji zobaczysz, jak utrzymać czytelność i organizację kodu w projekcie WPF, aby z łatwością wracać do niego w przyszłości.

45.1. Nazewnictwo kontroltek

Nazwy Twoich kontroltek w kodzie XAML muszą precyzyjnie informować o ich funkcji i typie. Używaj spójnego formatu.

MainWindow.xaml:

```

<Window x:Class="CleanApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Porządkowanie" Height="450" Width="800">
    <StackPanel Margin="10">
        <!-- Dobrze nazwane kontrolki -->
        <TextBox x:Name="FirstNameTextBox" Width="200" Margin="5" />
        <Button x:Name="SubmitButton" Content="Zapisz" Click="SubmitButton_Click" />
        <TextBlock x:Name="ErrorTextBlock" Foreground="Red" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace CleanApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void SubmitButton_Click(object sender, RoutedEventArgs e)
        {
            if (string.IsNullOrWhiteSpace(FirstNameTextBox.Text))
            {
                ErrorTextBlock.Text = "Imię jest wymagane!";
            }
        }
    }
}

```

```

    }
}
}

```

- **Sufiksy typów:** Dodawaj słowa `Button`, `TextBox`, `TextBlock` na końcu nazwy, aby w kodzie C# po wpisaniu nazwy od razu wiedzieć, z czym masz do czynienia.
- **Kontekst:** Omijaj ogólnikowe nazwy typu `TextBox1`, `Button2`.

45.2. Nazewnictwo metod

Metody powinny zaczynać się od czasownika i jasno opisywać akcję.

MainWindow.xaml:

```

<Window x:Class="CleanApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Metody" Height="450" Width="800">
    <StackPanel Margin="10">
        <Button Content="Uruchom" Click="StartProcessButton_Click" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace CleanApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void StartProcessButton_Click(object sender, RoutedEventArgs e)
        {
            ClearErrors();
            ValidateUserData();
            SaveToDatabase();
        }

        private void ClearErrors() { /* czyszczenie */ }
        private void ValidateUserData() { /* sprawdzanie */ }
        private void SaveToDatabase() { /* zapisz */ }
    }
}

```

- **Akcja najpierw:** Używaj słów `Get`, `Calculate`, `Save`, `Clear`.
- **Zdarzenia:** Event handlers zazwyczaj nazywają się według wzoru `NazwaKontrolki_Zdarzenie` (np. `StartProcessButton_Click`).

45.3. Metody pomocnicze

Kiedy widzisz, że dany kawałek kodu powtarza się w wielu miejscach, przenieś go do oddzielnej metody prywatnej.

MainWindow.xaml:

```

<Window x:Class="CleanApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Metody Pomocnicze" Height="450" Width="800">

```

```

<StackPanel>
    <Button Content="Akcja 1" Click="Action1_Click" />
    <Button Content="Akcja 2" Click="Action2_Click" />
    <TextBlock x:Name="LogTextBlock" />
</StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;

namespace CleanApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            private void Action1_Click(object sender, RoutedEventArgs e)
            {
                LogMessage("Wykonano akcję 1");
            }

            private void Action2_Click(object sender, RoutedEventArgs e)
            {
                LogMessage("Wykonano akcję 2");
            }

            // Metoda pomocnicza
            private void LogMessage(string message)
            {
                LogTextBlock.Text += $"[{DateTime.Now.ToShortTimeString()}] {message}\n";
            }
        }
    }
}

```

- **Mniej powtórzeń:** Zasada DRY (Don't Repeat Yourself) ułatwia późniejsze poprawki w kodzie.
- **Czytelność:** Kod w event handlerach staje się krótki i zrozumiały na pierwszy rzut oka.

45.4. Klasy serwisowe

Gdy masz dużo metod, które służą jednemu celowi (np. komunikacji z bazą danych lub pobieraniu plików), stwórz dla nich dedykowaną klasę zwaną serwisem.

MainWindow.xaml:

```

<Window x:Class="CleanApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Serwisy" Height="450" Width="800">
    <StackPanel>
        <Button Content="Zapisz do pliku" Click="Save_Click" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:


```

using System.Windows;

namespace CleanApp
{
    public partial class MainWindow : Window
    {
        private FileService _fileService;

        public MainWindow()
        {
            InitializeComponent();
            _fileService = new FileService();
        }

        private void Save_Click(object sender, RoutedEventArgs e)
        {
            _fileService.SaveText("dane.txt", "Przykładowa treść");
            MessageBox.Show("Zapisano!");
        }
    }
}

```

FileService.cs:

```

using System.IO;

namespace CleanApp
{
    public class FileService
    {
        public void SaveText(string path, string content)
        {
            File.WriteAllText(path, content);
        }
    }
}

```

- **Porządek plików:** Klasy wstawiaj do osobnych plików o nazwie odpowiadającej klasie.
- **Organizacja:** Mniej kodu w głównym oknie (tzw. "Fat Controller") zmniejsza szansę na pomyłkę.

45.5. Oddzielanie logiki od obsługi zdarzeń

Logika aplikacji nigdy nie powinna znajdować się bezpośrednio w obsłudze przycisku.

MainWindow.xaml:

```

<Window x:Class="CleanApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Oddzielanie" Height="450" Width="800">
    <StackPanel>
        <TextBox x:Name="NumberTextBox" />
        <Button Content="Sprawdź" Click="Check_Click" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace CleanApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void Check_Click(object sender, RoutedEventArgs e)
        {
            if (int.TryParse(NumberTextBox.Text, out int number))
            {
                bool isEven = MathLogic.IsEven(number);
                MessageBox.Show(isEven ? "Parzysta" : "Nieparzysta");
            }
        }
    }
}

```

MathLogic.cs:

```

namespace CleanApp
{
    public static class MathLogic
    {
        public static bool IsEven(int number)
        {
            return number % 2 == 0;
        }
    }
}

```

- **Użycie zwrotne:** Możesz wykorzystać tę samą logikę do sprawdzenia liczby w innym oknie aplikacji bez kopiowania kodu.

45.6. Komentarze w kodzie

Kod powinien być na tyle czytelny, by tłumaczył sam siebie, jednak komentarze są niezwykle istotne przy skomplikowanych algorytmach i wyjaśnianiu *dlaczego* coś zostało napisane w ten sposób.

MainWindow.xaml:

```

<Window x:Class="CleanApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Komentarze" Height="450" Width="800">
    <!-- Przycisk jest ukryty, ponieważ w fazie testów funkcja nie jest gotowa -->
    <Button Content="Testowa Funkcja" Visibility="Collapsed" />
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace CleanApp
{
    public partial class MainWindow : Window
    {

```

```

{
    public MainWindow()
    {
        InitializeComponent();
    }

    /// <summary>
    /// Oblicza skomplikowaną marżę dla produktu.
    /// Uwzględnia wahania walut.
    /// </summary>
    /// <param name="basePrice">Cena bazowa w PLN</param>
    /// <returns>Obliczona marża procentowa</returns>
    public decimal CalculateComplexMargin(decimal basePrice)
    {
        // Mnożnik 1.25 wynika z umowy zawartej z dostawcą w grudniu 2023.
        return basePrice * 1.25m;
    }
}

```

- **Tagi Summary:** Wpisując `///` nad metodą C#, automatycznie wygenerujesz blok dokumentacyjny używany potem przez środowisko programistyczne podczas podpowiadania kodu.
- **Wyjaśnienia decyzyjne:** Pisz dlaczego zrobisz obejście problemu (workaround), a nie tylko to, co robi kod.

45.7. Formatowanie kodu

Wcięcia (indentacja) i odpowiednie użycie spacji to podstawa estetyki. Kod źle sformatowany jest dużo trudniejszy w analizie.

MainWindow.xaml (Źle sformatowany):

```

<Window x:Class="CleanApp.MainWindow" xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
<StackPanel>
<Button Content="Klik" />
</StackPanel>
</Window>

```

MainWindow.xaml (Dobrze sformatowany):

```

<Window x:Class="CleanApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Formatowanie" Height="450" Width="800">
    <StackPanel>
        <Button Content="Klik" />
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace CleanApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}

```

```
}  
  
}
```

- **Zagnieżdżenia:** Kiedy kontrolka jest w innej kontrolce w XAML, zawsze rób wcięcie (Tab lub 4 spacje).
- **Skróty klawiszowe:** Pamiętaj, by regularnie korzystać ze skrótu `Ctrl + K, Ctrl + D` (w Visual Studio), który automatycznie sformatuje cały plik w poprawny sposób.

46. Budowanie, publikowanie i przekazywanie projektu

W tej sekcji zobaczysz, jak skompilować gotowy projekt w plik uruchomieniowy oraz jak zbudować z niego ostateczną paczkę dla klienta lub innego programisty.

46.1. Build projektu

Zanim uruchomisz aplikację, kod źródłowy musi zostać skompilowany, a pliki XAML przetworzone do plików wewnętrznych (BAML). Proces ten nazywa się budowaniem (Build).

MainWindow.xaml:

```
<Window x:Class="PublishApp.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Budowanie" Height="450" Width="800">  
    <Grid>  
        <TextBlock Text="Aplikacja skompilowana poprawnie!" HorizontalAlignment="Center" VerticalAlignment="Center" />  
    </Grid>  
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;  
  
namespace PublishApp  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
    }  
}
```

- **W terminalu:** Możesz wykonać to za pomocą komendy systemowej:

```
dotnet build
```

- **Sprawdzenie poprawności:** Budowanie upewnia się, że we wszystkich plikach nie ma błędów syntaktycznych.

46.2. Debug i Release

Podczas tworzenia aplikacji kompilujesz ją w trybie `Debug`. Gdy jest gotowa do przekazania, korzystasz z trybu `Release`.

MainWindow.xaml:

```
<Window x:Class="PublishApp.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Tryby Kompilacji" Height="450" Width="800">  
    <StackPanel>  
        <TextBlock x:Name="ModeTextBlock" Margin="20" />  
    </StackPanel>  
</Window>
```

```
</StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace PublishApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            #if DEBUG
                ModeTextBlock.Text = "Uruchomiono w trybie DEBUG";
            #else
                ModeTextBlock.Text = "Uruchomiono w trybie RELEASE";
            #endif
        }
    }
}
```

- **Tryb Debug:** Posiada włączone informacje niezbędne do breakpointów oraz wyłączoną optymalizację, co ułatwia śledzenie błędów.
- **Tryb Release:** Aplikacja działa szybciej i waży mniej, ponieważ kod zostaje mocno zoptymalizowany pod kątem wydajności użytkownika. Do kompilacji użyj polecenia `dotnet build -c Release`.

46.3. Folder wynikowy aplikacji

Gdy zrobisz Build projektu, pliki wykonywalne trafiają do specjalnych folderów.

MainWindow.xaml:

```
<Window x:Class="PublishApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Struktura plików" Height="450" Width="800">
    <TextBlock Text="Zajrzyj do katalogu bin/Debug" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace PublishApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Katalog `bin`:** Miejsce docelowe, w którym powstaje m.in. Twój plik `.exe` i `.dll`. Znajdziesz go wewnątrz `bin/Debug/net8.0-windows/` (lub dla innej wersji środowiska).
- **Katalog `obj`:** Tymczasowe pliki powiązane z procesem kompilacji, zazwyczaj niepotrzebne przy publikacji paczki.

46.4. Publikowanie folderowe

Polecenie Publish przygotowuje katalog, z którego osoba trzecia może po prostu włączyć aplikację. Wykonuje on dodatkowe zadania porządkujące zasoby i upewnia się o obecności wszystkich bibliotek zewnętrznych (DLL).

MainWindow.xaml:

```
<Window x:Class="PublishApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Publish" Height="450" Width="800">
    <TextBlock Text="Aplikacja opublikowana" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace PublishApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Komenda CLI:** Opublikowanie można uzyskać poprzez wpisanie:

```
dotnet publish -c Release
```

- **Katalog końcowy:** Twoje gotowe pliki znajdą się w `bin/Release/net8.0-windows/publish/`. Wystarczy skopiować zawartość tego katalogu docelowemu użytkownikowi.

46.5. Aplikacja zależna od runtime

Aplikacja Framework-Dependent (zależna od środowiska uruchomieniowego) zajmuje mało miejsca na dysku, ale aby działała, wymaga, aby na komputerze docelowym zainstalowany był pakiet .NET Desktop Runtime.

MainWindow.xaml:

```
<Window x:Class="PublishApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Runtime" Height="450" Width="800">
    <TextBlock Text="Zależne od systemu" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace PublishApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

```
}  
}  
}
```

- **Mniejszy rozmiar:** Wynikiem kompilacji jest paczka zajmująca ledwie kilka megabajtów.
- **Ryzyko:** Użytkownik nieposiadający środowiska .NET, zamiast otwarcia okna programu ujrzy systemowy monit z prośbą o doinstalowanie komponentów.

46.6. Aplikacja samowystarczalna

Zdecydowanie bezpieczniejszym podejściem z punktu widzenia doświadczenia docelowego odbiorcy jest Self-Contained Deployment (Aplikacja samowystarczalna). Oznacza to, że paczka ma wbudowane całe niezbędne biblioteki .NET.

MainWindow.xaml:

```
<Window x:Class="PublishApp.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Self-Contained" Height="450" Width="800">  
    <TextBlock Text="Działa wszędzie!" />  
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;  
  
namespace PublishApp  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
    }  
}
```

- **Rozmiar:** Zwiększa się waga paczki wbudowując podstawy bibliotek systemowych.
- **Komenda do przygotowania:** Budowanie z podaniem docelowej platformy (np. Windows na architekturze 64-bit):

```
dotnet publish -c Release -r win-x64 --self-contained
```

46.7. Plik .exe

Aplikacja po zbudowaniu wygeneruje przynajmniej dwa ważne pliki. Twój `NazwaAplikacji.dll` zawierający logikę biznesową, i sam uruchomieniowy plik `NazwaAplikacji.exe`, za pośrednictwem którego aplikacja faktycznie się budzi na systemach z rodziny Windows.

MainWindow.xaml:

```
<Window x:Class="PublishApp.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Plik wykonywalny" Height="450" Width="800">  
    <TextBlock Text="Kliknij w .exe" />  
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;
```

```
namespace PublishApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Skompresowany do 1 pliku:** W nowszych wersjach .NET możesz zapisać wszystko do pojedynczego pliku .exe używając atrybutu kompilacji:

```
dotnet publish -c Release -r win-x64 --self-contained -p:PublishSingleFile=true
```

47. Dokumentowanie działania aplikacji

W tej sekcji dowiesz się, jak wizualnie potwierdzać działanie programu. Kiedy przygotowujesz wersję aplikacji do przekazania do oceny (np. w celach weryfikacji poprawności), dokumentacja w formie zrzutów ekranu jest krytyczna. Udowadnia ona, że kod działa na Twoim środowisku.

47.1. Po co dokumentować stany aplikacji

Dokumentacja udowadnia, że Twoja aplikacja z powodzeniem się skompilowała i w pełni obsługuje wymagane zdarzenia bez błędów (tzw. crashów).

MainWindow.xaml:

```
<Window x:Class="DocApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Aplikacja" Height="450" Width="800">
    <StackPanel Margin="10">
        <TextBlock Text="Przykładowy widok do udokumentowania" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace DocApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Weryfikacja widoku:** Pokazujesz ostateczne formatowanie okna.
- **Dowód bezbłędności:** Chronisz siebie pokazując udane uruchomienie paczki tuż przed jej ostatecznym przesłaniem.

47.2. Zrzut ekranu stanu początkowego

Twój pierwszy krok to uruchomienie programu (np. kliknięcie klawisza F5 w Visual Studio). Następnie musisz uchwycić na obrazku aplikację dokładnie tak, jak prezentuje się bez interakcji.

MainWindow.xaml:


```
<Window x:Class="DocApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Stan początkowy" Height="450" Width="800">
    <StackPanel Margin="20">
        <TextBlock Text="Imię:" />
        <TextBox x:Name="NameBox" />
        <Button Content="Gotowe" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace DocApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Czysty układ:** Pokazujesz puste kontrolki, udowadniając brak błędnych wartości początkowych.
- **Pasek tytułowy:** Upewnij się, że widoczny jest tytuł i zarys całego okna.

47.3. Zrzuty ekranu po interakcjach

Jednym zdjęciem nie udowodnisz działania programu. Zawsze po uchwyceniu stanu zerowego wprowadzasz dane tak, jak postąpiłby rzeczywisty użytkownik, i następnie dokumentujesz zmianę stanu.

MainWindow.xaml:

```
<Window x:Class="DocApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Interakcja" Height="450" Width="800">
    <StackPanel Margin="20">
        <TextBox x:Name="InputBox" />
        <Button Content="Kliknij" Click="Action_Click" />
        <TextBlock x:Name="FeedbackText" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace DocApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

```
private void Action_Click(object sender, RoutedEventArgs e)
{
    FeedbackText.Text = $"Wpisałeś: {InputBox.Text}";
}
}
```

- **Wypełnianie formularza:** Wpisz dane do kontrolki, naciśnij **Kliknij**, i wykonaj zrzut uwieczniający prawidłowo obliczony wynik lub odpowiedź logiki.
- **Błędy walidacji:** Jeżeli program odrzuca puste pola wyrzucając stosowny komunikat błędu (tzw. **MessageBox**), zrób screen tego zjawiska! To pokazuje, że aplikacja jest odporna na błędy klienta.

47.4. Widoczne okno programu

Skupiając się na oknie roboczym programu upewnij się, że używasz odpowiednich narzędzi (np. narzędzie Wycinanie wbudowane w Windows, klawisze **Alt** + **PrintScreen** do chwycenia aktywnego okna).

MainWindow.xaml:

```
<Window x:Class="DocApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Wizualne ramy" Height="450" Width="800">
    <TextBlock Text="Złap całe moje okienko!" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace DocApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Minimalizacja szumu:** Skup się na uchwyceniu samego okna aplikacji WPF. Nie ma potrzeby ukazywać całego własnego pulpitu czy tapety, o ile specyfika zlecenia wyraźnie o to nie prosi.
- **Czytelność:** Jeśli czcionka jest bardzo mała, przed robieniem zrzutów zmień jej rozmiar, by ułatwić sprawdzanie poprawności wyników.

47.5. Widoczne środowisko programistyczne

Czasem wymaga się udokumentowania napisanego fragmentu kodu z edytora lub panelu z podziałem struktury (tzw. Solution Explorer). Robisz to najczęściej po to, aby wykazać, że Twoje pliki klas i pliki powiązane z wzorcem (ViewModels) są w poprawnych folderach.

MainWindow.xaml:

```
<Window x:Class="DocApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Struktura" Height="450" Width="800">
    <TextBlock Text="Ten widok znajduje się w folderze Views" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace DocApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Dokładność:** Zrób zdjęcie otwartego edytora, w którym z jednej strony widać okienko podglądu UI (Designer) a z drugiej kod XAML.
- **Dowód kompilacji:** Uchwycenie paska na samym dole IDE z zielonym napisem **Build Succeeded** utwierdza odbiorcę paczki w jakości Twojego rozwiązania.

47.6. Nazewnictwo plików ze zrzutami

Wrzucając zrzuty ekranu do paczki ZIP, nie pozostawiaj ich z domyślnymi, wygenerowanymi przez system długimi nazwami typu **Zrzut ekranu (2).png**.

MainWindow.xaml:

```
<Window x:Class="DocApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Paczka" Height="450" Width="800">
    <TextBlock Text="Plik graficzny musi coś opisywać!" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace DocApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Wytyczne numeracji:** Czasem z góry zakłada się określony format nazwy dla kolejnych screenów (np. **01.png**, **02.png**, **obraz_1.jpg**). Stosuj to co jest wymagane.
- **Zasada ogólna:** Zadbaj by obrazki układały się alfabetycznie podczas otwierania folderu – ułatwia to czytanie dokumentacji obrazkowej i proces weryfikacji.

47.7. Dokumentowanie błędów kompilacji

Jeżeli napotkasz nierozwiązywalny łańcuch błędów w aplikacji przed jej oddaniem i z jakichś przyczyn aplikacja się nie buduje, dobrą i dojrzałą inżynierską postawą jest zrobienie zdjęcia tym awariom.

MainWindow.xaml:

```
<Window x:Class="DocApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Błędy" Height="450" Width="800">
    <!-- Przykładowy celowy błąd (nieistniejący atrybut) -->
```

```
<!-- <Button NonExistingAttribute="Test" /> -->
<TextBlock Text="Zawsze opisz powód awarii" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace DocApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Wychwycenie problemu:** Dokumentując problem środowiska czy panel "Error List", ułatwisz zadanie sprawdzającemu. Lepiej pokazać, że świadomie wyciągnąłeś wnioski, niż zatajać problem.
- **Komentarz:** W pliku C# dodaj ewentualny krótki komentarz nad linią powodującą dany błąd.

48. Pakowanie projektu i plików źródłowych

Gdy aplikacja jest w 100% zaimplementowana, musisz zarchiwizować projekt programistyczny w taki sposób, aby mógł zostać przekazany i po rozpakowaniu poprawnie działał na komputerze recenzenta.

48.1. Co powinien zawierać przekazywany projekt

Zwykle zależy to od wytycznych zlecenia, ale w standardowym procesie nie oddajesz tylko jednej linijki i pojedynczego widoku. Oddajesz całą solucję.

MainWindow.xaml:

```
<Window x:Class="ArchiveApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Struktura paczki" Height="450" Width="800">
    <TextBlock Text="Przekazujesz całe źródło i środowisko" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace ArchiveApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Kod źródłowy:** Cały pakiet powiązanych plików pozwalających modyfikować aplikację w edytorze.
- **Baza projektu:** Pliki organizujące w rodzaju `.sln` oraz `.csproj`. Są to drogowskazy dla oprogramowania IDE ładującego Twój program na innej maszynie.

48.2. Pliki XAML i C#

Kod w środowisku WPF jest podzielony pomiędzy warstwę wyglądu oraz zachowania. Upewnij się, że nie zmodyfikowałeś niczego poza plikiem projektu.

App.xaml:

```
<Application x:Class="ArchiveApp.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="MainWindow.xaml">
    <Application.Resources>
    </Application.Resources>
</Application>
```

App.xaml.cs:

```
using System.Windows;

namespace ArchiveApp
{
    public partial class App : Application
    {
    }
}
```

- **Plik główny aplikacji (App.xaml):** Ten bardzo ważny plik znajduje się wyżej w strukturze projektu i mówi, jaki plik otwiera się zaraz po odpaleniu `exe`. Koniecznie prześlij go razem z projektem! Nie dotyczy to tylko okienek.
- **Powiązanie warstw:** Usuwanie plików z kodem ukrytym na zapleczu `.xaml.cs` i usilne kopiowanie tylko strony wizualnej uniemożliwi poprawne sprawdzenie Twoich umiejętności algorytmicznych.

48.3. Cały folder projektu

Najczęstszym błędem podczas wysyłania plików jest próba wejścia w pojedyncze podkatalogi i ręczne łapanie poszczególnych plików `.cs` zamiast zebrania całości otulającego je folderu.

MainWindow.xaml:

```
<Window x:Class="ArchiveApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Katalog główny" Height="450" Width="800">
    <TextBlock Text="Skopiuj nadrzędny folder, nie pojedyncze pliki" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace ArchiveApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Hierarchia folderów:** Nadrzędny folder z nazwą Twojego projektu (np. `KalkulatorApp`) zawiera wewnątrz cały skompilowany plik konfiguracyjny (np. `KalkulatorApp.csproj`), który zrzesza resztę. Jeśli go nie ma w paczce — sprawdzenie projektu z Visual Studio będzie niesamowicie uciążliwe.

48.4. Pliki wykonywalne

Osoba weryfikująca często na początku sprawdza jakość aplikacji przed zaglądnięciem w kod. Najprawdopodobniej uruchomi ona aplikację dwuklikiem myszki. Oznacza to, że paczka absolutnie musi skrywać wygenerowany kompilat.

MainWindow.xaml:

```
<Window x:Class="ArchiveApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Sprawdzanie działania" Height="450" Width="800">
    <TextBlock Text="Katalog bin musi być pełen" />
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace ArchiveApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Zbuduj zanim oddasz:** Naciśnij w Visual Studio opcję u góry menu **Build** → **Build Solution**. Jest to fundamentalne po dopisaniu chociażby ostatniej linijki. Inaczej Twój plik z rozszerzeniem **.exe** w katalogu **bin** będzie posiadał starszą, nieaktualną z kodem wersję!
- **Foldery wyjściowe:** Pliki takie zawsze mieszczą się w **bin\Debug\netX.0-windows** (gdzie X to wersja oprogramowania na którym pracowałeś).

48.5. Zasoby graficzne i pliki tekstowe

Zależności multimedialne na stałe wgrywane wewnątrz Twojego oprogramowania – grafiki tła, zdjęcia pobrane z sieci, pliki bazy tekstowej, powinny powędrować w Twoim projekcie razem z nim do finalnego użytkownika.

MainWindow.xaml:

```
<Window x:Class="ArchiveApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Dodatki" Height="450" Width="800">
    <StackPanel>
        <Image Source="Assets/ikona.png" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace ArchiveApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

```
}  
  
}
```

- **Folder zasobów:** Skopiuj przygotowane pliki do wydzielonych katalogów (np. `Assets` lub `Images`). Wtedy, w momencie robienia kopii pakietu, grafiki połączą się z jego zapleczem jako nierozdzielna całość.
- **Bez odniesień na sztywno:** Ścieżki takie jak `C:\Users\Nazwa\Obraz.jpg` w kodzie gwarantują usterkę paczki u kogokolwiek innego! Stosuj zawsze ścieżki względne, tzn. podawaj foldery od startu programu, np. `Assets/Obraz.jpg`.

48.6. Tworzenie archiwum .zip

Ostatnim etapem inżynierii przed wystaniem swojego efektu pracy w świat jest proces kompresji do jednego niewielkiego wolumenu bezstratnego o standardowym formacie zapisu `.zip`. Zbierasz całą uformowaną strukturę i zamykasz w archiwum, używając poleceń lub narzędzi z Twojego systemu operacyjnego.

MainWindow.xaml:

```
<Window x:Class="ArchiveApp.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Kompresja" Height="450" Width="800">  
    <TextBlock Text="Zawsze używaj formatu .zip" />  
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;  
  
namespace ArchiveApp  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
    }  
}
```

- **Formatowanie pliku:** Korzystając ze specyfikacji, unikaj formatów rzadziej natywnie obsługiwanych przez OS-y (np. nie korzystaj z formatu `.rar` czy `.7z`, chociaż są równie dobre pod względem wagi, stawiają wyższe wymagania techniczne od maszyny odbiorcy podczas weryfikacji paczki).
- **Bash Windows (PowerShell):** Możesz to wykonać szybko konsolowo:

```
Compress-Archive -Path "C:\Projekt\*" -DestinationPath "GotowaPaczka.zip"
```

- **Nazwa Archiwum:** Nazwij dokument dokładnie z narzuconymi przez menedżera, lub dokument wytycznych rygorami – zazwyczaj sprowadza się to do Twojego identyfikatora np. `123456789.zip`.

48.7. Kontrola poprawności paczki

Początkujący zapominają o trywialnej rzeczy – co prawda spakowali gotowy produkt, ale nie wypakowali z niego danych do testów przed ich bezwarunkowym wystaniem do oceny!

MainWindow.xaml:

```
<Window x:Class="ArchiveApp.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        Title="Finałna kontrola" Height="450" Width="800">  
    <TextBlock Text="Wypakuj, sprawdź i dopiero prześlij." />  
</Window>
```

MainWindow.xaml.cs:

```
using System.Windows;

namespace ArchiveApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

- **Symulacja po stronie docelowej:** Spakowany folder umieść koniecznie w odległym i niezależnym miejscu dysku na którym operowałeś (np. po prostu na Pulpicie, i dopiero w pełni z niego wypakuj do innej nazwy powłokę projektu).
- **Zasada dwukrotnego otworzenia:** Podejmij udaną próbę dwukrotnego wciśnięcia pliku `.exe` od wewnątrz rozpakowanego wolumenu by ujrzeć własne dzieło z perspektywy recenzenta, a po jego zamknięciu otwórz dwuklikiem plik projektu `.sln` – jeśli środowisko bez protestów odpali wgląd w kod – paczka powędruje szczęśliwie ku ewaluacji!

49. Projekty praktyczne od prostych do średnich

W tej sekcji zobaczysz dziesięć kompletnych, działających aplikacji, które krok po kroku przeprowadzają Cię przez budowę coraz bardziej zaawansowanych interfejsów i logiki.

49.1. Licznik kliknięć

Prosta aplikacja, w której kliknięcie przycisku inkrementuje wartość licznika wyświetlaną w interfejsie. Dodatkowo istnieje możliwość zresetowania licznika.

MainWindow.xaml:

```
<Window x:Class="LicznikKlikniec.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Licznik kliknięć" Height="300" Width="400">
    <StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
        <TextBlock x:Name="CounterTextBlock" Text="0" FontSize="72" FontWeight="Bold" HorizontalAlignment="Center"/>
        <StackPanel Orientation="Horizontal" HorizontalAlignment="Center">
            <Button Content="Kliknij mnie!" FontSize="20" Padding="15,5" Click="IncrementButton_Click"/>
            <Button Content="Resetuj" FontSize="20" Padding="15,5" Click="ResetButton_Click"/>
        </StackPanel>
    </StackPanel>
</Window>
```

MainWindow.xaml.cs:

```
using System;
using System.Windows;

namespace LicznikKlikniec
{
    public partial class MainWindow : Window
    {
        private int _clickCount = 0;

        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```



```

private void IncrementButton_Click(object sender, RoutedEventArgs e)
{
    _clickCount++;
    UpdateCounterDisplay();
}

private void ResetButton_Click(object sender, RoutedEventArgs e)
{
    _clickCount = 0;
    UpdateCounterDisplay();
}

private void UpdateCounterDisplay()
{
    CounterTextBlock.Text = _clickCount.ToString();
}
}

```

- **_clickCount** – prywatne pole przechowujące aktualny stan licznika.
- **UpdateCounterDisplay** – metoda pomocnicza zapobiegająca duplikowaniu kodu aktualizującego kontrolkę `TextBlock`.

49.2. Formularz danych osobowych

Aplikacja z formularzem pobierającym imię, nazwisko, wiek i płeć użytkownika. Po kliknięciu przycisku zapisz wyświetla podsumowanie danych w oknie dialogowym. Jeśli dane są niekompletne, wyświetla komunikat o błędzie.

MainWindow.xaml:

```

<Window x:Class="FormularzDanych.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Formularz danych osobowych" Height="400" Width="350">
    <Grid Margin="20">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <TextBlock Grid.Row="0" Text="Rejestracja użytkownika" FontSize="24" FontWeight="Bold" Margin="0,0,0,20" HorizontalAlignment="Center"/>

        <StackPanel Grid.Row="1" Margin="0,0,0,10">
            <TextBlock Text="Imię:" FontWeight="SemiBold"/>
            <TextBox x:Name="FirstNameTextBox" Height="25" Padding="2"/>
        </StackPanel>

        <StackPanel Grid.Row="2" Margin="0,0,0,10">
            <TextBlock Text="Nazwisko:" FontWeight="SemiBold"/>
            <TextBox x:Name="LastNameTextBox" Height="25" Padding="2"/>
        </StackPanel>

        <StackPanel Grid.Row="3" Margin="0,0,0,10">
            <TextBlock Text="Wiek:" FontWeight="SemiBold"/>
            <TextBox x:Name="AgeTextBox" Height="25" Padding="2" PreviewTextInput="AgeTextBox_PreviewTextInput"/>
        </StackPanel>

        <StackPanel Grid.Row="4" Margin="0,0,0,10">
            <TextBlock Text="Płeć:" FontWeight="SemiBold" Margin="0,0,0,5"/>

```

```

        <StackPanel Orientation="Horizontal">
            <RadioButton x:Name="MaleRadioButton" Content="Mężczyzna" Margin="0,0,10,0"/>
            <RadioButton x:Name="FemaleRadioButton" Content="Kobieta"/>
        </StackPanel>
    </StackPanel>

    <Button Grid.Row="6" Content="Zapisz dane" Height="40" FontSize="16" Click="SaveButton_Click"/>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Windows;
using System.Windows.Input;
using System.Text.RegularExpressions;

namespace FormularzDanych
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void AgeTextBox_PreviewTextInput(object sender, TextCompositionEventArgs e)
        {
            Regex regex = new Regex("[^0-9]+");
            e.Handled = regex.IsMatch(e.Text);
        }

        private void SaveButton_Click(object sender, RoutedEventArgs e)
        {
            string firstName = FirstNameTextBox.Text.Trim();
            string lastName = LastNameTextBox.Text.Trim();
            string ageText = AgeTextBox.Text.Trim();
            string gender = "";

            if (MaleRadioButton.IsChecked == true)
            {
                gender = "Mężczyzna";
            }
            else if (FemaleRadioButton.IsChecked == true)
            {
                gender = "Kobieta";
            }

            if (string.IsNullOrEmpty(firstName) || string.IsNullOrEmpty(lastName) || string.IsNullOrEmpty(ageText) || string.IsNullOrEmpty(gender))
            {
                MessageBox.Show("Wypełnij wszystkie pola przed zapisem!", "Błąd walidacji", MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }

            if (int.TryParse(ageText, out int age))
            {
                string message = $"Dane zapisane pomyślnie:\n\nImię: {firstName}\nNazwisko: {lastName}\nWiek: {age}\nPłeć: {gender}";
                MessageBox.Show(message, "Sukces", MessageBoxButton.OK, MessageBoxImage.Information);
            }
            else
            {

```

```

        MessageBox.Show("Wiek musi być poprawną liczbą!", "Błąd walidacji", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
}
}
}

```

- **PreviewTextInput** – zdarzenie pozwalające odrzucić wprowadzane znaki zanim trafią do TextBoxa (użyte do blokowania wpisywania liter do pola z wiekiem).
- **Regex** – wyrażenie regularne, dzięki któremu pozwalamy wyłącznie na cyfry od 0 do 9.
- **MessageBox.Show** – proste w użyciu okno dialogowe informujące użytkownika o sukcesie lub błędzie.

49.3. Generator haseł

Aplikacja pozwala wygenerować losowe hasło na podstawie określonych przez użytkownika parametrów (długość, zawartość cyfr, znaków specjalnych). Posiada również możliwość skopiowania hasła do schowka.

MainWindow.xaml:

```

<Window x:Class="GeneratorHasel.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Generator Haseł" Height="350" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="Wygenerowane hasło:" FontWeight="Bold"/>
        <TextBox x:Name="PasswordTextBox" Height="40" FontSize="20" IsReadOnly="True" TextAlignment="Center" VerticalContentAlignment="Center"/>

        <Button Content="Kopiuj do schowka" Click="CopyButton_Click" Width="150" HorizontalAlignment="Right" Margin="0,0,0,10"/>

        <TextBlock Text="Długość hasła (8-32):" FontWeight="SemiBold"/>
        <StackPanel Orientation="Horizontal">
            <Slider x:Name="LengthSlider" Minimum="8" Maximum="32" Value="12" Width="250" IsSnapToTickEnabled="True" TickFrequency="1"
                ValueChanged="LengthSlider_ValueChanged"/>
            <TextBlock x:Name="LengthTextBlock" Text="12" FontSize="16" VerticalAlignment="Center"/>
        </StackPanel>

        <CheckBox x:Name="IncludeNumbersCheckBox" Content="Zawsze dołączaj cyfry" IsChecked="True"/>
        <CheckBox x:Name="IncludeSpecialCheckBox" Content="Zawsze dołączaj znaki specjalne" IsChecked="True"/>

        <Button Content="Generuj hasło" Height="45" FontSize="18" Margin="0,20,0,0" Click="GenerateButton_Click"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Text;
using System.Windows;

namespace GeneratorHasel
{
    public partial class MainWindow : Window
    {
        private readonly string _lowercase = "abcdefghijklmnopqrstuvwxyz";
        private readonly string _uppercase = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        private readonly string _numbers = "0123456789";
        private readonly string _special = "!@#$%^&*()_+=[{}|;|:;.,.<?";
        private Random _random = new Random();

        public MainWindow()
        {
            InitializeComponent();
            GeneratePassword();
        }
    }
}

```

```

}

private void LengthSlider_ValueChanged(object sender, RoutedPropertyChangedEventArgs<double> e)
{
    if (LengthTextBlock != null)
    {
        LengthTextBlock.Text = e.NewValue.ToString("0");
    }
}

private void GenerateButton_Click(object sender, RoutedEventArgs e)
{
    GeneratePassword();
}

private void CopyButton_Click(object sender, RoutedEventArgs e)
{
    if (!string.IsNullOrEmpty>PasswordTextBox.Text))
    {
        Clipboard.SetText>PasswordTextBox.Text);
        MessageBox.Show("Skopiowano do schowka!", "Informacja", MessageBoxButton.OK, MessageBoxImage.Information);
    }
}

private void GeneratePassword()
{
    int length = (int)LengthSlider.Value;
    bool includeNumbers = IncludeNumbersCheckBox.IsChecked == true;
    bool includeSpecial = IncludeSpecialCheckBox.IsChecked == true;

    string allowedChars = _lowercase + _uppercase;

    StringBuilder password = new StringBuilder();

    if (includeNumbers)
    {
        allowedChars += _numbers;
        password.Append(_numbers[_random.Next(_numbers.Length)]);
    }

    if (includeSpecial)
    {
        allowedChars += _special;
        password.Append(_special[_random.Next(_special.Length)]);
    }

    while (password.Length < length)
    {
        password.Append(allowedChars[_random.Next(allowedChars.Length)]);
    }

    // Mieszanie znaków hasła
    char[] passwordArray = password.ToString().ToCharArray();
    for (int i = 0; i < passwordArray.Length; i++)
    {
        int swapIndex = _random.Next(passwordArray.Length);
        char temp = passwordArray[i];
        passwordArray[i] = passwordArray[swapIndex];
        passwordArray[swapIndex] = temp;
    }

    PasswordTextBox.Text = new string(passwordArray);
}

```

```
}  
  
}
```

- **Slider** – umożliwia płynny i bezbłędny wybór długości hasła.
- **Clipboard.SetText** – klasa systemowa pozwalająca skopiować wygenerowany tekst do schowka systemowego.
- **StringBuilder** – wydajniejsza metoda tworzenia stringa z pojedynczych znaków w pętli.

49.4. Nadawanie przesyłki

Zaawansowany formularz nadawania paczki, gdzie użytkownik wybiera gabaryt, podaje wymiary (jeśli wymagane) i adres dostawy. Aplikacja kalkuluje orientacyjną cenę na bieżąco.

MainWindow.xaml:

```
<Window x:Class="NadawaniePrzesylki.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Nadawanie przesyłki" Height="500" Width="450">  
    <Grid Margin="15">  
        <Grid.RowDefinitions>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="*" />  
            <RowDefinition Height="Auto"/>  
        </Grid.RowDefinitions>  
  
        <TextBlock Text="Kalkulator paczki" FontSize="22" FontWeight="Bold" Margin="0,0,0,15"/>  
  
        <GroupBox Grid.Row="1" Header="Wybierz gabaryt paczki" Margin="0,0,0,15">  
            <StackPanel Margin="10">  
                <RadioButton x:Name="SmallRadio" Content="Mała (do 5 kg, 15 PLN)" Checked="RecalculatePrice" IsChecked="True"/>  
                <RadioButton x:Name="MediumRadio" Content="Średnia (do 15 kg, 20 PLN)" Checked="RecalculatePrice"/>  
                <RadioButton x:Name="LargeRadio" Content="Duża (powyżej 15 kg, 30 PLN)" Checked="RecalculatePrice"/>  
            </StackPanel>  
        </GroupBox>  
  
        <CheckBox Grid.Row="2" x:Name="PriorityCheckBox" Content="Przesyłka priorytetowa (+5 PLN)" Margin="0,0,0,15" Checked="RecalculatePrice"  
Unchecked="RecalculatePrice"/>  
  
        <GroupBox Grid.Row="3" Header="Dane adresata" Margin="0,0,0,15">  
            <Grid Margin="10">  
                <Grid.RowDefinitions>  
                    <RowDefinition Height="Auto"/>  
                    <RowDefinition Height="Auto"/>  
                </Grid.RowDefinitions>  
                <Grid.ColumnDefinitions>  
                    <ColumnDefinition Width="Auto"/>  
                    <ColumnDefinition Width="*" />  
                </Grid.ColumnDefinitions>  
  
                <TextBlock Text="Ulica i numer:" VerticalAlignment="Center" Margin="0,0,10,10"/>  
                <TextBox Grid.Column="1" x:Name="StreetTextBox" Margin="0,0,0,10" Padding="2"/>  
  
                <TextBlock Grid.Row="1" Text="Kod i miasto:" VerticalAlignment="Center" Margin="0,0,10,0"/>  
                <Grid Grid.Row="1" Grid.Column="1">  
                    <Grid.ColumnDefinitions>  
                        <ColumnDefinition Width="60"/>  
                        <ColumnDefinition Width="*" />  
                    </Grid.ColumnDefinitions>  
                    <TextBox x:Name="PostalCodeTextBox" Margin="0,0,10,0" Padding="2"/>  
                    <TextBox Grid.Column="1" x:Name="CityTextBox" Padding="2"/>  
                </Grid>  
            </Grid>  
        </GroupBox>  
    </Grid>  
</Window>
```

```

        </Grid>
    </Grid>
</GroupBox>

<StackPanel Grid.Row="5" Orientation="Horizontal" HorizontalAlignment="SpaceBetween">
    <StackPanel Orientation="Horizontal">
        <TextBlock Text="Do zapłaty: " FontSize="18" FontWeight="Bold"/>
        <TextBlock x:Name="PriceTextBlock" Text="15 PLN" FontSize="18" FontWeight="Bold" Foreground="DarkGreen"/>
    </StackPanel>
    <Button Content="Nadaj paczkę" Width="120" Height="40" Click="SendButton_Click"/>
</StackPanel>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace NadawaniePrzesylki
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void RecalculatePrice(object sender, RoutedEventArgs e)
        {
            if (PriceTextBlock == null) return;

            decimal total = 0;

            if (SmallRadio.IsChecked == true) total += 15m;
            else if (MediumRadio.IsChecked == true) total += 20m;
            else if (LargeRadio.IsChecked == true) total += 30m;

            if (PriorityCheckBox.IsChecked == true)
            {
                total += 5m;
            }

            PriceTextBlock.Text = $"{total} PLN";
        }

        private void SendButton_Click(object sender, RoutedEventArgs e)
        {
            string street = StreetTextBox.Text.Trim();
            string postalCode = PostalCodeTextBox.Text.Trim();
            string city = CityTextBox.Text.Trim();

            if (string.IsNullOrEmpty(street) || string.IsNullOrEmpty(postalCode) || string.IsNullOrEmpty(city))
            {
                MessageBox.Show("Proszę podać wszystkie dane adresata.", "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }

            MessageBox.Show($"Zlecenie przyjęte do realizacji.\nKoszt: {PriceTextBlock.Text}", "Sukces", MessageBoxButton.OK,
                MessageBoxImage.Information);

            StreetTextBox.Clear();
        }
    }
}

```

```

        PostalCodeTextBox.Clear();
        CityTextBox.Clear();
        SmallRadio.IsChecked = true;
        PriorityCheckBox.IsChecked = false;
    }
}
}

```

- **Checked / Unchecked** – obsługa zdarzeń zmian dla przycisków typu RadioButton oraz CheckBox; dzięki nim wywołujemy funkcję natychmiastowego odświeżania ceny.
- **Zabezpieczenie przed null** – z racji inicjalizacji kontrolki z XAML w specyficznej kolejności, metoda podłączona pod zdarzenia odpala się wcześniej, dlatego warto zawsze sprawdzać `if (PriceTextBlock == null) return;`.

49.5. Dane paszportowe

Aplikacja z weryfikacją poprawności wpisywanych danych w formularzu oraz z logiką sprawdzania poprawności numeru paszportu i daty jego ważności. Ostrzega na czerwono, jeśli pole straci ostrość i nie spełnia wymogów walidacji.

MainWindow.xaml:

```

<Window x:Class="DanePaszportowe.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Dane paszportowe" Height="350" Width="400">
    <StackPanel Margin="20">
        <TextBlock Text="System paszportowy" FontSize="20" FontWeight="Bold" Margin="0,0,0,10"/>

        <TextBlock Text="Numer paszportu (2 litery, 7 cyfr):"/>
        <TextBox x:Name="PassportNumberTextBox" Height="25" LostFocus="PassportNumberTextBox_LostFocus"/>
        <TextBlock x:Name="PassportErrorTextBlock" Foreground="Red" FontSize="11" Visibility="Collapsed"/>

        <TextBlock Text="Data ważności:"/>
        <DatePicker x:Name="ExpirationDatePicker" SelectedDateChanged="ExpirationDatePicker_SelectedDateChanged"/>
        <TextBlock x:Name="DateErrorTextBlock" Foreground="Red" FontSize="11" Visibility="Collapsed"/>

        <Button Content="Zweryfikuj" Height="40" Margin="0,20,0,0" Click="VerifyButton_Click"/>
    </StackPanel>
</Window>

```

MainWindow.xaml.cs:

```

using System;
using System.Text.RegularExpressions;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;

namespace DanePaszportowe
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void PassportNumberTextBox_LostFocus(object sender, RoutedEventArgs e)
        {
            ValidatePassportNumber();
        }

        private void ExpirationDatePicker_SelectedDateChanged(object sender, SelectionChangedEventArgs e)

```

```

{
    ValidateExpirationDate();
}

private bool ValidatePassportNumber()
{
    string number = PassportNumberTextBox.Text.Trim();
    Regex regex = new Regex(@"^[A-Za-z]{2}\d{7}$");

    if (!regex.IsMatch(number))
    {
        PassportNumberTextBox.BorderBrush = Brushes.Red;
        PassportErrorTextBlock.Text = "Nieprawidłowy format. Wymagane 2 litery i 7 cyfr.";
        PassportErrorTextBlock.Visibility = Visibility.Visible;
        return false;
    }
    else
    {
        PassportNumberTextBox.BorderBrush = Brushes.Gray;
        PassportErrorTextBlock.Visibility = Visibility.Collapsed;
        return true;
    }
}

private bool ValidateExpirationDate()
{
    if (!ExpirationDatePicker.SelectedDate.HasValue)
    {
        DateErrorTextBlock.Text = "Proszę wybrać datę.";
        DateErrorTextBlock.Visibility = Visibility.Visible;
        return false;
    }

    if (ExpirationDatePicker.SelectedDate.Value < DateTime.Now)
    {
        DateErrorTextBlock.Text = "Paszport jest nieważny (data z przeszłości).";
        DateErrorTextBlock.Visibility = Visibility.Visible;
        return false;
    }

    DateErrorTextBlock.Visibility = Visibility.Collapsed;
    return true;
}

private void VerifyButton_Click(object sender, RoutedEventArgs e)
{
    bool isPassportValid = ValidatePassportNumber();
    bool isDateValid = ValidateExpirationDate();

    if (isPassportValid && isDateValid)
    {
        MessageBox.Show("Wszystkie dane są poprawne.", "Weryfikacja zakończona", MessageBoxButton.OK, MessageBoxImage.Information);
    }
    else
    {
        MessageBox.Show("Popraw błędy w formularzu przed kontynuacją.", "Błąd weryfikacji", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
}
}

```

- **LostFocus** – wydarzenie uruchamiające się w momencie, gdy użytkownik skończy pisać i kliknie poza kontrolką TextBox.

- **BorderBrush = Brushes.Red** – zmiana obramowania TextBoxa na czerwone podczas błędnej walidacji w celu wyróżnienia błędów.
- **DatePicker** – kontrolka służąca do wybierania daty z wbudowanego kalendarza.

49.6. Przeglądarka albumów

Aplikacja ładująca obiekty z kodów do listy (ListBox) i wyświetlająca szczegóły wybranego albumu obok. Pokazuje obrazek przypisany do konkretnego albumu, bazując na prostym cyklicznym wyborze.

MainWindow.xaml:

```
<Window x:Class="PrzegladarkaAlbumow.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Przeglądarka albumów" Height="400" Width="600">
    <Grid Margin="10">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="200"/>
            <ColumnDefinition Width="10"/>
            <ColumnDefinition Width="*/>
        </Grid.ColumnDefinitions>

        <ListBox x:Name="AlbumsListBox" SelectionChanged="AlbumsListBox_SelectionChanged" DisplayMemberPath="Title"/>

        <Border Grid.Column="2" BorderBrush="LightGray" BorderThickness="1" Padding="15">
            <StackPanel>
                <TextBlock x:Name="TitleTextBlock" FontSize="24" FontWeight="Bold" TextWrapping="Wrap"/>
                <TextBlock x:Name="ArtistTextBlock" FontSize="16" Foreground="Gray" Margin="0,5,0,20"/>

                <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
                    <TextBlock Text="Rok wydania: " FontWeight="SemiBold"/>
                    <TextBlock x:Name="YearTextBlock"/>
                </StackPanel>

                <StackPanel Orientation="Horizontal">
                    <TextBlock Text="Gatunek: " FontWeight="SemiBold"/>
                    <TextBlock x:Name="GenreTextBlock"/>
                </StackPanel>
            </StackPanel>
        </Border>
    </Grid>
</Window>
```

MainWindow.xaml.cs:

```
using System.Collections.Generic;
using System.Windows;
using System.Windows.Controls;

namespace PrzegladarkaAlbumow
{
    public class Album
    {
        public string Title { get; set; }
        public string Artist { get; set; }
        public int Year { get; set; }
        public string Genre { get; set; }
    }

    public partial class MainWindow : Window
    {
        private List<Album> _albums;
```

```

public MainWindow()
{
    InitializeComponent();
    LoadData();
}

private void LoadData()
{
    _albums = new List<Album>
    {
        new Album { Title = "Dark Side of the Moon", Artist = "Pink Floyd", Year = 1973, Genre = "Progressive Rock" },
        new Album { Title = "Thriller", Artist = "Michael Jackson", Year = 1982, Genre = "Pop" },
        new Album { Title = "Nevermind", Artist = "Nirvana", Year = 1991, Genre = "Grunge" },
        new Album { Title = "Abbey Road", Artist = "The Beatles", Year = 1969, Genre = "Rock" }
    };

    AlbumsListBox.ItemsSource = _albums;
    AlbumsListBox.SelectedIndex = 0; // Wybierz pierwszy element na start
}

private void AlbumsListBox_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (AlbumsListBox.SelectedItem is Album selectedAlbum)
    {
        TitleTextBlock.Text = selectedAlbum.Title;
        ArtistTextBlock.Text = selectedAlbum.Artist;
        YearTextBlock.Text = selectedAlbum.Year.ToString();
        GenreTextBlock.Text = selectedAlbum.Genre;
    }
}
}
}

```

- **Klasa pomocnicza Album** – reprezentacja modelu danych; wstrzyknięcie listy tych obiektów w `ItemsSource` to typowe i optymalne podejście.
- **DisplayMemberPath** – określamy, którą własność obiektu `Album` chcemy wyświetlać w `ListBox` jako tekst elementu.
- **SelectionChanged** – obsługuje aktualizowanie sąsiednich kontrolki na podstawie wybranego aktualnie elementu z listy.

49.7. Wzornik kolorów RGB

Rozbudowany projekt, który przy pomocy trzech suwaków pozwala manipulować kolorami tła prostokąta, pokazując aktualną wartość HEX generowanego koloru.

MainWindow.xaml:

```

<Window x:Class="WzornikKolorow.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Wzornik kolorów RGB" Height="350" Width="500">
    <Grid Margin="20">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="20"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

        <StackPanel VerticalAlignment="Center">
            <TextBlock Text="Czerwony (R):"/>
            <StackPanel Orientation="Horizontal">
                <Slider x:Name="RedSlider" Minimum="0" Maximum="255" Width="150" ValueChanged="ColorSlider_ValueChanged" SmallChange="1"
                    LargeChange="10"/>
                <TextBlock x:Name="RedValue" Text="0" Width="30" TextAlignment="Right"/>
            </StackPanel>
        </StackPanel>
    </Grid>

```

```

<TextBlock Text="Zielony (G):"/>
<StackPanel Orientation="Horizontal">
    <Slider x:Name="GreenSlider" Minimum="0" Maximum="255" Width="150" ValueChanged="ColorSlider_ValueChanged"/>
    <TextBlock x:Name="GreenValue" Text="0" Width="30" TextAlignment="Right"/>
</StackPanel>

<TextBlock Text="Niebieski (B):"/>
<StackPanel Orientation="Horizontal">
    <Slider x:Name="BlueSlider" Minimum="0" Maximum="255" Width="150" ValueChanged="ColorSlider_ValueChanged"/>
    <TextBlock x:Name="BlueValue" Text="0" Width="30" TextAlignment="Right"/>
</StackPanel>
</StackPanel>

<StackPanel Grid.Column="2" VerticalAlignment="Center">
    <Rectangle x:Name="ColorPreview" Width="150" Height="150" Stroke="Black" StrokeThickness="1">
        <Rectangle.Fill>
            <SolidColorBrush Color="Black"/>
        </Rectangle.Fill>
    </Rectangle>
    <TextBlock x:Name="HexCodeTextBlock" Text="#000000" FontSize="20" FontWeight="Bold" HorizontalAlignment="Center"/>
</StackPanel>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;
using System.Windows.Media;

namespace WzornikKolorow
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void ColorSlider_ValueChanged(object sender, RoutedPropertyChangedEventArgs<double> e)
        {
            if (RedValue == null || GreenValue == null || BlueValue == null || ColorPreview == null)
                return;

            byte r = (byte)RedSlider.Value;
            byte g = (byte)GreenSlider.Value;
            byte b = (byte)BlueSlider.Value;

            RedValue.Text = r.ToString();
            GreenValue.Text = g.ToString();
            BlueValue.Text = b.ToString();

            Color newColor = Color.FromRgb(r, g, b);

            // Tworzymy pędzel do wypełnienia prostokąta
            SolidColorBrush brush = new SolidColorBrush(newColor);
            ColorPreview.Fill = brush;

            HexCodeTextBlock.Text = $"#{r:X2}{g:X2}{b:X2}";
        }
    }
}

```

```
}  
  
}
```

- **Slider_ValueChanged** – zmapowane zdarzenie w XAML, współdzielone przez wszystkie trzy slidery do ciągłego aktualizowania koloru bez redundancji kodu.
- **Color.FromRgb** – statyczna metoda struktury Color pozwalająca łatwo utworzyć barwę z trzech bajtów.
- **HexCodeTextBlock.Text = \$"{r:X2}{g:X2}{b:X2}";** – eleganckie formatowanie stringów pozwalające wygenerować poprawny kod szesnastkowy. Format **X2** powoduje wypisanie zawsze dwóch znaków HEX.

49.8. Szyfr Cezara

Narzędzie służące do szyfrowania i deszyfrowania tekstów popularną metodą przesunięcia (szyfr Cezara). Pozwala podać dowolne przesunięcie używając liczby.

MainWindow.xaml:

```
<Window x:Class="SzyfrCezara.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Szyfr Cezara" Height="450" Width="600">  
    <Grid Margin="15">  
        <Grid.RowDefinitions>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="*" />  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="Auto"/>  
            <RowDefinition Height="*" />  
        </Grid.RowDefinitions>  
  
        <TextBlock Text="Tekst jawny:" FontWeight="SemiBold" />  
        <TextBox Grid.Row="1" x:Name="InputTextBox" TextWrapping="Wrap" AcceptsReturn="True" Margin="0,5,0,15" />  
  
        <StackPanel Grid.Row="2" Orientation="Horizontal" HorizontalAlignment="Center" Margin="0,0,0,15">  
            <TextBlock Text="Klucz (przesunięcie):" VerticalAlignment="Center" />  
            <TextBox x:Name="KeyTextBox" Text="3" Width="50" TextAlignment="Center" VerticalContentAlignment="Center" />  
            <Button Content="Szyfruj ↓" Width="100" Height="30" Click="EncryptButton_Click" />  
            <Button Content="Deszyfruj ↑" Width="100" Height="30" Click="DecryptButton_Click" />  
        </StackPanel>  
  
        <TextBlock Grid.Row="3" Text="Tekst zaszyfrowany:" FontWeight="SemiBold" />  
        <TextBox Grid.Row="4" x:Name="OutputTextBox" TextWrapping="Wrap" AcceptsReturn="True" Margin="0,5,0,0" IsReadOnly="True" Background="#FFF0F0" />  
    </Grid>  
</Window>
```

MainWindow.xaml.cs:

```
using System.Text;  
using System.Windows;  
  
namespace SzyfrCezara  
{  
    public partial class MainWindow : Window  
    {  
        public MainWindow()  
        {  
            InitializeComponent();  
        }  
  
        private void EncryptButton_Click(object sender, RoutedEventArgs e)  
        {  
            ProcessText(true);  
        }  
    }  
}
```

```

private void DecryptButton_Click(object sender, RoutedEventArgs e)
{
    ProcessText(false);
}

private void ProcessText(bool encrypt)
{
    if (!int.TryParse(KeyTextBox.Text, out int key))
    {
        MessageBox.Show("Klucz musi być liczbą całkowitą!", "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
        return;
    }

    if (!encrypt)
    {
        key = -key; // Odwracamy kierunek przesunięcia
    }

    string input = encrypt ? InputTextBox.Text : OutputTextBox.Text;
    StringBuilder output = new StringBuilder();

    foreach (char c in input)
    {
        if (char.IsLetter(c))
        {
            char offset = char.IsUpper(c) ? 'A' : 'a';
            // Skomplikowane wyliczenie gwarantuje, że nie wyjdziemy poza granice alfabetu także przy kluczach ujemnych
            char shifted = (char)((((c + key) - offset) % 26 + 26) % 26 + offset);
            output.Append(shifted);
        }
        else
        {
            // Znaki niebędące literami pozostawiamy bez zmian
            output.Append(c);
        }
    }

    if (encrypt)
        OutputTextBox.Text = output.ToString();
    else
        InputTextBox.Text = output.ToString();
}
}

```

- **AcceptsReturn="True"** – właściwość TextBoxa umożliwiająca wstawianie znaku nowej linii (Enter).
- **Prosta matematyka modularna** – $((c + key) - offset) \% 26 + 26) \% 26 + offset$ to bezpieczny sposób na wyliczenie przesunięcia znaków z zawijaniem dla alfabetu angielskiego.
- **Kierunek przesunięcia** – deszyfracja to tak naprawdę ta sama operacja co szyfrowanie, ale z zastosowaniem przeciwnego klucza (np. -3).

49.9. Lista zadań

Implementacja popularnej listy rzeczy do zrobienia (To-Do List). Posiada dodawanie i usuwanie elementów bazując na `ObservableCollection`, która automatycznie odświeża interfejs.

MainWindow.xaml:

```

<Window x:Class="ListaZadan.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Zarządzanie zadaniami" Height="400" Width="350">
    <Grid Margin="15">

```

```

<Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*/>
    <RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<StackPanel Orientation="Horizontal" Margin="0,0,0,10">
    <TextBox x:Name="TaskTextBox" Width="220" Margin="0,0,10,0" Padding="5" KeyDown="TaskTextBox_KeyDown"/>
    <Button Content="Dodaj" Width="80" Click="AddButton_Click"/>
</StackPanel>

<ListBox Grid.Row="1" x:Name="TasksListBox" SelectionMode="Single">
    <ListBox.ItemTemplate>
        <DataTemplate>
            <StackPanel Orientation="Horizontal" Margin="2">
                <TextBlock Text="• " FontWeight="Bold" Foreground="DarkBlue"/>
                <TextBlock Text="{Binding}"/>
            </StackPanel>
        </DataTemplate>
    </ListBox.ItemTemplate>
</ListBox>

<Button Grid.Row="2" Content="Usuń wybrane" Height="35" Margin="0,10,0,0" Click="DeleteButton_Click"/>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System.Collections.ObjectModel;
using System.Windows;
using System.Windows.Input;

namespace ListaZadan
{
    public partial class MainWindow : Window
    {
        private ObservableCollection<string> _tasks;

        public MainWindow()
        {
            InitializeComponent();
            _tasks = new ObservableCollection<string>();
            TasksListBox.ItemsSource = _tasks;
        }

        private void AddButton_Click(object sender, RoutedEventArgs e)
        {
            AddTask();
        }

        private void TaskTextBox_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.Key == Key.Enter)
            {
                AddTask();
            }
        }

        private void AddTask()
        {
            string task = TaskTextBox.Text.Trim();

```

```

        if (!string.IsNullOrEmpty(task))
        {
            _tasks.Add(task);
            TaskTextBox.Clear();
            TaskTextBox.Focus(); // Automatycznie zwraca kursor do wpisywania dalej
        }
    }

    private void DeleteButton_Click(object sender, RoutedEventArgs e)
    {
        if (TasksListBox.SelectedIndex != -1)
        {
            _tasks.RemoveAt(TasksListBox.SelectedIndex);
        }
        else
        {
            MessageBox.Show("Najpierw zaznacz element do usunięcia.", "Informacja", MessageBoxButton.OK, MessageBoxImage.Information);
        }
    }
}

```

- **ObservableCollection** – kluczowa klasa w C#. Po połączeniu z **ItemsSource** powoduje, że dodawanie lub usuwanie elementów od razu aktualizuje widoczny **ListBox**. Tradycyjna lista **List<T>** tego nie robi.
- **ItemTemplate** – zmiana wyglądu każdego wiersza listy. Dołączono wizualną kropkę przy każdym elemencie (tworząc listę punktowaną).
- **Focus()** – ułatwienie dla użytkownika (ergonomia). Po dodaniu zadania, pole tekstowe automatycznie czeka na kolejne wciśnięcia klawiszy.

49.10. Mini aplikacja wielowidokowa

Zamiast otwierania kilku osobnych okien, w jednym oknie (z lewym panelem menu) wstawiamy treść, która zmienia się dynamicznie. Służy do tego **Frame** i obiekty **Page**, lecz tutaj zbudujemy to poprzez przełączanie widoczności z użyciem zwykłych **Grid** w jednym pliku – jest to prostsze w mniejszych projektach.

MainWindow.xaml:

```

<Window x:Class="AplikacjaWielowidokowa.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Aplikacja menu" Height="400" Width="600">
    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="150"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

        <!-- Pasek boczny -->
        <Border Background="#FF333333">
            <StackPanel Margin="10">
                <TextBlock Text="MENU" Foreground="White" FontWeight="Bold" HorizontalAlignment="Center" Margin="0,10,0,20"/>
                <Button Content="Strona Główna" Padding="10" Background="Transparent" Foreground="White" BorderThickness="0" Click="NavHome_Click"/>
                <Button Content="Ustawienia" Padding="10" Background="Transparent" Foreground="White" BorderThickness="0" Click="NavSettings_Click"/>
                <Button Content="O autorze" Padding="10" Background="Transparent" Foreground="White" BorderThickness="0" Click="NavAbout_Click"/>
            </StackPanel>
        </Border>

        <!-- Kontener na widoki -->
        <Grid Grid.Column="1" Margin="20">
            <!-- Widok 1: Home -->
            <Grid x:Name="HomeView" Visibility="Visible">
                <TextBlock Text="Witaj na stronie głównej!" FontSize="28" HorizontalAlignment="Center" VerticalAlignment="Center"/>
            </Grid>

            <!-- Widok 2: Settings -->

```

```

        <Grid x:Name="SettingsView" Visibility="Collapsed">
            <StackPanel VerticalAlignment="Center">
                <TextBlock Text="Ustawienia aplikacji" FontSize="24" FontWeight="Bold" Margin="0,0,0,20"/>
                <CheckBox Content="Tryb ciemny (imitacja)"/>
                <CheckBox Content="Powiadomienia włączone" IsChecked="True" Margin="0,10,0,0"/>
            </StackPanel>
        </Grid>

        <!-- Widok 3: About -->
        <Grid x:Name="AboutView" Visibility="Collapsed">
            <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center">
                <TextBlock Text="Twórca: Programista WPF" FontSize="20"/>
                <TextBlock Text="Wersja: 1.0.0" FontSize="16" Foreground="Gray" Margin="0,10,0,0"/>
            </StackPanel>
        </Grid>
    </Grid>
</Grid>
</Window>

```

MainWindow.xaml.cs:

```

using System.Windows;

namespace AplikacjaWielowidokowa
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void NavHome_Click(object sender, RoutedEventArgs e)
        {
            HideAllViews();
            HomeView.Visibility = Visibility.Visible;
        }

        private void NavSettings_Click(object sender, RoutedEventArgs e)
        {
            HideAllViews();
            SettingsView.Visibility = Visibility.Visible;
        }

        private void NavAbout_Click(object sender, RoutedEventArgs e)
        {
            HideAllViews();
            AboutView.Visibility = Visibility.Visible;
        }

        private void HideAllViews()
        {
            HomeView.Visibility = Visibility.Collapsed;
            SettingsView.Visibility = Visibility.Collapsed;
            AboutView.Visibility = Visibility.Collapsed;
        }
    }
}

```


- **Visibility="Collapsed"** – ta właściwość powoduje, że Grid nie tylko staje się niewidoczny, ale dodatkowo znika z układu tak, jakby w ogóle go tam nie było. To pozwala nakładać na siebie kilka kontenerów.
- **HideAllViews()** – metoda narzędziowa. Przed pokazaniem nowego ekranu zawsze wyłączamy wszystkie pozostałe, aby uniknąć kolizji wyświetlania.

50. Kluczowe mechaniki do powtarzania

Zestawienie najpopularniejszych, wielokrotnie powtarzających się problemów z budową interfejsów w WPF. Znajdziesz tu szybkie przypomnienie i gotowy fragment kodu służący do wklejenia i dopasowania do swoich potrzeb.

50.1. Formularz z wieloma kontrolkami

Budowa formularzy wymaga poukładania opisu oraz kontrolki wejściowej, by estetycznie tworzyły wiersze. Zamiast budować tabelę komórka po komórce, używamy wierszy dla całych bloków połączonych w pionie. Narzędzie `StackPanel` zawarte w wierszach systematyzuje całość.

Złoty snippet (XAML):

```
<Grid>

    <Grid.RowDefinitions>

        <RowDefinition Height="Auto" />

        <RowDefinition Height="Auto" />

    </Grid.RowDefinitions>

    <StackPanel Grid.Row="0" Margin="0,0,0,10">

        <TextBlock Text="E-mail:" FontWeight="Bold" Margin="0,0,0,5" />

        <TextBox x:Name="EmailInput" Padding="5" />

    </StackPanel>

    <StackPanel Grid.Row="1" Margin="0,0,0,10">

        <TextBlock Text="Hasło:" FontWeight="Bold" Margin="0,0,0,5" />

        <PasswordBox x:Name="PasswordInput" Padding="5" />

    </StackPanel>

</Grid>
```

50.2. Główny przycisk akcji

Zawsze na dole każdego formularza dodajesz przycisk, który wykonuje właściwą logikę. Ten element powinien być dobrze widoczny i posiadać przypięte zdarzenie `Click`. W kodzie uruchamia on docelowe mechanizmy i ewentualne komunikaty.

Złoty snippet (XAML + C#):

```
<Button Content="Zatwierdź zmiany" Height="40" Background="DodgerBlue" Foreground="White" FontWeight="Bold" Click="SubmitAction_Click" />
```

```
private void SubmitAction_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Akcja wykonana pomyślnie!", "Sukces", MessageBoxButton.OK, MessageBoxImage.Information);
}
```

50.3. Dynamiczna zmiana tekstu

Interfejs musi na bieżąco reagować na to, co wpisuje użytkownik do TextBoxa. Za każdym wciśnięciem klawisza zmieniamy tekst innej kontrolki obok. Używamy do tego zdarzenia `TextChanged`.

Złoty snippet (XAML + C#):

```
<TextBox x:Name="InputBox" TextChanged="InputBox_TextChanged" />
<TextBlock x:Name="PreviewBlock" />
```

```
private void InputBox_TextChanged(object sender, TextChangedEventArgs e)
{
}
```

```
PreviewBlock.Text = "Wpisujesz: " + InputBox.Text;
}
```

50.4. Dynamiczna zmiana obrazu

Podmiana pliku graficznego z kodu jest niezbędna, jeśli widok zależy od stanu. Wymaga podania względnej ścieżki i stworzenia obiektu `BitmapImage`. Ten nowy obiekt nadpisuje dotychczasowy wstrzykiwany do kontrolki `Image`.

Złoty snippet (C#):

```
using System;
using System.Windows.Media.Imaging;

private void ChangeImage(string imageName)
{
    Uri imageUri = new Uri($"pack://application:,,,/Images/{imageName}.jpg");
    MyImageControl.Source = new BitmapImage(imageUri);
}
```

50.5. Zdarzenie LostFocus

To zdarzenie jest idealnym momentem na przeprowadzenie kontroli jakości, formatowanie numerów czy zablokowanie dalszych czynności. Odpala się gdy kursor wyjdzie z edytowanego pola tekstowego. W ten sposób nie nękamy użytkownika błędami, gdy ten jeszcze pisze.

Złoty snippet (XAML + C#):

```
<TextBox x:Name="PeselInput" LostFocus="PeselInput_LostFocus"/>
```

```
private void PeselInput_LostFocus(object sender, RoutedEventArgs e)
{
    if (PeselInput.Text.Length != 11)
    {
        MessageBox.Show("PESEL musi mieć 11 znaków.");
    }
}
```

50.6. Walidacja cyfr i długości tekstu

Często chcemy zezwolić wpisywać tylko i wyłącznie cyfry, albo tylko ograniczoną liczbę znaków. Używa się do tego wyrażeń regularnych na zdarzeniu odrzucającym niewłaściwe klawisze. Limit znaków wprowadzamy prosto z poziomu XAML.

Złoty snippet (XAML + C#):

```
←!— MaxLength rozwiązuje sprawę długości —→
<TextBox x:Name="PhoneInput" MaxLength="9" PreviewTextInput="PhoneInput_PreviewTextInput"/>
```

```
using System.Text.RegularExpressions;
using System.Windows.Input;

private void PhoneInput_PreviewTextInput(object sender, TextCompositionEventArgs e)
{
    // Odrzuca cokolwiek, co nie jest cyfrą (e.Handled = true oznacza odrzucenie znaku)
    Regex regex = new Regex("[^0-9]+");
    e.Handled = regex.IsMatch(e.Text);
}
```

50.7. Fallback do wartości domyślnej

Przy parsowaniu liczb z tekstu należy zabezpieczyć się przed sytuacją, w której pole jest puste albo tekst jest uszkodzony. Ochroni to aplikację przed nagłym wyrzuceniem błędu systemowego (Exception). W takim przypadku zawsze warto mieć bezpieczną alternatywę wpisaną na twardo.

Złoty snippet (C#):

```
string textValue = PriceInput.Text;
// W przypadku niepowodzenia (np. zły format), zmienna price pozostanie 0
if (!double.TryParse(textValue, out double price))
{
    price = 0; // Fallback do zera
}
```

50.8. Odczyt listy obiektów z pliku

Jest to klasyczny scenariusz wczytania linia po linii struktury z prostego pliku tekstowego. Przydaje się to do załadowania gotowej bazy danych na start działania programu. Należy upewnić się najpierw, czy plik istnieje, by nie wyrzucić wyjątku.

Złoty snippet (C#):

```
using System.IO;
using System.Collections.Generic;

private List<string> LoadData(string filePath)
{
    List<string> results = new List<string>();
    if (File.Exists(filePath))
    {
        string[] lines = File.ReadAllLines(filePath);
        results.AddRange(lines);
    }
    return results;
}
```

50.9. Indeks cykliczny

Mechanika przydatna bardzo często przy przycisku "Następne zdjęcie", gdzie przekroczenie rozmiaru listy powinno nas przenieść z powrotem na element zerowy (początek). Używa się do tego prostej matematyki z zastosowaniem operatora modulo. Taka pętla wyklucza błąd poza zakresem i jest bezawaryjna.

Złoty snippet (C#):

```
private int _currentIndex = 0;
private List<string> _items = new List<string> { "A", "B", "C" };

private void NextItem()
{
    // Jeśli indeks dojdzie do 3, zmieni się w 0
    _currentIndex = (_currentIndex + 1) % _items.Count;
    string current = _items[_currentIndex];
}
```

50.10. Suwaki RGB

Suwaki dostarczają idealnej pożywki do budowania barw na zasadzie RGB z zakresu 0 do 255. Do prostokąta, by zmienić mu kolor tła, wstrzykujemy pędzel zbudowany ze struktury `Color`. Wszystko operuje bez opóźnień.

Złoty snippet (C#):

```
using System.Windows.Media;

private void UpdateColor()
{
}
```

```

byte r = (byte)SliderR.Value;
byte g = (byte)SliderG.Value;
byte b = (byte)SliderB.Value;

MyRectangle.Fill = new SolidColorBrush(Color.FromRgb(r, g, b));
}

```

50.11. Zapis przez dialog systemowy

Zamiast zapisywać dane na twardo w jednym i tym samym miejscu na dysku, otwieramy natywne okienko Windows pozwalające wybrać ścieżkę zapisu pliku. Wymaga referencji do zasobów przestrzeni nazewniczej Win32. Ten mechanizm zapewnia maksymalną niezawodność zapisu.

Złoty snippet (C#):

```

using Microsoft.Win32;
using System.IO;

private void SaveData()
{
    SaveFileDialog saveDialog = new SaveFileDialog();
    saveDialog.Filter = "Plik tekstowy (*.txt)|*.txt";

    if (saveDialog.ShowDialog() == true)
    {
        File.WriteAllText(saveDialog.FileName, "Dane zapisane z aplikacji.");
    }
}

```

50.12. Stan bieżący i zatwierdzony

Aplikacje rzadko zapisują globalne ustawienia na bieżąco, częściej zatwierdzamy zmiany jednorazowo przyciskiem "Zapisz". Dlatego też potrzebujemy zmiennej z wersją roboczą oraz zmiennej przechowującej wynik ostateczny. Dopiero kliknięcie przesuwa wartość do faktycznego celu.

Złoty snippet (C#):

```

private string _draftName = "";
private string _savedName = "";

private void NameInput_TextChanged(object sender, TextChangedEventArgs e)
{
    _draftName = NameInput.Text; // Trzymamy roboczo
}

private void SaveAction_Click(object sender, RoutedEventArgs e)
{
    _savedName = _draftName; // Zatwierdzamy ostatecznie
    MessageBox.Show($"Zapisano nazwę: {_savedName}");
}

```

50.13. Atlas najważniejszych właściwości WPF

Atlas właściwości jest szybkim punktem odniesienia przy projektowaniu okien. WPF ma bardzo dużo właściwości, ale wiele z nich powtarza się w różnych kontrolkach, ponieważ kontrolki dziedziczą po wspólnych klasach, takich jak `Control`, `FrameworkElement` albo `ContentControl`. Dzięki temu `Margin`, `Padding`, `Width`, `Height`, `IsEnabled` czy `Visibility` działają podobnie w wielu miejscach.

WŁAŚCIWOŚĆ	GDZIE WYSTĘPUJE	ZNACZENIE	PRZYKŁAD XAML	PRZYKŁAD C# LUB UWAGA
Text	TextBox , TextBlock	tekst widoczny w kontrolce albo wpisany przez użytkownika	<TextBox Text="Jan" />	txtImie.Text = "Jan";
Content	Button , Label , CheckBox , RadioButton	zawartość kontrolki typu content control	<Button Content="Zapisz" />	btn.Content = "OK";
Header	GroupBox , TabItem , Expander	tytuł sekcji, karty albo panelu	<GroupBox Header="Dane" />	Nagłówek może być tekstem albo inną kontrolką.
Items	ComboBox , ListBox , MenuItem	ręcznie wpisana kolekcja elementów	<ComboBox><ComboBoxItem Content="A" /></ComboBox>	Dobre dla kilku stałych opcji.
ItemsSource	ComboBox , ListBox , DataGrid	źródło danych dla listy	<ListBox ItemsSource="{Binding Produkty}" />	Najczęściej kolekcja List<T> albo ObservableCollection<T>.
SelectedItem	ComboBox , ListBox , DataGrid	aktualnie wybrany obiekt	SelectedItem="{Binding WybranyProdukt}"	Zwraca cały obiekt, nie sam tekst.
SelectedIndex	ComboBox , ListBox , TabControl	indeks wybranego elementu, liczony od zera	SelectedIndex="0"	-1 oznacza brak wyboru.
SelectedValue	ComboBox , ListBox	wartość wybranej właściwości obiektu	SelectedValuePath="Id"	Użyteczne, gdy z obiektu potrzebny jest tylko identyfikator.
DisplayMemberPath	ComboBox , ListBox	nazwa właściwości pokazywanej jako tekst	DisplayMemberPath="Nazwa"	Pozwala pokazać Nazwa zamiast wyniku ToString().
IsChecked	CheckBox , RadioButton	stan zaznaczenia	<CheckBox IsChecked="True" />	Typ to bool? , bo CheckBox może mieć stan pośredni.
IsThreeState	CheckBox	włącza trzeci stan: null	IsThreeState="True"	Stan pośredni bywa używany dla opcji częściowo aktywnych.
Value	Slider , ProgressBar , DatePicker zależnie od kontrolki	bieżąca wartość kontrolki	<Slider Value="50" />	Przy Slider typem jest liczba.
Minimum	Slider , ProgressBar	najmniejsza dopuszczalna wartość	Minimum="0"	Ustaw razem z Maximum .
Maximum	Slider , ProgressBar	największa dopuszczalna wartość	Maximum="100"	Value powinno mieścić się w zakresie.
SmallChange	Slider	mały krok zmiany wartości	SmallChange="1"	Używany przy klawiaturze i małych przesunięciach.
LargeChange	Slider	większy krok zmiany wartości	LargeChange="10"	Przydatne przy szerokim zakresie wartości.
TickFrequency	Slider	odstęp między znacznikami	TickFrequency="10"	Działa razem z TickPlacement .

WŁAŚCIWOŚĆ	GDZIE WYSTĘPUJE	ZNACZENIE	PRZYKŁAD XAML	PRZYKŁAD C# LUB UWAGA
Visibility	większość elementów	widoczność elementu	Visibility="Collapsed"	Hidden zostawia miejsce, Collapsed usuwa element z układu.
IsEnabled	większość kontroltek	blokada używania kontrolki	IsEnabled="False"	Wyłączona kontrolka zwykle jest wyszarzona.
IsReadOnly	TextBox , DataGrid	pozwala czytać, ale nie edytować	IsReadOnly="True"	Lepsze niż IsEnabled="False" , gdy tekst ma pozostać czytelny.
IsDefault	Button	przycisk uruchamiany klawiszem Enter	IsDefault="True"	W oknie powinien być jeden główny przycisk.
IsCancel	Button	przycisk uruchamiany klawiszem Esc	IsCancel="True"	Często dla przycisku Anuluj .
Margin	FrameworkElement	odstęp na zewnątrz elementu	Margin="8"	Zapis lewo, góra, prawo, dół : Margin="5,10,5,0" .
Padding	wiele kontroltek	odstęp wewnątrz elementu	Padding="10,6"	Często poprawia czytelność przycisków i ramek.
Width	FrameworkElement	stała szerokość	Width="160"	Nie nadużywaj stałych szerokości w responsywnych układach.
Height	FrameworkElement	stała wysokość	Height="32"	Przy tekstach zwykle lepsze jest MinHeight .
MinWidth	FrameworkElement	minimalna szerokość	MinWidth="120"	Daje kontrolce dolną granicę rozmiaru.
MaxWidth	FrameworkElement	maksymalna szerokość	MaxWidth="400"	Chroni przed zbyt szerokimi polami.
MinHeight	FrameworkElement	minimalna wysokość	MinHeight="32"	Dobre dla pól i przycisków.
HorizontalAlignment	FrameworkElement	wyrównanie poziome	HorizontalAlignment="Stretch"	W Grid często używa się Stretch .
VerticalAlignment	FrameworkElement	wyrównanie pionowe	VerticalAlignment="Center"	Przy formularzach często Center .
HorizontalContentAlignment	Control	wyrównanie zawartości wewnątrz kontrolki	HorizontalContentAlignment="Center"	Dotyczy tekstu lub zawartości przycisku, listy, etykiety.
VerticalContentAlignment	Control	pionowe wyrównanie zawartości	VerticalContentAlignment="Center"	Pomaga przy kontrolkach o stałej wysokości.
Background	Control , Border , Panel	tło elementu	Background="LightYellow"	Przyjmuje pędzel, np. SolidColorBrush .
Foreground	Control , TextBlock	kolor tekstu	Foreground="DarkBlue"	Dotyczy tekstu i części kontroltek tekstowych.
BorderBrush	Control , Border	pędzel obramowania	BorderBrush="Gray"	Bez BorderThickness obramowanie może nie być widoczne.
BorderThickness	Control , Border	grubość obramowania	BorderThickness="1"	Może mieć cztery wartości: 1,0,1,0 .
CornerRadius	Border	zaokrąglenie rogów	CornerRadius="6"	Nie działa bezpośrednio na zwykłym Button .

WŁAŚCIWOŚĆ	GDZIE WYSTĘPUJE	ZNACZENIE	PRZYKŁAD XAML	PRZYKŁAD C# LUB UWAGA
FontFamily	kontrolki tekstowe	krój pisma	FontFamily="Segoe UI"	Domyślne fonty systemowe są zwykle najlepsze.
FontSize	kontrolki tekstowe	rozmiar tekstu	FontSize="16"	WPF używa jednostek niezależnych od DPI.
FontWeight	kontrolki tekstowe	grubość tekstu	FontWeight="Bold"	Typowe wartości: <code>Normal</code> , <code>SemiBold</code> , <code>Bold</code> .
FontStyle	kontrolki tekstowe	styl pisma	FontStyle="Italic"	Używaj oszczędnie dla wyróżnień.
TextAlignment	TextBlock, TextBox	wyrównanie tekstu	TextAlignment="Center"	Nie mylić z <code>HorizontalAlignment</code> .
TextWrapping	TextBlock, TextBox	zawijanie tekstu	TextWrapping="Wrap"	Włącza przechodzenie tekstu do nowej linii.
TextTrimming	TextBlock	skracać z wielokropkiem	TextTrimming="CharacterEllipsis"	Przydatne w wąskich kolumnach.
MaxLength	TextBox	maksymalna liczba znaków	MaxLength="30"	Ogranicza wpisywanie, ale nie zastępuje walidacji.
AcceptsReturn	TextBox	pozwała wpisywać wiele linii	AcceptsReturn="True"	Zwykle razem z paskiem przewijania.
AcceptsTab	TextBox	pozwała wpisać tabulator	AcceptsTab="True"	W zwykłych formularzach częściej zostawia się <code>False</code> .
TextBox.TextWrapping	TextBox	zawijanie tekstu w polu	TextWrapping="Wrap"	Dobre dla pól wielowierszowych.
VerticalScrollBarVisibility	TextBox, ScrollView	widoczność pionowego paska przewijania	VerticalScrollBarVisibility="Auto"	<code>Auto</code> pokazuje pasek tylko w razie potrzeby.
HorizontalScrollBarVisibility	TextBox, ScrollView	widoczność poziomego paska przewijania	HorizontalScrollBarVisibility="Disabled"	Przy zawijaniu tekstu zwykle niepotrzebny.
Source	Image	źródło obrazu	Source="Images/logo.png"	Plik powinien mieć poprawne ustawienia zasobu w projekcie.
Stretch	Image, kształty	sposób skalowania	Stretch="Uniform"	<code>Uniform</code> zachowuje proporcje.
Fill	Rectangle, Ellipse, Path	wypełnienie kształtu	Fill="Orange"	Odpowiednik tła dla kształtów.
Stroke	Rectangle, Ellipse, Path	kolor obrysu kształtu	Stroke="DarkOrange"	Działa razem ze <code>StrokeThickness</code> .
StrokeThickness	kształty	grubość obrysu	StrokeThickness="2"	Liczba w jednostkach WPF.
DataContext	FrameworkElement	kontekst danych dla bindingu	DataContext="{Binding}"	Dziedziczy się w dół drzewa kontrolki.
Command	Button, MenuItem	polecenie wykonywane przez kontrolkę	Command="{Binding ZapiszCommand}"	Podstawa podejścia MVVM.
CommandParameter	Button, MenuItem	parametr przekazany do polecenia	CommandParameter="{Binding}"	Często przekazuje aktualny obiekt.
ToolTip	większość elementów	podpowiedź po najechaniu myszą	ToolTip="Wpisz imię"	Może być prostym tekstem lub złożonym XAML.

WŁAŚCIWOŚĆ	GDZIE WYSTĘPUJE	ZNACZENIE	PRZYKŁAD XAML	PRZYKŁAD C# LUB UWAGA
<code>Tag</code>	<code>FrameworkElement</code>	dodatkowa wartość pomocnicza	<code>Tag="admin"</code>	Nie nadużywaj; lepszy jest model danych.
<code>Name</code> / <code>x:Name</code>	elementy XAML	nazwa elementu dostępna w C#	<code>x:Name="txtImie"</code>	W code-behind używa się tej nazwy jak pola klasy.
<code>Grid.Row</code>	elementy w <code>Grid</code>	numer wiersza	<code>Grid.Row="1"</code>	Numerowanie zaczyna się od zera.
<code>Grid.Column</code>	elementy w <code>Grid</code>	numer kolumny	<code>Grid.Column="2"</code>	Wymaga zdefiniowanych kolumn albo domyślnej kolumny 0.
<code>Grid.RowSpan</code>	elementy w <code>Grid</code>	rozciągnięcie na kilka wierszy	<code>Grid.RowSpan="2"</code>	Dobre dla większych paneli.
<code>Grid.ColumnSpan</code>	elementy w <code>Grid</code>	rozciągnięcie na kilka kolumn	<code>Grid.ColumnSpan="2"</code>	Często dla przycisków na dole formularza.
<code>DockPanel.Dock</code>	elementy w <code>DockPanel</code>	dokowanie do krawędzi	<code>DockPanel.Dock="Top"</code>	Ostatni element może wypełnić resztę miejsca.
<code>Canvas.Left</code>	elementy w <code>Canvas</code>	pozycja od lewej	<code>Canvas.Left="20"</code>	Układ absolutny, mniej elastyczny niż <code>Grid</code> .
<code>Canvas.Top</code>	elementy w <code>Canvas</code>	pozycja od góry	<code>Canvas.Top="40"</code>	Dobre do prostych rysunków, nie do formularzy.
<code>AutoGenerateColumns</code>	<code>DataGrid</code>	automatyczne tworzenie kolumn	<code>AutoGenerateColumns="False"</code>	<code>False</code> daje pełną kontrolę nad tabelą.
<code>CanUserAddRows</code>	<code>DataGrid</code>	możliwość dopisywania wierszy przez użytkownika	<code>CanUserAddRows="False"</code>	Wyłącz, gdy dane są tylko prezentowane.
<code>SelectionMode</code>	listy, <code>DataGrid</code>	sposób zaznaczania	<code>SelectionMode="Single"</code>	Dostępne wartości zależą od kontrolki.
<code>WindowStartupLocation</code>	<code>Window</code>	pozycja startowa okna	<code>WindowStartupLocation="CenterScreen"</code>	Często stosowane dla głównego okna.
<code>ResizeMode</code>	<code>Window</code>	możliwość zmiany rozmiaru	<code>ResizeMode="CanResize"</code>	<code>NoResize</code> blokuje zmianę rozmiaru.
<code>SizeToContent</code>	<code>Window</code>	dopasowanie okna do zawartości	<code>SizeToContent="WidthAndHeight"</code>	Użyteczne przy małych oknach dialogowych.

Przykład połączenia kilku właściwości w jednym formularzu:

```
<Grid Margin="16">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="Auto"/>
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <TextBlock Text="Imię:" VerticalAlignment="Center" Margin="0,0,8,8" />
    <TextBox x:Name="txtImie" Grid.Column="1" Margin="0,0,0,8" MaxLength="30" />

    <TextBlock Grid.Row="1" Text="Rola:" VerticalAlignment="Center" Margin="0,0,8,8" />
```



```
<ComboBox x:Name="cmbRole" Grid.Row="1" Grid.Column="1" SelectedIndex="0" Margin="0,0,0,8">
    <ComboBoxItem Content="Użytkownik" />
    <ComboBoxItem Content="Administrator" />
</ComboBox>

<Button Grid.Row="2" Grid.Column="1"
        Content="Zapisz"
        HorizontalAlignment="Right"
        Padding="14,6"
        IsDefault="True" />

</Grid>
```

50.14. Atlas najważniejszych zdarzeń WPF

Zdarzenie informuje aplikację, że coś wydarzyło się w interfejsie: kliknięto przycisk, zmieniono tekst, wybrano element listy albo zamknięto okno. W XAML podaje się nazwę metody obsługującej zdarzenie, a w C# implementuje się metodę o odpowiedniej sygnaturze.

ZDARZENIE	TYPOWE KONTROLKI	KIEDY WYSTĘPUJE	TYPOWE ARGUMENTY
Click	Button , MenuItem	po kliknięciu elementu	RoutedEventArgs
TextChanged	TextBox	po zmianie tekstu	TextChangedEventArgs
PasswordChanged	PasswordBox	po zmianie hasła	RoutedEventArgs
SelectionChanged	ComboBox , ListBox , DataGrid	po zmianie wyboru	SelectionChangedEventArgs
Checked	CheckBox , RadioButton	po zaznaczeniu	RoutedEventArgs
Unchecked	CheckBox , RadioButton	po odznaczeniu	RoutedEventArgs
Indeterminate	CheckBox	po przejściu w stan pośredni	RoutedEventArgs
ValueChanged	Slider , ProgressBar zależnie od kontroli	po zmianie wartości	RoutedPropertyChangedEventArgs<double>
SelectedDateChanged	DatePicker , Calendar	po zmianie daty	SelectionChangedEventArgs
Loaded	większość elementów	po załadowaniu elementu	RoutedEventArgs
Initialized	większość elementów	przy inicjalizacji elementu	EventArgs
Closing	Window	tuż przed zamknięciem okna	CancelEventArgs
Closed	Window	po zamknięciu okna	EventArgs
KeyDown	elementy z fokusem	po naciśnięciu klawisza	KeyEventArgs
KeyUp	elementy z fokusem	po puszczeniu klawisza	KeyEventArgs
PreviewKeyDown	elementy z fokusem	przed zwykłym KeyDown	KeyEventArgs
PreviewTextInput	pola tekstowe i elementy z fokusem	przed wpisaniem znaku tekstowego	TextCompositionEventArgs
MouseEnter	większość elementów	gdy kursor wejdzie nad element	MouseEventArgs
MouseLeave	większość elementów	gdy kursor opuści element	MouseEventArgs
MouseDown	większość elementów	po naciśnięciu przycisku myszy	MouseButtonEventArgs
MouseUp	większość elementów	po puszczeniu przycisku myszy	MouseButtonEventArgs
MouseDoubleClick	Control	po podwójnym kliknięciu	MouseButtonEventArgs
GotFocus	elementy przyjmujące fokus	gdy element dostanie fokus	RoutedEventArgs
LostFocus	elementy przyjmujące fokus	gdy element straci fokus	RoutedEventArgs
DropDownOpened	ComboBox	po otwarciu listy	EventArgs
DropDownClosed	ComboBox	po zamknięciu listy	EventArgs
Expanded	Expander , TreeViewItem	po rozwinięciu	RoutedEventArgs
Collapsed	Expander , TreeViewItem	po zwinieniu	RoutedEventArgs
ImageFailed	Image	gdy obraz nie może zostać wczytany	ExceptionRoutedEventArgs
ScrollChanged	ScrollViewer	po przewinięciu zawartości	ScrollChangedEventArgs
BeginningEdit	DataGrid	przed rozpoczęciem edycji komórki	DataGridBeginningEditEventArgs
CellEditEnding	DataGrid	przy kończeniu edycji komórki	DataGridCellEditEndingEventArgs
CanExecute	komendy	przy sprawdzaniu, czy komenda jest aktywna	CanExecuteRoutedEventArgs
Executed	komendy	po wykonaniu komendy	ExecutedRoutedEventArgs

Najczęściej spotykana sygnatura obsługi zdarzenia wygląda tak:

```
private void btnZapisz_Click(object sender, RoutedEventArgs e)
{
```

```
MessageBox.Show("Zapisano dane");  
}
```

Parametry metody mają konkretne znaczenie:

PARAMETR	ZNACZENIE
<code>sender</code>	obiekt, który wywołał zdarzenie; zwykle trzeba go rzutować na właściwy typ kontrolki
<code>e</code>	dodatkowe informacje o zdarzeniu
<code>e.OriginalSource</code>	element, od którego zdarzenie zaczęło drogę w drzewie kontrolek
<code>e.Handled</code>	ustawione na <code>true</code> może zatrzymać dalszą obsługę zdarzenia routed

Przykład użycia `sender`, gdy ten sam handler obsługuje kilka przycisków:

```
<StackPanel Orientation="Horizontal">  
    <Button Content="Mały" Tag="12" Click="ZmienRozmiar_Click"/>  
    <Button Content="Średni" Tag="16" Click="ZmienRozmiar_Click"/>  
    <Button Content="Duży" Tag="22" Click="ZmienRozmiar_Click"/>  
</StackPanel>
```

```
private void ZmienRozmiar_Click(object sender, RoutedEventArgs e)  
{  
    Button klikniety = (Button)sender;  
    double rozmiar = double.Parse(klikniety.Tag.ToString());  
    txtPodglad.FontSize = rozmiar;  
}
```

Zdarzenia `Preview...` są zdarzeniami tunelowymi. Najpierw przechodzą od okna w stronę elementu docelowego, a dopiero później zwykłe zdarzenie może przejść w drugą stronę. Dzięki temu można przechwycić klawisz albo kliknięcie wcześniej, zanim obsłuży je konkretna kontrolka.

51. Najczęstsze błędy (Wyjątki i błędy kompilacji)

Poniższa lista zawiera rzeczywiste wyjątki czasu wykonania platformy .NET oraz błędy kompilacji (kody CS), z którymi najczęściej spotkasz się w okienku Output lub Error List w programie Visual Studio.

51.1. `NullReferenceException`

Jeden z najczęstszych błędów w C#. Występuje, gdy kod próbuje odwołać się do składowej obiektu, który ma wartość `null`.

✗ Kod powodujący błąd:

```
Button mojPrzycisk = null;  
mojPrzycisk.Content = "Kliknij";
```

Komunikat błędu:

```
System.NullReferenceException: 'Object reference not set to an instance of an object.'
```

✓ Poprawny kod:

```
Button mojPrzycisk = new Button();  
mojPrzycisk.Content = "Kliknij";  
// Lub sprawdzenie null: if(mojPrzycisk != null) ...
```

Należy zawsze upewnić się, że obiekt został zainicjalizowany (`new`) przed odwołaniem się do jego właściwości lub metod.

51.2. XamlParseException

Wyjątek zgłaszany w trakcie ładowania pliku XAML (często przy uruchamianiu okna `InitializeComponent()`), gdy struktura dokumentu jest niepoprawna lub brakuje wymaganego zasobu.

✗ Kod powodujący błąd:

```
<Window ...>
    <Button Content="Test" Style="{StaticResource BrakujacyStyl}" />
</Window>
```

Komunikat błędu:

```
System.Windows.Markup.XamlParseException: 'Provide value on 'System.Windows.Baml2006.TypeConverterMarkupExtension' threw an exception.'
```

✓ Poprawny kod:

```
<Window.Resources>
    <Style x:Key="BrakujacyStyl" TargetType="Button" />
</Window.Resources>
<Button Content="Test" Style="{StaticResource BrakujacyStyl}" />
```

Błąd zazwyczaj zawiera szczegóły (tzw. InnerException) wskazujące konkretny wiersz w pliku XAML. Należy naprawić literówkę w zasobach lub usunąć niepoprawny znacznik.

51.3. InvalidOperationException (Aktualizacja UI z innego wątku)

W WPF elementy interfejsu (UI) mogą być modyfikowane tylko z głównego wątku (tzw. wątku UI). Próba ich edycji z innego wątku (np. z asynchronicznego zadania Task bez powrotu do UI) powoduje awarię.

✗ Kod powodujący błąd:

```
Task.Run(() => {
    txtStatus.Text = "Gotowe!";
});
```

Komunikat błędu:

```
System.InvalidOperationException: 'The calling thread cannot access this object because a different thread owns it.'
```

✓ Poprawny kod:

```
Task.Run(() => {
    Application.Current.Dispatcher.Invoke(() => {
        txtStatus.Text = "Gotowe!";
    });
});
```

Należy użyć mechanizmu `Dispatcher.Invoke` (lub `BeginInvoke`), aby przekazać operację modyfikacji interfejsu na odpowiedni wątek.

51.4. FormatException

Pojawia się, gdy format argumentu nie spełnia wymagań metody parsującej, na przykład przy próbie zamiany liter na liczbę.

✗ Kod powodujący błąd:

```
int wiek = int.Parse("Dwadzieścia");
```

Komunikat błędu:

```
System.FormatException: 'Input string was not in a correct format.'
```

✅ **Poprawny kod:**

```
if (int.TryParse("Dwadzieścia", out int wiek))
{
    // parsowanie udane
}
else
{
    // obsługa błędu
}
```

Zaleca się używanie metod `TryParse`, które nie rzucają wyjątku w przypadku błędu formatu, lecz zwracają wartość logiczną `false`.

51.5. Błąd CS0103 (Kompilacja)

Błąd kompilacji występujący, gdy użyta nazwa zmiennej, kontrolki lub metody nie istnieje w bieżącym pliku ani kontekście.

❌ **Kod powodujący błąd:**

```
btnWyslij.Content = "Wyślij"; // Brak elementu o x:Name="btnWyslij" w XAML
```

Komunikat błędu:

```
CS0103: The name 'btnWyslij' does not exist in the current context
```

✅ **Poprawny kod:**

```
←!— W pliku XAML: →
<Button x:Name="btnWyslij" />

// W C#:
btnWyslij.Content = "Wyślij";
```

Upewnij się, że kontrolka w XAML ma ustawiony atrybut `x:Name` zgodny z nazwą używaną w C# (wielkość liter ma znaczenie!).

51.6. Błąd CS1061 (Kompilacja)

Pojawia się, gdy typ (np. obiekt danej klasy) nie zawiera definicji wywoływanej metody lub właściwości.

❌ **Kod powodujący błąd:**

```
string tekst = "Witaj";
tekst.ReverseString();
```

Komunikat błędu:

```
CS1061: 'string' does not contain a definition for 'ReverseString'...
```

✅ **Poprawny kod:**

```
string tekst = "Witaj";
char[] tablica = tekst.ToCharArray();
```

```
Array.Reverse(tablica);  
string wynik = new string(tablica);
```

Należy sprawdzić dokumentację dla danego typu. Jeśli była to własna klasa, sprawdź literówki lub dodaj brakującą metodę/właściwość.

51.7. Błąd CS0246 (Kompilacja)

Oznacza problem z odnalezieniem typu lub przestrzeni nazw (namespace). Zwykle to efekt braku instrukcji `using` na górze pliku.

✗ Kod powodujący błąd:

```
ObservableCollection<string> lista;
```

Komunikat błędu:

```
CS0246: The type or namespace name 'ObservableCollection<>' could not be found (are you missing a using directive or an assembly reference?)
```

✓ Poprawny kod:

```
using System.Collections.ObjectModel;  
  
ObservableCollection<string> lista;
```

Dodaj brakującą przestrzeń nazw poprzez `using` na początku pliku lub wykorzystaj autouzupełnianie w Visual Studio (Ctrl + .).

51.8. InvalidCastException

Wyjątek zgłaszany w czasie działania programu, gdy próbuje się rzutować obiekt na typ, z którym nie jest on kompatybilny.

✗ Kod powodujący błąd:

```
object liczba = 10;  
string tekst = (string)liczba;
```

Komunikat błędu:

```
System.InvalidCastException: 'Unable to cast object of type 'System.Int32' to type 'System.String'.'
```

✓ Poprawny kod:

```
object liczba = 10;  
string tekst = liczba.ToString();
```

Należy rzutować obiekty tylko na takie typy, w których rzeczywiście zostały utworzone. Do konwersji używaj metod takich jak `ToString()`, `Convert.ToInt32()` itp.

51.9. ArgumentNullException

Podobne do `NullReferenceException`, ale zgłaszane, gdy null został jawnie przekazany do metody, która nie akceptuje takich wartości.

✗ Kod powodujący błąd:

```
string plik = null;  
File.ReadAllText(plik);
```

Komunikat błędu:

```
System.ArgumentNullException: 'Value cannot be null. (Parameter 'path')'
```

✅ **Poprawny kod:**

```
string plik = "dane.txt";
if (plik != null)
{
    File.ReadAllText(plik);
}
```

Należy weryfikować zmienne przed przekazaniem ich jako argumenty do wbudowanych lub zewnętrznych funkcji.

51.10. FileNotFoundException

Zgłaszany przy próbie odczytu pliku, który fizycznie nie istnieje pod wskazaną ścieżką.

❌ **Kod powodujący błąd:**

```
string tekst = File.ReadAllText("brak_pliku.txt");
```

Komunikat błędu:

```
System.IO.FileNotFoundException: 'Could not find file '.../brak_pliku.txt'.'
```

✅ **Poprawny kod:**

```
if (File.Exists("plik.txt"))
{
    string tekst = File.ReadAllText("plik.txt");
}
```

Przed operacją na plikach zawsze sprawdzaj ich istnienie za pomocą `File.Exists()`. Pamiętaj również o właściwych ścieżkach (względne vs bezwzględne).

51.11. IndexOutOfRangeException

Pojawia się, gdy program próbuje pobrać element tablicy po indeksie, który wykracza poza jej długość.

❌ **Kod powodujący błąd:**

```
int[] tablica = { 1, 2 };
int trzeciElement = tablica[2];
```

Komunikat błędu:

```
System.IndexOutOfRangeException: 'Index was outside the bounds of the array.'
```

✅ **Poprawny kod:**

```
int[] tablica = { 1, 2 };
if (tablica.Length > 2)
{
    int trzeciElement = tablica[2];
}
```

Upewnij się, że użyty indeks wynosi od 0 do `Length - 1`.

51.12. ArgumentOutOfRangeException

Podobne do `IndexOutOfRangeException`, ale stosowane przez metody kolekcje (np. `List<T>`), gdy podany argument (często indeks) przekracza akceptowalny zakres.

✗ Kod powodujący błąd:

```
List<string> lista = new List<string>();
lista.RemoveAt(0);
```

Komunikat błędu:

```
System.ArgumentOutOfRangeException: 'Index was out of range. Must be non-negative and less than the size of the collection.'
```

✓ Poprawny kod:

```
List<string> lista = new List<string>();
if (lista.Count > 0)
{
    lista.RemoveAt(0);
}
```

Zanim wywołasz metodę zależną od zawartości kolekcji, upewnij się, że nie jest ona pusta (`Count > 0`).

51.13. XamlParseException (Brak handlera zdarzeń)

Błąd, który zdarza się bardzo często, gdy skopiujemy kontrolkę z XAML i zapomnimy skopiować metody C# lub usuniemy metodę C# bez usunięcia atrybutu z XAML.

✗ Kod powodujący błąd:

```
<!-- XAML posiada Click, ale w kodzie C# nie ma takiej metody -->
<Button Click="Przycisk_Click" />
```

Komunikat błędu:

```
System.Windows.Markup.XamlParseException: 'Failed to create a 'Click' from the text 'Przycisk_Click'.' Line number '...'
```

✓ Poprawny kod:

```
<!-- Jeśli nie ma metody, usunąć atrybut z XAML -->
<Button />
```

Znajdź w podanej w błędzie linii XAML zdarzenie (np. `Click="..."`) i usuń je lub wygeneruj metodę w pliku `.xaml.cs`.

51.14. BindingExpression path error (Ostrzeżenie Output Window)

To błąd logiczny, który nie powoduje zamknięcia aplikacji, ale sprawia, że dane (Binding) się nie wyświetlają. Wynika z podania niewłaściwej nazwy właściwości w XAML.

✗ Kod powodujący błąd:

```
<TextBlock Text="{Binding Nazwisko}" />
```

Komunikat błędu:


```
System.Windows.Data Error: 40 : BindingExpression path error: 'Nazwisko' property not found on 'object' ''Person' (HashCode=...)'
```

✔ Poprawny kod:

```
<TextBlock Text="{Binding Nazwisko}" />
```

Należy otworzyć okno "Output" w Visual Studio w trakcie działania aplikacji, znaleźć błąd Bindingu i poprawić literówkę w atrybucie `Binding`.

51.15. UnauthorizedAccessException

Pojawia się, gdy próbujemy modyfikować plik systemowy lub plik tylko do odczytu bez odpowiednich uprawnień.

✗ Kod powodujący błąd:

```
File.WriteAllText("C:\\Windows\\System32\\plik.txt", "dane");
```

Komunikat błędu:

```
System.UnauthorizedAccessException: 'Access to the path '...' is denied.'
```

✔ Poprawny kod:

```
// Wybierz katalog, do którego użytkownik ma prawa zapisu:  
string path = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);  
File.WriteAllText(Path.Combine(path, "plik.txt"), "dane");
```

Pisz i czytaj dane z folderów dozwolonych (np. Moje Dokumenty, AppData) lub upewnij się, że program został uruchomiony jako administrator.

51.16. DivideByZeroException

Zgłaszany przy próbie podzielenia liczby całkowitej przez zero.

✗ Kod powodujący błąd:

```
int a = 10;  
int b = 0;  
int wynik = a / b;
```

Komunikat błędu:

```
System.DivideByZeroException: 'Attempted to divide by zero.'
```

✔ Poprawny kod:

```
int a = 10;  
int b = 0;  
if (b != 0)  
    int wynik = a / b;
```

Zabezpieczaj kod wykonujący dzielenie poprzez wcześniejsze sprawdzenie czy mianownik różni się od zera.

51.17. StackOverflowException

Pojawia się (często zawieszając program natychmiast), gdy stos wywołań zostanie przepełniony. Najczęściej powoduje go nieskończona rekurencja lub błędne wywoływanie zdarzeń wewnątrz pętli.

✗ Kod powodujący błąd:

```
public int Wiek
{
    get { return Wiek; } // Odwołuje się znów do właściwości zamiast do pola
}
```

Komunikat błędu:

Process is terminated due to StackOverflowException.

✅ Poprawny kod:

```
private int _wiek;
public int Wiek
{
    get { return _wiek; }
}
```

Szukaj nieskończonych pętli w akcesorach własności (getter/setter) lub nieskończonej rekurencji. Upewnij się, że modyfikacja danych w zdarzeniu `TextChanged` nie powoduje kolejnego zdarzenia `TextChanged` .

51.18. OutOfMemoryException

Wyjątek informujący o wyczerpaniu dostępnej pamięci operacyjnej.

❌ Kod powodujący błąd:

```
List<byte[]> duzeDane = new List<byte[]>();
while(true) duzeDane.Add(new byte[1000000]);
```

Komunikat błędu:

System.OutOfMemoryException: 'Exception of type 'System.OutOfMemoryException' was thrown.'

✅ Poprawny kod:

```
// Poprawna logika z ograniczeniem przydzielania pamięci
// oraz zwalnianiem obiektów lub tablic, których już nie potrzebujemy.
```

Zoptymalizuj przetwarzanie dużych plików. Unikaj ładowania wszystkiego naraz (np. wielkich list lub obrazów bez optymalizacji rozdzielczości).

51.19. InvalidOperationException (Kolekcje w pętli foreach)

Występuje podczas użycia instrukcji `foreach` , jeśli modyfikujemy kolekcję podczas przeglądania (dodajemy lub usuwamy z niej elementy).

❌ Kod powodujący błąd:

```
foreach (var item in lista)
{
    if (item == "usun") lista.Remove(item);
}
```

Komunikat błędu:

System.InvalidOperationException: 'Collection was modified; enumeration operation may not execute.'

✔ Poprawny kod:

```
for (int i = lista.Count - 1; i ≥ 0; i--)  
{  
    if (lista[i] == "usun") lista.RemoveAt(i);  
}
```

Zamiast `foreach` użyj tradycyjnej pętli `for` (najlepiej idąc od tyłu `i--`, aby indeksy nie ulegały przesunięciu po usunięciu elementu).

51.20. NotSupportedException

Zgłaszany przy użyciu metod klasy, która nie obsługuje wybranej operacji (np. modyfikowanie tablicy o stałym rozmiarze używając interfejsów kolekcji).

✖ Kod powodujący błąd:

```
int[] tablica = { 1, 2 };  
((IList)tablica).Add(3);
```

Komunikat błędu:

```
System.NotSupportedException: 'Collection was of a fixed size.'
```

✔ Poprawny kod:

```
List<int> lista = new List<int> { 1, 2 };  
lista.Add(3);
```

W przypadku konieczności dynamicznego dodawania/usuwania elementów wybieraj `List<T>` zamiast zwykłych tablic konwertowanych na interfejsy.